GOOGLE | QUESTIONS + FULL CODE

Study path, questions and full code

288 questions in 43 groups, in the same order and with the same numbers as the study path book. Each question shows its statement and examples, then its full program in the clean layout: the main question (helper functions, main function, tests), then every follow-up in its own labelled section with its own helpers, function and tests, and run_all_tests() at the bottom. The brute force and the long explanations are in the study path book, whose groups also start with the code template. The italic line under each title says what you change in the group's template for that question.

38 EASY	197 MEDIUM	53 HARD

1. Frontier search (BFS or DFS), Tier 1: spread, count and measure distance (9)

1. Flood Fill
2. Number of Islands
3. Rotten Oranges / Multi-Source BFS (taxis)
4. Solve Grid and Counting Problems
5. Find shortest relationship path using BFS
6. Farthest City for a New Data Center
7. Compute distance to nearest taxi in grid
8. Infection Spread on a Grid (Cellular Automaton)
9. Shortest Distance From All Buildings

2. Sweep over events in time order (9)

10. Meeting Rooms / Interval Scheduling Variants
11. Train / Flight Schedule Reachability
12. Solve Rooms and Top-K Streams
13. Compute servers needed for daily recurring jobs
14. Gate Entry / Exit Time Resolution with Tie-Break Rules
15. Meeting Rooms III Variant
16. Count Overlapping Rectangle Updates on a Grid
17. Compute city skyline outline
18. Compute Turnstile Crossing Times

3. Tree recursion, Tier 1: one function that visits a node, asks its children, and reports upward (9)

19. Given two trees, determine if the pre-order traversals of two trees are same or not.
20. Calculate the height of a binary tree.
21. Find right-side view of binary tree
22. Expression Tree Single-Leaf Mutation
23. Longest Consecutive Path in Binary Tree
24. Given the root of a binary tree, return the length of the diameter of the tree.
25. Implement zigzag level-order traversal
26. Repeatedly Delete Leaves Matching a Target Value From a Binary Tree
27. Delete an Entire Binary Tree

4. A heap of what is available: pop the best, put back the changed one (9)

28. Streaming Median (LC 295) with Follow-ups
29. Compute minimum number of rooms needed
30. Implement a Reusable Locker Allocation System
31. Simulate room assignments for scheduled meetings
32. Trip Expense Settlement
33. Playlist Shuffle No Adjacent Artists
34. Minimum Makespan on Identical CPUs
35. Design server allocation for multi-type nodes
36. Huffman Tree Construction

5. Recurrence on a position or a range (memo or table) (9)

37. Target Expressions from Ordered Digits
38. Count Word Decompositions into Chemical Element Symbols

39. Find the longest palindromic subsequence using dynamic programming.
40. Jump Game with Profit
41. Fence Painting Minimum Operations (Gemini Team)
42. Maximize 1D deliveries within distance and minimize total distance
43. Edit distance
44. Count Perfect Wake Numbers up to N
45. Word Break: All Sentences

6. Best-first search with a heap: Dijkstra and its minimax twin (9)

46. Path Minimizing the Maximum Cell (Swim in Rising Water)
47. Nearest Favorite City by Travel Time
48. Given a train schedule with departure city, arrival city, start time, end time, and duration, and given a source and destination, find the sequence of trains that gets you there at the earliest possible time.
49. Find the shortest path in a graph.
50. Given a matrix with a rat, bread, cat, and obstacles, find a path from the rat to the bread that maximizes the minimum distance from cat.
51. Minimize maximum height along a grid path
52. Find minimum threshold enabling grid path
53. Minimize travel time with optional meeting point
54. Compute minimax grid path and network delay

7. Backtracking: choose, recurse, undo (9)

55. Mahjong Winning Hand Detection (14 Tiles)
56. Count Prefix-Matching Simple Paths in a Labeled Graph
57. Generate Parentheses
58. American Football Score Combinations
59. Permutations
60. Determine if a 14-tile hand is winning
61. Count Passcode Patterns on a Keypad
62. Match people to questions using tags and priorities
63. Matrix Flower Placement with House Adjacency

8. Runs and groups: extend while it fits, restart or close when it does not (9)

64. Group Timestamps Into Close Intervals
65. First Out-of-Order Character
66. Longest Non-Decreasing Subarray with One Replacement
67. Piano Hand Movement Grouping
68. Find longest increasing contiguous subarray
69. Sum of Good Subarrays (Adjacent Diff ±1)
70. Sustained-Error Time Window
71. Longest Non-Decreasing Subarray From Two Arrays
72. Count Super Streaks

9. Union-find: merge the pairs, then ask who is together (8)

73. Count Islands in a Grid and Track the Count as Land Cells Are Added
74. 2D Points Connected Components by Radius
75. Count Clusters of 2D Points Within a Radius
76. Find A Threshold-Limited Path With Minimum Required Safety
77. Movie Recommendations via Transitive Similarity
78. Determine Reporting Relationships
79. Determine whether two nodes are related
80. Friend Graph with Timeline (Friend / Unfriend Events)

10. A dict of per-key state that you update, expire, or rank (8)

81. Robot Status Message Deduplication Window
82. Active Communication Top Users
83. Filter Repeated Robot Status Messages
84. Next-Word Predictor
85. Batch and Streaming Log Deduplication
86. Design a logger that returns top-K messages
87. Hit Counter
88. Website Activity Analytics: Unique Users + Session Time

11. Binary search on a yes/no condition (8)

89. First Index Reaching a Threshold (Binary Search)

90. Cutting Ribbons (LC 1891)
91. First Bad Version with Parallel Search Follow-up
92. Solve Banana Speed and Interval Merge
93. Given a list with response time for each server, calculate how long time it will take to process n requests.
94. Median of Two Sorted Arrays (LC 4) with a Follow-up Variant
95. Find the median of two sorted arrays.
96. Find longest duplicated substring

12. Greedy: scan once, carry the best choice so far, act only when forced (8)

97. Maximum Profit
98. Alarm for the Next Upcoming Birthday
99. Coins on the Table
100. Visible People in a Line (Last Person, Then Everyone)
101. Bounded Knapsack Variant
102. Find the Largest Monotone Increasing Number
103. Min boards to cover roof holes
104. Jump Game II (Minimum Jumps)

13. Sets and counters: remember the first collection, check the second against it (8)

105. Exclude First-K-Unique From Another List
106. Find the Duplicates
107. Positional Matches Between Two Lists
108. Contains Duplicate
109. Filter Common Elements Per Window (Two Lists)
110. Reconstruct original array from doubled shuffle
111. Min deletions to avoid overlap in first k
112. Smallest Missing Positive Integer in Linear Time and Constant Extra Space

14. bisect on a sorted list: find a position, count a range, keep it sorted (8)

113. Count People in a Postal Code Range
114. Sliding-Window Mean After Removing the Largest K Values
115. Russian Doll Envelopes
116. Count Sorted Strings With a Given Prefix
117. Streaming Insert + Find K-th Largest
118. Given positions of N kids and M ice cream sellers on a 1D line, find the smallest distance E such that every kid lies within the range [P - E, P + E] of at least one seller.
119. Find Maximum Envelope Nesting
120. Count of Smaller Numbers After Self

15. Prefix sums, Tier 1: precompute once, answer ranges and balance points in O(1) (8)

121. Parity Alternating Subarray Query
122. Cake Balance Line
123. Find the Pivot Index (Balanced Split) in O(n)
124. Range Duplicate Check with Constant-Time Queries
125. Find a Horizontal Cut That Bisects Rectangle Area
126. Check pivot after removing one element
127. Gold Chain Split into Two Equal Halves
128. Count the number of ways to split a string.

16. Sorted intervals: merge while they touch, then read the gaps or search (8)

129. Check an Interval Against Existing Intervals for Conflicts
130. Find Containing Range
131. Union of Sorted Interval Lists
132. Range Module (Insert and Point Query)
133. Meetings Minus a Do-Not-Schedule Block
134. Merge Intervals
135. Find safe travel intervals between planet influences
136. Implement a function that processes a stream of interval data.

17. Trie: nested dicts, one character per level (8)

137. URL Request Handler Router
138. Trie Prefix Search / Autocomplete
139. Build Prefix Lookup with a Trie
140. Greedy Longest-Match Dictionary Tokenizer
141. Check if all substrings are dictionary words

142. Return dictionary words matching a prefix
143. Implement Trie Insert and Exact-Word Search
144. Implement Longest-Match Text Replacement

18. A dict for lookup plus an ordered container for who is next (lazy deletion) (8)

145. Implement Batched Undo/Redo Layer
146. Design a restaurant waitlist system
147. Implement Python's itertools.tee
148. Store With Cheapest-Item Removal
149. Round-Robin Job Scheduler
150. Aggregate FizzBuzz Offer Matches
151. Restaurant Waitlist: join / delete / find_first_match
152. Implement LRU Cache.

19. Rolling state: the next state comes only from the previous one (7)

153. Climbing Stairs
154. Fibonacci Numbers
155. Maximize sum without choosing adjacent elements
156. Given n houses in a line with money in each, find the maximum amount a robber can steal, without stealing from two adjacent houses.
157. Paint House II
158. Write SQL code to publish the Fibonacci series.
159. Constrained Dice Roll Sequences

20. Two pointers: move i and j toward each other (or along two lists) (7)

160. Reverse a linked list.
161. Sort a binary array in linear time.
162. Is this a valid palindrome?
163. Reverse a Sentence
164. Find median of merged RLE arrays
165. Run-Length Vector Storage + Dot Product
166. Count Unique Triplets That Sum to Zero

21. Walk two strings at once with two indices (7)

167. Find the First Occurrence of a Substring
168. Determine if two words are sorted in lexicographic order.
169. Return all unique substrings of a given string.
170. Can board states be transformed?
171. Decide string transform with directional tokens and walls
172. Determine if two sentences are similar.
173. Find Dictionary Matches for a Jammed String

22. Simulate a machine: a class with state and one branch per kind of event (7)

174. Mario-Style Score Tracker (OOP)
175. Tic-Tac-Toe System with K Players and 3-in-a-Row Win
176. Design Tic-Tac-Toe With K Players
177. Infinite Deck Card Duel
178. Implement iterator merging lists with filtering
179. Design an elevator control system
180. Simulate In-Place Cellular State Updates

23. Top-K by frequency: Counter plus nsmallest (6)

181. Top-K Frequent in Large Logs
182. Rearrange Tasks With Cooldown
183. Implement frequency + distance top■K queries
184. Solve chat, grid paths, and car rentals
185. Top-K Chatters by Word Count
186. Find top/bottom-k words in list or stream

24. Digits and characters: peel one at a time and carry (6)

187. Caesar Cipher: Recover Key, Encrypt, and Decrypt
188. Add two big integers from digit lists
189. Solve several coding interview problems
190. Check Whether Digits Are Monotone Increasing
191. Check Digits and Combine Ranges

192. Permutation Sequence

25. Look at the input once and keep only a small summary (and count your calls) (6)

193. Two-Day Log Intersection under Memory Limit
194. Find shared objects across two log files
195. Streaming Points: Emit K Within Distance
196. Minimize calls to find all bad test pairs
197. Find All Bad Pairs Using runTest Oracle
198. Find maximum value in huge text file

26. Grid DP: each cell from its neighbours above and to the left (5)

199. Grid Path Count with Diagonal Moves and Checkpoints
200. Longest Same-Color Grid Sequence
201. Count Same-Color Squares in Matrix
202. Find the Longest Straight Run of One Color
203. Count same-color squares in a character grid

27. Nested structures: a stack of contexts, or recursion with a cache (5)

204. Decode String
205. Template Variable Expansion with Follow-Ups
206. Expand Recursive Key-Value Placeholders in a Template
207. Nested List Weight Sum (LC 339) with Follow-Ups
208. Nested Function Expression Evaluator

28. Frontier search (BFS or DFS), Tier 2: free steps, early exit, neighbours worked out on the fly (5)

209. Shortest Path Nodes and Escaping a Pursuer
210. Radio Broadcast Reachability
211. Minimum Direction Violations in a Directed Graph
212. Compute shortest paths with blocked nodes
213. Shortest Directed Cycle Through a Node

29. Topological sort (Kahn) and cycle detection (5)

214. Course Scheduling and Dependency Variants
215. Validate Race Results (Cycle Detection)
216. Solve tree, graph, sliding-window problems
217. Validate course catalog dependencies
218. Course Schedule

30. Counting pairs by looking up each partner in a Counter or a bucket (5)

219. Progressive Array Partitioning
220. K-diff Pairs in an Array
221. Count Axis-Aligned Squares Among Grid Points
222. Sum of Two Squares in Two Ways
223. Maximum-Area Rectangle from Planar Points

31. Tree DP: build a parent order once, then add up from the leaves (5)

224. Compute File System Total Size
225. Minimum-Cost Tree Disconnection
226. Given a large n-ary tree where the root node can pass information to one child per iteration, and in subsequent iterations, nodes can pass information to one child while the root continues with others, find the minimum number of iterations to reach all nodes.
227. Compute distance-sum from every tree node
228. Tree Distance Sum (Re-root DP)

32. Counting or reaching an amount: the knapsack loop (5)

229. Maximize Shelf Space (Unbounded Knapsack)
230. Coin Change
231. Partition an array into two sub-arrays with equal sum.
232. Explain how to find a target sum in an array.
233. Reverse Coin Change

33. Prefix sums, Tier 2: running total plus a dict of earlier totals (5)

234. Maximum-Sum Subarray With Equal Endpoints
235. Maximum Subarray Sum With a Minimum Length Constraint
236. Solve array-sum and city-community problems
237. Find the maximum subarray sum.

238. Given an integer array nums and an integer k, return true if nums has a subarray of at least two elements whose sum is a multiple of k.

34. Group by a canonical key (5)

239. Given an array of strings, return the words that can be typed using letters of the alphabet on only one row of American keyboard.
240. Check if all substrings are anagrams of words
241. Group anagrams
242. Given an array of strings, return groups of "buddy strings," where the strings have the same length and the distance between corresponding characters is the same.
243. Given two strings and two allowed operations, swapping positions of letters and converting all occurrences of one character to another, can the first string be transformed into the second?

35. K-way merge: a heap of the next item from each source (5)

244. Find k pairs with smallest sums
245. K Largest Elements Across Sorted Lists
246. Sort a nearly sorted (or K sorted) array.
247. Merge k sorted linked lists.
248. Return top-k subset sums in descending order

36. Tree recursion, Tier 2: pass a rule down, return a node or a yes/no up (5)

249. Next Birthday (BST In-Order Successor with Wraparound)
250. Lowest Common Ancestor
251. Determine if a given binary tree is a binary search tree (BST).
252. Find mode in a trinary search tree
253. Next Upcoming Birthday in a Family Tree

37. Random pick without repeats: swap the chosen item with the last one and pop (5)

254. Random Generator Without Repeats
255. Random Song Shuffler With a No-Repeat Window
256. Design set with O(1) random access
257. Unique Random Number Generator
258. Shuffle array using random integer API

38. Build and serialize: write a structure out, read it back with the same walk (5)

259. Construct a binary tree given its preorder and inorder traversals.
260. Write functions to serialize and deserialize a list of strings.
261. Print a Directory Tree
262. Implement a decode and encode function of a general N-ary tree.
263. Serialize and Deserialize a Graph

39. Games: win or lose with a cache, and guessing with feedback (5)

264. Choose optimal guesses for green-only Wordle
265. Wordle-Style 5-Letter Minimum-Guess Strategy
266. Determine winner in stack-merging game
267. Find secret word with match-count feedback
268. Babylon Tile-Stack Merge Game

40. Sliding window: grow on the right, shrink on the left while it is too bad (5)

269. Temperature Monitoring Data Structure
270. Shortest Subarray with K Distinct Integers
271. Find the minimum window substring.
272. Footpath Beauty After Repainting
273. Necklace Cut into Two Halves with Equal D/R Counts

41. Parent pointers: store who is above each node and climb (5)

274. Tree Node.
275. Employee Hierarchy Add / Delete
276. Validate parent array forms a tree
277. Lowest Common Ancestor of a Binary Tree III
278. Root a Tree So It Becomes a Binary Tree

42. Build or transform a grid with index arithmetic (5)

279. Filled ASCII Diamond
280. Connected Crop Arrangement on a Grid
281. Rotate Any m×n 2D Matrix

282. Solve matrix groups and recipe inventory
283. Range Sum Query 2D (Immutable Matrix)

43. Split into pieces, fix each piece, join (and pack words into lines) (5)

284. Implement string formatting and request tracking tasks
285. Snake Case to Camel Case with Repository-Wide Refactor
286. Google Doc Line Wrap (Word Wrap with Whitespace and Newlines)
287. Implement a function to justify a list of words within a given width.
288. Convert Valid Snake Case to Lower Camel Case

GROUP 1 OF 43 | 9 QUESTIONS, ONE CODE TEMPLATE | 1 E 6 M 2 H

Frontier search (BFS or DFS), Tier 1: spread, count and measure distance

1 Flood Fill — EASY

Template fit: starts = [(sr, sc)]; usable = same colour as the start; repaint every cell you reach; return early if old colour == new colour

QUESTION

You are given an m x n grid image where image[r][c] is the integer color of the pixel at row r , column c . You are also given a starting pixel (sr, sc) and a new_color . Perform a flood fill: starting from (sr, sc) , repaint that pixel and every pixel connected to it 4-directionally (up, down, left, right) that shares the SAME original color as the starting pixel, changing them all to new_color . Pixels connected only diagonally are not filled. Return the modified grid.

Function signature

```
from typing import List
def flood_fill(image: List[List[int]], sr: int, sc: int, new_color: int) ->
List[List[int]]:
    ...
```

Follow-up (as reported): Implement with DFS (recursive or explicit stack) or BFS; discuss the base case that prevents re-visiting when new_color equals the original color.

EXAMPLES

Example 1	
Input	`image = [[1,1,1],[1,1,0],[1,0,1]], sr = 1, sc = 1, new_color = 2`
Output	`[[2,2,2],[2,2,0],[2,0,1]]`
Why	`Starting color is 1. All 1-pixels connected 4-directionally to (1,1) become 2. The bottom-right 1 is only diagonally reachable, so it stays.`

Example 2	
Input	`image = [[0,0,0],[0,0,0]], sr = 0, sc = 0, new_color = 5`
Output	`[[5,5,5],[5,5,5]]`
Why	`The whole grid is one connected region of color 0.`

Example 3	
Input	`image = [[0,0,0],[0,1,0]], sr = 1, sc = 1, new_color = 1`
Output	`[[0,0,0],[0,1,0]]`
Why	`The start pixel already equals new_color, so nothing changes.`

CONSTRAINTS

- 1 <= m, n <= 50
- 0 <= image[r][c], new_color < 2^16
- 0 <= sr < m , 0 <= sc < n
- If the starting pixel already has new_color , the grid is returned unchanged (guard against infinite recursion).

FULL CODE | main question first, then every follow-up, tests at the end

```
# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ========================================================
```

```python
from collections import deque
import copy


# =========================================================
# MAIN QUESTION
# =========================================================
# ---- main question: helper functions ----
def neighbors(r, c, rows, cols):
    # the cells above, below, left and right of (r, c) that exist
    result = []
    for nr, nc in ((r + 1, c), (r - 1, c), (r, c + 1), (r, c - 1)):
        if 0 <= nr < rows and 0 <= nc < cols:
            result.append((nr, nc))
    return result


# ---- main question: main function ----
# Idea: spread over neighbours that still have the old colour and
# repaint them. If the new colour equals the old one there is nothing
# to do (and the loop below would never finish), so return at once.
def flood_fill(image, sr, sc, new_color):
    old = image[sr][sc]
    if old == new_color:
        return image
    rows, cols = len(image), len(image[0])
    image[sr][sc] = new_color
    stack = [(sr, sc)]
    while stack:
        r, c = stack.pop()
        for nr, nc in neighbors(r, c, rows, cols):
            if image[nr][nc] == old:
                image[nr][nc] = new_color
                stack.append((nr, nc))
    return image


# ---- main question: tests ----
```

```python
IMAGE = [[1, 1, 1], [1, 1, 0], [1, 0, 1]]
FILLED = [[2, 2, 2], [2, 2, 0], [2, 0, 1]]


def test_main_question():
    assert flood_fill(copy.deepcopy(IMAGE), 1, 1, 2) == FILLED
    assert flood_fill(copy.deepcopy(IMAGE), 1, 1, 1) == IMAGE      # same colour: nothing changes


# =========================================================
# FOLLOW-UP 1: Do it with BFS. Why is the early return needed?
# =========================================================
# ---- follow-up 1: function ----
# Answer: a queue (BFS) costs the same. If the new colour equals the
# old one, repainting never changes the test "has the old colour", so
# cells would be pushed again forever; the early return (or a visited
# set) stops that.
def flood_fill_bfs(image, sr, sc, new_color):
    old = image[sr][sc]
    if old == new_color:
        return image
    rows, cols = len(image), len(image[0])
    image[sr][sc] = new_color
    queue = deque([(sr, sc)])
    while queue:
        r, c = queue.popleft()
```

```
        for nr, nc in neighbors(r, c, rows, cols):
            if image[nr][nc] == old:
                image[nr][nc] = new_color
                queue.append((nr, nc))
    return image


# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert flood_fill_bfs(copy.deepcopy(IMAGE), 1, 1, 2) == FILLED
    assert flood_fill_bfs(copy.deepcopy(IMAGE), 1, 1, 1) == IMAGE


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
```

```
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

2 Number of Islands — MEDIUM

Template fit: loop over every cell; each unseen land cell starts a new frontier_search (share one seen set); islands += 1 per start

QUESTION

Given a grid of 0s (water) and 1s (land), count the islands. An island is a group of 1s connected up, down, left or right (diagonal cells are not connected).

Function signature

def num_islands(grid: list) -> int

EXAMPLES

Example 1	
Input	`[[1, 1, 0, 0], [1, 1, 0, 0], [0, 0, 1, 1], [0, 0, 1, 1]]`
Output	`2`
Why	`Two 2x2 blocks of land that only touch the water between them: 2 islands.`

Example 2	
Input	`[[1, 0, 1], [0, 1, 0], [1, 0, 1]]`
Output	`5`
Why	`Land that touches only diagonally is not connected, so every 1 is its own island.`

Example 3	
Input	`[[0, 0], [0, 0]]`
Output	`0`
Why	`No land, no islands.`

FULL CODE | main question first, then every follow-up, tests at the end

```python
# ======================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ======================================================

# ======================================================
# MAIN QUESTION
# ======================================================
# ---- main question: helper functions ----
def neighbors(r, c, rows, cols):
    # the cells beside (r, c) that are inside the grid
    result = []
    for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)):
        nr, nc = r + dr, c + dc
        if 0 <= nr < rows and 0 <= nc < cols:
            result.append((nr, nc))
    return result


def flood(grid, start, seen):
    # mark all land joined to `start` as seen and return how many cells it has;
    # a stack instead of recursion, so a huge island cannot overflow the call stack
    rows, cols = len(grid), len(grid[0])
    seen.add(start)
    stack = [start]
    size = 0
    while stack:
        r, c = stack.pop()
        size += 1
        for nr, nc in neighbors(r, c, rows, cols):
            if grid[nr][nc] == 1 and (nr, nc) not in seen:
                seen.add((nr, nc))
                stack.append((nr, nc))
    return size


def island_sizes(grid):
    # one number per island: how many cells it has
    sizes = []
    if not grid:
        return sizes
    seen = set()
    for r in range(len(grid)):
        for c in range(len(grid[0])):
            if grid[r][c] == 1 and (r, c) not in seen:     # land we have not visited starts a new island
                sizes.append(flood(grid, (r, c), seen))
    return sizes


# ---- main question: main function ----
# Idea: scan every cell; each unvisited land cell starts a new
# island, and the flood marks all its connected land as visited.
def num_islands(grid):
    return len(island_sizes(grid))


# ---- main question: tests ----
def test_main_question():
    g = [[1, 1, 0, 0], [1, 1, 0, 0], [0, 0, 1, 1], [0, 0, 1, 1]]
    assert num_islands(g) == 2
    assert num_islands([]) == 0


# ======================================================
```

```
# FOLLOW-UP 1: What if land can also be removed?
# ======================================================


# ======================================================
# FOLLOW-UP 2: How do you handle a grid too large for memory?
# ======================================================


# ======================================================
# FOLLOW-UP 4: The grid is huge and does not fit in memory
# ======================================================
# ---- follow-up 1: helper functions ----
class IslandCounter:
    # land cells arrive one at a time; keep the island count up to date
    # with union-find (each island is one group of joined cells)
    def __init__(self, rows, cols):
        self.rows = rows
        self.cols = cols
        self.parent = {}        # land cell -> a cell of its group
        self.count = 0

    def find(self, cell):
        # follow parents to the group's representative (path halving keeps it fast)
        while self.parent[cell] != cell:
            self.parent[cell] = self.parent[self.parent[cell]]
            cell = self.parent[cell]
        return cell

    def add_land(self, r, c):
        if (r, c) in self.parent:               # adding the same cell twice changes nothing
            return self.count
        self.parent[(r, c)] = (r, c)
        self.count += 1
        for nb in neighbors(r, c, self.rows, self.cols):
            if nb in self.parent:
                a = self.find((r, c))
                b = self.find(nb)
                if a != b:
                    self.parent[a] = b
                    self.count -= 1             # two islands became one
        return self.count


# ---- follow-up 1: tests ----
def test_follow_up_1():
    ic = IslandCounter(2, 2)
    counts = []
    for r, c in [(0, 0), (0, 1), (1, 1)]:
        counts.append(ic.add_land(r, c))
    assert counts == [1, 1, 1]
    ic = IslandCounter(3, 3)                    # two separate cells make two islands
    assert ic.add_land(0, 0) == 1
    assert ic.add_land(2, 2) == 2
    assert ic.add_land(1, 1) == 3               # the middle cell touches neither corner
# Removing land is harder: process the changes backwards (removal
# becomes adding) or rebuild the affected island.
# Too big for memory: read the grid a row at a time, keep only the
# group labels of the previous row, and merge labels with union-find.


# ======================================================
# FOLLOW-UP 3: Find the area of the largest island
# ======================================================
# ---- follow-up 3: function ----
```

```
# Answer: count the cells during each flood and keep the maximum.
def largest_island(grid):
    biggest = 0
    for size in island_sizes(grid):
        if size > biggest:
            biggest = size
    return biggest


# ---- follow-up 3: tests ----
def test_follow_up_3():
    g = [[1, 1, 0, 0], [1, 1, 0, 0], [0, 0, 1, 1], [0, 0, 1, 1]]
    assert largest_island(g) == 4
    assert largest_island([[0, 0]]) == 0


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

3 Rotten Oranges / Multi-Source BFS (taxis) MEDIUM

Template fit: starts = every rotten orange; usable = fresh orange; answer = rounds taken, or -1 if a fresh orange stays unreached

QUESTION

Each cell of a grid is 0 (empty), 1 (a fresh orange) or 2 (a rotten orange). Every minute, each fresh orange that is next to (up, down, left or right) a rotten one becomes rotten. Return the number of minutes until no fresh orange is left, or -1 if some fresh orange can never rot.

Function signature

def rotting_minutes(grid: list) -> int

EXAMPLES

Example 1	
Input	`[[2, 1, 1], [1, 1, 0], [0, 1, 1]]`
Output	`4`
Why	`The rot spreads outward from the corner and reaches the last orange after 4 minutes.`

Example 2	
Input	`[[2, 1, 1], [0, 1, 1], [1, 0, 1]]`
Output	`-1`
Why	`The orange at the bottom left is cut off by empty cells, so it never rots.`

QUESTION

Example 3	
Input	`[[0, 2]]`
Output	`0`
Why	`Nothing fresh to rot: 0 minutes.`

FULL CODE | main question first, then every follow-up, tests at the end

```
# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ========================================================

from collections import deque


# ========================================================
# MAIN QUESTION
# ========================================================
# ---- main question: helper functions ----
def inside(r, c, rows, cols):
    # is (r, c) a real cell of the grid?
    return 0 <= r < rows and 0 <= c < cols


def find_cells(grid, value):
    # every (row, column) whose cell holds `value`
    cells = []
    for r in range(len(grid)):
        for c in range(len(grid[0])):
            if grid[r][c] == value:
                cells.append((r, c))
    return cells


def bfs_distances(grid, starts, wall):
    # distance from the nearest start to every cell it can reach;
    # a cell holding `wall` can never be entered
    rows, cols = len(grid), len(grid[0])
    dist = {}
    queue = deque()
    for start in starts:
        dist[start] = 0
        queue.append(start)
    while queue:
        r, c = queue.popleft()
        for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)):
            nr, nc = r + dr, c + dc
            if not inside(nr, nc, rows, cols):
                continue
            if grid[nr][nc] == wall or (nr, nc) in dist:
                continue
            dist[(nr, nc)] = dist[(r, c)] + 1
            queue.append((nr, nc))
    return dist


# ---- main question: main function ----
# Idea: every rotten orange starts at minute 0 in the same queue;
# each BFS level is one minute. A fresh orange that is never
```

```python
# reached makes the answer -1.
def rotting_minutes(grid):
    rotten = find_cells(grid, 2)
    fresh = find_cells(grid, 1)
    dist = bfs_distances(grid, rotten, 0)      # rot spreads through any non-empty cell
    minutes = 0
    for cell in fresh:
        if cell not in dist:
            return -1
        if dist[cell] > minutes:
            minutes = dist[cell]
    return minutes


# the same search answers "distance to the nearest taxi"
def nearest_taxi(grid):
    # 'T' = taxi, '.' = empty, '#' = blocked; None where no taxi can reach
    taxis = find_cells(grid, "T")
    dist = bfs_distances(grid, taxis, "#")
    answer = []
    for r in range(len(grid)):
        row = []
        for c in range(len(grid[0])):
            row.append(dist.get((r, c)))
        answer.append(row)
    return answer


# ---- main question: tests ----
def test_main_question():
    assert rotting_minutes([[2, 1, 1], [1, 1, 0], [0, 1, 1]]) == 4
    assert rotting_minutes([[2, 0, 1]]) == -1       # a fresh orange walled off by an empty cell
    assert rotting_minutes([[0, 2]]) == 0
    assert nearest_taxi(["T.#", "..."]) == [[0, 1, None], [1, 2, 3]]
```

```python
# ============================================================
# FOLLOW-UP 1: Sources move or new taxis appear
# ============================================================
# Answer: do not redo everything. A new source only improves the
# cells it beats, so run a BFS from it that stops at cells it does
# not improve (written out in the taxi question of this group).
# Moving sources: recompute from scratch, or keep one BFS per source.


# ============================================================
# FOLLOW-UP 2: Why is multi-source BFS the same cost as single-source BFS?
# ============================================================
# Answer: every cell is put in the queue once, no matter how many
# sources there are, so the work is O(rows * cols) either way.


# ============================================================
# FOLLOW-UP 3: Signal variant: how long until every target gets it?
# ============================================================


# ============================================================
# FOLLOW-UP 4: A signal spreads from sources to targets
# ============================================================
# ---- follow-up 4: function ----
# Answer: one multi-source BFS from the sources, then the answer is
# the largest distance over the targets (-1 if a target is never
# reached, 0 if there are no targets). One BFS per source would
# cost O(rows * cols * sources).
def signal_time(rows, cols, sources, targets, blocked=()):
```

```
    grid = []
    for r in range(rows):
        grid.append([0] * cols)
    for r, c in blocked:
        grid[r][c] = 1                              # 1 = wall
    dist = bfs_distances(grid, list(sources), 1)
    slowest = 0
    for target in targets:
        if target not in dist:
            return -1
        if dist[target] > slowest:
            slowest = dist[target]
    return slowest
```

```
# ---- follow-up 4: tests ----
def test_follow_up_4():
    assert signal_time(1, 5, [(0, 0), (0, 4)], [(0, 1), (0, 2)]) == 2
    assert signal_time(1, 3, [(0, 0)], [(0, 2)], blocked={(0, 1)}) == -1
    assert signal_time(1, 3, [(0, 0)], []) == 0


# ======================================================
# RUN EVERYTHING
# ======================================================
def run_all_tests():
    test_main_question()
    test_follow_up_4()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

4 Solve Grid and Counting Problems — MEDIUM

Template fit: the island count again (queue version); the second part of the question is a short counting formula, no search

QUESTION

A software engineering intern phone screen included two coding tasks: 1.
Count islands in a grid Given an `m x n` grid of characters where `'1'` represents land and `'0'` represents water, return the number of islands. An island is formed by connecting adjacent land cells horizontally or vertically. You may assume all four edges of the grid are surrounded by water. 2.
Count ways to form a target word Given an array of uppercase letters, count how many different ways you can choose letters from the array so that the chosen letters can be rearranged to spell **`GOOGLE`**. Each array element can be used at most once, and letters at different indices are considered different choices even if they contain the same character. Design efficient algorithms for both tasks, and discuss the time and space complexity of your approach.

Part 1: Count Islands in a GridGiven a 2D grid of characters where each cell is either '1' (land) or '0' (water), return the number of islands. An island is a group of adjacent land cells connected only horizontally or vertically. Diagonal cells do not count as connected. You may assume the grid edges are surrounded by water.
Constraints
0 <= number of rows <= 3000 <= number of columns <= 300grid[r][c] is either '0' or '1'All rows have the same length

EXAMPLES

```
Input: []
```

Expected Output:
Explanation: Empty grid contains no land, so there are no islands.

QUESTION

```
Input: [['1']]
```

Expected Output:
Explanation: A single land cell forms one island.

Hints

Whenever you find an unvisited land cell, start a DFS or BFS to mark the entire island.Only 4-directional neighbors matter: up, down, left, and right.

Part 2: Count Ways to Form the Word GOOGLEGiven an array of uppercase letters, count how many different ways you can choose letters so that the chosen letters can be rearranged to spell GOOGLE. Each array element can be used at most once. Letters at different indices count as different choices even if they contain the same character. The word GOOGLE requires exactly 2 G's, 2 O's, 1 L, and 1 E.

Constraints

0 <= len(letters) <= 200000Each element is an uppercase English letter from 'A' to 'Z'The answer may be large, but fits in Python's integer type

Examples

Input: []

Expected Output:

Explanation: No letters means it is impossible to form GOOGLE.

Input: ['G','O','O','G','L','E']

Expected Output:

Explanation: There is exactly one way to choose all six letters.

Hints

First count how many G, O, L, and E letters appear in the array.You need to choose 2 of the G positions and 2 of the O positions, then 1 L and 1 E.

FULL CODE | main question first, then every follow-up, tests at the end

```python
# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

from collections import deque


# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
STEPS = ((1, 0), (-1, 0), (0, 1), (0, -1))


def count_letters(letters):
    # how many times each letter appears
    counts = {}
    for ch in letters:
        counts[ch] = counts.get(ch, 0) + 1
    return counts


def choose(n, k):
    # the number of ways to pick k items out of n (n choose k)
    if k > n:
        return 0
    ways = 1
```

```
    for i in range(k):
        ways = ways * (n - i) // (i + 1)
    return ways


# ---- main question: main function ----
# PART 1: number of islands ('1' is land)
# Idea: scan every cell; an unseen land cell starts a new island, and a
# BFS marks all of its land as seen so it is counted once.
def num_islands(grid):
    if not grid:
        return 0
    rows, cols = len(grid), len(grid[0])
    seen = set()
    count = 0
```

```
    for r in range(rows):
        for c in range(cols):
            if grid[r][c] != "1" or (r, c) in seen:
                continue
            count += 1
            seen.add((r, c))
            queue = deque([(r, c)])
            while queue:
                x, y = queue.popleft()
                for dx, dy in STEPS:
                    nx, ny = x + dx, y + dy
                    if not (0 <= nx < rows and 0 <= ny < cols):
                        continue
                    if grid[nx][ny] == "1" and (nx, ny) not in seen:
                        seen.add((nx, ny))
                        queue.append((nx, ny))
    return count


# PART 2: in how many ways can the letters of an array spell GOOGLE?
# Idea: for every letter of the word, pick that many of the copies
# available in the array; multiply the choices (follow-up 1 allows any word).
def ways_to_spell(letters, word="GOOGLE"):
    have = count_letters(letters)
    need = count_letters(word)
    total = 1
    for ch in need:
        total = total * choose(have.get(ch, 0), need[ch])
    return total


# ---- main question: tests ----
def test_main_question():
    assert num_islands([]) == 0
    assert num_islands([["1"]]) == 1
    assert num_islands(["11000", "11000", "00100", "00011"]) == 3
    assert ways_to_spell([]) == 0
    assert ways_to_spell(list("GOOGLE")) == 1
    assert ways_to_spell(list("GOOOGLE")) == 3          # three O's: choose 2 of them
    assert ways_to_spell(list("GGGOOLEE")) == 3 * 2      # an extra G and an extra E
    assert ways_to_spell(list("GOGLE")) == 0
    assert ways_to_spell(list("AABB"), "AB") == 4        # any target word


# ======================================================
```

```
# FOLLOW-UP 1: Any target word, not only GOOGLE
# ======================================================
# Answer: already handled by the `word` parameter of ways_to_spell above.
```

```
# ============================================================
# FOLLOW-UP 2: The letters must appear in order (a subsequence of the array spells the word)
# ============================================================
# ---- follow-up 2: function ----
# Answer: this is the "distinct subsequences" count. ways[j] is the
# number of ways to spell the first j letters of the word; go through
# the array and update j from high to low so one letter is not used twice.
def ways_in_order(letters, word="GOOGLE"):
    ways = [0] * (len(word) + 1)
    ways[0] = 1
    for ch in letters:
        for j in range(len(word), 0, -1):
            if word[j - 1] == ch:
                ways[j] += ways[j - 1]
    return ways[len(word)]


# ---- follow-up 2: tests ----
def test_follow_up_2():
    assert ways_in_order(list("GOOGLE")) == 1
    assert ways_in_order(list("GGOOGLE")) == 2
    assert ways_in_order(list("GOGOLE")) == 0
    assert ways_in_order(list("EGOOGL")) == 0


# ============================================================
# FOLLOW-UP 3: Islands arrive one cell at a time
# ============================================================
# Answer: keep the islands in union-find and update the count per
# new cell; the code is IslandCounter in the "Number of Islands" question.


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_2()
    print("all tests passed")
```

```
if __name__ == "__main__":
    run_all_tests()
```

5 Find shortest relationship path using BFS — MEDIUM

Template fit: one start; also store parent[v] = u when you first reach v; stop at the target and walk parent back to rebuild the path

QUESTION

People are connected by relationships, given as an adjacency dict that maps each person to the people they are directly related to. shortest_path(adj, s, t) returns the shortest chain of people from s to t (both ends included), or None when t cannot be reached.

Function signature

def shortest_path(adj: dict, s, t) -> list | None

EXAMPLES

QUESTION

Example 1	
Input	adj = {'ann': ['bob', 'cy'], 'bob': ['dee'], 'cy': ['dee', 'eli'], 'dee': ['fay'], 'eli': ['fay'], 'fay': []} shortest_path(adj, 'ann', 'fay') shortest_path(adj, 'ann', 'ann') shortest_path(adj, 'fay', 'ann')
Output	shortest_path(adj, 'ann', 'fay') -> ['ann', 'bob', 'dee', 'fay'] shortest_path(adj, 'ann', 'ann') -> ['ann'] shortest_path(adj, 'fay', 'ann') -> null
Why	Several chains of 4 people exist; BFS finds the first. A person is trivially related to themselves, and fay has no outgoing relationships.

FULL CODE | main question first, then every follow-up, tests at the end

```
# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

from collections import deque
import heapq


# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
def walk_back(parent, target):
    # follow the parent links from target back to the start, then reverse
    path = [target]
    while parent[path[-1]] is not None:
        path.append(parent[path[-1]])
    path.reverse()
    return path


# ---- main question: main function ----
# Idea: queue the source and mark it visited. Pop a person, queue
# each unvisited neighbour with parent = this person, and stop when
# the target is reached; the first time BFS reaches a node is by a
# shortest path.
def shortest_path(adj, s, t):
    if s == t:
        return [s]
    parent = {s: None}
    queue = deque([s])
    while queue:
        u = queue.popleft()
        for v in adj.get(u, []):
            if v not in parent:
                parent[v] = u
                if v == t:
                    return walk_back(parent, t)
                queue.append(v)
    return None
```

```
# ---- main question: tests ----
def test_main_question():
    adj = {"a": ["b", "c"], "b": ["d"], "c": ["d"], "d": ["e"], "e": []}
```

```
    assert shortest_path(adj, "a", "e") in (["a", "b", "d", "e"], ["a", "c", "d", "e"])
    assert shortest_path(adj, "a", "a") == ["a"]
    assert shortest_path(adj, "e", "a") is None


# ======================================================
# FOLLOW-UP 1: The graph is huge (a social network)
# ======================================================
# ---- follow-up 1: helper functions ----
def expand_one_level(adj, frontier, mine, other):
    # visit everything one step beyond `frontier`; `mine` / `other` hold the
    # distances found from this side / the opposite side.
    # Returns (new frontier, length of a path found, or None).
    new_frontier = []
    for u in frontier:
        for v in adj.get(u, []):
            if v in other:
                return new_frontier, mine[u] + 1 + other[v]
            if v not in mine:
                mine[v] = mine[u] + 1
                new_frontier.append(v)
    return new_frontier, None


# ---- follow-up 1: function ----
# Answer: search from both ends and stop when the two searches meet.
# With branching b and distance d that explores about 2 * b^(d/2)
# nodes instead of b^d. Always grow the smaller side.
def bidirectional(adj, s, t):
    if s == t:
        return 0
    dist_s, dist_t = {s: 0}, {t: 0}
    front_s, front_t = [s], [t]
    while front_s and front_t:
        if len(front_s) <= len(front_t):
            front_s, found = expand_one_level(adj, front_s, dist_s, dist_t)
        else:
            front_t, found = expand_one_level(adj, front_t, dist_t, dist_s)
        if found is not None:
            return found
    return None
```

```
# ---- follow-up 1: tests ----
def test_follow_up_1():
    both_ways = {"a": ["b", "c"], "b": ["a", "d"], "c": ["a", "d"],
                 "d": ["b", "c", "e"], "e": ["d"]}
    assert bidirectional(both_ways, "a", "e") == 3
    assert bidirectional(both_ways, "a", "a") == 0


# ======================================================
# FOLLOW-UP 2: Return every shortest path, or count them
# ======================================================
# ---- follow-up 2: function ----
# Answer: remember, for each node, its distance and every parent that
# is exactly one step closer; then list the paths by walking back
# through those parent lists (counting paths is the same walk).
def all_shortest_paths(adj, s, t):
    dist = {s: 0}
    parents = {}
    queue = deque([s])
    while queue:
        u = queue.popleft()
        for v in adj.get(u, []):
            if v not in dist:
```

```
                dist[v] = dist[u] + 1
                queue.append(v)
            if dist[v] == dist[u] + 1:
                if v not in parents:
                    parents[v] = []
                parents[v].append(u)
    if t not in dist:
        return []
    paths = []

    def build(node, tail):
        if node == s:
            paths.append([s] + tail)
            return
        for p in parents[node]:
            build(p, [node] + tail)

    build(t, [])
    return paths
```

```
# ---- follow-up 2: tests ----
def test_follow_up_2():
    adj = {"a": ["b", "c"], "b": ["d"], "c": ["d"], "d": ["e"], "e": []}
    paths = all_shortest_paths(adj, "a", "e")
    assert sorted(paths) == [["a", "b", "d", "e"], ["a", "c", "d", "e"]]


# ============================================================
# FOLLOW-UP 3: Relationships have different strengths (weights)
# ============================================================
# ---- follow-up 3: function ----
# Answer: BFS is no longer right; use Dijkstra with a heap (weights
# must not be negative). Keep the same parent map to rebuild the path.
def weighted_path(graph, s, t):
    dist = {s: 0}
    parent = {s: None}
    heap = [(0, s)]
    while heap:
        d, u = heapq.heappop(heap)
        if d > dist[u]:
            continue                        # an older, longer entry
        for v, w in graph.get(u, []):
            if v not in dist or d + w < dist[v]:
                dist[v] = d + w
                parent[v] = u
                heapq.heappush(heap, (d + w, v))
    if t not in dist:
        return None
    return dist[t], walk_back(parent, t)


# ---- follow-up 3: tests ----
def test_follow_up_3():
    graph = {"a": [("b", 1), ("c", 5)], "b": [("c", 1)]}
    assert weighted_path(graph, "a", "c") == (2, ["a", "b", "c"])
    assert weighted_path(graph, "c", "a") is None


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
```

```
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

6 Farthest City for a New Data Center

MEDIUM

Template fit: same on a graph: starts = the data centres, neighbors = adjacency list; answer = the city with the largest distance

QUESTION

Cities are nodes 0..num_cities-1 of an undirected, unweighted graph; edges[i] = [u, v] is a direct connection. You are given a non-empty list dcs of cities that already host a data center. You want to build a new data center in the city that is as far as possible from the nearest existing data center — that is, the city whose shortest-path distance (in number of edges) to its closest data center is maximized. Return that city's index. Break ties by choosing the smallest index. It is guaranteed every city can reach at least one data center.

Function signature

def farthest_city(num_cities: int, edges: list[list[int]], dcs: list[int]) -> int:
    ...

Follow-up (as reported): Run a single BFS seeded from all data centers at once (distance 0) instead of a BFS per data center.

EXAMPLES

Example 1	
Input	`num_cities = 5, edges = [[0,1],[1,2],[2,3],[3,4]], dcs = [0]`
Output	`4`
Why	`Distances from city 0 are 0,1,2,3,4; city 4 is farthest.`

Example 2	
Input	`num_cities = 5, edges = [[0,1],[1,2],[2,3],[3,4]], dcs = [0, 4]`
Output	`2`
Why	`Nearest-DC distances are 0->0, 1->1, 2->2, 3->1, 4->0; city 2 is farthest at distance 2.`

CONSTRAINTS

- 1 <= num_cities <= 10**5 , 0 <= len(edges) <= 2*10**5 .
- 1 <= len(dcs) <= num_cities , dcs are distinct valid city indices.

FULL CODE | main question first, then every follow-up, tests at the end

```
# ====================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ====================================================

from collections import deque
import heapq
import random


# ====================================================
# MAIN QUESTION
# ====================================================
```

```
# ---- main question: helper functions ----
def index_of_largest(values):
    # the position of the biggest number (the first one on a tie)
    best = 0
    for i in range(len(values)):
        if values[i] > values[best]:
            best = i
    return best


# ---- main question: main function ----
# Idea: put every data center in the queue at distance 0 and run one
# BFS; the first time a city is reached is by its nearest data center.
# The answer is the city with the biggest distance.
def farthest_city(num_cities, edges, dcs):
    adj = []
    for i in range(num_cities):
        adj.append([])
    for u, v in edges:
        adj[u].append(v)
        adj[v].append(u)
    dist = [-1] * num_cities
    queue = deque()
    for d in dcs:
        dist[d] = 0
        queue.append(d)
    while queue:
        u = queue.popleft()
        for v in adj[u]:
            if dist[v] == -1:
```

```
                dist[v] = dist[u] + 1
                queue.append(v)
    return index_of_largest(dist)


# ---- main question: tests ----
def test_main_question():
    path = [[0, 1], [1, 2], [2, 3], [3, 4]]
    assert farthest_city(5, path, [0]) == 4
    assert farthest_city(5, path, [0, 4]) == 2


# ======================================================
# FOLLOW-UP 1: The roads have lengths. How does it change?
# ======================================================
# ---- follow-up 1: function ----
# Answer: replace the queue by a min-heap (Dijkstra) that starts with
# every data center at distance 0. The first time a city is taken out
# of the heap its distance is final. O((V + E) log V).
def farthest_city_weighted(num_cities, roads, dcs):
    adj = []
    for i in range(num_cities):
        adj.append([])
    for u, v, w in roads:
        adj[u].append((v, w))
        adj[v].append((u, w))
    dist = [float("inf")] * num_cities
    heap = []
    for d in dcs:
        dist[d] = 0
        heap.append((0, d))
    heapq.heapify(heap)
    while heap:
        d, u = heapq.heappop(heap)
        if d > dist[u]:
```

```
            continue
        for v, w in adj[u]:
            if d + w < dist[v]:
                dist[v] = d + w
                heapq.heappush(heap, (dist[v], v))
    return index_of_largest(dist)


# ---- follow-up 1: tests ----
def test_follow_up_1():
```

```
    # with every road length 1 the weighted version must agree with the plain BFS
    random.seed(58)
    for _ in range(400):
        n = random.randint(2, 8)
        edges = []
        for i in range(1, n):
            edges.append([i, random.randrange(i)])        # keeps the graph connected
        for _ in range(random.randint(0, 3)):
            edges.append([random.randrange(n), random.randrange(n)])
        dcs = random.sample(range(n), random.randint(1, min(3, n)))
        unit = []
        for u, v in edges:
            unit.append([u, v, 1])
        assert farthest_city_weighted(n, unit, dcs) == farthest_city(n, edges, dcs)


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

7 Compute distance to nearest taxi in grid

MEDIUM

Template fit: starts = every taxi cell; usable = not a wall; the dist table is the answer, cells never reached stay -1

QUESTION

A grid has 'T' for a taxi, '#' for a blocked cell and '.' for an open cell. For every cell return the number of steps (up, down, left or right, never through '#') to the nearest taxi: 0 for a taxi, and -1 for a blocked cell or a cell no taxi can reach.

Function signature

def taxi_distances(grid: list) -> list

EXAMPLES

Example 1	
Input	`['T.#', '...', '#.T']`
Output	`[[0, 1, -1], [1, 2, 1], [-1, 1, 0]]`
Why	The two taxis are in opposite corners; the middle cell is 2 steps from both; the blocked cells show -1.

QUESTION

Example 2	
Input	['..', '..']
Output	[[-1, -1], [-1, -1]]
Why	No taxi at all: every open cell is unreachable.

FULL CODE | main question first, then every follow-up, tests at the end

```
# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ========================================================

from collections import deque


# ========================================================
# MAIN QUESTION
# ========================================================
# ---- main question: helper functions ----
STEPS = ((1, 0), (-1, 0), (0, 1), (0, -1))


def spread(grid, dist, queue, allow_improve=False):
    # grow the distances outward from the cells in `queue`;
    # with allow_improve a cell may be lowered when a closer source reaches it
    rows, cols = len(grid), len(grid[0])
    while queue:
        r, c = queue.popleft()
        for dr, dc in STEPS:
            nr, nc = r + dr, c + dc
            if not (0 <= nr < rows and 0 <= nc < cols):
                continue
            if grid[nr][nc] == "#":
                continue
            new = dist[r][c] + 1
            never_seen = dist[nr][nc] == -1
            if never_seen or (allow_improve and new < dist[nr][nc]):
                dist[nr][nc] = new
                queue.append((nr, nc))


def empty_distances(grid):
    # a table the size of the grid, -1 everywhere
    table = []
    for row in grid:
        table.append([-1] * len(row))
    return table


# ---- main question: main function ----
```

```
# Idea: 'T' taxi, '.' road, '#' wall. Start one BFS from every taxi
# at once; a cell gets its distance the first time it is reached,
# and cells no taxi can reach stay at -1.
def taxi_distances(grid):
    dist = empty_distances(grid)
    queue = deque()
    for r in range(len(grid)):
        for c in range(len(grid[0])):
            if grid[r][c] == "T":
```

```
                dist[r][c] = 0
                queue.append((r, c))
    spread(grid, dist, queue)
    return dist


# ---- main question: tests ----
def test_main_question():
    d = taxi_distances(["T...", ".#..", "...T"])
    assert d[0] == [0, 1, 2, 2]                    # the taxi at the bottom right is closer to (0, 3)
    assert d[2] == [2, 2, 1, 0]
    assert d[1][1] == -1


# ======================================================
# FOLLOW-UP 1: A new taxi appears
# ======================================================
# ---- follow-up 1: function ----
# Answer: start a BFS from the new taxi but only continue into cells
# where it improves the stored distance. Distances only go down, so
# the work is bounded by the cells whose answer really changes.
def add_taxi(grid, dist, r, c):
    dist[r][c] = 0
    spread(grid, dist, deque([(r, c)]), allow_improve=True)


# ---- follow-up 1: tests ----
def test_follow_up_1():
    grid = ["T...", ".#..", "...T"]
    d = taxi_distances(grid)
    add_taxi(grid, d, 0, 2)                        # a new taxi in the middle of the top row
    assert d[0] == [0, 1, 0, 1]
    assert d[2][0] == 2
    assert taxi_distances(["T#.", "##."])[0][2] == -1   # unreachable cells stay -1
```

```
# ======================================================
# FOLLOW-UP 2: Taxis move or disappear
# ======================================================
# Answer: a disappearing taxi can make many distances larger, so
# recompute with one BFS from the taxis that are left (or keep, per
# cell, the two nearest taxis so one removal is cheap).


# ======================================================
# FOLLOW-UP 3: Return which taxi is nearest, not only the distance
# ======================================================
# ---- follow-up 3: function ----
# Answer: store a taxi number in each cell; a cell copies the number
# of the cell that first reached it. Ties go to whichever got there
# first, so say how you want ties broken.
def nearest_taxi_ids(grid):
    rows, cols = len(grid), len(grid[0])
    owner = []
    for row in grid:
        owner.append([None] * len(row))
    queue = deque()
    taxi_id = 0
    for r in range(rows):
        for c in range(cols):
            if grid[r][c] == "T":
                owner[r][c] = taxi_id
                taxi_id += 1
                queue.append((r, c))
    while queue:
        r, c = queue.popleft()
```

```
        for dr, dc in STEPS:
            nr, nc = r + dr, c + dc
            if not (0 <= nr < rows and 0 <= nc < cols):
                continue
            if grid[nr][nc] != "#" and owner[nr][nc] is None:
                owner[nr][nc] = owner[r][c]
                queue.append((nr, nc))
    return owner


# ---- follow-up 3: tests ----
def test_follow_up_3():
    ids = nearest_taxi_ids(["T...", ".#..", "...T"])
    assert ids[0][0] == 0 and ids[2][3] == 1
    assert ids[1][1] is None
```

```
# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

8 Infection Spread on a Grid (Cellular Automaton) HARD

Template fit: starts = every infected cell; usable = healthy cell; the answer is the largest distance reached (number of rounds)

QUESTION

A grid holds '.' for a healthy cell, 'I' for an infected cell and any other character (such as '#') for a barrier that is never infected. Every step, all infected cells infect their healthy neighbours (up, down, left, right) at the same time. infection_times(grid) returns the step at which each infected cell was infected (0 for the cells infected at the start); spread_steps(grid) returns the number of steps until nothing new gets infected.

Function signature

def infection_times(grid: list) -> dict
def spread_steps(grid: list) -> int

EXAMPLES

Example 1	
Input	`['I..', '...', '..#']`
Output	`3`
Why	`The infection travels from the corner: (0,1) and (1,0) at step 1, then three cells at step 2, and so on. The barrier at the bottom right is never infected.`

Example 2	
Input	`['I...']`
Output	`{(0, 0): 0, (0, 1): 1, (0, 2): 2, (0, 3): 3}`
Why	`A one-row grid: one cell per step.`

QUESTION

Example 3	
Input	`['I#.', 'I#.']`
Output	`0`
Why	Both cells next to the barrier are already infected and the barrier blocks the right column, so nothing can spread: 0 steps.

FULL CODE | main question first, then every follow-up, tests at the end

```
# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ========================================================

# ========================================================
# MAIN QUESTION
# ========================================================
# ---- main question: helper functions ----
STEPS = ((1, 0), (-1, 0), (0, 1), (0, -1))


def find_infected(grid):
    # the cells that start infected ('I')
    cells = []
    for r in range(len(grid)):
        for c in range(len(grid[0])):
            if grid[r][c] == "I":
                cells.append((r, c))
    return cells


# ---- main question: main function ----
# Idea: only the cells infected in the last step can infect new
# cells, so keep a frontier. 'I' = infected at time 0, '.' =
# healthy, '#' = blocked. Returns the step at which each cell
# gets infected.
def infection_times(grid):
    rows, cols = len(grid), len(grid[0])
    when = {}
    frontier = find_infected(grid)
    for cell in frontier:
        when[cell] = 0
    step = 0
    while frontier:
        step += 1
        next_frontier = []
        for r, c in frontier:
            for dr, dc in STEPS:
                nr, nc = r + dr, c + dc
                if 0 <= nr < rows and 0 <= nc < cols:
                    if grid[nr][nc] == "." and (nr, nc) not in when:
                        when[(nr, nc)] = step       # every healthy neighbour is infected next step
```

```
                        next_frontier.append((nr, nc))
        frontier = next_frontier
    return when


def spread_steps(grid):
    # steps until nothing changes any more
    when = infection_times(grid)
```

```
    last = 0
    for step in when.values():
        if step > last:
            last = step
    return last


# ---- main question: tests ----
def test_main_question():
    assert spread_steps(["I..", "...", "..."]) == 4
    wall = ["I#.", "I#."]                          # a wall splits the grid: the far cell is never infected
    assert spread_steps(wall) == 0
    assert spread_steps(["..."]) == 0


# =========================================================
# FOLLOW-UP 1: Add a recovery delay: an infected cell recovers after d steps and cannot spread afterwards
# =========================================================
# ---- follow-up 1: function ----
# Answer: a cell infects every healthy neighbour on the very next
# step, so recovery does not change how long spreading takes; it
# only changes what the grid looks like at time t.
def grid_at(grid, t, recovery=None):
    when = infection_times(grid)
    result = []
    for r in range(len(grid)):
        line = ""
        for c in range(len(grid[0])):
            start = when.get((r, c))
            if start is None or start > t:
                line += grid[r][c]                 # not infected yet
            elif recovery and t >= start + recovery:
                line += "R"                        # recovered
            else:
                line += "I"
        result.append(line)
    return result
```

```
# ---- follow-up 1: tests ----
def test_follow_up_1():
    g = ["I..", "...", "..."]
    assert grid_at(g, 1) == ["II.", "I..", "..."]
    assert grid_at(g, 1, recovery=1) == ["RI.", "I..", "..."]
    assert grid_at(["I#.", "I#."], 9)[0] == "I#."


# =========================================================
# FOLLOW-UP 2: Why can you stop early?
# =========================================================
# Answer: when a step infects nobody new the frontier is empty and
# nothing can change again, so the loop ends by itself.


# =========================================================
# FOLLOW-UP 3: Make the spread probabilistic
# =========================================================
# Answer: infect each healthy neighbour only with probability p
# (random.random() < p) and repeat the whole simulation many times
# to estimate the average spread time.


# =========================================================
# RUN EVERYTHING
# =========================================================
```

```
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

9 Shortest Distance From All Buildings — HARD

Template fit: run it once per building (one start each); add each run into a total table and count how many buildings reached each cell

QUESTION

You are given a 2D grid where each cell is:
0 = empty land (a school can be built here), 1 = a house, 2 = a tree (an obstacle that cannot be entered or crossed). You want to build one school on an empty cell so that the total travel distance from every house to the school is minimized. Distance is measured by 4-directional steps (up/down/left/right); trees block movement. A valid school location must be reachable from every house. Return the coordinates (row, col) of the empty cell with the minimum total distance to all houses. If several cells tie on total distance, return the one that comes first in row-major order (smallest row, then smallest column). If no empty cell can reach all houses, return (-1, -1) .

Function signature

def shortest_distance_from_all_buildings(grid: list[list[int]]) -> tuple[int, int]:
    ...

Follow-up (as reported): (Reported) Minimize the average distance instead of the total. Since the number of houses is fixed, the minimizing cell is the same, but track the count to report the average.

EXAMPLES

Example 1	
Input	`grid = [[1, 0, 2, 0, 1], [0, 0, 0, 0, 0], [0, 0, 1, 0, 0]]`
Output	`(1, 2)`
Why	`Cell (1,2) reaches all three houses with total distance 3 + 3 + 1 = 7, the minimum over all reachable empty cells.`

Example 2	
Input	`grid = [[1, 0, 1]]`
Output	`(0, 1)`
Why	`The only empty cell is distance 1 from each house; total 2.`

Example 3	
Input	`grid = [[1, 2, 1]]`
Output	`(-1, -1)`
Why	`There is no empty cell, so no valid location exists.`

CONSTRAINTS

- 1 <= rows, cols <= 50
- Each cell is 0 , 1 , or 2 .
- There is at least one house.

FULL CODE | main question first, then every follow-up, tests at the end

```python
# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ========================================================

from collections import deque


# ========================================================
# MAIN QUESTION
# ========================================================
# ---- main question: helper functions ----
def bfs_from_house(grid, house, total, reached):
    # walk outward from one house over empty cells (0); add the distance to
    # `total` and count one more house in `reached` for every cell it reaches
    rows, cols = len(grid), len(grid[0])
    seen = {house}
    queue = deque([(house[0], house[1], 0)])
    while queue:
        r, c, d = queue.popleft()
        for nr, nc in ((r + 1, c), (r - 1, c), (r, c + 1), (r, c - 1)):
            if not (0 <= nr < rows and 0 <= nc < cols):
                continue
            if grid[nr][nc] != 0 or (nr, nc) in seen:
                continue
            seen.add((nr, nc))
            total[nr][nc] += d + 1
            reached[nr][nc] += 1
            queue.append((nr, nc, d + 1))


def school_totals(grid):
    # for every empty cell: the sum of the distances to all houses it can reach,
    # how many houses reach it, and the number of houses
    rows, cols = len(grid), len(grid[0])
    total = []
    reached = []
    for r in range(rows):
        total.append([0] * cols)
        reached.append([0] * cols)
    houses = 0
    for r in range(rows):
        for c in range(cols):
            if grid[r][c] == 1:
                houses += 1
                bfs_from_house(grid, (r, c), total, reached)
    return total, reached, houses


# ---- main question: main function ----
# Idea: one BFS per house adds its distances to the empty cells it
# reaches; a valid cell is one that every house reached. Pick the valid
# cell with the smallest total; (-1, -1) if there is none.
def shortest_distance_from_all_buildings(grid):
    total, reached, houses = school_totals(grid)
    best = None
    spot = (-1, -1)
    for r in range(len(grid)):
        for c in range(len(grid[0])):
            if grid[r][c] == 0 and reached[r][c] == houses:
                if best is None or total[r][c] < best:
                    best = total[r][c]
                    spot = (r, c)
    return spot
```

```python
# ---- main question: tests ----
def test_main_question():
    assert shortest_distance_from_all_buildings(
        [[1, 0, 2, 0, 1], [0, 0, 0, 0, 0], [0, 0, 1, 0, 0]]) == (1, 2)
    assert shortest_distance_from_all_buildings([[1, 0, 1]]) == (0, 1)
    assert shortest_distance_from_all_buildings([[1, 2, 1]]) == (-1, -1)


# ========================================================
# FOLLOW-UP 1: Minimize the average distance instead of the total
# ========================================================
# ---- follow-up 1: function ----
# Answer: with a fixed number of houses the average is total / houses,
# so the best cell is the same; divide once at the end to report it.
def best_school_average(grid):
    spot = shortest_distance_from_all_buildings(grid)
    if spot == (-1, -1):
        return spot, None
    total, reached, houses = school_totals(grid)
    return spot, total[spot[0]][spot[1]] / houses
```

```python
# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert best_school_average([[1, 0, 1]]) == ((0, 1), 1.0)
    assert best_school_average([[1, 2, 1]]) == ((-1, -1), None)


# ========================================================
# RUN EVERYTHING
# ========================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

Sweep over events in time order

10 Meeting Rooms / Interval Scheduling Variants

MEDIUM

Template fit: the template as written: the largest running count is the number of rooms

QUESTION

LC 253 (Meeting Rooms II) appears in many forms across phone screen and onsite coding rounds — minimum number of rooms, room assignment, cyclic time, daily server jobs, gate entry/exit. Common follow-ups push toward sweep line, min-heap, or distributed processing.

The base task in every variant: given a list of intervals (start, end) — or pickup/return pairs, meeting time pairs, server job runs — answer one or more of:

Min rooms / cars / servers needed to honor all intervals concurrently (the canonical min-meeting-rooms problem).
Assignment: which interval goes to which room/car, so the caller can use the rental record.
Single-interval overlap test between two intervals.

Variant: parking-lot capacity

Arrival and departure intervals represent cars occupying a parking lot. Compute the peak simultaneous occupancy using the same interval-overlap model.

Variant: cyclic time (24h wrap)

Intervals wrap around midnight; if an interval ends after the day boundary, it carries into the next day.
Follow-up: "if N is huge, how do you optimize?" — expect a sweep-line discussion.

Variant: daily-recurring server jobs

Same set of jobs runs every day. If a job can't finish in one day, truncate at midnight and resume next day.
A server can run multiple identical jobs that overlap on the same machine; different jobs need separate servers.
Follow-up: a single job that runs many days needs its own dedicated server.

Variant: on-call rotation consolidation

Input: on-call rotations {name, start, end} using half-open intervals.
Output maximal contiguous segments during which the active on-call set does not change; omit gaps with nobody on call.
Segments are sorted by start, and names in each segment are the active set for that time range.
Clean approach: sort starts, keep a min-heap by end time, and emit a segment whenever the next start or next end changes the active set.

Variant: gate entry / exit with priority rules

Array of (timestamp, action) where action in {enter, exit}; each index is a different person.
Return for each person what time they actually enter/exit, applying tie-break rules:

If the prior tick had an enter, enter wins on a tie.
If the prior tick had an exit, exit wins on a tie.
If the prior tick had no activity, exit wins.
Within the winning class, lower index wins.

Optimization: instead of stepping time tick-by-tick, jump through the sorted input timestamps to reach O(n).

Variant: Google Calendar free-time intersection

Input: requested attendees plus each person's busy calendar intervals.
Return every time range where all requested attendees are free.
Normalize each person's busy blocks into free gaps, then intersect the candidate ranges across attendees.

Variant: car-rental assignment

QUESTION

Input: rental orders containing an order ID, pickup time, and return time.
Assign every order to a car so that no assigned intervals overlap, minimize the number of cars, and return each car's complete order list. Model each car explicitly with its own rental record.
A current output variant instead returns a dictionary from each reservation ID to its assigned car ID; the follow-up asks for the final fleet size.
A return at time t and another pickup at time t may reuse the same car.
Follow-up: if the fleet is capped at K cars and all requests cannot be served, define how the allocator detects and handles the failure.

Variant: exact-concurrency intervals

Compute the maximum number of simultaneously active meetings.
Given a target concurrency X, also return every maximal time range during which exactly X meetings are active.
Define the event ordering when one meeting starts at the same timestamp another ends; this choice controls whether a zero-width overlap is counted.

Variant: maximum-profit job scheduling

Each job has startTime, endTime, and profit. Choose a set of non-overlapping jobs maximizing total profit. The candidate identified it as LC 1235 (Maximum Profit in Job Scheduling).
Asked as the first question of an L4 onsite round, followed by an unrelated range-query problem in the same slot.

On-call employee metadata variant

Each duty record includes employee ID, employee name, start time, and end time. Return the employee count and identifying information for each time segment.

EXAMPLES

```
Canonical min-meeting-rooms example: [[0,30],[5,10],[15,20]] -> 2.
On-call consolidation: A[10,50], B[20,60], C[30,40], D[30,40] -> [10,20]: A, [20,30]: A,B, [30,40]:
    A,B,C,D, [40,50]: A,B, [50,60]: B.
Single-interval overlap: intervalsOverlap([1,5],[4,7]) -> true.
```

Long-running job: a 36-hour daily job is impossible under the truncate rule; reserve a separate server.

FULL CODE | main question first, then every follow-up, tests at the end

```python
# ======================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ======================================================

import heapq


# ======================================================
# MAIN QUESTION
# ======================================================
# ---- main question: main functions ----
# The minimum number of rooms for meetings (start, end).
# Idea (sweep line): a +1 event at every start and a -1 event at every end;
# sort them so an END at time t comes before a START at time t (a room is
# free the moment a meeting ends), and track the largest running count.
def min_rooms(intervals):
    events = []
    for start, end in intervals:
        events.append((start, 1))
        events.append((end, -1))
    events.sort()                                   # (-1) sorts before (+1) at equal times
    running = 0
    best = 0
```

```
    for time, delta in events:
        running += delta
        if running > best:
            best = running
    return best


# VARIANT: which room does each meeting get?
def assign_rooms(intervals):
    order = []
    for i in range(len(intervals)):
        order.append((intervals[i][0], intervals[i][1], i))
    order.sort()
    heap = []
    rooms = [0] * len(intervals)
    used = 0
    for start, end, i in order:
        if heap and heap[0][0] <= start:
            freed_at, room = heapq.heappop(heap)    # reuse the room that frees first, if it is free
```

```
        else:
            room = used
            used += 1
        rooms[i] = room
        heapq.heappush(heap, (end, room))
    return rooms


# VARIANT: on-call consolidation. rotations are (name, start, end), half-open.
# Return the segments (start, end, set of people on call), merging neighbouring
# segments with the same people and leaving out the gaps.
def oncall_segments(rotations):
    changes = {}                                    # time -> the (name, +1 or -1) changes at that time
    for name, start, end in rotations:
        if start not in changes:
            changes[start] = []
        changes[start].append((name, 1))
        if end not in changes:
            changes[end] = []
        changes[end].append((name, -1))
    active = {}
    segments = []
    times = sorted(changes)
    for i in range(len(times) - 1):
        t = times[i]
        following = times[i + 1]
        for name, change in changes[t]:
            active[name] = active.get(name, 0) + change
        people = set()
        for name in active:
            if active[name] != 0:
                people.add(name)
        who = frozenset(people)
        if not who:
            continue                                # nobody on call: a gap
        if segments and segments[len(segments) - 1][1] == t and segments[len(segments) - 1][2] == who:
            segments[len(segments) - 1] = (segments[len(segments) - 1][0], following, who)
        else:
            segments.append((t, following, who))
    return segments


# ---- main question: tests ----
def test_main_question():
    base = [(0, 30), (5, 10), (15, 20)]
```

```
    assert min_rooms(base) == 2
    assert min_rooms([(5, 8), (8, 9)]) == 1        # touching meetings share a room
    rooms = assign_rooms(base)
    assert rooms[0] != rooms[1] and rooms[1] == rooms[2]
    assert oncall_segments([("A", 1, 5), ("B", 3, 8)]) == [(1, 3, frozenset("A")), (3, 5, frozenset("AB")), (
5, 8, frozenset("B"))]
    assert len(oncall_segments([("A", 1, 2), ("B", 4, 5)])) == 2     # a gap with nobody on call is left out


# ============================================================
# FOLLOW-UP 1: Meetings wrap around midnight (cyclic time)
# ============================================================
# ---- follow-up 1: function ----
# Answer: split every meeting that wraps ([s, e) with s >= e) into [s, 24h)
# and [0, e) and run the same sweep: the overlap at any clock time is
# unchanged. (A meeting exactly 24 hours long covers the whole day.)
def min_rooms_cyclic(intervals, day=24 * 60):
    parts = []
    for start, end in intervals:
        if start < end:
            parts.append((start, end))
        else:
            parts.append((start, day))             # wraps past midnight: split at the boundary
            parts.append((0, end))
    return min_rooms(parts)


# ---- follow-up 1: tests ----
def test_follow_up_1():
    wrap = [(22 * 60, 2 * 60), (23 * 60, 60), (0, 3 * 60)]
    assert min_rooms_cyclic(wrap) == 3


# ============================================================
# FOLLOW-UP 2: The input is huge and spread over many machines
# ============================================================
# Answer: each machine computes its own list of +1 / -1 events, sorted;
# merge the sorted event lists (a k-way merge, as in the interval-union
# question) and run the same running count: memory O(number of machines).


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
```

```
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

11 Train / Flight Schedule Reachability — MEDIUM

Template fit: sort legs by departure; a leg is usable if you have already reached its origin before it leaves; keep the earliest arrival per place

QUESTION

Given trains (or flights) as (from, to, departure, arrival), decide whether you can travel from a source to a destination, boarding a later leg only if it departs after you arrive. Graph reachability with time state, solved with BFS / DFS.

QUESTION

Input: a schedule of legs, each with a departure station, an arrival station, a departure time, and an arrival time; plus a source and a destination.
Output: whether the destination is reachable from the source, where a leg can only be taken if it departs after you arrive at its departure station.
The same prompt appears as a flight schedule between airports in new-grad R1.

EXAMPLES

```
trains = [
  ("A", "Y", dep=1, arr=20),
  ("A", "Y", dep=2, arr=5),
  ("Y", "B", dep=6, arr=9)
]
source = "A", destination = "B"

Output: true — the first A->Y train arrives too late for the Y->B connection, but the second one makes it
.
```

FULL CODE | main question first, then every follow-up, tests at the end

```python
# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

import random

INF = float("inf")


# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
def departure_of(leg):
    # a leg is (from station, to station, departure time, arrival time)
    return leg[2]


# ---- main question: main functions ----
# Which stations can you reach from `source`? Time is part of the state:
# you can only take a leg if you are already at its station BEFORE it leaves.
# Idea: sort the legs by departure time and keep, for every station, the
# EARLIEST time you can be there (the source: minus infinity, you are there
# from the start). Going through the legs in departure order, every leg that
# could feed the current one has already been handled. A leg is usable if we
# are at its station strictly before it departs (strict=False allows a
# connection at the same minute).
def earliest_arrivals(legs, source, strict=True):
    earliest = {source: -INF}
    for from_station, to_station, departs, arrives in sorted(legs, key=departure_of):
        have = earliest.get(from_station, INF)
        usable = have < departs or (not strict and have == departs)
        if usable and arrives < earliest.get(to_station, INF):
            earliest[to_station] = arrives
    return earliest


def reachable(legs, source, dest, strict=True):
    return dest in earliest_arrivals(legs, source, strict)
```

```
# ---- main question: tests ----
```

```
LEGS = [("A", "Y", 1, 20), ("A", "Y", 2, 5), ("Y", "B", 6, 9)]


def test_main_question():
    assert reachable(LEGS, "A", "B")
    assert not reachable(LEGS[:1] + LEGS[2:], "A", "B")        # without the early A to Y train the connectio
n is lost
    assert earliest_arrivals(LEGS, "A")["B"] == 9
    assert reachable(LEGS, "A", "A")
    touching = [("A", "B", 1, 5), ("B", "C", 5, 8)]
    assert not reachable(touching, "A", "C")                   # arriving at 5 and leaving at 5 is too tight
...
    assert reachable(touching, "A", "C", strict=False)         # ... unless same-minute connections are allow
ed


# ============================================================
# FOLLOW-UP 1: Return the earliest arrival time at the destination
# ============================================================
# Answer: earliest_arrivals already holds it: earliest_arrivals(legs, source)[dest].


# ============================================================
# FOLLOW-UP 2: Print the route, not just yes or no
# ============================================================
# ---- follow-up 2: function ----
# Answer: remember the leg that last improved each station's earliest
# arrival, then walk back from the destination to the source and reverse.
def itinerary(legs, source, dest, strict=True):
    earliest = {source: -INF}
    came_by = {}                                    # station -> the leg that got us there
    for leg in sorted(legs, key=departure_of):
        from_station, to_station, departs, arrives = leg
        have = earliest.get(from_station, INF)
        usable = have < departs or (not strict and have == departs)
        if usable and arrives < earliest.get(to_station, INF):
            earliest[to_station] = arrives
            came_by[to_station] = leg
    if dest not in earliest:
        return None
    route = []
    while dest != source:
        leg = came_by[dest]
        route.append(leg)
        dest = leg[0]
    route.reverse()
    return route
```

```

# ---- follow-up 2: tests ----
def reachable_slowly(legs, source, dest):
    # a different way to check: keep trying every leg until nothing changes
    earliest = {source: -INF}
    changed = True
    while changed:
        changed = False
        for from_station, to_station, departs, arrives in legs:
            if earliest.get(from_station, INF) < departs and arrives < earliest.get(to_station, INF):
                earliest[to_station] = arrives
                changed = True
    return dest in earliest


def test_follow_up_2():
```

```
    assert itinerary(LEGS, "A", "B") == [LEGS[1], LEGS[2]]
    assert itinerary(LEGS, "B", "A") is None
    random.seed(23)
    for round_number in range(300):
        legs = []
        for i in range(random.randint(0, 8)):
            departs = random.randint(0, 9)
            legs.append((random.choice("ABCD"), random.choice("ABCD"), departs, departs + random.randint(1, 4
)))
        source = random.choice("ABCD")
        dest = random.choice("ABCD")
        assert reachable(legs, source, dest) == reachable_slowly(legs, source, dest)


# =============================================================
# FOLLOW-UP 3: Many queries on the same schedule: what changes in the graph representation?
# =============================================================
# Answer: build the graph once with one NODE per (station, time) event and an
# edge for every leg plus "wait" edges between a station's consecutive times.
# Then every query is a search from the source's first node; answers for the
# same source can be shared.


# =============================================================
# RUN EVERYTHING
# =============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_2()
    print("all tests passed")
```

```
if __name__ == "__main__":
    run_all_tests()
```

12 Solve Rooms and Top-K Streams

MEDIUM

Template fit: sort starts and ends separately and walk two pointers: a start before the earliest end needs a new room; the stream part is a size-K heap

QUESTION

Two small tasks. (1) min_rooms(intervals): the minimum number of rooms needed to hold all meetings; a meeting may start exactly when another ends. (2) TopKStream(k): numbers arrive one at a time with add(x); top() returns the k largest numbers seen so far, largest first.

Function signature

def min_rooms(intervals: list) -> int
class TopKStream:
  def __init__(self, k: int)
  def add(self, x: int) -> None
  def top(self) -> list

EXAMPLES

Example 1	
Input	`[(0, 30), (5, 10), (15, 20)]`
Output	`2`
Why	`(5, 10) and (15, 20) both overlap the long meeting (0, 30): 2 rooms.`

QUESTION

Example 2	
Input	`[(7, 10), (2, 4)]`
Output	`1`
Why	`The second meeting ends before the first starts: one room is enough.`

Example 3	

Input	`s = TopKStream(3)`
	`s.add(5)`
	`s.add(1)`
	`s.add(9)`
	`s.add(7)`
	`s.top()`

Output	`s.top() -> [9, 7, 5]`
Why	`Of 5, 1, 9, 7 the three largest are 9, 7 and 5.`

FULL CODE | main question first, then every follow-up, tests at the end

```
# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ========================================================

import heapq


# ========================================================
# MAIN QUESTION
# ========================================================
# ---- main question: main functions ----
# PART 1: the minimum number of rooms for meetings (start, end).
# Idea: sort the starts and the ends SEPARATELY. For each start: if it is at
# or after the earliest end that is still unused, that room is reused (move the
# end pointer on); otherwise a new room is needed.
def min_rooms(intervals):
    starts = []
    ends = []
    for start, end in intervals:
        starts.append(start)
        ends.append(end)
    starts.sort()
    ends.sort()
    rooms = 0
    j = 0                                          # the earliest end not yet used
    for start in starts:
        if start >= ends[j]:
            j += 1                                 # the earliest-ending meeting is over: reuse its room
        else:
            rooms += 1
    return rooms


# PART 2: the k largest values seen so far in a stream (fixed k).
# Idea: a min-heap of size k: its root is the k-th largest. A new value
# enters only if it beats the root.
class TopKStream:
    def __init__(self, k):
        self.k = k
        self.heap = []
```

```
    def add(self, x):
        if len(self.heap) < self.k:
            heapq.heappush(self.heap, x)
        elif x > self.heap[0]:
            heapq.heapreplace(self.heap, x)

    def top(self):
        return sorted(self.heap, reverse=True)


# ---- main question: tests ----
def test_main_question():
    assert min_rooms([(0, 30), (5, 10), (15, 20)]) == 2
    assert min_rooms([(1, 3), (3, 5)]) == 1                    # a meeting may start the moment another end
s
    assert min_rooms([]) == 0
    stream = TopKStream(3)
    for x in [5, 1, 9, 7, 3, 8]:
        stream.add(x)
    assert stream.top() == [9, 8, 7]


# ========================================================
# FOLLOW-UP 1: Rooms: return which room each meeting uses
# ========================================================
# ---- follow-up 1: function ----
# Answer: sort by start and keep a min-heap of (end time, room). If the
# earliest-ending room is free at the next start, reuse it, otherwise open a
# new room. The heap never holds more rooms than the answer.
def assign_rooms(intervals):
    order = []
    for i in range(len(intervals)):
        order.append((intervals[i][0], intervals[i][1], i))
    order.sort()
    heap = []
    rooms = [0] * len(intervals)
    opened = 0
    for start, end, i in order:
        if heap and heap[0][0] <= start:
            freed_at, room = heapq.heappop(heap)
        else:
            room = opened
            opened += 1
        rooms[i] = room
        heapq.heappush(heap, (end, room))
    return rooms
```

```
# ---- follow-up 1: tests ----
def test_follow_up_1():
    rooms = assign_rooms([(0, 30), (5, 10), (15, 20)])
    assert rooms[0] != rooms[1] and rooms[1] == rooms[2]


# ========================================================
# FOLLOW-UP 2: Stream: K changes over time, or you need the median
# ========================================================
# ---- follow-up 2: function ----
# Answer: a heap of fixed size k cannot grow cheaply when k changes: keep
# the sorted values (or a balanced tree) instead. For the MEDIAN use two
# heaps: the lower half in a max-heap (negated numbers) and the upper half in
# a min-heap, with sizes differing by at most one.
class MedianStream:
    def __init__(self):
```

```
        self.low = []                               # max-heap (negated numbers)
        self.high = []                              # min-heap

    def add(self, x):
        heapq.heappush(self.low, -x)
        top = -heapq.heappop(self.low)
        heapq.heappush(self.high, top)
        if len(self.high) > len(self.low):
            back = heapq.heappop(self.high)
            heapq.heappush(self.low, -back)

    def median(self):
        if len(self.low) > len(self.high):
            return float(-self.low[0])
        return (-self.low[0] + self.high[0]) / 2


# ---- follow-up 2: tests ----
def test_follow_up_2():
    stream = MedianStream()
    for x in [5, 2, 8, 1]:
        stream.add(x)
    assert stream.median() == 3.5


# ========================================================
# FOLLOW-UP 3: Rooms: meetings on several floors with capacities
# ========================================================
```

```
# Answer: keep one heap per floor (or per room size); a meeting asks for
# the smallest room that is big enough and free, using the same idea: take the
# room that frees up first among the rooms that fit.


# ========================================================
# RUN EVERYTHING
# ========================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

13 Compute servers needed for daily recurring jobs — MEDIUM

Template fit: merge each job's own intervals first so one job counts once, then sweep all of them together

QUESTION

You operate a cluster of identical servers that run recurring **daily** jobs. For a single day, you are given a list of job execution intervals. Each interval has:

- **`job_id`** (string or integer),
- **`start_time`** (timestamp within that day),
- **`end_time`** (timestamp within that day), with **`start_time < end_time`**. Interpret each interval as a half-open time range **`[start_time, end_time)`** during which that job must be running on some server **on that day**. Rules and constraints:
- Each physical server can execute at most **one job at a time**.

QUESTION

- However, if multiple intervals in the input correspond to the **same `job_id`**, they may overlap in time **on the same server** (they are considered parts or retries of the same job and do not consume extra capacity).
- If a job's true execution would extend past midnight into the next day, it is conceptually **truncated at the end of the current day** for scheduling purposes; for each day you only see the portion that falls within that day's 24-hour window.

Tasks

1. Given all job intervals for a single day (potentially with repeated **`job_id`**s and overlapping intervals), compute the **minimum number of servers** required to schedule all the jobs under the above rules.
2. Describe the time and space complexity of your algorithm in terms of the number of intervals (m).

Follow-up

Some rare jobs are extremely long-running and conceptually span **multiple days** without restart. For such a multi-day job, you decide to reserve a **dedicated server** that is not shared with any other jobs, but which may still handle different intervals of that same **`job_id`**. Explain how you would extend or adapt your solution to:

- Detect which jobs require a dedicated server based on their multi-day duration or configuration.
- Account for these dedicated servers when computing the total number of servers needed for the system.

Part 1: Minimum Servers for One Day with Overlapping Same-Job IntervalsYou are given all job execution intervals for a single day. Each interval is half-open, meaning [start_time, end_time), and represents a time range during which a job must be running. A server can run at most one job at a time. However, if multiple intervals have the same job_id, they are considered parts or retries of the same job, so overlapping intervals with the same job_id consume only one server total during the overlap. Return the minimum number of servers required to run all intervals for the day.
Constraints
0 <= len(intervals) <= 2000000 <= job_id <= 10^90 <= start_time < end_time <= 1440Intervals are half-open: an interval ending at time t does not overlap an interval starting at time t.Multiple intervals may have the same job_id.

EXAMPLES

```
Input: ([],)
Expected Output: 0
```

Explanation: There are no jobs, so no servers are needed.

```
Input: ([[5, 100, 200], [5, 150, 250], [5, 240, 300]],)
Expected Output: 1
```

Explanation: All intervals belong to the same job_id. Even though they overlap, they can share one server.

Hints

At any moment, one server is needed for each distinct active job_id, not for each active interval.Try merging all intervals belonging to the same job_id first, then run a sweep-line algorithm over the merged intervals.
Part 2: Total Servers with Dedicated Multi-Day JobsYou are given true job execution intervals that may span multiple days. Some jobs require dedicated servers. A job is dedicated if its job_id is listed in configured_dedicated, or if any one of its true intervals has duration at least min_dedicated_duration minutes. A dedicated job uses one reserved server for that job_id and does not share that server with other jobs, though multiple intervals of the same dedicated job_id may use the same dedicated server. For a given target_day, compute the total number of

FULL CODE | main question first, then every follow-up, tests at the end

```
# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ========================================================
```

```python
import heapq


# ======================================================
# MAIN QUESTION
# ======================================================
# ---- main question: helper functions ----
def merge_by_job(intervals):
    # intervals of the SAME job may overlap but they run on one server, so
    # merge them first: (job, start, end) -> disjoint blocks (start, end, job)
    by_job = {}
    for job, start, end in intervals:
        if job not in by_job:
            by_job[job] = []
        by_job[job].append((start, end))
    blocks = []
    for job in by_job:
        current = None
        for start, end in sorted(by_job[job]):
            if current is not None and start <= current[1]:
                if end > current[1]:
                    current[1] = end
            else:
                if current is not None:
                    blocks.append((current[0], current[1], job))
                current = [start, end]
        blocks.append((current[0], current[1], job))
    return blocks


# ---- main question: main function ----
# Daily jobs (job id, start, end), half-open [start, end). How many servers
# are needed if a job's own overlapping runs share one server?
# Idea: group the intervals by job and merge each job's overlapping or touching
# intervals; then sweep the start and end events of all the merged blocks: the
# answer is the largest number of blocks running at the same moment (at equal
# times an end comes first). `dedicated` jobs (follow-up 1) get a server of
# their own and are left out of the sweep.
```

```python
def servers_needed(intervals, dedicated=()):
    dedicated = set(dedicated)
    shared = []
    used_dedicated = set()
    for interval in intervals:
        if interval[0] in dedicated:
            used_dedicated.add(interval[0])
        else:
            shared.append(interval)
    events = []
    for start, end, job in merge_by_job(shared):
        events.append((start, 1))
        events.append((end, -1))
    running = 0
    best = 0
    for time, delta in sorted(events):
        running += delta
        if running > best:
            best = running
    return best + len(used_dedicated)


# ---- main question: tests ----
JOBS = [("a", 0, 10), ("a", 5, 12), ("b", 3, 8), ("c", 10, 15)]


def test_main_question():
```

```
    assert servers_needed(JOBS) == 2                          # a is one block [0, 12); b overlaps it; c star
ts at 10
    assert servers_needed([("a", 0, 5), ("b", 5, 9)]) == 1    # touching half-open intervals share a server
    assert servers_needed([]) == 0


# =====================================================
# FOLLOW-UP 1: Some rare jobs run for several days without restart and get a dedicated server
# =====================================================
# Answer: take those jobs out of the daily intervals and add one server per
# such job: servers = (number of dedicated jobs) + (the largest overlap of the
# rest). That is the `dedicated` option.
# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert servers_needed(JOBS, dedicated=["b"]) == 3
    assert servers_needed(JOBS, dedicated=["a", "b"]) == 3


# =====================================================
```

```
# FOLLOW-UP 2: Jobs that cross midnight
# =====================================================
# ---- follow-up 2: function ----
# Answer: cut a job that wraps past midnight into two pieces: the part
# before midnight and the part after it (each day's schedule is then
# truncated at the end of the day, as the statement says).
def split_over_midnight(start, end, day=24 * 60):
    if start < end:
        return [(start, end)]
    return [(start, day), (0, end)]


# ---- follow-up 2: tests ----
def test_follow_up_2():
    assert split_over_midnight(1380, 60) == [(1380, 1440), (0, 60)]
    assert split_over_midnight(60, 120) == [(60, 120)]


# =====================================================
# FOLLOW-UP 3: Return which server each job runs on
# =====================================================
# ---- follow-up 3: function ----
# Answer: sort the merged blocks by start and keep a min-heap of (end time,
# server): reuse the server that frees up first if it is free, otherwise open a
# new one. The number of servers opened equals the largest overlap.
def assign_servers(intervals):
    heap = []
    plan = []
    opened = 0
    for start, end, job in sorted(merge_by_job(intervals)):
        if heap and heap[0][0] <= start:
            freed_at, server = heapq.heappop(heap)
        else:
            server = opened
            opened += 1
        plan.append((job, server))
        heapq.heappush(heap, (end, server))
    return plan, opened


# ---- follow-up 3: tests ----
def test_follow_up_3():
    plan, count = assign_servers(JOBS)
    assert count == 2 and len(plan) == 3
```

```
# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

14 Gate Entry / Exit Time Resolution with Tie-Break Rules — MEDIUM

Template fit: the same two-queue simulation with the tie-break rules from the statement

QUESTION

A gate lets one person through per second. events[i] = (time, action) for person i, where action is 'enter' or 'exit'. When several people want the gate in the same second: if the gate was not used in the previous second, people who exit go first; if it was used, the direction used in the previous second goes first again. Within one direction, people go in order of arrival (person id). Return the second at which each person passes.

Function signature

def gate_pass_times(events: list) -> list

EXAMPLES

Example 1	
Input	`[(0, 'enter'), (0, 'exit'), (1, 'enter'), (5, 'enter')]`
Output	`[1, 0, 2, 5]`
Why	At second 0 the gate was idle, so the exit (person 1) goes first. Person 0 follows at 1, person 2 (arrived at 1) at 2, and person 3 arrives alone at 5.

Example 2	
Input	`[(0, 'enter'), (1, 'enter'), (1, 'exit')]`
Output	`[0, 1, 2]`
Why	The gate was used for entering in the previous second, so the waiting enter keeps priority over the exit.

FULL CODE | main question first, then every follow-up, tests at the end

```
# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

from collections import deque


# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: main function ----
# People arrive at a gate wanting to enter or to exit (events[i] = (arrival
# time, "enter" or "exit")). The gate lets ONE person through per second.
```

```python
# Rules when several wait: if the gate was used the second before, the same
# direction keeps going; if it was idle, one direction wins (exit, by default).
# Within a direction, people go in order of arrival, then by number.
# Return the second each person goes through.
# Idea: an event-driven simulation with one queue per direction. Sort the
# people by (arrival time, number). When nobody waits, jump the clock straight
# to the next arrival (so huge timestamps cost nothing); if that leaves at
# least one idle second, forget the previous direction. Then everyone who has
# arrived joins their queue, and one person goes through.
# (`idle_first` is follow-up 4: who wins when the gate was idle.)
def gate_pass_times(events, idle_first="exit"):
    n = len(events)
    arrivals = []
    for i in range(n):
        arrivals.append((events[i][0], i))
    arrivals.sort()
    lines = {"enter": deque(), "exit": deque()}
    result = [0] * n
    t = 0
    next_person = 0
    previous = None                                  # the direction used in the second before
    last_used = -10                                  # the last second the gate was used
    while next_person < n or lines["enter"] or lines["exit"]:
        if not lines["enter"] and not lines["exit"]:
            if arrivals[next_person][0] > t:
                t = arrivals[next_person][0]         # nobody waits: jump to the next arrival
            if t > last_used + 1:
                previous = None                      # an idle second forgets the last direction
        while next_person < n and arrivals[next_person][0] <= t:
```

```python
            person = arrivals[next_person][1]
            lines[events[person][1]].append(person)   # everyone who has arrived joins their line
            next_person += 1
        if lines["enter"] and lines["exit"]:
            if previous is not None:
                want = previous                      # keep the direction used last second
            else:
                want = idle_first                    # idle gate: the tie-break rule decides
        elif lines["enter"]:
            want = "enter"
        else:
            want = "exit"
        person = lines[want].popleft()
        result[person] = t
        previous = want
        last_used = t
        t += 1
    return result


# ---- main question: tests ----
def test_main_question():
    both = [(0, "enter"), (0, "exit")]
    assert gate_pass_times(both) == [1, 0]                      # one enter and one exit together: exit firs
t
    assert gate_pass_times(both, "enter") == [0, 1]             # with "enter" winning the idle tie it swaps
    assert gate_pass_times([(0, "enter"), (0, "enter"), (1, "exit")]) == [0, 1, 2]    # the same direction co
ntinues
    assert gate_pass_times([(0, "enter"), (10, "enter"), (10, "exit")]) == [0, 11, 10]  # a long idle gap res
ets the state
    assert gate_pass_times([(10 ** 9, "enter"), (10 ** 9, "exit")]) == [10 ** 9 + 1, 10 ** 9]
    assert gate_pass_times([]) == []


# ================================================
# FOLLOW-UP 1: Why does the optimization need care across an empty gap?
```

```
# ============================================================
# Answer: jumping the clock over an empty gap is only safe if the "previous
# direction" is forgotten when at least one idle second passes; otherwise a
# person arriving after the gap would be treated as if the gate had just been
# used in the same direction.


# ============================================================
# FOLLOW-UP 2: What is the complexity, and what if events come online?
# ============================================================
# Answer: O(n log n) for the sort and O(n) for the loop. Online, keep the two
# queues and the clock: handle a person when the clock reaches their arrival.
```

```
# ============================================================
# FOLLOW-UP 3: Timestamps go up to 10^9
# ============================================================
# Answer: nothing changes: the clock jumps over empty stretches, so the work is
# one step per person, not per second.


# ============================================================
# FOLLOW-UP 4: The tie-break when the gate was idle differs in this version of the question
# ============================================================
# Answer: make the idle winner a parameter: only the line that picks the
# direction changes. With "exit" it is the standard rule; with "enter" an
# enter and an exit arriving together swap places (see the second test above).


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

15 Meeting Rooms III Variant — MEDIUM

Template fit: split meetings that cross midnight into two pieces first, then the same sweep

QUESTION

Given an array of meeting time intervals where each interval is [start, end], determine the minimum number of conference rooms required to hold all meetings without overlap. Variant: times are cyclic within a 24-hour period (meetings can wrap around midnight).

EXAMPLES

Example 1	
Input	`intervals = [[0, 30], [5, 10], [15, 20]]`
Output	`2`
Why	`Meeting [0,30] overlaps with [5,10] and [15,20], so we need 2 rooms.`

QUESTION

Example 2	
Input	`intervals = [[7, 10], [2, 4]]`
Output	`1`
Why	`No overlap, 1 room is sufficient.`

Example 3	
Input	`intervals = [[22, 2], [23, 1], [0, 3]]`
Output	`3`
Why	`All three meetings overlap around midnight.`

CONSTRAINTS

- 1 <= intervals.length <= 10^4
- 0 <= start < end <= 24 * 60 (minutes in a day)
- Meetings may wrap around (e.g., [23:00, 01:00])

FULL CODE | main question first, then every follow-up, tests at the end

```python
# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ========================================================

import heapq
import random

DAY = 24


# ========================================================
# MAIN QUESTION
# ========================================================
# ---- main question: helper functions ----
def pieces(meetings, day=DAY):
    # a meeting whose end is before its start wraps past midnight: split it
    # into the part before midnight and the part after it
    parts = []
    for start, end in meetings:
        if end < start:
            parts.append((start, day))
            parts.append((0, end))
        else:
            parts.append((start, end))
    return parts


# ---- main question: main function ----
# How many rooms are needed? Meetings are half-open [start, end), and a
# meeting may wrap past midnight (end < start).
# Idea: turn every meeting into one or two ordinary intervals, make a +1
# event at every start and a -1 event at every end, sort the events (at the
# same time an END comes first: a meeting ending at 5 and one starting at 5
# can share a room) and take the biggest running count.
def min_rooms_sweep(meetings, day=DAY):
    events = []
    for start, end in pieces(meetings, day):
        events.append((start, 1))
        events.append((end, -1))
    events.sort()                                   # (-1) sorts before (+1) at equal times
```

```
    running = 0
    best = 0
```

```
    for time, step in events:
        running += step
        if running > best:
            best = running
    return best


# ---- main question: tests ----
def test_main_question():
    assert min_rooms_sweep([[0, 30], [5, 10], [15, 20]]) == 2
    assert min_rooms_sweep([[7, 10], [2, 4]]) == 1
    assert min_rooms_sweep([[22, 2], [23, 1], [0, 3]]) == 3
    assert min_rooms_sweep([]) == 0
    assert min_rooms_sweep([[1, 5], [5, 9]]) == 1            # one ends exactly when the next starts: same ro
om


# =========================================================
# FOLLOW-UP 1: A wrapped meeting must stay in a single room
# =========================================================
# ---- follow-up 1: helper functions ----
def hours(meeting, day=DAY):
    # the hours a meeting occupies on the 24-hour circle
    start, end = meeting
    length = (end - start) % day
    if length == 0:
        length = day
    occupied = set()
    for k in range(length):
        occupied.add((start + k) % day)
    return occupied


# ---- follow-up 1: function ----
# Answer: this is circular-arc colouring, NP-hard in general, so use the
# practical method: cut the circle at the hour where the FEWEST meetings are
# running; the meetings that cross the cut get their own rooms (blocked
# before their start and after the cut); place the rest greedily, each in the
# room that has been free the longest and still fits.
def assign_single_rooms(meetings, day=DAY):
    busy = []
    for hour in range(day):
        count = 0
        for meeting in meetings:
            if hour in hours(meeting, day):
                count += 1
```

```
        busy.append(count)
    cut = 0
    for hour in range(day):
        if busy[hour] < busy[cut]:
            cut = hour
    line = []                                          # rotate so the cut is time 0: intervals on a line
    for start, end in meetings:
        new_start = (start - cut) % day
        length = (end - start) % day
        if length == 0:
            length = day
        line.append((new_start, new_start + length))
    free_at = []                                       # per room: when it is free
    deadline = []                                      # per room: it must be empty again by then
    room_of = {}
    for i in range(len(line)):
```

```
        start, end = line[i]
        if end > day:                                   # crosses the cut: busy until end - day, and again fro
m start on
            room_of[i] = len(free_at)
            free_at.append(end - day)
            deadline.append(start)
    others = []
    for i in range(len(line)):
        if i not in room_of:
            others.append((line[i], i))
    others.sort()
    for (start, end), i in others:
        best_room = None
        for room in range(len(free_at)):
            if free_at[room] <= start and end <= deadline[room]:
                if best_room is None or free_at[room] > free_at[best_room]:
                    best_room = room                    # the room that has been free the longest
        if best_room is None:
            best_room = len(free_at)
            free_at.append(0)
            deadline.append(day)
        room_of[i] = best_room
        free_at[best_room] = end
    rooms = []
    for i in range(len(meetings)):
        rooms.append(room_of[i])
    return rooms


# ---- follow-up 1: tests ----
```

```
def valid_assignment(meetings, rooms, day=DAY):
    # no room may hold two meetings in the same hour
    used = set()
    for i in range(len(meetings)):
        for hour in hours(meetings[i], day):
            if (rooms[i], hour) in used:
                return False
            used.add((rooms[i], hour))
    return True


def test_follow_up_1():
    two = [(22, 2), (1, 5)]
    assert valid_assignment(two, assign_single_rooms(two))
    random.seed(22)
    for round_number in range(400):
        meetings = []
        for i in range(random.randint(0, 7)):
            start, end = random.sample(range(DAY), 2)
            meetings.append((start, end))
        assert valid_assignment(meetings, assign_single_rooms(meetings))


# =========================================================
# FOLLOW-UP 2: Return which room each meeting uses
# =========================================================


# =========================================================
# FOLLOW-UP 4: Return which room each meeting uses
# =========================================================
# ---- follow-up 2: function ----
# Answer: sort by start and keep a min-heap of (end time, room): reuse the
# room that frees up first if it is free, otherwise open a new room. (For
# wrapped meetings place them first so their rooms are fixed.)
```

```
def assign_rooms(meetings):
    order = []
    for i in range(len(meetings)):
        order.append((meetings[i][0], i))
    order.sort()
    heap = []
    room_of = {}
    rooms_opened = 0
    for start, i in order:
        end = meetings[i][1]
        if heap and heap[0][0] <= start:
            freed_at, room = heapq.heappop(heap)
        else:
            room = rooms_opened
            rooms_opened += 1
        room_of[i] = room
        heapq.heappush(heap, (end, room))
    rooms = []
    for i in range(len(meetings)):
        rooms.append(room_of[i])
    return rooms


# ---- follow-up 2: tests ----
def test_follow_up_2():
    flat = [[0, 30], [5, 10], [15, 20]]
    rooms = assign_rooms(flat)
    assert rooms == [0, 1, 1]
    assert len(set(rooms)) == min_rooms_sweep(flat)
    assert valid_assignment(flat, rooms)


# ========================================================
# FOLLOW-UP 3: Days can repeat (a wrapped meeting recurs every day)
# ========================================================
# Answer: the circle repeats every day, so the sweep over one day (with
# wrapped meetings split at midnight) already counts the overlap on every day.


# ========================================================
# RUN EVERYTHING
# ========================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

16 Count Overlapping Rectangle Updates on a Grid — HARD

Template fit: the same idea in two dimensions: a difference array (+1, -1, -1, +1 at the corners) and two running-sum passes

QUESTION

Count Overlapping Rectangle Updates on a Grid Implement

`range_add_counts(n, rectangles)`. Start with an **`n x n`** matrix of zeroes. Each rectangle is represented as **`[row1, col1, row2, col2]`** with inclusive, zero-based coordinates. For every rectangle, add one to each matrix cell inside that rectangle. Return the final matrix, where each cell therefore contains the number of input rectangles covering it.

QUESTION

Example

For **`n = 3`** and **`rectangles = [[0, 0, 1, 1], [1, 1, 2, 2]]`**, return:

```
[ [1, 1, 0], [1, 2, 1], [0, 1, 1] ]
```

Constraints

- **`1 <= n <= 500`**
- **`0 <= len(rectangles) <= 100_000`**
- **`0 <= row1 <= row2 < n`**
- **`0 <= col1 <= col2 < n`**

EXAMPLES

Example 1	
Input	`3, [(0, 0, 1, 1), (1, 1, 2, 2)]`
Output	`[[1, 1, 0], [1, 2, 1], [0, 1, 1]]`
Why	The cell (1, 1) is covered by both rectangles, so it is counted twice.

Example 2	
Input	`2, []`
Output	`[[0, 0], [0, 0]]`
Why	No rectangles: all zeros.

FULL CODE | main question first, then every follow-up, tests at the end

```python
# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ========================================================

# ========================================================
# MAIN QUESTION
# ========================================================
# ---- main question: main function ----
# An n x n grid; each rectangle [r1, c1, r2, c2] (corners inclusive) adds 1
# to every cell inside. Return the final count of every cell.
# Idea: a 2D DIFFERENCE array. For each rectangle mark only its 4 corners:
# +1 at (r1, c1), -1 at (r1, c2 + 1), -1 at (r2 + 1, c1), +1 at
# (r2 + 1, c2 + 1). Then two prefix-sum passes (along every row, then down
# every column) spread the marks over the cells. Cost O(R + n^2) for R
# rectangles instead of O(R * n^2).
def range_add_counts(n, rects):
    d = []
    for i in range(n + 1):
        d.append([0] * (n + 1))
    for r1, c1, r2, c2 in rects:
        d[r1][c1] += 1
        d[r1][c2 + 1] -= 1
        d[r2 + 1][c1] -= 1
        d[r2 + 1][c2 + 1] += 1
    for r in range(n + 1):                          # prefix sums along every row ...
        for c in range(1, n + 1):
            d[r][c] += d[r][c - 1]
    for c in range(n + 1):                          # ... then down every column
        for r in range(1, n + 1):
            d[r][c] += d[r - 1][c]
    counts = []
```

```
    for r in range(n):
        counts.append(d[r][:n])
    return counts


# ---- main question: tests ----
RECTS = [[0, 0, 1, 1], [1, 1, 2, 2]]


def test_main_question():
    assert range_add_counts(3, RECTS) == [[1, 1, 0], [1, 2, 1], [0, 1, 1]]
```

```
# =========================================================
# FOLLOW-UP 1: How many cells are covered by at least k rectangles?
# =========================================================
# ---- follow-up 1: function ----
# Answer: build the count table with the difference array and count the
# cells whose value is at least k. O(R + n^2).
def cells_at_least(n, rects, k):
    total = 0
    for row in range_add_counts(n, rects):
        for value in row:
            if value >= k:
                total += 1
    return total


# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert cells_at_least(3, RECTS, 2) == 1
    assert cells_at_least(3, RECTS, 1) == 7


# =========================================================
# FOLLOW-UP 2: The grid is huge (10^9 x 10^9) but there are few rectangles
# =========================================================
# ---- follow-up 2: function ----
# Answer: coordinate compression. Only the corner rows and columns matter, so
# keep just those (at most 2R of each) and run the same difference array on
# that small grid; each compressed cell stands for a whole BLOCK of real cells,
# so add its area (block height * block width) when its count is at least k.
def cells_at_least_compressed(rects, k):
    row_set = set()
    column_set = set()
    for r1, c1, r2, c2 in rects:
        row_set.add(r1)
        row_set.add(r2 + 1)
        column_set.add(c1)
        column_set.add(c2 + 1)
    rows = sorted(row_set)
    cols = sorted(column_set)
    row_index = {}
    for i in range(len(rows)):
        row_index[rows[i]] = i
    col_index = {}
```

```
    for j in range(len(cols)):
        col_index[cols[j]] = j
    d = []
    for i in range(len(rows) + 1):
        d.append([0] * (len(cols) + 1))
    for r1, c1, r2, c2 in rects:
        a = row_index[r1]
        b = col_index[c1]
```

```
        e = row_index[r2 + 1]
        f = col_index[c2 + 1]
        d[a][b] += 1
        d[a][f] -= 1
        d[e][b] -= 1
        d[e][f] += 1
    total = 0
    column_sums = [0] * (len(cols) + 1)             # the prefix sums down each column
    for i in range(len(rows) - 1):
        running = 0                                 # the prefix sum along this row
        for j in range(len(cols) - 1):
            column_sums[j] += d[i][j]
            running += column_sums[j]
            if running >= k:
                total += (rows[i + 1] - rows[i]) * (cols[j + 1] - cols[j])
    return total


# ---- follow-up 2: tests ----
def test_follow_up_2():
    assert cells_at_least_compressed(RECTS, 2) == 1          # the same answer as the plain grid
    assert cells_at_least_compressed(RECTS, 1) == 7
    assert cells_at_least_compressed([[0, 0, 10 ** 9, 10 ** 9], [5, 5, 10, 10]], 2) == 36


# ======================================================
# FOLLOW-UP 3: Rectangles are added and the count of a cell is queried online
# ======================================================
# ---- follow-up 3: function ----
# Answer: a 2D Fenwick tree (the difference-array idea as a binary indexed
# tree): add a rectangle by changing its 4 corners and read a cell as a prefix
# sum, both in O(log^2 n).
class Fenwick2D:
    def __init__(self, n):
        self.n = n
        self.tree = []
        for i in range(n + 2):
            self.tree.append([0] * (n + 2))

    def add_corner(self, r, c, value):
        i = r + 1
        while i <= self.n + 1:
            j = c + 1
            while j <= self.n + 1:
                self.tree[i][j] += value
                j += j & -j                          # j & -j is the lowest set bit of j
            i += i & -i

    def add_rect(self, r1, c1, r2, c2):
        self.add_corner(r1, c1, 1)
        self.add_corner(r1, c2 + 1, -1)
        self.add_corner(r2 + 1, c1, -1)
        self.add_corner(r2 + 1, c2 + 1, 1)

    def get(self, r, c):
        total = 0
        i = r + 1
        while i > 0:
            j = c + 1
            while j > 0:
                total += self.tree[i][j]
                j -= j & -j
            i -= i & -i
        return total
```

```
# ---- follow-up 3: tests ----
def test_follow_up_3():
    grid = Fenwick2D(3)
    for rect in RECTS:
        grid.add_rect(rect[0], rect[1], rect[2], rect[3])
    expected = range_add_counts(3, RECTS)
    for r in range(3):
        for c in range(3):
            assert grid.get(r, c) == expected[r][c]


# ========================================================
# RUN EVERYTHING
# ========================================================
def run_all_tests():
    test_main_question()
```

```
    test_follow_up_1()
    test_follow_up_2()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

17 Compute city skyline outline — HARD

Template fit: events at each building's left and right edge; a max-heap of active heights; record a point when the top height changes

QUESTION

You are given a list of rectangular buildings in a 2D city skyline. Each building is represented by three integers **`[L, R, H]`**:

- **`L`**: the x-coordinate of the left edge of the building,
- **`R`**: the x-coordinate of the right edge of the building (**`L < R`**),
- **`H`**: the height of the building. All buildings stand on a flat horizontal line (y = 0) and extend vertically up to height **`H`**. Buildings may overlap in the x-dimension. Your task is to compute the *skyline* formed by these buildings, as seen from a distance. The skyline should be represented as a list of **key points `[x, height]`** that describe the outer contour of the union of all buildings. A **key point** is a point where the height of the skyline changes. The output list must satisfy all of the following:

1. The list is sorted by increasing **`x`**.
2. No two consecutive points have the same height (i.e., the height must change at each key point).
3. The first point has a height greater than 0.
4. The last point has height 0, marking the end of the skyline.

Input

- An integer **`n`** — the number of buildings.
- A list of **`n`** buildings, where each building is given as a triple **`[L, R, H]`** of integers. You may assume:
- **`1 <= n <= 10^4`**
- **`0 <= L < R <= 10^9`**
- **`1 <= H <= 10^9`**

Output

QUESTION

Return a list of key points representing the skyline. Each key point is a pair **`[x, height]`** (or a 2-element array/tuple) of integers.

Example

Input:

- **`n = 3`** - buildings = **`[[2, 9, 10], [3, 7, 15], [5, 12, 12]]`** One valid output:
- **`[[2, 10], [3, 15], [7, 12], [12, 0]]`** Explanation (informal):
- From x = 2 to 3, the highest building has height 10.
- From x = 3 to 7, the highest building has height 15.
- From x = 7 to 12, the remaining highest building has height 12.
- After x = 12, there are no buildings, so the height drops to 0. Design an algorithm to compute the skyline key points for the given buildings.

EXAMPLES

Example 1	
Input	**`[[2, 9, 10], [3, 7, 15], [5, 12, 12], [15, 20, 10], [19, 24, 8]]`**
Output	**`[[2, 10], [3, 15], [7, 12], [12, 0], [15, 10], [20, 8], [24, 0]]`**
Why	**Buildings are [left, right, height]; the outline lists the points where the height changes.**

Example 2	
Input	**`[[0, 2, 3]]`**
Output	**`[[0, 3], [2, 0]]`**
Why	**One building: a point at its left edge and one at its right edge, where the height returns to 0.**

FULL CODE | main question first, then every follow-up, tests at the end

```python
# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

import heapq


# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: main function ----
# Buildings are [left, right, height]. Return the skyline: the "key points"
# [x, height] where the height of the outline changes.
# Idea: a sweep line from left to right with a max-heap of the buildings
# that are active (entries (-height, right edge), push negatives to get a
# max-heap). Start events carry -height so that, at the same x, taller starts
# come first and starts come before ends; end events carry 0. Buildings that
# have ended are removed LAZILY: only when they reach the top of the heap. After
# each event, the top of the heap is the current height; write a key point
# only when it differs from the last one. The ground (height 0) is always in the heap.
def skyline(buildings):
    events = []
    for left, right, height in buildings:
        events.append((left, -height, right))        # a start
        events.append((right, 0, 0))                 # an end
```

```
    events.sort()
    points = []
    heap = [(0, float("inf"))]                      # the ground, which never ends
    for x, minus_height, right in events:
        while heap[0][1] <= x:
            heapq.heappop(heap)                     # a building that ended at or before x
        if minus_height != 0:
            heapq.heappush(heap, (minus_height, right))
        top = -heap[0][0]
        if not points or points[len(points) - 1][1] != top:
            points.append([x, top])
    return points


# ---- main question: tests ----
BUILDINGS = [[2, 9, 10], [3, 7, 15], [5, 12, 12], [15, 20, 10], [19, 24, 8]]
OUTLINE = [[2, 10], [3, 15], [7, 12], [12, 0], [15, 10], [20, 8], [24, 0]]
```

```
def test_main_question():
    assert skyline(BUILDINGS) == OUTLINE
    assert skyline([[0, 2, 3], [2, 5, 3]]) == [[0, 3], [5, 0]]
    assert skyline([]) == []
    assert skyline([[1, 2, 1], [1, 2, 2], [1, 2, 3]]) == [[1, 3], [2, 0]]


# ========================================================
# FOLLOW-UP 1: Divide and conquer instead of a heap
# ========================================================
# ---- follow-up 1: helper functions ----
def merge_skylines(a, b):
    # sweep two skylines with the current height of each; the height of the
    # merged skyline is the bigger of the two
    merged = []
    i = 0
    j = 0
    height_a = 0
    height_b = 0
    while i < len(a) or j < len(b):
        take_a = i < len(a) and (j == len(b) or a[i][0] <= b[j][0])
        if take_a:
            x = a[i][0]
            height_a = a[i][1]
            i += 1
        else:
            x = b[j][0]
            height_b = b[j][1]
            j += 1
        if i < len(a) and a[i][0] == x:
            continue                                # both skylines change at x: wait for the second
        if j < len(b) and b[j][0] == x:
            continue
        top = height_a
        if height_b > top:
            top = height_b
        if not merged or merged[len(merged) - 1][1] != top:
            merged.append([x, top])
    return merged


# ---- follow-up 1: function ----
# Answer: build the skyline of each half of the buildings and merge the
```

```
# two by sweeping both lists with the current height of each. T(n) = 2 T(n/2)
# + O(n) = O(n log n), the same as the heap.
```

```python
def skyline_dc(buildings):
    if not buildings:
        return []
    if len(buildings) == 1:
        left, right, height = buildings[0]
        return [[left, height], [right, 0]]
    middle = len(buildings) // 2
    return merge_skylines(skyline_dc(buildings[:middle]), skyline_dc(buildings[middle:]))


# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert skyline_dc(BUILDINGS) == OUTLINE
    assert skyline_dc([]) == []


# ========================================================
# FOLLOW-UP 2: Coordinates up to 10^9 or buildings of a very large count
# ========================================================
# Answer: the sweep only touches the building edges, never the coordinates
# between them, so 10^9 coordinates cost nothing. For a huge number of buildings,
# split them across machines (divide and conquer) and merge the skylines.


# ========================================================
# FOLLOW-UP 3: Output the area under the skyline
# ========================================================
# ---- follow-up 3: function ----
# Answer: add up height * width over every pair of neighbouring key
# points: one extra pass over the output.
def area_under(points):
    area = 0
    for i in range(len(points) - 1):
        x, height = points[i]
        area += height * (points[i + 1][0] - x)
    return area


# ---- follow-up 3: tests ----
def test_follow_up_3():
    assert area_under(OUTLINE) == 212               # 10*1 + 15*4 + 12*5 + 10*5 + 8*4
```

```python
# ========================================================
# RUN EVERYTHING
# ========================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

18 Compute Turnstile Crossing Times

HARD

Template fit: sort people by arrival time; two queues (entering, exiting) with a rule for who goes first at each second

QUESTION

A turnstile lets one person through per second. events[i] = (arrival time, direction) for person i, with direction 'enter' or 'exit'. When people in both directions wait in the same second: if the turnstile was not used in the previous second, people exiting go first; otherwise the direction used in the previous second goes first again. People of the same direction go in order of arrival (person id). Return the second at which each person crosses.

Function signature

def turnstile_times(events: list) -> list

EXAMPLES

Example 1	
Input	`[(0, 'exit'), (0, 'enter'), (0, 'enter'), (3, 'exit')]`
Output	`[0, 1, 2, 3]`
Why	The idle turnstile lets the exit (person 0) go first at second 0, then the two entering people follow at 1 and 2; person 3 arrives alone at 3.

Example 2	
Input	`[(0, 'enter'), (0, 'exit'), (0, 'exit'), (0, 'enter')]`
Output	`[2, 0, 1, 3]`
Why	Idle turnstile: the exits (persons 1 and 2) go first at seconds 0 and 1, then the entering people (persons 0 and 3) at seconds 2 and 3.

FULL CODE | main question first, then every follow-up, tests at the end

```
# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ========================================================

from collections import deque


# ========================================================
# MAIN QUESTION
# ========================================================
# ---- main question: main function ----
# events[i] = (arrival time, "enter" or "exit"). One person passes the
# turnstile per second. If it was used in the second before, the same
# direction keeps going when both lines wait; if it was idle, `idle_first`
# wins (exit, by default). Return the second each person goes through.
# Idea: an event-driven simulation with two queues. Sort the arrivals once;
# when both queues are empty jump straight to the next arrival time, and if
# the jump leaves at least one idle second, reset the previous direction to
# "none". Then everyone who has arrived joins a line, and one person goes.
def turnstile_times(events, idle_first="exit"):
    n = len(events)
    arrivals = []
    for i in range(n):
        arrivals.append((events[i][0], i))
    arrivals.sort()                                    # by arrival time, then by number
    lines = {"enter": deque(), "exit": deque()}
    result = [0] * n
    t = 0
    next_person = 0
```

```
    previous = None
    last_used = -10
    while next_person < n or lines["enter"] or lines["exit"]:
        if not lines["enter"] and not lines["exit"]:
            if arrivals[next_person][0] > t:
                t = arrivals[next_person][0]           # nobody is waiting: jump to the next arrival
            if t > last_used + 1:
                previous = None                        # an idle second forgets the last direction
        while next_person < n and arrivals[next_person][0] <= t:
            person = arrivals[next_person][1]
            lines[events[person][1]].append(person)    # everyone who has arrived joins a line
            next_person += 1
        if lines["enter"] and lines["exit"]:
            if previous is not None:
                want = previous                        # both lines: keep the direction used last second
            else:
                want = idle_first                      # after an idle gap the tie-break decides
        elif lines["enter"]:
            want = "enter"
        else:
            want = "exit"
        person = lines[want].popleft()
        result[person] = t
        previous = want
        last_used = t
        t += 1
    return result


# ---- main question: tests ----
def test_main_question():
    both = [(0, "enter"), (0, "exit")]
    assert turnstile_times(both) == [1, 0]                      # idle tie: the exit goes first
    assert turnstile_times(both, "enter") == [0, 1]             # a version where enter wins the idle tie
    assert turnstile_times([(0, "enter"), (0, "enter"), (1, "exit")]) == [0, 1, 2]     # the direction in use
 keeps going
    assert turnstile_times([(0, "enter"), (10, "enter"), (10, "exit")]) == [0, 11, 10]  # an idle second rese
ts the state
    assert turnstile_times([(10 ** 9, "enter"), (10 ** 9, "exit")]) == [10 ** 9 + 1, 10 ** 9]
    assert turnstile_times([]) == []


# ======================================================
# FOLLOW-UP 1: The priority when the turnstile was idle is different in this version of the question
# ======================================================
# Answer: keep the same loop and make the idle-state winner a parameter
# (`idle_first`). Only the line that picks the direction changes, which is why
# you should write the state machine first.


# ======================================================
# FOLLOW-UP 2: Timestamps are up to 10^9
# ======================================================
# Answer: no problem: the clock jumps over empty stretches, one loop step per person.


# ======================================================
# FOLLOW-UP 3: Two turnstiles or a capacity of two people per second
# ======================================================
# Answer: let two people go through per second: take up to two from the
# queues in each step (same direction rule), or keep one "previous direction"
# per turnstile.
```

```python

# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

GROUP 3 OF 43 | 9 QUESTIONS, ONE CODE TEMPLATE | 3 E 4 M 2 H

Tree recursion, Tier 1: one function that visits a node, asks its children, and reports upward

19 Given two trees, determine if the pre-order traversals of two trees are same or not. EASY

Template fit: walk both trees in preorder with an explicit stack (push right, then left) and compare value by value; stop at the first difference

QUESTION

Given two binary trees, determine whether their pre-order traversals (node, then left subtree, then right subtree) produce the same sequence of values. The trees themselves may have different shapes.

Function signature

def same_preorder(a: TreeNode, b: TreeNode) -> bool

EXAMPLES

Example 1	
Input	`[1, 2, 3], [1, 2, 3]`
Output	`true`
Why	`Identical trees.`

Example 2	
Input	`[1, 2, null, 3], [1, null, 2, null, 3]`
Output	`true`
Why	`Different shapes (a chain going left versus a chain going right) but both read 1, 2, 3.`

Example 3	
Input	`[1, 2, 3], [1, 3, 2]`
Output	`false`
Why	`1, 2, 3 versus 1, 3, 2.`

FULL CODE | main question first, then every follow-up, tests at the end

```
# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ========================================================

# ========================================================
# MAIN QUESTION
# ========================================================
# ---- main question: helper functions ----
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right


def push_children(stack, node):
    # right goes in first so that the LEFT child comes out first (pre-order)
    if node.right:
```

```
        stack.append(node.right)
    if node.left:
        stack.append(node.left)


# ---- main question: main function ----
# Idea: walk both trees in pre-order at the same time with two stacks
# and compare the values one by one. The first mismatch stops the work
# and no list of values is built. If one walk ends before the other,
# the sequences differ.
def same_preorder(a, b):
    stack_a = []
    stack_b = []
    if a:
        stack_a.append(a)
    if b:
        stack_b.append(b)
    while stack_a and stack_b:
        x = stack_a.pop()
        y = stack_b.pop()
        if x.val != y.val:
            return False
        push_children(stack_a, x)
        push_children(stack_b, y)
```

```
    return not stack_a and not stack_b


# ---- main question: tests ----
def two_example_trees():
    t1 = TreeNode(1, TreeNode(2), TreeNode(3))
    t2 = TreeNode(1, TreeNode(2, None, TreeNode(3)))       # a different shape, same pre-order 1, 2, 3
    return t1, t2


def test_main_question():
    t1, t2 = two_example_trees()
    assert same_preorder(t1, t2)
    assert same_preorder(None, None)
    assert not same_preorder(t1, None)
    assert not same_preorder(t1, TreeNode(1, TreeNode(2)))


# =========================================================
# FOLLOW-UP 1: What if the interviewer meant the trees must be identical?
# =========================================================


# =========================================================
# FOLLOW-UP 3: They want identical trees, not just sequences
# =========================================================


# =========================================================
# FOLLOW-UP 4: They want identical trees (structure and values), not just the same pre-order values
# =========================================================
# ---- follow-up 1: functions ----
# Answer: compare structure as well as values: walk both trees together;
# a node must match in value, and both children must match; two missing
# nodes match, one missing node does not. Equivalently, write each tree
# out in pre-order with a marker ("#") for every missing child; that text
# is unique, so equal text means identical trees.
def same_tree(a, b):
    stack = [(a, b)]
    while stack:
        x, y = stack.pop()
```

```
        if x is None and y is None:
            continue
        if x is None or y is None or x.val != y.val:
            return False
```

```
        stack.append((x.left, y.left))
        stack.append((x.right, y.right))
    return True


def serialise(root):
    parts = []
    stack = [root]
    while stack:
        node = stack.pop()
        if node is None:
            parts.append("#")
            continue
        parts.append(str(node.val))
        stack.append(node.right)
        stack.append(node.left)
    return ",".join(parts)


# ---- follow-up 1: tests ----
def test_follow_up_1():
    t1, t2 = two_example_trees()
    assert not same_tree(t1, t2)
    assert serialise(t1) != serialise(t2)
    assert same_tree(t1, TreeNode(1, TreeNode(2), TreeNode(3)))
    assert same_tree(None, None)
    assert serialise(t1) == "1,2,#,#,3,#,#"


# =============================================================
# FOLLOW-UP 2: Do it with O(1) extra space
# =============================================================
# ---- follow-up 2: functions ----
MISSING = object()          # "this walk has ended"


# Answer: Morris traversal visits the nodes in pre-order using a
# temporary link from the rightmost node of the left subtree back to the
# node, so no stack is needed. It changes the tree while it runs (and puts
# it back at the end), so it is not safe for other readers at the same
# time: mention that trade-off. The generator must be run to the end,
# otherwise a temporary link is left in the tree.
def morris_preorder(root):
    node = root
    while node:
```

```
        if node.left is None:
            yield node.val
            node = node.right
            continue
        pred = node.left                          # the last node visited inside the left subtree
        while pred.right and pred.right is not node:
            pred = pred.right
        if pred.right is None:
            yield node.val
            pred.right = node                     # leave a link to come back
            node = node.left
        else:
            pred.right = None                     # second visit: remove the link
            node = node.right
```

```
def same_preorder_constant_space(a, b):
    one = morris_preorder(a)
    two = morris_preorder(b)
    same = True
    while True:
        x = next(one, MISSING)
        y = next(two, MISSING)
        if x is MISSING and y is MISSING:
            break
        if x != y:
            same = False                          # never stop early: that would leave links in the trees
    return same


# ---- follow-up 2: tests ----
def test_follow_up_2():
    t1, t2 = two_example_trees()
    assert list(morris_preorder(t2)) == [1, 2, 3]
    assert same_preorder_constant_space(t1, t2)
    assert not same_preorder_constant_space(t1, TreeNode(1, TreeNode(9), TreeNode(3)))
    assert t1.left.right is None and t1.right.left is None        # the trees are left exactly as they were


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
```

```
    test_follow_up_2()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

20 Calculate the height of a binary tree. EASY

Template fit: no update needed; return 1 + max(left, right) and read the value at the root

QUESTION

Given the root of a binary tree, return its height: the number of nodes on the longest path from the root down to a leaf (an empty tree has height 0).

Function signature

def height(root: TreeNode) -> int

EXAMPLES

Example 1	
Input	`[3, 9, 20, null, null, 15, 7]`
Output	`3`
Why	`Levels: [3], [9, 20], [15, 7]: height 3.`

Example 2	
Input	`[1, 2]`
Output	`2`
Why	`A root with one child.`

QUESTION

Example 3	
Input	`[]`
Output	`0`
Why	An empty tree.

FULL CODE | main question first, then every follow-up, tests at the end

```
# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

from collections import deque


# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right


# ---- main question: main function ----
# Idea: process the tree one level at a time with a queue and count the
# levels. No recursion, so a deep tree cannot overflow the stack; the
# memory used is the widest level. Height is counted in NODES: an empty
# tree is 0 and a single node is 1 (follow-up 1 explains edges).
def height(root):
    levels = 0
    queue = deque()
    if root:
        queue.append(root)
    while queue:
        levels += 1
        for i in range(len(queue)):              # exactly the nodes of this level
            node = queue.popleft()
            if node.left:
                queue.append(node.left)
            if node.right:
                queue.append(node.right)
    return levels


# ---- main question: tests ----
def example_tree():
    return TreeNode(3, TreeNode(9), TreeNode(20, TreeNode(15), TreeNode(7)))
```

```


def test_main_question():
    assert height(example_tree()) == 3
    assert height(None) == 0
    assert height(TreeNode(1)) == 1
    chain = TreeNode(1, TreeNode(2, TreeNode(3)))
    assert height(chain) == 3
```

```python
# ============================================================
# FOLLOW-UP 1: Nodes or edges?
# ============================================================
# ---- follow-up 1: function ----
# Answer: height in nodes is what height() returns; in edges subtract one
# (an empty tree is -1, a single node is 0). Say which one you use
# before you code.
def height_edges(root):
    return height(root) - 1


# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert height_edges(example_tree()) == 2
    assert height_edges(None) == -1
    assert height_edges(TreeNode(1)) == 0


# ============================================================
# FOLLOW-UP 2: Diameter (the longest path between any two nodes)
# ============================================================
# ---- follow-up 2: function ----
# Answer: the recursion returns the height of a subtree, and at every
# node the best path THROUGH it is left height + right height. Keep the
# biggest such sum: one pass, O(n). The diameter here is in edges.
def diameter(root):
    best = 0

    def depth(node):
        nonlocal best
        if node is None:
            return 0
        left = depth(node.left)
        right = depth(node.right)
        if left + right > best:
            best = left + right
        if left > right:
            return 1 + left
        return 1 + right

    depth(root)
    return best


# ---- follow-up 2: tests ----
def test_follow_up_2():
    assert diameter(example_tree()) == 3
    assert diameter(None) == 0
    assert diameter(TreeNode(1)) == 0


# ============================================================
# FOLLOW-UP 3: Is the tree balanced?
# ============================================================
# ---- follow-up 3: function ----
# Answer: at EVERY node the heights of the two sides differ by at most
# one. Return the height, or -1 as soon as a subtree is unbalanced, so
# the whole check is one O(n) pass.
def is_balanced(root):
    def depth(node):
        if node is None:
            return 0
        left = depth(node.left)
        if left == -1:
```

```
            return -1
        right = depth(node.right)
        if right == -1:
            return -1
        difference = left - right
        if difference < 0:
            difference = -difference
        if difference > 1:
            return -1
        if left > right:
            return 1 + left
        return 1 + right

    return depth(root) != -1
```

```
# ---- follow-up 3: tests ----
def test_follow_up_3():
    assert is_balanced(example_tree())
    assert is_balanced(None)
    chain = TreeNode(1, TreeNode(2, TreeNode(3)))
    assert not is_balanced(chain)


# =============================================================
# RUN EVERYTHING
# =============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

21 Find right-side view of binary tree — EASY

Template fit: pass depth down; the first node you reach at a new depth (visiting right before left) is the one you can see

QUESTION

You are given the root of a binary tree. From the **right side** of the tree, at each depth you can see exactly one node: the rightmost node at that depth. Write a function that returns a list of the values of the nodes visible when looking at the tree from the right side, ordered from **top to bottom**. ---

Function Signature

(one possible form)

```
rightSideView(root: TreeNode) -> List[int]
```

Where **`TreeNode`** is defined as:

```
class TreeNode { int val; TreeNode left; TreeNode right; }
```

Example

Consider this binary tree:

```
1 / \ 2 3 \ \ 5 4
```

The right-side view is:

QUESTION

```
[1, 3, 4]
```

Explain the algorithm you would use, including its time and space complexity. You may use either breadth-first search (BFS) or depth-first search (DFS).

EXAMPLES

Example 1	
Input	`[1, 2, 3, null, 5, null, 4]`
Output	`[1, 3, 4]`
Why	`Looking from the right you see 1, then 3 (hiding 2), then 4 (hiding 5).`

Example 2	
Input	`[1, 2]`
Output	`[1, 2]`
Why	`A left child is visible when there is no right child at that depth.`

Example 3	
Input	`[]`
Output	`[]`
Why	`An empty tree shows nothing.`

FULL CODE | main question first, then every follow-up, tests at the end

```python
# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

from collections import deque


# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right


# ---- main question: main function ----
# Idea: visit the RIGHT child before the left one and pass the depth
# down. The first node met at a depth that is not recorded yet is the
# rightmost node of that depth. Memory is O(height).
def right_side_view(root):
    view = []

    def visit(node, depth):
        if node is None:
            return
        if depth == len(view):
            view.append(node.val)                # first node seen at this depth
        visit(node.right, depth + 1)
        visit(node.left, depth + 1)
```

```
    visit(root, 0)
    return view


# ---- main question: tests ----
def example_tree():
    #     1
    #    / \
    #   2   3
    #    \   \
    #     5   4
    return TreeNode(1, TreeNode(2, None, TreeNode(5)), TreeNode(3, None, TreeNode(4)))


def test_main_question():
    assert right_side_view(example_tree()) == [1, 3, 4]
    assert right_side_view(None) == []
    chain = TreeNode(1, TreeNode(2, TreeNode(3)))            # a left-only tree is still seen from the right
    assert right_side_view(chain) == [1, 2, 3]


# ======================================================
# FOLLOW-UP 1: Left side view, or both
# ======================================================
# ---- follow-up 1: function ----
# Answer: for the left view swap the order of the two recursive calls.
# For both views at once, go level by level and take the first and the
# last node of each level.
def left_and_right_views(root):
    left_view = []
    right_view = []
    queue = deque()
    if root:
        queue.append(root)
    while queue:
        left_view.append(queue[0].val)
        right_view.append(queue[-1].val)
        for i in range(len(queue)):
            node = queue.popleft()
            if node.left:
                queue.append(node.left)
            if node.right:
                queue.append(node.right)
    return left_view, right_view


# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert left_and_right_views(example_tree()) == ([1, 2, 5], [1, 3, 4])
    assert left_and_right_views(None) == ([], [])


# ======================================================
# FOLLOW-UP 2: Avoid recursion
# ======================================================
# ---- follow-up 2: function ----
# Answer: go level by level with a queue and keep only the LAST node of
# every level (or use your own stack of (node, depth) and push the left
# child first so the right one is taken first).
def right_side_view_bfs(root):
    view = []
    queue = deque()
    if root:
```

```
        queue.append(root)
    while queue:
        view.append(queue[-1].val)
        for i in range(len(queue)):
            node = queue.popleft()
            if node.left:
                queue.append(node.left)
            if node.right:
                queue.append(node.right)
    return view


# ---- follow-up 2: tests ----
def test_follow_up_2():
    assert right_side_view_bfs(example_tree()) == [1, 3, 4]
    assert right_side_view_bfs(None) == []


# ========================================================
# FOLLOW-UP 3: Top view or bottom view
# ========================================================
# ---- follow-up 3: function ----
# Answer: give every node a horizontal distance (left child: -1, right
# child: +1 compared with its parent) and go level by level. The FIRST
# node seen at a distance is the top view there, the LAST one is the
# bottom view.
def top_and_bottom_view(root):
    top = {}
    bottom = {}
    queue = deque()
    if root:
        queue.append((root, 0))
    while queue:
        node, distance = queue.popleft()
        if distance not in top:
            top[distance] = node.val
```

```
        bottom[distance] = node.val
        if node.left:
            queue.append((node.left, distance - 1))
        if node.right:
            queue.append((node.right, distance + 1))
    top_view = []
    bottom_view = []
    for distance in sorted(top):
        top_view.append(top[distance])
        bottom_view.append(bottom[distance])
    return top_view, bottom_view


# ---- follow-up 3: tests ----
def test_follow_up_3():
    top, bottom = top_and_bottom_view(example_tree())
    assert top == [2, 1, 3, 4] and bottom == [2, 5, 3, 4]


# ========================================================
# RUN EVERYTHING
# ========================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    test_follow_up_3()
    print("all tests passed")
```

```
if __name__ == "__main__":
    run_all_tests()
```

22 Expression Tree Single-Leaf Mutation — MEDIUM

Template fit: post-order evaluation (children first, then apply the operator); to test one leaf flip, re-evaluate with that leaf changed

QUESTION

A boolean expression tree has leaves holding True or False and internal nodes AND, OR, XOR (two children) or NOT (one child). For each leaf in turn, imagine flipping only that leaf and report the value of the whole tree after the flip (every flip is independent of the others). Return the leaves and the list of answers, in leaf order.

Function signature

def flip_answers(root) -> (list, list)

(the leaves, the root value

after flipping each one)

EXAMPLES

Example 1	

Input	`t = Node('AND', Node(val=True), Node('OR', Node(val=False), Node(val=True)))`
	`evaluate(t)`
	`[leaf.val for leaf in flip_answers(t)[0]]`
	`flip_answers(t)[1]`

Output	`evaluate(t) -> true`
	`[leaf.val for leaf in flip_answers(t)[0]] -> [true, false, true]`
	`flip_answers(t)[1] -> [false, true, false]`

Why	`The tree is True AND (False OR True) = True. Flipping the first leaf gives False, flipping the False leaf changes nothing (the OR is still True), flipping the last leaf makes the OR False.`

FULL CODE | main question first, then every follow-up, tests at the end

```
# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
class Node:
    def __init__(self, op=None, left=None, right=None, val=None):
        # op is "AND", "OR", "XOR", "NOT", or None for a leaf (which holds val)
        self.op = op
        self.val = val
        self.left = left
        self.right = right


def combine(op, a, b=None):
    # the value of one gate from its input values (NOT has only one input)
    if op == "AND":
        return a and b
    if op == "OR":
        return a or b
```

```
    if op == "XOR":
        return a != b
    return not a


def right_value(value, node):
    # the value of the right child, or None when there is none (NOT)
    if node.right is None:
        return None
    return value[node.right]


def evaluate(node):
    # plain recursive evaluation (used as the checker in the tests)
    if node.op is None:
        return node.val
    a = evaluate(node.left)
    b = None
    if node.right is not None:
```

```
        b = evaluate(node.right)
    return combine(node.op, a, b)


def build_cache(root):
    # one pass that stores, for every node, its parent and its value,
    # and a list of the leaves in left-to-right order
    parent = {root: None}
    value = {}
    leaves = []
    order = []
    stack = [root]
    while stack:
        node = stack.pop()
        order.append(node)
        for child in (node.right, node.left):
            if child is not None:
                parent[child] = node
                stack.append(child)
    for node in reversed(order):                # children come before their parents
        if node.op is None:
            value[node] = node.val
        else:
            value[node] = combine(node.op, value[node.left], right_value(value, node))
    for node in order:                          # `order` is root, left subtree, right subtree
        if node.op is None:
            leaves.append(node)
    return parent, value, leaves


# ---- main question: main function ----
# Idea: store every node's value and parent once. To flip one leaf, walk
# UP from it recomputing only the nodes whose value really changes, and
# stop as soon as a value stays the same. Read the root's value, then put
# every changed value back for the next leaf.
def flip_answers(root):
    parent, value, leaves = build_cache(root)
    answers = []
    for leaf in leaves:
        undo = []
        node = leaf
        new = not value[leaf]
        while node is not None and value[node] != new:
            undo.append((node, value[node]))
            value[node] = new
```

```
            node = parent[node]
            if node is not None:
                new = combine(node.op, value[node.left], right_value(value, node))
        answers.append(value[root])
        for node, old in reversed(undo):          # restore for the next leaf
            value[node] = old
    return leaves, answers


# ---- main question: tests ----
def example_expression():
    # (True AND True) XOR (NOT False)
    return Node("XOR", Node("AND", Node(val=True), Node(val=True)), Node("NOT", Node(val=False)))


def test_main_question():
    tree = example_expression()
    leaves, answers = flip_answers(tree)
    assert len(leaves) == 3
    for i in range(len(leaves)):                  # every cached answer equals a full re-evaluation
        leaves[i].val = not leaves[i].val
        assert evaluate(tree) == answers[i]
        leaves[i].val = not leaves[i].val
    assert evaluate(tree) is False                # the tree is unchanged afterwards


# =============================================================
# FOLLOW-UP 1: The node interface has no parent pointer
# =============================================================
# Answer: build the parent map yourself in the first pass (build_cache
# does: parent[node]) instead of reading node.parent.


# =============================================================
# FOLLOW-UP 2: Apply several flips at once, or flips that stay
# =============================================================
# Answer: do not restore the values: flip a leaf, walk up and update, and
# leave the cache as it is for the next flip. For a batch, flip all the
# leaves first and recompute the affected nodes bottom-up once.


# =============================================================
# FOLLOW-UP 3: Which leaves change the answer at all?
# =============================================================
```

```
# =============================================================
# FOLLOW-UP 4: Which leaves can change the answer at all?
# =============================================================
# ---- follow-up 3: function ----
# Answer: pass a "matters" flag down from the root. Through AND the
# other side must be True, through OR the other side must be False,
# through XOR and NOT the flag always passes. The leaves that arrive
# with the flag still set are the ones that can change the answer. One
# pass, O(n).
def sensitive_leaves(root):
    parent, value, leaves = build_cache(root)
    found = []
    stack = [(root, True)]
    while stack:
        node, matters = stack.pop()
        if node.op is None:
            if matters:
                found.append(node)
            continue
        left, right = node.left, node.right
        left_ok = matters
```

```
        right_ok = matters
        if node.op == "AND":
            left_ok = matters and value[right]
            right_ok = matters and value[left]
        elif node.op == "OR":
            left_ok = matters and not value[right]
            right_ok = matters and not value[left]
        stack.append((left, left_ok))
        if right is not None:
            stack.append((right, right_ok))
    return found


# ---- follow-up 3: tests ----
def test_follow_up_3():
    tree = example_expression()
    leaves, answers = flip_answers(tree)
    base = evaluate(tree)
    matters = sensitive_leaves(tree)
    for leaf in leaves:                          # compare with really flipping each leaf
        leaf.val = not leaf.val
        changes = evaluate(tree) != base
        leaf.val = not leaf.val
```

```
        assert changes == (leaf in matters)


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

23 Longest Consecutive Path in Binary Tree — MEDIUM

Template fit: pass the parent's value and the run length down; extend the run when value == parent + 1, else restart at 1

QUESTION

Given a binary tree, return the length of the longest path that goes from a parent down to a child where every value is exactly 1 more than the previous one (counted in nodes).

Function signature

def longestConsecutive(root: TreeNode) -> int

EXAMPLES

Example 1	
Input	`[1, null, 3, 2, 4, null, null, null, 5]`
Output	`3`
Why	`3 -> 4 -> 5 is the longest increasing-by-one downward path.`

QUESTION

Example 2	
Input	[2, null, 3, 2, null, 1]
Output	2
Why	2 -> 3 is the longest; 3 -> 2 and 2 -> 1 go the wrong way.

FULL CODE | main question first, then every follow-up, tests at the end

```
# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ========================================================

# ========================================================
# MAIN QUESTION
# ========================================================
# ---- main question: helper functions ----
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right


# ---- main question: main function ----
# Idea: go down the tree carrying the length of the run that ends at
# the current node. A child that is exactly parent + 1 continues the
# run; any other child starts a new run of 1. Keep the best run seen.
# The length is counted in NODES (say that choice out loud).
# This version uses a stack, so it is also the answer to follow-up 2
# (no recursion).
def longestConsecutive(root):
    if root is None:
        return 0
    best = 0
    stack = [(root, 1)]
    while stack:
        node, run = stack.pop()
        if run > best:
            best = run
        for child in (node.left, node.right):
            if child is None:
                continue
            if child.val == node.val + 1:
                stack.append((child, run + 1))
            else:
                stack.append((child, 1))
    return best


# ---- main question: tests ----
```

```
def example_tree():
    #   1
    #    \
    #     3
    #    / \
    #   2   4
    #        \
    #         5
    return TreeNode(1, None, TreeNode(3, TreeNode(2), TreeNode(4, None, TreeNode(5))))
```

```
def test_main_question():
    assert longestConsecutive(example_tree()) == 3          # 3 -> 4 -> 5
    assert longestConsecutive(None) == 0
    assert longestConsecutive(TreeNode(7)) == 1
    assert longestConsecutive(TreeNode(2, TreeNode(2), TreeNode(2))) == 1   # equal values do not extend a ru
n


# ============================================================
# FOLLOW-UP 1: What does the any-direction version return for a single node, or for duplicate values?
# ============================================================
# Answer: a single node is a path of 1; equal values never extend a
# path (the next value must be exactly +1 or -1). See the tests of follow-up 4.


# ============================================================
# FOLLOW-UP 2: Can you avoid recursion?
# ============================================================
# Answer: yes, longestConsecutive above already uses an explicit stack.


# ============================================================
# FOLLOW-UP 3: Return the path itself, not only its length
# ============================================================
# ---- follow-up 3: function ----
# Answer: carry the run itself (a list of values) instead of its length
# and remember the longest list.
def longest_consecutive_path(root):
    best_path = []
    if root is None:
        return best_path
    stack = [(root, [root.val])]
    while stack:
        node, run = stack.pop()
        if len(run) > len(best_path):
            best_path = run
        for child in (node.left, node.right):
            if child is None:
                continue
            if child.val == node.val + 1:
                stack.append((child, run + [child.val]))
            else:
                stack.append((child, [child.val]))
    return best_path


# ---- follow-up 3: tests ----
def test_follow_up_3():
    assert longest_consecutive_path(example_tree()) == [3, 4, 5]
    assert longest_consecutive_path(None) == []


# ============================================================
# FOLLOW-UP 4: The path may go in any direction (LeetCode 549)
# ============================================================


# ============================================================
# FOLLOW-UP 5: The values must increase by 1 or decrease by 1 along the path but not mix
# ============================================================
# ---- follow-up 4: function ----
# Answer: for every node keep two numbers: the longest chain going down
# that INCREASES by 1 each step (up_chain) and the longest that DECREASES
```

```
# by 1 (down_chain). The best path that bends at this node joins the two
# chains: up_chain + down_chain - 1 (the node is counted once). Children
# are handled before their parents.
def longest_any_direction(root):
    if root is None:
        return 0
    order = []
    stack = [root]
    while stack:
        node = stack.pop()
        order.append(node)
        for child in (node.left, node.right):
            if child is not None:
                stack.append(child)
    up_chain = {}
    down_chain = {}
    best = 1
```

```
    for node in reversed(order):                # children come before their parents
        up = 1
        down = 1
        for child in (node.left, node.right):
            if child is None:
                continue
            if child.val == node.val + 1 and up_chain[child] + 1 > up:
                up = up_chain[child] + 1
            if child.val == node.val - 1 and down_chain[child] + 1 > down:
                down = down_chain[child] + 1
        up_chain[node] = up
        down_chain[node] = down
        if up + down - 1 > best:
            best = up + down - 1
    return best


# ---- follow-up 4: tests ----
def test_follow_up_4():
    assert longest_any_direction(TreeNode(2, TreeNode(1), TreeNode(3))) == 3     # 1 -> 2 -> 3 bends at 2
    assert longest_any_direction(TreeNode(5)) == 1                               # a single node
    assert longest_any_direction(TreeNode(2, TreeNode(2), TreeNode(2))) == 1      # duplicates do not count
    assert longest_any_direction(example_tree()) == 4                           # 2 -> 3 -> 4 -> 5 bends at 3


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_3()
    test_follow_up_4()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

24 Given the root of a binary tree, return the length of the diameter of the tree. MEDIUM

Template fit: best = max(best, left + right); return 1 + max(left, right) (the diameter)

QUESTION

Given the root of a binary tree, create a function diameterOfTree that determines and returns the length of the diameter of the tree, where the diameter of a tree is defined as the number of nodes along the longest path between any two nodes in the tree. For example, given the following binary tree: // tree 1 / \ 2 3 / \ 4 5 The diameter is 4, which is the length of the path [4, 2, 1, 3] or [5, 2, 1, 3].Watch Ali, SDE @ Amazon, answer the question: "Given the root of a binary tree, return the length of the diameter of the tree."No answers contributed by the community yet.Interview experiences6 shared‹›Uber · 3 months ago"interview process is very lengthy took around 5 weeks, and expectations is very high in each round, for LLD working code is required, also dont mess your HM round since i felt like it contained more weightage"Software Engineer · Mid Level / L4Meta · 4 months ago"The interview went smoothly overall. 1 round of phone screen followed by the full loop of 4 rounds packed in a day. 2 coding 1 behavior 1 system design."Software Engineer · Senior / L5Google · 9 months ago"I interviewed for an AI Engineer internship, but the process was mostly DSA-heavy. I had an online coding round first, then a technical round with binary tree and DP questions, then another harder DSA round with DP and linked list questions, and finally an HR round. The online round covered arrays and strings, and the later rounds got progressively more difficult. The only non-DSA part was the last

EXAMPLES

Example 1	
Input	`[1, 2, 3, 4, 5]`
Output	`4`
Why	`The longest path 4 - 2 - 1 - 3 has 4 nodes.`

Example 2	
Input	`[1]`
Output	`1`
Why	`A single node.`

FULL CODE | main question first, then every follow-up, tests at the end

```python
# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right


# ---- main question: main function ----
# Idea: one post-order pass. depth(node) returns the height of the
# node's subtree (in nodes) and, on the way, notes the longest path that
# passes THROUGH this node: left height + right height + 1 nodes. Every
# node is visited once. The diameter is counted in NODES, as the
# question says (follow-up 3 shows edges).
def diameter_nodes(root):
```

```
    best = 0

    def depth(node):
        nonlocal best
        if node is None:
            return 0
        left = depth(node.left)
        right = depth(node.right)
        if left + right + 1 > best:
            best = left + right + 1
        if left > right:
            return left + 1
        return right + 1

    depth(root)
    return best


# ---- main question: tests ----
def test_main_question():
```

```
    root = TreeNode(1, TreeNode(2, TreeNode(4), TreeNode(5)), TreeNode(3))
    assert diameter_nodes(root) == 4              # 4 -> 2 -> 1 -> 3
    assert diameter_nodes(None) == 0
    assert diameter_nodes(TreeNode(1)) == 1


# ========================================================
# FOLLOW-UP 1: An n-ary tree
# ========================================================
# ---- follow-up 1: helper functions ----
class NNode:
    def __init__(self, val, children=()):
        self.val = val
        self.children = list(children)


def two_largest(numbers):
    # the biggest and the second biggest number (0 when there are fewer)
    first = 0
    second = 0
    for x in numbers:
        if x > first:
            second = first
            first = x
        elif x > second:
            second = x
    return first, second


# ---- follow-up 1: function ----
# Answer: at every node add the TWO tallest child heights (not just left
# and right). Track the top two in one pass over the children: O(d) per
# node (sorting them would cost O(d log d)).
def diameter_nary(root):
    best = 0

    def depth(node):
        nonlocal best
        heights = []
        for child in node.children:
            heights.append(depth(child))
        first, second = two_largest(heights)
        if first + second + 1 > best:
            best = first + second + 1
        return first + 1
```

```
    if root is None:
        return 0
    depth(root)
    return best


# ---- follow-up 1: tests ----
def test_follow_up_1():
    root = NNode(1, [NNode(2, [NNode(5), NNode(6)]), NNode(3), NNode(4)])
    assert diameter_nary(root) == 4              # 5 -> 2 -> 1 -> 3 has 4 nodes
    assert diameter_nary(NNode(1)) == 1
    assert diameter_nary(None) == 0


# ======================================================
# FOLLOW-UP 2: Edges have weights, or you want the path itself
# ======================================================
# ---- follow-up 2: function ----
# Answer: add the edge weight to each child's height before comparing.
# (For the path itself also store which child gave each of the two best
# heights and rebuild it from the node that produced the best sum.)
# Here children are pairs (weight, node) and the answer is the largest
# total weight of a path.
def diameter_weighted(root):
    best = 0

    def down(node):
        nonlocal best
        lengths = []
        for weight, child in node.children:
            lengths.append(weight + down(child))
        first, second = two_largest(lengths)
        if first + second > best:
            best = first + second
        return first

    if root is None:
        return 0
    down(root)
    return best


# ---- follow-up 2: tests ----
def test_follow_up_2():
```

```
    root = NNode(1, [(5, NNode(2, [(1, NNode(4))])), (2, NNode(3))])
    assert diameter_weighted(root) == 8          # 4 -1- 2 -5- 1 -2- 3  = 1 + 5 + 2


# ======================================================
# FOLLOW-UP 3: Count the diameter in nodes instead of edges
# ======================================================
# ---- follow-up 3: function ----
# Answer: a path of k edges has k + 1 nodes, so the two numbers differ
# by one. State which convention the interviewer wants before you code.
def diameter_edges(root):
    if root is None:
        return 0
    return diameter_nodes(root) - 1


# ---- follow-up 3: tests ----
def test_follow_up_3():
    root = TreeNode(1, TreeNode(2, TreeNode(4), TreeNode(5)), TreeNode(3))
    assert diameter_edges(root) == 3
    assert diameter_edges(None) == 0
```

```
# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

25 Implement zigzag level-order traversal

MEDIUM

Template fit: collect values by depth; reverse every odd level (or fill each level from the other end)

QUESTION

Given the root of a binary tree, return its values level by level, reading the first level from left to right, the next from right to left, and so on, alternating. The result is a list of lists, one per level. Aim for O(n) time.

Function signature

def zigzag_level_order(root: TreeNode) -> list

EXAMPLES

Example 1	
Input	`[3, 9, 20, null, null, 15, 7]`
Output	`[[3], [20, 9], [15, 7]]`
Why	`Level 0 is [3], level 1 is read right to left [20, 9], level 2 left to right [15, 7].`

Example 2	
Input	`[1]`
Output	`[[1]]`
Why	`A single node.`

Example 3	
Input	`[]`
Output	`[]`
Why	`An empty tree has no levels.`

FULL CODE | main question first, then every follow-up, tests at the end

```
# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

from collections import deque


# ============================================================
# MAIN QUESTION
```

```python
# ============================================================
# ---- main question: helper functions ----
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right


# ---- main question: main function ----
# Idea: go level by level with a queue. Read the first level left to
# right, the next one right to left, and so on: keep a flag and reverse
# the level list whenever the flag says "right to left".
def zigzag_level_order(root):
    result = []
    queue = deque()
    if root:
        queue.append(root)
    left_to_right = True
    while queue:
        level = []
        for i in range(len(queue)):               # exactly the nodes of this level
            node = queue.popleft()
            level.append(node.val)
            if node.left:
                queue.append(node.left)
            if node.right:
                queue.append(node.right)
        if not left_to_right:
            level.reverse()
        result.append(level)
        left_to_right = not left_to_right
    return result
```

```python
# ---- main question: tests ----
def example_tree():
    return TreeNode(3, TreeNode(9), TreeNode(20, TreeNode(15), TreeNode(7)))


def test_main_question():
    assert zigzag_level_order(example_tree()) == [[3], [20, 9], [15, 7]]
    assert zigzag_level_order(TreeNode(1)) == [[1]]
    assert zigzag_level_order(None) == []


# ============================================================
# FOLLOW-UP 1: Bottom-up level order (leaf level first)
# ============================================================
# ---- follow-up 1: function ----
# Answer: do the normal level-order traversal and reverse the list of
# levels at the end.
def level_order_bottom_up(root):
    levels = []
    queue = deque()
    if root:
        queue.append(root)
    while queue:
        level = []
        for i in range(len(queue)):
            node = queue.popleft()
            level.append(node.val)
            if node.left:
                queue.append(node.left)
            if node.right:
```

```
                queue.append(node.right)
        levels.append(level)
    levels.reverse()
    return levels


# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert level_order_bottom_up(example_tree()) == [[15, 7], [9, 20], [3]]


# ========================================================
# FOLLOW-UP 2: Do it with DFS
```

```
# ========================================================
# ---- follow-up 2: function ----
# Answer: pass the depth down and add each value to levels[depth]; at the
# end reverse the odd levels. The recursion depth is the tree height.
def zigzag_dfs(root):
    levels = []

    def visit(node, depth):
        if node is None:
            return
        if depth == len(levels):
            levels.append([])
        levels[depth].append(node.val)
        visit(node.left, depth + 1)
        visit(node.right, depth + 1)

    visit(root, 0)
    for depth in range(1, len(levels), 2):        # the odd levels are read right to left
        levels[depth].reverse()
    return levels


# ---- follow-up 2: tests ----
def test_follow_up_2():
    assert zigzag_dfs(example_tree()) == [[3], [20, 9], [15, 7]]
    assert zigzag_dfs(None) == []


# ========================================================
# FOLLOW-UP 3: N-ary tree
# ========================================================
# ---- follow-up 3: helper functions ----
class NNode:
    def __init__(self, val, children=()):
        self.val = val
        self.children = list(children)


# ---- follow-up 3: function ----
# Answer: loop over a list of children instead of left and right; the
# direction flag works in the same way.
def zigzag_nary(root):
    result = []
    queue = deque()
    if root:
```

```
        queue.append(root)
    left_to_right = True
    while queue:
        level = []
        for i in range(len(queue)):
            node = queue.popleft()
```

```
            level.append(node.val)
            for child in node.children:
                queue.append(child)
        if not left_to_right:
            level.reverse()
        result.append(level)
        left_to_right = not left_to_right
    return result


# ---- follow-up 3: tests ----
def test_follow_up_3():
    root = NNode(1, [NNode(2, [NNode(5)]), NNode(3), NNode(4)])
    assert zigzag_nary(root) == [[1], [4, 3, 2], [5]]


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

26 Repeatedly Delete Leaves Matching a Target Value From a Binary Tree — HARD

Template fit: set node.left = dfs(node.left) and node.right likewise; return None when the node became a leaf with the target value

QUESTION

Given a binary tree and an integer **`target`**, delete every leaf whose value equals **`target`**. Deleting a leaf can turn its parent into a leaf; if that parent's value also equals **`target`**, it must be deleted as well, and so on, until no leaf with value **`target`** remains. Return the resulting tree, which is empty if the root itself ends up deleted.

Function Signature

`remove_target_leaves(tree: list[int | None], target: int) -> list[int | None]`

Tree Encoding

The input and the output use the same level-order encoding:

- An empty tree is **`[]`**.
- Otherwise the first element is the root's value. Present nodes are then visited in breadth-first order, left to right, and for each one its left child and then its right child are appended, using **`None`** for an absent child. Children of absent nodes are not listed.
- Trailing **`None`** values are omitted. The input list is always well formed, and the output must be in exactly this canonical form.

Rules

- A leaf is a node with no children.
- A node whose value equals **`target`** but that still has at least one child after all deletions is kept.
- Kept nodes never move: every kept node keeps its original parent and its side (left or right).
- The returned tree contains no leaf whose value equals **`target`**.

Constraints

- The input tree has between 1 and 5000 nodes, and its height is at most 500.

QUESTION

- Node values and **`target`** are integers in **`[-1000000000, 1000000000]`**.

Examples

Input: **`tree = [5,7,7,7,3,None,7], target = 7`**
Output: **`[5,7,None,None,3]`** The leaf 7 under the root's left child is deleted, but that left child still has the child 3, so it stays. On the right side, deleting the lower 7 turns the root's right child into a leaf with value 7, so it is deleted too.
Input: **`tree = [3,3,1], target = 3`**
Output: **`[3,None,1]`** Only the left leaf is deleted. The root's value is 3, but it still has a child.
Input: **`tree = [4,4,4], target = 4`**
Output: **`[]`**

EXAMPLES

Example 1	
Input	`tree = [1, 2, 3, 2, null, 2, 4], target = 2 and tree = [1, 3, 3, 3, 2], target = 3`
Output	`values(remove_leaves(tree([1, 2, 3, 2, None, 2, 4]), 2)[0]) -> [1, null, 3, null, 4]` `values(remove_leaves(tree([1, 3, 3, 3, 2]), 3)[0]) -> [1, 3, null, null, 2]`
Why	`remove_leaves returns (the new root, how many leaves were removed). Deleting a leaf can turn its parent into a leaf, which is deleted too if it also equals the target: in the first tree the 2s go and the root's left branch disappears entirely.`

FULL CODE | main question first, then every follow-up, tests at the end

```python
# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ========================================================

from collections import deque


# ========================================================
# MAIN QUESTION
# ========================================================
# ---- main question: helper functions ----
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right


def build_tree(values):
    # level-order list with None for a missing child: [1, 2, 3, 2, None, 2, 4]
    if not values:
        return None
    root = TreeNode(values[0])
    queue = deque([root])
    i = 1
    while queue and i < len(values):
        node = queue.popleft()
        if i < len(values) and values[i] is not None:
            node.left = TreeNode(values[i])
            queue.append(node.left)
        i += 1
        if i < len(values) and values[i] is not None:
            node.right = TreeNode(values[i])
            queue.append(node.right)
        i += 1
```

```
    return root


def tree_to_list(root):
    # the same encoding, with the trailing Nones removed
    result = []
    queue = deque([root])
    while queue:
        node = queue.popleft()
        if node is None:
            result.append(None)
        else:
            result.append(node.val)
            queue.append(node.left)
            queue.append(node.right)
    while result and result[-1] is None:
        result.pop()
    return result


def remove_leaves(root, target):
    # post-order: handle both children first, then look at the node itself.
    # Returns (the new root, how many nodes were deleted) -- follow-up 3.
    deleted = 0

    def visit(node):
        nonlocal deleted
        if node is None:
            return None
        node.left = visit(node.left)
        node.right = visit(node.right)
        if node.left is None and node.right is None and node.val == target:
            deleted += 1                          # a leaf with the target value disappears
            return None
        return node

    return visit(root), deleted


# ---- main question: main function ----
# Idea: one post-order pass. Process both children first, then the node:
# if it has become a leaf and its value is the target, it is removed
# (its parent then sees None). That is exactly how a deleted leaf can
# turn its parent into a new leaf.
def remove_target_leaves(tree, target):
    root, deleted = remove_leaves(build_tree(tree), target)
    return tree_to_list(root)


# ---- main question: tests ----
def test_main_question():
    assert remove_target_leaves([1, 2, 3, 2, None, 2, 4], 2) == [1, None, 3, None, 4]
```

```
    assert remove_target_leaves([1, 3, 3, 3, 2], 3) == [1, 3, None, None, 2]
    assert remove_target_leaves([1, 2, None, 2, None, 2], 2) == [1]
    assert remove_target_leaves([5], 5) == []
    assert remove_target_leaves([], 5) == []


# ======================================================
# FOLLOW-UP 1: The tree is a 100,000-node chain. How do you avoid a stack overflow?
# ======================================================
# ---- follow-up 1: function ----
# Answer: collect the nodes in pre-order with your own stack and remember
# each node's parent; then walk that list BACKWARDS (children before
```

```python
# parents) and unlink every node that is a matching leaf. That keeps the
# post-order guarantee without recursion.
def remove_leaves_iterative(root, target):
    if root is None:
        return None, 0
    parent = {root: None}
    order = []
    stack = [root]
    while stack:
        node = stack.pop()
        order.append(node)
        for child in (node.left, node.right):
            if child is not None:
                parent[child] = node
                stack.append(child)
    deleted = 0
    for node in reversed(order):                  # children are handled before their parents
        if node.left or node.right or node.val != target:
            continue
        deleted += 1
        up = parent[node]
        if up is None:
            return None, deleted
        if up.left is node:
            up.left = None
        else:
            up.right = None
    return root, deleted


# ---- follow-up 1: tests ----
def test_follow_up_1():
    chain = TreeNode(5)
```

```python
    node = chain
    for i in range(99999):
        node.left = TreeNode(5)
        node = node.left
    new_root, deleted = remove_leaves_iterative(chain, 5)
    assert new_root is None and deleted == 100000
    root, deleted = remove_leaves_iterative(build_tree([1, 2, 3, 2, None, 2, 4]), 2)
    assert tree_to_list(root) == [1, None, 3, None, 4] and deleted == 3


# ======================================================
# FOLLOW-UP 2: Delete any node whose whole subtree sums to zero, or whose value matches a predicate
# ======================================================
# Answer: the same post-order pass: after both children are handled,
# decide with the new rule (the sum of the node and its remaining
# children, or a yes/no function of the value) instead of "is it a
# leaf with the target value".


# ======================================================
# FOLLOW-UP 3: Return how many nodes were deleted
# ======================================================
# Answer: remove_leaves above already returns the count next to the new root.
# ---- follow-up 3: tests ----
def test_follow_up_3():
    root, deleted = remove_leaves(build_tree([1, 2, 3, 2, None, 2, 4]), 2)
    assert deleted == 3


# ======================================================
# RUN EVERYTHING
# ======================================================
```

```
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

27 Delete an Entire Binary Tree

HARD

Template fit: post-order: free the children first, then the node itself

QUESTION

You are given the root of a binary tree (not a BST); each node has only left and right child pointers. You must "free" every node so that after the call the tree is empty. Because a node must not be freed before its children, the correct freeing order is a post-order traversal. Implement delete_tree(root) that frees all nodes and returns the list of node values in the exact order they are freed (post-order: left subtree, right subtree, then node).

Function signature

```
from typing import Optional
class TreeNode:
    def __init__(self, val: int = 0,
                 left: "Optional[TreeNode]" = None,
                 right: "Optional[TreeNode]" = None) -> None:
        self.val = val
        self.left = left
        self.right = right
def delete_tree(root: Optional[TreeNode]) -> list[int]:
    """Free every node and return the values in the order freed (post-order)."""
    ...
```

Follow-up (as reported): Recursive DFS uses O(h) stack space. Achieve O(1) extra space by repeatedly rotating the tree (Morris-style): while the root has a left child, rotate it up; once the root has no left child, free the root and move to its right child.

EXAMPLES

Example 1	
Input	`root = 1 / \ 2 3`
Output	`[2, 3, 1]`
Why	`Post-order frees the left child, then the right child, then the root.`

Example 2	
Input	`root = None`
Output	`[]`
Why	`An empty tree frees nothing.`

Example 3	
Input	`root = 1 / 2 / 3`
Output	`[3, 2, 1]`
Why	`A left-leaning chain frees deepest-first.`

CONSTRAINTS

QUESTION

- 0 <= number of nodes <= 10^5 .
- Node values are integers.

FULL CODE | main question first, then every follow-up, tests at the end

```
# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ========================================================

# ========================================================
# MAIN QUESTION
# ========================================================
# ---- main question: helper functions ----
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right


def build_tree(shape):
    # shape is None or (value, left shape, right shape)
    if shape is None:
        return None
    value, left, right = shape
    return TreeNode(value, build_tree(left), build_tree(right))


# ---- main question: main function ----
# Idea: a node may only be freed AFTER its children, so free the left
# subtree, then the right subtree, then the node itself (post-order).
# "Freeing" a node clears its two pointers; the values are returned in
# the order they were freed.
def delete_tree(root):
    freed = []

    def free(node):
        if node is None:
            return
        free(node.left)
        free(node.right)
        freed.append(node.val)
        node.left = None
        node.right = None

    free(root)
    return freed
```

```


# ---- main question: tests ----
def test_main_question():
    assert delete_tree(build_tree((1, (2, None, None), (3, None, None)))) == [2, 3, 1]
    assert delete_tree(None) == []
    assert delete_tree(build_tree((1, (2, (3, None, None), None), None))) == [3, 2, 1]


# ========================================================
# FOLLOW-UP 1: Do it with O(1) extra space
# ========================================================
# ---- follow-up 1: function ----
```

```python
# Answer: rotate instead of recursing. While the root has a left child,
# rotate it up (the left child becomes the root and the old root becomes
# its right child). When the root has no left child, free it and move to
# its right child. No stack is used (the order of freeing is no longer
# post-order, but every node is freed once).
def delete_tree_constant_space(root):
    freed = []
    while root:
        if root.left:
            left = root.left
            root.left = left.right
            left.right = root
            root = left
        else:
            following = root.right
            freed.append(root.val)
            root.right = None
            root = following
    return freed


# ---- follow-up 1: tests ----
def test_follow_up_1():
    tree = build_tree((1, (2, (4, None, None), (5, None, None)), (3, None, None)))
    assert sorted(delete_tree_constant_space(tree)) == [1, 2, 3, 4, 5]
    assert delete_tree_constant_space(None) == []


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

GROUP 4 OF 43 | 9 QUESTIONS, ONE CODE TEMPLATE | 8 M 1 H

A heap of what is available: pop the best, put back the changed one

28 Streaming Median (LC 295) with Follow-ups

MEDIUM

Template fit: two heaps: a max-heap for the lower half and a min-heap for the upper half; keep them balanced

QUESTION

Design a structure that receives numbers one at a time and can report the median of everything received so far at any moment. addNum(x) adds a number; findMedian() returns the middle value, or the mean of the two middle values when the count is even.

Function signature

class MedianFinder:
  def addNum(self, x: float) -> None
  def findMedian(self) -> float

EXAMPLES

Example 1

Input	`m = MedianFinder()`
	`m.addNum(5)`
	`m.addNum(2)`
	`m.findMedian()`
	`m.addNum(8)`
	`m.findMedian()`
	`m.addNum(1)`
	`m.findMedian()`

Output	`m.findMedian() -> 3.5`
	`m.findMedian() -> 5`
	`m.findMedian() -> 3.5`

Why	`Sorted so far: [2, 5] -> 3.5; [2, 5, 8] -> 5; [1, 2, 5, 8] -> 3.5.`

FULL CODE | main question first, then every follow-up, tests at the end

```
# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

import heapq


# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: main class ----
# Idea: keep the numbers in two heaps. The LOWER half is in a max-heap
# (store the negatives, since heapq is a min-heap) and the UPPER half in a
# min-heap. Their sizes differ by at most one (the lower half may have one
# more) and every number in the lower half is at most every number in the
# upper half. The median is the top of the lower heap, or the average of the
# two tops when the sizes are equal. addNum is O(log n), findMedian O(1).
class MedianFinder:
    def __init__(self):
        self.low = []                           # max-heap (negated numbers)
        self.high = []                          # min-heap
```

```
    def addNum(self, x):
        heapq.heappush(self.low, -x)               # go through the lower half so the halves stay ordered
        biggest_of_low = -heapq.heappop(self.low)
        heapq.heappush(self.high, biggest_of_low)
        if len(self.high) > len(self.low):         # keep the lower half the same size or one bigger
            smallest_of_high = heapq.heappop(self.high)
            heapq.heappush(self.low, -smallest_of_high)

    def findMedian(self):
        if len(self.low) > len(self.high):
            return float(-self.low[0])
        return (-self.low[0] + self.high[0]) / 2


# ---- main question: tests ----
def test_main_question():
    m = MedianFinder()
    for x in [5, 2, 8]:
        m.addNum(x)
    assert m.findMedian() == 5
```

```
    m.addNum(1)
    assert m.findMedian() == 3.5


# ========================================================
# FOLLOW-UP 1: Almost every value is in a small range (say 0 to 100). Can you do better than two heaps?
# ========================================================
# ---- follow-up 1: function ----
# Answer: yes. A counting array (one counter per possible value) gives O(1)
# inserts, and finding the median is one pass over at most V counters:
# O(V) per query for a fixed V. A moving pointer to the median bucket would
# make queries amortised O(1).
class CountingMedian:
    def __init__(self, max_value):
        self.counts = [0] * (max_value + 1)
        self.n = 0

    def addNum(self, x):
        self.counts[x] += 1
        self.n += 1

    def kth(self, k):
        # the k-th smallest number (1 = the smallest): one pass over the counters
        seen = 0
        for value in range(len(self.counts)):
            seen += self.counts[value]
            if seen >= k:
                return value

    def findMedian(self):
        lower = self.kth((self.n + 1) // 2)
        upper = self.kth(self.n // 2 + 1)
        return (lower + upper) / 2


# ---- follow-up 1: tests ----
def test_follow_up_1():
    counting = CountingMedian(100)
    for x in [5, 2, 8, 1]:
        counting.addNum(x)
    assert counting.findMedian() == 3.5


# ========================================================
# FOLLOW-UP 2: Now support an arbitrary k-th smallest, not only the median
```

```python
# ========================================================
# ---- follow-up 2: function ----
# Answer: two heaps only work for ONE fixed rank. Use an order-statistics
# structure: a Fenwick tree over the (compressed) values, where tree
# nodes hold counts and kth() walks down the tree, or a balanced search tree
# whose nodes know the size of their subtree. Both give any k-th smallest in
# O(log V).
class Fenwick:
    def __init__(self, size):
        self.size = size
        self.tree = [0] * (size + 1)

    def addNum(self, value):
        i = value + 1
        while i <= self.size:
            self.tree[i] += 1
            i += i & -i                          # i & -i is the lowest set bit of i

    def kth(self, k):
        # walk down the tree to the k-th smallest value (k starts at 1)
        position = 0
        step = 1
        while step * 2 <= self.size:
            step *= 2                            # the biggest power of two that fits
        while step > 0:
            following = position + step
            if following <= self.size and self.tree[following] < k:
                position = following
                k -= self.tree[following]
            step = step // 2
        return position


# ---- follow-up 2: tests ----
def test_follow_up_2():
    fenwick = Fenwick(100)
    for x in [5, 2, 8, 1]:
        fenwick.addNum(x)
    ranks = []
    for k in (1, 2, 3, 4):
        ranks.append(fenwick.kth(k))
    assert ranks == [1, 2, 5, 8]


# ========================================================
```

```python
# FOLLOW-UP 3: What if the data stream is too large to store?
# ========================================================
# Answer: exact medians need all the data (or a value range). If an
# approximation is fine, keep a fixed-size summary (a histogram of buckets, or a
# quantile sketch) and read the median from it.


# ========================================================
# FOLLOW-UP 4: The median of only the last K values
# ========================================================
# ---- follow-up 4: function ----
# Answer: keep the window in order of arrival (to know what expires) and
# in a sorted list (to read the middle). Insert the new value into its sorted
# place and remove the expired one. O(K) per update because a list shifts;
# two heaps with lazy deletion give O(log K).
class WindowMedian:
    def __init__(self, k):
        self.k = k
        self.order = []                          # the last k values, oldest first
        self.sorted = []                         # the same values in sorted order
```

```
    def addNum(self, x):
        self.order.append(x)
        position = 0
        while position < len(self.sorted) and self.sorted[position] < x:
            position += 1
        self.sorted.insert(position, x)
        if len(self.order) > self.k:
            old = self.order.pop(0)
            self.sorted.remove(old)                # removes one copy of the expired value

    def findMedian(self):
        n = len(self.sorted)
        return (self.sorted[(n - 1) // 2] + self.sorted[n // 2]) / 2


# ---- follow-up 4: tests ----
def test_follow_up_4():
    window = WindowMedian(3)
    for x in [9, 1, 5, 7]:
        window.addNum(x)
    assert window.findMedian() == 5                # the window is [1, 5, 7]


# ============================================================
```

```
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    test_follow_up_4()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

29 Compute minimum number of rooms needed — MEDIUM

Template fit: heap of end times; a meeting reuses the room that frees first if it ended, otherwise push a new room

QUESTION

Problem

You are given a list of meetings, each with a start time and end time. A single room can host only one meeting at a time. Two meetings **overlap** if one starts before the other ends (treat meetings as half-open intervals **[start, end)** so a meeting ending at time `t` does not conflict with another starting at time `t`).

Task

Return the **minimum number of rooms** required to host all meetings.

Input

- `intervals`: an array of `n` pairs `[start, end]` where `0 <= start <= end`.

Output

- An integer: the minimum number of rooms needed.

Constraints (typical interview bounds)

- `1 <= n <= 200000`
- Times are integers in a reasonable range (e.g., `0 . 10^9`).

Example

QUESTION

- Input: **`[[0,30],[5,10],[15,20]]`**
- Output: **2** (One room can host **`[0,30]`**, the other can host **`[5,10]`** then **`[15,20]`**.)

You are given a list of meetings, where each meeting is represented as [start, end]. A single room can host only one meeting at a time. Meetings are treated as half-open intervals [start, end), so a meeting ending at time t does not conflict with another meeting starting at time t. Return the minimum number of rooms required to host all meetings. If start == end, the meeting occupies no time and does not require a room.

Constraints

0 <= n <= 2000000 <= start <= end <= 10^9

EXAMPLES

```
Input: ([[0,30],[5,10],[15,20]],)
Expected Output: 2
Explanation: [0,30] overlaps with both [5,10] and [15,20], so at most 2 rooms are needed at the same
    time.
Input: ([[7,10],[2,4],[4,7],[10,12]],)
Expected Output: 1
```

Explanation: These meetings only touch at endpoints, so one room can host all of them in sequence.

Hints

Try turning each meeting into two events: one when it starts and one when it ends.Because intervals are half-open, if a meeting ends at the same time another starts, process the ending event first.

FULL CODE | main question first, then every follow-up, tests at the end

```python
# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ========================================================

import heapq


# ========================================================
# MAIN QUESTION
# ========================================================
# ---- main question: main function ----
# Idea: handle the meetings in order of start time and keep a min-heap of
# the END times of the rooms in use. If the room that ends earliest is free
# by this meeting's start, reuse it (replace its end time); otherwise open a
# new room (push). The size of the heap at the end is the number of rooms
# needed. Meetings are half-open [start, end): a meeting ending at t does not
# clash with one starting at t, and an empty meeting needs no room.
def min_meeting_rooms(intervals):
    meetings = []
    for start, end in intervals:
        if start < end:
            meetings.append((start, end))
    meetings.sort()
    ends = []
    for start, end in meetings:
        if ends and ends[0] <= start:
            heapq.heapreplace(ends, end)          # the earliest room is free: reuse it
        else:
            heapq.heappush(ends, end)
    return len(ends)


# ---- main question: tests ----
def test_main_question():
```

```
    assert min_meeting_rooms([(0, 30), (5, 10), (15, 20)]) == 2
    assert min_meeting_rooms([(7, 10), (2, 4)]) == 1
    assert min_meeting_rooms([(1, 3), (3, 5), (4, 4)]) == 1        # touching meetings share a room; empty on
es need none
    assert min_meeting_rooms([]) == 0


# =====================================================
# FOLLOW-UP 1: Can one person attend all meetings (LeetCode 252)?
```

```
# =====================================================
# ---- follow-up 1: function ----
# Answer: sort by start and check that every meeting ends no later than the
# next one starts. O(n log n) time, O(1) extra space.
def can_attend_all(intervals):
    ordered = sorted(intervals)
    for i in range(len(ordered) - 1):
        if ordered[i][1] > ordered[i + 1][0]:
            return False
    return True


# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert not can_attend_all([(0, 30), (5, 10), (15, 20)])
    assert can_attend_all([(7, 10), (2, 4)])


# =====================================================
# FOLLOW-UP 2: Return which room each meeting uses
# =====================================================
# ---- follow-up 2: function ----
# Answer: a heap of (end time, room number): reuse the room that frees up
# first, otherwise open the next room number, and write down the room for
# every meeting.
def assign_rooms(intervals):
    order = []
    for i in range(len(intervals)):
        order.append((intervals[i][0], intervals[i][1], i))
    order.sort()                                    # by start time (the index is kept)
    heap = []
    rooms = [None] * len(intervals)
    rooms_used = 0
    for start, end, i in order:
        if heap and heap[0][0] <= start:
            freed_at, room = heapq.heappop(heap)
        else:
            room = rooms_used
            rooms_used += 1
        rooms[i] = room
        heapq.heappush(heap, (end, room))
    return rooms


# ---- follow-up 2: tests ----
```

```
def test_follow_up_2():
    rooms = assign_rooms([(0, 30), (5, 10), (15, 20)])
    assert rooms[0] != rooms[1] and rooms[1] == rooms[2]


# =====================================================
# FOLLOW-UP 3: Meetings arrive online
# =====================================================
# ---- follow-up 3: function ----
# Answer: keep the heap of end times between calls: take out every end at
```

```
# or before the new start (those rooms are free), then push the new end.
# The size of the heap is the number of rooms in use right now; the largest
# size seen is the answer so far. (Meetings must come in order of start time.)
class RoomTracker:
    def __init__(self):
        self.ends = []
        self.peak = 0

    def book(self, start, end):
        while self.ends and self.ends[0] <= start:
            heapq.heappop(self.ends)              # that meeting is over: its room is free
        heapq.heappush(self.ends, end)
        if len(self.ends) > self.peak:
            self.peak = len(self.ends)
        return len(self.ends)


# ---- follow-up 3: tests ----
def test_follow_up_3():
    tracker = RoomTracker()
    in_use = []
    for start, end in [(0, 30), (5, 10), (15, 20)]:
        in_use.append(tracker.book(start, end))
    assert in_use == [1, 2, 2]
    assert tracker.peak == 2


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    test_follow_up_3()
```

```
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

30 Implement a Reusable Locker Allocation System — MEDIUM

Template fit: heap of freed locker numbers; take the smallest freed one before handing out a new number

QUESTION

Problem

A locker bank has lockers numbered from 1 through **n**. Process operations in order. **`store(customer)`** assigns the smallest available locker and returns its number, or **`-1`** if that customer already has a locker or none is available. **`pickup(customer)`** releases and returns that customer's locker, or **`-1`** if no locker is assigned. **`last()`** returns the locker number from the most recent successful **`store`**, or **`-1`** if there has not been one. Return one result per operation.

Constraints & Assumptions

- **`1 <= n <= 200,000`**.
- There are at most 200,000 operations.
- Customer IDs are non-empty strings.
- A released locker becomes immediately available for reuse.

Clarifications

- Failed store and pickup operations do not change **`last()`**.

QUESTION

- The allocation rule is always the smallest currently free locker number.
- A customer can store again after a successful pickup.

Examples

```
n = 3 operations = [store("a"), store("b"), pickup("a"), store("c"), last()] output = [1, 2, 1, 1, 1]
```

Hints

```
Track free lockers Use an ordered structure that can remove and reinsert the minimum locker efficiently.
```

```
Track ownership A map from customer ID to locker supports validation and pickup.
```

Process a fresh locker bank with lockers numbered 1 through n. Operations are ["store", customer], ["pickup", customer], or ["last"]. Store assigns and returns the smallest available locker, or -1 if the customer already owns one or none is free. Pickup releases and returns that customer's locker, or -1 if none is assigned. Last reports the locker from the most recent successful store, or -1 before any success. Return one integer per operation.
Constraints
1 <= n <= 200000.operations contains at most 200000 encoded store, pickup, or last lists.Customer IDs are non-empty strings.A released locker becomes immediately available for reuse.Failed store and pickup operations do not change last.A customer can store again after a successful pickup.

EXAMPLES

```
Input: (3, [['store', 'a'], ['store', 'b'], ['pickup', 'a'], ['store', 'c'], ['last']])
Expected Output: [1, 2, 1, 1, 1]
```

Explanation: This is the source example; the released smallest locker is reused.

```
Input: (5, [])
Expected Output: []
```

Explanation: An empty batch produces no per-operation results.

Hints

Keep available locker numbers in a min-oriented structure.Use a customer-to-locker map to validate stores and pickups.

FULL CODE | main question first, then every follow-up, tests at the end

```python
# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

import heapq
import threading


# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: main class ----
# n lockers numbered 1 .. n. store(customer) gives the smallest free locker
# number (-1 if the customer already has one or none is free); pickup(customer)
# frees the customer's locker and returns its number (-1 if there is none);
# last() is the locker of the latest SUCCESSFUL store (-1 if none yet).
# Idea: a min-heap of RELEASED locker numbers plus a counter of the next
# never-used number. A released locker is always smaller than any never-used
# one, so: take from the heap if it is not empty, else take the counter if it
# is at most n. No list of 10^9 lockers is ever built (follow-up 1).
class Lockers:
```

```
    def __init__(self, n):
        self.n = n
        self.next_new = 1                       # the next locker number never used
        self.free = []                          # released lockers, smallest on top
        self.of = {}                            # customer -> locker number
        self.last_store = -1

    def store(self, customer):
        if customer in self.of:
            return -1
        if self.free:
            locker = heapq.heappop(self.free)
        elif self.next_new <= self.n:
            locker = self.next_new
            self.next_new += 1
        else:
            return -1
        self.of[customer] = locker
        self.last_store = locker
        return locker
```

```
    def pickup(self, customer):
        if customer not in self.of:
            return -1
        locker = self.of.pop(customer)
        heapq.heappush(self.free, locker)       # it is available again at once
        return locker

    def last(self):
        return self.last_store                  # failed operations never change this


# ---- main question: tests ----
def process(n, operations):
    box = Lockers(n)
    answers = []
    for operation in operations:
        if operation[0] == "store":
            answers.append(box.store(operation[1]))
        elif operation[0] == "pickup":
            answers.append(box.pickup(operation[1]))
        else:
            answers.append(box.last())
    return answers


def test_main_question():
    operations = [["store", "a"], ["store", "b"], ["pickup", "a"], ["store", "c"], ["last"]]
    assert process(3, operations) == [1, 2, 1, 1, 1]
    assert process(2, [["last"], ["pickup", "x"]]) == [-1, -1]              # nothing stored yet, a missing
pickup
    full = [["store", "a"], ["store", "a"], ["store", "b"], ["store", "c"]]
    assert process(2, full) == [1, -1, 2, -1]                                 # a customer who has one, and
a full bank
    assert process(2, full + [["last"]])[-1] == 2                             # failed operations leave last
() alone


# =========================================================
# FOLLOW-UP 1: There are 10^9 lockers
# =========================================================
# ---- follow-up 1: tests ----
# Answer: nothing to change: only the released lockers are stored, and a
# counter stands for all the never-used ones.
def test_follow_up_1():
    assert process(10 ** 9, [["store", "a"], ["last"]]) == [1, 1]
```

```
# ============================================================
```

```
# FOLLOW-UP 2: Many threads call store and pickup
# ============================================================
# ---- follow-up 2: function ----
# Answer: protect the heap, the counter and the customer map with ONE lock,
# so the same locker number is never given out twice (or use finer locks
# around the pair (heap, counter)).
class SafeLockers:
    def __init__(self, n):
        self.lockers = Lockers(n)
        self.lock = threading.Lock()

    def store(self, customer):
        with self.lock:
            return self.lockers.store(customer)

    def pickup(self, customer):
        with self.lock:
            return self.lockers.pickup(customer)


# ---- follow-up 2: tests ----
def test_follow_up_2():
    safe = SafeLockers(2)
    assert safe.store("a") == 1 and safe.pickup("a") == 1


# ============================================================
# FOLLOW-UP 3: Lockers of different sizes
# ============================================================
# Answer: keep one heap of released numbers and one "next new" counter per
# SIZE; a customer asks for a size and the smallest locker of that size (or
# the next larger size when none is free) is given.


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    print("all tests passed")


if __name__ == "__main__":
```

```
    run_all_tests()
```

31 Simulate room assignments for scheduled meetings

MEDIUM

Template fit: a heap of free room numbers and a heap of (end time, room); a delayed meeting takes the earliest-ending room and shifts later

QUESTION

There are n rooms numbered 0 to n - 1 and a list of meetings (start, end). Meetings are handled in order of start time. A meeting takes the lowest-numbered free room; if every room is busy it waits until the room that frees up first becomes free and then runs for its full original length, so it ends later. Return the room that hosted the most meetings (the lowest room number on a tie).

QUESTION

Function signature

def mostBookedRoom(n: int, meetings: list) -> int

EXAMPLES

Example 1	
Input	`2, [[0, 10], [1, 5], [2, 7], [3, 4]]`
Output	`0`
Why	`Rooms 0 and 1 each host two meetings; the lower number wins the tie.`

Example 2	
Input	`3, [[1, 20], [2, 10], [3, 5], [4, 9], [6, 8]]`
Output	`1`
Why	`With three rooms, room 1 hosts two of the five meetings and no room hosts more.`

FULL CODE | main question first, then every follow-up, tests at the end

```python
# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

import heapq


# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: main function ----
# n rooms (0 .. n - 1) and meetings (start, end). Meetings are handled in
# order of start time; a meeting takes the lowest-numbered free room. If
# every room is busy it WAITS for the room that frees up first and then
# runs for its full original length (so it ends later). Which room hosts the
# most meetings (the lowest number on a tie)?
# Idea: two heaps. `free` holds the numbers of the free rooms (lowest on
# top); `busy` holds (time the room becomes free, room). For each meeting,
# first move the rooms that finished by its start from `busy` to `free`.
# If a room is free take the lowest one; otherwise take the room that
# frees up first and push the meeting back by the waiting time.
# (`by="time"` is follow-up 3: rank rooms by their total busy time.)
def mostBookedRoom(n, meetings, by="count"):
    free = list(range(n))
    heapq.heapify(free)
    busy = []
    used = [0] * n
    for start, end in sorted(meetings):
        while busy and busy[0][0] <= start:
            finished_at, room = heapq.heappop(busy)
            heapq.heappush(free, room)                      # this room is free again
        if free:
            room = heapq.heappop(free)
            heapq.heappush(busy, (end, room))
        else:
            finished_at, room = heapq.heappop(busy)         # wait for the room that frees up first
            heapq.heappush(busy, (finished_at + (end - start), room))
        if by == "count":
            used[room] += 1
        else:
            used[room] += end - start                       # the meeting's own length, not the delay
```

```
    best = 0
```

```
    for room in range(n):
        if used[room] > used[best]:
            best = room                                    # ties keep the lowest room number
    return best


# ---- main question: tests ----
def test_main_question():
    assert mostBookedRoom(2, [[0, 10], [1, 5], [2, 7], [3, 4]]) == 0
    assert mostBookedRoom(3, [[1, 20], [2, 10], [3, 5], [4, 9], [6, 8]]) == 1
    assert mostBookedRoom(1, [[0, 1], [0, 1]]) == 0           # a single room takes everything


# ========================================================
# FOLLOW-UP 1: What if meetings arrive online and cannot be reordered?
# ========================================================
# Answer: the same two heaps work, because meetings are handled in start
# order. If they can arrive out of order you need to know the earliest
# future start, so keep a buffer of the next few meetings and decide only
# when the order is certain.


# ========================================================
# FOLLOW-UP 2: How do you report the busiest room by total busy time instead of meeting count?
# ========================================================


# ========================================================
# FOLLOW-UP 3: Rank rooms by total busy time instead of the number of meetings
# ========================================================
# Answer: add each meeting's own length (not the delay) to the room's total
# instead of 1: the `by="time"` option above. The choice of room does not change.
# ---- follow-up 3: tests ----
def test_follow_up_3():
    meetings = [[0, 100], [1, 2], [2, 3], [3, 4]]
    assert mostBookedRoom(2, meetings, "count") == 1
    assert mostBookedRoom(2, meetings, "time") == 0


# ========================================================
# RUN EVERYTHING
# ========================================================
def run_all_tests():
    test_main_question()
    test_follow_up_3()
```

```
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

32 Trip Expense Settlement — MEDIUM

Template fit: a heap of creditors and a heap of debtors; the biggest of each settles min(amount) and the remainder goes back

QUESTION

A group of friends go on a trip and each friend may pay for others. Every record gives the payer, the amount and the list of people who share it equally (the payer may be one of them). Compute a small set of payments that settles everyone's debts: each payment is {payer, amount, payee}.

Function signature

def settle_greedy(records: list) -> list

EXAMPLES

Example 1

Input	`trip = [{'payer': 'alice', 'amount': 4000, 'payees': ['bob', 'jess', 'alice', 'sam']}, {'payer': 'jess', 'amount': 2000, 'payees': ['jess', 'alice']}]` `settle_greedy(trip)`
Output	`settle_greedy(trip) -> [{'payer': 'bob', 'amount': 1000, 'payee': 'alice'}, {'payer': 'sam', 'amount': 1000, 'payee': 'alice'}]`
Why	`Alice paid 4000 shared four ways and owes 1000 of Jess's 2000; Bob and Sam each owe 1000, and Jess is even. Two payments settle everything.`

Example 2

Input	`trip = [{'payer': 'alice', 'amount': 3000, 'payees': ['alice', 'bob', 'cy']}]` `settle_greedy(trip)`
Output	`settle_greedy(trip) -> [{'payer': 'bob', 'amount': 1000, 'payee': 'alice'}, {'payer': 'cy', 'amount': 1000, 'payee': 'alice'}]`
Why	`Bob and Cy each owe Alice their 1000 share.`

CONSTRAINTS

- 1 <= payments.length <= 1000
- 1 <= amount <= 10^6
- 1 <= payees.length <= 100

FULL CODE | main question first, then every follow-up, tests at the end

```
# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ========================================================

import heapq
import random


# ========================================================
# MAIN QUESTION
# ========================================================
# ---- main question: helper functions ----
def split_equally(amount, people):
    # share an amount (in whole CENTS, never floats) equally; the cents that
    # do not divide go to the first people in sorted order, so the rule is
    # always the same (follow-up 2)
    people = sorted(people)
    base = amount // len(people)
    extra = amount % len(people)
    shares = {}
    for i in range(len(people)):
```

```
        shares[people[i]] = base
        if i < extra:
            shares[people[i]] += 1
    return shares


def shares_of(record):
    # who owes what for one record. `payees` is a list (equal split) or a
    # dictionary {person: amount} for different shares (follow-up 3).
    payees = record["payees"]
    if isinstance(payees, dict):
        total = 0
        for share in payees.values():
            total += share
        if total != record["amount"]:
            raise ValueError("shares do not add up")
        return payees
    return split_equally(record["amount"], payees)


def balances(records):
```

```
    # the net balance of every person: what they paid minus what they owe.
    # Positive: the group owes them money. All balances add up to zero.
    net = {}
    for record in records:
        payer = record["payer"]
        net[payer] = net.get(payer, 0) + record["amount"]
        shares = shares_of(record)
        for person in shares:
            net[person] = net.get(person, 0) - shares[person]
    result = {}
    for person in net:
        if net[person] != 0:
            result[person] = net[person]
    return result


# ---- main question: main function ----
# Idea: the people who are owed money (positive balance) go into one
# max-heap and the people who owe money into another (push negatives to get a
# max-heap with heapq). Repeatedly let the biggest debtor pay the biggest
# creditor as much as one of them can (the smaller of the two amounts), and
# put back whoever still has something left. At least one person is settled
# per payment, so there are at most n - 1 payments.
def settle_greedy(records):
    net = balances(records)
    owed = []                                   # creditors: (minus what they are owed, person)
    debt = []                                   # debtors: (minus what they owe, person)
    for person in net:
        if net[person] > 0:
            owed.append((-net[person], person))
        else:
            debt.append((net[person], person))
    heapq.heapify(owed)
    heapq.heapify(debt)
    payments = []
    while owed and debt:
        owed_minus, creditor = heapq.heappop(owed)
        debt_minus, debtor = heapq.heappop(debt)
        pay = min(-owed_minus, -debt_minus)
        payments.append({"payer": debtor, "amount": pay, "payee": creditor})
        if -owed_minus > pay:
            heapq.heappush(owed, (owed_minus + pay, creditor))
        if -debt_minus > pay:
            heapq.heappush(debt, (debt_minus + pay, debtor))
```

```
    return payments
```

```


# ---- main question: tests ----
def payer_of(payment):
    return payment["payer"]


def test_main_question():
    trip = [{"payer": "alice", "amount": 4000, "payees": ["bob", "jess", "alice", "sam"]},
            {"payer": "jess", "amount": 2000, "payees": ["jess", "alice"]}]
    got = settle_greedy(trip)
    wanted = [{"payer": "bob", "amount": 1000, "payee": "alice"},
              {"payer": "sam", "amount": 1000, "payee": "alice"}]
    assert sorted(got, key=payer_of) == wanted
    assert balances(trip) == {"alice": 2000, "bob": -1000, "sam": -1000}


# =========================================================
# FOLLOW-UP 1: Do you need the true minimum number of payments?
# =========================================================
# ---- follow-up 1: function ----
# Answer: that is NP-hard in general (it is a zero-sum partition problem).
# The greedy answer needs at most n - 1 payments and is usually accepted. For
# an exact answer: people split into groups that settle among themselves
# (the group's balances add up to 0), and a group of g people needs g - 1
# payments, so find the MOST groups with a DP over subsets of people
# (fine up to about 20 people).
def min_payments_exact(values):
    n = len(values)
    total = [0] * (2 ** n)                          # total[mask] = the balances of the people in the subset `
mask`
    groups = [0] * (2 ** n)                         # groups[mask] = the most zero-sum groups inside `mask`
    for mask in range(1, 2 ** n):
        best = 0
        for i in range(n):
            if (mask // 2 ** i) % 2 == 1:           # person i is in this subset
                total[mask] += values[i]
                if groups[mask - 2 ** i] > best:
                    best = groups[mask - 2 ** i]
        groups[mask] = best
        if total[mask] == 0:
            groups[mask] += 1
    return n - groups[2 ** n - 1]


# the same answer by backtracking: settle the first open balance against
```

```
# every balance of the opposite sign (skipping equal ones, which give the same result)
def min_payments_search(values):
    values = [v for v in values if v != 0]

    def settle_from(start):
        while start < len(values) and values[start] == 0:
            start += 1
        if start == len(values):
            return 0
        best = None
        tried = set()
        for j in range(start + 1, len(values)):
            if values[j] * values[start] < 0 and values[j] not in tried:
                tried.add(values[j])
                values[j] += values[start]
                payments = 1 + settle_from(start + 1)
                values[j] -= values[start]
```

```
                if best is None or payments < best:
                    best = payments
        return best

    return settle_from(0)


# ---- follow-up 1: tests ----
def everyone_settled(payments, net):
    # after the payments every balance must be zero
    left = dict(net)
    for pay in payments:
        left[pay["payer"]] = left.get(pay["payer"], 0) + pay["amount"]
        left[pay["payee"]] = left.get(pay["payee"], 0) - pay["amount"]
    for person in left:
        if left[person] != 0:
            return False
    return True


def test_follow_up_1():
    assert min_payments_exact([5, -5]) == 1
    assert min_payments_exact([3, -3, 2, -2]) == 2
    assert min_payments_search([3, -3, 2, -2]) == 2
    random.seed(20)
    for round_number in range(100):
        records = []
        for i in range(random.randint(1, 5)):
```

```
            records.append({"payer": random.choice("abcdef"),
                            "amount": random.randint(1, 500),
                            "payees": random.sample("abcdef", random.randint(1, 4))})
        net = balances(records)
        total = 0
        for value in net.values():
            total += value
        assert total == 0
        payments = settle_greedy(records)
        assert everyone_settled(payments, net)
        assert len(payments) <= max(len(net) - 1, 0)
        values = list(net.values())
        assert min_payments_exact(values) == min_payments_search(values) <= len(payments)


# =============================================================
# FOLLOW-UP 2: Amounts that do not divide evenly (cents)
# =============================================================
# Answer: work in whole cents and decide where the remainder goes (here to
# the first people in sorted order; or to the payer). Never use floats for
# money, and make the rule deterministic: split_equally above.
# ---- follow-up 2: tests ----
def test_follow_up_2():
    assert split_equally(1000, ["c", "a", "b"]) == {"a": 334, "b": 333, "c": 333}      # 10.00 split three wa
ys


# =============================================================
# FOLLOW-UP 3: Different shares per person instead of equal split
# =============================================================
# Answer: give each record an explicit amount per person (a dictionary in
# `payees`); the net balance is computed the same way: shares_of.
# ---- follow-up 3: tests ----
def test_follow_up_3():
    custom = [{"payer": "x", "amount": 90, "payees": {"x": 10, "y": 30, "z": 50}}]
    assert balances(custom) == {"x": 80, "y": -30, "z": -50}
```

```
# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    test_follow_up_3()
```

```
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

33 Playlist Shuffle No Adjacent Artists

MEDIUM

Template fit: pop the artist with the most songs left, hold it for one turn, then push it back with a smaller count

QUESTION

Given the artist of every song in a playlist, return an order of the artists in which as few neighbours as possible are by the same artist: with no repeats next to each other whenever that is possible.

Function signature

def shuffle_no_adjacent(artists: list) -> list

EXAMPLES

Example 1	
Input	`['a', 'a', 'b', 'b', 'c']`
Output	`['a', 'b', 'a', 'b', 'c']`
Why	Two a's and two b's alternate; c fits wherever it likes.

Example 2	
Input	`['a', 'a', 'a', 'b']`
Output	`['a', 'b', 'a', 'a']`
Why	Three songs by 'a' cannot all be separated by a single 'b': the best is to keep two a's together at the end.

Example 3	
Input	`['a', 'a']`
Output	`['a', 'a']`
Why	A single artist: nothing can be done.

CONSTRAINTS

- 1 <= songs.length <= 10^4

FULL CODE | main question first, then every follow-up, tests at the end

```
# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================
```

```python
from collections import deque
import heapq
import random


# =====================================================
# MAIN QUESTION
# =====================================================
# ---- main question: helper functions ----
def count_items(items):
    counts = {}
    for item in items:
        counts[item] = counts.get(item, 0) + 1
    return counts


def adjacent_pairs(artists):
    # how many neighbouring songs are by the same artist?
    pairs = 0
    for i in range(1, len(artists)):
        if artists[i] == artists[i - 1]:
            pairs += 1
    return pairs


# ---- main question: main function ----
# Given the artist of every song, return an order with as few neighbours as
# possible by the same artist (none at all when that is possible).
# Idea (greedy with a max-heap): keep (songs left, artist) in a heap.
# At every step play the artist with the MOST songs left, but hold that
# artist back for one step so they cannot play twice in a row; put them back
# in the heap after the next artist has played. If only the artist just
# played is left, a repeat cannot be avoided.
def shuffle_no_adjacent(artists):
    counts = count_items(artists)
    heap = []
    for artist in counts:
        heap.append((-counts[artist], artist))        # the most songs left first; ties alphabetical
```

```python
    heapq.heapify(heap)
    order = []
    held = None                                       # the artist who just played (held back one step)
    while heap or held is not None:
        if heap:
            minus_left, artist = heapq.heappop(heap)
        else:
            minus_left, artist = held                 # only that artist is left: a forced repeat
            held = None
        order.append(artist)
        if held is not None:
            heapq.heappush(heap, held)                # the artist held back may play again now
            held = None
        if minus_left + 1 < 0:
            held = (minus_left + 1, artist)           # one song fewer left; hold this artist back
    return order


# ---- main question: tests ----
def forced_repeats(counts):
    # the fewest unavoidable neighbouring pairs (follow-up 3): the other n - m songs
    # split the biggest artist (m songs) into at most n - m + 1 groups, so at
    # least m - (n - m + 1) = 2m - n - 1 pairs touch
    total = 0
    biggest = 0
    for count in counts:
        total += count
```

```
        if count > biggest:
            biggest = count
    forced = 2 * biggest - total - 1
    if forced < 0:
        return 0
    return forced


def test_main_question():
    assert shuffle_no_adjacent(["a", "a", "b", "b", "c"]) == ["a", "b", "a", "b", "c"]
    assert shuffle_no_adjacent(["a", "a", "a", "b"]) == ["a", "b", "a", "a"]     # two a's must touch
    assert shuffle_no_adjacent(["a", "a"]) == ["a", "a"]
    assert shuffle_no_adjacent([]) == []
    random.seed(20)
    for round_number in range(150):                    # random playlists: the number of touching pairs is th
e minimum
        artists = []
        for i in range(random.randint(1, 7)):
            artists.append(random.choice("ABC"))
```

```
        order = shuffle_no_adjacent(artists)
        assert sorted(order) == sorted(artists)
        counts = list(count_items(artists).values())
        assert adjacent_pairs(order) == forced_repeats(counts)


# =====================================================
# FOLLOW-UP 1: The shuffle should look random, not always the same arrangement
# =====================================================


# =====================================================
# FOLLOW-UP 2: Songs by the same artist must be at least K apart
# =====================================================
# ---- follow-up 1: helper functions ----
def release(waiting, heap):
    # the artist that has waited longest may play again (if it has songs left)
    entry = waiting.popleft()
    if entry[0] < 0:
        heapq.heappush(heap, entry)


def take_oldest(waiting, heap):
    # break the gap on purpose: the first waiting artist that still has songs plays early
    while waiting:
        entry = waiting.popleft()
        if entry[0] < 0:
            heapq.heappush(heap, entry)
            return


# ---- follow-up 1: function ----
# Answer: for songs (artist, title): break ties between equally frequent
# artists at RANDOM (a random number in the heap entry) and pick the song
# inside an artist at random. The result is valid but not uniform among all
# valid shuffles (that would need counting arrangements). For "at least K
# apart" (follow-up 2) hold an artist back for K - 1 songs in a waiting
# queue (`gap` = K; gap 2 means never twice in a row); strict=True returns
# None when the gap cannot be kept.
def shuffle_playlist(songs, gap=2, strict=False, rng=random):
    by_artist = {}
    for artist, title in songs:
        if artist not in by_artist:
            by_artist[artist] = []
        by_artist[artist].append(title)
```

```
    for titles in by_artist.values():
        rng.shuffle(titles)                             # random song order inside an artist
    heap = []
    for artist in by_artist:
        heap.append((-len(by_artist[artist]), rng.random(), artist))
    heapq.heapify(heap)
    waiting = deque()
    order = []
    while len(order) < len(songs):
        if not heap:
            if strict:
                return None                             # everyone left is still waiting
            take_oldest(waiting, heap)
        minus_left, tie, artist = heapq.heappop(heap)
        order.append((artist, by_artist[artist].pop()))
        waiting.append((minus_left + 1, rng.random(), artist))   # waits even with no songs left: it marks sp
acing
        if len(waiting) >= gap:
            release(waiting, heap)
    return order


# ---- follow-up 1: tests ----
def test_follow_up_1():
    rng = random.Random(7)
    songs = [("A", "song1"), ("A", "song2"), ("B", "song3"), ("B", "song4"), ("C", "song5")]
    mixed = shuffle_playlist(songs, rng=rng)
    assert sorted(mixed) == sorted(songs)
    artists_in_order = []
    for artist, title in mixed:
        artists_in_order.append(artist)
    assert adjacent_pairs(artists_in_order) == 0
    different = set()
    for i in range(30):
        different.add(tuple(shuffle_playlist(songs, rng=rng)))
    assert len(different) > 1                           # runs differ from each other but are all valid
    heavy = [("A", str(i)) for i in range(4)] + [("B", "x")]
    assert shuffle_playlist(heavy, strict=True, rng=rng) is None
    assert shuffle_playlist([], rng=rng) == []


# ---- follow-up 2: tests ----
def test_follow_up_2():
    rng = random.Random(8)
    trio = []
    for artist in "ABC":
        for i in range(2):
            trio.append((artist, artist + str(i)))
    far = shuffle_playlist(trio, 3, True, rng)          # at least 3 apart: three artists, two songs each
    assert sorted(far) == sorted(trio)
    for i in range(len(far)):
        for back in range(1, 3):
            if i - back >= 0:
                assert far[i - back][0] != far[i][0]


# ========================================================
# FOLLOW-UP 3: What is the minimum number of unavoidable repeats?
# ========================================================
# Answer: forced_repeats(counts) above: with n songs and a biggest artist
# with m songs, the other n - m songs split that artist into at most
# n - m + 1 groups, so at least 2m - n - 1 neighbouring pairs of that
# artist are forced (if that is positive).
# ---- follow-up 3: tests ----
def test_follow_up_3():
    assert forced_repeats([4, 1]) == 2
```

```
    assert forced_repeats([2, 2, 1]) == 0
    assert forced_repeats([5]) == 4


# ====================================================
# RUN EVERYTHING
# ====================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

34 Minimum Makespan on Identical CPUs

MEDIUM

Template fit: the template as written: heap of the times the CPUs become free

QUESTION

You have C identical CPUs and a list release_times , where release_times[i] is the earliest time job i may start. Every job runs non-preemptively for the same fixed duration S . When a job's release time arrives it may start on any free CPU; if all CPUs are busy it waits and starts on the first CPU that becomes free (never before its release time). Each CPU runs one job at a time. Scheduling optimally, compute the minimum possible makespan: the time at which the last job finishes.

Function signature

def min_makespan(release_times: list[int], num_cpus: int, duration: int) -> int:

    ...

Follow-up (as reported): (Reported) The naive approach tracked an O(n*C) table of CPU availability; reduce it to a min-heap of the C next-free times, popping the earliest-free CPU per job after sorting jobs by release time.

EXAMPLES

Example 1	
Input	release_times = [0, 1, 2, 3], num_cpus = 2, duration = 3
Output	7
Why	CPU-A: job@0 -> [0,3), job@2 starts at 3 -> [3,6). CPU-B: job@1 -> [1,4), job@3 starts at 4 -> [4,7). Last finish = 7.

Example 2	
Input	release_times = [0, 0, 0], num_cpus = 1, duration = 2
Output	6
Why	One CPU runs them back to back: [0,2), [2,4), [4,6).

Example 3	
Input	release_times = [5], num_cpus = 3, duration = 4
Output	9
Why	The single job starts at its release time 5, finishes at 9.

CONSTRAINTS

- 1 <= C <= 10^5
- 0 <= len(release_times) <= 10^5
- 0 <= release_times[i] <= 10^9

QUESTION

- 1 <= S <= 10^9
- If there are no jobs, the makespan is 0 .
- Target complexity: O(n log C) time, O(C) space.

FULL CODE | main question first, then every follow-up, tests at the end

```
# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ========================================================

import heapq


# ========================================================
# MAIN QUESTION
# ========================================================
# ---- main question: main function ----
# Jobs become available at the times in `release_times`; every job takes
# `duration`; there are `num_cpus` identical CPUs. When is everything done
# (the makespan) if each job goes to the CPU that frees up first, but never
# starts before its release time?
# Idea: handle the jobs in release order and keep a min-heap of the times
# at which the CPUs become free (all 0 at the start).
def min_makespan(release_times, num_cpus, duration):
    free = [0] * num_cpus
    last = 0
    for release in sorted(release_times):
        cpu_free = heapq.heappop(free)             # the CPU that frees up first
        start = release
        if cpu_free > start:
            start = cpu_free
        heapq.heappush(free, start + duration)
        if start + duration > last:
            last = start + duration
    return last


# ---- main question: tests ----
def test_main_question():
    assert min_makespan([0, 1, 2, 3], 2, 3) == 7
    assert min_makespan([0, 0, 0], 1, 2) == 6
    assert min_makespan([5], 3, 4) == 9


# ========================================================
# FOLLOW-UP 1: The first idea tracked every CPU's availability in a list. Why is the heap better?
# ========================================================
# ---- follow-up 1: function ----
```

```
# Answer: with a list you have to scan all C CPUs for the earliest free one
# on every job: O(n * C). A min-heap gives the earliest free time in O(1),
# takes it out and puts the new time back in O(log C), so the total is
# O(n log C) after the sort.
def min_makespan_scan(release_times, num_cpus, duration):
    free = [0] * num_cpus
    for release in sorted(release_times):
        earliest = 0
        for i in range(num_cpus):                  # the scan the heap avoids
            if free[i] < free[earliest]:
                earliest = i
        start = release
```

```
        if free[earliest] > start:
            start = free[earliest]
        free[earliest] = start + duration
    if not release_times:
        return 0
    last = 0
    for t in free:
        if t > last:
            last = t
    return last


# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert min_makespan_scan([0, 1, 2, 3], 2, 3) == 7
    for jobs in ([0, 0, 0, 5, 9], [3, 1, 2], []):
        for cpus in (1, 2, 3):
            assert min_makespan_scan(jobs, cpus, 4) == min_makespan(jobs, cpus, 4)


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

35 Design server allocation for multi-type nodes

MEDIUM

Template fit: a heap per type of (flexibility, server); skip servers already allocated (lazy deletion)

QUESTION

Design a service that manages a pool of server nodes. Each node supports several instance types. getServer(count, types) hands out up to count servers that together cover all the requested types, preferring the least flexible servers (the ones that support the fewest types) so that versatile servers stay free, and returns None (allocating nothing) if the request cannot be met. releaseServer(ids) returns servers to the pool.

Function signature

class Servers:

def __init__(self, servers: dict)

id -> set of instance types

def getServer(self, count: int, types: list) -> list | None

def releaseServer(self, ids: list) -> None

EXAMPLES

QUESTION

Example 1

Input	`s = Servers({1: {'a'}, 2: {'a', 'b'}, 3: {'b'}, 4: {'a', 'b', 'c'}})`
	`s.getServer(2, ['a', 'b'])`
	`s.getServer(2, ['a', 'b'])`
	`s.getServer(1, ['a'])`
	`s.releaseServer([1])`
	`s.getServer(1, ['a'])`

Output	`s.getServer(2, ['a', 'b']) -> [1, 3]`
	`s.getServer(2, ['a', 'b']) -> [2, 4]`
	`s.getServer(1, ['a']) -> null`
	`s.getServer(1, ['a']) -> [1]`

Why	The first request takes the two least flexible servers (1 for a, 3 for b); the second takes 2 and 4; nothing is left for the third; after server 1 is released it can serve type a again.

FULL CODE | main question first, then every follow-up, tests at the end

```
# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ========================================================

import heapq


# ========================================================
# MAIN QUESTION
# ========================================================
# ---- main question: main class ----
# Servers support several instance types. getServer(count, types) must
# return `count` free servers that TOGETHER cover all the requested types
# (None if it cannot). To keep the versatile servers free for later, use the
# LEAST flexible suitable server first.
# Idea: for every type keep a heap of (number of types the server supports,
# server id) for the servers that support it: the least flexible is on top.
# A server that gets allocated is not removed from the other heaps at once
# ("lazy deletion"): it is skipped and dropped when it reaches the top.
# Pick the servers first and commit only if the whole request works.
class Servers:
    def __init__(self, servers):
        # servers: id -> set of instance types
        self.types = {}
        for server in servers:
            self.types[server] = set(servers[server])
        self.allocated = set()
        self.heaps = {}                                 # type -> heap of (flexibility, server id)
        self.lease = {}                                 # allocated server -> time its lease runs out (follow-
up 4)
        for server in self.types:
            for t in self.types[server]:
                if t not in self.heaps:
                    self.heaps[t] = []
                heapq.heappush(self.heaps[t], (len(self.types[server]), server))

    def best_for(self, t, skip):
        # the best free server of type t that is not in `skip` (None if there is none)
        heap = self.heaps.get(t, [])
        looked_past = []
        while heap:
            flexibility, server = heap[0]
```

```
            if server in self.allocated:
```

```
                heapq.heappop(heap)                    # lazy deletion
            elif server in skip:
                looked_past.append(heapq.heappop(heap))
            else:
                break
        best = None
        if heap:
            best = heap[0]
        for item in looked_past:
            heapq.heappush(heap, item)                 # put back what we only looked past
        return best

    def covers(self, chosen, t):
        # does one of the chosen servers already support type t?
        for server in chosen:
            if t in self.types[server]:
                return True
        return False

    def getServer(self, count, types):
        chosen = []
        for t in types:
            if self.covers(chosen, t):
                continue
            top = self.best_for(t, set(chosen))
            if top is None:
                return None
            chosen.append(top[1])
        while len(chosen) < count:                     # fill up with the least flexible free servers
            skip = set(chosen)
            options = []
            for t in types:
                found = self.best_for(t, skip)
                if found is not None:
                    options.append(found)
            if not options:
                return None
            chosen.append(min(options)[1])
        if len(chosen) > count:
            return None                                # more servers were needed to cover the types than all
owed
        for server in chosen:
            self.allocated.add(server)
        return chosen

    def releaseServer(self, ids):
```

```
        for server in ids:
            if server in self.allocated:
                self.allocated.discard(server)
                self.lease.pop(server, None)
                for t in self.types[server]:
                    heapq.heappush(self.heaps[t], (len(self.types[server]), server))

    # follow-up 4: a client that dies without releasing its servers
    def rent(self, count, types, now, ttl):
        got = self.getServer(count, types)
        if got:
            for server in got:
                self.lease[server] = now + ttl
        return got

    def expire(self, now):
        late = []
```

```
        for server in self.lease:
            if self.lease[server] <= now:
                late.append(server)
        self.releaseServer(late)
        return late


# ---- main question: tests ----
def test_main_question():
    pool = Servers({1: {"gpu"}, 2: {"gpu", "big"}, 3: {"big"}, 4: {"cpu"}})
    assert pool.getServer(2, ["gpu", "big"]) == [1, 3]        # the least flexible server of each type goes f
irst
    assert pool.getServer(1, ["gpu", "big"]) == [2]           # the versatile server 2 was still there
    assert pool.getServer(1, ["gpu"]) is None                 # nothing is left for a gpu request
    assert pool.getServer(1, ["cpu", "gpu"]) is None          # does not fit in one server: nothing changes
    assert pool.getServer(1, ["cpu"]) == [4]
    pool.releaseServer([2])
    assert pool.getServer(1, ["gpu"]) == [2]


# =========================================================
# FOLLOW-UP 1: Many clients call at the same time
# =========================================================
# Answer: protect the pool with one lock (as in the locker question), or
# shard it by instance type so requests for different types do not wait for each other.


# =========================================================
# FOLLOW-UP 2: Clients keep asking for rare types and starve
```

```
# =========================================================
# Answer: keep some servers of each rare type in reserve, or let a request
# wait in a queue per type so older requests are served first.


# =========================================================
# FOLLOW-UP 3: Servers fail while allocated
# =========================================================
# Answer: have the servers send heartbeats; when a server stops, treat it as
# released and tell its client (who can ask for a replacement).


# =========================================================
# FOLLOW-UP 4: A client dies without releasing its servers
# =========================================================
# Answer: give every allocation a LEASE (an expiry time). A periodic sweep
# returns the servers whose lease ran out to the free pool, and a live client
# renews its lease. rent() sets the lease and expire() returns the servers
# of dead clients to the pool.
# ---- follow-up 4: tests ----
def test_follow_up_4():
    pool = Servers({1: {"a"}})
    assert pool.rent(1, ["a"], now=0, ttl=10) == [1]
    assert pool.getServer(1, ["a"]) is None
    assert pool.expire(now=5) == []
    assert pool.expire(now=10) == [1]
    assert pool.getServer(1, ["a"]) == [1]


# =========================================================
# RUN EVERYTHING
# =========================================================
def run_all_tests():
    test_main_question()
    test_follow_up_4()
    print("all tests passed")
```

```
if __name__ == "__main__":
    run_all_tests()
```

36 Huffman Tree Construction — HARD

Template fit: pop the two smallest weights, push their sum as a new node; repeat until one node remains

QUESTION

Given symbol frequencies, build the Huffman code: repeatedly merge the two least frequent items into one until a single tree is left; the path from the root gives each symbol its bit string. Return the code of every symbol. To make the answer unique, the codes are assigned canonically: shorter codes first, then alphabetical order.

Function signature

def huffman_codes(freq: dict) -> dict

EXAMPLES

Example 1	
Input	{'a': 5, 'b': 9, 'c': 12, 'd': 13, 'e': 16, 'f': 45}
Output	{'f': '0', 'c': '100', 'd': '101', 'e': '110', 'a': '1110', 'b': '1111'}
Why	The frequent symbol f gets a 1-bit code; the rare ones get longer codes.

Example 2	
Input	{'x': 3, 'y': 8}
Output	{'x': '0', 'y': '1'}
Why	Two symbols always get one bit each.

Example 3	
Input	{'z': 10}
Output	{'z': '0'}
Why	A single symbol still needs a code of one bit.

FULL CODE | main question first, then every follow-up, tests at the end

```
# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

import heapq
from collections import deque


# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
def huffman_lengths(freq):
    # the code length of every symbol = its depth in the Huffman tree.
    # Idea: put every symbol in a min-heap by frequency; repeatedly take the
    # TWO lightest nodes and merge them under a new node whose weight is the
    # sum; every symbol below the merged node gets one level deeper.
    if not freq:
```

```
        return {}
    symbols = sorted(freq)
    if len(symbols) == 1:
        return {symbols[0]: 1}                      # a lone symbol still needs one bit
    heap = []
    for i in range(len(symbols)):
        heap.append((freq[symbols[i]], i, [symbols[i]]))   # (weight, tie-break, the symbols below)
    heapq.heapify(heap)
    depth = {}
    for symbol in symbols:
        depth[symbol] = 0
    tie = len(heap)
    while len(heap) > 1:
        weight1, t1, below1 = heapq.heappop(heap)
        weight2, t2, below2 = heapq.heappop(heap)
        for symbol in below1 + below2:
            depth[symbol] += 1
        heapq.heappush(heap, (weight1 + weight2, tie, below1 + below2))
        tie += 1
    return depth


def to_binary(number, length):
    # the number written in binary with exactly `length` digits
```

```
    digits = ""
    for i in range(length):
        digits = str(number % 2) + digits
        number = number // 2
    return digits


def canonical_codes(lengths):
    # CANONICAL codes (follow-up 2 and 4): sort the symbols by (length,
    # symbol) and give them consecutive binary numbers, shifting left when
    # the length grows. A decoder can rebuild all codes from the lengths alone
    # (this is how DEFLATE stores its tables).
    ordered = []
    for symbol in lengths:
        ordered.append((lengths[symbol], symbol))
    ordered.sort()
    code = 0
    previous_length = 0
    codes = {}
    for length, symbol in ordered:
        code = code * 2 ** (length - previous_length)   # a longer length shifts the code left
        codes[symbol] = to_binary(code, length)
        code += 1
        previous_length = length
    return codes


# ---- main question: main function ----
# Idea: the lengths come from the min-heap merging above; the actual
# codes are then assigned in the canonical way.
def huffman_codes(freq):
    return canonical_codes(huffman_lengths(freq))


# ---- main question: tests ----
FREQ = {"a": 5, "b": 9, "c": 12, "d": 13, "e": 16, "f": 45}


def decode(bits, codes):
    # read bits until they match a code (the codes are prefix-free)
    symbol_of = {}
```

```
    for symbol in codes:
        symbol_of[codes[symbol]] = symbol
    text = ""
    current = ""
```

```
    for bit in bits:
        current += bit
        if current in symbol_of:
            text += symbol_of[current]
            current = ""
    return text


def total_cost(freq, lengths):
    cost = 0
    for symbol in freq:
        cost += freq[symbol] * lengths[symbol]
    return cost


def test_main_question():
    lengths = huffman_lengths(FREQ)
    assert lengths["f"] == 1                        # the most frequent symbol is next to the root
    assert total_cost(FREQ, lengths) == 224         # the cost of the optimal tree
    codes = huffman_codes(FREQ)
    for x in codes.values():
        for y in codes.values():
            assert x == y or not y.startswith(x)    # prefix-free: no code starts another code
    bits = ""
    for ch in "abfadc":
        bits += codes[ch]
    assert decode(bits, codes) == "abfadc"
    assert huffman_codes({"x": 3}) == {"x": "0"}
    assert huffman_codes({}) == {}


# =======================================================
# FOLLOW-UP 1: Why is the greedy choice optimal?
# =======================================================
# Answer: the two LIGHTEST symbols can always be siblings at the deepest
# level of some optimal tree (swapping a heavier symbol with a lighter one
# at a shallower level never makes the cost bigger). Merging them leaves a
# smaller problem of the same kind, so repeating the choice is optimal.


# =======================================================
# FOLLOW-UP 2: How do you make the code canonical, and why would you?
# =======================================================
```

```
# =======================================================
# FOLLOW-UP 4: Make the codes canonical so only the code lengths need to be stored
# =======================================================
# Answer: canonical_codes above: keep only each symbol's code LENGTH, sort
# the symbols by (length, symbol) and give consecutive binary numbers. The
# decoder needs just the lengths instead of the whole tree.


# =======================================================
# FOLLOW-UP 3: Time complexity if symbols arrive already sorted?
# =======================================================
# ---- follow-up 3: function ----
# Answer: use TWO queues: one holds the sorted leaves, the other the merged
# nodes (which are created in non-decreasing weight order). Always take the
# smaller front of the two queues. That is O(n) after the sort.
def lengths_two_queues(sorted_items):
```

```
    leaves = deque()
    depth = {}
    for symbol, weight in sorted_items:
        leaves.append((weight, [symbol]))
        depth[symbol] = 0
    merged = deque()

    def take_smaller():
        if not merged or (leaves and leaves[0][0] <= merged[0][0]):
            return leaves.popleft()
        return merged.popleft()

    while len(leaves) + len(merged) > 1:
        weight1, below1 = take_smaller()
        weight2, below2 = take_smaller()
        for symbol in below1 + below2:
            depth[symbol] += 1
        merged.append((weight1 + weight2, below1 + below2))
    return depth


# ---- follow-up 3: tests ----
def test_follow_up_3():
    items = sorted(FREQ.items(), key=lambda pair: pair[1])      # the symbols ordered by weight
    assert total_cost(FREQ, lengths_two_queues(items)) == 224


# =============================================================
# RUN EVERYTHING
```

```
# =============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

Recurrence on a position or a range (memo or table)

37 Target Expressions from Ordered Digits

MEDIUM

*Template fit: best(l, r) = every value reachable from digits l..r; split at m and combine the two sides with +, - and **

QUESTION

Given a list of digits in order and a target number, put +, - or * between the digits (never reorder them and never glue two digits into one number) and add parentheses wherever you like. Return every distinct expression that evaluates to the target.

Function signature

def expressions_for_target(digits: list, target: int) -> list

EXAMPLES

Example 1	
Input	`[2, 3, 4], 14`
Output	`['2*(3+4)', '2+3*4']`
Why	`2 * (3 + 4) = 14 and 2 + 3 * 4 = 14 are the only two ways.`

Example 2	
Input	`[1, 2, 3], 6`
Output	`['1*2*3', '1+2+3']`
Why	`Three digits can reach 6 several ways.`

Example 3	
Input	`[1, 2], 100`
Output	`[]`
Why	`No way to reach 100 with these digits.`

FULL CODE | main question first, then every follow-up, tests at the end

```
# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
PRECEDENCE = {"+": 1, "-": 1, "*": 2}
ATOM = 3                                  # a single digit never needs brackets


def needs_brackets(op, child_precedence, is_right_child):
    # a child that binds weaker than the operator needs brackets; on the RIGHT
    # of a minus even an equal one does: 5-(3-1) is not 5-3-1
    if child_precedence < PRECEDENCE[op]:
        return True
    if is_right_child and op == "-" and child_precedence == PRECEDENCE[op]:
        return True
    return False
```

```
def join(left_text, left_precedence, op, right_text, right_precedence):
    if needs_brackets(op, left_precedence, False):
        left_text = "(" + left_text + ")"
    if needs_brackets(op, right_precedence, True):
        right_text = "(" + right_text + ")"
    return left_text + op + right_text


def expressions_over(digits, left, right, memo):
    # every value that digits[left:right] can make, with every expression that makes it:
    # {value: {(text, precedence), ...}}
    if (left, right) in memo:
        return memo[(left, right)]
    result = {}
    if right - left == 1:
        result[digits[left]] = {(str(digits[left]), ATOM)}
    for middle in range(left + 1, right):                    # try every split point
        lefts = expressions_over(digits, left, middle, memo)
        rights = expressions_over(digits, middle, right, memo)
        for left_value in lefts:
```

```
            for right_value in rights:
                results_of = {"+": left_value + right_value,
                              "-": left_value - right_value,
                              "*": left_value * right_value}
                for op in results_of:
                    value = results_of[op]
                    if value not in result:
                        result[value] = set()
                    for left_text, left_precedence in lefts[left_value]:
                        for right_text, right_precedence in rights[right_value]:
                            text = join(left_text, left_precedence, op, right_text, right_precedence)
                            result[value].add((text, PRECEDENCE[op]))
    memo[(left, right)] = result
    return result


# ---- main question: main function ----
# Idea (interval DP): an expression over digits[l:r] is two smaller
# expressions over a split point joined by +, - or *. For every range
# remember, per value, the set of expressions that make it. Brackets are
# added only where precedence needs them.
def expressions_for_target(digits, target):
    table = expressions_over(digits, 0, len(digits), {})
    texts = set()
    if target in table:
        for text, precedence in table[target]:
            texts.add(text)
    return sorted(texts)


# ---- main question: tests ----
def test_main_question():
    answer = expressions_for_target([2, 3, 4], 14)
    assert "2*(3+4)" in answer and "2+3*4" in answer
    assert "5-(3-1)" in expressions_for_target([5, 3, 1], 3)        # subtraction needs brackets on the right
    assert expressions_for_target([2, 3, 4], 100) == []


# ========================================================
# FOLLOW-UP 1: How do you handle N digits where listing every expression is too much?
# ========================================================
# Answer: the number of expressions grows far faster than the number of
# values. Ask whether a yes/no, a count or one example is enough, and keep
```

```
# only the SET OF VALUES per range (follow-up 3).
```

```
# ============================================================
# FOLLOW-UP 2: What if concatenating digits (23) or division is allowed?
# ============================================================
# Answer: concatenation adds more "leaf" values to each range (the number
# written by digits[l:r] itself); division adds one more operator, but
# skip it when the divisor is 0 and decide how to treat remainders.


# ============================================================
# FOLLOW-UP 3: Only a yes/no answer is needed for many digits
# ============================================================
# ---- follow-up 3: function ----
# Answer: store just the set of reachable values per range instead of
# the expression strings; strings blow up long before the value sets do.
def target_reachable(digits, target):
    values = {}

    def reachable(left, right):
        if (left, right) in values:
            return values[(left, right)]
        found = set()
        if right - left == 1:
            found.add(digits[left])
        for middle in range(left + 1, right):
            for a in reachable(left, middle):
                for b in reachable(middle, right):
                    found.add(a + b)
                    found.add(a - b)
                    found.add(a * b)
        values[(left, right)] = found
        return found

    return target in reachable(0, len(digits))


# ---- follow-up 3: tests ----
def test_follow_up_3():
    assert target_reachable([2, 3, 4], 14)
    assert not target_reachable([2, 3, 4], 100)
    assert target_reachable([1] * 8, 8)


# ============================================================
# RUN EVERYTHING
```

```
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

38 Count Word Decompositions into Chemical Element Symbols

MEDIUM

Template fit: bottom-up ways[i]: add ways[i+1] if the one-letter symbol exists and ways[i+2] if the two-letter symbol exists

QUESTION

Chemical elements have symbols of one or two letters (H, He, Si, Cs ...). Given a word, count the ways to write it as a sequence of element symbols (ignore upper and lower case). Different splits count as different ways even if they spell the same word.

Function signature

def count_decompositions(word: str) -> int

EXAMPLES

Example 1	
Input	'Physics'
Output	4
Why	P-H-Y-Si-C-S, P-H-Y-Si-Cs, P-H-Y-S-I-C-S and P-H-Y-S-I-Cs: 4 ways.

Example 2	
Input	'Ho'
Output	2
Why	'Ho' is one symbol (holmium) and also H + O: 2 ways.

Example 3	
Input	'xqz'
Output	0
Why	No split works (x, y... cannot all be symbols).

FULL CODE | main question first, then every follow-up, tests at the end

```
# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ========================================================

# ========================================================
# MAIN QUESTION
# ========================================================
# ---- main question: helper functions ----
ELEMENTS = set("""h he li be b c n o f ne na mg al si
p s cl ar k ca sc ti v cr mn fe co ni cu zn ga ge as
se br kr rb sr y zr nb mo tc ru rh pd ag cd in sn sb
te i xe cs ba la ce pr nd pm sm eu gd tb dy ho er tm
yb lu hf ta w re os ir pt au hg tl pb bi po at rn fr
ra ac th pa u np pu am cm bk cf es fm md no lr rf db
sg bh hs mt ds rg cn nh fl mc lv ts og""".split())


# ---- main question: main function ----
# Idea: ways[i] = the number of ways to split the END of the word, from
# position i on. A symbol has one or two letters, so
# ways[i] = ways[i + 1] (if word[i] is a symbol)
#         + ways[i + 2] (if word[i:i + 2] is a symbol).
# An empty ending has exactly one way. `mod` is for follow-up 5.
def count_decompositions(word, symbols=ELEMENTS, mod=None):
    w = word.lower()
    n = len(w)
```

```
    ways = [0] * (n + 2)
    ways[n] = 1
    for i in range(n - 1, -1, -1):
        if w[i] in symbols:
            ways[i] += ways[i + 1]
        if i + 2 <= n and w[i:i + 2] in symbols:
            ways[i] += ways[i + 2]
        if mod:
            ways[i] = ways[i] % mod
    return ways[0]


# ---- main question: tests ----
def test_main_question():
    assert count_decompositions("Physics") == 4
    assert count_decompositions("") == 1
```

```
# ============================================================
# FOLLOW-UP 1: How would you list the decompositions, not just count them?
# ============================================================


# ============================================================
# FOLLOW-UP 3: List the decompositions instead of counting them
# ============================================================
# ---- follow-up 1: function ----
# Answer: the same recursion, but build the list of parts chosen. The
# number of results can be exponential, so ask whether a count or a list
# is wanted.
def list_decompositions(word, symbols=ELEMENTS):
    w = word.lower()
    n = len(w)
    memo = {}

    def splits(i):
        # every way to split the ending of the word that starts at i
        if i == n:
            return [[]]
        if i in memo:
            return memo[i]
        result = []
        for size in (1, 2):
            part = w[i:i + size]
            if len(part) == size and part in symbols:
                for rest in splits(i + size):
                    result.append([part] + rest)
        memo[i] = result
        return result

    return splits(0)


# ---- follow-up 1: tests ----
def test_follow_up_1():
    parts = list_decompositions("Physics")
    assert len(parts) == 4
    assert ["p", "h", "y", "si", "c", "s"] in parts


# ============================================================
```

```
# FOLLOW-UP 2: What if the dictionary is large and symbols can be long?
# ============================================================
```

```python
# ============================================================
# FOLLOW-UP 4: A larger dictionary with long entries
# ============================================================
# ---- follow-up 2: function ----
# Answer: keep the dictionary in a set and remember the length L of
# the LONGEST entry, so each position looks only at pieces up to L
# letters (or use a trie): O(L) lookups per position instead of O(n).
def count_with_max_len(word, symbols):
    longest = 0
    for symbol in symbols:
        if len(symbol) > longest:
            longest = len(symbol)
    w = word.lower()
    n = len(w)
    ways = [0] * (n + 1)
    ways[n] = 1
    for i in range(n - 1, -1, -1):
        for size in range(1, longest + 1):
            if i + size <= n and w[i:i + size] in symbols:
                ways[i] += ways[i + size]
    return ways[0]


# ---- follow-up 2: tests ----
def test_follow_up_2():
    assert count_with_max_len("Physics", ELEMENTS) == 4


# ============================================================
# FOLLOW-UP 5: Count modulo a prime, or only test whether any decomposition exists
# ============================================================
# ---- follow-up 5: function ----
# Answer: replace the addition by addition modulo the prime (the `mod`
# parameter above), or by "or" for a yes/no answer. The structure is the same.
def exists_decomposition(word, symbols=ELEMENTS):
    return count_decompositions(word, symbols) > 0


# ---- follow-up 5: tests ----
def test_follow_up_5():
    assert exists_decomposition("Physics")
```

```python
    assert not exists_decomposition("xq")                    # no element is "x" or "xq"
    assert count_decompositions("Physics", mod=3) == 1       # 4 % 3


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    test_follow_up_5()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

39 Find the longest palindromic subsequence using dynamic programming. MEDIUM

Template fit: table over (i, j): equal ends add 2 to the inside; otherwise the better of dropping either end

QUESTION

Given a string s, return the length of its longest palindromic subsequence: the longest sequence of letters of s, taken in order but not necessarily next to each other, that reads the same forwards and backwards.

Function signature

def lps_length(s: str) -> int

EXAMPLES

Example 1	
Input	'bbbab'
Output	4
Why	'bbbb' is a subsequence of 'bbbab' and a palindrome.

Example 2	
Input	'cbbd'
Output	2
Why	'bb' (the two b's) is the longest.

Example 3	
Input	'a'
Output	1
Why	A single letter.

FULL CODE | main question first, then every follow-up, tests at the end

```
# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

import random


# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
def lps_table(s):
    # dp[i][j] = the length of the longest palindromic SUBSEQUENCE of s[i..j].
    # Equal end letters are both used (+2); otherwise drop one end.
    n = len(s)
    dp = []
    for i in range(n):
        dp.append([0] * n)
    for i in range(n - 1, -1, -1):
        dp[i][i] = 1
        for j in range(i + 1, n):
            if s[i] == s[j]:
                dp[i][j] = dp[i + 1][j - 1] + 2
            elif dp[i + 1][j] >= dp[i][j - 1]:
                dp[i][j] = dp[i + 1][j]
            else:
                dp[i][j] = dp[i][j - 1]
```

```
    return dp


# ---- main question: main function ----
# Idea (interval DP): dp[i][j] is the longest palindromic subsequence of
# s[i..j]. Fill i from the end of the string to the start and j from i + 1
# up. Answer: dp[0][n - 1].
def lps_length(s):
    if not s:
        return 0
    return lps_table(s)[0][len(s) - 1]


# ---- main question: tests ----
def test_main_question():
```

```
    assert lps_length("bbbab") == 4
    assert lps_length("cbbd") == 2
    assert lps_length("") == 0


# ========================================================
# FOLLOW-UP 1: Return the subsequence itself
# ========================================================
# ---- follow-up 1: function ----
# Answer: keep the whole table and walk from (0, n - 1): equal end letters
# are both taken (they go into the answer); otherwise move towards the side
# with the larger value. The walk back needs O(n^2) memory.
def lps_string(s):
    if not s:
        return ""
    dp = lps_table(s)
    i = 0
    j = len(s) - 1
    left_half = ""
    middle = ""
    while i <= j:
        if i == j:
            middle = s[i]
            break
        if s[i] == s[j]:
            left_half += s[i]
            i += 1
            j -= 1
        elif dp[i + 1][j] >= dp[i][j - 1]:
            i += 1
        else:
            j -= 1
    return left_half + middle + left_half[::-1]


# ---- follow-up 1: tests ----
def is_subsequence(small, big):
    # do the letters of `small` appear in `big` in the same order?
    position = 0
    for ch in big:
        if position < len(small) and ch == small[position]:
            position += 1
    return position == len(small)
```

```
def test_follow_up_1():
    assert lps_string("bbbab") == "bbbb"
    assert lps_string("a") == "a"
    random.seed(4)
    for round_number in range(300):
```

```
        s = ""
        for i in range(random.randint(0, 10)):
            s += random.choice("abc")
        found = lps_string(s)
        assert is_subsequence(found, s)                    # the answer is really a subsequence of s
        assert found == found[::-1]                        # and a palindrome
        assert len(found) == lps_length(s)


# ========================================================
# FOLLOW-UP 2: Minimum number of insertions (or deletions) to make the string a palindrome
# ========================================================
# ---- follow-up 2: function ----
# Answer: keep the longest palindromic subsequence and fix everything
# outside it: n - LPS(s) insertions (or deletions).
def min_insertions(s):
    return len(s) - lps_length(s)


# ---- follow-up 2: tests ----
def test_follow_up_2():
    assert min_insertions("abca") == 1
    assert min_insertions("race") == 3
    assert min_insertions("aba") == 0


# ========================================================
# FOLLOW-UP 3: Longest palindromic substring instead (contiguous)
# ========================================================
# ---- follow-up 3: function ----
# Answer: expand around each of the 2n - 1 centres (a letter, or the gap
# between two letters): O(n^2). Manacher's algorithm does it in O(n). No
# subsequence DP is needed.
def longest_palindromic_substring(s):
    best = ""
    for centre in range(2 * len(s) - 1):
        i = centre // 2
        j = i + centre % 2
        while i >= 0 and j < len(s) and s[i] == s[j]:
            i -= 1
```

```
            j += 1
        if j - i - 1 > len(best):
            best = s[i + 1:j]
    return best


# ---- follow-up 3: tests ----
def test_follow_up_3():
    assert longest_palindromic_substring("abacc") == "aba"
    assert longest_palindromic_substring("cbbd") == "bb"
    assert longest_palindromic_substring("") == ""


# ========================================================
# RUN EVERYTHING
# ========================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
```

```
    run_all_tests()
```

40 Jump Game with Profit — MEDIUM

Template fit: best[i] = max over every j before i that can reach i (j + arr[j] >= i) of best[j] + profit[i]

QUESTION

You stand at index 0 of an array. From index i you may jump to any index from i + 1 up to i + arr[i]. Landing on index i earns profit[i] (the start earns profit[0] too). Return the largest total profit of a path that ends exactly at the last index, or -1 if the last index cannot be reached.

Function signature

def max_profit_fast(arr: list, profit: list) -> int

EXAMPLES

Example 1	
Input	`[2, 3, 1, 1, 4], [1, 5, 2, 3, 4]`
Output	`15`
Why	`Jumping one index at a time visits (and collects) everything: 1 + 5 + 2 + 3 + 4 = 15.`

Example 2	
Input	`[1, 0, 1], [1, 1, 1]`
Output	`-1`
Why	`From index 1 you cannot jump at all (arr[1] = 0), so the last index is unreachable.`

FULL CODE | main question first, then every follow-up, tests at the end

```
# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

import heapq

NEG = float("-inf")


# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: main function ----
# Reading A: arr[i] is the farthest you may jump from cell i, profit[i]
# is gained when you land on i; maximise the total profit on the way to
# the last cell (-1 if it cannot be reached).
# Idea: best[i] = the largest profit possible when standing on i. You can
# land on i from any earlier cell j that reaches it (j + arr[j] >= i):
# best[i] = profit[i] + the largest best[j] among those. Keep the
# candidates in a MAX-heap (push negatives) and drop a candidate for good
# as soon as it can no longer reach the current cell.
def max_profit_fast(arr, profit):
    n = len(arr)
    best = [NEG] * n
    best[0] = profit[0]
    heap = [(-best[0], arr[0])]                 # (minus the profit, how far it reaches)
    for i in range(1, n):
        while heap and heap[0][1] < i:
            heapq.heappop(heap)                 # this cell can no longer reach i or anything after
```

```
        if heap:
            best[i] = -heap[0][0] + profit[i]
            heapq.heappush(heap, (-best[i], i + arr[i]))
    if best[n - 1] == NEG:
        return -1
    return best[n - 1]


# ---- main question: tests ----
def test_main_question():
    assert max_profit_fast([2, 3, 1, 1, 4], [1, 5, 2, 3, 4]) == 15
    assert max_profit_fast([1, 0, 1], [1, 1, 1]) == -1
```

```
# ============================================================
# FOLLOW-UP 1: Speed up reading A to near-linear time
# ============================================================


# ============================================================
# FOLLOW-UP 4: Make the profit DP faster
# ============================================================
# Answer: the heap above is the speed-up: instead of looking at every
# earlier cell (O(n^2)), the best usable candidate is always at the top of
# the heap, so the whole thing is O(n log n).


# ============================================================
# FOLLOW-UP 2: What if you want to know whether the end is reachable at all (plain Jump Game)?
# ============================================================


# ============================================================
# FOLLOW-UP 3: Only reachability, no profit (LeetCode 55)
# ============================================================
# ---- follow-up 3: function ----
# Answer: greedy. Keep the farthest index reachable so far; if the
# current index is beyond it, you are stuck. O(n) time, O(1) space.
def can_reach_end(arr):
    farthest = 0
    for i in range(len(arr)):
        if i > farthest:
            return False
        if i + arr[i] > farthest:
            farthest = i + arr[i]
    return True


# ---- follow-up 3: tests ----
def test_follow_up_3():
    assert can_reach_end([2, 3, 1, 1, 4])
    assert not can_reach_end([1, 0, 1])


# ============================================================
# FOLLOW-UP 5: Fewest jumps (LeetCode 45)
# ============================================================
# ---- follow-up 5: function ----
```

```
# Answer: a BFS by levels, done greedily: track where the current jump
# can end (`end`) and how far any cell seen so far can reach
# (`farthest`). When we get to `end` we must jump: count it and extend
# `end` to `farthest`. O(n).
def fewest_jumps(arr):
    jumps = 0
    end = 0
```

```
    farthest = 0
    for i in range(len(arr) - 1):
        if i + arr[i] > farthest:
            farthest = i + arr[i]
        if i == end:
            jumps += 1
            end = farthest
    return jumps


# ---- follow-up 5: tests ----
def test_follow_up_5():
    assert fewest_jumps([2, 3, 1, 1, 4]) == 2
    assert fewest_jumps([0]) == 0


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_3()
    test_follow_up_5()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

41 Fence Painting Minimum Operations (Gemini Team) MEDIUM

Template fit: range (l, r): paint one stripe at the lowest height, then solve the pieces on both sides; a sparse table finds the lowest point fast

QUESTION

A fence has n planks, plank i is a[i] units tall. One stroke is either horizontal (paints one row of height h across a run of neighbouring planks that are all at least h tall) or vertical (paints one plank completely, bottom to top). Return the smallest number of strokes needed to paint the whole fence.

Function signature

def paint(a: list) -> int

EXAMPLES

Example 1	
Input	`[2, 2, 1, 2, 1]`
Output	`3`
Why	`Row 1 across all five planks, row 2 across the first two planks, and one vertical stroke on plank 4: 3 strokes.`

Example 2	
Input	`[1, 1, 1]`
Output	`1`
Why	`One horizontal stroke of height 1 across all three planks paints the whole fence.`

Example 3	
Input	`[5]`
Output	`1`
Why	`One tall plank: a single vertical stroke beats five horizontal ones.`

FULL CODE | main question first, then every follow-up, tests at the end

```
# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ========================================================

# ========================================================
# MAIN QUESTION
# ========================================================
# ---- main question: helper functions ----
def sparse_table(a):
    # table[j][i] = the index of the smallest plank among a[i : i + 2**j].
    # Built once, it answers "where is the smallest plank in a[l..r]?" in O(1).
    table = [list(range(len(a)))]
    j = 1
    while (1 << j) <= len(a):
        previous = table[j - 1]
        row = []
        for i in range(len(a) - (1 << j) + 1):
            x = previous[i]
            y = previous[i + (1 << (j - 1))]
            if a[x] <= a[y]:
                row.append(x)
            else:
                row.append(y)
        table.append(row)
        j += 1
    return table


def smallest_index(table, a, left, right):
    # the index of the smallest plank in a[left..right]: two overlapping blocks cover the range
    k = 0
    while (1 << (k + 1)) <= right - left + 1:
        k += 1
    x = table[k][left]
    y = table[k][right - (1 << k) + 1]
    if a[x] <= a[y]:
        return x
    return y


def paint_strokes(a, width=None):
    # returns (the number of strokes, the list of strokes). `width` is the
```

```
    # width of the horizontal brush in planks (None = as wide as you like).
    if len(a) == 0:
        return 0, []
    table = sparse_table(a)

    def solve(left, right, base):
        # paint planks left..right, whose lowest `base` rows are already painted
        if left > right:
            return 0, []
        low = smallest_index(table, a, left, right)
        size = right - left + 1
        per_row = 1
        if width is not None:
            per_row = (size + width - 1) // width          # a narrow brush needs several strokes per row
        band = (a[low] - base) * per_row                   # horizontal strokes across the whole range
        left_count, left_strokes = solve(left, low - 1, a[low])
        right_count, right_strokes = solve(low + 1, right, a[low])
        horizontal = band + left_count + right_count
        if size <= horizontal:                             # painting every plank upright is cheaper
```

```
            strokes = []
            for i in range(left, right + 1):
                strokes.append(("vertical", i))
            return size, strokes
        strokes = []
        for row in range(base + 1, a[low] + 1):
            strokes.append(("horizontal", row, left, right))
        return horizontal, strokes + left_strokes + right_strokes

    return solve(0, len(a) - 1, 0)


# ---- main question: main function ----
# Idea: look at the SHORTEST plank in a range. Either paint the rows up
# to its height across the whole range (one stroke per row) and then
# solve the two sides on top of that, or paint every plank of the range
# upright (one stroke per plank): take the cheaper. The sparse table
# finds the shortest plank in O(1) and every plank is the split point at
# most once, so only n ranges are visited.
def paint(a, width=None):
    return paint_strokes(a, width)[0]


# ---- main question: tests ----
def test_main_question():
    assert paint([2, 2, 1, 2, 1]) == 3
```

```
    assert paint([]) == 0
    assert paint([3]) == 1
    assert paint([4, 4, 4, 4]) == 4                     # a flat fence: one stroke per row
    assert paint([5, 0, 5]) == 2                        # tall thin planks are cheaper painted upright


# ======================================================
# FOLLOW-UP 1: The horizontal brush is only W planks wide
# ======================================================
# Answer: the `width` parameter: a horizontal band over a range of
# `size` planks costs ceil(size / width) strokes per row.
# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert paint([2, 2, 2, 2], width=2) == 4
    assert paint([2, 2, 2, 2], width=4) == 2


# ======================================================
# FOLLOW-UP 2: Return the strokes, not just the count
# ======================================================
# Answer: paint_strokes records which option won in each range and returns
# the strokes: horizontal bands as ("horizontal", row, left, right) and
# upright strokes as ("vertical", plank).
# ---- follow-up 2: tests ----
def test_follow_up_2():
    count, strokes = paint_strokes([2, 2, 1, 2, 1])
    assert count == len(strokes) == 3


# ======================================================
# FOLLOW-UP 3: Is it the same as largest rectangle in a histogram?
# ======================================================
# Answer: it is related but not the same. Both split at the shortest bar,
# but painting chooses between a horizontal band and upright strokes for each
# range, while the histogram problem maximises one rectangle's area.


# ======================================================
# RUN EVERYTHING
```

```
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    print("all tests passed")
```

```
if __name__ == "__main__":
    run_all_tests()
```

42 Maximize 1D deliveries within distance and minimize total distance — MEDIUM

Template fit: table over (orders used, drivers used): skip an order, skip a driver, or match them when they are within D (count, then minus distance)

QUESTION

Orders and drivers are points on a line. A driver can deliver an order that is at most D away; every driver takes at most one order and every order is delivered by at most one driver. First maximize the number of deliveries; among all plans with that many deliveries, minimize the total distance driven. Return (number of deliveries, total distance).

Function signature

def deliveries(orders: list, drivers: list, D: int) -> tuple

EXAMPLES

Example 1	
Input	`[1, 5, 9], [2, 6, 20], 2`
Output	`(2, 2)`
Why	`1 is served by the driver at 2, and 5 by the driver at 6: 2 deliveries with total distance 1 + 1 = 2.`

Example 2	
Input	`[1, 5, 9], [2, 6, 20], 11`
Output	`(3, 13)`
Why	`With D = 11 the driver at 20 can also take the order at 9 (distance 11): 1 + 1 + 11 = 13 for 3 deliveries.`

Example 3	
Input	`[3, 4], [], 5`
Output	`(0, 0)`
Why	`No drivers, no deliveries.`

FULL CODE | main question first, then every follow-up, tests at the end

```
# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
```

```python
def better(x, y):
    # the better of two (deliveries, minus distance) pairs: more deliveries
    # first, then less distance (a bigger "minus distance")
    if x >= y:
        return x
    return y


# ---- main question: main function ----
# Idea: sort the orders and the drivers (positions on a line). Let
# dp[i][j] be the best pair (number of deliveries, MINUS the total
# distance) using only the first i orders and the first j drivers. For
# the next order and driver you can skip the order, skip the driver, or
# match them if they are at most D apart. Pairs compare lexicographically,
# so more deliveries wins and then the smaller total distance.
def deliveries(orders, drivers, D):
    o = sorted(orders)
    d = sorted(drivers)
    dp = []
    for i in range(len(o) + 1):
        dp.append([(0, 0)] * (len(d) + 1))
    for i in range(1, len(o) + 1):
        for j in range(1, len(d) + 1):
            best = better(dp[i - 1][j], dp[i][j - 1])          # skip the order, or skip the driver
            gap = o[i - 1] - d[j - 1]
            if gap < 0:
                gap = -gap
            if gap <= D:                                       # or match them
                count, minus_distance = dp[i - 1][j - 1]
                best = better(best, (count + 1, minus_distance - gap))
            dp[i][j] = best
    count, minus_distance = dp[len(o)][len(d)]
    return count, -minus_distance
```

```python
# ---- main question: tests ----
def test_main_question():
    assert deliveries([1, 5], [2, 6], 1) == (2, 2)
    assert deliveries([1, 3], [2, 4], 1) == (2, 2)
    assert deliveries([], [1], 5) == (0, 0)
    assert deliveries([1, 2], [2, 3], 0) == (1, 0)            # a limit of 0 needs identical positions


# ======================================================
# FOLLOW-UP 1: Only the number of deliveries matters
# ======================================================
# ---- follow-up 1: function ----
# Answer: two pointers on the sorted lists: match when the two are
# within D, otherwise move on the one that is further left. O(n log n).
# It maximises the count but not the total distance.
def max_count(orders, drivers, D):
    o = sorted(orders)
    d = sorted(drivers)
    i = 0
    j = 0
    count = 0
    while i < len(o) and j < len(d):
        gap = o[i] - d[j]
        if gap < 0:
            gap = -gap
        if gap <= D:
            count += 1
            i += 1
            j += 1
        elif o[i] < d[j]:
            i += 1                                             # this order is too far left for every driver
```

```
        else:
            j += 1
    return count


# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert max_count([1, 3], [2, 4], 1) == 2
    assert max_count([1, 10], [5], 2) == 0


# ============================================================
# FOLLOW-UP 2: A driver can take several orders (capacity c)
```

```
# ============================================================
# ---- follow-up 2: function ----
# Answer: list every driver `cap` times (as separate points) and use the
# same DP; or model it as min-cost flow with capacities.
def with_capacity(orders, drivers, D, cap):
    repeated = []
    for driver in drivers:
        for copy in range(cap):
            repeated.append(driver)
    return deliveries(orders, repeated, D)


# ---- follow-up 2: tests ----
def test_follow_up_2():
    assert with_capacity([1, 2, 3], [2], 1, 3) == (3, 2)


# ============================================================
# FOLLOW-UP 3: Minimise the largest single distance instead of the total
# ============================================================
# ---- follow-up 3: function ----
# Answer: binary search the largest allowed distance D: "can everyone
# who can be served be served with limit D?" is false for small D and true
# from some D on, and the two-pointer count answers it.
def smallest_limit(orders, drivers):
    target = min(len(orders), len(drivers))
    low = 0
    high = 10 ** 9
    while low < high:
        mid = (low + high) // 2
        if max_count(orders, drivers, mid) >= target:
            high = mid
        else:
            low = mid + 1
    return low


# ---- follow-up 3: tests ----
def test_follow_up_3():
    assert smallest_limit([1, 5], [2, 9]) == 4


# ============================================================
# RUN EVERYTHING
# ============================================================
```

```
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    test_follow_up_3()
    print("all tests passed")
```

```
if __name__ == "__main__":
    run_all_tests()
```

43 Edit distance — HARD

Template fit: table over two strings, two rows: equal letters copy the diagonal, otherwise 1 + the smallest of the three neighbours

QUESTION

Given two words a and b, return the edit distance: the smallest number of single-letter operations (insert a letter, delete a letter, or replace a letter) that turn a into b.

Function signature

def edit_distance(a: str, b: str) -> (int, list)

(the distance, the whole DP

table)

EXAMPLES

Example 1	
Input	`a = 'horse', b = 'ros'`
Output	`edit_distance('horse', 'ros')[0] -> 3`
Why	`horse -> rorse (replace h) -> rose (delete r) -> ros (delete e): 3 operations.`

Example 2	
Input	`a = 'intention', b = 'execution'`
Output	`edit_distance('intention', 'execution')[0] -> 5`
Why	`intention -> execution takes 5 operations.`

Example 3	
Input	`a = '', b = 'abc'`
Output	`edit_distance('', 'abc')[0] -> 3`
Why	`From the empty word, insert every letter.`

FULL CODE | main question first, then every follow-up, tests at the end

```
# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

from collections import deque


# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
def smallest(x, y, z):
    result = x
    if y < result:
        result = y
    if z < result:
        result = z
```

```
    return result


# ---- main question: main function ----
# Idea: dp[i][j] = the fewest edits that turn the first i letters of a
# into the first j letters of b. With equal last letters it is
# dp[i-1][j-1]; otherwise 1 + the best of replace (dp[i-1][j-1]),
# delete (dp[i-1][j]) and insert (dp[i][j-1]). The first row and column
# are plain deletes / inserts. Returns (the distance, the whole table).
def edit_distance(a, b):
    m = len(a)
    n = len(b)
    dp = []
    for i in range(m + 1):
        dp.append([0] * (n + 1))
    for i in range(m + 1):
        dp[i][0] = i                                # delete i letters
    for j in range(n + 1):
        dp[0][j] = j                                # insert j letters
    for i in range(1, m + 1):
        for j in range(1, n + 1):
            if a[i - 1] == b[j - 1]:
                dp[i][j] = dp[i - 1][j - 1]         # same letter: no edit
            else:
                dp[i][j] = 1 + smallest(dp[i - 1][j - 1], dp[i - 1][j], dp[i][j - 1])
```

```
    return dp[m][n], dp


# RELATED: are the words at most k edits apart?
def within_k(a, b, k):
    if abs(len(a) - len(b)) > k:
        return False                                # lengths alone need more than k edits
    return edit_distance(a, b)[0] <= k


# RELATED: the word-ladder version: change one letter at a time and every
# word must be in the list; BFS over the words
def ladder_length(begin, end, words):
    words = set(words)
    if end not in words:
        return 0
    queue = deque([(begin, 1)])
    seen = {begin}
    while queue:
        word, steps = queue.popleft()
        if word == end:
            return steps
        for i in range(len(word)):
            for c in "abcdefghijklmnopqrstuvwxyz":
                following = word[:i] + c + word[i + 1:]
                if following in words and following not in seen:
                    seen.add(following)
                    queue.append((following, steps + 1))
    return 0


# ---- main question: tests ----
def test_main_question():
    assert edit_distance("horse", "ros")[0] == 3
    assert edit_distance("", "abc")[0] == 3
    assert edit_distance("same", "same")[0] == 0
    assert within_k("kitten", "sitting", 3)
    assert not within_k("kitten", "sitting", 2)
    words = ["hot", "dot", "dog", "lot", "log", "cog"]
    assert ladder_length("hit", "cog", words + ["hit"]) == 5
```

```
    assert ladder_length("hit", "cog", ["hot", "dot"]) == 0


# ======================================================
# FOLLOW-UP 1: Return the actual operations
```

```
# ======================================================
# ---- follow-up 1: function ----
# Answer: keep the whole table and walk back from dp[m][n]: if the
# letters match, step diagonally; otherwise take the neighbour that is one
# smaller and write down the replace, delete or insert. The walk back needs
# the whole table: O(m * n) memory.
def edit_operations(a, b):
    distance, dp = edit_distance(a, b)
    i = len(a)
    j = len(b)
    operations = []
    while i > 0 or j > 0:
        if i > 0 and j > 0 and a[i - 1] == b[j - 1]:
            i -= 1
            j -= 1
        elif i > 0 and j > 0 and dp[i][j] == dp[i - 1][j - 1] + 1:
            operations.append("replace " + a[i - 1] + " with " + b[j - 1])
            i -= 1
            j -= 1
        elif i > 0 and dp[i][j] == dp[i - 1][j] + 1:
            operations.append("delete " + a[i - 1])
            i -= 1
        else:
            operations.append("insert " + b[j - 1])
            j -= 1
    operations.reverse()
    return operations


# ---- follow-up 1: tests ----
def test_follow_up_1():
    operations = edit_operations("horse", "ros")
    assert len(operations) == 3
    assert edit_operations("same", "same") == []


# ======================================================
# FOLLOW-UP 2: Are the strings exactly one edit apart (LeetCode 161)?
# ======================================================
# ---- follow-up 2: function ----
# Answer: no table needed. If the lengths differ by more than one, say
# no. Walk to the first place where the letters differ and compare the rest:
# equal lengths: skip one letter in each; otherwise skip one in the longer
# one. O(n) time, O(1) space.
def one_edit_apart(a, b):
```

```
    if abs(len(a) - len(b)) > 1:
        return False
    if len(a) > len(b):
        a, b = b, a                             # a is now the shorter (or equal) one
    for i in range(len(a)):
        if a[i] != b[i]:
            if len(a) == len(b):
                return a[i + 1:] == b[i + 1:]   # one replaced letter
            return a[i:] == b[i + 1:]           # one extra letter in b
    return len(a) != len(b)                     # no difference: only an extra letter at the end counts


# ---- follow-up 2: tests ----
def test_follow_up_2():
```

```
    assert one_edit_apart("cat", "cut")
    assert one_edit_apart("cat", "cats")
    assert not one_edit_apart("cat", "cat")
    assert not one_edit_apart("cat", "dog")


# =====================================================
# FOLLOW-UP 3: Operations have different costs, or only a small edit distance is of interest (at most k)
# =====================================================
# Answer: different costs: use the cost of each operation instead of 1 in
# the formula. At most k: only cells within k of the diagonal can matter,
# so fill a band of width 2k + 1: O(n * k).


# =====================================================
# RUN EVERYTHING
# =====================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

44 Count Perfect Wake Numbers up to N

HARD

Template fit: digit DP: cache best(remaining digits, used-digit mask, last digit, falling?) and walk along N with a tight flag

QUESTION

A number is perfect if its digits are all different, it contains no 0, and no digit is a valley (a digit strictly smaller than both of its neighbours). Part 1: decide whether a number is perfect. Part 2: count the perfect numbers from 1 to N.

Function signature

def is_perfect(n: int) -> bool

def count_up_to(N: int) -> int

EXAMPLES

Example 1	
Input	196
Output	true
Why	Digits 1, 9, 6 are different, no zero, and 9 is a peak, not a valley.

Example 2	
Input	1546
Output	false
Why	The 4 in 1546 sits between 5 and 6, both larger: a valley.

Example 3	
Input	99
Output	81
Why	Every number 1-9 and every two-digit number with different non-zero digits: 9 + 72 = 81.

FULL CODE | main question first, then every follow-up, tests at the end

```python
# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ========================================================

# ========================================================
# MAIN QUESTION
# ========================================================
# ---- main question: helper functions ----
def digits_of(n):
    # the digits of n as a list of numbers: 1203 -> [1, 2, 0, 3]
    result = []
    for ch in str(n):
        result.append(int(ch))
    return result


def choose(n, k):
    # the number of ways to pick k things out of n
    ways = 1
    for i in range(k):
        ways = ways * (n - i) // (i + 1)
    return ways


def with_k_digits(k):
    # how many perfect numbers have exactly k digits (closed form, follow-up 4):
    # pick k different digits out of 1..9; the biggest is the peak and every
    # other digit goes left or right of it in sorted order: choose(9, k) * 2^(k-1)
    if k < 1:
        return 0
    return choose(9, k) * 2 ** (k - 1)


# ---- main question: main functions ----
# Idea: a number is perfect when it has no zero, no repeated digit, and
# no "valley": no digit smaller than both of its neighbours.
def is_perfect(n):
    d = digits_of(n)
    seen = set()
    for digit in d:
        if digit == 0 or digit in seen:
            return False
```

```python
        seen.add(digit)
    for i in range(1, len(d) - 1):
        if d[i - 1] > d[i] and d[i] < d[i + 1]:
            return False
    return True


# PART 2: how many perfect numbers are there from 1 to n?
# Idea (digit DP): numbers with fewer digits come from the closed form.
# For numbers with the SAME number of digits, build them digit by digit
# from the left, remembering: which digits are used (a bitmask), the last
# digit, whether we already went DOWN (`falling`: going up after that
# would be a valley), and whether we are still equal to the limit
# (`tight`: then the next digit may not exceed n's digit).
def count_up_to(n):
    if n < 1:
        return 0
    digits = digits_of(n)
    size = len(digits)
    total = 0
    for k in range(1, size):
```

```
        total += with_k_digits(k)
    memo = {}

    def count_from(position, used, last, falling, tight):
        if position == size:
            return 1
        key = (position, used, last, falling, tight)
        if key in memo:
            return memo[key]
        limit = 9
        if tight:
            limit = digits[position]
        count = 0
        for d in range(1, limit + 1):
            if (used >> d) % 2 == 1:
                continue                            # digit already used
            now_falling = falling
            if position > 0:
                if d > last and falling:
                    continue                        # going up after going down: a valley
                if d < last:
                    now_falling = True
            count += count_from(position + 1, used + (1 << d), d, now_falling, tight and d == digits[position
])
        memo[key] = count
```

```
        return count

    return total + count_from(0, 0, 0, False, True)


# ---- main question: tests ----
def test_main_question():
    for good in (196, 23, 12463):
        assert is_perfect(good)
    for bad in (1546, 320, 34321):
        assert not is_perfect(bad)
    for limit in (1, 9, 10, 99, 100, 500, 2000):         # the DP agrees with testing every number
        slow = 0
        for n in range(1, limit + 1):
            if is_perfect(n):
                slow += 1
        assert count_up_to(limit) == slow


# ============================================================
# FOLLOW-UP 1: Why does the DP only need a "falling" flag instead of the full history?
# ============================================================
# Answer: a valley is "down, then up". Once we have gone down, the only
# thing that matters is that we never go up again; which digits came
# before is irrelevant (the used-digit mask handles repeats).


# ============================================================
# FOLLOW-UP 3: What is the closed form without a DP?
# ============================================================


# ============================================================
# FOLLOW-UP 4: Is there a closed form for k-digit numbers?
# ============================================================
# Answer: with_k_digits(k) above: choose(9, k) * 2^(k - 1).


# ============================================================
# FOLLOW-UP 2: How do you count the numbers in a range [A, B]?
```

```
# ============================================================


# ============================================================
# FOLLOW-UP 5: Count perfect numbers in a range [A, B]
```

```
# ============================================================
# ---- follow-up 2: function ----
# Answer: inclusion-exclusion: count(B) - count(A - 1). The DP depends
# on the upper bound only and is cheap, so call it twice.
def count_between(a, b):
    return count_up_to(b) - count_up_to(a - 1)


# ---- follow-up 2: tests ----
def test_follow_up_2():
    assert count_between(10, 99) == count_up_to(99) - count_up_to(9)
    assert with_k_digits(1) == 9
    assert with_k_digits(2) == 72                       # every 2-digit number with different non-zero digit
s


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_2()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

45 Word Break: All Sentences — HARD

Template fit: best(start) = the list of sentences for the suffix; for every word that matches at start, put it in front of each sentence of best(end)

QUESTION

You are given a string s and a dictionary word_dict of distinct words. Return ALL possible ways to segment s into a sequence of space-separated words such that every word in the sequence appears in word_dict . A dictionary word may be reused any number of times. Each returned sentence is a single string of the chosen words joined by single spaces. The sentences may be returned in any order.

Function signature

from typing import List
def word_break_all(s: str, word_dict: List[str]) -> List[str]:
    ...

Follow-up (as reported): Solve with backtracking: at each position try every dictionary word that is a prefix of the remaining string, then recurse. Add memoization keyed on the suffix (or start index) to reuse partial results and avoid exponential re-work.

EXAMPLES

Example 1	
Input	`s = "catsanddog", word_dict = ["cat","cats","and","sand","dog"]`
Output	`["cats and dog", "cat sand dog"]`
Why	`Two segmentations use only dictionary words.`

QUESTION

Example 2	
Input	`s = "pineapplepenapple", word_dict = ["apple","pen","applepen","pine","pineapple"]`
Output	`["pine apple pen apple", "pineapple pen apple", "pine applepen apple"]`
Why	`All three segmentations consist entirely of dictionary words.`

Example 3	
Input	`s = "catsandog", word_dict = ["cats","dog","sand","and","cat"]`
Output	`[]`
Why	`No segmentation covers the whole string with dictionary words.`

CONSTRAINTS

- 1 <= len(s) <= 20
- 1 <= len(word_dict) <= 1000
- 1 <= len(word) <= 10 for each word; words consist of lowercase letters and are distinct.
- If no valid segmentation exists, return an empty list [] .

FULL CODE | main question first, then every follow-up, tests at the end

```
# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ========================================================

# ========================================================
# MAIN QUESTION
# ========================================================
# ---- main question: main function ----
# Idea (memoised search on the start index): the sentences that can be
# made from the ending of s that starts at `start` do not depend on how
# we got there, so compute them once per start. Try every word that
# begins at `start` and put it in front of every sentence of the rest.
def word_break_all(s, word_dict):
    words = set(word_dict)
    memo = {}

    def sentences(start):
        if start == len(s):
            return [""]                                  # nothing left: one empty sentence
        if start in memo:
            return memo[start]
        result = []
        for end in range(start + 1, len(s) + 1):
            word = s[start:end]
            if word in words:
                for rest in sentences(end):
                    if rest == "":
                        result.append(word)
                    else:
                        result.append(word + " " + rest)
        memo[start] = result
        return result

    return sentences(0)


# ---- main question: tests ----
def test_main_question():
```

```
    found = word_break_all("catsanddog", ["cat", "cats", "and", "sand", "dog"])
    assert sorted(found) == ["cat sand dog", "cats and dog"]
    assert word_break_all("catsandog", ["cats", "dog", "sand", "and", "cat"]) == []
    pineapple = word_break_all("pineapplepenapple", ["apple", "pen", "applepen", "pine", "pineapple"])
    assert len(pineapple) == 3


# ============================================================
# FOLLOW-UP 1: Only count the segmentations
# ============================================================
# ---- follow-up 1: function ----
# Answer: a DP over the endings: ways[i] is the sum of ways[j] over
# every word s[i:j]. No strings are built, so it is O(n * L) with a set of words.
def count_sentences(s, word_dict):
    words = set(word_dict)
    ways = [0] * (len(s) + 1)
    ways[len(s)] = 1
    for i in range(len(s) - 1, -1, -1):
        for j in range(i + 1, len(s) + 1):
            if s[i:j] in words:
                ways[i] += ways[j]
    return ways[0]


# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert count_sentences("catsanddog", ["cat", "cats", "and", "sand", "dog"]) == 2
    assert count_sentences("catsandog", ["cats", "dog", "sand", "and", "cat"]) == 0


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

GROUP 6 OF 43 | 9 QUESTIONS, ONE CODE TEMPLATE | 5 M 4 H

Best-first search with a heap: Dijkstra and its minimax twin

46 Path Minimizing the Maximum Cell (Swim in Rising Water) MEDIUM

Template fit: the same minimax loop with a given start and end cell (Swim in Rising Water)

QUESTION

You get a grid of heights, a start cell and an end cell, and you may move to any of the four neighbouring cells. The cost of a path is the largest height on it (start and end included). Return the smallest cost over all paths from start to end.

Function signature

def min_max_path(g: list, start: tuple, end: tuple) -> int

EXAMPLES

Example 1	
Input	`[[0, 2], [1, 3]], (0, 0), (1, 1)`
Output	`3`
Why	`Both routes pass the cell of height 3 at the end, so the best cost is 3.`

Example 2	
Input	`[[0, 1, 2, 3, 4], [24, 23, 22, 21, 5], [12, 13, 14, 15, 16], [11, 17, 18, 19, 20], [10, 9, 8, 7, 6]], (0, 0), (4, 4)`
Output	`16`
Why	`A 5x5 board: the best route has to climb to 16 somewhere, and no route stays lower.`

FULL CODE | main question first, then every follow-up, tests at the end

```
# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ========================================================

import heapq

INF = float("inf")


# ========================================================
# MAIN QUESTION
# ========================================================
# ---- main question: helper functions ----
def neighbors(r, c, rows, cols):
    # the cells above, below, left and right of (r, c) that exist
    result = []
    for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)):
        nr, nc = r + dr, c + dc
        if 0 <= nr < rows and 0 <= nc < cols:
            result.append((nr, nc))
    return result


# ---- main question: main function ----
# Idea: Dijkstra on the running maximum. Heap entries are (cost so
# far, cell); taking a cell out fixes its cost; a neighbour gets
# max(cost, its own value). Stop when the end cell comes out; skip
```

```
# outdated heap entries.
def min_max_path(g, start, end):
    rows, cols = len(g), len(g[0])
    best = {start: g[start[0]][start[1]]}
    heap = [(best[start], start)]
    while heap:
        cost, cell = heapq.heappop(heap)
        if cost > best[cell]:
            continue                            # an outdated entry
        if cell == end:
            return cost
        for nr, nc in neighbors(cell[0], cell[1], rows, cols):
            new_cost = cost
            if g[nr][nc] > new_cost:
                new_cost = g[nr][nc]            # highest cell so far, or this one
```

```
            if new_cost < best.get((nr, nc), INF):
                best[(nr, nc)] = new_cost
                heapq.heappush(heap, (new_cost, (nr, nc)))
    return None


# the same idea on a weighted graph: every edge must be below `limit`,
# and the cost of a path is its heaviest edge
def min_bottleneck_edge(graph, s, t, limit):
    # graph[u] = [(v, weight), ...]
    best = {s: 0}
    heap = [(0, s)]
    while heap:
        cost, u = heapq.heappop(heap)
        if cost > best[u]:
            continue
        if u == t:
            return cost
        for v, w in graph.get(u, []):
            if w >= limit:
                continue                        # edges at or above the limit are not allowed
            new_cost = cost
            if w > new_cost:
                new_cost = w
            if new_cost < best.get(v, INF):
                best[v] = new_cost
                heapq.heappush(heap, (new_cost, v))
    return -1                                   # no admissible path


# the related plain Dijkstra: when does a signal reach every node?
def network_delay(n, edges, src):
    graph = {}
    for u, v, w in edges:
        if u not in graph:
            graph[u] = []
        graph[u].append((v, w))
    dist = {src: 0}
    heap = [(0, src)]
    while heap:
        d, u = heapq.heappop(heap)
        if d > dist[u]:
            continue
        for v, w in graph.get(u, []):
            if d + w < dist.get(v, INF):
```

```
                dist[v] = d + w
                heapq.heappush(heap, (d + w, v))
    if len(dist) < n:
        return -1                               # some node is never reached
```

```
    worst = 0
    for d in dist.values():
        if d > worst:
            worst = d
    return worst


# ---- main question: tests ----
def test_main_question():
    g = [[0, 1, 2, 3, 4], [24, 23, 22, 21, 5], [12, 13, 14, 15, 16],
         [11, 17, 18, 19, 20], [10, 9, 8, 7, 6]]
    assert min_max_path(g, (0, 0), (4, 0)) == 16
    assert min_max_path(g, (2, 2), (2, 2)) == 14           # start equals end: just that cell
    graph = {0: [(1, 5), (2, 9)], 1: [(2, 3)]}
    assert min_bottleneck_edge(graph, 0, 2, 10) == 5
    assert min_bottleneck_edge(graph, 0, 2, 5) == -1
    chain = [(0, 1, 2), (1, 2, 3)]
    assert network_delay(3, chain, 0) == 5
    assert network_delay(3, [(0, 1, 2)], 0) == -1


# ========================================================
# FOLLOW-UP 1: Maximise the minimum distance to a cat instead
# ========================================================
# ---- follow-up 1: function ----
# Answer: the same bottleneck shape with the goal flipped: score each
# cell by its distance to the cat, always extend the SAFEST frontier
# cell (a max-heap: push negatives), and the path is as safe as its
# closest cell. "#" is water and cannot be entered.
def safest_path_score(grid, start, target, cat):
    rows, cols = len(grid), len(grid[0])

    def cat_distance(r, c):
        return abs(r - cat[0]) + abs(c - cat[1])

    best = {start: cat_distance(start[0], start[1])}
    heap = [(-best[start], start)]
    while heap:
        negative, cell = heapq.heappop(heap)
        safety = -negative
        if safety < best[cell]:
            continue
        if cell == target:
            return safety
        for nr, nc in neighbors(cell[0], cell[1], rows, cols):
            if grid[nr][nc] == "#":
                continue
            new_safety = safety
            if cat_distance(nr, nc) < new_safety:
                new_safety = cat_distance(nr, nc)      # only as safe as the closest cell
            if new_safety > best.get((nr, nc), -1):
                best[(nr, nc)] = new_safety
                heapq.heappush(heap, (-new_safety, (nr, nc)))
    return None


# ---- follow-up 1: tests ----
def test_follow_up_1():
    grid = ["...", ".#.", "..."]
    assert safest_path_score(grid, (0, 0), (2, 2), (0, 2)) == 2


# ========================================================
# FOLLOW-UP 2: Why is Dijkstra correct for a max-along-path cost?
# ========================================================
```

```
# Answer: Dijkstra needs one property: extending a path never makes it
# cheaper. For "max along the path" that is true (the max can only stay
# or grow), so the first time a cell leaves the heap its cost is final.


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

47 Nearest Favorite City by Travel Time — MEDIUM

Template fit: combine = cost + w; stop when a favourite is popped (keep popping equal distances to break ties on the smaller index)

QUESTION

You are given a network of num_cities cities (0..num_cities-1) connected by weighted, undirected roads roads[i] = [u, v, w] where w > 0 is the travel time. Given a start city and a non-empty list favorites of favorite cities, return the favorite city reachable from start in the shortest total travel time. Break ties by smallest city index. If none of the favorite cities is reachable, return -1 .

Function signature

```
def nearest_favorite(num_cities: int, roads: list[list[int]],
                     start: int, favorites: list[int]) -> int:
    ...
```

EXAMPLES

Example 1	
Input	`num_cities = 5, roads = [[0,1,2],[0,2,5],[1,3,1],[2,4,1]], start = 0, favorites = [3, 4]`
Output	`3`
Why	`Shortest times from 0: to 3 = 2+1 = 3, to 4 = 5+1 = 6. City 3 is nearer.`

Example 2	
Input	`num_cities = 3, roads = [[0,1,4]], start = 0, favorites = [2]`
Output	`-1`
Why	`City 2 is disconnected from start, so no favorite is reachable.`

CONSTRAINTS

- 1 <= num_cities <= 10**5 , 0 <= len(roads) <= 2*10**5 , 1 <= w <= 10**9 .
- The interviewer required the optimal solution (Dijkstra with a priority queue).

FULL CODE | main question first, then every follow-up, tests at the end

```
# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================
```

```python
import heapq

INF = float("inf")


# ========================================================
# MAIN QUESTION
# ========================================================
# ---- main question: helper functions ----
def build_roads(num_cities, roads):
    # roads are two-way: [city, city, travel time]
    adj = []
    for i in range(num_cities):
        adj.append([])
    for u, v, w in roads:
        adj[u].append((v, w))
        adj[v].append((u, w))
    return adj


# ---- main question: main function ----
# Idea: Dijkstra from the start finalises cities in order of distance,
# so the first favourite taken out is the nearest. Cities at the same
# distance: the smaller number wins, so keep going while d does not
# exceed the best distance found. Returns -1 if none is reachable.
def nearest_favorite(num_cities, roads, start, favorites):
    adj = build_roads(num_cities, roads)
    dist = [INF] * num_cities
    dist[start] = 0
    heap = [(0, start)]
    best_distance = None
    best_city = -1
    while heap:
        d, u = heapq.heappop(heap)
        if d > dist[u]:
            continue
        if best_distance is not None and d > best_distance:
            break                                   # everything left is farther
        if u in favorites and (best_distance is None or u < best_city):
            best_distance = d
            best_city = u
        for v, w in adj[u]:
            if d + w < dist[v]:
                dist[v] = d + w
                heapq.heappush(heap, (dist[v], v))
    return best_city


# ---- main question: tests ----
def test_main_question():
    roads = [[0, 1, 2], [0, 2, 5], [1, 3, 1], [2, 4, 1]]
    assert nearest_favorite(5, roads, 0, [3, 4]) == 3
    assert nearest_favorite(3, [[0, 1, 4]], 0, [2]) == -1


# ========================================================
# FOLLOW-UP 1: Answer it for every possible start city in one run
# ========================================================
# ---- follow-up 1: function ----
# Answer: turn the search around: ONE Dijkstra that starts from every
# favourite city at distance 0, storing for each city the pair
# (distance, which favourite). Roads are two-way, so this is the same
# distance as searching from the city.
def nearest_favorite_for_all(num_cities, roads, favorites):
    adj = build_roads(num_cities, roads)
```

```
    best = []
    for i in range(num_cities):
        best.append((INF, -1))
    heap = []
    for f in favorites:
        best[f] = (0, f)
        heapq.heappush(heap, (0, f, f))
    while heap:
        d, fav, u = heapq.heappop(heap)
        if (d, fav) > best[u]:
            continue
        for v, w in adj[u]:
            candidate = (d + w, fav)
            if candidate < best[v]:
                best[v] = candidate
                heapq.heappush(heap, (candidate[0], fav, v))
    answers = []
    for d, fav in best:
```

```
        if d < INF:
            answers.append(fav)
        else:
            answers.append(-1)
    return answers


# ---- follow-up 1: tests ----
def test_follow_up_1():
    roads = [[0, 1, 2], [0, 2, 5], [1, 3, 1], [2, 4, 1]]
    everyone = nearest_favorite_for_all(5, roads, [3, 4])
    for city in range(5):
        assert everyone[city] == nearest_favorite(5, roads, city, [3, 4])


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

48 Given a train schedule with departure city, arrival city, start time, end time, and duration, and given a source and destination, find the sequence of trains that gets you there at the earliest possible time. MEDIUM

Template fit: cost = arrival time; neighbors only offers trains that leave at or after the current time; combine = the train's arrival time

QUESTION

A schedule lists trains as (from, to, departure time, arrival time). Starting in city src at time start, find the sequence of trains that gets you to dst at the earliest possible time. You can only board a train that leaves at or after the time you arrive in its departure city. Return (arrival time, the trains you take), or None if dst cannot be reached.

Function signature

def earliest_arrival(trains: list, src, dst, start=0) -> tuple | None

EXAMPLES

QUESTION

Example 1	
Input	`[('A', 'B', 1, 3), ('B', 'C', 4, 6), ('A', 'C', 2, 9), ('B', 'C', 2, 5)], 'A', 'C'`
Output	`(6, [('A', 'B', 1, 3), ('B', 'C', 4, 6)])`
Why	A -> B arrives at 3; the train B -> C that leaves at 2 is already gone, so take the one leaving at 4 and arrive at 6 (the direct A -> C train arrives later, at 9).

Example 2	
Input	`[('A', 'B', 1, 3), ('B', 'C', 4, 6)], 'C', 'A'`
Output	`null`
Why	No train leaves C, so A cannot be reached from C.

FULL CODE | main question first, then every follow-up, tests at the end

```
# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

import heapq
import random

DAY = 24 * 60          # times are minutes from midnight


# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
# A train is a tuple (from_city, to_city, departure_time, arrival_time).
def trains_by_city(trains):
    # city -> the trains that leave from it
    table = {}
    for train in trains:
        if train[0] not in table:
            table[train[0]] = []
        table[train[0]].append(train)
    return table


# ---- main question: main function ----
# Idea: Dijkstra on arrival time. best[city] is the earliest moment you
# can be there. From a city you may board any train that leaves at or
# after your arrival (plus `layover`, follow-up 1; no layover is needed
# in the city you start in). Remember which train improved each city to
# rebuild the sequence. Returns (arrival time, trains taken) or None.
def earliest_arrival(trains, src, dst, start=0, layover=0):
    leaving = trains_by_city(trains)
    best = {src: start}
    took = {}                                   # city -> the train that got us there
    heap = [(start, src)]
    while heap:
        time, city = heapq.heappop(heap)
        if time > best[city]:
            continue                            # an outdated entry
        if city == dst:
            break
```

```
        wait = layover
        if city == src:
            wait = 0
        for train in leaving.get(city, []):
            _from, to_city, departs, arrives = train
            if departs >= time + wait and (to_city not in best or arrives < best[to_city]):
                best[to_city] = arrives
                took[to_city] = train
                heapq.heappush(heap, (arrives, to_city))
    if dst not in best:
        return None
    route = []
    city = dst
    while city != src:                          # walk back from the destination
        route.append(took[city])
        city = took[city][0]
    route.reverse()
    return best[dst], route


# ---- main question: tests ----
TRAINS = [("A", "B", 60, 120),
          ("B", "C", 130, 200),
          ("A", "C", 100, 400),
          ("B", "C", 110, 150)]


def test_main_question():
    arrival, route = earliest_arrival(TRAINS, "A", "C")
    assert arrival == 200
    assert route == [("A", "B", 60, 120), ("B", "C", 130, 200)]
    assert earliest_arrival(TRAINS, "C", "A") is None
    assert earliest_arrival(TRAINS, "A", "A") == (0, [])
    example = [("A", "B", 1, 3), ("B", "C", 4, 6), ("A", "C", 2, 9), ("B", "C", 2, 5)]
    assert earliest_arrival(example, "A", "C") == (6, [("A", "B", 1, 3), ("B", "C", 4, 6)])


# =========================================================
# FOLLOW-UP 1: A minimum layover is required when changing trains
# =========================================================
# ---- follow-up 1: tests ----
# Answer: the boarding test becomes  departs >= arrival + layover  for
# every city except where you start (the `layover` parameter above). The
# earliest-arrival idea still holds, so Dijkstra stays correct.
def test_follow_up_1():
```

```
    # a 15 minute layover rules out the 130 departure (arrive 120, 120 + 15 = 135):
    # the direct train wins
    arrival, route = earliest_arrival(TRAINS, "A", "C", layover=15)
    assert arrival == 400 and len(route) == 1


# =========================================================
# FOLLOW-UP 2: Fewest transfers instead of earliest arrival
# =========================================================
# ---- follow-up 2: function ----
# Answer: do rounds. Round k holds, for every city, the earliest arrival
# using at most k trains (like Bellman-Ford). Stop at the first round
# that reaches the destination. (Or Dijkstra on the pair
# (number of trains, arrival time).)
def fewest_trains(trains, src, dst, start=0, layover=0):
    best = {src: (start, [])}                   # city -> (earliest arrival, trains used)
    for round_number in range(len(trains)):
        after = dict(best)
        for train in trains:
            from_city, to_city, departs, arrives = train
            if from_city not in best:
```

```
                continue
            time, route = best[from_city]
            wait = layover
            if from_city == src:
                wait = 0
            if departs < time + wait:
                continue
            if to_city not in after or arrives < after[to_city][0]:
                after[to_city] = (arrives, route + [train])
        if dst in after:
            return after[dst]
        best = after
    return None


# ---- follow-up 2: tests ----
def test_follow_up_2():
    arrival, route = fewest_trains(TRAINS, "A", "C")
    assert len(route) == 1 and arrival == 400


# ========================================================
# FOLLOW-UP 3: The schedule repeats daily
# ========================================================
```

```
# ---- follow-up 3: function ----
# Answer: unroll the timetable into absolute times (add 24 hours for
# every day); a train that arrives after midnight has an arrival
# number smaller than its departure, so add one day to it. The same
# Dijkstra then works on absolute times.
def daily(trains, days):
    unrolled = []
    for day in range(days):
        shift = day * DAY
        for from_city, to_city, departs, arrives in trains:
            if arrives < departs:
                arrives = arrives + DAY
            unrolled.append((from_city, to_city, departs + shift, arrives + shift))
    return unrolled


def route_is_valid(route, src, dst, start, layover, arrival):
    # does the route start at src, end at dst, never board before it may, and end at `arrival`?
    city, time = src, start
    for from_city, to_city, departs, arrives in route:
        wait = layover
        if city == src:
            wait = 0
        if from_city != city or departs < time + wait:
            return False
        city, time = to_city, arrives
    return city == dst and time == arrival


# ---- follow-up 3: tests ----
def test_follow_up_3():
    week = daily([("A", "B", 1400, 60)], 2)
    assert week[0][3] == 1500 and week[1][2] == 2840      # the 23:20 train arrives at 01:00 next day
    late = daily([("A", "B", 600, 700)], 2)
    assert earliest_arrival(late, "A", "B")[0] == 700
    random.seed(7)
    for round_number in range(300):                         # random timetables: every answer is a legal rout
e
        trains = []
        for k in range(random.randint(1, 9)):
            a, b = random.sample("ABCD", 2)
```

```
            departs = random.randint(0, 100)
            trains.append((a, b, departs, departs + random.randint(1, 50)))
        gap = random.choice([0, 5, 20])
        answer = earliest_arrival(trains, "A", "D", layover=gap)
        if answer is not None:
```

```
            assert route_is_valid(answer[1], "A", "D", 0, gap, answer[0])


# ====================================================
# RUN EVERYTHING
# ====================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

49 Find the shortest path in a graph. MEDIUM

Template fit: combine = cost + w; also store parent[v]; stop at the target and rebuild the path

QUESTION

A directed graph is given as a list of weighted edges (u, v, w) with non-negative weights. Return the total length and the nodes of a shortest path from s to t, or None if t cannot be reached.

Function signature

def dijkstra_path(edges: list, s, t) -> tuple | None

(distance, path)

EXAMPLES

Example 1	
Input	`[('A', 'B', 1), ('A', 'C', 4), ('B', 'C', 2), ('C', 'D', 1)], 'A', 'D'`
Output	`(4, ['A', 'B', 'C', 'D'])`
Why	`A -> B -> C -> D costs 1 + 2 + 1 = 4; the direct edge A -> C costs 4, which is longer overall.`

Example 2	
Input	`[('A', 'B', 1), ('B', 'C', 2), ('C', 'D', 1)], 'D', 'A'`
Output	`null`
Why	`There is no edge leading from D back to A.`

FULL CODE | main question first, then every follow-up, tests at the end

```
# ====================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ====================================================

from collections import deque
import heapq
```

```
INF = float("inf")


# ======================================================
# MAIN QUESTION
# ======================================================
# ---- main question: helper functions ----
def make_graph(edges):
    # node -> list of (neighbour, weight); an edge is (u, v) or (u, v, weight)
    graph = {}
    for edge in edges:
        u, v = edge[0], edge[1]
        weight = 1
        if len(edge) == 3:
            weight = edge[2]
        if u not in graph:
            graph[u] = []
        graph[u].append((v, weight))
    return graph


def walk_back(parent, node):
    # follow the parent links back to the start, then reverse
    path = []
    while node is not None:
        path.append(node)
        node = parent[node]
    path.reverse()
    return path


# ---- main question: main function ----
# Idea: keep the best known distance to every node and a min-heap of
# (distance, node). Take the closest unsettled node, relax its edges
# (try to improve each neighbour), and remember who improved whom.
```

```
# Works for weights that are not negative. Returns (distance, path).
def dijkstra_path(edges, s, t):
    graph = make_graph(edges)
    dist = {s: 0}
    parent = {s: None}
    heap = [(0, s)]
    while heap:
        d, u = heapq.heappop(heap)
        if d > dist[u]:
            continue                        # an outdated heap entry
        if u == t:
            return d, walk_back(parent, u)  # the first time t comes out of the heap it is final
        for v, w in graph.get(u, []):
            if d + w < dist.get(v, INF):
                dist[v] = d + w
                parent[v] = u
                heapq.heappush(heap, (d + w, v))
    return None


# ---- main question: tests ----
def test_main_question():
    edges = [(0, 1, 4), (0, 2, 1), (2, 1, 2), (1, 3, 1)]
    assert dijkstra_path(edges, 0, 3) == (4, [0, 2, 1, 3])
    assert dijkstra_path(edges, 3, 0) is None
    assert dijkstra_path(edges, 0, 0) == (0, [0])


# ======================================================
```

```
# FOLLOW-UP 1: The graph is unweighted
# ======================================================
# ---- follow-up 1: function ----
# Answer: a BFS with a parent map finds a shortest path in O(V + E);
# Dijkstra would be correct but needlessly slow.
def bfs_path(edges, s, t):
    graph = make_graph(edges)
    parent = {s: None}
    queue = deque([s])
    while queue:
        u = queue.popleft()
        if u == t:
            path = walk_back(parent, u)
            return len(path) - 1, path
        for v, w in graph.get(u, []):
            if v not in parent:
```

```
                parent[v] = u
                queue.append(v)
    return None


# ---- follow-up 1: tests ----
def test_follow_up_1():
    edges = [(0, 1, 4), (0, 2, 1), (2, 1, 2), (1, 3, 1)]
    assert bfs_path(edges, 0, 3) == (2, [0, 1, 3])


# ======================================================
# FOLLOW-UP 2: Weights can be negative
# ======================================================
# ---- follow-up 2: function ----
# Answer: Dijkstra can give wrong answers. Use Bellman-Ford: relax
# every edge n - 1 times (O(V * E)). If one more round still improves a
# distance, a negative cycle is reachable and shortest paths do not exist.
def bellman_ford(n, edges, s):
    dist = [INF] * n
    dist[s] = 0
    for round_number in range(n - 1):
        changed = False
        for u, v, w in edges:
            if dist[u] + w < dist[v]:
                dist[v] = dist[u] + w
                changed = True
        if not changed:
            break
    for u, v, w in edges:
        if dist[u] + w < dist[v]:
            raise ValueError("negative cycle")
    return dist


# ---- follow-up 2: tests ----
def test_follow_up_2():
    negative = [(0, 1, 5), (1, 2, -3)]
    assert bellman_ford(4, negative, 0) == [0, 5, 2, INF]
    try:
        bellman_ford(2, [(0, 1, 1), (1, 0, -3)], 0)
        raise AssertionError("negative cycle not found")
    except ValueError:
        pass
```

```
# ======================================================
# FOLLOW-UP 3: Weights are only 0 or 1, or the graph is huge and you know the target
# ======================================================
```

```python
# ---- follow-up 3: function ----
# Answer: with weights 0 and 1 use a deque (0-1 BFS) in O(V + E): a
# free edge goes to the FRONT, a cost-1 edge to the BACK. For a huge
# graph with a known target use A* (Dijkstra ordered by distance plus
# an estimate that never overestimates) or search from both ends.
def zero_one_bfs(n, edges, s):
    graph = make_graph(edges)
    dist = [INF] * n
    dist[s] = 0
    dq = deque([s])
    while dq:
        u = dq.popleft()
        for v, w in graph.get(u, []):
            if dist[u] + w < dist[v]:
                dist[v] = dist[u] + w
                if w == 0:
                    dq.appendleft(v)
                else:
                    dq.append(v)
    return dist


# ---- follow-up 3: tests ----
def test_follow_up_3():
    cheap = [(0, 1, 1), (1, 2, 0), (0, 2, 1)]
    assert zero_one_bfs(3, cheap, 0) == [0, 1, 1]


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

50 Given a matrix with a rat, bread, cat, and obstacles, find a path from the rat to the bread that maximizes the minimum distance from cat. MEDIUM

Template fit: maximise the minimum distance to the cat along the path: push negated values, combine = min(value, cat distance of the cell)

QUESTION

A grid contains a rat R, a piece of bread B, a cat C, walls

and open cells '.'. Move

the rat from R to B through open cells (up, down, left, right). The safety of a path is the smallest distance to the cat of any cell on it (distances count steps around the walls). Return the largest safety you can achieve, or -1 if the bread cannot be reached.

Function signature

def safest_path(grid: list) -> int

EXAMPLES

QUESTION

Example 1	
Input	`['R..', '...', 'C.B']`
Output	`2`
Why	The path along the top and right side keeps at least 2 steps from the cat at the bottom left.

Example 2	
Input	`['R#B']`
Output	`-1`
Why	The wall blocks the only route to the bread.

FULL CODE | main question first, then every follow-up, tests at the end

```
# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

from collections import deque
import heapq

INF = 10 ** 9


# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
def open_neighbors(r, c, grid):
    # the cells beside (r, c) that exist and are not an obstacle ("#")
    result = []
    for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)):
        nr, nc = r + dr, c + dc
        if 0 <= nr < len(grid) and 0 <= nc < len(grid[0]):
            if grid[nr][nc] != "#":
                result.append((nr, nc))
    return result


def find(grid, letter):
    # where is the letter ("R" rat, "B" bread, "C" cat)?
    for r in range(len(grid)):
        for c in range(len(grid[0])):
            if grid[r][c] == letter:
                return (r, c)
    return None


def cat_distances(grid):
    # one BFS from the cat gives every cell's distance to the cat
    dist = []
    for row in grid:
        dist.append([INF] * len(row))
    cat = find(grid, "C")
    if cat is None:
        return dist
```

```
    dist[cat[0]][cat[1]] = 0
    queue = deque([cat])
    while queue:
        r, c = queue.popleft()
        for nr, nc in open_neighbors(r, c, grid):
            if dist[nr][nc] == INF:
                dist[nr][nc] = dist[r][c] + 1
                queue.append((nr, nc))
    return dist


# ---- main question: main function ----
# Idea: a path is only as safe as its cell closest to the cat. Always
# extend the safest frontier cell (like Dijkstra, but with "max of the
# minimum" instead of "min of the sum"; the heap is a max-heap, so we
# push negative values). Returns that safety, or None if no path exists.
def safest_path(grid):
    dist = cat_distances(grid)
    start = find(grid, "R")
    goal = find(grid, "B")
    best = {start: dist[start[0]][start[1]]}
    heap = [(-best[start], start)]
    while heap:
        negative, cell = heapq.heappop(heap)
        safety = -negative
        if cell == goal:
            return safety
        if safety < best[cell]:
            continue                                    # an outdated entry
        for nr, nc in open_neighbors(cell[0], cell[1], grid):
            new_safety = safety
            if dist[nr][nc] < new_safety:
                new_safety = dist[nr][nc]               # the path is only as safe as its closest cell
            if new_safety > best.get((nr, nc), -1):
                best[(nr, nc)] = new_safety
                heapq.heappush(heap, (-new_safety, (nr, nc)))
    return None


# ---- main question: tests ----
GRID = ["R.....",
        ".###..",
        "...C.B"]
```

```
def test_main_question():
    assert cat_distances(GRID)[0][0] == 5
    assert safest_path(GRID) == 2                       # the bread itself is 2 steps from the cat
    assert safest_path(["R#B"]) is None                 # walled in
    assert safest_path(["R.B"]) == INF                  # no cat at all: every path is perfectly safe


# ======================================================
# FOLLOW-UP 1: Use binary search instead of a heap
# ======================================================
# ---- follow-up 1: helper functions ----
def shortest_path_at_least(grid, dist, d):
    # BFS using only cells at least d steps from the cat; returns the
    # shortest such path (a list of cells) or None
    start = find(grid, "R")
    goal = find(grid, "B")
    if dist[start[0]][start[1]] < d:
        return None
    parent = {start: None}
    queue = deque([start])
    while queue:
        cell = queue.popleft()
```

```
        if cell == goal:
            path = []
            while cell is not None:
                path.append(cell)
                cell = parent[cell]
            path.reverse()
            return path
        for nxt in open_neighbors(cell[0], cell[1], grid):
            if nxt not in parent and dist[nxt[0]][nxt[1]] >= d:
                parent[nxt] = cell
                queue.append(nxt)
    return None


# ---- follow-up 1: function ----
# Answer: binary search the safety d; for each d ask a BFS whether the
# rat can reach the bread using only cells at least d from the cat.
# The answer "yes" holds for small d and "no" for big d, so binary
# search works: O(m * n * log(m + n)).
def safest_by_search(grid):
    dist = cat_distances(grid)
    if shortest_path_at_least(grid, dist, 0) is None:
        return None
```

```
    low, high = 0, INF
    while low < high:
        mid = (low + high + 1) // 2
        if shortest_path_at_least(grid, dist, mid) is not None:
            low = mid
        else:
            high = mid - 1
    return low


# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert safest_by_search(GRID) == 2
    assert safest_by_search(["R#B"]) is None


# ========================================================
# FOLLOW-UP 2: Among the safest paths, return the shortest one
# ========================================================
# ---- follow-up 2: function ----
# Answer: first find the best safety, then run an ordinary BFS that is
# only allowed on cells at least that far from the cat. That is the
# shortest of the safest paths.
def shortest_safest_path(grid):
    best = safest_path(grid)
    if best is None:
        return None
    return shortest_path_at_least(grid, cat_distances(grid), best)


# ---- follow-up 2: tests ----
def test_follow_up_2():
    path = shortest_safest_path(GRID)
    assert path[0] == (0, 0) and path[-1] == (2, 5)
    dist = cat_distances(GRID)
    closest = INF
    for r, c in path:
        if dist[r][c] < closest:
            closest = dist[r][c]
    assert closest == 2
```

```
# ============================================================
# FOLLOW-UP 3: The cat moves one step toward the rat each turn
# ============================================================
```

```
# Answer: now a cell's danger depends on the turn. Search over states
# (cell, turn): the cat can be at most `turn` steps away from where it
# started, so a cell is safe at turn t only if its distance to the cat
# is greater than t.


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

51 Minimize maximum height along a grid path

HARD

Template fit: the same minimax loop on a height grid

QUESTION

You are given a grid of heights and walk from the top-left cell to the bottom-right cell, moving up, down, left or right. The cost of a path is the largest height of any cell on it. Return the smallest possible cost (and, with with_path=True, a path that achieves it).

Function signature

def min_max_path(H: list, with_path=False)

EXAMPLES

Example 1	
Input	`[[1, 9], [2, 3]]`
Output	`3`
Why	`Down then right passes cells 1, 2, 3: the largest is 3; going right first would have to cross the 9.`

Example 2	
Input	`[[1, 2], [4, 3]], with_path = true`
Output	`(3, [(0, 0), (0, 1), (1, 1)])`
Why	`With with_path=True you also get the route.`

FULL CODE | main question first, then every follow-up, tests at the end

```
# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

import heapq
```

```
INF = float("inf")


# ======================================================
# MAIN QUESTION
# ======================================================
# ---- main question: helper functions ----
def neighbors(r, c, rows, cols):
    # the cells above, below, left and right of (r, c) that exist
    result = []
    for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)):
        nr, nc = r + dr, c + dc
        if 0 <= nr < rows and 0 <= nc < cols:
            result.append((nr, nc))
    return result


def reachable(H, limit):
    # can we go top-left to bottom-right using only cells of height <= limit?
    rows, cols = len(H), len(H[0])
    if H[0][0] > limit:
        return False
    seen = {(0, 0)}
    stack = [(0, 0)]
    while stack:
        r, c = stack.pop()
        if (r, c) == (rows - 1, cols - 1):
            return True
        for nr, nc in neighbors(r, c, rows, cols):
            if (nr, nc) not in seen and H[nr][nc] <= limit:
                seen.add((nr, nc))
                stack.append((nr, nc))
    return False


# ---- main question: main function ----
```

```
# Idea: Dijkstra where a path costs its highest cell. Heap of (highest
# cell so far, cell): take the smallest, extend to each neighbour with
# cost max(current, neighbour), keep only improvements.
def min_max_path(H, with_path=False):
    rows, cols = len(H), len(H[0])
    best = {(0, 0): H[0][0]}
    parent = {}
    heap = [(H[0][0], 0, 0)]
    while heap:
        cost, r, c = heapq.heappop(heap)
        if cost > best[(r, c)]:
            continue
        if (r, c) == (rows - 1, cols - 1):
            if not with_path:
                return cost
            path = [(r, c)]
            while (r, c) in parent:
                r, c = parent[(r, c)]
                path.append((r, c))
            path.reverse()
            return cost, path
        for nr, nc in neighbors(r, c, rows, cols):
            new_cost = cost
            if H[nr][nc] > new_cost:
                new_cost = H[nr][nc]
            if new_cost < best.get((nr, nc), INF):
                best[(nr, nc)] = new_cost
                parent[(nr, nc)] = (r, c)
                heapq.heappush(heap, (new_cost, nr, nc))
```

```
    return None


# the same answer by binary search over the sorted heights
def min_max_path_bsearch(H):
    seen = set()
    for row in H:
        for h in row:
            seen.add(h)
    heights = sorted(seen)
    low, high = 0, len(heights) - 1
    while low < high:
        mid = (low + high) // 2
        if reachable(H, heights[mid]):
            high = mid                  # this height is enough: try lower
        else:
            low = mid + 1               # not enough: go higher
    return heights[low]


# ---- main question: tests ----
def test_main_question():
    H = [[0, 2, 1], [1, 3, 1], [2, 2, 1]]
    assert min_max_path(H) == min_max_path_bsearch(H)
    cost, path = min_max_path(H, True)
    assert path[0] == (0, 0) and path[-1] == (2, 2)
    highest = 0
    for r, c in path:
        if H[r][c] > highest:
            highest = H[r][c]
    assert highest == cost
    assert min_max_path([[7]]) == 7
    grid = [[0, 1, 2], [4, 3, 5], [8, 7, 6]]
    assert min_max_path(grid) == 6
    assert min_max_path_bsearch(grid) == 6


# =========================================================
# FOLLOW-UP 1: The cost is the largest difference between adjacent cells on the path (LeetCode 1631)
# =========================================================
# ---- follow-up 1: function ----
# Answer: the same Dijkstra with max, but a step costs the absolute
# height DIFFERENCE instead of the cell value. (The binary-search
# version also works: allow an edge only if its difference is at most the limit.)
def min_effort_path(H):
    rows, cols = len(H), len(H[0])
    best = {(0, 0): 0}
    heap = [(0, 0, 0)]
    while heap:
        cost, r, c = heapq.heappop(heap)
        if cost > best[(r, c)]:
            continue
        if (r, c) == (rows - 1, cols - 1):
            return cost
        for nr, nc in neighbors(r, c, rows, cols):
            step = H[nr][nc] - H[r][c]
            if step < 0:
                step = -step
            new_cost = cost
            if step > new_cost:
                new_cost = step
            if new_cost < best.get((nr, nc), INF):
                best[(nr, nc)] = new_cost
                heapq.heappush(heap, (new_cost, nr, nc))
```

```
    return None


# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert min_effort_path([[1, 2, 2], [3, 8, 2], [5, 3, 5]]) == 2


# =========================================================
# FOLLOW-UP 2: Water rises over time (the cell is usable from time H[r][c])
# =========================================================
# Answer: same as the main question: you may stand on a cell once the
# time reaches its height, so the answer is min_max_path.


# =========================================================
# FOLLOW-UP 3: Return the path, not only the value
# =========================================================
# Answer: min_max_path(H, with_path=True) stores each cell's parent and
# walks back from the goal.


# =========================================================
# RUN EVERYTHING
# =========================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

52 Find minimum threshold enabling grid path

HARD

Template fit: the same minimax loop: the smallest threshold that lets you walk from corner to corner

QUESTION

You are given a grid of heights. Find the smallest threshold T such that you can walk from the top-left cell to the bottom-right cell, moving up, down, left or right, using only cells of height at most T. Return T together with one path that achieves it.

Function signature

def min_max_path(H: list) -> (int, list)

EXAMPLES

Example 1	
Input	`[[1, 3], [2, 2]]`
Output	`(2, [(0, 0), (1, 0), (1, 1)])`
Why	`Going down then right uses cells 1, 2, 2: the largest is 2, and no path stays below that.`

Example 2	
Input	`[[7]]`
Output	`(7, [(0, 0)])`
Why	`A single cell: the threshold is its own height.`

FULL CODE | main question first, then every follow-up, tests at the end

```
# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

import heapq

INF = float("inf")


# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
def neighbors(r, c, rows, cols):
    # the cells above, below, left and right of (r, c) that exist
    result = []
    for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)):
        nr, nc = r + dr, c + dc
        if 0 <= nr < rows and 0 <= nc < cols:
            result.append((nr, nc))
    return result


def bottleneck_path(H, use_difference):
    # Dijkstra where a path costs the MAX of its steps (not their sum). A
    # step costs the height of the cell entered, or (use_difference) the
    # height difference between the two cells. Returns (cost, path).
    rows, cols = len(H), len(H[0])
    start_cost = 0
    if not use_difference:
        start_cost = H[0][0]
    best = {(0, 0): start_cost}
    parent = {}
    heap = [(start_cost, 0, 0)]
    while heap:
        cost, r, c = heapq.heappop(heap)
        if cost > best[(r, c)]:
            continue                                    # an outdated entry
        if (r, c) == (rows - 1, cols - 1):
            path = [(r, c)]
            while (r, c) in parent:
                r, c = parent[(r, c)]
                path.append((r, c))
            path.reverse()
            return cost, path
        for nr, nc in neighbors(r, c, rows, cols):
            if use_difference:
                step = H[nr][nc] - H[r][c]
                if step < 0:
                    step = -step
            else:
                step = H[nr][nc]
            new_cost = cost
            if step > new_cost:
                new_cost = step
            if new_cost < best.get((nr, nc), INF):
                best[(nr, nc)] = new_cost
                parent[(nr, nc)] = (r, c)
                heapq.heappush(heap, (new_cost, nr, nc))
    return None, []


# ---- main question: main function ----
```

```python
# Idea: Dijkstra with cost = the highest cell on the path. The first
# time a cell leaves the heap its cost is final. (Water rising over
# time is the same problem: a cell is usable once the water reaches
# its height.) Returns (threshold, path).
def min_max_path(H):
    return bottleneck_path(H, False)


# ---- main question: tests ----
def test_main_question():
    H = [[0, 1, 2], [4, 3, 5], [8, 7, 6]]
    cost, path = min_max_path(H)
    assert cost == 6
    assert path[0] == (0, 0) and path[-1] == (2, 2)
    highest = 0
    for r, c in path:
        if H[r][c] > highest:
            highest = H[r][c]
    assert highest == cost
    assert min_max_path([[5]]) == (5, [(0, 0)])


# ============================================================
# FOLLOW-UP 1: The cost is the largest difference between adjacent cells on the path (LeetCode 1631)
```

```python
# ============================================================
# ---- follow-up 1: function ----
# Answer: the same Dijkstra with max, but a step costs the absolute
# height DIFFERENCE instead of the cell value. (A binary search also
# works: allow an edge only if its difference is at most the limit.)
def min_effort_path(H):
    return bottleneck_path(H, True)


# ---- follow-up 1: tests ----
def test_follow_up_1():
    effort, path = min_effort_path([[1, 2, 2], [3, 8, 2], [5, 3, 5]])
    assert effort == 2
    assert min_effort_path([[7]]) == (0, [(0, 0)])


# ============================================================
# FOLLOW-UP 2: Water rises over time (the cell is usable from time H[r][c])
# ============================================================
# Answer: same as the main question: you may stand on a cell once the
# time reaches its height, so the answer is the highest cell on the
# best path, which is exactly min_max_path.


# ============================================================
# FOLLOW-UP 3: Return the path, not only the value
# ============================================================
# Answer: already returned: every improvement stores its parent cell,
# and the path is rebuilt by walking back from the goal.


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
```

```
    run_all_tests()
```

53 Minimize travel time with optional meeting point — HARD

Template fit: three runs: from a, from b, and from the destination on the reversed graph; answer = min over meeting points m of max(da, db) + dd

QUESTION

A weighted graph has n nodes and edges (u, v, w) (undirected unless directed=True). Two people start at nodes a and b and both must end at dest. They may meet at some node m and walk the rest of the way together, so the total time is max(time a to m, time b to m) + time m to dest. Return [the minimum total time, the meeting node] (or [-1, -1] when dest cannot be reached). Meeting at dest itself, or at a or b, is allowed.

Function signature

def meeting_point(n: int, edges: list, a: int, b: int, dest: int, directed=False) -> list

EXAMPLES

Example 1	
Input	`5, [(0, 1, 2), (3, 1, 1), (1, 4, 5)], 0, 3, 4`
Output	`[7, 1]`
Why	`They meet at node 1: a needs 2, b needs 1, then 5 more to node 4: 2 + 5 = 7. Meeting at node 4 itself also takes 7, and the smaller node number is reported.`

Example 2	
Input	`3, [(0, 1, 1)], 0, 1, 2`
Output	`[-1, -1]`
Why	`Node 2 is not connected to anything.`

FULL CODE | main question first, then every follow-up, tests at the end

```
# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

import heapq

INF = float("inf")


# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
def dijkstra(graph, source):
    # shortest distance from source to every node (INF if unreachable)
    dist = [INF] * len(graph)
    dist[source] = 0
    heap = [(0, source)]
    while heap:
        d, u = heapq.heappop(heap)
        if d > dist[u]:
            continue
        for v, w in graph[u]:
            if d + w < dist[v]:
                dist[v] = d + w
                heapq.heappush(heap, (d + w, v))
```

```
    return dist


def build(n, edges, directed):
    # the forward graph, and the reversed graph (used to get distances TO the destination)
    forward = []
    reverse = []
    for i in range(n):
        forward.append([])
        reverse.append([])
    for u, v, w in edges:
        forward[u].append((v, w))
        reverse[v].append((u, w))
        if not directed:
            forward[v].append((u, w))
            reverse[u].append((v, w))
```

```
    return forward, reverse


# ---- main question: main function ----
# Idea: both travellers start together, meet at some node m (the one
# who arrives first waits), then travel together to dest. Run Dijkstra
# from A, from B, and from dest on the reversed graph (distance from
# every node TO dest). The best meeting node minimises
# max(distA, distB) + distance to dest. Meeting AT dest is the same as
# "no meeting". Returns [total time, meeting node] or [-1, -1].
def meeting_point(n, edges, a, b, dest, directed=False):
    forward, reverse = build(n, edges, directed)
    dist_a = dijkstra(forward, a)
    dist_b = dijkstra(forward, b)
    to_dest = dijkstra(reverse, dest)
    best_total = INF
    best_node = -1
    for m in range(n):
        wait_for_both = dist_a[m]
        if dist_b[m] > wait_for_both:
            wait_for_both = dist_b[m]
        total = wait_for_both + to_dest[m]
        if total < best_total:
            best_total = total
            best_node = m
    if best_total == INF:
        return [-1, -1]
    return [best_total, best_node]


# ---- main question: tests ----
STAR = [(0, 2, 1), (1, 2, 2), (2, 3, 4), (0, 3, 9), (1, 3, 9)]


def test_main_question():
    assert meeting_point(4, STAR, 0, 1, 3) == [6, 2]        # meet at 2: max(1, 2) + 4 = 6; alone costs 9
    assert meeting_point(3, [(0, 1, 1)], 0, 1, 2) == [-1, -1]   # nobody can reach the destination
    assert meeting_point(4, STAR, 0, 0, 3)[0] == 5          # same start: they walk together
    assert meeting_point(3, [(0, 1, 1), (1, 2, 1)], 0, 1, 2, True) == [2, 1]   # one-way roads


# ========================================================
# FOLLOW-UP 1: More than two travellers
# ========================================================
# ---- follow-up 1: function ----
```

```
# Answer: one Dijkstra per traveller and one from the destination; for
# each node the group waits for the last traveller, then shares the
# last leg. (If the cost is the total distance walked, add the
```

```
# distances instead of taking the maximum.)
def meeting_point_many(n, edges, starts, dest, directed=False):
    forward, reverse = build(n, edges, directed)
    all_dists = []
    for s in starts:
        all_dists.append(dijkstra(forward, s))
    to_dest = dijkstra(reverse, dest)
    best_total = INF
    best_node = -1
    for m in range(n):
        wait_for_all = 0
        for dist in all_dists:
            if dist[m] > wait_for_all:
                wait_for_all = dist[m]
        if wait_for_all + to_dest[m] < best_total:
            best_total = wait_for_all + to_dest[m]
            best_node = m
    if best_total == INF:
        return [-1, -1]
    return [best_total, best_node]


# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert meeting_point_many(4, STAR, [0, 1, 1], 3) == [6, 2]


# ========================================================
# FOLLOW-UP 2: Meeting is optional
# ========================================================
# Answer: already covered: meeting at the destination costs
# max(distA, distB) + 0, and each traveller going alone is just
# dist(a, dest); take the cheaper of "meet somewhere" and "go alone".


# ========================================================
# FOLLOW-UP 3: Edge weights change over time (traffic)
# ========================================================
# Answer: the edge cost becomes a function of the time you enter it.
# Dijkstra still works as long as leaving later never lets you arrive
# earlier (the FIFO property); otherwise search over (node, time).
```

```
# ========================================================
# RUN EVERYTHING
# ========================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

54 Compute minimax grid path and network delay

HARD

Template fit: combine = max(cost, height of the next cell); the answer is the cost at the bottom-right cell

QUESTION

Two tasks. (1) minimax_path(H): given a grid of heights, find a path from the top-left to the bottom-right cell (moving up, down, left or right) whose largest height is as small as possible, and return that largest height. (2) network_delay(n, edges, source, node_delay=None): a signal starts at source and travels along directed weighted edges (u, v, w); return the time until every node has received it, or -1 if some node never does. node_delay[u], when given, is a pause at node u before it forwards the signal.

Function signature

def minimax_path(H: list) -> int

def network_delay(n: int, edges: list, source: int, node_delay=None) -> int

EXAMPLES

Example 1	
Input	`[[1, 5, 1], [2, 2, 5]]`
Output	`5`
Why	`Both ways around pass a cell of height 5, so the answer is 5.`

Example 2	
Input	`4, [(0, 1, 2), (1, 2, 3), (0, 3, 10)], 0`
Output	`10`
Why	`The signal reaches node 1 at 2, node 2 at 5, node 3 at 10: everyone has it after 10.`

Example 3	
Input	`3, [(0, 1, 1)], 0`
Output	`-1`
Why	`Node 2 has no incoming edge: it is never reached.`

Example 4	
Input	`4, [(0, 1, 2), (1, 2, 3), (0, 3, 10)], 0, node_delay = [1, 0, 0, 0]`
Output	`11`
Why	`With a pause of 1 at node 0, every arrival is 1 later: node 3 is reached at 11.`

FULL CODE | main question first, then every follow-up, tests at the end

```
# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

import heapq

INF = float("inf")


# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
def neighbors(r, c, rows, cols):
    # the cells above, below, left and right of (r, c) that exist
```

```
    result = []
    for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)):
        nr, nc = r + dr, c + dc
        if 0 <= nr < rows and 0 <= nc < cols:
            result.append((nr, nc))
    return result


# ---- main question: main function ----
# PART 1: walk from the top-left to the bottom-right so that the
# HIGHEST cell on the path is as low as possible.
# Idea: Dijkstra where the cost of a path is its maximum (not a sum).
# Heap of (highest cell so far, cell); pop the smallest.
def minimax_path(H):
    rows, cols = len(H), len(H[0])
    best = {(0, 0): H[0][0]}
    heap = [(H[0][0], 0, 0)]
    while heap:
        cost, r, c = heapq.heappop(heap)
        if cost > best[(r, c)]:
            continue
        if (r, c) == (rows - 1, cols - 1):
            return cost
        for nr, nc in neighbors(r, c, rows, cols):
            new_cost = cost
            if H[nr][nc] > new_cost:
                new_cost = H[nr][nc]                # the cost of a path is its maximum, never a sum
```

```
            if new_cost < best.get((nr, nc), INF):
                best[(nr, nc)] = new_cost
                heapq.heappush(heap, (new_cost, nr, nc))
    return -1


# PART 2: when does a signal sent from `source` reach every node?
# Plain Dijkstra; -1 if some node is never reached. `node_delay` is
# an optional processing delay paid when leaving each node (follow-up 3).
def network_delay(n, edges, source, node_delay=None):
    graph = []
    for i in range(n):
        graph.append([])
    for u, v, w in edges:
        graph[u].append((v, w))
    dist = [INF] * n
    dist[source] = 0
    heap = [(0, source)]
    while heap:
        d, u = heapq.heappop(heap)
        if d > dist[u]:
            continue
        pause = 0
        if node_delay:
            pause = node_delay[u]
        for v, w in graph[u]:
            if d + pause + w < dist[v]:
                dist[v] = d + pause + w
                heapq.heappush(heap, (dist[v], v))
    worst = 0
    for d in dist:
        if d > worst:
            worst = d
    if worst == INF:
        return -1
    return worst
```

```
# ---- main question: tests ----
def test_main_question():
    H = [[0, 2, 1], [1, 3, 1], [2, 2, 1]]
    assert minimax_path(H) == 2
    assert minimax_path([[5]]) == 5
    edges = [(0, 1, 2), (1, 2, 3), (0, 2, 10)]
    assert network_delay(3, edges, 0) == 5
```

```
    assert network_delay(4, edges, 0) == -1              # node 3 cannot be reached


# ========================================================
# FOLLOW-UP 1: Part 1 with a union-find (sort cells, add them one by one)
# ========================================================
# ---- follow-up 1: helper functions ----
class UnionFind:
    def __init__(self):
        self.parent = {}

    def find(self, x):
        while self.parent[x] != x:
            self.parent[x] = self.parent[self.parent[x]]
            x = self.parent[x]
        return x


# ---- follow-up 1: function ----
# Answer: add the cells from the lowest to the highest and join each
# new cell to its neighbours that are already added. The moment the
# start and the goal are in one group, that height is the answer:
# O(mn log mn) for the sort.
def minimax_path_union_find(H):
    rows, cols = len(H), len(H[0])
    cells = []
    for r in range(rows):
        for c in range(cols):
            cells.append((H[r][c], r, c))
    cells.sort()
    uf = UnionFind()
    for h, r, c in cells:
        uf.parent[(r, c)] = (r, c)
        for nr, nc in neighbors(r, c, rows, cols):
            if (nr, nc) in uf.parent:
                uf.parent[uf.find((r, c))] = uf.find((nr, nc))
        start, goal = (0, 0), (rows - 1, cols - 1)
        if start in uf.parent and goal in uf.parent and uf.find(start) == uf.find(goal):
            return h
    return -1


# ---- follow-up 1: tests ----
def test_follow_up_1():
    H = [[0, 2, 1], [1, 3, 1], [2, 2, 1]]
```

```
    assert minimax_path_union_find(H) == minimax_path(H)
    assert minimax_path_union_find([[5]]) == 5


# ========================================================
# FOLLOW-UP 2: Part 2: weights are huge
# ========================================================
# Answer: nothing changes: Python integers do not overflow. In a language
# with fixed-size integers use 64 bits and a safe "infinity" value.


# ========================================================
```

```python
# FOLLOW-UP 3: Part 2: the signal can be blocked at some nodes, or each node adds a processing delay
# ========================================================
# ---- follow-up 3: tests ----
# Answer: a processing delay is added when leaving a node (the
# `node_delay` parameter); a blocked node is simply never relaxed.
def test_follow_up_3():
    big = [(0, 1, 10 ** 9), (1, 2, 10 ** 9)]
    assert network_delay(3, big, 0) == 2 * 10 ** 9
    assert network_delay(3, big, 0, [1, 1, 1]) == 2 * 10 ** 9 + 2


# ========================================================
# RUN EVERYTHING
# ========================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

Backtracking: choose, recurse, undo

55 Mahjong Winning Hand Detection (14 Tiles) — MEDIUM

Template fit: take the smallest tile; try it as a triple, then as the start of a run; undo the counts after each try

QUESTION

Mountain View onsite coding round 1: given a hand of 14 tiles, decide whether it is a winning mahjong hand. The candidate must clarify the rule set (typically: 4 melds of three identical tiles or three consecutive ranks + 1 pair).

Input: array of 14 integers representing tile values (clarify suit / range during the interview).
A winning hand consists of:

One pair (two identical tiles), and
Four melds, each being either:

A triplet (3 identical tiles), or
A run (3 consecutive ranks in the same suit).

Minority variant: A newer version instead specifies one pair plus two melds while retaining a 14-tile input. Because that accounts for only eight tiles, clarify the intended meld count and how the remaining six tiles should be handled before coding.

Output: boolean.

EXAMPLES

Example 1	
Input	`[1, 1, 1, 2, 3, 4, 11, 12, 13, 21, 21, 21, 31, 31]`
Output	`true`
Why	`Tiles are numbers: 1-9 one suit, 11-19 a second suit, 21-29 a third, 31 and up honours. Pair 31-31, triplets 1-1-1 and 21-21-21, and sequences 2-3-4 and 11-12-13.`

Example 2	
Input	`[1, 1, 1, 2, 3, 4, 11, 12, 13, 21, 21, 21, 31, 32]`
Output	`false`
Why	`31 and 32 are different honours, so there is no pair to finish the hand.`

FULL CODE | main question first, then every follow-up, tests at the end

```
# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ========================================================

# ========================================================
# MAIN QUESTION
# ========================================================
# ---- main question: helper functions ----
# A tile is a number: suit * 10 + rank for the numbered suits (suits 0 to
# 2, ranks 1 to 9), and 30 or more for honour tiles, which never form
# runs. A winning hand is ONE pair plus `melds` melds (4 melds: 14 tiles).
# A meld is a triplet (three equal tiles) or a run (three consecutive
# ranks of one suit).
def is_run_start(tile, count):
    if tile >= 30 or tile % 10 > 7:
```

```
        return False                                # honours never run; a run cannot start at rank 8 or 9
    return count.get(tile + 1, 0) > 0 and count.get(tile + 2, 0) > 0


def smallest_tile(count):
    smallest = None
    for tile in count:
        if count[tile] > 0 and (smallest is None or tile < smallest):
            smallest = tile
    return smallest


def melds_left(count, need):
    # can the tiles in `count` (tile -> how many) form exactly `need` melds?
    tile = smallest_tile(count)
    if tile is None:
        return need == 0
    if need == 0:
        return False
    # the smallest tile is either in a triplet ...
    if count[tile] >= 3:
        count[tile] -= 3
        ok = melds_left(count, need - 1)
        count[tile] += 3                            # undo
        if ok:
            return True
    # ... or the start of a run of three
```

```
    if is_run_start(tile, count):
        for t in (tile, tile + 1, tile + 2):
            count[t] -= 1
        ok = melds_left(count, need - 1)
        for t in (tile, tile + 1, tile + 2):
            count[t] += 1                           # undo
        return ok
    return False


# ---- main question: main function ----
# Idea: count the tiles. Try every tile value that has at least two tiles
# as the PAIR; then the rest must form `melds` melds: always take the
# SMALLEST remaining tile and try the (at most two) ways to use it.
# melds=2 is the smaller variant with a pair and two melds, 8 tiles
# (follow-up 1).
def is_winning_hand(hand, melds=4):
    if len(hand) != 2 + 3 * melds:
        return False                                # one pair plus the melds must use every tile
    count = {}
    for tile in hand:
        count[tile] = count.get(tile, 0) + 1
    for tile in list(count):
        if count[tile] >= 2:
            count[tile] -= 2
            ok = melds_left(count, melds)
            count[tile] += 2
            if ok:
                return True
    return False


# ---- main question: tests ----
WIN = [1, 1, 1, 2, 3, 4, 11, 12, 13, 21, 22, 23, 31, 31]


def test_main_question():
    assert is_winning_hand(WIN)
```

```
    lose = [1, 1, 1, 2, 3, 5, 11, 12, 13, 21, 22, 23, 31, 31]      # 2, 3, 5 is not a run
    assert not is_winning_hand(lose)
    honours = [30, 30, 31, 31, 31, 32, 32, 32, 33, 33, 33, 34, 34, 34]       # honours only make triplets or
the pair
    assert is_winning_hand(honours)
    assert not is_winning_hand(WIN[:13])                                      # a wrong tile count is never a
 win
```

```
# ============================================================
# FOLLOW-UP 1: A variant needs only two melds plus a pair (8 tiles of 14)
# ============================================================
# ---- follow-up 1: tests ----
# Answer: ask what happens to the other six tiles before coding. If they
# are simply not used, check whether ANY 8 of the 14 tiles form a pair and
# two melds (the same search, tried over the subsets). If the hand is
# only 8 tiles long, the `melds=2` parameter above is all that changes.
def test_follow_up_1():
    assert is_winning_hand([5, 5, 1, 2, 3, 7, 7, 7], 2)
    assert not is_winning_hand([5, 5, 1, 2, 4, 7, 7, 7], 2)


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

56 Count Prefix-Matching Simple Paths in a Labeled Graph

MEDIUM

Template fit: extend the path along edges whose label matches the next query letter, with a bitmask of visited nodes; count per depth

QUESTION

A directed graph has labelled vertices (labels[i] is the label of vertex i) and edges (u, v). A simple path never visits a vertex twice. For a query word, count for every prefix length L (1 up to the length of the word) the number of simple paths with L vertices whose labels spell the first L letters of the word. Return the list of counts.

Function signature

def prefix_paths(labels: list, edges: list, query: str) -> list

EXAMPLES

Example 1	
Input	`['a', 'b', 'c', 'a'], [(0, 1), (1, 2), (3, 1)], 'abc'`
Output	`[2, 2, 2]`
Why	`Two paths spell 'a' (vertices 0 and 3), and both continue to b (vertex 1) and then c (vertex 2).`

Example 2	
Input	`['a', 'a'], [(0, 1), (1, 0)], 'aaa'`
Output	`[2, 2, 0]`
Why	`Two paths spell 'a' and two spell 'aa' (0 -> 1 and 1 -> 0); none spells 'aaa' because that would reuse a vertex.`

FULL CODE | main question first, then every follow-up, tests at the end

```
# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ========================================================

# ========================================================
# MAIN QUESTION
# ========================================================
# ---- main question: main function ----
# Every vertex has a label (a letter). A SIMPLE path never visits a vertex
# twice. counts[d] is the number of simple paths whose first d + 1
# labels spell the first d + 1 letters of the query.
# Idea: DFS with pruning. Start from every vertex whose label is the first
# query letter. From a vertex go only to an UNVISITED neighbour whose
# label is the NEXT query letter; add one to the count for the current
# length at each step, and undo the visit when coming back.
# `undirected=True` lets every edge be used in both directions (follow-up 1).
def prefix_paths(labels, edges, query, undirected=False):
    adj = {}
    for v in range(len(labels)):
        adj[v] = []
    for u, v in edges:
        adj[u].append(v)
        if undirected:
            adj[v].append(u)
    counts = [0] * len(query)
    visited = set()

    def go(u, depth):
        counts[depth] += 1
        if depth + 1 == len(query):
            return
        for w in adj[u]:
            if w in visited:
                continue                            # a simple path never revisits a vertex
            if labels[w] == query[depth + 1]:
                visited.add(w)
                go(w, depth + 1)
                visited.remove(w)                   # undo

    for start in range(len(labels)):
        if len(query) > 0 and labels[start] == query[0]:
            visited.add(start)
            go(start, 0)
            visited.remove(start)
    return counts


# ---- main question: tests ----
LABELS = ["a", "b", "a", "b"]
EDGES = [(0, 1), (1, 2), (2, 3), (3, 0)]              # a directed square: a > b > a > b > back to the first
a


def test_main_question():
    assert prefix_paths(LABELS, EDGES, "aba") == [2, 2, 2]
    assert prefix_paths(LABELS, EDGES, "ababa") == [2, 2, 2, 2, 0]      # the 5th letter would revisit a vert
ex
    assert prefix_paths(LABELS, EDGES, "") == []


# ========================================================
# FOLLOW-UP 1: The edges are undirected
```

```
# ============================================================
# ---- follow-up 1: tests ----
# Answer: add each edge in both directions (the `undirected` parameter).
def test_follow_up_1():
    assert prefix_paths(LABELS, EDGES, "ab", True) == [2, 4]


# ============================================================
# FOLLOW-UP 2: Count only the paths that spell the whole query
# ============================================================
# ---- follow-up 2: function ----
# Answer: the last number of the list is exactly that. For long queries on
# bigger graphs: if vertices MAY repeat (walks, not simple paths), memoise on
# (vertex, depth) or do the DP below; for simple paths the visited set
# prevents memoisation.
def whole_query_paths(labels, edges, query, undirected=False):
    counts = prefix_paths(labels, edges, query, undirected)
    if len(counts) == 0:
        return 0
    return counts[-1]


# ---- follow-up 2: tests ----
def test_follow_up_2():
    assert whole_query_paths(LABELS, EDGES, "aba") == 2
    assert whole_query_paths(LABELS, EDGES, "") == 0
```

```
# ============================================================
# FOLLOW-UP 3: The graph has up to 100,000 vertices
# ============================================================
# ---- follow-up 3: function ----
# Answer: simple paths cannot be counted fast (the visited set is the
# problem). For WALKS (vertices may repeat) a DP over (depth, vertex) is
# enough: ways[v] is the number of walks that spell the query so far and
# end in v. It costs O(length of the query * edges) and works for very
# large graphs.
def prefix_walks(labels, edges, query):
    adj = {}
    for v in range(len(labels)):
        adj[v] = []
    for u, v in edges:
        adj[u].append(v)
    ways = {}
    for v in range(len(labels)):
        if len(query) > 0 and labels[v] == query[0]:
            ways[v] = 1
    counts = []
    if len(query) > 0:
        counts.append(sum(ways.values()))
    for letter in query[1:]:
        following = {}
        for u in ways:
            for w in adj[u]:
                if labels[w] == letter:
                    following[w] = following.get(w, 0) + ways[u]
        ways = following
        counts.append(sum(ways.values()))
    return counts


# ---- follow-up 3: tests ----
def test_follow_up_3():
    assert prefix_walks(LABELS, EDGES, "abab") == [2, 2, 2, 2]
    assert prefix_walks(LABELS, EDGES, "ababa") == [2, 2, 2, 2, 2]     # a walk may reuse vertices, a simple
path may not
```

```
    assert prefix_walks(LABELS, EDGES, "") == []


# ====================================================
# RUN EVERYTHING
# ====================================================
def run_all_tests():
```

```
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

57 Generate Parentheses — MEDIUM

Template fit: choices are '(' and ')'; allowed when opened < n, and ')' only when closed < opened; complete at length 2n

QUESTION

Generate all combinations of well-formed parentheses. A parenthesis combination is well-formed if each opening parenthesis "(" is closed by a matching closing parenthesis ")" in the correct order. For example, "()", "()()", and "((()))" are all combinations of well-formed parentheses, while ")(", "((" and "(()" are not. Examples n = 1 output: ["()"] n = 2 output: ["(())", "()()"] n = 3 output: ["((()))", "(()())", "(())()", "()(())", "()()()"] n = 4 output should contain: ["(((())))", "((()()))", "((())())", "((()))()", "(()(()))", "(()()())", "(()())()", "(())(())", "(())()()", "()((()))", "()(()())", "()(())()", "()()(())", "()()()()"]The problem is to generate all combinations of well-formed parentheses given n pairs of parentheses. A combination is well-formed if every opening parenthesis has a corresponding closing parenthesis and they are properly nested. Recursive Approach The most common and intuitive way to solve this problem is by using recursion with backtracking. Here's a step-by-step breakdown of the approach: Define the Problem: We need to build strings of length 2n using n opening (and n closing) parentheses. At any point in the string, the number of closing parentheses should not exceed the number of opening parentheses to ensure the sequence is valid. Recursive Function: We use a recursive helper function to build the strings character by character. The function tracks the remaining number of opening and closing parentheses that can still be added. It also

EXAMPLES

Example 1	
Input	3
Output	['((()))', '(()())', '(())()', '()(())', '()()()']
Why	Five well-formed combinations of three pairs.

Example 2	
Input	1
Output	['()']
Why	One pair.

FULL CODE | main question first, then every follow-up, tests at the end

```
# ====================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
```

```python
# ============================================================

# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: main function ----
# Idea (backtracking): build the string one character at a time. Add "(" as
# long as fewer than n have been placed; add ")" only while the number
# of ")" is smaller than the number of "(" (so it closes something). That
# rule keeps every string valid. At length 2n the string is complete.
def generate(n):
    result = []

    def build(text, opens, closes):
        if len(text) == 2 * n:
            result.append(text)
            return
        if opens < n:
            build(text + "(", opens + 1, closes)
        if closes < opens:
            build(text + ")", opens, closes + 1)

    build("", 0, 0)
    return result


# ---- main question: tests ----
def test_main_question():
    assert generate(1) == ["()"]
    assert generate(2) == ["(())", "()()"]
    assert sorted(generate(3)) == ["((()))", "(()())", "(())()", "()(())", "()()()"]
    assert generate(0) == [""]


# ============================================================
# FOLLOW-UP 1: Return only the k-th string, or just the number of strings
# ============================================================
# ---- follow-up 1: helper functions ----
MEMO = {}
```

```python

def completions(opens_left, balance):
    # in how many ways can a string be finished when opens_left "(" are
    # still to be placed and `balance` brackets are open (unclosed)?
    key = (opens_left, balance)
    if key in MEMO:
        return MEMO[key]
    if opens_left == 0:
        result = 1                                  # only the closing brackets are left: one way
    else:
        result = completions(opens_left - 1, balance + 1)
        if balance > 0:
            result += completions(opens_left, balance - 1)
    MEMO[key] = result
    return result


# ---- follow-up 1: functions ----
# Answer: the NUMBER of strings is the Catalan number C(n) =
# (2n choose n) / (n + 1), which is completions(n, 0). For the k-th
# string (in sorted order, "(" before ")") count at each step the
# strings that continue with "(" and skip them if k is larger
# (unranking), with a small DP over (opens left, balance).
def catalan(n):
    return completions(n, 0)
```

```
def kth(n, k):
    if k < 1 or k > catalan(n):
        raise ValueError("k is out of range")
    text = ""
    opens_left = n
    balance = 0
    while len(text) < 2 * n:
        ways = 0
        if opens_left > 0:
            ways = completions(opens_left - 1, balance + 1)       # strings that continue with "("
        if opens_left > 0 and k <= ways:
            text += "("
            opens_left -= 1
            balance += 1
        else:
            k -= ways                                  # skip all the strings that start with "("
            text += ")"
            balance -= 1
```

```
    return text


# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert catalan(3) == 5 and catalan(10) == 16796
    for n in range(0, 8):
        assert catalan(n) == len(generate(n))
    for n in range(1, 7):
        everything = sorted(generate(n))
        for k in range(1, len(everything) + 1):
            assert kth(n, k) == everything[k - 1]


# ======================================================
# FOLLOW-UP 2: Several bracket types, such as () [] {}
# ======================================================
# ---- follow-up 2: function ----
# Answer: multiply the choices. Any type may be OPENED (while fewer than n
# pairs were opened) and only the type on TOP of a stack of open brackets
# may be closed.
def generate_multi(n, kinds="()[]{}"):
    pairs = []
    for i in range(0, len(kinds), 2):
        pairs.append(kinds[i:i + 2])
    result = []

    def build(text, stack, opened):
        if len(text) == 2 * n:
            result.append(text)
            return
        if opened < n:
            for open_char, close_char in pairs:
                build(text + open_char, stack + [close_char], opened + 1)
        if len(stack) > 0:
            build(text + stack[-1], stack[:-1], opened)

    build("", [], 0)
    return result


# ---- follow-up 2: tests ----
def test_follow_up_2():
    for n in range(1, 5):                              # Catalan(n) shapes times 3 type choices per pair
        got = generate_multi(n, "()[]{}")
```

```
        assert len(got) == catalan(n) * 3 ** n
        assert len(set(got)) == len(got)
    assert sorted(generate_multi(1, "()[]")) == ["()", "[]"]


# =====================================================
# FOLLOW-UP 3: Avoid recursion
# =====================================================
# ---- follow-up 3: function ----
# Answer: use your own stack of states (text, opens, closes) instead of the
# call stack. The pruning rules stay the same. ")" is pushed first so
# "(" is handled first and the strings come out in sorted order.
def generate_iter(n):
    result = []
    stack = [("", 0, 0)]
    while stack:
        text, opens, closes = stack.pop()
        if len(text) == 2 * n:
            result.append(text)
            continue
        if closes < opens:
            stack.append((text + ")", opens, closes + 1))
        if opens < n:
            stack.append((text + "(", opens + 1, closes))
    return result


# ---- follow-up 3: tests ----
def test_follow_up_3():
    for n in range(0, 8):
        assert generate_iter(n) == sorted(generate(n))


# =====================================================
# RUN EVERYTHING
# =====================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
```

```
    run_all_tests()
```

58 American Football Score Combinations

MEDIUM

Template fit: choices are the scoring plays from index i onward (the same play may repeat); complete when the remaining score is 0

QUESTION

Bryan's DP question was: enumerate all the ways a football team can accumulate a given total number of points. In American football a single scoring play is worth one of: 2 (safety), 3 (field goal), 6 (touchdown), 7 (touchdown + extra point), or 8 (touchdown + 2-point conversion). Given a target total , return every distinct combination of scoring plays whose values sum to total . Two combinations are the same if they use the same multiset of play values (order does not matter). Each combination should be returned as a list of play values sorted in non-decreasing order. The overall list of combinations may be in any order. A play value may be used any number of times.

QUESTION

Function signature

def football_combinations(total: int) -> list[list[int]]:
    """Return all distinct multisets of {2,3,6,7,8} that sum to total, each sorted ascending."""
    ...

Follow-up (as reported): If you only need the count of combinations (not the lists), do it with an O(total) DP table (unbounded coin-change counting).

EXAMPLES

Example 1	
Input	`total = 6`
Output	`[[2, 2, 2], [3, 3], [6]]`
Why	`2+2+2 = 3+3 = 6 = 6.`

Example 2	
Input	`total = 7`
Output	`[[2, 2, 3], [7]]`
Why	`2+2+3 = 7 and the single play 7.`

Example 3	
Input	`total = 0`
Output	`[[]]`
Why	`The empty combination is the only way to score 0.`

CONSTRAINTS

- 0 <= total <= 60
- Allowed play values are exactly [2, 3, 6, 7, 8] .
- If total == 0 , the only combination is the empty one, so return [[]] .

FULL CODE | main question first, then every follow-up, tests at the end

```
# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ========================================================

# ========================================================
# MAIN QUESTION
# ========================================================
# ---- main question: main function ----
# A team can score 2, 3, 6, 7 or 8 points at a time. List every
# combination of plays that adds up to `total`.
# Idea (backtracking): choose the plays in NON-DECREASING order, so every
# combination is listed exactly once (2 + 3 and 3 + 2 are the same
# combination). `start` is the index of the smallest play we may still use;
# the same play may be used again, so the index stays the same after a
# choice. Undo the choice (pop) after the recursive call.
def football_combinations(total):
    plays = (2, 3, 6, 7, 8)
    result = []

    def dfs(start, left, path):
        if left == 0:
```

```
            result.append(path[:])
            return
        for i in range(start, len(plays)):
            if plays[i] <= left:
                path.append(plays[i])
                dfs(i, left - plays[i], path)
                path.pop()

    dfs(0, total, [])
    return result


# ---- main question: tests ----
def test_main_question():
    assert football_combinations(6) == [[2, 2, 2], [3, 3], [6]]
    assert football_combinations(7) == [[2, 2, 3], [7]]
    assert football_combinations(0) == [[]]
    assert football_combinations(1) == []


# =====================================================
```

```
# FOLLOW-UP 1: You only need how many combinations there are
# =====================================================
# ---- follow-up 1: function ----
# Answer: use the unbounded coin DP: ways[0] = 1, and for each play value v,
# ways[t] += ways[t - v] for t from v up to the total. Looping over the
# values on the OUTSIDE counts combinations (not orders). O(total * values)
# time and O(total) space.
def count_combinations(total):
    ways = [1] + [0] * total
    for value in (2, 3, 6, 7, 8):
        for t in range(value, total + 1):
            ways[t] += ways[t - value]
    return ways[total]


# ---- follow-up 1: tests ----
def test_follow_up_1():
    for total in range(0, 19):
        assert count_combinations(total) == len(football_combinations(total))


# =====================================================
# RUN EVERYTHING
# =====================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

59 Permutations

MEDIUM

Template fit: the template as written

QUESTION

Given an array of distinct integers, return all possible permutations of the array (every rearrangement of its elements), in any order.

Function signature

def permute(nums: list) -> list

EXAMPLES

Example 1	
Input	`[1, 2, 3]`
Output	`[[1, 2, 3], [1, 3, 2], [2, 1, 3], [2, 3, 1], [3, 1, 2], [3, 2, 1]]`
Why	`Six orders of three numbers.`

Example 2	
Input	`[0, 1]`
Output	`[[0, 1], [1, 0]]`
Why	`Two numbers have two orders.`

Example 3	
Input	`[1]`
Output	`[[1]]`
Why	`One number has a single permutation.`

FULL CODE | main question first, then every follow-up, tests at the end

```
# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: main function ----
# Idea (backtracking): fill the positions from left to right. At each
# position try every value that is not used yet: choose it (mark it used,
# put it in the current list), recurse for the next position, then UNDO
# the choice (remove it, unmark it). Only valid prefixes are ever
# extended, so every complete list is a permutation.
def permute(nums):
    result = []
    current = []
    used = [False] * len(nums)

    def go():
        if len(current) == len(nums):
            result.append(current[:])              # a copy, because `current` keeps changing
            return
        for i in range(len(nums)):
            if not used[i]:
                used[i] = True
                current.append(nums[i])
                go()
```

```
                current.pop()
                used[i] = False

    go()
    return result


# ---- main question: tests ----
def test_main_question():
    assert permute([1, 2, 3]) == [[1, 2, 3], [1, 3, 2], [2, 1, 3], [2, 3, 1], [3, 1, 2], [3, 2, 1]]
    assert permute([0, 1]) == [[0, 1], [1, 0]]
    assert permute([1]) == [[1]]
    assert permute([]) == [[]]
```

```
# =========================================================
# FOLLOW-UP 1: The input has duplicates (Permutations II)
# =========================================================
# ---- follow-up 1: function ----
# Answer: sort first, and skip a value that equals the previous one unless
# the previous one is already used in the current list. That forces equal
# values to appear in a fixed order, which removes the duplicate lists
# without a set.
def permute_unique(nums):
    nums = sorted(nums)
    result = []
    current = []
    used = [False] * len(nums)

    def go():
        if len(current) == len(nums):
            result.append(current[:])
            return
        for i in range(len(nums)):
            if used[i]:
                continue
            if i > 0 and nums[i] == nums[i - 1] and not used[i - 1]:
                continue                            # the same value as before and it is not in use
            used[i] = True
            current.append(nums[i])
            go()
            current.pop()
            used[i] = False

    go()
    return result


# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert permute_unique([1, 1, 2]) == [[1, 1, 2], [1, 2, 1], [2, 1, 1]]
    assert permute_unique([2, 2]) == [[2, 2]]


# =========================================================
# FOLLOW-UP 2: Return the next lexicographic permutation in place
# =========================================================
# ---- follow-up 2: function ----
# Answer: find the LAST position i with a[i] < a[i + 1], swap a[i] with the
# smallest bigger value to its right (the last bigger one, because the
```

```
# part after i is in decreasing order), then reverse the part after i so
# it is the smallest. O(n) time, O(1) extra space. Returns False (and
# wraps to the smallest order) when it was the last permutation.
def next_permutation(a):
    i = len(a) - 2
```

```
    while i >= 0 and a[i] >= a[i + 1]:
        i -= 1                                  # find the last position that can still increase
    if i < 0:
        a.reverse()
        return False
    j = len(a) - 1
    while a[j] <= a[i]:
        j -= 1
    a[i], a[j] = a[j], a[i]
    a[i + 1:] = a[i + 1:][::-1]                 # the part after i is decreasing: make it the smallest
    return True


# ---- follow-up 2: tests ----
def test_follow_up_2():
    a = [1, 2, 3]
    seen = [a[:]]
    while next_permutation(a):
        seen.append(a[:])
    assert seen == permute([1, 2, 3])           # repeating it lists all permutations in order
    assert a == [1, 2, 3]                       # and wraps around at the end


# =========================================================
# FOLLOW-UP 3: Return only the k-th permutation of 1..n
# =========================================================
# ---- follow-up 3: helper functions ----
def factorial(n):
    result = 1
    for i in range(2, n + 1):
        result *= i
    return result


# ---- follow-up 3: function ----
# Answer: the factorial number system. The permutations of 1..n come in
# blocks of (n - 1)!, so the first value is the unused value at index
# (k - 1) // (n - 1)!; continue with the remainder on the remaining
# values. O(n^2), no listing.
def kth_permutation(n, k):
    digits = list(range(1, n + 1))
    result = []
    k -= 1
    for left in range(n, 0, -1):
        block = factorial(left - 1)
        result.append(digits.pop(k // block))
        k %= block
    return result


# ---- follow-up 3: tests ----
def test_follow_up_3():
    for n in range(1, 6):
        everything = permute(list(range(1, n + 1)))
        for k in range(1, len(everything) + 1):
            assert kth_permutation(n, k) == everything[k - 1]


# =========================================================
# RUN EVERYTHING
# =========================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
```

```
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

60 Determine if a 14-tile hand is winning

MEDIUM

Template fit: the same try-triple-then-run recursion over a count array, after choosing the pair

QUESTION

A hand is 14 tiles with values 1 to 9 of one suit (you may reorder them). It wins if it splits into one pair (two identical tiles) and four melds of three tiles, where a meld is either a triplet (three identical tiles) or a sequence (three consecutive values). Return whether the hand is winning.

Function signature

def is_winning_hand(tiles: list) -> bool

EXAMPLES

Example 1	
Input	`[1, 1, 1, 2, 3, 4, 5, 5, 5, 6, 7, 8, 9, 9]`
Output	`true`
Why	`Pair 9-9, triplets 1-1-1 and 5-5-5, sequences 2-3-4 and 6-7-8.`

Example 2	
Input	`[1, 2, 3, 4, 5, 6, 7, 8, 9, 1, 1, 2, 4, 6]`
Output	`false`
Why	`These tiles cannot be split into a pair and four melds.`

FULL CODE | main question first, then every follow-up, tests at the end

```
# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
# A hand is made of tiles with values 1 to 9 in one suit. It WINS when its
# 14 tiles split into ONE pair (two equal tiles) and FOUR melds. A meld is
# a triplet (three equal tiles) or a run (three consecutive values).
def count_tiles(tiles):
    # counts[v] = how many tiles with value v (index 0 is unused)
    counts = [0] * 12
    for tile in tiles:
        counts[tile] += 1
    return counts


def melds_ok(counts, v=1):
    # can the tiles in counts be split into melds only?
    while v <= 9 and counts[v] == 0:
        v += 1                                      # the smallest value that still has tiles
    if v > 9:
```

```
        return True                                  # no tiles left
    # The smallest tile must be in a triplet or at the START of a run
    # (nothing smaller is left to take it), so only these two choices exist.
    if counts[v] >= 3:
        counts[v] -= 3
        ok = melds_ok(counts, v)
        counts[v] += 3                               # undo
        if ok:
            return True
    if v <= 7 and counts[v + 1] > 0 and counts[v + 2] > 0:
        counts[v] -= 1
        counts[v + 1] -= 1
        counts[v + 2] -= 1
        ok = melds_ok(counts, v)
        counts[v] += 1
        counts[v + 1] += 1
        counts[v + 2] += 1                           # undo
        return ok
```

```
    return False


def is_winning_counts(counts):
    # try every value that has two tiles as the PAIR; the rest must be melds
    for pair in range(1, 10):
        if counts[pair] >= 2:
            counts[pair] -= 2
            ok = melds_ok(counts)
            counts[pair] += 2
            if ok:
                return True
    return False


# ---- main question: main function ----
def is_winning_hand(tiles):
    return is_winning_counts(count_tiles(tiles))


# ---- main question: tests ----
def test_main_question():
    win = [1, 1, 1, 2, 3, 4, 5, 5, 5, 6, 7, 8, 9, 9]
    assert is_winning_hand(win)
    lose = [1, 2, 4, 5, 7, 8, 1, 2, 4, 5, 7, 8, 9, 9]         # 3 and 6 are missing, so the runs cannot be fo
rmed
    assert not is_winning_hand(lose)


# ========================================================
# FOLLOW-UP 1: Which tile would complete a 13-tile hand?
# ========================================================
# ---- follow-up 1: function ----
# Answer: try each of the nine values as the 14th tile and run the same
# check: 9 checks of a tiny search. That is how a player finds the tiles
# they are waiting for.
def waiting_tiles(tiles13):
    waiting = []
    for tile in range(1, 10):
        if is_winning_hand(tiles13 + [tile]):
            waiting.append(tile)
    return waiting


# ---- follow-up 1: tests ----
def test_follow_up_1():
```

```
    wait = [1, 1, 1, 2, 3, 4, 5, 5, 5, 6, 7, 8, 9]
    assert 9 in waiting_tiles(wait)
    assert waiting_tiles(wait) != []


# ======================================================
# FOLLOW-UP 2: More suits or special hands (seven pairs)
# ======================================================
# ---- follow-up 2: functions ----
# Answer: melds never cross suits, so check each suit on its own: exactly
# one suit holds the pair (its tile count is 2 more than a multiple of
# 3), the others must be melds only. Seven pairs is a separate count
# check: 14 tiles, every value exactly twice.
def is_seven_pairs(tiles):
    if len(tiles) != 14:
        return False
    counts = {}
    for tile in tiles:
        counts[tile] = counts.get(tile, 0) + 1
    for tile in counts:
        if counts[tile] != 2:
            return False
    return True


def is_winning_multi(hand):
    # hand is a list of (suit, value)
    suits = {}
    for suit, value in hand:
        if suit not in suits:
            suits[suit] = []
        suits[suit].append(value)
    pair_suits = []
    for suit in suits:
        if len(suits[suit]) % 3 == 2:
            pair_suits.append(suit)
    if len(pair_suits) != 1:
        return False
    for suit in suits:
        counts = count_tiles(suits[suit])
        if suit == pair_suits[0]:
            if not is_winning_counts(counts):
                return False
        elif len(suits[suit]) % 3 != 0 or not melds_ok(counts):
            return False
```

```
    return True


# ---- follow-up 2: tests ----
def test_follow_up_2():
    assert is_seven_pairs([1, 1, 2, 2, 3, 3, 4, 4, 5, 5, 6, 6, 7, 7])
    assert not is_seven_pairs([1, 1, 1, 1, 2, 2, 3, 3, 4, 4, 5, 5, 6, 6])
    hand = [("m", v) for v in (1, 1, 1, 2, 3)] + [("p", v) for v in (4, 5, 6, 7, 7, 7)] + [("s", v) for v in
(2, 2, 2)]
    assert is_winning_multi(hand)
    assert not is_winning_multi(hand[:-1])


# ======================================================
# FOLLOW-UP 3: Why is the "smallest remaining value" rule enough?
# ======================================================
# Answer: the smallest tile can only be in a triplet or at the START of a
# run, because a run containing it as its middle or end would need a
# smaller tile, and none is left. So at every step there are at most two
# choices, and a decision for the smallest tile never has to be reconsidered.
```

```python
# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

61 Count Passcode Patterns on a Keypad

MEDIUM

Template fit: the choice is the next adjacent unused key; complete at k keys; count instead of storing

QUESTION

A keypad is a rows x cols grid of keys (the classic phone dial is 3x3):
1 2 3 4 5 6 7 8 9 A passcode pattern is a path across keys where each move goes only to an orthogonally adjacent key (horizontal or vertical — no diagonals), and no key is visited more than once. The order matters: 1 -> 2 -> 3 -> 6 and 6 -> 3 -> 2 -> 1 are different patterns. Given the grid dimensions and an integer k , return the number of distinct passcode patterns that use exactly k keys.

Function signature

def count_patterns(rows: int, cols: int, k: int) -> int:
"""Return the number of distinct passcode patterns using exactly k keys,
moving only to orthogonally adjacent, unvisited keys on a rows x cols grid."""
...

Follow-up (as reported): Count patterns of length at least k instead of exactly k . Hint: sum the DFS counts over all lengths from k to rows*cols .

EXAMPLES

Example 1	
Input	`rows = 3, cols = 3, k = 1`
Output	`9`
Why	`Each of the 9 keys alone is one pattern.`

Example 2	
Input	`rows = 3, cols = 3, k = 2`
Output	`24`
Why	`There are 12 adjacent key pairs; each can be traversed in 2 directions, giving 24 patterns.`

Example 3	
Input	`rows = 3, cols = 3, k = 3`
Output	`44`
Why	`Summing over each possible middle key b, deg(b)*(deg(b)-1) directed 3-key paths: corners give 4*2=8, edge-mids 4*6=24, center 12; total 44.`

CONSTRAINTS

- 1 <= rows, cols <= 4 .
- 1 <= k <= rows * cols .

QUESTION

- A single key (k = 1) is a valid pattern of length 1.
- Count directed patterns (a path and its reverse are counted separately).

FULL CODE | main question first, then every follow-up, tests at the end

```
# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ========================================================

# ========================================================
# MAIN QUESTION
# ========================================================
# ---- main question: helper functions ----
def neighbours(r, c, rows, cols):
    # the keys right next to key (r, c) on the keypad
    result = []
    for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)):
        nr = r + dr
        nc = c + dc
        if 0 <= nr < rows and 0 <= nc < cols:
            result.append((nr, nc))
    return result


# ---- main question: main function ----
# Count the patterns of exactly k keys: start on any key and keep moving to
# an ADJACENT key that is not used yet.
# Idea (DFS with backtracking): from the current key try every unused
# neighbour: mark it used, recurse with one more key, then UNMARK it.
# A pattern is complete when it has k keys.
def count_patterns(rows, cols, k):
    def dfs(r, c, used, seen):
        if used == k:
            return 1
        total = 0
        for nr, nc in neighbours(r, c, rows, cols):
            if (nr, nc) not in seen:
                seen.add((nr, nc))
                total += dfs(nr, nc, used + 1, seen)
                seen.remove((nr, nc))                # undo
        return total

    total = 0
    for r in range(rows):
        for c in range(cols):
            total += dfs(r, c, 1, {(r, c)})
    return total
```

```
# ---- main question: tests ----
def test_main_question():
    assert count_patterns(3, 3, 1) == 9
    assert count_patterns(3, 3, 2) == 24                # one per direction of every edge
    assert count_patterns(3, 3, 3) == 44


# ========================================================
# FOLLOW-UP 1: Count patterns that use at least k keys
# ========================================================
# ---- follow-up 1: function ----
```

```python
# Answer: the same DFS, but count one at every depth that is at least k
# and keep extending instead of stopping at k.
def count_patterns_at_least(rows, cols, k):
    def dfs(r, c, used, seen):
        total = 0
        if used >= k:
            total = 1
        for nr, nc in neighbours(r, c, rows, cols):
            if (nr, nc) not in seen:
                seen.add((nr, nc))
                total += dfs(nr, nc, used + 1, seen)
                seen.remove((nr, nc))
        return total

    total = 0
    for r in range(rows):
        for c in range(cols):
            total += dfs(r, c, 1, {(r, c)})
    return total


# ---- follow-up 1: tests ----
def test_follow_up_1():
    for rows, cols in ((1, 4), (2, 2), (2, 3), (3, 3)):
        for k in range(1, 4):
            expected = 0
            for length in range(k, rows * cols + 1):
                expected += count_patterns(rows, cols, length)
            assert count_patterns_at_least(rows, cols, k) == expected


# ============================================================
```

```python
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

62 Match people to questions using tags and priorities

MEDIUM

Template fit: try each question of a person; if it is taken, recursively ask its owner to move (augmenting path); undo by not recording on failure

QUESTION

A set of people and a set of interview questions: adj[p] lists the questions person p is allowed to answer (their tags match). Each person is given at most one question and each question goes to at most one person. Return the largest number of people that can be matched.

Function signature

def max_matching(adj: list) -> int

EXAMPLES

QUESTION

Example 1	
Input	`[[0, 1], [0], [1, 2]]`
Output	`3`
Why	`Person 0 takes question 0, person 1 has only question 0 so person 0 moves to question 1, person 2 takes question 2: all three are matched.`

Example 2	
Input	`[[0], [0], [0]]`
Output	`1`
Why	`Three people can only answer the same single question: just one match.`

FULL CODE | main question first, then every follow-up, tests at the end

```python
# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
def build_graph(people, questions):
    # a person can take a question when they share at least one tag.
    # people: [(name, priority, tags)]   questions: [(id, tags, capacity)]
    adj = {}
    for name, priority, tags in people:
        adj[name] = []
        for question_id, question_tags, capacity in questions:
            if len(set(tags) & set(question_tags)) > 0:
                adj[name].append(question_id)
    return adj


# ---- main question: main function ----
# Idea (Kuhn's augmenting paths): adj[p] is the list of questions person p
# accepts. For each person run a DFS over the questions they accept: if a
# question is free, take it; if it belongs to someone else, try to move
# that owner to ANOTHER question (a recursive try). A `seen` set makes sure
# no question is tried twice in one attempt. The number of people who
# found a question is the size of the maximum matching.
def max_matching(adj):
    owner = {}                                      # question -> the person who holds it

    def try_assign(person, seen):
        for question in adj[person]:
            if question in seen:
                continue
            seen.add(question)
            if question not in owner or try_assign(owner[question], seen):
                owner[question] = person
                return True
        return False

    matched = 0
    for person in range(len(adj)):
```

```
        if try_assign(person, set()):
            matched += 1
    return matched


# ---- main question: tests ----
def test_main_question():
    assert max_matching([[0, 1], [0], [1, 2]]) == 3          # person 1 takes 0, so person 0 moves to 1, pers
on 2 takes 2
    assert max_matching([[0], [0], [0]]) == 1
    assert max_matching([[], []]) == 0
    people = [("ann", 1, {"dp"}), ("bob", 5, {"dp", "graph"}), ("cy", 3, {"graph"})]
    questions = [("q1", {"dp"}, 1), ("q2", {"graph"}, 1)]
    assert build_graph(people, questions) == {"ann": ["q1"], "bob": ["q1", "q2"], "cy": ["q2"]}


# =====================================================
# FOLLOW-UP 1: Priorities: higher-priority people must be served first
# =====================================================


# =====================================================
# FOLLOW-UP 3: A question may take several people (capacity c)
# =====================================================
# ---- follow-up 1: helper functions ----
def priority_of(person):
    return person[1]                                    # a person is (name, priority, tags)


# ---- follow-up 1: function ----
# Answer: try the people in order of decreasing priority and run the same
# augmenting step. Matchings of this kind form a "matroid", so this greedy
# order ends with the matched set of the highest total priority. A
# question with capacity c holds up to c people: it has a free place when
# fewer than c people hold it, otherwise try to move one of its holders
# to another question.
def assign(people, questions):
    adj = build_graph(people, questions)
    capacity = {}
    held = {}                                           # question -> the people who hold its places
    for question_id, tags, cap in questions:
        capacity[question_id] = cap
        held[question_id] = []

    def try_assign(person, seen):
        for question in adj[person]:
```

```
            if question in seen:
                continue
            seen.add(question)
            if len(held[question]) < capacity[question]:
                held[question].append(person)           # a free place
                return True
            for other in list(held[question]):          # or someone here who can move elsewhere
                if try_assign(other, seen):
                    held[question].remove(other)
                    held[question].append(person)
                    return True
        return False

    for name, priority, tags in sorted(people, key=priority_of, reverse=True):
        try_assign(name, set())
    result = {}
    for question in held:
        for person in held[question]:
            result[person] = question
    return result
```

```python
# ---- follow-up 1: tests ----
def test_follow_up_1():
    people = [("ann", 1, {"dp"}), ("bob", 5, {"dp", "graph"}), ("cy", 3, {"graph"})]
    questions = [("q1", {"dp"}, 1), ("q2", {"graph"}, 1)]
    got = assign(people, questions)
    assert len(got) == 2                          # the matching is still the largest possible
    assert got["bob"] in ("q1", "q2")             # bob (priority 5) is served
    both = assign(people[:2], [("q1", {"dp"}, 2)])   # a question with two places takes two people
    assert set(both) == {"ann", "bob"}
    assert assign([("x", 1, {"a"})], [("q", {"b"}, 1)]) == {}      # nobody shares a tag: nobody is matched


# ======================================================
# FOLLOW-UP 2: Edges have weights (how good a match is)
# ======================================================
# Answer: now it is a maximum-WEIGHT matching: use the Hungarian
# algorithm (O(n^3)) or min-cost max-flow. Kuhn's algorithm only maximises
# the NUMBER of matches.


# ======================================================
# RUN EVERYTHING
# ======================================================
```

```python
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

63 Matrix Flower Placement with House Adjacency

HARD

Template fit: place row by row; reject a row early when the houses two rows above are still not next to a flower

QUESTION

A grid has houses (H) and empty cells (0). Place flowers (F) on empty cells so that every row and every column holds at most one flower (exactly one of each when exactly_one is True) and every house has a flower directly beside it (up, down, left or right). Return one valid grid as a list of strings, or None if there is none.

Function signature

def place_flowers(grid: list, exactly_one=False) -> list | None

EXAMPLES

Example 1	
Input	`['H000', 'H000', '000H', '0000']`
Output	`['HF00', 'H0F0', 'F00H', '000F']`
Why	Flowers at (0,1), (2,0) and (1,3): one per row and column, and each house touches a flower.

Example 2	
Input	`['H000', 'H000', '000H', '0000'], exactly_one = true`
Output	`['HF00', 'H0F0', 'F00H', '000F']`
Why	Exactly one flower in every row and column: a fourth flower at (3,2) completes it.

QUESTION

Example 3	
Input	['H', 'H']
Output	null
Why	The two houses can only be served by the same column, so no valid placement exists.

FULL CODE | main question first, then every follow-up, tests at the end

```
# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ========================================================

# ========================================================
# MAIN QUESTION
# ========================================================
# ---- main question: helper functions ----
STEPS = ((1, 0), (-1, 0), (0, 1), (0, -1))


# ---- main question: main function ----
# The grid has houses "H" and empty cells "0". Put at most one flower "F"
# in every row and every column, only on empty cells, so that every house
# has a flower right next to it (above, below, left or right).
# Idea: backtracking row by row. For each row either put a flower in a
# free column (remember the used columns) or leave the row empty; undo the
# choice afterwards. Houses are checked LATE: when we start row r, the
# rows before r are decided, so the houses in row r - 2 can no longer
# get a new neighbour and must already be satisfied. That prunes early.
# exactly_one=True: a flower in EVERY row and EVERY column (follow-up 3);
# find_all=True: return every solution (follow-up 4).
def place_flowers(grid, exactly_one=False, find_all=False):
    rows = len(grid)
    cols = len(grid[0])
    cells = []
    for row in grid:
        cells.append(list(row))
    houses = []
    for r in range(rows):
        for c in range(cols):
            if cells[r][c] == "H":
                houses.append((r, c))
    found = []

    def house_ok(r, c):
        # is some neighbour of this house a flower?
        for dr, dc in STEPS:
            a = r + dr
            b = c + dc
            if 0 <= a < rows and 0 <= b < cols and cells[a][b] == "F":
                return True
```

```
        return False

    def row_houses_ok(r):
        for hr, hc in houses:
            if hr == r and not house_ok(hr, hc):
                return False
        return True

    def solve(r, used):
```

```
        if len(found) > 0 and not find_all:
            return                                  # one solution is enough
        if r >= 2 and not row_houses_ok(r - 2):
            return                                  # a house in row r - 2 can no longer be served
        if r == rows:
            full = len(used) == cols
            if row_houses_ok(rows - 1) and (full or not exactly_one):
                found.append(["".join(line) for line in cells])
            return
        for c in range(cols):                       # try a flower in every free column of this row
            if c not in used and cells[r][c] == "0":
                cells[r][c] = "F"
                used.add(c)
                solve(r + 1, used)
                used.remove(c)                      # undo
                cells[r][c] = "0"
        if not exactly_one:
            solve(r + 1, used)                      # or leave the row empty

    solve(0, set())
    if find_all:
        return found
    if len(found) > 0:
        return found[0]
    return None


# ---- main question: tests ----
def test_main_question():
    grid = ["H00", "0H0", "00H"]
    solution = place_flowers(grid)
    assert solution is not None
    for r in range(3):
        for c in range(3):
            if grid[r][c] == "H":
                has_flower = False
```

```
                for dr, dc in STEPS:
                    x = r + dr
                    y = c + dc
                    if 0 <= x < 3 and 0 <= y < 3 and solution[x][y] == "F":
                        has_flower = True
                assert has_flower                   # every house has a flower beside it
    for row in solution:
        assert row.count("F") <= 1                  # at most one flower per row
    for c in range(3):
        column = [row[c] for row in solution]
        assert column.count("F") <= 1               # and per column
    assert place_flowers(["H"]) is None             # a lone house in a 1 x 1 grid cannot be served


# =====================================================
# FOLLOW-UP 1: What changes between "at most one" and "exactly one" flower per row and column?
# =====================================================


# =====================================================
# FOLLOW-UP 3: Exactly one flower per row and column
# =====================================================
# Answer: with "exactly one" a row may not be left empty (skip the "leave the
# row empty" branch) and at the end every column must be used. That is
# the exactly_one flag. It also prunes much harder: only n! arrangements
# remain.
# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert place_flowers(["000", "000"], True) is None          # 2 rows, 3 columns: no flower in every col
```

```
umn
    assert place_flowers(["00", "00"], True) is not None


# ====================================================
# FOLLOW-UP 2: How would you speed up the search?
# ====================================================
# Answer: use a bitmask for the used columns, stop as soon as a house can
# no longer be served (the delayed check above), try rows with houses
# first, and memoise on (row, used columns, the flowers of the last two
# rows) because nothing else matters for the houses still to be checked.


# ====================================================
# FOLLOW-UP 4: Count the solutions instead of listing them
# ====================================================
# ---- follow-up 4: function ----
```

```
# Answer: the same recursion, but add one to a counter at the end instead of
# copying the grid. With a bitmask of used columns and memoisation it
# becomes a DP. (Here the solutions are listed with find_all and counted.)
def count_solutions(grid, exactly_one=False):
    return len(place_flowers(grid, exactly_one, True))


# ---- follow-up 4: tests ----
def test_follow_up_4():
    assert count_solutions(["00", "00"], True) == 2              # the two diagonals
    assert count_solutions(["00", "00"]) == 7                    # none, 4 singles, and the 2 diagonals


# ====================================================
# RUN EVERYTHING
# ====================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_4()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

GROUP 8 OF 43 | 9 QUESTIONS, ONE CODE TEMPLATE | 2 E 7 M

Runs and groups: extend while it fits, restart or close when it does not

64 Group Timestamps Into Close Intervals — EASY

Template fit: close the group when the gap to the next timestamp is at least k; otherwise extend it

QUESTION

You are given an array of positive integer timestamps in ascending order and an integer k . Partition the timestamps into maximal runs such that within each run, the difference between every pair of adjacent values is strictly less than k . A run breaks wherever an adjacent difference is >= k . Return each run as an interval [start, end] (its first and last value), in order.

Function signature

def group_into_intervals(timestamps: list[int], k: int) -> list[list[int]]:

    ...

Follow-up (as reported): One pass, O(n) time and O(1) extra space (besides the output).

EXAMPLES

Example 1	
Input	`timestamps = [1,2,3,4,11,14,15,17,21,22,23], k = 2`
Output	`[[1,4],[11,11],[14,15],[17,17],[21,23]]`
Why	`Adjacent diffs of 1 stay in a run; a diff of >=2 (e.g. 4->11, 11->14, 15- >17, 17->21) starts a new run.`

Example 2	
Input	`timestamps = [5], k = 100`
Output	`[[5,5]]`
Why	`A single timestamp forms a run by itself.`

Example 3	
Input	`timestamps = [1,2,3], k = 1`
Output	`[[1,1],[2,2],[3,3]]`
Why	`A diff of 1 is not strictly less than k=1, so every element stands alone.`

CONSTRAINTS

- 1 <= len(timestamps) <= 10^5 , values strictly increasing.
- k >= 1 .

FULL CODE | main question first, then every follow-up, tests at the end

```
# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ========================================================

import random


# ========================================================
# MAIN QUESTION
# ========================================================
```

```python
# ---- main question: main function ----
# Idea: one pass. Remember where the current run started; whenever the
# gap to the previous timestamp reaches k, close the run [start, previous]
# and start a new one at this timestamp.
def group_into_intervals(timestamps, k):
    if not timestamps:
        return []
    runs = []
    start = timestamps[0]
    for i in range(1, len(timestamps)):
        previous = timestamps[i - 1]
        current = timestamps[i]
        if current - previous >= k:
            runs.append([start, previous])
            start = current
    runs.append([start, timestamps[len(timestamps) - 1]])
    return runs


# ---- main question: tests ----
def test_main_question():
    result = group_into_intervals([1, 2, 3, 4, 11, 14, 15, 17, 21, 22, 23], 2)
    assert result == [[1, 4], [11, 11], [14, 15], [17, 17], [21, 23]]
    assert group_into_intervals([5], 100) == [[5, 5]]
    assert group_into_intervals([1, 2, 3], 1) == [[1, 1], [2, 2], [3, 3]]
    assert group_into_intervals([], 3) == []


# ========================================================
# FOLLOW-UP 1: A run must also never last k seconds or more from its first value (a session limit)
# ========================================================
# ---- follow-up 1: function ----
```

```python
# Answer: compare with the run's START instead of the previous value:
# close the run when current - start >= k (that also covers the
# neighbour-gap rule). Still one pass.
def group_by_span(timestamps, k):
    if not timestamps:
        return []
    runs = []
    start = timestamps[0]
    previous = timestamps[0]
    for i in range(1, len(timestamps)):
        current = timestamps[i]
        if current - start >= k:
            runs.append([start, previous])
            start = current
        previous = current
    runs.append([start, previous])
    return runs


# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert group_by_span([1, 2, 3, 4, 5, 6], 3) == [[1, 3], [4, 6]]
    random.seed(45)
    for round_number in range(400):
        timestamps = sorted(random.sample(range(1, 40), random.randint(0, 9)))
        k = random.randint(1, 6)
        for first, last in group_by_span(timestamps, k):
            assert last - first < k                        # no run lasts k seconds or more


# ========================================================
# RUN EVERYTHING
# ========================================================
```

```
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

65 First Out-of-Order Character

EASY

Template fit: fits = s[i] >= s[i-1]; return i at the first item that does not fit

QUESTION

You are given a string of lowercase English letters. Scanning left to right, the string is considered "in order" as long as each character is greater than or equal to the character before it (a non-decreasing sequence; equal adjacent characters are allowed). Return the index of the FIRST character that breaks this rule — that is, the first index i >= 1 where s[i] < s[i-1] . If the entire string is non-decreasing, return -1 .

Function signature

def first_out_of_order(s: str) -> int:

...

Follow-up (as reported): Solve in a single O(N) pass with O(1) extra space; state the time/space complexity as the interviewer expected.

EXAMPLES

Example 1	
Input	`"abcda"`
Output	`4`
Why	`a<=b<=c<=d holds, but at index 4 'a' < 'd' (s[3]), so 4 is returned.`

Example 2	
Input	`"aabbz"`
Output	`-1`
Why	`Every character is >= the previous one; fully non-decreasing.`

Example 3	
Input	`"cba"`
Output	`1`
Why	`At index 1, 'b' < 'c', the first violation.`

Example 4	
Input	`""`
Output	`-1`
Why	`An empty string is trivially in order.`

CONSTRAINTS

- 0 <= len(s) <= 10^5 s consists only of lowercase English letters.
- An empty string or a single-character string is trivially in order -> return -1 .

FULL CODE | main question first, then every follow-up, tests at the end

```python
# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

import random


# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: main function ----
# Idea: one pass, comparing each letter with the one before it. The
# order is broken exactly where a letter is SMALLER than the previous
# letter; return that index, or -1 if the string is in order.
def first_out_of_order(s):
    for i in range(1, len(s)):
        if s[i] < s[i - 1]:
            return i
    return -1


# ---- main question: tests ----
def test_main_question():
    assert first_out_of_order("abcda") == 4
    assert first_out_of_order("aabbz") == -1
    assert first_out_of_order("cba") == 1
    assert first_out_of_order("") == -1


# ============================================================
# FOLLOW-UP 1: Report every break and the longest in-order run
# ============================================================
# ---- follow-up 1: function ----
# Answer: collect every index where s[i] < s[i - 1]. The in-order runs are
# the stretches between breaks, so the longest run is the biggest gap
# between neighbouring break positions (counting the two ends of the string).
def order_breaks(s):
    breaks = []
    for i in range(1, len(s)):
        if s[i] < s[i - 1]:
            breaks.append(i)
    return breaks
```

```python


def longest_in_order_run(s):
    edges = [0] + order_breaks(s) + [len(s)]
    longest = 0
    for i in range(len(edges) - 1):
        if edges[i + 1] - edges[i] > longest:
            longest = edges[i + 1] - edges[i]
    return longest


# ---- follow-up 1: tests ----
def longest_run_slowly(s):
    # check by walking the string once more, counting the current run
    longest = 0
    run = 0
    for i in range(len(s)):
        if i > 0 and s[i] >= s[i - 1]:
            run += 1
        else:
            run = 1
```

```
        if run > longest:
            longest = run
    return longest


def test_follow_up_1():
    assert order_breaks("abcda") == [4]
    random.seed(84)
    for round_number in range(400):
        s = ""
        for i in range(random.randint(0, 9)):
            s += random.choice("abcd")
        assert longest_in_order_run(s) == longest_run_slowly(s)


# ======================================================
# RUN EVERYTHING
# ======================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")
```

```
if __name__ == "__main__":
    run_all_tests()
```

66 Longest Non-Decreasing Subarray with One Replacement

MEDIUM

Template fit: left[i] and right[i] run lengths; the best answer joins the run before and after one changed element

QUESTION

Find the maximum length of a contiguous non-decreasing subarray if at most one element may be changed to any value. Some rounds start from the plain longest-increasing-subarray question and add the one-replacement follow-up; those versions ask for an increasing run, so clarify strict vs non-strict.

Input: an integer array nums.
You may change at most one element to any value.
Return the maximum length of a contiguous subarray that is non-decreasing (nums[i] <= nums[i + 1] for every adjacent pair inside it) after the change.

Variant: other rounds ask for a strictly increasing run and open with the no-replacement version (longest increasing subarray) before adding the one-replacement follow-up — clarify strict vs non-strict before coding.

Expect to discuss and compare the complexity of several approaches; a brute-force first pass is usually followed by a request to optimize.

EXAMPLES

```
nums = [1, 0, 3, 4, 5, 2, 3]
Output: 5
Change nums[1] from 0 to 1 -> [1, 1, 3, 4, 5, 2, 3]
[1, 1, 3, 4, 5] is a non-decreasing subarray of length 5.
```

FULL CODE | main question first, then every follow-up, tests at the end

```
# ======================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ======================================================
```

```
# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
def in_order(x, y, strict):
    # may y follow x? non-decreasing: x <= y; strictly increasing: x < y
    if strict:
        return x < y
    return x <= y


def run_lengths(a, strict):
    # left[i] = length of the run that ENDS at i, right[i] = the run that STARTS at i
    n = len(a)
    left = [1] * n
    right = [1] * n
    for i in range(1, n):
        if in_order(a[i - 1], a[i], strict):
            left[i] = left[i - 1] + 1
    for i in range(n - 2, -1, -1):
        if in_order(a[i], a[i + 1], strict):
            right[i] = right[i + 1] + 1
    return left, right


def bigger(x, y):
    if x > y:
        return x
    return y


# ---- main question: main function ----
# Idea: changing a[i] can join the run on its left and the run on its
# right: left[i-1] + 1 + right[i+1], but only if some value fits between
# the neighbours (a[i-1] <= a[i+1]). Otherwise it extends just the longer
# of the two runs. At the ends there is only one side to join.
# (strict=True is follow-up 1: strictly increasing runs, and then the
# neighbours must differ by at least 2 so that a whole number fits between.)
```

```
def longest_with_one_change(a, strict=False):
    n = len(a)
    if n <= 2:
        return n                                        # nothing to fix
    gap = 0
    if strict:
        gap = 2
    left, right = run_lengths(a, strict)
    best = 1
    for i in range(n):
        left_run = 0
        if i > 0:
            left_run = left[i - 1]
        right_run = 0
        if i + 1 < n:
            right_run = right[i + 1]
        if left_run == 0 or right_run == 0 or a[i + 1] - a[i - 1] >= gap:
            best = bigger(best, left_run + 1 + right_run)      # the change joins both runs
        else:
            best = bigger(best, bigger(left_run, right_run) + 1)   # it joins only the longer one
    return best


# the warm-up: no replacement allowed
def longest_without_change(a):
    best = 0
    run = 0
```

```
    for i in range(len(a)):
        if i > 0 and a[i - 1] < a[i]:
            run += 1
        else:
            run = 1
        best = bigger(best, run)
    return best


# ---- main question: tests ----
def test_main_question():
    assert longest_with_one_change([1, 0, 3, 4, 5, 2, 3]) == 5      # change 0 to 1: [1, 1, 3, 4, 5]
    assert longest_with_one_change([]) == 0
    assert longest_with_one_change([7]) == 1
    assert longest_without_change([1, 3, 5, 4, 7]) == 3


# ========================================================
```

```
# FOLLOW-UP 1: The run must be strictly increasing
# ========================================================
# Answer: use < instead of <= when building the runs, and when the
# replaced value must fit between its neighbours require
# a[i+1] - a[i-1] >= 2 (a whole number strictly between them exists).
# Clear up "strict or not" before you write code.
# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert longest_with_one_change([1, 2, 2, 3, 4], True) == 4
    assert longest_with_one_change([1, 1, 1, 1], True) == 2


# ========================================================
# RUN EVERYTHING
# ========================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

67 Piano Hand Movement Grouping

MEDIUM

Template fit: start a new group when adding the note would make max - min exceed k

QUESTION

A pianist plays a sequence of notes in order (a bigger number is a higher key). The hand can stay in one position while every note it plays lies within a span of k (highest minus lowest at most k). Starting a new hand position costs a move. Return the smallest number of hand positions needed to play all the notes in order.

Function signature

def groups(notes: list, k: int) -> int

EXAMPLES

Example 1	
Input	`[1, 3, 5, 7, 9, 11], 4`
Output	`2`
Why	`[1, 3, 5] fits in a span of 4, then [7, 9, 11]: 2 positions.`

QUESTION

Example 2	
Input	[1, 2, 3, 4, 5], 4
Output	1
Why	All five notes fit in a span of 4: one position.

Example 3	
Input	[5, 9, 1], 4
Output	2
Why	5 and 9 fit together, then 1 is too far from them: 2 positions.

FULL CODE | main question first, then every follow-up, tests at the end

```
# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ========================================================

# ========================================================
# MAIN QUESTION
# ========================================================
# ---- main question: helper functions ----
def hand_positions(notes, k):
    # split the notes into groups that one hand position (a span of at most
    # k) can play, as early as possible. Each group is (start index,
    # lowest note, highest note).
    groups_found = []
    for i in range(len(notes)):
        x = notes[i]
        if groups_found:
            start, low, high = groups_found[-1]
            new_low = low
            if x < new_low:
                new_low = x
            new_high = high
            if x > new_high:
                new_high = x
            if new_high - new_low <= k:                # the note still fits the current span
                groups_found[-1] = (start, new_low, new_high)
                continue
        groups_found.append((i, x, x))                 # otherwise the hand must move: a new group starts
    return groups_found


# ---- main question: main function ----
# Idea: a greedy scan. Keep the lowest and highest note of the current
# group; extend the group while high - low stays within k, otherwise
# close it and start a new group at this note. The answer is the number
# of hand placements.
def groups(notes, k):
    return len(hand_positions(notes, k))


# ---- main question: tests ----
def test_main_question():
    assert groups([1, 3, 5, 7, 9, 11], 4) == 2          # the example: 2 placements for reach 4
```

```
    assert groups([], 4) == 0
    assert groups([2, 2, 3], 0) == 2                    # a reach of 0: a new placement for every change
```

```python
# ============================================================
# FOLLOW-UP 1: Return where each group starts and where to place the hand
# ============================================================
# ---- follow-up 1: function ----
# Answer: remember the start index when a group opens; a group ends just
# before the next one starts. Put the hand on the lowest note of the group.
# The state changes are the same.
def plan(notes, k):
    spots = hand_positions(notes, k)
    result = []
    for n in range(len(spots)):
        start, low, high = spots[n]
        if n + 1 < len(spots):
            end = spots[n + 1][0] - 1
        else:
            end = len(notes) - 1
        result.append((start, end, low))
    return result


# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert plan([1, 3, 5, 7, 9, 11], 4) == [(0, 2, 1), (3, 5, 7)]


# ============================================================
# FOLLOW-UP 2: Count lifts, not placements
# ============================================================
# ---- follow-up 2: function ----
# Answer: the first placement is free and every later one is a lift, so
# lifts = placements - 1 (never below 0). The examples differ on this, so
# ask and say which one you count.
def lifts(notes, k):
    placements = groups(notes, k)
    if placements == 0:
        return 0
    return placements - 1


# ---- follow-up 2: tests ----
def test_follow_up_2():
```

```python
    assert lifts([1, 2, 3, 4, 5], 4) == 0
    assert lifts([5, 9, 1], 4) == 1


# ============================================================
# FOLLOW-UP 3: The hand may slide without a lift (costing something)
# ============================================================
# Answer: now each step between groups costs min(lift cost, slide cost x
# distance moved). Keep the same groups, and add the cheaper of the two
# options for every move.


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    print("all tests passed")
```

```
if __name__ == "__main__":
    run_all_tests()
```

68 Find longest increasing contiguous subarray — MEDIUM

Template fit: fits = a[i] > a[i-1]; also keep a left run and a right run per index so one change can join two runs

QUESTION

Given an array you may change at most one element to any value (an integer when integers=True, any real number otherwise). Return the length of the longest strictly increasing contiguous subarray you can obtain.

Function signature

def longest_one_change(a: list, integers: bool = True) -> int

EXAMPLES

Example 1	
Input	[1, 2, 3, 2, 5, 6]
Output	6
Why	Change the second 2 to 4 and the whole array 1, 2, 3, 4, 5, 6 increases.

Example 2	
Input	[1, 3, 2, 4]
Output	3
Why	Between 1 and 2 there is no integer, so the array 1, 3, 2, 4 only improves to a run of 3.

Example 3	
Input	[1, 3, 2, 4], integers = false
Output	4
Why	With real numbers, 3 can become 1.5 and the whole array increases.

FULL CODE | main question first, then every follow-up, tests at the end

```
# ====================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ====================================================

# ====================================================
# MAIN QUESTION
# ====================================================
# ---- main question: helper functions ----
def bigger(x, y):
    if x > y:
        return x
    return y


# ---- main question: main function ----
# Idea: left[i] is the length of the strictly increasing run that ENDS
# at i, right[i] the length of the one that STARTS at i. Try every
# position as the one changed value: it can join the run on its left, the
# run on its right, or both if a value fits between its neighbours (for
# whole numbers the neighbours must differ by at least 2; for real
# numbers by more than 0).
```

```
def longest_one_change(a, integers=True):
    n = len(a)
    if n == 0:
        return 0
    left = [1] * n
    right = [1] * n
    for i in range(1, n):
        if a[i] > a[i - 1]:
            left[i] = left[i - 1] + 1
    for i in range(n - 2, -1, -1):
        if a[i] < a[i + 1]:
            right[i] = right[i + 1] + 1
    best = 1
    for i in range(n):
        best = bigger(best, left[i])                    # no change at all
        if i > 0:
            best = bigger(best, left[i - 1] + 1)        # change a[i] to continue the run on its left
        if i < n - 1:
            best = bigger(best, right[i + 1] + 1)       # or to start the run on its right
        if 0 < i < n - 1:                               # or to join both, if a value fits between
            room = a[i + 1] - a[i - 1]
```

```
            if integers:
                fits = room >= 2
            else:
                fits = room > 0
            if fits:
                best = bigger(best, left[i - 1] + 1 + right[i + 1])
    if best > n:
        return n
    return best


# ---- main question: tests ----
def test_main_question():
    assert longest_one_change([1, 2, 3, 2, 5, 6]) == 6
    assert longest_one_change([1, 3, 2, 4]) == 3
    assert longest_one_change([1, 3, 5, 4, 7]) == 5
    assert longest_one_change([5, 4, 3, 2, 1]) == 2
    assert longest_one_change([7]) == 1
    assert longest_one_change([]) == 0
    assert longest_one_change([1, 2, 3]) == 3           # never more than the array length


# ======================================================
# FOLLOW-UP 1: No modification allowed (LeetCode 674)
# ======================================================
# ---- follow-up 1: function ----
# Answer: one pass with the length of the current strictly increasing
# run; start again at 1 when a[i] <= a[i - 1]. O(n) time, O(1) space.
def longest_increasing(a):
    best = 0
    run = 0
    for i in range(len(a)):
        if i > 0 and a[i - 1] < a[i]:
            run += 1
        else:
            run = 1
        best = bigger(best, run)
    return best


# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert longest_increasing([1, 3, 5, 4, 7]) == 3
    assert longest_increasing([]) == 0
```

```python
# ============================================================
# FOLLOW-UP 2: Up to k modifications
# ============================================================
# Answer: use a sliding window that is allowed to contain at most k
# "bad" positions (places where the order breaks and has to be repaired);
# shrink from the left when there are more.


# ============================================================
# FOLLOW-UP 3: Values are real numbers, not integers
# ============================================================
# Answer: the `integers` parameter: with real numbers a value fits
# between two neighbours whenever they differ at all (room > 0), but with
# whole numbers they must differ by at least 2.
# ---- follow-up 3: tests ----
def test_follow_up_3():
    assert longest_one_change([1, 3, 2, 4], integers=False) == 4
    assert longest_one_change([1, 2, 2, 3]) == 3           # equal neighbours leave no room for a whole numbe
r
    assert longest_one_change([1, 2, 2, 3], integers=False) == 4


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

69 Sum of Good Subarrays (Adjacent Diff ±1) MEDIUM

Template fit: keep the running length and running sum of the current up-run and down-run; add both to the total at every index

QUESTION

MLE L5 coding round: sum over all contiguous subarrays whose elements form a strictly monotonic ±1 step sequence (every adjacent pair differs by exactly +1, OR every adjacent pair differs by exactly -1). Required time complexity is O(N).

Input: integer array arr of length N.
A subarray is good if its consecutive differences are either all +1 (monotone increasing by 1) or all -1 (monotone decreasing by 1).
Singleton subarrays are always good.
Output: the sum of element-sums across all good subarrays (each good subarray contributes the sum of its elements once).
Required: O(N) time.

EXAMPLES

```
arr = [3, 5, 6, 7, 6]
```

Good subarrays

```
  singletons:           [3], [5], [6], [7], [6]
  monotone +1 runs:     [5, 6], [6, 7], [5, 6, 7]
  monotone -1 runs:     [7, 6]
Not good: [3, 5] (diff 2), [6, 7, 6] (mixes +1 then -1).
```

FULL CODE | main question first, then every follow-up, tests at the end

```python
# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: main function ----
# A subarray is GOOD when every step between neighbours is +1, or every
# step is -1 (a single element is good). Return the sum of the elements
# of all good subarrays.
# Idea: go left to right and, for each direction, keep the total of the
# sums of all good subarrays that END at the current element, and how
# many there are. When the next element x continues a +1 run, every old
# subarray grows by x and the new one [x] appears: the total grows by
# (count + 1) * x. A break restarts the total at x. The single [x] is in
# both the "up" and the "down" total, so subtract it once.
def good_subarray_sum(a):
    total = 0
    up_sum = 0
    up_count = 0
    down_sum = 0
    down_count = 0
    for i in range(len(a)):
        x = a[i]
        step = None
        if i > 0:
            step = x - a[i - 1]
        if step == 1:
            up_sum += (up_count + 1) * x
            up_count += 1
        else:
            up_sum = x
            up_count = 1
        if step == -1:
            down_sum += (down_count + 1) * x
            down_count += 1
        else:
            down_sum = x
            down_count = 1
        total += up_sum + down_sum - x
    return total
```

```python


# ---- main question: tests ----
def test_main_question():
    # singletons 27 + [5,6] 11 + [6,7] 13 + [5,6,7] 18 + [7,6] 13
    assert good_subarray_sum([3, 5, 6, 7, 6]) == 82
    assert good_subarray_sum([]) == 0
    assert good_subarray_sum([4]) == 4


# ============================================================
# FOLLOW-UP 1: Why is the singleton subtracted?
# ============================================================
# Answer: a one-element subarray [x] is good in BOTH directions, so it
# is inside up_sum and inside down_sum; counting it twice would be wrong.


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
```

```
    test_main_question()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

70 Sustained-Error Time Window

MEDIUM

Template fit: from the given second extend left and right while errors >= baseline; compare the time span with the interval

QUESTION

Reconstructing the "two arrays - Error and Time" question. You are given two parallel arrays: times (timestamps in seconds, strictly increasing) and errors (the error reading recorded at each timestamp), where errors[i] was recorded at times[i] . You are also given three integers: second , interval , and baseline . Determine whether there exists a contiguous run of readings that:

1. Includes the reading taken at timestamp second (guaranteed to exist exactly once in times), 2. Has every reading in the run satisfy error >= baseline , and 3. Spans a duration of at least interval seconds — that is, times[right] - times[left] >= interval , where left and right are the endpoints of the run. Return True if such a run exists, otherwise False . Note that if the reading at second is itself below baseline , the answer is immediately False .

Function signature

```
def has_sustained_error_window(
    times: list[int],
    errors: list[int],
    second: int,
    interval: int,
    baseline: int,
) -> bool:
    """Return True if a run of readings all >= baseline includes second and spans >=
interval seconds."""
    ...
```

EXAMPLES

Example 1	
Input	`times = [1, 2, 3, 4, 5], errors = [10, 20, 5, 30, 40], second = 1, interval = 1, baseline = 8`
Output	`True`
Why	`Starting at time 1 (err 10 >= 8) the run extends to time 2 (err 20 >= 8) but stops at time 3 (err 5 < 8). The run covers times 1..2, a span of 1 >= interval 1.`

Example 2	
Input	`times = [1, 2, 3, 4, 5], errors = [10, 20, 5, 30, 40], second = 4, interval = 2, baseline = 8`
Output	`False`
Why	`Time 4 (err 30 >= 8) can only extend right to time 5 (err 40 >= 8); time 3 (err 5) blocks going left. The run covers times 4..5, span 1 < interval 2.`

Example 3	
Input	`times = [1, 2, 3], errors = [10, 4, 10], second = 2, interval = 0, baseline = 8`
Output	`False`
Why	`The reading at second=2 is 4 < baseline 8, so no valid run includes it.`

QUESTION

CONSTRAINTS

- 1 <= len(times) == len(errors) <= 10^5 times is strictly increasing; 0 <= times[i] <= 10^9
- -10^9 <= errors[i] <= 10^9 , baseline in the same range second is one of the values in times ; 0 <= interval <= 10^9

FULL CODE | main question first, then every follow-up, tests at the end

```
# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ========================================================

# ========================================================
# MAIN QUESTION
# ========================================================
# ---- main question: main function ----
# The readings are (time, error count) pairs in time order. Is the error
# count AT `second` part of a stretch of readings, all at or above the
# `baseline`, that lasts at least `interval` seconds?
# Idea: the longest stretch around that reading is the best one, so only
# its two ends matter: expand to the left and right while the readings
# stay at or above the baseline.
def has_sustained_error_window(times, errors, second, interval, baseline):
    at = times.index(second)
    if errors[at] < baseline:
        return False
    left = at
    right = at
    while left > 0 and errors[left - 1] >= baseline:
        left -= 1
    while right + 1 < len(times) and errors[right + 1] >= baseline:
        right += 1
    return times[right] - times[left] >= interval


# ---- main question: tests ----
def test_main_question():
    times = [1, 2, 3, 4, 5]
    errors = [10, 20, 5, 30, 40]
    assert has_sustained_error_window(times, errors, 1, 1, 8) is True
    assert has_sustained_error_window(times, errors, 4, 2, 8) is False
    assert has_sustained_error_window(times, errors, 3, 0, 8) is False


# ========================================================
# FOLLOW-UP 1: You get thousands of (second, interval) queries on the same data
# ========================================================
# ---- follow-up 1: helper functions ----
def run_bounds(times, errors, baseline):
    # for every reading: the index where its stretch (readings at or above
```

```
    # the baseline) starts and ends; one pass from the left, one from the right
    n = len(times)
    left = [0] * n
    right = [0] * n
    for i in range(n):
        left[i] = i
        if i > 0 and errors[i - 1] >= baseline and errors[i] >= baseline:
            left[i] = left[i - 1]
    for i in range(n - 1, -1, -1):
        right[i] = i
        if i + 1 < n and errors[i + 1] >= baseline and errors[i] >= baseline:
            right[i] = right[i + 1]
```

```
    return left, right


def find_index(times, second):
    # binary search in the sorted times: the index of the first time >= second
    low = 0
    high = len(times)
    while low < high:
        middle = (low + high) // 2
        if times[middle] < second:
            low = middle + 1
        else:
            high = middle
    return low


# ---- follow-up 1: function ----
# Answer: compute once, for every reading, where its stretch starts and
# ends (run_bounds); then each query is a binary search for the reading
# plus two lookups: O(log n) per query.
def sustained_query(times, errors, bounds, second, interval, baseline):
    at = find_index(times, second)
    if errors[at] < baseline:
        return False
    left, right = bounds
    return times[right[at]] - times[left[at]] >= interval


# ---- follow-up 1: tests ----
def test_follow_up_1():
    times = [1, 2, 3, 4, 5, 8, 9]
    errors = [10, 20, 5, 30, 40, 50, 9]
    bounds = run_bounds(times, errors, 8)
```

```
    for second in times:
        for interval in (0, 1, 2, 3):
            expected = has_sustained_error_window(times, errors, second, interval, 8)
            assert sustained_query(times, errors, bounds, second, interval, 8) == expected


# ========================================================
# RUN EVERYTHING
# ========================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

71 Longest Non-Decreasing Subarray From Two Arrays

MEDIUM

Template fit: two run lengths per index, one for each array you may take the value from

QUESTION

You are given two integer arrays nums1 and nums2 , both of length n . You must construct a third array nums3 of length n where, for each index i , nums3[i] is chosen to be either nums1[i] or nums2[i] (chosen independently per index). Return the length of the longest non-decreasing subarray (a contiguous run) achievable in nums3 over all valid choices.

QUESTION

Function signature

def longest_non_decreasing_subarray(nums1: list[int], nums2: list[int]) -> int:

...

Follow-up (as reported): (Reported) The interviewer first asked for the longest contiguous non-decreasing subarray of a single array, then generalized to picking per index from two arrays; also asked for the time complexity and to write test cases.

EXAMPLES

Example 1	
Input	`nums1 = [2, 3, 1], nums2 = [1, 2, 1]`
Output	`2`
Why	`Choosing nums3 = [2, 3, ...] gives the run [2, 3] of length 2. No choice yields a non-decreasing run of length 3.`

Example 2	
Input	`nums1 = [1, 3, 2, 1], nums2 = [2, 2, 3, 4]`
Output	`4`
Why	`nums3 = [1, 2, 3, 4] (pick nums1[0], nums2[1], nums2[2], nums2[3]) is non-decreasing over all 4 positions.`

Example 3	
Input	`nums1 = [1, 1], nums2 = [2, 2]`
Output	`2`
Why	`Any choice, e.g. [1, 1], is non-decreasing across both.`

CONSTRAINTS

- 1 <= n <= 10^5
- nums1 and nums2 have the same length n .
- 1 <= nums1[i], nums2[i] <= 10^9
- Target complexity: O(n) time, O(1) extra space.

FULL CODE | main question first, then every follow-up, tests at the end

```
# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
def bigger(x, y):
    if x > y:
        return x
    return y


# ---- main question: main function ----
# At every index you may take the value from nums1 or from nums2. Find
# the longest non-decreasing subarray you can build.
# Idea (two-state DP): dp1 = the best run ending here if we take nums1[i],
# dp2 = the same if we take nums2[i]. Each can extend the run that ended
```

```python
# at the previous index with either choice, if the order holds.
def longest_non_decreasing_subarray(nums1, nums2):
    dp1 = 1
    dp2 = 1
    best = 1
    for i in range(1, len(nums1)):
        new1 = 1
        new2 = 1
        if nums1[i - 1] <= nums1[i]:
            new1 = bigger(new1, dp1 + 1)
        if nums2[i - 1] <= nums1[i]:
            new1 = bigger(new1, dp2 + 1)
        if nums1[i - 1] <= nums2[i]:
            new2 = bigger(new2, dp1 + 1)
        if nums2[i - 1] <= nums2[i]:
            new2 = bigger(new2, dp2 + 1)
        dp1 = new1
        dp2 = new2
        best = bigger(best, bigger(dp1, dp2))
    return best


# ---- main question: tests ----
```

```python
def test_main_question():
    assert longest_non_decreasing_subarray([2, 3, 1], [1, 2, 1]) == 2
    assert longest_non_decreasing_subarray([1, 3, 2, 1], [2, 2, 3, 4]) == 4
    assert longest_non_decreasing_subarray([1, 1], [2, 2]) == 2


# ============================================================
# FOLLOW-UP 1: First solve it for a single array, then generalize to picking per index from two arrays
# ============================================================
# ---- follow-up 1: function ----
# Answer: for one array: run = run + 1 if nums[i - 1] <= nums[i] else 1,
# and keep the maximum: O(n) time, O(1) space. The two-array version keeps
# one run length per CHOICE instead of one.
def longest_non_decreasing_one(nums):
    if not nums:
        return 0
    best = 1
    run = 1
    for i in range(1, len(nums)):
        if nums[i - 1] <= nums[i]:
            run += 1
        else:
            run = 1
        best = bigger(best, run)
    return best


# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert longest_non_decreasing_one([1, 2, 2, 1, 5]) == 3
    assert longest_non_decreasing_one([]) == 0
    assert longest_non_decreasing_one([3]) == 1


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")
```

```
if __name__ == "__main__":
    run_all_tests()
```

72 Count Super Streaks — MEDIUM

Template fit: sort by time; the same type within T continues the streak; when it breaks, count it if size >= N and span >= X

QUESTION

A player produces a list of events, each (event_id, event_type, event_time) . Sort events by event_time . A streak is a maximal run of events, consecutive in time order, in which every event has the same event_type AND the gap between each event and the previous one in the run is at most T seconds. A streak ends when the next event has a different event_type or its gap from the previous event exceeds T . A streak is a Super Streak if it contains at least N events AND spans at least X seconds (span = last event time minus first event time within the streak). Return the number of Super Streaks.

Function signature

```
def count_super_streaks(events: list[tuple[int, str, int]],
                        N: int, T: int, X: int) -> int:
    ...
```

Follow-up (as reported): Follow-up 1: events are tagged with user_id ; return a per-user count of Super Streaks. Follow-up 2: after processing, support count_streaks(user_id, time_A, time_B) returning how many of that user's Super Streaks completed entirely within [time_A, time_B] . Store each user's Super-Streak end-times sorted and answer with two binary searches in O(log n) per query.

EXAMPLES

Example 1	
Input	`events = [(1,"A",0),(2,"A",10),(3,"A",20),(4,"A",30),(5,"A",40)], N=4, T=10, X=30`
Output	`1`
Why	`One streak of 5 A-events, gaps all 10 (<=T), span 40 (>=30), size 5 (>=4). Super.`

Example 2	
Input	`events = [(1,"A",0),(2,"A",10),(3,"A",20),(4,"A",30),(5,"A",40), (6,"A",55),(7,"A",56),(8,"A",57)], N=4, T=10, X=30`
Output	`1`
Why	`The jump 40->55 (gap 15 > T) breaks the run. The first streak [0..40] is Super; the second [55,56,57] has only 3 events and span 2, so not Super.`

Example 3	
Input	`events = [(1,"A",0),(2,"B",5),(3,"A",10)], N=2, T=10, X=1`
Output	`0`
Why	`Type changes each step, so every streak has length 1 — none reaches N=2.`

CONSTRAINTS

- 0 <= len(events) <= 10^5 .
- N >= 1 , T >= 0 , X >= 0 ; times are non-negative integers, possibly with duplicate timestamps.

FULL CODE | main question first, then every follow-up, tests at the end

```
# ====================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
```

```
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
def event_time(event):
    # an event is (id, type, time) or (id, type, time, user)
    return event[2]


def find_streaks(events, N, T, X):
    # A STREAK is a run of events of the same type where each event comes at
    # most T after the previous one. It is a SUPER streak if it has at least N
    # events and lasts at least X from its first to its last event.
    # Returns the (start time, end time) of every super streak.
    spans = []
    start = None
    previous = None
    kind = None
    size = 0
    for event in sorted(events, key=event_time):
        event_type = event[1]
        time = event[2]
        if kind == event_type and time - previous <= T:
            size += 1                                   # the open streak continues
        else:
            if size > 0 and size >= N and previous - start >= X:
                spans.append((start, previous))         # the open streak was a super streak
            start = time
            kind = event_type
            size = 1
        previous = time
    if size > 0 and size >= N and previous - start >= X:
        spans.append((start, previous))
    return spans


# ---- main question: main function ----
# Idea: one pass over the events sorted by time. Keep only the open
```

```
# streak (when it started, the time of its last event, its size and its
# type); when an event does not continue it, check whether it was a
# super streak and start a new one.
def count_super_streaks(events, N, T, X):
    return len(find_streaks(events, N, T, X))


# ---- main question: tests ----
EVENTS = [(1, "A", 0), (2, "A", 10), (3, "A", 20), (4, "A", 30), (5, "A", 40)]


def test_main_question():
    assert count_super_streaks(EVENTS, 4, 10, 30) == 1
    more = EVENTS + [(6, "A", 55), (7, "A", 56), (8, "A", 57)]
    assert count_super_streaks(more, 4, 10, 30) == 1       # the second run is too short
    assert count_super_streaks([], 1, 1, 0) == 0


# ============================================================
# FOLLOW-UP 1: Events carry a user_id. Return the number of super streaks for each user
# ============================================================
# ---- follow-up 1: helper functions ----
def group_by_user(events):
```

```
    # events are (id, type, time, user); returns {user: [(id, type, time), ...]}
    by_user = {}
    for event_id, event_type, time, user in events:
        if user not in by_user:
            by_user[user] = []
        by_user[user].append((event_id, event_type, time))
    return by_user


# ---- follow-up 1: function ----
# Answer: group the events by user and run the same pass on each group
# (or keep one open streak per user in a single pass). Cost O(n log n).
def super_streaks_per_user(events, N, T, X):
    counts = {}
    by_user = group_by_user(events)
    for user in by_user:
        counts[user] = count_super_streaks(by_user[user], N, T, X)
    return counts


# ---- follow-up 1: tests ----
def test_follow_up_1():
```

```
    tagged = [(1, "A", 0, "u"), (2, "A", 10, "u"), (3, "A", 20, "u"), (4, "A", 30, "u"),
              (5, "B", 5, "v"), (6, "B", 6, "v")]
    assert super_streaks_per_user(tagged, 4, 10, 30) == {"u": 1, "v": 0}


# ========================================================
# FOLLOW-UP 2: Then answer count_streaks(user_id, time_A, time_B): how many of that user's super streaks fini
shed entirely inside [A, B]?
# ========================================================
# ---- follow-up 2: function ----
# Answer: record each super streak as (start, end) when it closes, per
# user (they do not overlap, so they are already sorted by start). A
# query counts the streaks with start >= A and end <= B. (For many
# queries, binary-search the first start >= A.)
class StreakIndex:
    def __init__(self, events, N, T, X):
        self.spans = {}
        by_user = group_by_user(events)
        for user in by_user:
            self.spans[user] = find_streaks(by_user[user], N, T, X)

    def count_streaks(self, user, a, b):
        count = 0
        for start, end in self.spans.get(user, []):
            if start >= a and end <= b:
                count += 1
        return count


# ---- follow-up 2: tests ----
def test_follow_up_2():
    tagged = [(1, "A", 0, "u"), (2, "A", 10, "u"), (3, "A", 20, "u"), (4, "A", 30, "u"),
              (5, "A", 100, "u"), (6, "A", 110, "u"), (7, "A", 120, "u"), (8, "A", 130, "u")]
    index = StreakIndex(tagged, 4, 10, 30)
    assert index.count_streaks("u", 0, 200) == 2
    assert index.count_streaks("u", 0, 50) == 1
    assert index.count_streaks("u", 5, 200) == 1           # the first streak started before 5
    assert index.count_streaks("nobody", 0, 200) == 0


# ========================================================
# RUN EVERYTHING
# ========================================================
```

```
def run_all_tests():
    test_main_question()
    test_follow_up_1()
```

```
    test_follow_up_2()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

Union-find: merge the pairs, then ask who is together

73 Count Islands in a Grid and Track the Count as Land Cells Are Added

MEDIUM

Template fit: each new land cell: groups += 1, then union it with every land neighbour (groups -= 1 per real merge); report groups after each addition

QUESTION

A map is a grid with **`m`** rows and **`n`** columns in which every cell is land (**`1`**) or water (**`0`**). An island is a maximal group of land cells connected horizontally or vertically. Cells that touch only at a corner are not connected. You are given the initial map and a sequence of additions, each of which turns one cell into land. Report the number of islands before any addition, and again after each addition.

Function Signature

```
def island_counts(grid: list[list[int]], additions: list[tuple[int, int]]) -> list[int]:
```

Rules

- **`grid[r][c]`** is **`1`** for land and **`0`** for water, where **`r`** is the row index and **`c`** is the column index.
- An addition **`(r, c)`** turns cell **`(r, c)`** into land. If that cell is already land, either initially or because of an earlier addition, the map does not change.
- Additions are applied in the given order. Land never turns back into water.
- Return a list of length **`len(additions) + 1`**. Element **`0`** is the number of islands in the initial grid. For **`i >= 1`**, element **`i`** is the number of islands after the first **`i`** additions have been applied.

Constraints

- **`1 <= m <= 1000`**, **`1 <= n <= 1000`** and **`m * n <= 200000`**
- Every row of **`grid`** has exactly **`n`** entries, each **`0`** or **`1`**.
- **`0 <= len(additions) <= 200000`**
- Every addition **`(r, c)`** satisfies **`0 <= r < m`** and **`0 <= c < n`**. The same cell may appear more than once.
- Every count is at most **`m * n`**, so all values fit in a 32-bit signed integer.

Examples

Example 1

```
Input: grid = [[1, 1, 0, 0], [0, 0, 0, 1], [1, 0, 0, 1]], additions = []
Output: [3]
```

The islands are {(0, 0), (0, 1)}, {(1, 3), (2, 3)} and {(2, 0)}. With no additions, the only element is the initial count.

Example 2

```
Input: grid = [[0, 0, 0], [0, 0, 0], [0, 0, 0]], additions = [(0, 0), (0, 1), (1, 2), (2, 1), (1, 1), (1, 1)]
Output: [0, 1, 1, 2, 3, 1, 1]
```

The grid starts with no land. (0, 0) creates an island and (0, 1) joins it. (1, 2) and (2, 1) each touch no land, so each creates a new island. (1, 1) touches all three islands and merges them into one. The second (1, 1) is already land, so nothing changes.

Example 3

```
Input: grid = [[1, 1, 0, 0], [0, 0, 0, 1], [1, 0, 0, 1]], additions = [(0, 2), (0, 3), (1, 0)]
Output: [3, 3, 2, 1]
```

QUESTION

(0, 2) joins the top-left island. It touches (1, 3) only at a corner, so the count stays 3. (0, 3) connects the top-left island to the island at (1, 3) and (2, 3), giving 2. (1, 0) connects the merged island to (2, 0), giving 1.

FULL CODE | main question first, then every follow-up, tests at the end

```
# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ========================================================

# ========================================================
# MAIN QUESTION
# ========================================================
# ---- main question: helper functions ----
STEPS = ((1, 0), (-1, 0), (0, 1), (0, -1))


class Islands:
    # land cells are added one at a time; the island count is kept up to date
    def __init__(self):
        self.parent = {}       # land cell -> its parent cell (union-find)
        self.size = {}         # size of the island, stored at its leader
        self.count = 0
        self.largest = 0       # used by follow-up 3

    def find(self, cell):
        while self.parent[cell] != cell:
            self.parent[cell] = self.parent[self.parent[cell]]
            cell = self.parent[cell]
        return cell

    def add(self, r, c):
        if (r, c) in self.parent:
            return                                  # already land
        self.parent[(r, c)] = (r, c)
        self.size[(r, c)] = 1
        self.count += 1                             # a new island ...
        if self.largest < 1:
            self.largest = 1
        for dr, dc in STEPS:
            neighbour = (r + dr, c + dc)
            if neighbour in self.parent:
                self.union((r, c), neighbour)       # ... that may join others

    def union(self, a, b):
        a, b = self.find(a), self.find(b)
        if a == b:
            return
```

```
        if self.size[a] < self.size[b]:
            a, b = b, a                             # the small island joins the big one
        self.parent[b] = a
        self.size[a] += self.size[b]
        self.count -= 1                             # two islands became one
        if self.size[a] > self.largest:
            self.largest = self.size[a]


def build_islands(grid):
    # put every land cell (1) of the starting grid into the structure
    islands = Islands()
    for r in range(len(grid)):
```

```
        for c in range(len(grid[0])):
            if grid[r][c] == 1:
                islands.add(r, c)
    return islands


# ---- main question: main function ----
# Idea: build the starting islands with unions, then for every added
# cell: ignore it if it is already land; otherwise it is a new island
# (+1) and every time it is joined to a different island the count
# goes down by one. Returns the count before any addition and after each one.
def island_counts(grid, additions):
    islands = build_islands(grid)
    counts = [islands.count]
    for r, c in additions:
        islands.add(r, c)
        counts.append(islands.count)
    return counts


# ---- main question: tests ----
def test_main_question():
    grid = [[1, 0, 0], [0, 0, 0], [0, 0, 1]]
    adds = [(0, 1), (1, 1), (2, 1), (0, 1)]
    assert island_counts(grid, adds) == [2, 2, 2, 1, 1]     # a repeated cell is ignored
    assert island_counts([[1, 0], [0, 1]], []) == [2]      # corners do not connect cells


# ========================================================
# FOLLOW-UP 3: Report the size of the largest island after each addition
# ========================================================
# ---- follow-up 3: function ----
```

```
# Answer: keep a size per leader (union by size) and a running maximum,
# which only grows when land is added; every step stays near-constant time.
def largest_after_each(grid, additions):
    islands = build_islands(grid)
    sizes = [islands.largest]
    for r, c in additions:
        islands.add(r, c)
        sizes.append(islands.largest)
    return sizes


# ---- follow-up 3: tests ----
def test_follow_up_3():
    grid = [[1, 0, 0], [0, 0, 0], [0, 0, 1]]
    adds = [(0, 1), (1, 1), (2, 1), (0, 1)]
    assert largest_after_each(grid, adds) == [1, 2, 3, 5, 5]
    big = Islands()                                  # a huge grid is fine: only land cells are stored
    big.add(10 ** 9, 10 ** 9)
    big.add(10 ** 9, 10 ** 9 + 1)
    assert big.count == 1 and big.largest == 2


# ========================================================
# RUN EVERYTHING
# ========================================================
def run_all_tests():
    test_main_question()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

74 2D Points Connected Components by Radius

MEDIUM

Template fit: pairs = points within distance r, found by bucketing points into cells of size r and checking only the 9 nearby cells; answer = groups

QUESTION

Given 2D points and radius **r**, connect two points when their distance is at most **r**; connectivity is transitive, and the output is the final number of clusters. The interviewer accepted or preferred a brute-force neighbor scan after discussing optimization ideas.

Input: a list of 2D points and a radius r.
Two points are connected if their Euclidean distance is <= r.
Connectivity is transitive: if A connects to B and B connects to C, all three belong to the same cluster.
Return the number of final clusters.

EXAMPLES

Example 1	
Input	`[(0, 0), (1, 0), (10, 10), (10, 11), (20, 20)], 2`
Output	`3`
Why	`Points within distance 2 of each other form a cluster: two near the origin, two near (10, 10), one alone.`

Example 2	
Input	`[(0, 0), (1, 0), (2, 0), (3, 0)], 1`
Output	`1`
Why	`A chain is one cluster even though its ends are far apart.`

FULL CODE | main question first, then every follow-up, tests at the end

```
# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ========================================================

# ========================================================
# MAIN QUESTION
# ========================================================
# ---- main question: helper functions ----
class UnionFind:
    def __init__(self, n):
        self.parent = list(range(n))

    def find(self, x):
        while self.parent[x] != x:
            self.parent[x] = self.parent[self.parent[x]]
            x = self.parent[x]
        return x

    def union(self, a, b):
        self.parent[self.find(a)] = self.find(b)


def near(p, q, r):
    # compare SQUARED distances: no square root, and exact for integer points
    dx = p[0] - q[0]
    dy = p[1] - q[1]
    return dx * dx + dy * dy <= r * r
```

```python
# ---- main question: main function ----
# Idea: put each point in the cell (x // r, y // r). Two points within
# distance r are in the same or in neighbouring cells, so only those 9
# cells are compared; union every close pair and count the groups.
def count_clusters(points, r):
    size = r
    if size <= 0:
        size = 1
    cells = {}
    for i in range(len(points)):
        key = (points[i][0] // size, points[i][1] // size)
        if key not in cells:
            cells[key] = []
        cells[key].append(i)
    uf = UnionFind(len(points))
    for i in range(len(points)):
        cx, cy = points[i][0] // size, points[i][1] // size
        for dx in (-1, 0, 1):
            for dy in (-1, 0, 1):
                for j in cells.get((cx + dx, cy + dy), []):
                    if j > i and near(points[i], points[j], r):
                        uf.union(i, j)
    groups = 0
    for i in range(len(points)):
        if uf.find(i) == i:
            groups += 1
    return groups


# ---- main question: tests ----
def count_clusters_slow(points, r):
    # compare every pair: slow but obviously right, used to check the fast one
    uf = UnionFind(len(points))
    for i in range(len(points)):
        for j in range(i + 1, len(points)):
            if near(points[i], points[j], r):
                uf.union(i, j)
    roots = set()
    for i in range(len(points)):
        roots.add(uf.find(i))
    return len(roots)


def test_main_question():
    assert count_clusters([(0, 0), (3, 0), (6, 0)], 3) == 1     # joined only through the middle point
    assert count_clusters([(0, 0), (3, 0), (7, 0)], 3) == 2     # the far point stays alone
    assert count_clusters([(1, 1), (1, 1), (2, 2)], 0) == 2     # radius 0 joins only identical points
    assert count_clusters([], 5) == 0
    points = []
    for i in range(30):
        points.append((i * 7 % 13, i * 5 % 11))
    for r in (1, 2, 3):
        assert count_clusters(points, r) == count_clusters_slow(points, r)


# ============================================================
# FOLLOW-UP 1: Why compare squared distances?
# ============================================================
```

```python
# Answer: a square root is slow and inexact. dist <= r means exactly the
# same as dist^2 <= r^2 for non-negative numbers, and with integer
# coordinates the comparison is exact, so nothing goes wrong right at
# the boundary (see near() above).
```

```python
# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

75 Count Clusters of 2D Points Within a Radius — MEDIUM

Template fit: identical to the line above: cell buckets give the candidate pairs, answer = groups

QUESTION

Given 2D points and a radius r, two points are connected when their distance is at most r. A cluster is a group of points connected directly or through chains of other points. Return the number of clusters.

Function signature

def count_clusters(points: list, r: float) -> int

EXAMPLES

Example 1	
Input	`[(0, 0), (1, 0), (10, 10), (10, 11), (20, 20)], 2`
Output	`3`
Why	`Three groups: the two points near the origin, the two near (10, 10), and the lone point at (20, 20).`

Example 2	
Input	`[(0, 0), (1, 0), (2, 0)], 1`
Output	`1`
Why	`A chain: (0,0)-(1,0)-(2,0) are linked neighbour to neighbour, so it is one cluster.`

FULL CODE | main question first, then every follow-up, tests at the end

```python
# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
class Clusters:
    # points arrive one by one; the cluster count is kept up to date.
    # Two points are in one cluster when a chain of points, each within
    # distance r of the next, joins them.
    def __init__(self, r):
        self.r = r
        self.cells = {}          # grid cell (side r) -> indices of the points inside it
        self.points = []
        self.parent = []         # union-find over the point indices
        self.components = 0
```

```
    def find(self, x):
        while self.parent[x] != x:
            self.parent[x] = self.parent[self.parent[x]]
            x = self.parent[x]
        return x

    def near(self, p, q):
        dx = p[0] - q[0]
        dy = p[1] - q[1]
        return dx * dx + dy * dy <= self.r * self.r      # squared distances: no square root

    def add(self, x, y):
        i = len(self.points)
        self.points.append((x, y))
        self.parent.append(i)
        self.components += 1
        side = self.r
        if side == 0:
            side = 1
        cx, cy = x // side, y // side
        if (cx, cy) not in self.cells:
            self.cells[(cx, cy)] = []
        self.cells[(cx, cy)].append(i)
```

```
        # a point within r can only sit in this cell or one of the 8 around it
        for dx in (-1, 0, 1):
            for dy in (-1, 0, 1):
                for j in self.cells.get((cx + dx, cy + dy), []):
                    if j != i and self.near((x, y), self.points[j]):
                        a, b = self.find(i), self.find(j)
                        if a != b:
                            self.parent[a] = b
                            self.components -= 1
        return self.components

    def largest(self):
        # size of the biggest cluster
        sizes = {}
        for i in range(len(self.points)):
            root = self.find(i)
            sizes[root] = sizes.get(root, 0) + 1
        biggest = 0
        for size in sizes.values():
            if size > biggest:
                biggest = size
        return biggest


# ---- main question: main function ----
# Idea: put each point in the cell (x // r, y // r). A point within
# distance r of another is in the same cell or one of the 8 cells
# around it, so only those are compared; union the close pairs.
def count_clusters(points, r):
    clusters = Clusters(r)
    for x, y in points:
        clusters.add(x, y)
    return clusters.components


# ---- main question: tests ----
def test_main_question():
    line = [(0, 0), (3, 0), (6, 0), (20, 20)]
    assert count_clusters(line, 3) == 2
    assert count_clusters([], 3) == 0
    assert count_clusters([(1, 1), (1, 1)], 0) == 1
```

```
# ============================================================
# FOLLOW-UP 1: Report the size of the largest cluster
```

```
# ============================================================
# ---- follow-up 1: function ----
# Answer: count the points per root after the unions. The unions
# themselves are the same.
def largest_cluster(points, r):
    clusters = Clusters(r)
    for x, y in points:
        clusters.add(x, y)
    return clusters.largest()


# ---- follow-up 1: tests ----
def test_follow_up_1():
    line = [(0, 0), (3, 0), (6, 0), (20, 20)]
    assert largest_cluster(line, 3) == 3


# ============================================================
# FOLLOW-UP 2: Points arrive one at a time
# ============================================================
# Answer: add() above already does it: put the point in its cell,
# union it with every point within r in the 9 cells around, and
# components goes down by one for each union that really joined two groups.
# ---- follow-up 2: tests ----
def test_follow_up_2():
    c = Clusters(2)
    counts = []
    for p in [(0, 0), (10, 10), (1, 1), (5, 5)]:
        counts.append(c.add(p[0], p[1]))
    assert counts == [1, 2, 2, 3]
    assert c.add(3, 3) == 4          # (3, 3) is alone ...
    assert c.add(4, 4) == 3          # ... then (4, 4) bridges it to (5, 5)


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    print("all tests passed")


if __name__ == "__main__":
```

```
    run_all_tests()
```

76 Find A Threshold-Limited Path With Minimum Required Safety

MEDIUM

Template fit: pairs = edges sorted from the highest safety down; union until find(s) == find(t); the answer is the safety of that last edge

QUESTION

A graph has n nodes and edges (u, v, safety). The safety of a path is the smallest safety among its edges. Using only edges whose safety is at least threshold (no limit when threshold is None), return the best (largest) safety you can reach going from s to t, or -1 if no such path exists. A path from a node to itself has infinite safety.

Function signature

def best_path_safety(n: int, edges: list, s: int, t: int, threshold=None) -> int

QUESTION

EXAMPLES

Example 1	
Input	3, [(0, 1, 5), (1, 2, 3), (0, 2, 1)], 0, 2
Output	3
Why	Direct edge 0-2 has safety 1; the route 0-1-2 has safety min(5, 3) = 3, which is better.

Example 2	
Input	3, [(0, 1, 5), (1, 2, 3), (0, 2, 1)], 0, 2, threshold = 4
Output	-1
Why	Only edges with safety at least 4 may be used: 0-1 is allowed but nothing leads to node 2.

Example 3	
Input	3, [(0, 1, 5)], 1, 1
Output	infinity
Why	Source and target are the same node.

FULL CODE | main question first, then every follow-up, tests at the end

```
# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

import heapq


# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
class UnionFind:
    def __init__(self, n):
        self.parent = list(range(n))

    def find(self, x):
        while self.parent[x] != x:
            self.parent[x] = self.parent[self.parent[x]]
            x = self.parent[x]
        return x

    def union(self, a, b):
        self.parent[self.find(a)] = self.find(b)


def safety_of(edge):
    # an edge is (u, v, safety)
    return edge[2]


# ---- main question: main function ----
# Idea: the safety of a path is its weakest edge. Add edges from the
# safest to the least safe; the edge that finally joins s and t is the
# weakest edge of the best path. -1 if s and t never join (or only
# edges below `threshold` are allowed and that is not enough).
```

```
def best_path_safety(n, edges, s, t, threshold=None):
    if s == t:
        return float("inf")
    uf = UnionFind(n)
    for u, v, w in sorted(edges, key=safety_of, reverse=True):
        if threshold is not None and w < threshold:
            break                                   # follow-up 3: weaker edges are forbidden
```

```
        uf.union(u, v)
        if uf.find(s) == uf.find(t):
            return w
    return -1


# ---- main question: tests ----
def test_main_question():
    edges = [(0, 1, 5), (1, 2, 3), (0, 2, 1), (2, 3, 4)]
    assert best_path_safety(4, edges, 0, 3) == 3          # 0-1-2-3 has weakest edge 3; 0-2-3 only 1
    assert best_path_safety(3, [(0, 1, 9)], 0, 2) == -1    # not connected
    assert best_path_safety(3, [], 1, 1) == float("inf")    # source equals target


# ======================================================
# FOLLOW-UP 1: Use a max-heap search instead of Union-Find
# ======================================================
# ---- follow-up 1: function ----
# Answer: Dijkstra where the value of a path is its weakest edge, and
# we maximise it: take the best path first (a max-heap, so push the
# negated value) and extend it with min(value, edge): O(E log V).
def best_path_safety_heap(n, edges, s, t, threshold=None):
    graph = []
    for i in range(n):
        graph.append([])
    for u, v, w in edges:
        if threshold is None or w >= threshold:
            graph[u].append((v, w))
            graph[v].append((u, w))
    best = [-1] * n
    best[s] = float("inf")
    heap = [(-best[s], s)]
    while heap:
        neg, u = heapq.heappop(heap)
        value = -neg
        if value < best[u]:
            continue                                # an outdated entry
        for v, w in graph[u]:
            candidate = value
            if w < candidate:
                candidate = w                       # extending a path can only lower its weakest edge
            if candidate > best[v]:
                best[v] = candidate
                heapq.heappush(heap, (-candidate, v))
    return best[t]
```

```


# ---- follow-up 1: tests ----
def test_follow_up_1():
    edges = [(0, 1, 5), (1, 2, 3), (0, 2, 1), (2, 3, 4)]
    assert best_path_safety_heap(4, edges, 0, 3) == 3
    assert best_path_safety_heap(4, edges, 0, 3, 4) == -1     # edges below the threshold are forbidden
    assert best_path_safety_heap(3, [(0, 1, 9)], 0, 2) == -1


# ======================================================
# FOLLOW-UP 2: Answer many (source, target) queries
```

```python
# ============================================================
# Answer: build the union-find once while adding edges strongest
# first and remember when each pair of groups merged (a Kruskal
# reconstruction tree), or sort the queries too and answer them offline.


# ============================================================
# FOLLOW-UP 3: Edges below a threshold are forbidden
# ============================================================
# Answer: already handled by the `threshold` parameter: stop adding
# edges as soon as one is weaker than the threshold.
# ---- follow-up 3: tests ----
def test_follow_up_3():
    edges = [(0, 1, 5), (1, 2, 3), (0, 2, 1), (2, 3, 4)]
    assert best_path_safety(4, edges, 0, 3, 4) == -1


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

77 Movie Recommendations via Transitive Similarity

MEDIUM

Template fit: pairs = similar movies (use a dict instead of a list); group = every movie with the same root as the target, sorted by rating

QUESTION

You are given a target movie name, a dictionary ratings mapping each movie to its numeric rating, an integer n , and a list similarities of [movieX, movieY] pairs meaning the two movies are directly similar. Similarity is transitive: if A is similar to B and B is similar to C, then A, B, and C all belong to the same similarity group. Return up to n recommended movies from the target's similarity group, excluding the target itself, ordered by rating descending (break ties alphabetically by name). If n == 1 , this returns the single highest- rated similar movie. If the group has fewer than n other movies, return all of them.

Function signature

def recommend(target: str, ratings: dict[str, float], n: int,
similarities: list[list[str]]) -> list[str]:
...

EXAMPLES

Example 1	
Input	`target = "A", ratings = {"A": 8, "B": 5, "C": 9, "D": 7, "E": 6}, n = 1, similarities = [["A", "B"], ["B", "C"]]`
Output	`["C"]`
Why	`A, B, C form one group. Excluding A, the highest rated is C (9).`

QUESTION

Example 2	
Input	`target = "A", ratings = {"A": 8, "B": 5, "C": 9, "D": 7, "E": 6}, n = 2, similarities = [["A", "B"], ["B", "C"]]`
Output	`["C", "B"]`
Why	`Similar to A are {B (5), C (9)}; sorted by rating descending -> [C, B].`

Example 3	
Input	`target = "D", ratings = {"A": 8, "B": 5, "C": 9, "D": 7, "E": 6}, n = 3, similarities = [["A", "B"], ["B", "C"]]`
Output	`[]`
Why	`D is in no similarity pair, so its group is just {D}; excluding D leaves nothing.`

CONSTRAINTS

- Every movie mentioned appears as a key in ratings .
- 1 <= n ; ratings are numbers, names are distinct strings.

FULL CODE | main question first, then every follow-up, tests at the end

```
# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ========================================================

# ========================================================
# MAIN QUESTION
# ========================================================
# ---- main question: helper functions ----
class UnionFind:
    # union-find over names (any hashable value); unknown names join on first use
    def __init__(self):
        self.parent = {}

    def find(self, x):
        if x not in self.parent:
            self.parent[x] = x
        while self.parent[x] != x:
            self.parent[x] = self.parent[self.parent[x]]
            x = self.parent[x]
        return x

    def union(self, a, b):
        self.parent[self.find(a)] = self.find(b)


def best_first(movies, ratings):
    # the movies ordered by rating, highest first (ties by name)
    pairs = []
    for movie in movies:
        pairs.append((-ratings[movie], movie))
    pairs.sort()
    ordered = []
    for pair in pairs:
        ordered.append(pair[1])
    return ordered


# ---- main question: main function ----
# Idea: movies joined by "similar" pairs (directly or through other
```

```
# movies) form one group. Take the target's group and rank it by rating.
def recommend(target, ratings, n, similarities):
    uf = UnionFind()
    for x, y in similarities:
        uf.union(x, y)
    root = uf.find(target)
    group = []
    for movie in list(uf.parent):
        if movie != target and uf.find(movie) == root:
            group.append(movie)
    return best_first(group, ratings)[:n]


# ---- main question: tests ----
def test_main_question():
    r = {"A": 8, "B": 5, "C": 9, "D": 7, "E": 6}
    similar = [["A", "B"], ["B", "C"]]
    assert recommend("A", r, 1, similar) == ["C"]
    assert recommend("A", r, 2, similar) == ["C", "B"]
    assert recommend("D", r, 3, similar) == []


# ======================================================
# FOLLOW-UP 1: Thousands of recommendation queries on the same data
# ======================================================
# ---- follow-up 1: function ----
# Answer: build the groups once with union-find and sort each group by
# rating once, remembering each movie's group. A query then only looks
# up the group and takes the first n movies (skipping the target): O(n).
class Recommender:
    def __init__(self, ratings, similarities):
        uf = UnionFind()
        for x, y in similarities:
            uf.union(x, y)
        members = {}                        # leader -> movies of that group
        self.group_of = {}
        for movie in list(uf.parent):
            root = uf.find(movie)
            self.group_of[movie] = root
            if root not in members:
                members[root] = []
            members[root].append(movie)
        self.groups = {}
        for root in members:
            self.groups[root] = best_first(members[root], ratings)

    def recommend(self, target, n):
        if target not in self.group_of:
            return []
        picked = []
        for movie in self.groups[self.group_of[target]]:
            if movie != target and len(picked) < n:
                picked.append(movie)
        return picked


# ---- follow-up 1: tests ----
def test_follow_up_1():
    r = {"A": 8, "B": 5, "C": 9, "D": 7, "E": 6}
    rec = Recommender(r, [["A", "B"], ["B", "C"]])
    assert rec.recommend("A", 2) == ["C", "B"]
    assert rec.recommend("D", 3) == []
```

```python
# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

78 Determine Reporting Relationships

MEDIUM

Template fit: pairs = peer links; keep the boss at the root and carry it over when two groups merge

QUESTION

Build an in-memory organization model that supports three operations on employee IDs:

1. **`add_manager(a, b)`**: employee **a** is the direct manager of employee **b**.
2. **`add_peer(a, b)`**: employees **a** and **b** share the same direct manager.
3. **`is_manager(a, b)`**: return whether employee **a** is a manager of employee **b**. The answer should account for relationships that can be inferred from prior operations. For example, if **`add_manager(m, x)`** and **`add_peer(x, y)`** were previously recorded, then **`is_manager(m, y)`** should return **`true`**. You may assume the input is logically consistent and does not create invalid cycles in the org chart. Explain how you would design the data structures and implement these operations efficiently.

Implement an in-memory organization model that processes operations on employee IDs in order. Each operation is a tuple **`(op, a, b)`** where:

- **`("add_manager", a, b)`**: employee **a** becomes the direct manager of employee **b**.
- **`("add_peer", a, b)`**: employees **a** and **b** share the same direct manager.
- **`("is_manager", a, b)`**: ask whether employee **a** is a direct or indirect manager of employee **b**. Relationships can be inferred from earlier operations. For example, after **`add_manager(m, x)`** and **`add_peer(x, y)`**, **`is_manager(m, y)`** should be **`True`**. Process the operations sequentially and return a list of answers for all **`is_manager`** queries, in order. Notes:
- The input is logically consistent.
- No operation creates a cycle in the org chart.
- Repeating the same valid relationship is allowed.
- An employee is not considered their own manager.

Constraints
0 <= len(operations) <= 20000Employee IDs are integers in the range [-10^9, 10^9]At most 10000 distinct employee IDs appearThe sequence of operations is logically consistent and does not create cycles

EXAMPLES

```
Input: ([("add_manager", 1, 2), ("add_peer", 2, 3), ("is_manager", 1, 3)])
Expected Output: [True]
```

Explanation: Employee 2 reports to 1. Since 2 and 3 are peers, 3 also reports to 1.

```
Input: ([("add_manager", 2, 4), ("is_manager", 1, 4), ("add_manager", 1, 2), ("is_manager", 1, 4),
    ("is_manager", 2, 4), ("is_manager", 4, 2)])
Expected Output: [False, True, True, False]
```

QUESTION

Explanation: Before 2 gets a manager, 1 is not a manager of 4. After adding 1 as manager of 2, 1 becomes an indirect manager of 4. Hints**`add_peer(a, b)`** means **a** and **b** share one direct manager, but they are not interchangeable as managers. Group such employees with Union-Find.Once a peer group learns its direct manager, every current member of that group gets the same parent pointer. To answer **`is_manager(a, b)`**, walk upward from **b** through parent pointers.

FULL CODE | main question first, then every follow-up, tests at the end

```
# ====================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ====================================================

# ====================================================
# MAIN QUESTION
# ====================================================
# ---- main question: main function ----
# Idea: union-find joins peers (people with the same manager); the
# leader of every peer group stores that group's manager.
# add_peer(a, b)    a and b have the same manager
# add_manager(a, b) a is the direct manager of b (and of b's peers)
class Org:
    def __init__(self):
        self.parent = {}       # union-find over people
        self.boss = {}         # leader of a peer group -> the manager of the group

    def find(self, x):
        if x not in self.parent:
            self.parent[x] = x
        while self.parent[x] != x:
            self.parent[x] = self.parent[self.parent[x]]
            x = self.parent[x]
        return x

    def add_peer(self, a, b):
        ra, rb = self.find(a), self.find(b)
        if ra == rb:
            return
        if ra in self.boss and rb in self.boss and self.boss[ra] != self.boss[rb]:
            raise ValueError("conflicting managers")          # follow-up 2
        self.parent[rb] = ra
        if rb in self.boss and ra not in self.boss:
            self.boss[ra] = self.boss[rb]                     # the merged group keeps the known manager

    def add_manager(self, a, b):
        self.find(a)
        root = self.find(b)
        if root in self.boss and self.boss[root] != a:
            raise ValueError("conflicting managers")          # follow-up 2
        self.boss[root] = a
```

```
    def manager_of(self, x):
        return self.boss.get(self.find(x))

    def is_manager(self, a, b):
        # climb b's chain of managers; `seen` stops a cycle
        seen = set()
        person = b
        while person is not None and person not in seen:
            seen.add(person)
            person = self.manager_of(person)
            if person == a:
                return True
```

```
        return False


# ---- main question: tests ----
def test_main_question():
    o = Org()
    o.add_manager("ceo", "vp")
    o.add_manager("vp", "dev")
    o.add_peer("dev", "qa")
    assert o.is_manager("vp", "qa")               # qa shares dev's manager
    assert o.is_manager("ceo", "qa")
    assert not o.is_manager("qa", "vp")
    assert not o.is_manager("dev", "qa")
    p = Org()                                     # peers added before the manager is known
    p.add_peer("a", "b")
    p.add_manager("boss", "a")
    assert p.is_manager("boss", "b")


# ======================================================
# FOLLOW-UP 1: Return the closest common manager of two employees
# ======================================================
# ---- follow-up 1: function ----
# Answer: collect the chain of managers above one person into a set,
# then climb from the other person until you meet it: O(depth).
def common_manager(org, a, b):
    chain = set()
    person = org.manager_of(a)
    while person is not None and person not in chain:
        chain.add(person)
        person = org.manager_of(person)
    seen = set()
    person = org.manager_of(b)
```

```
    while person is not None and person not in seen:
        if person in chain:
            return person
        seen.add(person)
        person = org.manager_of(person)
    return None


# ---- follow-up 1: tests ----
def test_follow_up_1():
    o = Org()
    o.add_manager("ceo", "vp")
    o.add_manager("vp", "dev")
    o.add_peer("dev", "qa")
    assert common_manager(o, "dev", "qa") == "vp"
    assert common_manager(o, "vp", "qa") == "ceo"


# ======================================================
# FOLLOW-UP 2: Facts can be wrong or contradictory
# ======================================================
# Answer: two peer groups with different managers, or a second manager
# for the same group, is a contradiction; the Org methods above raise
# ValueError instead of silently picking one.
# ---- follow-up 2: tests ----
def test_follow_up_2():
    o = Org()
    o.add_manager("ceo", "vp")
    o.add_manager("vp", "dev")
    try:
        o.add_manager("ceo", "dev")
        raise AssertionError("should conflict")
```

```
    except ValueError:
        pass


# ============================================================
# FOLLOW-UP 3: Remove a relationship
# ============================================================
# Answer: union-find cannot split a group. Keep the list of facts and
# rebuild the structure without the removed fact (or use an offline
# "dynamic connectivity" method if removals are frequent).


# ============================================================
```

```
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

79 Determine whether two nodes are related

MEDIUM

Template fit: pairs = the given edges; the query is find(a) == find(b)

QUESTION

Nodes 0 to n - 1 are linked by edges, and two nodes are related when a chain of edges joins them (links work in both directions; the graph may be disconnected and may contain cycles). Support union(a, b) to add a link and related(a, b) to answer the query.

Function signature

class Relations:
  def __init__(self, n: int, edges=())
  def union(self, a: int, b: int) -> None
  def related(self, a: int, b: int) -> bool

EXAMPLES

Example 1

Input	`r = Relations(5, [(0, 1), (1, 2)])`
	`r.related(0, 2)`
	`r.related(0, 3)`
	`r.union(3, 4)`
	`r.related(3, 4)`
	`r.related(2, 4)`

Output	`r.related(0, 2) -> true`
	`r.related(0, 3) -> false`
	`r.related(3, 4) -> true`
	`r.related(2, 4) -> false`

Why	`0, 1, 2 form one group and 3, 4 another until they are joined; the two groups are never linked to each other.`

FULL CODE | main question first, then every follow-up, tests at the end

```
# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

from collections import deque


# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: main function ----
# Idea: union the two ends of every relationship; two nodes are
# related exactly when they have the same leader (root).
class Relations:
    def __init__(self, n, edges=()):
        self.parent = list(range(n))
        self.size = [1] * n
        for u, v in edges:
            self.union(u, v)

    def find(self, x):
        # walk up to the leader; path halving keeps the trees flat
        while self.parent[x] != x:
            self.parent[x] = self.parent[self.parent[x]]
            x = self.parent[x]
        return x

    def union(self, a, b):
        # relations can arrive over time: near-constant time each
        ra, rb = self.find(a), self.find(b)
        if ra == rb:
            return
        if self.size[ra] < self.size[rb]:
            ra, rb = rb, ra                # the smaller group joins the bigger one
        self.parent[rb] = ra
        self.size[ra] += self.size[rb]

    def related(self, a, b):
        return self.find(a) == self.find(b)


# ---- main question: tests ----
```

```
def test_main_question():
    r = Relations(5, [(0, 1), (1, 2)])
    assert r.related(0, 2) and not r.related(0, 3)
    r.union(3, 4)
    assert r.related(3, 4) and not r.related(2, 4)
    r.union(2, 3)
    assert r.related(0, 4)
    assert r.related(1, 1)


# ============================================================
# FOLLOW-UP 1: "Related" means ancestor or descendant, or sharing an ancestor
# ============================================================
# ---- follow-up 1: helper functions ----
def ancestors(parent, x):
    # x and everything above it; the check `not in seen` stops a corrupt cycle
    seen = []
    while x is not None and x not in seen:
        seen.append(x)
        x = parent.get(x)
    return seen
```

```
# ---- follow-up 1: function ----
# Answer: keep a parent map; list the ancestors of both people and
# they are related when the two lists share a person.
def kin(parent, a, b):
    up_a = ancestors(parent, a)
    up_b = ancestors(parent, b)
    for person in up_a:
        if person in up_b:
            return True
    return False


# ---- follow-up 1: tests ----
def test_follow_up_1():
    parent = {"kid": "mom", "mom": "gran", "uncle": "gran", "stranger": None}
    assert kin(parent, "kid", "gran")
    assert kin(parent, "kid", "uncle")
    assert not kin(parent, "kid", "stranger")
    assert kin({"a": "b", "b": "a"}, "a", "b")         # a corrupt cycle cannot loop forever


# ============================================================
```

```
# FOLLOW-UP 2: Relationships arrive over time and you must answer queries between updates
# ============================================================
# Answer: that is exactly what union() above is for: each new
# relationship is one union, and related() can be asked at any moment.


# ============================================================
# FOLLOW-UP 3: The graph is directed and "related" means reachability from a to b
# ============================================================
# ---- follow-up 3: function ----
# Answer: union-find ignores direction, so it cannot answer this. Search
# (BFS) per query, or find strongly connected components once and
# answer reachability on the condensed graph when there are many queries.
def reachable(graph, a, b):
    seen = {a}
    queue = deque([a])
    while queue:
        u = queue.popleft()
        if u == b:
            return True
        for v in graph.get(u, []):
            if v not in seen:
                seen.add(v)
                queue.append(v)
    return False


# ---- follow-up 3: tests ----
def test_follow_up_3():
    g = {"a": ["b"], "b": ["c"]}
    assert reachable(g, "a", "c")
    assert not reachable(g, "c", "a")                # direction matters


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_3()
```

```
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

80 Friend Graph with Timeline (Friend / Unfriend Events) HARD

Template fit: pairs = friend events sorted by time; the answer is the time of the union that makes groups == 1

QUESTION

There are n people numbered 0 to n - 1. Each event is (time, a, b, kind). A 'friend' event makes a and b friends from that time on, and two people are connected when a chain of friends links them. Return the earliest time at which all n people are connected, or -1 if that never happens.

Function signature

def earliest_all_connected(n: int, events: list) -> int

EXAMPLES

Example 1	
Input	3, [(1, 0, 1, 'friend'), (5, 1, 2, 'friend')]
Output	5
Why	After time 1, {0, 1} are connected; at time 5 person 2 joins, so everyone is connected at 5.

Example 2	
Input	4, [(1, 0, 1, 'friend'), (2, 1, 2, 'friend')]
Output	-1
Why	Person 3 is never befriended, so the group never becomes one.

FULL CODE | main question first, then every follow-up, tests at the end

```
# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ========================================================

from collections import deque


# ========================================================
# MAIN QUESTION
# ========================================================
# ---- main question: helper functions ----
def event_time(event):
    # events are (time, a, b, "friend" or "unfriend"); sort by the time
    return event[0]


class UnionFind:
    # union by size WITHOUT path compression, so every link keeps the time it was made
    # (follow-up 4 needs those times)
    def __init__(self, n):
        self.parent = list(range(n))      # everybody starts as their own leader
        self.size = [1] * n               # group sizes decide who hangs under whom
```

```
        self.link_time = [0] * n          # the time a person was linked under its parent
        self.groups = n                   # how many separate groups exist now

    def find(self, x):
        # walk up to the leader of x's group
        while self.parent[x] != x:
            x = self.parent[x]
        return x

    def union(self, a, b, time):
        ra, rb = self.find(a), self.find(b)
        if ra == rb:
            return False                  # already together: nothing changes
        if self.size[ra] < self.size[rb]:
            ra, rb = rb, ra               # ra is the bigger group
        self.parent[rb] = ra              # the smaller group hangs under the bigger one
        self.size[ra] += self.size[rb]
        self.link_time[rb] = time
        self.groups -= 1                  # two groups became one
        return True
```

```
# ---- main question: main function ----
# Idea: sort the events by time and union the two people of every
# friend event. The first time only one group is left is the answer;
# -1 if that never happens.
def earliest_all_connected(n, events):
    if n <= 1:
        return 0
    uf = UnionFind(n)
    for time, a, b, kind in sorted(events, key=event_time):
        if kind == "friend":
            uf.union(a, b, time)
            if uf.groups == 1:
                return time
    return -1


# ---- main question: tests ----
def test_main_question():
    events = [(1, 0, 1, "friend"), (5, 1, 2, "friend")]      # people A, B, C are 0, 1, 2
    assert earliest_all_connected(3, events) == 5
    assert earliest_all_connected(3, events[:1]) == -1       # a missing friendship: never connected
    assert earliest_all_connected(1, []) == 0


# ========================================================
# FOLLOW-UP 1: Why does unfriending break plain Union-Find?
# ========================================================
# Answer: a union merges two sets and forgets which edge caused it,
# so removing that edge cannot tell which members should separate.


# ========================================================
# FOLLOW-UP 2: What is the simple answer if the interviewer wants something quick?
# ========================================================
# ---- follow-up 2: function ----
# Answer: rebuild the graph from the events up to time t and run a BFS
# for each query: O(E + N) per query. State that cost first, then
# explain how the offline method avoids it.
def connected_with_unfriends(n, events, a, b, t):
    edges = set()
    for time, x, y, kind in sorted(events, key=event_time):
        if time > t:
            break
```

```
        if x < y:
            pair = (x, y)
        else:
            pair = (y, x)
        if kind == "friend":
            edges.add(pair)               # friend adds the edge
        else:
            edges.discard(pair)           # unfriend removes it
    adj = {}
    for x, y in edges:
        adj.setdefault(x, []).append(y)
        adj.setdefault(y, []).append(x)
    seen = {a}
    queue = deque([a])
    while queue:
        u = queue.popleft()
        if u == b:
            return True
        for v in adj.get(u, []):
            if v not in seen:
                seen.add(v)
                queue.append(v)
    return False


# ---- follow-up 2: tests ----
def test_follow_up_2():
    events = [(1, 0, 1, "friend"), (5, 1, 2, "friend"), (7, 1, 2, "unfriend")]
    assert connected_with_unfriends(3, events, 0, 2, 6)
    assert not connected_with_unfriends(3, events, 0, 2, 8)      # the unfriend broke the link


# ========================================================
# FOLLOW-UP 3: Report how many friend groups exist after each event (friends only)
# ========================================================
# ---- follow-up 3: function ----
# Answer: union-find with a group counter: it goes down by one when a
# union joins two different groups. Record it after each event.
def groups_after_each_event(n, events):
    uf = UnionFind(n)
    counts = []
    for time, a, b, kind in sorted(events, key=event_time):
        if kind == "friend":
            uf.union(a, b, time)
            counts.append(uf.groups)
```

```
    return counts


# ---- follow-up 3: tests ----
def test_follow_up_3():
    events = [(1, 0, 1, "friend"), (5, 1, 2, "friend")]
    assert groups_after_each_event(3, events) == [2, 1]        # groups: 3 -> 2 -> 1


# ========================================================
# FOLLOW-UP 4: Queries ask "were A and B connected at time t?" with no unfriending
# ========================================================
# ---- follow-up 4: function ----
# Answer: no path compression, and each link remembers its time. Two
# people were connected at time t when the NEWEST link on the path
# between them is not later than t.
def connected_at(uf, a, b, t):
    if a == b:
        return True
    newest_to = {a: 0}                    # climbing from a: ancestor -> newest link time so far
    newest = 0
```

```
    x = a
    while uf.parent[x] != x:
        if uf.link_time[x] > newest:
            newest = uf.link_time[x]
        x = uf.parent[x]
        newest_to[x] = newest
    newest = 0
    y = b
    while y not in newest_to:            # climb from b until we reach an ancestor of a
        if uf.parent[y] == y:
            return False                 # b's leader reached without meeting a
        if uf.link_time[y] > newest:
            newest = uf.link_time[y]
        y = uf.parent[y]
    if newest_to[y] > newest:
        newest = newest_to[y]
    return newest <= t


# ---- follow-up 4: tests ----
def test_follow_up_4():
    uf = UnionFind(3)
    for time, a, b in [(1, 0, 1), (5, 1, 2)]:
        uf.union(a, b, time)
```

```
    assert not connected_at(uf, 0, 2, 4)       # A and C are apart at time 4 ...
    assert connected_at(uf, 0, 2, 5)           # ... and together at time 5
    assert connected_at(uf, 1, 1, 0)


# ======================================================
# RUN EVERYTHING
# ======================================================
def run_all_tests():
    test_main_question()
    test_follow_up_2()
    test_follow_up_3()
    test_follow_up_4()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

A dict of per-key state that you update, expire, or rank

81 Robot Status Message Deduplication Window

EASY

Template fit: the template as written: show a message only if it was not shown in the last 10 seconds

QUESTION

Status events arrive from a robot as (time, message). Show a message only if the same message was not shown during the previous 10 seconds; a hidden message does not restart the timer, and exactly 10 seconds later it is shown again. show(t, msg) returns True when the message should be displayed.

Function signature

class Dedup:
    def show(self, t: int, msg: str) -> bool

EXAMPLES

Example 1

Input	`d = Dedup()`
	`d.show(0, 'low battery')`
	`d.show(4, 'low battery')`
	`d.show(6, 'door open')`
	`d.show(9, 'low battery')`
	`d.show(10, 'low battery')`

Output	`d.show(0, 'low battery') -> true`
	`d.show(4, 'low battery') -> false`
	`d.show(6, 'door open') -> true`
	`d.show(9, 'low battery') -> false`
	`d.show(10, 'low battery') -> true`

Why	The repeats at 4 and 9 are hidden and do not move the timer; at time 10 it is exactly 10 seconds after the last display, so it shows again.

FULL CODE | main question first, then every follow-up, tests at the end

```
# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ========================================================

# ========================================================
# MAIN QUESTION
# ========================================================
# ---- main question: helper functions ----
def first_at_least(items, x):
    # binary search in a sorted list: the first index whose value is >= x
    low = 0
    high = len(items)
    while low < high:
        middle = (low + high) // 2
        if items[middle] < x:
            low = middle + 1
        else:
            high = middle
    return low


def first_greater(items, x):
```

```
    # binary search in a sorted list: the first index whose value is > x
    low = 0
    high = len(items)
    while low < high:
        middle = (low + high) // 2
        if items[middle] <= x:
            low = middle + 1
        else:
            high = middle
    return low


# ---- main question: main class ----
# show(t, msg) says whether the message may be displayed now: not if it was
# displayed less than `window` seconds ago. Events arrive in time order.
# Idea: a dictionary with the time each message was last DISPLAYED. If the
# gap is still inside the window, hide the message WITHOUT changing the stored
# time (only a displayed event starts a new window); otherwise display it and
# store the new time. boundary_allowed says whether exactly `window` seconds later
# may show.
```

```
class Dedup:
    def __init__(self, window=10, boundary_allowed=True):
        self.window = window
        self.last = {}
        self.boundary_allowed = boundary_allowed

    def too_soon(self, gap):
        if self.boundary_allowed:
            return gap < self.window
        return gap <= self.window

    def show(self, t, msg):
        last = self.last.get(msg)
        if last is not None and self.too_soon(t - last):
            return False
        self.last[msg] = t
        return True


# ---- main question: tests ----
def test_main_question():
    d = Dedup()
    answers = []
    for t in (10, 13, 21):
        answers.append(d.show(t, "solar"))
    assert answers == [True, False, True]                # 13 is hidden and does not move the window
    d = Dedup()
    d.show(0, "a")
    assert d.show(10, "a")                               # exactly 10 seconds later shows by default
    d = Dedup(boundary_allowed=False)
    d.show(0, "a")
    assert not d.show(10, "a")                           # ... but is hidden if the boundary is not allowed


# ======================================================
# FOLLOW-UP 1: Events can arrive out of order
# ======================================================
# ---- follow-up 1: function ----
# Answer: decide the contract first. One option: judge a late event against
# the stored time with the absolute difference. Better: keep a SORTED list of
# the displayed times per message and check whether ANY displayed time is
# within `window` seconds on either side (two binary searches).
class DedupOutOfOrder:
    def __init__(self, window=10):
        self.window = window
```

```
        self.shown = {}                                 # message -> sorted list of displayed times

    def show(self, t, msg):
        if msg not in self.shown:
            self.shown[msg] = []
        times = self.shown[msg]
        low = first_greater(times, t - self.window)  # the first displayed time later than t - window
        high = first_at_least(times, t + self.window)
        if low < high:
            return False                                # a display within the window on one of the two sides
        times.insert(first_at_least(times, t), t)
        return True


# ---- follow-up 1: tests ----
def test_follow_up_1():
    dedup = DedupOutOfOrder()
    assert dedup.show(20, "x")
    assert not dedup.show(15, "x")                  # a late event close to a displayed one is hidden
    assert dedup.show(5, "x")                       # a late event far from every display shows


# ========================================================
# RUN EVERYTHING
# ========================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

82 Active Communication Top Users

MEDIUM

Template fit: a set of seen pairs, a Counter of partners per user, and buckets by count to read the top users in O(1)

QUESTION

A stream of messages like 'A-B' means users A and B talked. Each distinct pair counts once (A-B and B-A are the same pair) and a pair of a user with themselves is ignored. A user's count is how many different partners they have. Support add(pair), most_active() (the set of users with the highest count) and top_k(k) (k users by count, ties broken by name).

Function signature

class Active:
  def add(self, pair: str) -> None
  def most_active(self) -> set
  def top_k(self, k: int) -> list

EXAMPLES

QUESTION

Example 1

Input	`a = Active()`
	`a.add('A-B')`
	`a.add('B-C')`
	`a.add('B-A')`
	`a.most_active()`
	`a.top_k(2)`

Output	`a.most_active() -> {'B'}` `a.top_k(2) -> ['B', 'A']`
Why	B talked to A and C (2 partners). The repeated pair B-A adds nothing. A and C have 1 each, so the top 2 are B, then A (ties by name).

FULL CODE | main question first, then every follow-up, tests at the end

```
# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

import heapq


# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: main class ----
# A stream of messages like "A-B" means users A and B talked. Each DISTINCT
# pair counts once (A-B and B-A are the same pair) and a pair of a user with
# themselves is ignored. A user's count is how many different partners they have.
# add(pair), most_active() = the set of users with the highest count,
# top_k(k) = k users by count, ties broken by name.
# Idea: keep the set of edges seen (to drop repeats), a count per user, and
# BUCKETS: count -> the set of users with exactly that count. A new edge moves
# both users up one bucket, and most_active just reads the top bucket: O(1).
# (count_self=True is follow-up 1: a user talking to themselves counts, once.)
class Active:
    def __init__(self, count_self=False):
        self.edges = set()                      # the distinct pairs seen so far
        self.count = {}                         # user -> how many different partners
        self.bucket = {}                        # count -> the users with exactly that count
        self.top = 0                            # the highest count right now (it never drops)
        self.count_self = count_self

    def bump(self, user):
        old = self.count.get(user, 0)
        if old > 0:
            self.bucket[old].discard(user)      # leave bucket `old` ...
        self.count[user] = old + 1
        if old + 1 not in self.bucket:
            self.bucket[old + 1] = set()
        self.bucket[old + 1].add(user)          # ... and join the next bucket
        if old + 1 > self.top:
            self.top = old + 1

    def add(self, pair):
        a, b = pair.split("-")
        if a == b and not self.count_self:
            return                              # a user talking to themselves is ignored
        if a < b:
```

```
            edge = (a, b)                       # A-B and B-A are the same edge
        else:
            edge = (b, a)
        if edge in self.edges:
            return                              # a repeated pair counts only once
        self.edges.add(edge)
        self.bump(a)
        if a != b:
            self.bump(b)                        # a self pair counts once, not twice

    def most_active(self):
        if self.top == 0:
            return set()
        return set(self.bucket[self.top])

    def top_k(self, k):
        rows = []
        for user in self.count:
            rows.append((-self.count[user], user))
        best = heapq.nsmallest(k, rows)
        users = []
        for minus_count, user in best:
            users.append(user)
        return users


# ---- main question: tests ----
def test_main_question():
    a = Active()
    for pair in ["A-B", "B-C", "B-A"]:
        a.add(pair)
    assert a.most_active() == {"B"}                    # B talks to A and C; the repeat B-A is ignored
    assert a.count["B"] == 2 and a.count["A"] == 1
    assert a.top_k(2) == ["B", "A"]
    a.add("C-C")
    assert a.count["C"] == 1                           # self pairs are ignored by default
    assert Active().most_active() == set()


# ========================================================
# FOLLOW-UP 1: Self communication (A-A) should now count
# ========================================================
# Answer: pass a flag to the class (count_self=True): a self pair is one
```

```
# edge that adds one to that user's count, once. The set of edges already
# drops repeated self pairs.
# ---- follow-up 1: tests ----
def test_follow_up_1():
    s = Active(count_self=True)
    s.add("C-C")
    s.add("C-C")
    assert s.count["C"] == 1


# ========================================================
# FOLLOW-UP 2: Top-K with a changing K, or a fixed-size priority queue
# ========================================================
# Answer: top_k above builds the ranking on demand with a heap, so k can
# change on every call. For a FIXED k keep a min-heap of size k (updated when a
# count changes), at the price of lazy deletion of old counts.


# ========================================================
# FOLLOW-UP 3: The space claim
# ========================================================
# Answer: the memory is O(number of distinct pairs) for the edge set plus
```

```
# O(users) for the counts and buckets. Without the edge set (no dedup) it would
# be O(users).


# ====================================================
# RUN EVERYTHING
# ====================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

83 Filter Repeated Robot Status Messages

MEDIUM

Template fit: the same rule applied to a stream of (timestamp, message) pairs; keep what is accepted

QUESTION

A robot status stream is a list of (timestamp, message). Show a message when it has never been shown, or when strictly more than gap seconds (default 10) have passed since it was last shown. Messages are compared exactly (case-sensitive), a hidden repeat does not restart the interval, and the shown messages keep their input order. Return the shown (timestamp, message) pairs.

Function signature

def filter_status(stream: list, gap: int = 10) -> list

EXAMPLES

Example 1	
Input	`[(0, 'ok'), (5, 'ok'), (10, 'ok'), (11, 'ok'), (11, 'OK')]`
Output	`[(0, 'ok'), (11, 'ok'), (11, 'OK')]`
Why	`'ok' at 5 and 10 are hidden (not more than 10 seconds after 0); at 11 it is 11 seconds later, so it shows; 'OK' is a different message.`

Example 2	
Input	`[(0, 'low'), (3, 'low'), (7, 'low')], gap = 2`
Output	`[(0, 'low'), (3, 'low'), (7, 'low')]`
Why	`A smaller gap lets repeats through sooner.`

FULL CODE | main question first, then every follow-up, tests at the end

```
# ====================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ====================================================

import heapq


# ====================================================
# MAIN QUESTION
# ====================================================
# ---- main question: main function ----
# The stream is a list of (timestamp, message). Show a message only if MORE
```

```python
# than `gap` seconds have passed since it was last SHOWN; a hidden repeat
# changes nothing (it does not move the last-shown time).
# Idea: a dictionary from message to the time it was last shown. One lookup
# per message.
def filter_status(stream, gap=10):
    last_shown = {}
    shown = []
    for ts, msg in stream:
        if msg not in last_shown or ts - last_shown[msg] > gap:
            last_shown[msg] = ts
            shown.append((ts, msg))
    return shown


# ---- main question: tests ----
STREAM = [(0, "a"), (5, "a"), (10, "a"), (11, "a"), (12, "b"), (22, "a")]


def test_main_question():
    # 10 is hidden (not MORE than 10 after 0); 11 shows
    assert filter_status(STREAM) == [(0, "a"), (11, "a"), (12, "b"), (22, "a")]
    assert filter_status([(0, "A"), (1, "a")]) == [(0, "A"), (1, "a")]       # messages are case sensitive


# ========================================================
# FOLLOW-UP 1: The stream never ends; memory must be bounded
# ========================================================
# ---- follow-up 1: function ----
# Answer: a message older than the gap can never be blocked again, so
# forget it. Keep a min-heap of (time shown, message) and, before every
```

```python
# decision, remove the entries older than the gap; delete the dictionary key
# only if it still points at that time (it may have been shown again since).
class BoundedFilter:
    def __init__(self, gap=10):
        self.gap = gap
        self.last = {}                              # message -> the time it was last shown
        self.heap = []                              # (time shown, message), oldest first

    def show(self, ts, msg):
        while self.heap and ts - self.heap[0][0] > self.gap:
            old_ts, old_msg = heapq.heappop(self.heap)
            if self.last.get(old_msg) == old_ts:
                del self.last[old_msg]              # not shown again since: forget it
        if msg in self.last:
            return False
        self.last[msg] = ts
        heapq.heappush(self.heap, (ts, msg))
        return True


# ---- follow-up 1: tests ----
def test_follow_up_1():
    bounded = BoundedFilter()
    answers = []
    for ts, msg in STREAM:
        answers.append(bounded.show(ts, msg))
    assert answers == [True, False, False, True, True, True]
    assert len(bounded.last) <= 2


# ========================================================
# FOLLOW-UP 2: Several robots send messages
# ========================================================
```

```
# ============================================================
# FOLLOW-UP 3: The interval depends on the message severity
# ============================================================
# ---- follow-up 2: function ----
# Answer: use (robot, message) as the key (or one filter per robot): the
# same logic applies per key. For severity, look up the gap for that severity
# in a table (with a default).
class PerRobotFilter:
    def __init__(self, gaps, default=10):
        self.gaps = gaps                          # severity -> its gap
        self.default = default
        self.last = {}

    def show(self, robot, ts, msg, severity=None):
        key = (robot, msg)
        gap = self.gaps.get(severity, self.default)
        last = self.last.get(key)
        if last is not None and ts - last <= gap:
            return False
        self.last[key] = ts
        return True


# ---- follow-up 2: tests ----
def test_follow_up_2():
    robots = PerRobotFilter({"high": 2})
    assert robots.show("r1", 0, "x")
    assert not robots.show("r1", 9, "x")
    assert robots.show("r2", 1, "x")                  # another robot
    assert robots.show("r1", 1, "y", "high")
    assert robots.show("r1", 4, "y", "high")          # a stricter severity: a gap of 2 is enough


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

84 Next-Word Predictor — MEDIUM

Template fit: defaultdict(Counter): counts[a][b] += 1 for neighbouring words; remember the best next word per word

QUESTION

Train on sentences, then predict the word that most often follows a given word. train(sentence) learns every pair of neighbouring words; predict(word) returns the word that followed it most often (ties go to the alphabetically smaller word) or None if nothing ever followed it. predict must run in O(1).

Function signature

class Predictor:
  def train(self, sentence: str) -> None
  def predict(self, word: str) -> str | None

EXAMPLES

QUESTION

Example 1

Input	`p = Predictor()`
	`p.train('I love apples')`
	`p.train('I love pears')`
	`p.train('you love pears too')`
	`p.predict('love')`
	`p.predict('I')`
	`p.predict('too')`

Output	`p.predict('love') -> 'pears'`
	`p.predict('I') -> 'love'`
	`p.predict('too') -> null`

Why	`After 'love' came 'apples' once and 'pears' twice, so 'pears' wins. Nothing ever followed 'too'.`

FULL CODE | main question first, then every follow-up, tests at the end

```
# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

import heapq


# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: main class ----
# train(sentence) learns every pair of neighbouring words; predict(word)
# returns the word that followed it most often (ties: the alphabetically
# smaller word), or None. predict must be O(1).
# Idea: counts[word][next] is the number of times `next` followed `word`.
# After every increment compare that one count with the best successor kept for
# the word (a higher count wins, then the smaller word). predict then only has to
# read the stored best word.
class Predictor:
    def __init__(self):
        self.counts = {}                        # word -> {next word: how often}
        self.best = {}                          # word -> the most likely next word so far

    def train(self, sentence):
        words = sentence.split()
        for i in range(len(words) - 1):
            self.update(words[i], words[i + 1])

    def update(self, word, following):
        if word not in self.counts:
            self.counts[word] = {}
        table = self.counts[word]
        table[following] = table.get(following, 0) + 1
        current = self.best.get(word)
        if current is None:
            self.best[word] = following
        elif table[following] > table[current]:
            self.best[word] = following
        elif table[following] == table[current] and following < current:
            self.best[word] = following             # a tie goes to the smaller word

    def predict(self, word):
```

```
        return self.best.get(word)


# ---- main question: tests ----
def test_main_question():
    p = Predictor()
    p.train("i love dogs")
    p.train("i love cats")
    p.train("i love cats and dogs")
    assert p.predict("love") == "cats"             # "love" is followed by cats twice, dogs once
    assert p.predict("zebra") is None
    q = Predictor()
    q.train("a z")
    q.train("a b")
    assert q.predict("a") == "b"                   # a tie goes to the alphabetically smaller word


# ========================================================
# FOLLOW-UP 1: Return the top K predictions
# ========================================================
# ---- follow-up 1: function ----
# Answer: the table of counts per word is already there: pick the k best
# (count, word) rows with a heap (heapq.nsmallest on (-count, word)): O(d log k)
# per query for d different followers (or keep a sorted structure per word if
# queries are much more common than training).
def top_k(predictor, word, k):
    rows = []
    for following, count in predictor.counts.get(word, {}).items():
        rows.append((-count, following))
    best = heapq.nsmallest(k, rows)
    words = []
    for minus_count, following in best:
        words.append(following)
    return words


# ---- follow-up 1: tests ----
def test_follow_up_1():
    p = Predictor()
    p.train("i love dogs")
    p.train("i love cats")
    p.train("i love cats and dogs")
    assert top_k(p, "love", 2) == ["cats", "dogs"]
    assert top_k(p, "zebra", 2) == []
```

```


# ========================================================
# FOLLOW-UP 2: Use two words of context (trigrams)
# ========================================================
# ---- follow-up 2: function ----
# Answer: the key becomes the PAIR of previous words and everything else is
# the same, at the price of more memory. Try the longest context first and
# back off to a shorter one when that context was never seen. orders=(2, 1)
# means "two words of context, then one".
class ContextPredictor:
    def __init__(self, orders=(2, 1)):
        self.orders = sorted(orders, reverse=True)        # the longest context first
        self.counts = {}                                  # (order, context words) -> {next word: how often}
        self.best = {}                                    # (order, context words) -> the most likely next wo
rd

    def train(self, sentence):
        words = sentence.split()
        for k in self.orders:
            for i in range(len(words) - k):
                key = (k, tuple(words[i:i + k]))
                self.update(key, words[i + k])
```

```
    def update(self, key, following):
        if key not in self.counts:
            self.counts[key] = {}
        table = self.counts[key]
        table[following] = table.get(following, 0) + 1
        current = self.best.get(key)
        if current is None or table[following] > table[current]:
            self.best[key] = following
        elif table[following] == table[current] and following < current:
            self.best[key] = following

    def predict(self, *context):
        for k in self.orders:
            if len(context) >= k:
                key = (k, tuple(context[len(context) - k:]))
                if key in self.best:
                    return self.best[key]
        return None


# ---- follow-up 2: tests ----
def test_follow_up_2():
    p = ContextPredictor()
```

```
    p.train("i love dogs")
    p.train("i love cats")
    p.train("i love cats and dogs")
    assert p.predict("love") == "cats"
    assert p.predict("i", "love") == "cats"             # two words of context are tried first
    assert p.predict("we", "love") == "cats"            # an unseen pair backs off to the last word
    assert p.predict("zebra") is None


# ========================================================
# FOLLOW-UP 3: The corpus keeps changing and old text should fade
# ========================================================
# Answer: multiply the old counts by a factor below 1 from time to time
# (exponential decay), or count only the most recent N sentences with a
# queue and subtract the counts of the sentence that leaves.


# ========================================================
# RUN EVERYTHING
# ========================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

85 Batch and Streaming Log Deduplication

MEDIUM

Template fit: dict message -> first (or latest) timestamp depending on the policy; sort the result at the end

QUESTION

Given log entries (timestamp, message), keep one entry per message and return the kept entries sorted by timestamp. With policy 'first' keep the entry seen first for each message; with policy 'latest' keep the one with the latest timestamp.

Function signature

def dedupe(logs: list, policy: str = 'first') -> list

EXAMPLES

Example 1	
Input	`[(5, 'a'), (1, 'b'), (9, 'a'), (3, 'c'), (2, 'b')]`
Output	`[(1, 'b'), (3, 'c'), (5, 'a')]`
Why	`Messages 'a' and 'b' each appear twice; the first one seen is kept.`

Example 2	
Input	`[(5, 'a'), (1, 'b'), (9, 'a'), (3, 'c'), (2, 'b')], policy = 'latest'`
Output	`[(2, 'b'), (3, 'c'), (9, 'a')]`
Why	`With 'latest', the later timestamp of each repeated message wins.`

FULL CODE | main question first, then every follow-up, tests at the end

```python
# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

import heapq


# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: main function ----
# logs are (timestamp, message) pairs. Keep ONE entry per message: the
# first one seen (policy "first") or the newest (policy "latest").
# Idea: one pass with a dictionary from message to the kept timestamp; then
# sort the kept entries by time for the output.
def dedupe(logs, policy="first"):
    kept = {}
    for ts, msg in logs:
        if msg not in kept:
            kept[msg] = ts
        elif policy == "latest" and ts > kept[msg]:
            kept[msg] = ts
    entries = []
    for msg in kept:
        entries.append((kept[msg], msg))
    entries.sort()
    return entries


# ---- main question: tests ----
def test_main_question():
    logs = [(5, "b"), (1, "a"), (9, "a"), (3, "b")]
    assert dedupe(logs, "first") == [(1, "a"), (5, "b")]
    assert dedupe(logs, "latest") == [(5, "b"), (9, "a")]
```

```
# ============================================================
# FOLLOW-UP 1: Late data: how do you decide when it is safe to emit?
# ============================================================


# ============================================================
```

```
# FOLLOW-UP 4: Only release entries once they are safe (late data may still arrive)
# ============================================================
# ---- follow-up 4: function ----
# Answer: use a WATERMARK: release only entries whose timestamp is at most
# (the largest timestamp seen minus the allowed lateness), in time order.
# Anything that arrives older than the watermark goes to a late-data path
# (here: it is dropped). Entries wait in a min-heap; an entry that was replaced
# by a newer one is skipped when it reaches the top ("lazy deletion").
class StreamDedup:
    def __init__(self, policy="first", lateness=0):
        self.policy = policy
        self.lateness = lateness
        self.kept = {}                              # message -> the timestamp currently kept
        self.heap = []                              # (timestamp, message); may hold outdated entries
        self.max_ts = None

    def watermark(self):
        # everything at or below this is safe to release
        if self.max_ts is None:
            return None
        return self.max_ts - self.lateness

    def add(self, ts, msg):
        mark = self.watermark()
        if mark is not None and ts < mark:
            return False                            # too late: older than the watermark
        if self.max_ts is None or ts > self.max_ts:
            self.max_ts = ts
        old = self.kept.get(msg)
        if old is None or (self.policy == "latest" and ts > old):
            self.kept[msg] = ts
            heapq.heappush(self.heap, (ts, msg))
        return True

    def get_next(self):
        mark = self.watermark()
        while self.heap:
            ts, msg = self.heap[0]
            if self.kept.get(msg) != ts:
                heapq.heappop(self.heap)            # this entry was replaced: skip it
                continue
            if ts > mark:
                return None                         # not safe yet: later data may still arrive
            heapq.heappop(self.heap)
            del self.kept[msg]
```

```
            return ts, msg
        return None


# ---- follow-up 4: tests ----
def test_follow_up_4():
    s = StreamDedup("first", lateness=2)
    for ts, msg in [(1, "a"), (3, "c"), (5, "b")]:
        assert s.add(ts, msg)
    assert s.get_next() == (1, "a")                 # the watermark is 5 - 2 = 3: release 1, then 3
    assert s.get_next() == (3, "c")
```

```
    assert s.get_next() is None                    # 5 is still above the watermark
    assert not s.add(2, "late")                    # data older than the watermark is dropped
    t = StreamDedup("latest")
    t.add(1, "x")
    t.add(4, "x")
    assert t.get_next() == (4, "x")                # the "latest" policy replaced the kept entry
    assert t.get_next() is None


# ============================================================
# FOLLOW-UP 2: Memory must stay bounded
# ============================================================
# Answer: only messages that can still repeat must be remembered: with a
# watermark, forget a message once its entry has been released and it is
# older than the lateness window (the streaming class does this by deleting
# a message when it is released).


# ============================================================
# FOLLOW-UP 3: Keep the entry with the most occurrences instead
# ============================================================
# Answer: keep a count per message as well and keep the entry with the largest
# count (ties: the earliest); the dictionary then maps a message to (count, ts).


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_4()
    print("all tests passed")
```

```
if __name__ == "__main__":
    run_all_tests()
```

86 Design a logger that returns top-K messages

MEDIUM

Template fit: dict message -> list of timestamps; top_k counts each list inside the time range with bisect, then takes the k largest

QUESTION

Design a logger. log(ts, msg) records a message at time ts (timestamps arrive in non-decreasing order). top_k(k, start, end) returns the k most frequent messages whose timestamps fall in [start, end]; leave start and end out to use the whole history. Ties are broken alphabetically.

Function signature

class Logger:
    def log(self, ts: int, msg: str) -> None
    def top_k(self, k: int, start=None, end=None) -> list

EXAMPLES

QUESTION

Example 1

Input	`lg = Logger()`
	`lg.log(1, 'disk full')`
	`lg.log(2, 'timeout')`
	`lg.log(3, 'timeout')`
	`lg.log(4, 'disk full')`
	`lg.log(5, 'timeout')`
	`lg.top_k(1)`
	`lg.top_k(2, 4, 5)`
	`lg.top_k(2, 1, 1)`

Output	`lg.top_k(1) -> ['timeout']`
	`lg.top_k(2, 4, 5) -> ['disk full', 'timeout']`
	`lg.top_k(2, 1, 1) -> ['disk full']`

Why	Overall 'timeout' occurred 3 times. In the window [4, 5] 'disk full' and 'timeout' occur once each (tie, alphabetical). The window [1, 1] only holds 'disk full'.

FULL CODE | main question first, then every follow-up, tests at the end

```
# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ========================================================

import heapq


# ========================================================
# MAIN QUESTION
# ========================================================
# ---- main question: helper functions ----
def first_at_least(items, x):
    # binary search in a sorted list: the first index whose value is >= x
    low = 0
    high = len(items)
    while low < high:
        middle = (low + high) // 2
        if items[middle] < x:
            low = middle + 1
        else:
            high = middle
    return low


def first_greater(items, x):
    # binary search in a sorted list: the first index whose value is > x
    low = 0
    high = len(items)
    while low < high:
        middle = (low + high) // 2
        if items[middle] <= x:
            low = middle + 1
        else:
            high = middle
    return low


# ---- main question: main class ----
# log(ts, msg) records a message at time ts (timestamps never go backwards).
# top_k(k, start, end) returns the k most frequent messages with a timestamp in
# [start, end] (the whole history if they are left out); ties alphabetically.
```

```
# Idea: keep, per message, the list of its timestamps. They arrive in
```

```
# order, so every list stays sorted. The number of times a message occurs in
# [start, end] is two binary searches: (first index > end) - (first index >= start).
# A heap then picks the k best (count, message) rows.
class Logger:
    def __init__(self):
        self.times = {}                          # message -> its timestamps, in order of arrival

    def log(self, ts, msg):
        if msg not in self.times:
            self.times[msg] = []
        self.times[msg].append(ts)

    def top_k(self, k, start=None, end=None):
        rows = []
        for msg in self.times:
            stamps = self.times[msg]
            low = 0
            high = len(stamps)
            if start is not None:
                low = first_at_least(stamps, start)
            if end is not None:
                high = first_greater(stamps, end)
            if high > low:
                rows.append((-(high - low), msg))      # minus the count: the most frequent sorts first
        best = heapq.nsmallest(k, rows)
        messages = []
        for minus_count, msg in best:
            messages.append(msg)
        return messages

    def expire(self, before):
        # bounded memory (follow-up 2): forget timestamps older than `before`
        for msg in list(self.times):
            stamps = self.times[msg]
            self.times[msg] = stamps[first_at_least(stamps, before):]
            if not self.times[msg]:
                del self.times[msg]


# ---- main question: tests ----
def test_main_question():
    logger = Logger()
    for ts, msg in [(1, "a"), (2, "b"), (3, "a"), (4, "c"), (5, "a"), (6, "b")]:
        logger.log(ts, msg)
    assert logger.top_k(2) == ["a", "b"]
```

```
    assert logger.top_k(3, 4, 6) == ["a", "b", "c"]          # only the window [4, 6]: a once, b once, c once
    assert logger.top_k(2, 100, 200) == []
    logger.expire(4)
    assert logger.top_k(5) == ["a", "b", "c"]


# ========================================================
# FOLLOW-UP 1: Print a message only if it was not printed in the last 10 seconds (LeetCode 359)
# ========================================================
# ---- follow-up 1: function ----
# Answer: keep a dictionary message -> the time it was last PRINTED, and
# compare: print if the gap is at least 10 (and update the time only when you
# print). O(1) per call.
class RateLimiter:
    def __init__(self, gap=10):
        self.gap = gap
        self.last = {}
```

```
    def should_print(self, ts, msg):
        last = self.last.get(msg)
        if last is not None and ts - last < self.gap:
            return False
        self.last[msg] = ts                          # only a printed message starts a new window
        return True


# ---- follow-up 1: tests ----
def test_follow_up_1():
    limiter = RateLimiter()
    answers = []
    for t in (1, 2, 11, 12, 21):
        answers.append(limiter.should_print(t, "x"))
    assert answers == [True, False, True, False, True]


# ========================================================
# FOLLOW-UP 2: Memory must stay bounded
# ========================================================
# Answer: forget old data: expire(before) above drops the timestamps older
# than a limit, and removes messages that have none left. Run it from time to
# time (or on every log call for the oldest entries).


# ========================================================
# FOLLOW-UP 3: Very high top_k frequency of calls
```

```
# ========================================================
# Answer: do not rescan every message on each call: keep a running count per
# message and a heap or sorted structure that is updated when a count changes,
# and cache the answer for the whole history until the next log call.


# ========================================================
# RUN EVERYTHING
# ========================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

87 Hit Counter — MEDIUM

Template fit: one deque of hit times; drop the ones older than 300 seconds; the answer is the length

QUESTION

Design a hit counter. hit(timestamp) records a hit at that second, and get_hits(timestamp) returns the number of hits in the past 300 seconds (timestamps never go backwards).

Function signature

class QueueCounter:
    def hit(self, ts: int) -> None
    def get_hits(self, ts: int) -> int

EXAMPLES

QUESTION

Example 1

Input	`h = QueueCounter()`
	`h.hit(1)`
	`h.hit(2)`
	`h.hit(3)`
	`h.get_hits(4)`
	`h.hit(300)`
	`h.get_hits(300)`
	`h.get_hits(301)`

Output	`h.get_hits(4) -> 3`
	`h.get_hits(300) -> 4`
	`h.get_hits(301) -> 3`

Why	At time 301 the hit at time 1 is exactly 300 seconds old and no longer counts.

CONSTRAINTS

- 1 <= timestamp <= 2 * 10^9
- Timestamps are monotonically increasing

FULL CODE | main question first, then every follow-up, tests at the end

```
# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

import random
import threading
from collections import deque

WINDOW = 300                                    # seconds


# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: main class ----
# hit(ts) records a hit at second ts; get_hits(ts) returns the number of
# hits in the past 300 seconds (timestamps never go backwards).
# Idea: keep a queue of the hit times in order of arrival. get_hits removes
# the hits from the front while the oldest is outside the window; what is left
# is the answer. Every hit is added once and removed once: O(1) amortised.
class QueueCounter:
    def __init__(self):
        self.hits = deque()

    def hit(self, ts):
        self.hits.append(ts)

    def get_hits(self, ts):
        while self.hits and self.hits[0] <= ts - WINDOW:     # keep only the hits in (ts - 300, ts]
            self.hits.popleft()
        return len(self.hits)


# ---- main question: tests ----
def test_main_question():
    counter = QueueCounter()
    counter.hit(1)
    counter.hit(2)
```

```
    counter.hit(3)
    assert counter.get_hits(4) == 3
    counter.hit(300)
    assert counter.get_hits(300) == 4
```

```
    assert counter.get_hits(301) == 3


# ========================================================
# FOLLOW-UP 1: Millions of hits per second
# ========================================================
# ---- follow-up 1: function ----
# Answer: use 300 BUCKETS (one per second of the window) instead of one
# entry per hit: hit is O(1), get_hits adds up 300 buckets (constant), and
# memory is constant. A bucket that still holds an older second is reset when it
# is reused.
class BucketCounter:
    def __init__(self):
        self.stamp = [0] * WINDOW              # which second each bucket holds
        self.count = [0] * WINDOW

    def hit(self, ts):
        slot = ts % WINDOW
        if self.stamp[slot] != ts:
            self.stamp[slot] = ts              # the bucket still holds an old second: reuse it
            self.count[slot] = 0
        self.count[slot] += 1

    def get_hits(self, ts):
        total = 0
        for slot in range(WINDOW):
            if 0 <= ts - self.stamp[slot] < WINDOW:
                total += self.count[slot]
        return total


# ---- follow-up 1: tests ----
def test_follow_up_1():
    counter = BucketCounter()
    for ts in (1, 2, 3, 300):
        counter.hit(ts)
    assert counter.get_hits(300) == 4
    assert counter.get_hits(301) == 3
    assert counter.get_hits(600) == 0


# ========================================================
# FOLLOW-UP 2: Timestamps can arrive out of order
# ========================================================
# ---- follow-up 2: helper functions ----
```

```
def count_not_after(items, x):
    # binary search in a sorted list: how many items are <= x
    low = 0
    high = len(items)
    while low < high:
        middle = (low + high) // 2
        if items[middle] <= x:
            low = middle + 1
        else:
            high = middle
    return low


# ---- follow-up 2: function ----
# Answer: a queue no longer works. Keep the times in a SORTED list and count
```

```
# with two binary searches: the hits up to ts minus the hits up to ts - 300.
# (The buckets also work: they do not care about the order inside the window.)
class SortedCounter:
    def __init__(self):
        self.hits = []

    def hit(self, ts):
        position = count_not_after(self.hits, ts)
        self.hits.insert(position, ts)         # insert so the list stays sorted

    def get_hits(self, ts):
        return count_not_after(self.hits, ts) - count_not_after(self.hits, ts - WINDOW)


# ---- follow-up 2: tests ----
def test_follow_up_2():
    for make in (QueueCounter, BucketCounter, SortedCounter):
        counter = make()
        for ts in (1, 2, 3, 300):
            counter.hit(ts)
        assert counter.get_hits(300) == 4
        assert counter.get_hits(301) == 3               # the hit at second 1 is 300 seconds old: it is out
        assert counter.get_hits(599) == 1
        assert counter.get_hits(600) == 0
        assert counter.get_hits(1000) == 0
    random.seed(24)
    for make in (QueueCounter, BucketCounter, SortedCounter):
        counter = make()
        log = []
        now = 0
        for step in range(500):
            now += random.randint(0, 40)
            if random.random() < 0.6:
                counter.hit(now)
                log.append(now)
            expected = 0
            for t in log:
                if now - t < WINDOW:
                    expected += 1
            assert counter.get_hits(now) == expected
    late = SortedCounter()                                  # out-of-order input only works with the sorted vers
ion
    for ts in (50, 10, 400, 30):
        late.hit(ts)
    assert late.get_hits(400) == 1
    assert late.get_hits(310) == 2


# ========================================================
# FOLLOW-UP 3: Thread safety
# ========================================================
# ---- follow-up 3: function ----
# Answer: guard the counter with a lock so that check-then-update steps
# (like resetting a bucket) are atomic; or keep a counter per thread and add
# them up when reading. The cost is one lock per call: contention, not
# complexity, is the concern.
class Locked:
    def __init__(self, counter):
        self.counter = counter
        self.lock = threading.Lock()

    def hit(self, ts):
        with self.lock:
            self.counter.hit(ts)
```

```
    def get_hits(self, ts):
        with self.lock:
            return self.counter.get_hits(ts)


# ---- follow-up 3: tests ----
def test_follow_up_3():
    shared = Locked(BucketCounter())

    def spam():
        for i in range(1000):
            shared.hit(7)

    workers = []
    for i in range(4):
        workers.append(threading.Thread(target=spam))
    for worker in workers:
        worker.start()
    for worker in workers:
        worker.join()
    assert shared.get_hits(7) == 4000


# =============================================================
# RUN EVERYTHING
# =============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

88 Website Activity Analytics: Unique Users + Session Time

MEDIUM

Template fit: dict activity -> set of users for unique counts; dict user -> sorted events to add up login-to-logout session time

QUESTION

Each record has a user_id, a timestamp ('YYYY-MM-DD HH:MM:SS') and an activity. Return (1) the number of distinct users per activity and (2) the average number of seconds a user spends on the site. A user's time is the sum of the gaps from each 'login' to the next 'logout'; a login without a logout counts until the user's last event.

Function signature

def activity_stats(records: list) -> (dict, float)

EXAMPLES

Example 1	

Input	`rec = lambda u, t, a: {'user_id': u, 'timestamp': '2026-01-01 ' + t, 'activity': a}`
	`records = [rec(1, '10:00:00', 'login'), rec(1, '10:10:00', 'click'), rec(1, '10:30:00', 'logout'), rec(2, '10:05:00', 'login'), rec(2, '10:20:00', 'logout')]`
	`activity_stats(records)`

Output	`activity_stats(records) -> ({'click': 1, 'login': 2, 'logout': 2}, 1350)`
Why	User 1 spent 30 minutes (1800 s), user 2 spent 15 minutes (900 s): the average is 1350 s. Both users logged in and out; only user 1 clicked.

FULL CODE | main question first, then every follow-up, tests at the end

```
# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

from datetime import datetime

FORMAT = "%Y-%m-%d %H:%M:%S"


# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
def when(record):
    # the timestamp text of a record as a date and time
    return datetime.strptime(record["timestamp"], FORMAT)


def unique_users_by_activity(records):
    # per activity: how many DIFFERENT users did it (a set counts each user once)
    users = {}
    for record in records:
        if record["activity"] not in users:
            users[record["activity"]] = set()
        users[record["activity"]].add(record["user_id"])
    counts = {}
    for activity in sorted(users):
        counts[activity] = len(users[activity])
    return counts


def session_seconds(events):
    # one user's (time, activity) rows, in any order: the seconds spent logged in.
    # Sort by time first (follow-up 1: rows may arrive out of order). A second
    # login before a logout is ignored; a login with no logout ends at the
    # user's last event.
    total = 0
    start = None
    for t, activity in sorted(events):
        if activity == "login" and start is None:
            start = t
        elif activity == "logout" and start is not None:
            total += (t - start).total_seconds()
            start = None
    if start is not None:
        last = events[0][0]
        for t, activity in events:
            if t > last:
                last = t
        total += (last - start).total_seconds()
    return total


def average_time_per_user(records):
    # the average session time over EVERY user who appears (state the denominator)
    events = {}
    for record in records:
        if record["user_id"] not in events:
            events[record["user_id"]] = []
        events[record["user_id"]].append((when(record), record["activity"]))
    if not events:
        return 0.0
```

```
    total = 0
    for user in events:
        total += session_seconds(events[user])
    return total / len(events)


# ---- main question: main function ----
# records are {"user_id", "activity", "timestamp"}. Return the number of
# unique users per activity and the average session time per user.
# Idea: one pass groups the records, then each user's rows are paired up
# login -> logout.
def activity_stats(records):
    return unique_users_by_activity(records), average_time_per_user(records)


# ---- main question: tests ----
def record(user, minute, activity):
    return {"user_id": user, "activity": activity, "timestamp": "2024-07-26 10:" + str(minute).zfill(2) + ":0
0"}


def test_main_question():
    data = [record(1, 0, "login"), record(2, 5, "login"), record(1, 10, "view"), record(1, 15, "purchase"),
            record(2, 20, "view"), record(3, 25, "login"), record(1, 30, "logout")]
    unique, average = activity_stats(data)
    assert unique == {"login": 3, "logout": 1, "purchase": 1, "view": 2}
```

```
    assert average == 900                                # user 1: 1800 s, user 2: 900 s (no logout), user
 3: 0
    assert activity_stats([]) == ({}, 0.0)


# ========================================================
# FOLLOW-UP 1: Users with several sessions, missing logouts or out-of-order rows
# ========================================================
# Answer: sort each user's events by time before pairing them (session_seconds
# does); two sessions add up; a second login before a logout is ignored (or
# closes the old session, depending on the contract you agree on); a missing
# logout ends the session at the user's last event.
# ---- follow-up 1: tests ----
def test_follow_up_1():
    shuffled = [record(1, 30, "logout"), record(1, 0, "login")]
    assert average_time_per_user(shuffled) == 1800          # out-of-order rows are sorted first
    two = [record(1, 0, "login"), record(1, 5, "logout"), record(1, 10, "login"), record(1, 20, "logout")]
    assert average_time_per_user(two) == 900                # two sessions add up: 300 + 600 = 900


# ========================================================
# RUN EVERYTHING
# ========================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

Binary search on a yes/no condition

89 First Index Reaching a Threshold (Binary Search) EASY

Template fit: x = an index; ok(x) = arr[x] >= k on a sorted array

QUESTION

Given a non-decreasing (sorted ascending, duplicates allowed) array arr and a target value k , return the smallest index i such that arr[i] >= k — the first position where the array "reaches" the value k . If no element is >= k , return -1 . Solve in O(log n) time and O(1) extra space.

Function signature

def first_index_at_least(arr: list[int], k: int) -> int:

    ...

Follow-up (as reported): The interviewer probed space complexity; note the solution is already O(1) space. If arr is a huge sorted stream/file, describe reading via random access rather than loading it all.

EXAMPLES

Example 1	
Input	`arr = [1, 2, 2, 3, 5, 7], k = 2`
Output	`1`
Why	`arr[1] = 2 is the first element >= 2.`

Example 2	
Input	`arr = [1, 2, 2, 3, 5, 7], k = 4`
Output	`4`
Why	`The first element >= 4 is 5 at index 4.`

Example 3	
Input	`arr = [1, 2, 2, 3, 5, 7], k = 8`
Output	`-1`
Why	`No element is >= 8.`

CONSTRAINTS

- 0 <= len(arr) <= 10**7
- Values and k are integers.

FULL CODE | main question first, then every follow-up, tests at the end

```
# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ========================================================

# ========================================================
# MAIN QUESTION
# ========================================================
# ---- main question: main function ----
# Idea: the array is sorted, so the values below k come first and the
# values at least k come after. Halve the range while keeping that
# boundary inside it; when the range is one spot, that spot is the first
```

```python
# value >= k. -1 if every value is smaller.
def first_index_at_least(arr, k):
    lo = 0
    hi = len(arr)
    while lo < hi:
        mid = (lo + hi) // 2
        if arr[mid] >= k:
            hi = mid                        # mid could be the answer: keep it
        else:
            lo = mid + 1                    # mid is too small: the answer is after it
    if lo < len(arr):
        return lo
    return -1


# ---- main question: tests ----
DATA = [1, 2, 2, 3, 5, 7]


def test_main_question():
    assert first_index_at_least(DATA, 2) == 1
    assert first_index_at_least(DATA, 4) == 4
    assert first_index_at_least(DATA, 8) == -1
    assert first_index_at_least([], 1) == -1


# ======================================================
# FOLLOW-UP 1: The array is a huge sorted file with random access and an unknown length
# ======================================================
# ---- follow-up 1: function ----
# Answer: probe positions 1, 2, 4, 8, ... (doubling) until the value
```

```python
# there is at least k or the position is past the end, then binary search
# inside the last doubled range. O(log position) reads, and the data is
# never loaded. get(i) returns the i-th value or raises IndexError past the end.
def first_at_least_unbounded(get, k):
    def value(i):
        try:
            return get(i)
        except IndexError:
            return float("inf")             # past the end counts as "bigger than anything"

    hi = 1
    while value(hi - 1) < k:
        hi *= 2
        if hi > (1 << 40):
            return -1
    lo = hi // 2
    while lo < hi:
        mid = (lo + hi) // 2
        if value(mid) >= k:
            hi = mid
        else:
            lo = mid + 1
    if value(lo) == float("inf"):
        return -1
    return lo


# ---- follow-up 1: tests ----
def get_from_data(i):
    return DATA[i]


def test_follow_up_1():
    assert first_at_least_unbounded(get_from_data, 2) == 1
```

```
    assert first_at_least_unbounded(get_from_data, 4) == 4
    assert first_at_least_unbounded(get_from_data, 1) == 0
    assert first_at_least_unbounded(get_from_data, 8) == -1
    assert first_at_least_unbounded(get_from_data, 7) == 5


# =====================================================
# RUN EVERYTHING
# =====================================================
def run_all_tests():
    test_main_question()
```

```
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

90 Cutting Ribbons (LC 1891) MEDIUM

Template fit: x = the piece length; ok(x) = the ribbons give at least k pieces; you want the LARGEST ok, so search the first x that fails and subtract 1

QUESTION

You have ribbons of given lengths and an integer k. Each ribbon may be cut into pieces of one common integer length L (leftover bits are thrown away). Return the largest L that gives at least k pieces in total, or 0 if even length 1 cannot give k pieces.

Function signature

def max_length(ribbons: list, k: int) -> int

EXAMPLES

Example 1	
Input	`[9, 7, 5], 3`
Output	`5`
Why	`Length 5 gives 1 + 1 + 1 = 3 pieces, length 6 only 2.`

Example 2	
Input	`[5, 7], 12`
Output	`1`
Why	`Length 1 already gives 12 pieces.`

Example 3	
Input	`[1, 2], 10`
Output	`0`
Why	`Total length is 3 but 10 pieces are needed: impossible, so 0.`

FULL CODE | main question first, then every follow-up, tests at the end

```
# =====================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# =====================================================
```

```python
# ======================================================
# MAIN QUESTION
# ======================================================
# ---- main question: helper functions ----
def pieces(ribbons, length):
    # how many pieces of this length can we cut? (the check for a GIVEN length)
    total = 0
    for r in ribbons:
        total += r // length
    return total


def upper_bound(ribbons, k):
    # no piece is longer than the longest ribbon, and k pieces of length L
    # need k * L in total, so L <= sum // k
    longest = 0
    total = 0
    for r in ribbons:
        total += r
        if r > longest:
            longest = r
    if total // k < longest:
        return total // k
    return longest


# ---- main question: main function ----
# Idea: if k pieces of length L can be cut, k pieces of any SHORTER
# length can too, so "can we cut k pieces?" goes from yes to no as L
# grows: binary search the largest L that is still yes (whole numbers).
# The answer is 0 when it is impossible.
def max_length(ribbons, k):
    low = 1
    high = upper_bound(ribbons, k)
    best = 0
    while low <= high:
        mid = (low + high) // 2
        if pieces(ribbons, mid) >= k:
```

```python
            best = mid                    # mid works: remember it and try longer
            low = mid + 1
        else:
            high = mid - 1
    return best


# the same idea when lengths may be fractions: a fixed number of halvings
def max_length_float(ribbons, k, rounds=50):
    low = 0.0
    high = float(max(ribbons))
    for i in range(rounds):
        mid = (low + high) / 2
        count = 0
        for r in ribbons:
            count += int(r // mid)
        if count >= k:
            low = mid                     # enough pieces: the answer is at least mid
        else:
            high = mid
    return low


# ---- main question: tests ----
def test_main_question():
    assert max_length([9, 7, 5], 3) == 5
```

```
    assert max_length([7, 5, 9], 4) == 4
    assert max_length([1, 2], 10) == 0                # not enough ribbon in total
    assert abs(max_length_float([1], 3) - 1 / 3) < 1e-9
    assert max_length_float([1], 2) > 0.49


# ======================================================
# FOLLOW-UP 1: Why does the running time depend on the value range and not just on the number of ribbons?
# ======================================================
# Answer: the binary search halves the range of LENGTHS, so it makes about
# log2(max length) checks, and each check costs O(number of ribbons).


# ======================================================
# FOLLOW-UP 2: What if the candidate length is given to you?
# ======================================================
# ---- follow-up 2: tests ----
# Answer: then no search is needed: the check pieces(ribbons, length)
# alone says whether k pieces can be cut.
```

```
def test_follow_up_2():
    assert pieces([9, 7, 5], 5) == 3


# ======================================================
# FOLLOW-UP 3: What is a good upper bound for the search?
# ======================================================


# ======================================================
# FOLLOW-UP 4: Give the search a tighter upper bound
# ======================================================
# ---- follow-up 4: tests ----
# Answer: the longest ribbon is a bound, and a better one is
# min(longest, sum // k): k pieces of length L need k * L in total.
# It saves a few probes and returns 0 at once when the sum is below k.
def test_follow_up_4():
    assert upper_bound([9, 7, 5], 3) == 7
    assert upper_bound([1, 2], 10) == 0


# ======================================================
# RUN EVERYTHING
# ======================================================
def run_all_tests():
    test_main_question()
    test_follow_up_2()
    test_follow_up_4()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

91 First Bad Version with Parallel Search Follow-up

MEDIUM

Template fit: x = the version number; ok(x) = is_bad(x)

QUESTION

Versions are numbered 1 to n and from some version on every version is bad. The function is_bad(v) tells whether version v is bad. Find the first bad version with as few calls to is_bad as possible.

Function signature

def first_bad_version(n: int, is_bad) -> int

EXAMPLES

Example 1	
Input	bad_from = 4 first_bad_version(10, lambda v: v >= bad_from)
Output	first_bad_version(10, lambda v: v >= bad_from) -> 4
Why	Versions 4 to 10 are bad, so 4 is the first; binary search needs about log2(10) calls.

Example 2	
Input	bad_from = 1 first_bad_version(1, lambda v: v >= bad_from)
Output	first_bad_version(1, lambda v: v >= bad_from) -> 1
Why	With a single version that is bad, the answer is 1.

FULL CODE | main question first, then every follow-up, tests at the end

```
# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

from concurrent.futures import ThreadPoolExecutor
from threading import Lock


# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: main function ----
# Idea: is_bad is monotone (all good versions, then all bad ones), so
# keep the invariant "the first bad version is in [lo, hi]" and halve the
# interval. Use lo + (hi - lo) // 2 in languages with fixed-size integers.
def first_bad_version(n, is_bad):
    lo = 1
    hi = n
    while lo < hi:
        mid = lo + (hi - lo) // 2
        if is_bad(mid):
            hi = mid                        # the first bad version is mid or earlier
        else:
            lo = mid + 1
    return lo


# ---- main question: tests ----
class Oracle:
    # a pretend is_bad that remembers how often it was called
    def __init__(self, first_bad):
        self.first_bad = first_bad
```

```
        self.calls = []
        self.lock = Lock()

    def __call__(self, version):
        with self.lock:
            self.calls.append(version)
        return version >= self.first_bad


CASES = ((1, 1), (10, 1), (10, 10), (1000, 637))
```

```


def test_main_question():
    for n, bad in CASES:
        assert first_bad_version(n, Oracle(bad)) == bad
    oracle = Oracle(637)
    first_bad_version(1000, oracle)
    assert len(oracle.calls) <= 10                    # about log2(1000) probes


# ========================================================
# FOLLOW-UP 1: What does parallelism buy you, and what does it cost?
# ========================================================
# Answer: it cuts the number of sequential ROUNDS from log2(n) to about
# log_(k+1)(n) with k workers, so wall-clock time drops when each probe is
# slow. It increases the total number of probes (about k per round instead
# of 1), so it costs more calls and money.


# ========================================================
# FOLLOW-UP 2: What if the oracle is rate limited or can fail?
# ========================================================
# Answer: cache every answer (never ask twice), retry a failed call a
# few times with a pause, and cap the number of parallel probes at the
# rate limit.


# ========================================================
# FOLLOW-UP 3: Count how many rounds each approach needs
# ========================================================


# ========================================================
# FOLLOW-UP 4: Show how many rounds and calls the parallel search uses
# ========================================================
# ---- follow-up 3: helper functions ----
def probe_points(lo, hi, workers):
    # versions spread evenly over [lo, hi - 1]
    step = (hi - lo + 1) // (workers + 1)
    if step < 1:
        step = 1
    points = set()
    for i in range(workers):
        p = lo + step * (i + 1) - 1
        if p > hi - 1:
```

```
            p = hi - 1
        points.add(p)
    return sorted(points)


# ---- follow-up 3: function ----
# Answer: one binary search needs log2(n) rounds of one probe each. With k
# workers each round shrinks the interval by about k + 1, so about
# log_(k+1)(n) rounds with k probes each. For n = 10^9: 30 rounds with one
```

```
# probe per round, but only 10 rounds with 7 probes per round. Total calls
# are at most k times the number of rounds.
def first_bad_parallel(n, is_bad, workers=4):
    lo = 1
    hi = n                                  # invariant: the answer is in [lo, hi]
    answers = {}
    rounds = 0
    with ThreadPoolExecutor(workers) as pool:
        while lo < hi:
            points = probe_points(lo, hi, workers)
            fresh = []
            for p in points:
                if p not in answers:
                    fresh.append(p)
            results = list(pool.map(is_bad, fresh))    # ask about the new points at the same time
            for i in range(len(fresh)):
                answers[fresh[i]] = results[i]
            rounds += 1
            first_bad_point = None
            for p in points:
                if answers[p]:
                    first_bad_point = p
                    break
            if first_bad_point is None:
                lo = points[-1] + 1                    # every probe was good: the answer is after them
            else:
                for p in points:
                    if p < first_bad_point:
                        lo = p + 1                     # the last good probe before the first bad one
                hi = first_bad_point
    return lo, rounds, len(answers)


# ---- follow-up 3: tests ----
def test_follow_up_3():
    for n, bad in CASES:
```

```
        answer, rounds, used = first_bad_parallel(n, Oracle(bad), workers=4)
        assert answer == bad
    answer, rounds, used = first_bad_parallel(10 ** 6, Oracle(123456), workers=7)
    assert answer == 123456 and rounds < 20           # fewer rounds than the 20 of a plain binary search
    assert used <= 7 * rounds


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

92 Solve Banana Speed and Interval Merge

MEDIUM

Template fit: x = the eating speed; ok(x) = total hours (ceil of pile / x summed) <= h; first_true over 1..max pile

QUESTION

During two technical phone screens for a new graduate software engineering role, the candidate was asked to solve two coding problems: minimum eating speed and merging overlapping intervals.

Constraints & Assumptions

- Provide clear algorithms, correctness reasoning, complexity, and edge cases.
- For minimum eating speed, each hour can be spent on only one pile.
- For interval merging, intervals overlap when the next start is less than or equal to the current merged end.
- Use sorted or binary-search approaches where appropriate.

Clarifying Questions

to Ask - Are **`piles`** and **h** always positive?

- Is **h** at least the number of piles?
- Are interval endpoints inclusive?
- Can intervals be empty or unsorted?

Part 1

- Minimum Eating Speed Given **`piles`** and **h**, return the minimum integer speed **k** such that all piles can be eaten within **h** hours.

What This Part Should Cover

- Binary search on answer from 1 to **`max(piles)`**.
- Feasibility function using sum of **`ceil(pile / k)`**.
- Why feasibility is monotonic.
- Complexity and edge cases.

Part 2

- Merge Overlapping Intervals Given intervals **`[start, end]`**, merge all overlapping intervals and return non-overlapping intervals sorted by start.

What This Part Should Cover

- Sort by start.
- Maintain current merged interval.
- Extend on overlap, otherwise append and start a new interval.
- Complexity and edge cases.

What a Strong

Answer Covers - Recognizes monotonic binary search for eating speed.

- Uses integer-safe ceiling division.
- Correctly defines interval overlap.
- Gives clean code and complexity for both problems.

Follow-up Questions

- How would you avoid floating-point division in the banana problem?
- What if **h** is smaller than the number of piles?
- How do you merge touching intervals like **`[1,3]`** and **`[3,5]`**?

QUESTION

- How would you handle streaming intervals?Minimum Eating Speed (Koko Eating Bananas)Koko loves to eat bananas. There are **n** piles of bananas; the i-th pile has **`piles[i]`** bananas. The guards will return in **h** hours. Koko can decide her constant bananas-per-hour eating speed **k**. Each hour she chooses one pile and eats **k** bananas from it. If that pile has fewer than **k** bananas, she eats all of them and does not eat more during that hour. An hour is spent on only one pile. Return the minimum integer speed **k** such that she can finish all the bananas within **h** hours. Example:
- **`piles = [3, 6, 7, 11]`**, **`h = 8`** -> **`4`**
- **`piles = [30, 11, 23, 4, 20]`**, **`h = 5`** -> **`30`**
- **`piles = [30, 11, 23, 4, 20]`**, **`h = 6`** -> **`23`** Constraints: **`1 <= piles.length <= h`**, **`1 <= piles[i] <= 10^9`**.

Constraints

1 <= piles.length <= h1 <= piles[i] <= 10^9h can be as large as 10^9

EXAMPLES

```
Input: ([3, 6, 7, 11], 8)
Expected Output: 4
Explanation: At k=4: ceil(3/4)+ceil(6/4)+ceil(7/4)+ceil(11/4) = 1+2+2+3 = 8 hours. k=3 needs 1+2+3+4 =
    10 > 8, so 4 is minimal.
Input: ([30, 11, 23, 4, 20], 5)
Expected Output: 30
```

Explanation: With exactly 5 piles and 5 hours, each pile must be eaten in one hour, so k must be at least max(piles) = 30.

Hints

If speed k finishes in time, any larger speed also finishes in time. This monotonicity lets you binary search on the answer.The answer lies in [1, max(piles)]: speed 1 is the slowest, and max(piles) finishes every pile in at most one hour.Hours needed at speed k is sum of ceil(pile / k). Use integer-safe ceiling: (pile + k - 1) // k to avoid floating point.Merge Overlapping IntervalsGiven an array of intervals where **`intervals[i] = [start_i, end_i]`**, merge all overlapping intervals and return an array of the non-overlapping intervals that cover all the intervals in the input, sorted by start. Two intervals overlap when the next start is less than or equal to the current merged end

FULL CODE | main question first, then every follow-up, tests at the end

```python
# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
def hours_needed(piles, speed):
    # each hour she works on ONE pile, so every pile takes a rounded-up number of hours
    total = 0
    for p in piles:
        total += (p + speed - 1) // speed
    return total


# ---- main question: main functions ----
# PART 1 (bananas): the smallest eating speed that finishes all piles
# within h hours. Hours needed never go UP when the speed goes up, so
# binary search the speed between 1 and the biggest pile. -1 if h is
# smaller than the number of piles (each pile takes at least one hour).
def min_eating_speed(piles, h):
    if h < len(piles):
```

```
        return -1
    low = 1
    high = max(piles)
    while low < high:
        mid = (low + high) // 2
        if hours_needed(piles, mid) <= h:
            high = mid                    # fast enough: try slower
        else:
            low = mid + 1
    return low


# PART 2 (intervals): merge overlapping intervals. Sort by start; an
# interval that starts before (or when) the last merged one ends is
# absorbed by it. Sharing one point counts as overlapping.
def merge_intervals(intervals):
    merged = []
    for start, end in sorted(intervals):
        if merged and start <= merged[-1][1]:
            if end > merged[-1][1]:
                merged[-1][1] = end
        else:
            merged.append([start, end])
    return merged


# ---- main question: tests ----
def test_main_question():
    assert min_eating_speed([3, 6, 7, 11], 8) == 4
    assert min_eating_speed([30, 11, 23, 4, 20], 5) == 30
    assert min_eating_speed([30, 11, 23, 4, 20], 6) == 23
    assert min_eating_speed([1, 1, 1], 2) == -1       # fewer hours than piles
    assert hours_needed([3, 6, 7, 11], 4) == 8
    assert merge_intervals([[1, 3], [2, 6], [8, 10], [15, 18]]) == [[1, 6], [8, 10], [15, 18]]
    assert merge_intervals([[1, 4], [4, 5]]) == [[1, 5]]
    assert merge_intervals([]) == []


# ============================================================
# FOLLOW-UP 1: Bananas: h varies per query, or the piles change
# ============================================================
# Answer: for many queries on the same piles, sort the piles and keep
# prefix sums so hours_needed costs O(log n) instead of O(n); when piles
# change, keep the structure updated (a balanced tree or Fenwick tree).


# ============================================================
# FOLLOW-UP 3: Bananas: the guards come back in fewer hours than piles
# ============================================================
# Answer: then no speed works (every pile needs at least one hour), which
# is why min_eating_speed returns -1 when h < number of piles.


# ============================================================
# FOLLOW-UP 2: Intervals: insert one new interval into an already merged list (LeetCode 57)
# ============================================================
# ---- follow-up 2: function ----
# Answer: copy the intervals that end before the new one starts, absorb
# the ones that overlap it, then copy the rest. O(n), no sort needed.
def insert_interval(merged, new):
    result = []
    i = 0
    n = len(merged)
    while i < n and merged[i][1] < new[0]:            # 1) intervals completely before the new one
```

```
        result.append(merged[i])
        i += 1
    start, end = new[0], new[1]
    while i < n and merged[i][0] <= end:              # 2) intervals that overlap it are absorbed
        if merged[i][0] < start:
            start = merged[i][0]
        if merged[i][1] > end:
            end = merged[i][1]
        i += 1
    result.append([start, end])
    while i < n:                                      # 3) the rest comes after
        result.append(merged[i])
        i += 1
    return result


# ---- follow-up 2: tests ----
def test_follow_up_2():
    assert insert_interval([[1, 3], [6, 9]], [2, 5]) == [[1, 5], [6, 9]]
    assert insert_interval([], [4, 8]) == [[4, 8]]
    assert insert_interval([[1, 2], [5, 6]], [3, 4]) == [[1, 2], [3, 4], [5, 6]]


# ====================================================
# RUN EVERYTHING
# ====================================================
def run_all_tests():
    test_main_question()
    test_follow_up_2()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

93 Given a list with response time for each server, calculate how long time it will take to process n requests. MEDIUM

Template fit: x = the elapsed time; ok(x) = sum(x // time_i) >= n requests

QUESTION

Servers work in parallel and server i needs times[i] time units to process one request (optionally server i only starts at starts[i]). Return the earliest time at which n requests have been processed in total.

Function signature

def time_for(times: list, n: int, starts=None) -> int

EXAMPLES

Example 1	
Input	`[1, 2], 3`
Output	`2`
Why	`By time 2 the first server has done 2 requests and the second one: 3 in total.`

Example 2	
Input	`[3], 4`
Output	`12`
Why	`One server needs 3 units per request: 4 requests take 12.`

QUESTION

Example 3	
Input	[2, 1], 6, starts = [0, 5]
Output	8
Why	The second server only starts at time 5, so the first one does most of the work.

FULL CODE | main question first, then every follow-up, tests at the end

```
# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ========================================================

import heapq


# ========================================================
# MAIN QUESTION
# ========================================================
# ---- main question: helper functions ----
def start_of(starts, i):
    # the startup delay of server i (0 when no delays are given)
    if starts:
        return starts[i]
    return 0


def done_by(times, t, starts=None):
    # how many requests do all servers finish within time t? Server i needs
    # times[i] per request and only begins after its startup delay (follow-up 1).
    total = 0
    for i in range(len(times)):
        working_time = t - start_of(starts, i)
        if working_time > 0:
            total += working_time // times[i]
    return total


# ---- main question: main function ----
# Idea: binary search on the TOTAL TIME. In time t, server i finishes
# t // times[i] requests, and done_by(t) never decreases as t grows, so the
# answer is the smallest t with done_by(t) >= n.
# starts[i] is an optional startup delay (follow-up 1).
def time_for(times, n, starts=None):
    if n <= 0:
        return 0
    low = 0
    high = start_of(starts, 0) + times[0] * n      # the first server alone is always enough
    for i in range(len(times)):
        if start_of(starts, i) + times[i] * n < high:
            high = start_of(starts, i) + times[i] * n
```

```
    while low < high:
        mid = (low + high) // 2
        if done_by(times, mid, starts) >= n:
            high = mid
        else:
            low = mid + 1
    return low
```

```python
# ---- main question: tests ----
def test_main_question():
    assert time_for([1, 2, 3], 6) == 4
    assert time_for([4], 3) == 12
    assert time_for([5, 5], 0) == 0
    assert time_for([2, 3], 1) == 2


# ========================================================
# FOLLOW-UP 1: Servers may fail, or there is a startup time per server
# ========================================================
# ---- follow-up 1: tests ----
# Answer: add the startup time to the check: server i finishes
# (t - start_i) // r_i requests (nothing before it starts). The check
# is still monotone in t, so the binary search still works. A failed
# server is the same as one that starts "never": drop it.
def test_follow_up_1():
    assert time_for([1, 5], 2, starts=[10, 0]) == 10   # server 0 starts at 10, so server 1 does the work


# ========================================================
# FOLLOW-UP 2: The inverse question: how many servers are needed to finish within a given time?
# ========================================================
# ---- follow-up 2: helper functions ----
def ceil_div(a, b):
    # a / b rounded up
    return (a + b - 1) // b


# ---- follow-up 2: function ----
# Answer: no search is needed: each server (response time r) finishes
# t // r requests, so divide and round up.
def servers_needed(n, r, t):
    per_server = t // r
    if per_server == 0:
        return None                        # not even one request fits in time t
    return ceil_div(n, per_server)


# ---- follow-up 2: tests ----
def test_follow_up_2():
    assert servers_needed(10, 4, 8) == 5
    assert servers_needed(10, 4, 3) is None
    assert servers_needed(1, 7, 7) == 1


# ========================================================
# FOLLOW-UP 3: Requests have different sizes
# ========================================================
# ---- follow-up 3: function ----
# Answer: now it is a scheduling problem (minimise the finish time of the
# busiest server), which is NP-hard in general. The greedy rule "give the
# biggest request to the least loaded server" (a heap of loads) is
# within 4/3 of the best possible.
def makespan(sizes, servers):
    loads = [0] * servers
    heapq.heapify(loads)
    for size in sorted(sizes, reverse=True):
        lightest = heapq.heappop(loads)
        heapq.heappush(loads, lightest + size)
    busiest = 0
    for load in loads:
        if load > busiest:
            busiest = load
    return busiest
```

```
# ---- follow-up 3: tests ----
def test_follow_up_3():
    assert makespan([5, 5, 4, 4, 3, 3, 3], 3) == 11   # the best split is 9, 9, 9; greedy gets 11 (11/9 < 4/3
)
    assert makespan([], 2) == 0


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    test_follow_up_3()
```

```
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

94 Median of Two Sorted Arrays (LC 4) with a Follow-up Variant

HARD

Template fit: x = how many items come from the shorter array's left side; ok(x) = the left part's largest <= the right part's smallest

QUESTION

Given two sorted arrays of numbers, return the median of all their elements together, without merging them into one array. The median is the middle element when the total count is odd, and the average of the two middle elements when it is even. Aim for O(log(min(m, n))) time.

Function signature

def median_two(a: list, b: list) -> float

EXAMPLES

Example 1	
Input	[1, 3], [2]
Output	2
Why	Combined [1, 2, 3]: the middle element is 2.

Example 2	
Input	[1, 2], [3, 4]
Output	2.5
Why	Combined [1, 2, 3, 4]: the two middle elements are 2 and 3, so the median is 2.5.

Example 3	
Input	[], [7, 9]
Output	8
Why	One array may be empty: the median is just the other array's median.

FULL CODE | main question first, then every follow-up, tests at the end

```
# ============================================================
# HOW THIS FILE IS ORGANISED
```

```
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

import bisect
import heapq
import random

INF = float("inf")


# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
def left_border(arr, i):
    # the last of the first i items of arr (minus infinity when i is 0)
    if i > 0:
        return arr[i - 1]
    return -INF


def right_border(arr, i):
    # the first item after the first i items (plus infinity when there is none)
    if i < len(arr):
        return arr[i]
    return INF


def bigger(a, b):
    if a > b:
        return a
    return b


def smaller(a, b):
    if a < b:
        return a
    return b


# ---- main question: main function ----
```

```
# Idea: make a the shorter array. Binary search i, the number of items of
# a in the left half, and take half - i items from b, so that the left
# half holds (m + n + 1) // 2 items. The cut is right when every left
# item is <= every right item; then the median is read off the four items
# next to the cuts.
def median_two(a, b):
    if len(a) > len(b):
        a, b = b, a
    m = len(a)
    n = len(b)
    if m + n == 0:
        raise ValueError("no elements")
    low = 0
    high = m
    half = (m + n + 1) // 2
    while low <= high:
        i = (low + high) // 2
        j = half - i
        a_left, a_right = left_border(a, i), right_border(a, i)
        b_left, b_right = left_border(b, j), right_border(b, j)
        if a_left <= b_right and b_left <= a_right:        # a valid cut
            if (m + n) % 2 == 1:
```

```
                return bigger(a_left, b_left)
            return (bigger(a_left, b_left) + smaller(a_right, b_right)) / 2
        if a_left > b_right:
            high = i - 1
        else:
            low = i + 1


# ---- main question: tests ----
def test_main_question():
    assert median_two([1, 3], [2]) == 2
    assert median_two([1, 2], [3, 4]) == 2.5
    assert median_two([], [5]) == 5


# ======================================================
# FOLLOW-UP 1: Now there are k sorted arrays instead of two. How do you find the median?
# ======================================================
# ---- follow-up 1: function ----
# Answer: binary search on the ANSWER VALUE. For a candidate x count the
# items that are <= x with one bisect per array (O(k log n)); the k-th
# smallest is the smallest x whose count reaches k.
def median_k_value(arrays):
```

```
    arrays = [a for a in arrays if a]
    total = 0
    for a in arrays:
        total += len(a)
    if total == 0:
        raise ValueError("no elements")

    def kth(rank):
        low = arrays[0][0]
        high = arrays[0][-1]
        for a in arrays:
            low = smaller(low, a[0])
            high = bigger(high, a[-1])
        while low < high:
            mid = (low + high) // 2
            count = 0
            for a in arrays:
                count += bisect.bisect_right(a, mid)
            if count >= rank:
                high = mid
            else:
                low = mid + 1
        return low

    if total % 2 == 1:
        return kth(total // 2 + 1)
    return (kth(total // 2) + kth(total // 2 + 1)) / 2


# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert median_k_value([[1, 3], [2], [4, 5, 6]]) == 3.5


# ======================================================
# FOLLOW-UP 2: What is the heap version and when would you choose it?
# ======================================================
# ---- follow-up 2: function ----
# Answer: a min-heap holds the front item of every array. Pop the
# smallest again and again (like merging) until the middle one or two
# items have been seen. It is simple and fine when k is small or the
# arrays are streams you cannot index.
```

```python
def median_k_heap(arrays):
    heap = []
    total = 0
    for i in range(len(arrays)):
        total += len(arrays[i])
        if arrays[i]:
            heap.append((arrays[i][0], i, 0))
    if total == 0:
        raise ValueError("no elements")
    heapq.heapify(heap)
    low_rank = (total - 1) // 2
    high_rank = total // 2
    seen = 0
    middle = []
    while heap:
        value, i, j = heapq.heappop(heap)
        if seen >= low_rank:
            middle.append(value)
        if seen >= high_rank:
            break
        seen += 1
        if j + 1 < len(arrays[i]):
            heapq.heappush(heap, (arrays[i][j + 1], i, j + 1))
    if total % 2 == 1:
        return middle[0]
    return (middle[0] + middle[1]) / 2


# ---- follow-up 2: tests ----
def test_follow_up_2():
    assert median_k_heap([[1, 3], [2], [4, 5, 6]]) == 3.5
    random.seed(59)
    for round_number in range(300):                    # both versions must agree
        arrays = []
        for k in range(random.randint(1, 5)):
            numbers = []
            for i in range(random.randint(0, 6)):
                numbers.append(random.randint(-9, 9))
            numbers.sort()
            arrays.append(numbers)
        if sum([len(a) for a in arrays]) == 0:
            continue
        assert median_k_heap(arrays) == median_k_value(arrays)


# ============================================================
# RUN EVERYTHING
# ============================================================
```

```python
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

95 Find the median of two sorted arrays. HARD

Template fit: the same partition search on the shorter array

QUESTION

Given two sorted arrays, return the median of all their elements together. The median is the middle value when the total count is odd and the average of the two middle values when it is even. The best solutions run in O(log(min(m, n))).

Function signature

def findMedianSortedArrays(nums1: list, nums2: list) -> float

EXAMPLES

Example 1	
Input	`[1, 3], [2]`
Output	`2`
Why	`Combined: 1, 2, 3.`

Example 2	
Input	`[1, 2], [3, 4]`
Output	`2.5`
Why	`Combined: 1, 2, 3, 4: the middle two are 2 and 3.`

Example 3	
Input	`[], [5]`
Output	`5`
Why	`One empty array.`

FULL CODE | main question first, then every follow-up, tests at the end

```
# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

from bisect import bisect_right

INF = float("inf")


# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
def left_border(arr, i):
    # the last of the first i items of arr (minus infinity when i is 0)
    if i > 0:
        return arr[i - 1]
    return -INF


def right_border(arr, i):
    # the first item after the first i items (plus infinity when there is none)
    if i < len(arr):
        return arr[i]
    return INF
```

```
def bigger(a, b):
    if a > b:
        return a
    return b


def kth_of_two(a, b, k):
    # the k-th smallest (k starts at 1) of two sorted arrays. Take i items
    # from a and k - i items from b as the "left part". The cut is right when
    # every left item is <= every right item, and only the four items next to
    # the two cuts have to be compared. Binary search on i: O(log n).
    # (This also answers follow-up 3: any k-th smallest, not only the median.)
    if len(a) > len(b):
        a, b = b, a
    low = k - len(b)
    if low < 0:
        low = 0
    high = k
    if len(a) < high:
        high = len(a)
    while low <= high:
        i = (low + high) // 2
        j = k - i
        if left_border(a, i) > right_border(b, j):
            high = i - 1                    # took too many from a
        elif left_border(b, j) > right_border(a, i):
            low = i + 1                     # took too few from a
        else:
            return bigger(left_border(a, i), left_border(b, j))
    raise ValueError("k is out of range")


# ---- main question: main function ----
# Idea: binary search how many items of the shorter array go into the
# left half. The median is the middle item (odd total) or the mean of
# the two middle items (even total).
def findMedianSortedArrays(nums1, nums2):
    total = len(nums1) + len(nums2)
    if total == 0:
        raise ValueError("no data")
    if total % 2 == 1:
        return kth_of_two(nums1, nums2, total // 2 + 1)
    low = kth_of_two(nums1, nums2, total // 2)
    high = kth_of_two(nums1, nums2, total // 2 + 1)
    return (low + high) / 2


# ---- main question: tests ----
def test_main_question():
    assert findMedianSortedArrays([1, 3], [2]) == 2
    assert findMedianSortedArrays([1, 2], [3, 4]) == 2.5
    assert findMedianSortedArrays([], [5]) == 5


# ======================================================
# FOLLOW-UP 1: What if there are k sorted arrays instead of two?
# ======================================================
# ---- follow-up 1: helper functions ----
def count_at_most(arrays, v):
    # how many items are <= v in all the arrays together
    total = 0
    for arr in arrays:
        total += bisect_right(arr, v)
    return total
```

```python
# ---- follow-up 1: function ----
# Answer: binary search on the ANSWER VALUE instead of on a position. For
# a candidate value v, count with bisect how many items are <= v in each
# array; the k-th smallest is the smallest v with at least k items <= v.
# (Whole-number values.)
def kth_of_many(arrays, k):
    arrays = [arr for arr in arrays if arr]
    low = arrays[0][0]
    high = arrays[0][-1]
    for arr in arrays:
        if arr[0] < low:
            low = arr[0]
        if arr[-1] > high:
            high = arr[-1]
    while low < high:
        mid = (low + high) // 2
        if count_at_most(arrays, mid) >= k:
            high = mid
        else:
            low = mid + 1
    return low


def median_of_many(arrays):
    total = 0
    for arr in arrays:
        total += len(arr)
    if total % 2 == 1:
        return kth_of_many(arrays, total // 2 + 1)
    return (kth_of_many(arrays, total // 2) + kth_of_many(arrays, total // 2 + 1)) / 2


# ---- follow-up 1: tests ----
def test_follow_up_1():
    arrays = [[1, 3], [2], [4, 5, 6]]
    assert median_of_many(arrays) == 3.5
    assert kth_of_many(arrays, 4) == 4
```

```python
# ============================================================
# FOLLOW-UP 2: What if the arrays are too large for memory?
# ============================================================
# Answer: use the binary search on the VALUE above: it only needs "how
# many items are <= v", which can be computed with a binary search on
# each sorted file (random access) or one streaming pass per probe. The
# data never has to be loaded.


# ============================================================
# FOLLOW-UP 3: Find the k-th smallest element, not just the median
# ============================================================
# ---- follow-up 3: tests ----
# Answer: kth_of_two (above) is exactly that; the median is k = (m + n) // 2 + 1.
def test_follow_up_3():
    assert kth_of_two([1, 3, 5], [2, 4, 6], 4) == 4
    assert kth_of_two([1, 3, 5], [], 2) == 3


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
```

```
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

96 Find longest duplicated substring — HARD

Template fit: x = the substring length; ok(x) = some substring of that length appears twice (rolling hash); search the largest ok

QUESTION

You are given a string **s** consisting of lowercase English letters. A **substring** of **s** is any contiguous sequence of characters, i.e., **`s[i..j)`** for **`0 <= i < j <= len(s)`**. Design an algorithm to find **any one** of the longest substrings that appears **at least twice** in **s**. The two occurrences may overlap. If no substring appears at least twice, return the empty string.

Requirements

- Input: a string **s** of length **n** (e.g., **`1 <= n <= 2 × 10^5`**).
- Output: a string representing one of the longest duplicated substrings.
- Aim for an algorithm substantially better than the naive **`O(n^2)`** substring comparison approach.

Examples

- **`s = "banana"`**
- Possible duplicated substrings: "a", "an", "ana" - Longest length is 3 ("ana"), so acceptable outputs include **`"ana"`**.
- **`s = "abcd"`**
- No duplicated substring, so output should be **`""`** (empty string). Describe your algorithm, its time and space complexity, and how it scales for large **n**.

EXAMPLES

Example 1	
Input	`'banana'`
Output	`'ana'`
Why	`'ana' occurs twice (overlapping is allowed).`

Example 2	
Input	`'abcd'`
Output	`''`
Why	`No substring repeats.`

Example 3	
Input	`'aaaaa'`
Output	`'aaaa'`
Why	`'aaaa' appears at positions 0 and 1.`

FULL CODE | main question first, then every follow-up, tests at the end

```
# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
```

```
# ============================================================

# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
MOD = (1 << 61) - 1          # a big prime: hash values stay small
BASE = 131


def find_length(s, length):
    # is there a substring of this length that appears twice? Return one, or None.
    # Rolling hash: the hash of every window is updated in O(1) when the
    # window slides by one character.
    codes = []
    for c in s:
        codes.append(ord(c))
    power = pow(BASE, length, MOD)                    # BASE ** length, used to remove the left character
    h = 0
    for i in range(length):
        h = (h * BASE + codes[i]) % MOD
    seen = {h: [0]}                                   # hash -> start positions of windows with it
    for i in range(1, len(s) - length + 1):
        h = (h * BASE - codes[i - 1] * power + codes[i + length - 1]) % MOD
        if h in seen:
            for j in seen[h]:
                if s[j:j + length] == s[i:i + length]:    # equal hashes may be a collision: compare the text
                    return s[i:i + length]
        else:
            seen[h] = []
        seen[h].append(i)
    return None


# ---- main question: main function ----
# Idea: if a substring of length L appears twice, so does every shorter
# one, so binary search the length. For each length find out whether a
# repeat exists with the rolling hash (Rabin-Karp) above.
def longest_dup(s):
    low = 1
```

```
    high = len(s) - 1
    best = ""
    while low <= high:
        mid = (low + high) // 2
        found = find_length(s, mid)
        if found:
            best = found
            low = mid + 1                 # a repeat exists: try longer
        else:
            high = mid - 1
    return best


# ---- main question: tests ----
def test_main_question():
    assert longest_dup("banana") == "ana"
    assert longest_dup("abcd") == ""
    assert len(longest_dup("aaaaa")) == 4


# ============================================================
# FOLLOW-UP 1: Avoid hash collisions entirely
# ============================================================
# ---- follow-up 1: function ----
# Answer: use a suffix array with the LCP (longest common prefix) of
```

```
# neighbours: sort all suffixes; the longest duplicate is the largest
# common prefix between two neighbours in that order. Sorting the
# suffixes directly is O(n^2 log n); real suffix array algorithms are
# O(n log n) or O(n).
def longest_dup_suffix(s):
    suffixes = []
    for i in range(len(s)):
        suffixes.append(s[i:])
    suffixes.sort()
    best = ""
    for index in range(len(suffixes) - 1):
        a = suffixes[index]
        b = suffixes[index + 1]
        k = 0
        while k < len(a) and k < len(b) and a[k] == b[k]:
            k += 1
        if k > len(best):
            best = a[:k]
    return best
```

```
# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert len(longest_dup_suffix("banana")) == 3
    assert longest_dup_suffix("abcd") == ""
    for s in ("abracadabra", "mississippi", "abab"):
        assert len(longest_dup(s)) == len(longest_dup_suffix(s))


# ============================================================
# FOLLOW-UP 2: The substring must appear at least k times, or must not overlap
# ============================================================
# ---- follow-up 2: function ----
# Answer: for "at least k times", count the windows with each text (need
# k of them). For "not overlapping", count only start positions that are
# at least `length` apart (greedy from the left). This simple version
# lists every window of each length instead of hashing.
def longest_repeated(s, times=2, non_overlapping=False):
    best = ""
    n = len(s)
    for length in range(1, n):
        spots = {}                                  # window text -> start positions
        for i in range(n - length + 1):
            piece = s[i:i + length]
            if piece not in spots:
                spots[piece] = []
            spots[piece].append(i)
        found = None
        for text in spots:
            starts = spots[text]
            if non_overlapping:
                taken = 0
                last = -length
                for p in starts:
                    if p - last >= length:
                        taken += 1
                        last = p
            else:
                taken = len(starts)
            if taken >= times:
                found = text
                break
        if found is None:
            break                                   # no repeat of this length, so none of any longer lengt
h
        best = found
```

```
    return best


# ---- follow-up 2: tests ----
def test_follow_up_2():
    assert longest_repeated("banana", 3) == "a"             # three occurrences needed
    assert len(longest_repeated("aaaa", 2, True)) == 2      # "aa" twice without overlapping
    assert len(longest_repeated("aaaa", 2)) == 3            # overlapping copies are allowed


# ========================================================
# FOLLOW-UP 3: The string is huge and does not fit in memory
# ========================================================
# Answer: the binary search on the length still works; for each length
# stream the string once and keep only the hashes (a Bloom filter or a
# hash set on disk), then verify the candidate positions in a second pass.


# ========================================================
# RUN EVERYTHING
# ========================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

GROUP 12 OF 43 | 8 QUESTIONS, ONE CODE TEMPLATE | 3 E 5 M

Greedy: scan once, carry the best choice so far, act only when forced

97 Maximum Profit — EASY

Template fit: pick the cheapest and the dearest city with a fixed tie-break (min and max once)

QUESTION

You and your friend have decided to start an international trading business. You want to earn a profit through some ol' fashioned arbitrage: You'll buy widgets in one country and sell them in another at a higher price. Your friend has already found the current market prices for widgets in cities around the world, and you'd like to use this data to pick which cities to buy and sell widgets in. Write a function max_profit(prices) that efficiently finds the two cities that maximize your profit (i.e. largest difference in prices).
Input: A dictionary with cities as keys and prices as values Output: An array containing the names of the cities (min, max) If the prices dictionary is empty, return None/null (depending on the programming language used). If there is 0 profit to be earned, return None/null as well. If you are using C++, return {"", ""} in such cases. Example Pythonprices = {'London': 72, 'New York': 70, 'Tokyo': 67, 'Miami': 62} max_profit(prices)

=> ['Miami', 'London']Hint 1You can think of this as two sub-problems

: Finding the lowest price, and finding the highest price.Hint 2While you could compare every pair of cities, it's more efficient to find the min and max values.Hint 3You can keep track of the lowest and highest values and update them as you iterate through the list.Challenge 1Does your solution work in O(N) time and O(1) space? If not, you can be more efficient!Challenge 2What happens if the price is the same in every city? Does your function still return

EXAMPLES

Example 1	
Input	`{'A': 3, 'B': 10, 'C': 1}`
Output	`9`
Why	`Buy in the cheapest city (C, price 1) and sell in the dearest (B, price 10): profit 9.`

Example 2	
Input	`{'A': 5}`
Output	`0`
Why	`With a single city there is nothing to trade.`

FULL CODE | main question first, then every follow-up, tests at the end

```
# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ========================================================

# ========================================================
# MAIN QUESTION
# ========================================================
# ---- main question: helper functions ----
def bigger(a, b):
    if a > b:
        return a
    return b


def smaller(a, b):
```

```
    if a < b:
        return a
    return b


# ---- main question: main function ----
# Idea: prices is a dictionary city -> price. Buy in the cheapest city
# and sell in the dearest one: the order of the cities does not matter,
# so the best profit is highest price - lowest price (0 with one city).
def max_profit_prices(prices):
    if len(prices) < 2:
        return 0
    lowest = None
    highest = None
    for city in prices:
        price = prices[city]
        if lowest is None or price < lowest:
            lowest = price
        if highest is None or price > highest:
            highest = price
    return highest - lowest


# ---- main question: tests ----
def test_main_question():
    assert max_profit_prices({"a": 10, "b": 4, "c": 9}) == 6
    assert max_profit_prices({"a": 3}) == 0
```

```
    assert max_profit_prices({}) == 0


# ======================================================
# FOLLOW-UP 1: The prices are a time series: you must buy before you sell (LeetCode 121)
# ======================================================
# ---- follow-up 1: function ----
# Answer: keep the lowest price seen so far and the best difference
# (today's price minus that lowest): O(n) time, O(1) space. The
# dictionary version could ignore the order; a time series cannot.
def best_one_trade(series):
    lowest = float("inf")
    best = 0
    for price in series:
        lowest = smaller(lowest, price)
        best = bigger(best, price - lowest)
    return best


# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert best_one_trade([7, 1, 5, 3, 6, 4]) == 5
    assert best_one_trade([7, 6, 4, 3, 1]) == 0


# ======================================================
# FOLLOW-UP 2: Allow many trades (LeetCode 122) or a limit of k trades
# ======================================================
# ---- follow-up 2: functions ----
# Answer: many trades: add up every rise from one day to the next. At
# most k trades: a DP over (trades used, holding a share or not): O(n * k).
def best_many_trades(series):
    total = 0
    for i in range(1, len(series)):
        if series[i] > series[i - 1]:
            total += series[i] - series[i - 1]
    return total
```

```
def best_k_trades(series, k):
    if len(series) == 0 or k == 0:
        return 0
    hold = [float("-inf")] * (k + 1)       # best money while holding a share, with t trades started
    cash = [0] * (k + 1)                   # best money with no share, with t trades finished
    for price in series:
        for t in range(1, k + 1):
            hold[t] = bigger(hold[t], cash[t - 1] - price)      # buy, using the cash of t - 1 finished trade
s
            cash[t] = bigger(cash[t], hold[t] + price)          # or sell
    return cash[k]


# ---- follow-up 2: tests ----
def test_follow_up_2():
    assert best_many_trades([7, 1, 5, 3, 6, 4]) == 7
    assert best_k_trades([3, 2, 6, 5, 0, 3], 2) == 7
    assert best_k_trades([1, 2, 3], 1) == 2
    assert best_k_trades([], 3) == 0


# =====================================================
# FOLLOW-UP 3: Transaction fee or cooldown
# =====================================================
# ---- follow-up 3: function ----
# Answer: a two-state DP per day: hold = best money while holding a
# share, cash = best money without one.
#   hold = max(hold, cash - price)        cash = max(cash, hold + price - fee)
def best_with_fee(series, fee):
    hold = float("-inf")
    cash = 0
    for price in series:
        hold = bigger(hold, cash - price)
        cash = bigger(cash, hold + price - fee)
    return cash


# ---- follow-up 3: tests ----
def test_follow_up_3():
    assert best_with_fee([1, 3, 2, 8, 4, 9], 2) == 8


# =====================================================
# RUN EVERYTHING
# =====================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    test_follow_up_3()
    print("all tests passed")
```

```
if __name__ == "__main__":
    run_all_tests()
```

98 Alarm for the Next Upcoming Birthday

EASY

Template fit: minimum over people of the circular gap (their day - today) modulo 365, ties by name

QUESTION

You maintain a list of people and their birthdays. Given today's date and a list of (name, month, day) records, return the name of the person whose birthday comes soonest starting from today. Birthdays recur every year, so if no birthday remains later this calendar year, the soonest one is the earliest birthday next year. A birthday that falls exactly on today counts as 0 days away (soonest). Ignore leap years; treat every date simply as a (month, day) pair. If two people share the soonest birthday, return the one that is alphabetically first by name.

Function signature

def next_birthday(today: tuple[int, int], people: list[tuple[str, int, int]]) -> str:

...

EXAMPLES

Example 1	
Input	`today = (10, 7), people = [("Alice", 10, 9), ("Bob", 1, 15), ("Carol", 12, 25)]`
Output	`"Alice"`
Why	`Alice's birthday is 2 days away; Carol's is later this year; Bob's is next January. Alice is soonest.`

Example 2	
Input	`today = (12, 31), people = [("Alice", 1, 1), ("Dan", 12, 31)]`
Output	`"Dan"`
Why	`Dan's birthday is today (0 days away), which beats Alice's tomorrow.`

Example 3	
Input	`today = (6, 1), people = [("Eve", 5, 30), ("Finn", 5, 30)]`
Output	`"Eve"`
Why	`Both birthdays already passed this year, so both are ~next-year and tie; alphabetical order picks "Eve".`

CONSTRAINTS

- 1 <= len(people) <= 10**4
- 1 <= month <= 12 , 1 <= day <= 31 (assume valid calendar dates). today is a valid (month, day) pair.

FULL CODE | main question first, then every follow-up, tests at the end

```python
# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ========================================================

from bisect import bisect_left
import random


# ========================================================
# MAIN QUESTION
# ========================================================
# ---- main question: helper functions ----
MONTH_DAYS = [31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31]      # a non-leap year
```

```
def day_of_year(month, day):
    # 1 January is day 1
    total = day
    for m in range(month - 1):
        total += MONTH_DAYS[m]
    return total


# ---- main question: main function ----
# Idea: the number of days from today until a birthday is
# (its day of the year - today's day of the year) mod 365, so a birthday
# that already passed this year wraps around to next year. Take the
# smallest number of days; on a tie the name that comes first.
# today is (month, day); people are (name, month, day).
def next_birthday(today, people):
    now = day_of_year(today[0], today[1])
    best_name = None
    best_days = None
    for name, month, day in people:
        days_away = (day_of_year(month, day) - now) % 365
        if best_days is None or days_away < best_days or (days_away == best_days and name < best_name):
            best_days = days_away
            best_name = name
    return best_name


# ---- main question: tests ----
```

```
def test_main_question():
    people = [("Alice", 10, 9), ("Bob", 1, 15), ("Carol", 12, 25)]
    assert next_birthday((10, 7), people) == "Alice"
    assert next_birthday((12, 31), [("Alice", 1, 1), ("Dan", 12, 31)]) == "Dan"     # today counts (0 days aw
ay)
    assert next_birthday((6, 1), [("Eve", 5, 30), ("Finn", 5, 30)]) == "Eve"        # a tie: the first name


# =======================================================
# FOLLOW-UP 1: Many queries with different dates. How do you make each one fast?
# =======================================================
# ---- follow-up 1: helper functions ----
# Answer: sort the birthdays by (day of the year, name) ONCE. For a query
# binary-search the first birthday at or after today's day of the year;
# if there is none, wrap around to the first birthday in the list.
# O(log p) per query after an O(p log p) setup.
class BirthdayBook:
    def __init__(self, people):
        self.items = []
        for name, month, day in people:
            self.items.append((day_of_year(month, day), name))
        self.items.sort()
        self.days = []
        for d, name in self.items:
            self.days.append(d)

    def next_birthday(self, today):
        at = bisect_left(self.days, day_of_year(today[0], today[1]))
        return self.items[at % len(self.items)][1]


# ---- follow-up 1: tests ----
def test_follow_up_1():
    random.seed(50)
    names = ["Ann", "Bo", "Cy", "Di", "Ed"]
    for round_number in range(200):
        people = []
        for name in names:
```

```
            people.append((name, random.randint(1, 12), random.randint(1, 28)))
        book = BirthdayBook(people)
        today = (random.randint(1, 12), random.randint(1, 28))
        assert book.next_birthday(today) == next_birthday(today, people)


# =====================================================
# RUN EVERYTHING
```

```
# =====================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

99 Coins on the Table

EASY

Template fit: scan the events with one counter: add on '+', subtract on '-' but never below 0

QUESTION

There is a table that starts with 0 coins on it. You are given a string events , where each character is a single event applied left to right: '+' places one coin on the table. '-' removes one coin from the table. The table can never hold a negative number of coins: a '-' event when the table is empty has no effect (it is ignored). After processing every event, return the number of coins on the table.

Function signature

def coins_on_table(events: str) -> int:

    ...

Follow-up (as reported): Generalize so each event is an integer delta (e.g. "+3" , "-2"), still clamping the running total at 0. Hint: parse tokens instead of single characters.

EXAMPLES

Example 1	
Input	events = "+++-"
Output	2
Why	+1, +1, +1, then -1 -> 2 coins remain.

Example 2	
Input	events = "----"
Output	0
Why	Every removal happens on an empty table and is ignored.

Example 3	
Input	events = "+-+-+"
Output	1
Why	1, 0, 1, 0, 1 -> 1 coin remains.

CONSTRAINTS

- 0 <= len(events) <= 10^5
- Every character of events is either '+' or '-' .

QUESTION

- An empty string returns 0 .

FULL CODE | main question first, then every follow-up, tests at the end

```
# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ========================================================

import random


# ========================================================
# MAIN QUESTION
# ========================================================
# ---- main question: main function ----
# Idea: one counter that can never go below zero. "+" puts a coin on the
# table; "-" takes one, but only if there is one (otherwise it stays 0):
# total = max(0, total + delta) after every event.
def coins_on_table(events):
    total = 0
    for event in events:
        if event == "+":
            total += 1
        elif total > 0:
            total -= 1
    return total


# ---- main question: tests ----
def test_main_question():
    assert coins_on_table("+++-") == 2
    assert coins_on_table("----") == 0
    assert coins_on_table("+-+-+") == 1
    assert coins_on_table("") == 0


# ========================================================
# FOLLOW-UP 1: Each event is an integer delta like +3 or -2, still never below zero
# ========================================================
# ---- follow-up 1: function ----
# Answer: split the text into tokens, turn each into a number with int()
# and apply total = max(0, total + delta). The clamp must happen after
# EVERY event, not once at the end.
def coins_with_deltas(events):
    total = 0
    for token in events.split():
        total += int(token)
        if total < 0:
            total = 0
    return total


# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert coins_with_deltas("+3 -2 -5 +4") == 4
    random.seed(16)
    for round_number in range(300):                # +1 / -1 deltas must agree with the plus/minus version
        s = ""
        for i in range(random.randint(0, 12)):
            s += random.choice("+-")
        tokens = []
```

```
        for c in s:
            if c == "+":
                tokens.append("+1")
            else:
                tokens.append("-1")
        assert coins_with_deltas(" ".join(tokens)) == coins_on_table(s)


# ========================================================
# RUN EVERYTHING
# ========================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

100 Visible People in a Line (Last Person, Then Everyone) — MEDIUM

Template fit: scan from the end carrying the tallest person seen between; count a person when no taller one blocks the view

QUESTION

Given heights of people in a line, A can see B if nobody between them is taller than both. Part 1: count how many people the last person can see. Part 2: return that count for every person. Written in a plain editor with no compilation.

Input: an array of heights representing a line of people.
Visibility rule: person A can see person B if no one standing between them is taller than both A and B.
Part 1: return how many people the last person in line can see.
Part 2: for the same array, return, for every person in the line, how many people they can see.

EXAMPLES

```
[1, 10, 6, 7, 9, 8, 2, 4, 3, 5] -> 6 (the last person, height 5, can see 3, 4, 2, 8, 9, 10).
```

FULL CODE | main question first, then every follow-up, tests at the end

```
# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ========================================================

# ========================================================
# MAIN QUESTION
# ========================================================
# ---- main question: helper functions ----
def bigger(a, b):
    if a > b:
        return a
    return b


# ---- main question: main function ----
# Part 1: how many people can the LAST person see? Person k is seen by
# the last person when nobody standing strictly between them is taller
# than both of them. Walk from the second-last person to the front and
# keep the tallest height seen so far between us.
def last_sees(a):
    if len(a) == 0:
```

```
        return 0
    last = a[-1]
    between = float("-inf")                     # the tallest person strictly between k and the last person
    count = 0
    for k in range(len(a) - 2, -1, -1):
        if between <= bigger(last, a[k]):
            count += 1
        between = bigger(between, a[k])
    return count


# ---- main question: tests ----
LINE = [1, 10, 6, 7, 9, 8, 2, 4, 3, 5]


def test_main_question():
    assert last_sees(LINE) == 6
    assert last_sees([]) == 0 and last_sees([7]) == 0
    assert last_sees([5, 5, 5]) == 2                # equal heights never block each other
```

```
# ========================================================
# FOLLOW-UP 1: Part 2: return the number of people each person can see
# ========================================================
# ---- follow-up 1: functions ----
# Answer: count, for every person, how many people they see looking
# right; do the same on the reversed line for looking left; add the two.
# Looking right from i: everyone up to the first STRICTLY taller person t
# is visible, and after t only the people who are at least as tall as
# everybody from t on (the chain of "next at least as tall" people).
# Monotonic stacks give "next at least as tall" and "next strictly
# taller" for every position in O(n).
def see_right(a):
    n = len(a)
    next_ge = [None] * n                        # next person at least as tall
    next_gt = [None] * n                        # next person strictly taller
    stack = []
    for k in range(n - 1, -1, -1):
        while stack and a[stack[-1]] < a[k]:
            stack.pop()
        if stack:
            next_ge[k] = stack[-1]
        stack.append(k)
    stack = []
    for k in range(n - 1, -1, -1):
        while stack and a[stack[-1]] <= a[k]:
            stack.pop()
        if stack:
            next_gt[k] = stack[-1]
        stack.append(k)
    chain = [0] * n                             # how many "at least as tall" steps follow k
    for k in range(n - 1, -1, -1):
        if next_ge[k] is not None:
            chain[k] = 1 + chain[next_ge[k]]
    result = []
    for i in range(n):
        t = next_gt[i]
        if t is None:
            result.append(n - 1 - i)
        else:
            result.append(t - i + chain[t])
    return result


def see_all(a):
    # looking left is looking right on the reversed line
```

```
    left = see_right(a[::-1])[::-1]
    right = see_right(a)
    total = []
    for i in range(len(a)):
        total.append(left[i] + right[i])
    return total


# ---- follow-up 1: tests ----
def can_see_slow(a, i, j):
    # i sees j when nobody strictly between them is taller than both
    low, high = i, j
    if i > j:
        low, high = j, i
    for k in range(low + 1, high):
        if a[k] > bigger(a[i], a[j]):
            return False
    return True


def test_follow_up_1():
    assert see_all(LINE)[-1] == 6
    assert see_all([5, 5, 5]) == [2, 2, 2]
    for a in ([3, 1, 2], [2, 2, 1, 3], [1, 2, 3, 4], [4, 3, 2, 1], LINE):     # compare with checking every p
air
        for i in range(len(a)):
            count = 0
            for j in range(len(a)):
                if i != j and can_see_slow(a, i, j):
                    count += 1
            assert see_all(a)[i] == count


# ============================================================
# FOLLOW-UP 2: How do ties and the direction of sight change the answer?
# ============================================================
# Answer: with "strictly taller blocks" (used above) equal heights see
# each other; if equal heights also block, the stack pops on <= instead of
# <. Looking left is the same code on the reversed line; "who can see ME"
# is the same count seen from the other side.


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
```

```
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

101 Bounded Knapsack Variant

MEDIUM

Template fit: sort items by value, highest first; take min(available count, remaining capacity) of each

QUESTION

Each item type has a value and an available count; every item weighs 1. Given capacity W, maximize total value. DP or DFS + memo works at O(n·W), greedy by value after sorting is O(n log n), and the best answer uses quickselect for O(n).

Input: for each item type, its value and the number of copies available. Every item has weight 1.
Input: a maximum total weight W.
Output: the maximum total value achievable without exceeding W.

EXAMPLES

Example 1	
Input	`[(10, 2), (6, 3), (3, 5)], 4`
Output	`32`
Why	`Items are (value, available count), each weighs 1. Take both 10s and two 6s: 20 + 12 = 32.`

Example 2	
Input	`[(10, 2), (6, 3), (3, 5)], 7`
Output	`44`
Why	`A bigger capacity takes more items.`

FULL CODE | main question first, then every follow-up, tests at the end

```
# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

import random


# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: main function ----
# Idea: every item weighs 1, so every slot of the bag is worth the same
# and the dearest copies should go in first. items are (value, count)
# pairs and W is the number of slots. Sort by value, highest first; for
# each type take as many copies as still fit and add value * taken.
def max_value_sorted(items, W):
    total = 0
    for value, count in sorted(items, reverse=True):
        take = count
        if W < take:
            take = W                                # only W slots are left
        total += value * take
        W -= take
        if W == 0:
            break
    return total


# the same answer in O(n) expected time: quickselect finds where the W-th
# slot falls without sorting everything
def max_value_select(items, W):
    total = 0
```

```
    pool = list(items)
    while pool and W > 0:
        pivot = random.choice(pool)[0]
        high = []
        middle = []
        low = []
        for value, count in pool:
            if value > pivot:
                high.append((value, count))
            elif value == pivot:
```

```
                middle.append((value, count))
            else:
                low.append((value, count))
        high_count = 0
        for value, count in high:
            high_count += count
        if high_count >= W:
            pool = high                          # the bag fills up inside the dearer types
            continue
        for value, count in high:
            total += value * count               # every dearer copy fits
        W -= high_count
        middle_count = 0
        for value, count in middle:
            middle_count += count
        take = middle_count
        if W < take:
            take = W
        total += pivot * take
        W -= take
        pool = low
    return total


# ---- main question: tests ----
def test_main_question():
    demo = [(3, 2), (5, 1), (4, 4)]
    assert max_value_sorted(demo, 4) == 17           # 5 + 4 + 4 + 4
    assert max_value_select(demo, 4) == 17
    assert max_value_sorted([], 5) == 0
    assert max_value_sorted([(7, 3)], 0) == 0
    assert max_value_sorted([(7, 3)], 10) == 21
    random.seed(9)
    for round_number in range(200):
        items = []
        for i in range(random.randint(0, 6)):
            items.append((random.randint(1, 9), random.randint(1, 4)))
        W = random.randint(0, 12)
        assert max_value_select(items, W) == max_value_sorted(items, W)


# =====================================================
# FOLLOW-UP 1: Why is greedy correct here but not for a general knapsack?
# =====================================================
# Answer: with unit weights all slots are equal, so swapping a cheaper
```

```
# copy for a dearer one never hurts (an exchange argument). With
# different weights a dear item can waste space that two smaller items
# would use better.


# =====================================================
# FOLLOW-UP 2: What if the weights are not all 1?
# =====================================================
# ---- follow-up 2: function ----
```

```python
# Answer: it is a real bounded knapsack and greedy fails; use a DP over
# the capacity. Splitting each count into chunks of 1, 2, 4, ... copies
# keeps the number of items small. items are (value, weight, count).
def knapsack_bounded(items, W):
    best = [0] * (W + 1)                    # best[w] = best value with capacity w
    for value, weight, count in items:
        chunk = 1
        while count > 0:
            take = chunk
            if count < take:
                take = count
            count -= take
            cost = take * weight
            gain = take * value
            for w in range(W, cost - 1, -1):      # backwards so one chunk is used once
                if best[w - cost] + gain > best[w]:
                    best[w] = best[w - cost] + gain
            chunk *= 2
    return best[W]


# ---- follow-up 2: tests ----
def test_follow_up_2():
    mixed = [(10, 5, 1), (6, 3, 2)]
    assert knapsack_bounded(mixed, 6) == 12
    unit = [(3, 1, 2), (5, 1, 1), (4, 1, 4)]          # with every weight 1 it must agree with the greedy
    assert knapsack_bounded(unit, 4) == 17


# ============================================================
# FOLLOW-UP 3: W is up to 10^9. Which version still works?
# ============================================================
# Answer: the greedy (sort by value, take from the top) only depends on
# the number of item TYPES, not on W, so it still works. The DP needs an
# array of size W and is out of the question.
```

```python
# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_2()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

102 Find the Largest Monotone Increasing Number — MEDIUM

Template fit: scan the digits right to left; when a digit is smaller than the one before it, lower the earlier digit and fill 9s after it

QUESTION

Given an integer n, return the largest number not greater than n whose digits never decrease from left to right (for example 1223 qualifies, 1232 does not).

Function signature

def largest_monotone(n: int) -> int

EXAMPLES

QUESTION

Example 1	
Input	10
Output	9
Why	10 itself has a drop (1 then 0); the largest valid number below it is 9.

Example 2	
Input	1234
Output	1234
Why	1234 is already monotone.

Example 3	
Input	332
Output	299
Why	332: lower the first 3 to 2 and fill the rest with 9s: 299.

FULL CODE | main question first, then every follow-up, tests at the end

```
# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ========================================================

# ========================================================
# MAIN QUESTION
# ========================================================
# ---- main question: main function ----
# Idea: the largest number <= n whose digits never decrease. Scan the
# digits from RIGHT to LEFT. When a digit is smaller than the one before
# it (a drop), the left digit must go down by one, and every digit after
# it can become 9 (the largest choice). Remember where the nines start.
def largest_monotone(n):
    digits = list(str(n))
    nines_from = len(digits)
    for i in range(len(digits) - 1, 0, -1):
        if digits[i - 1] > digits[i]:
            digits[i - 1] = str(int(digits[i - 1]) - 1)
            nines_from = i
    for i in range(nines_from, len(digits)):
        digits[i] = "9"
    return int("".join(digits))


# ---- main question: tests ----
def is_monotone(n):
    text = str(n)
    for i in range(len(text) - 1):
        if text[i] > text[i + 1]:
            return False
    return True


def largest_monotone_slow(n):
    # count down from n until the digits never decrease (obviously right, slow)
    while not is_monotone(n):
        n -= 1
    return n
```

```
def test_main_question():
    assert largest_monotone(10) == 9
```

```
    assert largest_monotone(1234) == 1234
    assert largest_monotone(332) == 299
    assert largest_monotone(0) == 0
    for n in range(0, 3000):
        assert largest_monotone(n) == largest_monotone_slow(n)


# =====================================================
# FOLLOW-UP 1: The number is given as a string with 100,000 digits
# =====================================================
# ---- follow-up 1: function ----
# Answer: the scan only needs the digit characters, so keep the digits in
# a list and never convert to an integer: O(d). A leading zero (10 turns
# into 09) is dropped at the end.
def largest_monotone_string(digits):
    d = list(digits)
    nines_from = len(d)
    for i in range(len(d) - 1, 0, -1):
        if d[i - 1] > d[i]:
            d[i - 1] = str(int(d[i - 1]) - 1)
            nines_from = i
    for i in range(nines_from, len(d)):
        d[i] = "9"
    start = 0
    while start < len(d) - 1 and d[start] == "0":
        start += 1
    return "".join(d[start:])


# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert largest_monotone_string("332") == "299"
    assert largest_monotone_string("10") == "9"
    for n in range(0, 3000):
        assert largest_monotone_string(str(n)) == str(largest_monotone(n))
    big = "9" * 50000 + "1" * 50000                    # a 100,000 digit input is fine as a string
    assert largest_monotone_string(big)[:3] == "899"


# =====================================================
# FOLLOW-UP 2: Return the smallest monotone number that is at least n instead
# =====================================================
# ---- follow-up 2: function ----
# Answer: scan LEFT to right for the first digit that is bigger than its
# right neighbour, and copy that digit over the whole rest (332 becomes
```

```
# 333). The part before it is already in order.
def smallest_monotone_at_least(n):
    d = list(str(n))
    for i in range(len(d) - 1):
        if d[i] > d[i + 1]:
            for j in range(i + 1, len(d)):
                d[j] = d[i]
            break
    return int("".join(d))


# ---- follow-up 2: tests ----
def test_follow_up_2():
    assert smallest_monotone_at_least(332) == 333
    assert smallest_monotone_at_least(120) == 122
```

```
    for n in range(0, 3000):
        up = n
        while not is_monotone(up):
            up += 1
        assert smallest_monotone_at_least(n) == up


# ======================================================
# FOLLOW-UP 3: Digits must be strictly increasing
# ======================================================
# Answer: the same right-to-left scan, but a drop is also an equal pair
# (digit[i - 1] >= digit[i]), and the filling after the cut is
# 9, 8, 7... going leftwards (the largest strictly increasing ending).


# ======================================================
# RUN EVERYTHING
# ======================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

103 Min boards to cover roof holes

MEDIUM

Template fit: the template as written

QUESTION

Holes in a roof are at integer positions along a line (the same position may be listed twice). A board of the given length covers that many consecutive positions. Return the fewest boards that cover every hole.

Function signature

def min_boards(holes: list, length: int) -> int

EXAMPLES

Example 1	
Input	`[1, 2, 4, 7, 8], 3`
Output	`3`
Why	`Boards cover [1..3], [4..6] and [7..9].`

Example 2	
Input	`[1, 10], 5`
Output	`2`
Why	`The holes are far apart: one board each.`

Example 3	
Input	`[5, 5, 5], 1`
Output	`1`
Why	`Repeated positions need only one board.`

FULL CODE | main question first, then every follow-up, tests at the end

```python
# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ========================================================

from bisect import bisect_left


# ========================================================
# MAIN QUESTION
# ========================================================
# ---- main question: main function ----
# Idea (greedy): sort the holes. The leftmost hole that is not covered
# yet must start a new board, and the best place for it is exactly at
# that hole (starting earlier covers nothing more). That board covers
# every hole below start + length. Count the boards.
def min_boards(holes, length):
    count = 0
    covered_to = float("-inf")              # the last position the boards so far cover
    for position in sorted(set(holes)):
        if position > covered_to:
            count += 1
            covered_to = position + length - 1
    return count


# ---- main question: tests ----
def test_main_question():
    assert min_boards([1, 2, 4, 5, 9], 3) == 3
    assert min_boards([], 3) == 0
    assert min_boards([5, 5, 5], 1) == 1


# ========================================================
# FOLLOW-UP 1: You have only K boards: what is the smallest board length that covers all holes?
# ========================================================
# ---- follow-up 1: function ----
# Answer: binary search the board length L. For a given L the greedy tells
# how many boards are needed; "at most K boards" gets easier as L grows,
# so binary search works: O(n log range).
def smallest_length(holes, k):
    points = sorted(set(holes))
    if not points:
        return 0
    low = 1
    high = points[-1] - points[0] + 1         # one board that covers everything
    while low < high:
        mid = (low + high) // 2
        if min_boards(points, mid) <= k:
            high = mid                        # long enough: try shorter
        else:
            low = mid + 1
    return low


# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert smallest_length([1, 2, 4, 5, 9], 2) == 5       # boards cover 1..5 and 9..13
    assert smallest_length([1, 2, 4, 5, 9], 1) == 9
    assert smallest_length([], 1) == 0


# ========================================================
# FOLLOW-UP 2: Boards of different lengths with a cost each
```

```python
# ============================================================
# ---- follow-up 2: function ----
# Answer: the greedy argument breaks. Use a DP over the sorted holes:
# best[i] = the cheapest way to cover the holes from i onward = the
# minimum over the board types of (cost + best[j]), where j is the first
# hole that a board of that type placed at hole i does NOT reach.
def min_cost(holes, boards):
    # boards is a list of (length, cost); you may use any number of each
    points = sorted(set(holes))
    best = [0] * (len(points) + 1)
    for i in range(len(points) - 1, -1, -1):
        best[i] = float("inf")
        for length, cost in boards:
            j = bisect_left(points, points[i] + length)
            if cost + best[j] < best[i]:
                best[i] = cost + best[j]
    return best[0]


# ---- follow-up 2: tests ----
def test_follow_up_2():
    assert min_cost([1, 2, 3, 10], [(3, 5), (1, 3)]) == 8     # a long board costs 5, a short one 3
    assert min_cost([1, 2, 3, 4], [(2, 3), (4, 5)]) == 5
```

```python
# ============================================================
# FOLLOW-UP 3: Holes are in a grid and boards are 1 x L horizontally
# ============================================================
# ---- follow-up 3: function ----
# Answer: boards cannot cross rows, so run the same greedy on every row
# and add the counts. (If boards may also be vertical it becomes a
# covering problem on a bipartite graph: Konig's theorem.)
def boards_per_row(grid, length):
    total = 0
    for row in grid:
        holes = []
        for c in range(len(row)):
            if row[c] == "#":
                holes.append(c)
        total += min_boards(holes, length)
    return total


# ---- follow-up 3: tests ----
def test_follow_up_3():
    assert boards_per_row(["#.#.", "..##", "...."], 2) == 3


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

104 Jump Game II (Minimum Jumps) MEDIUM

Template fit: scan the indices carrying the farthest reach; when you hit the end of the current jump, you must jump again: jumps += 1, end = farthest

QUESTION

You are given a 0-indexed array of non-negative integers nums . You start at index 0. From any index i , you can jump to any index in the range [i + 1, i + nums[i]] (i.e., nums[i] is the maximum forward jump length from i). Return the minimum number of jumps required to reach the last index len(nums)

- 1 . The test cases guarantee that the last index is always reachable.

Function signature

```
def jump(nums: list[int]) -> int:
    """Return the minimum number of jumps to reach the last index."""
    ...
```

Follow-up (as reported): Solve it in O(n) time and O(1) extra space with a greedy "current reach vs. farthest reach" scan rather than O(n) DP.

EXAMPLES

Example 1	
Input	`nums = [2, 3, 1, 1, 4]`
Output	`2`
Why	`Jump 1 step from index 0 to 1, then 3 steps to the last index.`

Example 2	
Input	`nums = [2, 3, 0, 1, 4]`
Output	`2`
Why	`0 -> 1 -> 4.`

Example 3	
Input	`nums = [0]`
Output	`0`
Why	`Already at the last index.`

CONSTRAINTS

- 1 <= len(nums) <= 10^4
- 0 <= nums[i] <= 1000
- If the array has length 1 you are already at the last index, so the answer is 0.

FULL CODE | main question first, then every follow-up, tests at the end

```
# ====================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ====================================================

import random


# ====================================================
# MAIN QUESTION
# ====================================================
# ---- main question: main function ----
# Idea (greedy by ranges): each jump lets you reach a whole range of
```

```
# indexes. Walk through the array and remember the farthest index you
# can reach from anything seen so far. When you pass the end of the
# range of the current jump, you must take another jump, and the new
# range ends at that farthest index.
def jump(nums):
    jumps = 0
    current_end = 0                      # the last index reachable with `jumps` jumps
    farthest = 0
    for i in range(len(nums) - 1):
        if i + nums[i] > farthest:
            farthest = i + nums[i]
        if i == current_end:             # the range of this jump is used up: take another
            jumps += 1
            current_end = farthest
    return jumps


# ---- main question: tests ----
def test_main_question():
    assert jump([2, 3, 1, 1, 4]) == 2
    assert jump([2, 3, 0, 1, 4]) == 2
    assert jump([0]) == 0
    assert jump([1, 1, 1, 1]) == 3


# ======================================================
# FOLLOW-UP 1: Return the actual indexes you jump through, not just the count
# ======================================================
# ---- follow-up 1: function ----
# Answer: remember, for the current range, the index that reaches
```

```
# farthest. When the range ends, that index is where the NEXT jump lands.
# Add it to the path (the first range, which is just index 0, adds
# nothing). The last landing is always the last index. Still O(n).
def jump_path(nums):
    last = len(nums) - 1
    path = [0]
    if last == 0:
        return path
    current_end = 0
    farthest = 0
    best_index = 0                       # the index that reaches farthest so far
    for i in range(last):
        if i + nums[i] > farthest:
            farthest = i + nums[i]
            best_index = i
        if i == current_end:
            if i > 0:
                path.append(best_index)
            current_end = farthest
    path.append(last)
    return path


# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert jump_path([2, 3, 1, 1, 4]) == [0, 1, 4]
    assert jump_path([0]) == [0]
    assert jump_path([3, 1, 1, 1]) == [0, 3]
    random.seed(5)
    for round_number in range(300):              # every jump is legal and the length matches jump()
        nums = []
        for i in range(random.randint(1, 10)):
            nums.append(random.randint(1, 4))
        path = jump_path(nums)
        assert path[0] == 0 and path[-1] == len(nums) - 1
```

```
        for k in range(len(path) - 1):
            assert 0 < path[k + 1] - path[k] <= nums[path[k]]
        assert len(path) - 1 == jump(nums)


# ====================================================
# RUN EVERYTHING
# ====================================================
def run_all_tests():
    test_main_question()
```

```
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

GROUP 13 OF 43 | 8 QUESTIONS, ONE CODE TEMPLATE | 4 E 3 M 1 H

Sets and counters: remember the first collection, check the second against it

105 Exclude First-K-Unique From Another List

EASY

Template fit: the set holds the first k distinct values of list 1; keep list 2's values that are not in it

QUESTION

You are given two lists list1 and list2 and an integer k . Scan list1 left to right and collect the first k distinct values it contains (in order of first appearance; if list1 has fewer than k distinct values, take all of them). Then return list2 with every occurrence of any of those collected values removed, preserving the order of the remaining elements.

Function signature

def exclude_first_k_unique(list1: list[int], list2: list[int], k: int) -> list[int]:

...

EXAMPLES

Example 1	
Input	`list1 = [3, 3, 1, 4, 1, 5], list2 = [1, 2, 3, 4, 3, 5], k = 2`
Output	`[2, 4, 5]`
Why	`First 2 distinct values of list1 are {3, 1}. Removing all 3s and 1s from list2 leaves [2, 4, 5].`

Example 2	
Input	`list1 = [7], list2 = [7, 8, 9], k = 3`
Output	`[8, 9]`
Why	`list1 has only one distinct value (7); remove all 7s from list2.`

CONSTRAINTS

- 0 <= len(list1), len(list2) <= 10**5 , 1 <= k .
- Values are integers.

FULL CODE | main question first, then every follow-up, tests at the end

```
# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ========================================================

# ========================================================
# MAIN QUESTION
# ========================================================
# ---- main question: helper functions ----
def first_k_unique(list1, k):
    # the first k DISTINCT values of list1, as a set
    blocked = set()
    for x in list1:
        if len(blocked) == k:
            break
        blocked.add(x)
    return blocked


# ---- main question: main function ----
```

```python
# Take the first k distinct values of list1 and remove them from list2.
# Idea: put those values in a set; a set lookup is O(1), so filtering
# list2 takes one pass.
def exclude_first_k_unique(list1, list2, k):
    blocked = first_k_unique(list1, k)
    result = []
    for y in list2:
        if y not in blocked:
            result.append(y)
    return result


# ---- main question: tests ----
def test_main_question():
    assert exclude_first_k_unique([3, 3, 1, 4, 1, 5], [1, 2, 3, 4, 3, 5], 2) == [2, 4, 5]
    assert exclude_first_k_unique([7], [7, 8, 9], 3) == [8, 9]


# ========================================================
# FOLLOW-UP 1: list2 is a huge stream. How do you avoid loading it?
# ========================================================
# ---- follow-up 1: function ----
# Answer: build the blocked set from list1 first (O(k) memory), then
```

```python
# yield the stream's values one at a time, skipping the blocked ones. The
# result is a generator, so list2 is never loaded.
def exclude_stream(list1, stream, k):
    blocked = first_k_unique(list1, k)
    for y in stream:
        if y not in blocked:
            yield y


# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert list(exclude_stream([3, 3, 1, 4, 1, 5], iter([1, 2, 3, 4, 3, 5]), 2)) == [2, 4, 5]


# ========================================================
# RUN EVERYTHING
# ========================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

106 Find the Duplicates — EASY

Template fit: values that appear in both lists, each reported once (set intersection, or two pointers on sorted lists)

QUESTION

Given two sorted arrays arr1 and arr2 of passport numbers, implement a function findDuplicates that returns an array of all passport numbers that are both arrays. Note that the output array should be sorted in ascending order. Let N and M be the lengths of arr1 and arr2, respectively. Solve for two cases and analyze the time & space complexities of your solutions: M ~ N - the array lengths are approximately the same M >> N - arr2 is much bigger than arr1. Example input: arr1 = [1, 2, 3, 5, 6, 7], arr2 = [3, 6, 7, 8, 20] output: [3, 6, 7]

since only these three values

QUESTION

are both in arr1 and arr2Hint 1Start with a brute force solution and optimize from there.Hint 2Note that the question asks for a sorted array. What techniques work well on sorted arrays?Hint 3If M >> N (case #2) is giving you trouble, consider how you might use the values of N, the shorter array, to increase the efficiency of your answer.Challenge 1Have you taken care to avoid array out-of-bound runtime errors?First, the brute force solution. Try two nested loops. For every value arr1[i] in the outer loop, we look it up in arr2 in the inner loop. The time complexity of this algorithm in the worst case scenario is O(N*M). Not very efficient. We can do better than that. Case 1 (M ~ N) We can use the fact the arrays are sorted to traverse both of them in an in-order manner at the same time. The general idea of the algorithm is to use two indices, i and j, for arr1 and arr2, respectively. Every time one of indices,

EXAMPLES

Example 1	
Input	`[1, 3, 5, 7], [3, 4, 5, 8]`
Output	`[3, 5]`
Why	`Two sorted lists of passport numbers: 3 and 5 appear in both.`

Example 2	
Input	`[1, 2], [3, 4]`
Output	`[]`
Why	`Nothing in common.`

FULL CODE | main question first, then every follow-up, tests at the end

```
# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

import random

END = object()


# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
def first_at_least(values, x, start):
    # binary search in a sorted list, looking only from index `start` on:
    # the first index whose value is >= x
    low = start
    high = len(values)
    while low < high:
        middle = (low + high) // 2
        if values[middle] < x:
            low = middle + 1
        else:
            high = middle
    return low


def bit_length(n):
    # how many binary digits n has (roughly log2 of n)
    bits = 0
    while n > 0:
        bits += 1
        n = n // 2
```

```
    return bits


# ---- main question: main functions ----
# Find the values that appear in BOTH sorted arrays (each result once).
# CASE 1: the arrays have similar sizes: two pointers. Walk both arrays
# together and move on the pointer that points at the smaller value; when
# they are equal, record the value once and move both. O(N + M) time.
def common_two_pointers(a, b):
```

```
    result = []
    i = 0
    j = 0
    while i < len(a) and j < len(b):
        if a[i] < b[j]:
            i += 1
        elif a[i] > b[j]:
            j += 1
        else:
            if not result or result[len(result) - 1] != a[i]:
                result.append(a[i])
            i += 1
            j += 1
    return result


# CASE 2: one array is much shorter: binary-search every item of the short
# array in the long one. O(N log M); each search starts where the previous
# one stopped.
def common_binary_search(small, big):
    result = []
    position = 0
    for x in small:
        position = first_at_least(big, x, position)
        if position == len(big):
            break
        if big[position] == x and (not result or result[len(result) - 1] != x):
            result.append(x)
    return result


# Choose by cost: N log M against N + M.
def common(a, b):
    if len(a) <= len(b):
        small = a
        big = b
    else:
        small = b
        big = a
    if len(small) * bit_length(len(big)) < len(a) + len(b):
        return common_binary_search(small, big)
    return common_two_pointers(a, b)


# ---- main question: tests ----
```

```
def test_main_question():
    a = [1, 2, 3, 5, 6, 7]
    b = [3, 6, 7, 8, 20]
    assert common_two_pointers(a, b) == [3, 6, 7]
    assert common_binary_search(b, a) == [3, 6, 7]
    assert common(a, b) == [3, 6, 7]
    assert common([], [1]) == []
    assert common([1, 1, 2], [1, 1, 3]) == [1]
    assert common([5, 999999], list(range(0, 1000000, 3))) == [999999]      # one short array against a very
long one
```

```
# ====================================================
# FOLLOW-UP 1: Three or more arrays
# ====================================================
# ---- follow-up 1: function ----
# Answer: intersect pairwise, the smallest arrays first so the running
# result shrinks quickly: O(total size). With K arrays you can also keep K
# pointers and move the one at the smallest value.
def common_many(arrays):
    ordered = sorted(arrays, key=len)
    result = ordered[0]
    for other in ordered[1:]:
        result = common(result, other)
        if not result:
            break
    return result


# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert common_many([[1, 2, 3], [7, 8], [3]]) == []
    assert common_many([[1, 2, 3, 4], [2, 3, 9], [2, 3, 5, 6, 7]]) == [2, 3]


# ====================================================
# FOLLOW-UP 2: The arrays are too big for memory
# ====================================================
# ---- follow-up 2: function ----
# Answer: read both as streams (two pointers need only the CURRENT item of
# each), or split both by value range into files that fit in memory and
# intersect them part by part.
def common_stream(a_items, b_items):
    a_stream = iter(a_items)
    b_stream = iter(b_items)
    x = next(a_stream, END)
```

```
    y = next(b_stream, END)
    last = END
    while x is not END and y is not END:
        if x < y:
            x = next(a_stream, END)
        elif x > y:
            y = next(b_stream, END)
        else:
            if x != last:
                yield x
                last = x
            x = next(a_stream, END)
            y = next(b_stream, END)


# ---- follow-up 2: tests ----
def test_follow_up_2():
    assert list(common_stream(iter([1, 2, 3, 5, 6, 7]), iter([3, 6, 7, 8, 20]))) == [3, 6, 7]
    random.seed(23)
    for round_number in range(300):
        x = sorted(random.sample(range(40), random.randint(0, 9)))
        y = sorted(random.sample(range(40), random.randint(0, 9)))
        expected = []
        for value in x:
            if value in y:
                expected.append(value)
        assert common_two_pointers(x, y) == expected
        assert common_binary_search(x, y) == expected
        assert common(x, y) == expected
```

```
        assert list(common_stream(x, y)) == expected


# ============================================================
# FOLLOW-UP 3: Which one do you choose when N = 1,000 and M = 10^9?
# ============================================================
# Answer: binary search: 1,000 * 30 steps is far fewer than 10^9, so the
# long array is hardly read at all (common() makes this choice by comparing
# the costs).


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
```

```
    test_follow_up_1()
    test_follow_up_2()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

107 Positional Matches Between Two Lists

EASY

Template fit: no set needed: zip both lists and keep a where a == b

QUESTION

You are given two lists of equal length, first and second . Return, in order, the values at every position where the two lists agree — that is, all first[i] such that first[i] == second[i] .

Function signature

def positional_matches(first: list[int], second: list[int]) -> list[int]:
    ...

EXAMPLES

Example 1	
Input	`first = [3, 1, 4, 1, 5, 5, 7, 9], second = [1, 2, 3, 4, 5, 6, 7, 8]`
Output	`[5, 7]`
Why	`Positions 4 (5 == 5) and 6 (7 == 7) match. (The report described "matches" between a digits-of-pi list and an index list.)`

Example 2	
Input	`first = [1, 2, 3], second = [4, 5, 6]`
Output	`[]`
Why	`No position matches.`

CONSTRAINTS

- len(first) == len(second) , 0 <= length <= 10**6 .
- Values are integers.

FULL CODE | main question first, then every follow-up, tests at the end

```
# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
```

```
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

import random


# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: main function ----
# Idea: walk both lists together and keep the value wherever they agree at
# the same position. The shorter list decides how far to go.
def positional_matches(first, second):
    matches = []
    for i in range(min(len(first), len(second))):
        if first[i] == second[i]:
            matches.append(first[i])
    return matches


# ---- main question: tests ----
def test_main_question():
    assert positional_matches([3, 1, 4, 1, 5, 5, 7, 9], [1, 2, 3, 4, 5, 6, 7, 8]) == [5, 7]
    assert positional_matches([1, 2, 3], [4, 5, 6]) == []


# ============================================================
# FOLLOW-UP 1: Return the positions instead of the values, lazily for a huge input
# ============================================================
# ---- follow-up 1: function ----
# Answer: write a GENERATOR that yields each matching position as it finds
# it. Nothing is stored, so memory is O(1), and the caller can stop early.
def matching_positions(first, second):
    for i in range(min(len(first), len(second))):
        if first[i] == second[i]:
            yield i


# ---- follow-up 1: tests ----
def test_follow_up_1():
    random.seed(64)
```

```
    for round_number in range(300):
        n = random.randint(0, 9)
        a = []
        b = []
        for i in range(n):
            a.append(random.randint(0, 3))
            b.append(random.randint(0, 3))
        values = []
        for position in matching_positions(a, b):
            values.append(a[position])
        assert values == positional_matches(a, b)


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
```

```
run_all_tests()
```

108 Contains Duplicate

EASY

Template fit: add values to a set as you go; the first value already in the set means a duplicate

QUESTION

Given an integer array nums , return True if any value appears at least twice in the array, and False if every element is distinct.

Function signature

def contains_duplicate(nums: list[int]) -> bool:
    """Return True if any value appears more than once."""
    ...

EXAMPLES

Example 1	
Input	`nums = [1, 2, 3, 1]`
Output	`True`

Example 2	
Input	`nums = [1, 2, 3, 4]`
Output	`False`

Example 3	
Input	`nums = [1, 1, 1, 3, 3, 4, 3, 2, 4, 2]`
Output	`True`

CONSTRAINTS

- 1 <= len(nums) <= 10^5
- -10^9 <= nums[i] <= 10^9

FULL CODE | main question first, then every follow-up, tests at the end

```python
# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

import random


# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: main function ----
# Idea: remember the values seen so far in a set; meeting a value that is
# already in the set means a duplicate. O(n) time, O(n) memory.
def contains_duplicate(nums):
    seen = set()
    for x in nums:
        if x in seen:
            return True
        seen.add(x)
    return False
```

```
# ---- main question: tests ----
def test_main_question():
    assert contains_duplicate([1, 2, 3, 1]) is True
    assert contains_duplicate([1, 2, 3, 4]) is False
    assert contains_duplicate([]) is False


# ========================================================
# FOLLOW-UP 1: What if you cannot use extra memory?
# ========================================================
# ---- follow-up 1: function ----
# Answer: sort (in place if allowed) and compare neighbours: duplicates
# become adjacent. O(n log n) time and no extra set.
def contains_duplicate_sorted(nums):
    ordered = sorted(nums)
    for i in range(len(ordered) - 1):
        if ordered[i] == ordered[i + 1]:
            return True
    return False
```

```
# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert contains_duplicate_sorted([1, 2, 3, 1]) is True
    assert contains_duplicate_sorted([1, 2, 3, 4]) is False


# ========================================================
# FOLLOW-UP 2: Only count it if the two equal values are at most k positions apart
# ========================================================
# ---- follow-up 2: function ----
# Answer: keep a set of just the last k values (a sliding window): check
# the new value against it, add it, and drop the value that just left the
# window. O(n).
def duplicate_within_k(nums, k):
    window = set()
    for i in range(len(nums)):
        if nums[i] in window:
            return True
        window.add(nums[i])
        if len(window) > k:
            window.remove(nums[i - k])
    return False


# ---- follow-up 2: tests ----
def test_follow_up_2():
    random.seed(7)
    for round_number in range(300):
        nums = []
        for i in range(random.randint(0, 9)):
            nums.append(random.randint(0, 8))
        k = random.randint(1, 5)
        slow = False
        for i in range(len(nums)):                  # compare every pair that is at most k apart
            for j in range(i + 1, min(len(nums), i + k + 1)):
                if nums[i] == nums[j]:
                    slow = True
        assert duplicate_within_k(nums, k) == slow


# ========================================================
# RUN EVERYTHING
# ========================================================
def run_all_tests():
```

```
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

109 Filter Common Elements Per Window (Two Lists) MEDIUM

Template fit: build a set per window of the first list; keep the matching window of the second list minus that set

QUESTION

You are given two lists a and b of equal length n and a window size k that divides n . Split both lists into consecutive, non-overlapping windows of size k (window 0 is indices 0..k-1 , window 1 is k..2k-1 , and so on). For each window, remove from b 's window every value that also appears in the corresponding window of a . Concatenate the surviving elements of b , preserving their original order, and return that list.

Function signature

def filter_windows(a: list[int], b: list[int], k: int) -> list[int]:
    ...

Follow-up (as reported): The interviewer asked to do the removal in place / O(1) extra space. Naive in-place removal becomes O(n*k); discuss compaction with a write pointer to keep it O(n) time.

EXAMPLES

Example 1	
Input	`a = [1, 2, 3, 4, 5, 6], b = [3, 7, 8, 6, 9, 5], k = 3`
Output	`[7, 8, 9]`
Why	`Window 0: a={1,2,3}, b=[3,7,8] -> remove 3 -> [7,8]. Window 1: a={4,5,6}, b=[6,9,5] -> remove 6 and 5 -> [9]. Concatenate -> [7,8,9].`

Example 2	
Input	`a = [1, 1], b = [1, 2], k = 2`
Output	`[2]`
Why	`Single window: a={1}, b=[1,2] -> remove 1 -> [2].`

CONSTRAINTS

- 0 <= n <= 10**5 , n % k == 0 , 1 <= k <= n (for n > 0).
- Values are integers.

FULL CODE | main question first, then every follow-up, tests at the end

```
# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: main function ----
# Cut both lists into windows of k elements. In every window keep only the
# values of b that do NOT appear in the same window of a.
# Idea: put the window of a into a set (a set lookup is O(1)), then keep the
# elements of the window of b that are not in it.
```

```python
def filter_windows(a, b, k):
    result = []
    for start in range(0, len(a), k):
        seen = set(a[start:start + k])
        for x in b[start:start + k]:
            if x not in seen:
                result.append(x)
    return result


# ---- main question: tests ----
def test_main_question():
    assert filter_windows([1, 2, 3, 4, 5, 6], [3, 7, 8, 6, 9, 5], 3) == [7, 8, 9]
    assert filter_windows([1, 1], [1, 2], 2) == [2]


# ============================================================
# FOLLOW-UP 1: Do it in place: no new output list
# ============================================================
# ---- follow-up 1: function ----
# Answer: keep a write position `w`. For each window build the set of a's
# window, then copy every surviving element of b to b[w] and move w on; at the
# end cut b down to length w. Time O(n), extra space O(k) for one window's set.
def filter_windows_in_place(a, b, k):
    w = 0
    for start in range(0, len(a), k):
        seen = set(a[start:start + k])
        for j in range(start, min(start + k, len(b))):
            if b[j] not in seen:
                b[w] = b[j]
```

```python
                w += 1
    del b[w:]
    return b


# ---- follow-up 1: tests ----
def test_follow_up_1():
    b = [3, 7, 8, 6, 9, 5]
    assert filter_windows_in_place([1, 2, 3, 4, 5, 6], b, 3) == [7, 8, 9]
    assert b == [7, 8, 9]


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

110 Reconstruct original array from doubled shuffle

MEDIUM

Template fit: Counter of the values; go from the smallest, pairing each x with 2x and using both counts up

QUESTION

An array of numbers was doubled: for every element x of the original, the value 2x was added, and then all the numbers were shuffled. Given the shuffled array changed, return an original array (in any order), or an empty list if changed cannot be a doubled array. Zero and negative numbers are possible.

Function signature

def find_original(changed: list) -> list

EXAMPLES

Example 1	
Input	`[1, 3, 4, 2, 6, 8]`
Output	`[1, 3, 4]`
Why	`Original [1, 3, 4] gives doubles [2, 6, 8].`

Example 2	
Input	`[1, 3, 2]`
Output	`[]`
Why	`An odd number of values cannot be doubled.`

Example 3	
Input	`[0, 0, 0, 0]`
Output	`[0, 0]`
Why	`Zeros pair with zeros.`

Example 4	
Input	`[-2, -1, -4, -2]`
Output	`[-1, -2]`
Why	`Negatives work too: -2 is the double of -1.`

FULL CODE | main question first, then every follow-up, tests at the end

```
# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ========================================================

# ========================================================
# MAIN QUESTION
# ========================================================
# ---- main question: helper functions ----
def count_values(values):
    counts = {}
    for v in values:
        counts[v] = counts.get(v, 0) + 1
    return counts


def size_then_value(v):
    # the sort key: smaller absolute value first (a negative number sorts by
    # its size, so -1 comes before 2)
    return (abs(v), v)
```

```python
# ---- main question: main function ----
# `changed` holds every original x and also its double 2x, shuffled. Find
# the originals (or return [] if `changed` is not such an array).
# Idea: count the values and go through them from the SMALLEST absolute
# value: the smallest remaining value can only be an original, never someone's
# double. Every copy of x needs its own copy of 2x: if there are fewer copies of
# 2x, fail; otherwise use them up and write x down that many times. Zero is its
# own double, so zeros pair with each other. (Going by absolute value also makes
# negatives work: -2 is the double of -1, and |-1| < |-2|: follow-up 3.)
def find_original(changed):
    if len(changed) % 2 == 1:
        return []
    count = count_values(changed)
    originals = []
    for x in sorted(count, key=size_then_value):
        if count[x] == 0:
            continue                                    # all copies of x were used as doubles
        if x == 0:
            if count[0] % 2 == 1:
                return []
            for i in range(count[0] // 2):
```

```python
                originals.append(0)
            count[0] = 0
            continue
        if count.get(2 * x, 0) < count[x]:
            return []                                   # not enough doubles for every x
        for i in range(count[x]):
            originals.append(x)
        count[2 * x] -= count[x]
        count[x] = 0
    return originals


# ---- main question: tests ----
def test_main_question():
    assert sorted(find_original([1, 3, 4, 2, 6, 8])) == [1, 3, 4]
    assert find_original([6, 3, 0, 1]) == []
    assert find_original([1]) == []
    assert find_original([0, 0, 0, 0]) == [0, 0]
    assert find_original([0, 0, 0]) == []
    assert sorted(find_original([-1, -2, 4, 2])) == [-1, 2]       # -2 is the double of -1


# ========================================================
# FOLLOW-UP 1: Return the originals in the order they appear in the input
# ========================================================
# ---- follow-up 1: function ----
# Answer: first compute the multiset of originals, then scan the input once
# and keep each element while its remaining original count is positive.
def find_original_in_order(changed):
    originals = find_original(changed)
    if not originals and changed:
        return []
    left = count_values(originals)
    result = []
    for x in changed:
        if left.get(x, 0) > 0:
            left[x] -= 1
            result.append(x)
    return result


# ---- follow-up 1: tests ----
```

```python
def test_follow_up_1():
    assert find_original_in_order([4, 1, 2, 8, 2, 4]) == [4, 1, 2]
```

```python
# ============================================================
# FOLLOW-UP 2: Values are small non-negative integers
# ============================================================
# ---- follow-up 2: function ----
# Answer: use a counting ARRAY instead of sorting and a dictionary: go
# through the values from small to large, O(n + range).
def find_original_small(changed, limit):
    if len(changed) % 2 == 1:
        return []
    count = [0] * (2 * limit + 2)
    for x in changed:
        count[x] += 1
    if count[0] % 2 == 1:
        return []
    originals = []
    for i in range(count[0] // 2):
        originals.append(0)
    for x in range(1, limit + 1):
        if count[x] > 0:
            if count[2 * x] < count[x]:
                return []
            for i in range(count[x]):
                originals.append(x)
            count[2 * x] -= count[x]
    return originals


# ---- follow-up 2: tests ----
def test_follow_up_2():
    assert sorted(find_original_small([1, 2, 4, 8], 8)) == [1, 4]
    assert find_original_small([1, 3], 3) == []


# ============================================================
# FOLLOW-UP 3: Why does processing by absolute value work with negatives?
# ============================================================
# Answer: for a negative x the double 2x is further from zero, so the value
# with the smallest absolute value is always an original; going by absolute
# value handles negatives exactly like positives.


# ============================================================
# RUN EVERYTHING
# ============================================================
```

```python
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

111 Min deletions to avoid overlap in first k

MEDIUM

Template fit: Counter of the left part and of the right part; for each value in both, delete min(left count, right count)

QUESTION

Split an array into its first k elements and the rest. Return the minimum number of elements to delete so that no value appears on both sides (for every shared value delete all copies from the side that has fewer).

Function signature

def min_deletions(a: list, k: int) -> int

EXAMPLES

Example 1	
Input	`[1, 2, 2, 3, 2, 1], 3`
Output	`2`
Why	`Left [1, 2, 2], right [3, 2, 1]: value 1 appears once on each side (delete 1), value 2 appears twice on the left and once on the right (delete 1): 2 deletions.`

Example 2	
Input	`[1, 2, 3, 4], 2`
Output	`0`
Why	`No value is shared between [1, 2] and [3, 4].`

FULL CODE | main question first, then every follow-up, tests at the end

```
# ====================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ====================================================

# ====================================================
# MAIN QUESTION
# ====================================================
# ---- main question: helper functions ----
def count_values(values):
    counts = {}
    for v in values:
        counts[v] = counts.get(v, 0) + 1
    return counts


# ---- main question: main function ----
# The values in a[:k] and in a[k:] must not share any value. Delete as few
# elements as possible.
# Idea: count every value in the first part and in the rest. For a value
# that occurs on BOTH sides, one side has to lose ALL of its copies, so delete
# the copies on the side where there are fewer. Add that up over the shared values.
def min_deletions(a, k):
    left = count_values(a[:k])
    right = count_values(a[k:])
    total = 0
    for v in left:
        if v in right:
            if left[v] < right[v]:
                total += left[v]
            else:
                total += right[v]
    return total
```

```
# which indices to delete (either side may lose elements)
def deletions_plan(a, k):
    left = count_values(a[:k])
    right = count_values(a[k:])
    indices = []
    for v in left:
        if v in right:
            if left[v] <= right[v]:
                side = range(k)                         # the first part has fewer copies: delete those
            else:
                side = range(k, len(a))
            for i in side:
                if a[i] == v:
                    indices.append(i)
    indices.sort()
    return indices


# ---- main question: tests ----
def test_main_question():
    assert min_deletions([1, 2, 2, 3, 2, 1], 3) == 2           # first part [1, 2, 2], rest [3, 2, 1]: 1 and
2 are shared
    assert min_deletions([1, 2, 3], 1) == 0
    assert min_deletions([], 0) == 0
    a = [1, 2, 2, 3, 2, 1]
    plan = deletions_plan(a, 3)
    assert len(plan) == min_deletions(a, 3)
    first = set()
    rest = set()
    for i in range(len(a)):
        if i not in plan:
            if i < 3:
                first.add(a[i])
            else:
                rest.add(a[i])
    assert not (first & rest)                                  # nothing is shared any more


# ========================================================
# FOLLOW-UP 1: Only elements of the first part may be deleted
# ========================================================
# ---- follow-up 1: function ----
# Answer: then every shared value must go from the first part entirely, so
# the cost is the number of first-part elements whose value occurs in the
# rest (one set lookup per element).
def min_deletions_first_only(a, k):
    rest = set(a[k:])
    count = 0
    for x in a[:k]:
        if x in rest:
            count += 1
    return count
```

```
# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert min_deletions_first_only([1, 2, 2, 3, 2, 1], 3) == 3


# ========================================================
# FOLLOW-UP 2: Elements have weights (deleting some is more costly)
# ========================================================
# ---- follow-up 2: function ----
# Answer: use the sum of the weights of each value on each side instead of
```

```python
# counts; the choice for each value is still independent, so take the cheaper
# side. Still O(n).
def min_deletions_weighted(a, weights, k):
    left = {}
    right = {}
    for i in range(len(a)):
        if i < k:
            left[a[i]] = left.get(a[i], 0) + weights[i]
        else:
            right[a[i]] = right.get(a[i], 0) + weights[i]
    total = 0
    for v in left:
        if v in right:
            if left[v] < right[v]:
                total += left[v]
            else:
                total += right[v]
    return total


# ---- follow-up 2: tests ----
def test_follow_up_2():
    a = [1, 2, 2, 3, 2, 1]
    weights = [1, 5, 5, 1, 1, 1]                    # the first two 2's are heavy: delete the other side's co
py of 2
    assert min_deletions_weighted(a, weights, 3) == 1 + 1
    assert min_deletions_weighted(a, [1] * 6, 3) == min_deletions(a, 3)


# ============================================================
# FOLLOW-UP 3: The statement may differ from this reading
# ============================================================
# Answer: ask: must the two parts have no value in common, or only no
# equal values at the same position? Which parts may lose elements? State
# your reading before you code.
```

```python
# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

112 Smallest Missing Positive Integer in Linear Time and Constant Extra Space

HARD

Template fit: use the array itself as the set: swap each value x into slot x - 1, then the first slot with the wrong value is the answer

QUESTION

You are given an unsorted list of integers **nums**, which may contain negative numbers, zeros, duplicates, and values far larger than the length of the list. Return the smallest positive integer, that is the smallest integer that is at least **1**, that does not appear in **nums**. In the interview, a first solution that ran in linear time but used extra memory proportional to the input was accepted as a starting point, and the interviewer then asked for the same result using only constant extra space. Aim for that tighter target.

Function Signature

QUESTION

```
def first_missing_positive(nums: list[int]) -> int:
```

Rules

- Target complexity: **`O(n)`** time and **`O(1)`** auxiliary space, where **`n = len(nums)`**. The list may be rearranged or overwritten in place; its contents after the call do not matter.
- Return only the missing value.

Constraints

- **`1 <= len(nums) <= 10^5`**
- **`-2147483648 <= nums[i] <= 2147483647`**
- The answer is a positive integer that fits in a signed 32-bit integer.

Examples

Example 1

- Input: **`nums = [4, -2, 1, 2]`**
- Output: **`3`**
- Explanation: **`1`** and **`2`** are present and **`3`** is not.

Example 2

- Input: **`nums = [5, 6, 100, -3]`**
- Output: **`1`**
- Explanation: **`1`** does not appear, so it is the answer, however large the other values are.

Example 3

- Input: **`nums = [1, 2, 2, 3]`**
- Output: **`4`**
- Explanation: **`1`**, **`2`** and **`3`** all appear (**`2`** twice), so the smallest missing positive integer is **`4`**.

FULL CODE | main question first, then every follow-up, tests at the end

```
# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ========================================================

# ========================================================
# MAIN QUESTION
# ========================================================
# ---- main question: main functions ----
# Find the smallest positive integer that is missing from the array.
# Idea (cyclic placement): the answer is between 1 and n + 1, so only
# values in 1..n matter. Put each value v at index v - 1 by swapping: for
# each position, while the value there is in 1..n and not yet at its home
# index, swap it home. Every swap puts one value in its final place, so there
# are at most n swaps in total: O(n) time, O(1) extra space. Then the first
# index that does not hold index + 1 gives the answer.
def first_missing_positive(nums):
    a = list(nums)                                  # work on a copy: the input stays untouched
    n = len(a)
    for i in range(n):
        while 1 <= a[i] <= n and a[a[i] - 1] != a[i]:
            home = a[i] - 1
            a[i], a[home] = a[home], a[i]
    for i in range(n):
        if a[i] != i + 1:
```

```
            return i + 1
    return n + 1


# the easy version (say it first): O(n) extra memory
def first_missing_set(nums):
    seen = set(nums)
    v = 1
    while v in seen:
        v += 1
    return v


# ---- main question: tests ----
CASES = [([3, 4, -1, 1], 2), ([1, 2, 0], 3), ([7, 8, 9, 11, 12], 1), ([], 1), ([1, 1, 1], 2), ([2, 1], 3)]


def test_main_question():
```

```
    for nums, expected in CASES:
        assert first_missing_positive(nums) == expected
        assert first_missing_set(nums) == expected
    original = [3, 4, -1, 1]
    first_missing_positive(original)
    assert original == [3, 4, -1, 1]              # the input list is left untouched


# ========================================================
# FOLLOW-UP 1: Do it without swapping (sign marking)
# ========================================================
# ---- follow-up 1: function ----
# Answer: replace the values outside 1..n by n + 1 (they cannot matter).
# Then for every value v in 1..n make the entry at index v - 1 NEGATIVE as a
# "seen" flag. The first entry that is still positive marks the missing
# number. Same O(n) time and O(1) extra space.
def first_missing_marking(nums):
    a = list(nums)
    n = len(a)
    for i in range(n):
        if a[i] <= 0 or a[i] > n:
            a[i] = n + 1
    for i in range(n):
        v = abs(a[i])
        if v <= n:
            a[v - 1] = -abs(a[v - 1])
    for i in range(n):
        if a[i] > 0:
            return i + 1
    return n + 1


# ---- follow-up 1: tests ----
def test_follow_up_1():
    for nums, expected in CASES:
        assert first_missing_marking(nums) == expected


# ========================================================
# FOLLOW-UP 2: The array must not be modified
# ========================================================
# Answer: then you cannot swap or mark in the array itself. Copy it (O(n)
# memory, as above), or use the set version, or binary-search the answer when
# you may count: for a guess g, count how many values are in 1..g.
```

```
# ========================================================
```

```python
# FOLLOW-UP 3: Smallest missing non-negative integer, or the k-th missing positive
# ========================================================
# ---- follow-up 3: function ----
# Answer: for the smallest missing non-negative number start the search at 0
# instead of 1. For the k-th missing positive in a SORTED array: the number of
# missing values before index i is a[i] - (i + 1), which only grows, so binary
# search the first index where it reaches k.
def kth_missing_positive(sorted_nums, k):
    low = 0
    high = len(sorted_nums)
    while low < high:
        middle = (low + high) // 2
        if sorted_nums[middle] - (middle + 1) < k:
            low = middle + 1
        else:
            high = middle
    return low + k


# ---- follow-up 3: tests ----
def test_follow_up_3():
    assert kth_missing_positive([2, 3, 4, 7, 11], 5) == 9      # the missing ones: 1, 5, 6, 8, 9
    assert kth_missing_positive([1, 2, 3, 4], 2) == 6


# ========================================================
# RUN EVERYTHING
# ========================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

bisect on a sorted list: find a position, count a range, keep it sorted

113 Count People in a Postal Code Range

EASY

Template fit: sort the codes once; the answer is the template as written, never below 0

QUESTION

You are given a list of people, each as a tuple (name, postal_code) . Build a structure that can answer many queries of the form: "how many people have a postal code within the inclusive range [low, high] ?" The interviewer explicitly wanted the optimal query implementation — not a linear scan of the people (or a scan over the range and a hash-map lookup for each value). Preprocess once, then answer each query in O(log n) . Implement a class with: a constructor that accepts the list of people, and count_in_range(low, high) returning the number of people whose postal code is in [low, high] inclusive.

Function signature

```
from typing import List, Tuple
class PostalRegistry:
    def __init__(self, people: List[Tuple[str, int]]) -> None:
        """Preprocess the (name, postal_code) pairs for fast range counts."""
        ...
    def count_in_range(self, low: int, high: int) -> int:
        """Return how many people have postal_code in [low, high] inclusive."""
        ...
```

Follow-up (as reported): Also return the list of matching names. Hint: sort by (postal_code, name) and slice between bisect_left(low) and bisect_right(high) .

EXAMPLES

Example 1

Input	`people = [("Alice", 90210), ("Bob", 10001), ("Cara", 90211), ("Dan", 30301)]` `count_in_range(90000, 99999)`
Output	`2`
Why	`Alice (90210) and Cara (90211) fall in the range.`

Example 2

Input	`same people; count_in_range(10001, 30301)`
Output	`2`
Why	`Bob (10001) and Dan (30301); range is inclusive on both ends.`

Example 3

Input	`same people; count_in_range(50000, 60000)`
Output	`0`
Why	`No postal codes in that range.`

CONSTRAINTS

- 0 <= number of people <= 10^5 ; the same postal code may appear for multiple people (duplicates allowed).
- Postal codes are integers, 0 <= postal_code <= 10^9 .
- 0 <= low <= high <= 10^9 .
- Each count_in_range call must run in O(log n) after an O(n log n) preprocessing step.

FULL CODE | main question first, then every follow-up, tests at the end

```python
# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ========================================================

from bisect import bisect_left, bisect_right


# ========================================================
# MAIN QUESTION
# ========================================================
# ---- main question: main function ----
# Idea: keep only the postal codes, sorted. The codes in the range
# [low, high] form one block of the sorted list: bisect_left finds where
# it starts, bisect_right where it ends. The count is the block's size.
# people is a list of (name, postal code).
class PostalRegistry:
    def __init__(self, people):
        self.codes = []
        for name, code in people:
            self.codes.append(code)
        self.codes.sort()

    def count_in_range(self, low, high):
        count = bisect_right(self.codes, high) - bisect_left(self.codes, low)
        if count < 0:
            return 0                        # low is bigger than high
        return count


# ---- main question: tests ----
PEOPLE = [("Alice", 90210), ("Bob", 10001), ("Cara", 90211), ("Dan", 30301)]


def test_main_question():
    registry = PostalRegistry(PEOPLE)
    assert registry.count_in_range(90000, 99999) == 2
    assert registry.count_in_range(10001, 30301) == 2
    assert registry.count_in_range(50000, 60000) == 0
    assert registry.count_in_range(99999, 90000) == 0


# ========================================================
```

```python
# FOLLOW-UP 1: Also return the names of the people in the range
# ========================================================
# ---- follow-up 1: function ----
# Answer: keep (code, name) pairs sorted. Find the two ends with bisect
# (from (low, "") up to just before (high + 1, "")) and slice: the names
# come out ordered by code, then by name. O(log n + size of the answer).
class NamedPostalRegistry:
    def __init__(self, people):
        self.items = []
        for name, code in people:
            self.items.append((code, name))
        self.items.sort()

    def names_in_range(self, low, high):
        start = bisect_left(self.items, (low, ""))
        end = bisect_left(self.items, (high + 1, ""))
        names = []
        for code, name in self.items[start:end]:
            names.append(name)
        return names
```

```python
# ---- follow-up 1: tests ----
def test_follow_up_1():
    registry = NamedPostalRegistry(PEOPLE)
    assert registry.names_in_range(10001, 30301) == ["Bob", "Dan"]
    assert registry.names_in_range(90000, 99999) == ["Alice", "Cara"]
    assert registry.names_in_range(1, 2) == []


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

114 Sliding-Window Mean After Removing the Largest K Values

MEDIUM

Template fit: keep the window in a sorted list; insort the new value, remove the old one with bisect_left, and track the sum of the top-K region

QUESTION

For every window of a fixed size over the array, remove the k largest values of the window (repeated values count separately) and compute the mean of the rest. Return the means in window order as reduced fractions written 'numerator/denominator'. Update the windows incrementally instead of sorting each one again.

Function signature

def trimmed_window_means(nums: list, window: int, k: int) -> list

EXAMPLES

Example 1	
Input	[1, 2, 3, 4, 5], 3, 1
Output	['3/2', '5/2', '7/2']
Why	Window [1,2,3] without its largest value 3 averages (1+2)/2 = 3/2; then 5/2 and 7/2.

Example 2	
Input	[2, 4, 6, 8], 2, 0
Output	['3/1', '5/1', '7/1']
Why	With k = 0 nothing is removed: plain window means.

FULL CODE | main question first, then every follow-up, tests at the end

```python
# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

from bisect import bisect_left, bisect_right
from fractions import Fraction
import random
```

```
# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
def as_text(fraction):
    # a reduced fraction as text: Fraction(5, 2) -> "5/2"
    return str(fraction.numerator) + "/" + str(fraction.denominator)


class Window:
    # the values of the current window in a SORTED list, with three sums kept
    # up to date while values come and go: the whole sum (total), the sum of
    # the k largest values (top) and the sum of the k smallest values (bottom)
    def __init__(self, k):
        self.k = k
        self.values = []
        self.total = 0
        self.top = 0
        self.bottom = 0

    def insert(self, x):
        k = self.k
        values = self.values
        old_length = len(values)
        place = bisect_right(values, x)
        values.insert(place, x)
        self.total += x
        # the top region is the last k places of the new list
        if old_length < k:
            self.top += x                                   # the list is still short: everything is "top"
        elif place >= old_length + 1 - k:
            self.top += x - values[old_length - k]          # x entered it, the value just below it left
        # the bottom region is the first k places
```

```
        if old_length < k:
            self.bottom += x
        elif place < k:
            self.bottom += x - values[k]                    # x entered it, the value now at index k left

    def remove(self, x):
        k = self.k
        values = self.values
        old_length = len(values)
        place = bisect_left(values, x)
        self.total -= x
        if old_length <= k:
            self.top -= x
            self.bottom -= x
        else:
            if place >= old_length - k:
                self.top += values[old_length - k - 1] - x      # the value just below the region joins it
            if place < k:
                self.bottom += values[k] - x                    # the value at index k joins the region
        del values[place]


# ---- main question: main function ----
# Idea: for every window of the given size, drop its k largest values
# and take the mean of the rest. Keep the window in a sorted list and keep
# the sum of its k largest values up to date while one value enters and
# one leaves, so no window is sorted again. With both=True the k smallest
# are dropped as well (a trimmed mean, follow-up 4). Each mean is
# returned as an exact reduced fraction in text, like "5/2".
def trimmed_window_means(nums, window, k, both=False):
    w = Window(k)
    means = []
```

```
    for i in range(len(nums)):
        w.insert(nums[i])
        if i >= window:
            w.remove(nums[i - window])
        if i >= window - 1:
            kept = window - k
            dropped = w.top
            if both:
                kept = window - 2 * k
                dropped = w.top + w.bottom
            means.append(as_text(Fraction(w.total - dropped, kept)))
    return means
```

```
# ---- main question: tests ----
def slow_means(nums, window, k, both=False):
    # sort every window again (obviously right, slow) to check the fast version
    means = []
    for i in range(window - 1, len(nums)):
        piece = sorted(nums[i - window + 1:i + 1])
        if both:
            kept = piece[k:window - k]
        else:
            kept = piece[:window - k]
        total = 0
        for v in kept:
            total += v
        means.append(as_text(Fraction(total, len(kept))))
    return means


def test_main_question():
    assert trimmed_window_means([5, 1, 4], 3, 1) == ["5/2"]
    assert trimmed_window_means([3, 3, 3, 3], 3, 1) == ["3/1", "3/1"]       # duplicates at the boundary
    random.seed(33)
    for round_number in range(300):
        nums = []
        for i in range(random.randint(3, 14)):
            nums.append(random.randint(-6, 6))
        window = random.randint(2, len(nums))
        k = random.randint(0, window - 1)
        assert trimmed_window_means(nums, window, k) == slow_means(nums, window, k)


# ======================================================
# FOLLOW-UP 1: Make the update O(log w) in the worst case
# ======================================================
# Answer: inserting into a Python list shifts items (O(w)). For a true
# O(log w) update use a balanced tree or a Fenwick tree over the value
# ranks that also stores sums, so "sum of the k largest" is one query.


# ======================================================
# FOLLOW-UP 3: Duplicates on the boundary
# ======================================================
# Answer: remove exactly k COPIES: it does not matter which equal value is
# dropped because they are equal. The sorted list handles this naturally
# (bisect_right to insert, bisect_left to remove).
```

```


# ======================================================
# FOLLOW-UP 2: Remove the smallest K instead, or both ends (trimmed mean)
# ======================================================
```

```python
# ============================================================
# FOLLOW-UP 4: Trim the smallest K as well (a trimmed mean)
# ============================================================
# ---- follow-up 4: tests ----
# Answer: the Window class also keeps the sum of the K smallest values (the
# mirror image of the top sum), so the trimmed mean is
# (total - top - bottom) / (window - 2K): trimmed_window_means(..., both=True).
def test_follow_up_4():
    nums = [5, 1, 4, 2, 8, 3, 3, 9]
    assert trimmed_window_means(nums, 5, 1, True) == slow_means(nums, 5, 1, True)
    random.seed(34)
    for round_number in range(300):
        nums = []
        for i in range(random.randint(3, 14)):
            nums.append(random.randint(-6, 6))
        window = random.randint(3, len(nums)) if len(nums) >= 3 else 3
        if window > len(nums):
            continue
        k = random.randint(0, (window - 1) // 2)
        assert trimmed_window_means(nums, window, k, True) == slow_means(nums, window, k, True)


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_4()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

115 Russian Doll Envelopes

MEDIUM

Template fit: sort by width ascending and height descending, then the longest increasing run of heights using a tails list and bisect_left

QUESTION

LeetCode 354. Maximum chain of envelopes that strictly fit inside each other. Sort + LIS reduction.

Input: list of (width, height) envelopes.
Output: longest chain where each envelope strictly fits in the next.
Trick: sort by width ascending, height descending (when widths tie); run LIS (Longest Increasing Subsequence) on heights.
LIS with patience-sort gives O(n log n).

EXAMPLES

```
[[5,4],[6,4],[6,7],[2,3]] -> 3.
```

FULL CODE | main question first, then every follow-up, tests at the end

```python
# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

from bisect import bisect_left
```

```python
# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
def width_then_taller_first(envelope):
    # sort key: width ascending; for EQUAL widths the taller one first
    return (envelope[0], -envelope[1])


# ---- main question: main function ----
# Idea: sort by (width ascending, height descending), then find the
# longest strictly increasing run of heights with a binary search.
# tails[k] is the smallest possible last height of a chain of k + 1
# envelopes; each new height either extends the longest chain or lowers
# one of the tails.
def max_envelopes(envelopes):
    ordered = sorted(envelopes, key=width_then_taller_first)
    tails = []
    for width, height in ordered:
        i = bisect_left(tails, height)
        if i == len(tails):
            tails.append(height)
        else:
            tails[i] = height
    return len(tails)


# ---- main question: tests ----
def test_main_question():
    assert max_envelopes([[5, 4], [6, 4], [6, 7], [2, 3]]) == 3
    assert max_envelopes([]) == 0
    assert max_envelopes([[1, 1], [1, 2], [1, 3]]) == 1             # equal widths cannot nest, however the h
eights go
    assert max_envelopes([[4, 5], [4, 6], [6, 7], [2, 3], [1, 1]]) == 4
```

```python
# ============================================================
# FOLLOW-UP 1: Why sort heights descending when widths tie?
# ============================================================
# Answer: envelopes with the SAME width cannot nest. Putting the taller
# one first means that, among equal widths, heights go DOWN, and a
# strictly increasing run of heights can use at most one of them. Sorted
# ascending, the run could wrongly pick several of the same width.


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

116 Count Sorted Strings With a Given Prefix

MEDIUM

Template fit: lo = bisect_left(words, prefix); hi = bisect_left(words, prefix + a very large character); the answer is hi - lo

QUESTION

Given a lexicographically sorted array of strings words and a query string prefix , return the number of strings in words that start with prefix . Because words is sorted, all matching strings occupy a contiguous range; find that range with binary search in O(log n * L) time (where L is the prefix length), not by scanning.

Function signature

def count_with_prefix(words: list[str], prefix: str) -> int:

    ...

Follow-up (as reported): Many repeated queries: cache counts in a hash map keyed by prefix. All queries unique: build a trie of all words storing subtree sizes, so each query is O(L) .

EXAMPLES

Example 1	
Input	`words = ["app", "apple", "application", "banana", "bat"], prefix = "app"`
Output	`3`
Why	`"app", "apple", "application" all start with "app".`

Example 2	
Input	`words = ["app", "apple", "application", "banana", "bat"], prefix = "ba"`
Output	`2`
Why	`"banana" and "bat".`

Example 3	
Input	`words = ["app", "apple", "application", "banana", "bat"], prefix = "cat"`
Output	`0`
Why	`No string starts with "cat".`

CONSTRAINTS

- 0 <= len(words) <= 10**6 , sorted in non-decreasing lexicographic order.
- 0 <= len(prefix) <= 100 . An empty prefix matches every string.

FULL CODE | main question first, then every follow-up, tests at the end

```
# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

from bisect import bisect_left


# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: main function ----
# Idea: in a sorted list the words that start with the prefix form ONE
# block. It starts at the first word >= prefix and ends just before the
# first word >= prefix followed by the biggest possible character. Two
# binary searches find both ends; the answer is the size of the block.
def count_with_prefix(words, prefix):
    start = bisect_left(words, prefix)
```

```
    end = bisect_left(words, prefix + "\uffff")
    return end - start


# ---- main question: tests ----
def test_main_question():
    words = ["app", "apple", "application", "banana", "bat"]
    assert count_with_prefix(words, "app") == 3
    assert count_with_prefix(words, "ba") == 2
    assert count_with_prefix(words, "cat") == 0
    assert count_with_prefix(words, "") == 5


# ========================================================
# FOLLOW-UP 1: Thousands of different queries on the same list. Can each be O(L)?
# ========================================================
# ---- follow-up 1: function ----
# Answer: build a trie once in which every node stores how many words pass
# through it. A query walks down the prefix (L steps) and returns the
# count stored in the node it reaches (0 if the path breaks).
class PrefixCounter:
    def __init__(self, words):
        self.total = len(words)
        self.root = {"count": 0, "children": {}}
        for word in words:
```

```
            node = self.root
            for ch in word:
                if ch not in node["children"]:
                    node["children"][ch] = {"count": 0, "children": {}}
                node = node["children"][ch]
                node["count"] += 1             # one more word passes through this node

    def count(self, prefix):
        node = self.root
        for ch in prefix:
            if ch not in node["children"]:
                return 0
            node = node["children"][ch]
        if prefix == "":
            return self.total
        return node["count"]


# ---- follow-up 1: tests ----
def test_follow_up_1():
    words = ["app", "apple", "application", "banana", "bat"]
    counter = PrefixCounter(words)
    for prefix in ("app", "ba", "cat", "", "apple", "applesauce"):
        assert counter.count(prefix) == count_with_prefix(words, prefix)


# ========================================================
# RUN EVERYTHING
# ========================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

117 Streaming Insert + Find K-th Largest

MEDIUM

Template fit: insort every new value; the k-th largest is sorted_list[-1 - k]

QUESTION

Design a structure with insert(x), which adds a number, and findLargest(k), which returns the k-th largest number inserted so far (k = 0 is the largest), counting repeated values separately, or None when fewer than k + 1 numbers exist.

Function signature

class StreamKth:
    def insert(self, x: float) -> None
    def findLargest(self, k: int) -> float | None

EXAMPLES

Example 1

Input	**s = StreamKth()** **s.insert(3)** **s.insert(3)** **s.insert(2)** **s.findLargest(0)** **s.findLargest(1)** **s.findLargest(2)** **s.findLargest(3)**
Output	**s.findLargest(0) -> 3** **s.findLargest(1) -> 3** **s.findLargest(2) -> 2** **s.findLargest(3) -> null**
Why	**Sorted descending the numbers are 3, 3, 2, so the 0th, 1st and 2nd largest are 3, 3 and 2, and there is no 3rd.**

FULL CODE | main question first, then every follow-up, tests at the end

```
# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

import heapq
from bisect import bisect_left, insort


# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: main function ----
# Idea: keep every value in an ascending list. insort puts a new value
# in its place with a binary search (O(log n) to find the spot, O(n) to
# shift the others). The k-th largest (k = 0 is the largest, as in the
# examples) is then simply a[-1 - k].
class StreamKth:
    def __init__(self):
        self.a = []

    def insert(self, x):
        insort(self.a, x)

    def findLargest(self, k):
        if 0 <= k < len(self.a):
            return self.a[-1 - k]
        return None
```

```
    # follow-ups 1 and 3: values can also be deleted. A sorted list can
    # delete: find a copy of x with a binary search and remove it.
    def delete(self, x):
        i = bisect_left(self.a, x)
        if i < len(self.a) and self.a[i] == x:
            del self.a[i]
            return True
        return False


# ---- main question: tests ----
def test_main_question():
    s = StreamKth()
    for x in (3, 3, 2):
```

```
        s.insert(x)
    assert [s.findLargest(k) for k in range(3)] == [3, 3, 2]
    assert s.findLargest(3) is None
    assert s.delete(3) and not s.delete(9)             # deletes work too
    assert [s.findLargest(k) for k in range(2)] == [3, 2]


# =======================================================
# FOLLOW-UP 1: What if values can also be deleted?
# =======================================================


# =======================================================
# FOLLOW-UP 3: Values can also be deleted
# =======================================================
# Answer: a heap cannot delete an arbitrary value efficiently (finding it
# costs O(k)), but the sorted list can: StreamKth.delete above.


# =======================================================
# FOLLOW-UP 2: What is the trade-off between the two designs?
# =======================================================
# ---- follow-up 2: function ----
# Answer: the heap is the smallest and fastest when k is FIXED and small
# (O(log k) per insert, O(k) memory) but it cannot change k or delete.
# The sorted list answers any k and is simplest, at O(n) memory and a
# linear shift on every insert.
class FixedKth:
    def __init__(self, k):
        self.k = k
        self.heap = []                      # a min-heap holding the k largest values

    def insert(self, x):
        heapq.heappush(self.heap, x)
        if len(self.heap) > self.k:
            heapq.heappop(self.heap)        # drop the smallest: keep only the k largest

    def kth_largest(self):
        if len(self.heap) < self.k:
            return None
        return self.heap[0]                 # the smallest of the k largest


# ---- follow-up 2: tests ----
def test_follow_up_2():
```

```
    f = FixedKth(2)
    assert f.kth_largest() is None
    for x in (5, 1, 9, 7):
        f.insert(x)
```

```
    assert f.kth_largest() == 7            # the two largest are 9 and 7


# ======================================================
# RUN EVERYTHING
# ======================================================
def run_all_tests():
    test_main_question()
    test_follow_up_2()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

118 Given positions of N kids and M ice cream sellers on a 1D line, find the smallest distance E such that every kid lies within the range [P - E, P + E] of at least one seller. MEDIUM

Template fit: for each kid bisect into the sorted sellers and take the nearer of the two neighbours; the answer is the largest of those distances

QUESTION

Kids and ice cream sellers stand at positions on a line. Find the smallest distance E such that every kid is within E of at least one seller (the kid at position K is served by a seller at P when P - E <= K <= P + E).

Function signature

def smallest_e(kids: list, sellers: list) -> int

EXAMPLES

Example 1	
Input	[1, 5, 9], [2, 8]
Output	3
Why	The kid at 5 is 3 away from both sellers (at 2 and 8); the others are 1 away.

Example 2	
Input	[4], [4]
Output	0
Why	A kid standing on a seller needs distance 0.

FULL CODE | main question first, then every follow-up, tests at the end

```
# ======================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ======================================================

from bisect import bisect_left


# ======================================================
# MAIN QUESTION
# ======================================================
# ---- main question: helper functions ----
def distance_to_nearest(sorted_sellers, kid):
    # a binary search finds where the kid would stand among the sellers; the
    # nearest seller is the one right at or after that spot, or the one before it
```

```
    i = bisect_left(sorted_sellers, kid)
    nearest = float("inf")
    if i < len(sorted_sellers):
        nearest = sorted_sellers[i] - kid
    if i > 0 and kid - sorted_sellers[i - 1] < nearest:
        nearest = kid - sorted_sellers[i - 1]
    return nearest


# ---- main question: main function ----
# Idea: every kid should go to the nearest seller, so E is the WORST such
# distance over all kids. Sort the sellers once and binary search each kid.
def smallest_e(kids, sellers):
    sellers = sorted(sellers)
    worst = 0
    for kid in kids:
        d = distance_to_nearest(sellers, kid)
        if d > worst:
            worst = d
    return worst


# ---- main question: tests ----
def test_main_question():
    assert smallest_e([1, 5, 9], [2, 8]) == 3
    assert smallest_e([4], [4]) == 0
```

```
# ============================================================
# FOLLOW-UP 1: Both lists are already sorted, or you want to avoid the binary search
# ============================================================
# ---- follow-up 1: function ----
# Answer: walk the kids in order with ONE pointer on the sellers that only
# moves forward while the next seller is at least as close. O(N + M)
# (after sorting, if the lists were not sorted).
def smallest_e_sorted(kids, sellers):
    kids = sorted(kids)
    sellers = sorted(sellers)
    worst = 0
    j = 0
    for kid in kids:
        while j + 1 < len(sellers):
            if abs(sellers[j + 1] - kid) > abs(sellers[j] - kid):
                break                       # the next seller is farther: stay
            j += 1
        if abs(sellers[j] - kid) > worst:
            worst = abs(sellers[j] - kid)
    return worst


# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert smallest_e_sorted([1, 5, 9], [2, 8]) == 3
    assert smallest_e_sorted([4], [4]) == 0


# ============================================================
# FOLLOW-UP 2: Each seller can serve at most C kids
# ============================================================
# ---- follow-up 2: helper functions ----
def feasible_capacity(kids, sellers, e, cap):
    # can every kid be served when a seller takes at most `cap` kids and
    # only kids within e? Going left to right, each seller takes the
    # leftmost kids that are still unserved and in range.
    i = 0
    for s in sellers:
```

```
        if i < len(kids) and kids[i] < s - e:
            return False                    # this kid is too far left for every later seller
        taken = 0
        while i < len(kids) and taken < cap:
            if kids[i] > s + e:
                break
            i += 1
            taken += 1
    return i == len(kids)


# ---- follow-up 2: function ----
# Answer: now it is an assignment problem. Binary search the radius E and
# test it with the greedy left-to-right matching above.
def smallest_e_capacity(kids, sellers, cap):
    kids = sorted(kids)
    sellers = sorted(sellers)
    if len(kids) > cap * len(sellers):
        return None                         # not enough room at all
    low = 0
    high = max(kids[-1], sellers[-1]) - min(kids[0], sellers[0])
    while low < high:
        mid = (low + high) // 2
        if feasible_capacity(kids, sellers, mid, cap):
            high = mid
        else:
            low = mid + 1
    return low


# ---- follow-up 2: tests ----
def test_follow_up_2():
    assert smallest_e_capacity([1, 2, 3], [2], 1) is None       # three kids, room for one
    assert smallest_e_capacity([1, 2, 3], [1, 3, 9], 1) == 6
    assert smallest_e_capacity([1, 2], [5, 6], 1) == 4


# ========================================================
# FOLLOW-UP 3: Sellers can be placed anywhere (minimise the radius with S sellers)
# ========================================================
# ---- follow-up 3: helper functions ----
def sellers_needed(kids, e):
    # greedy: the leftmost kid not covered yet, at x, gets a seller at x + e,
    # who covers every kid up to x + 2e
    count = 0
    reach = None
    for kid in sorted(kids):
        if reach is None or kid > reach:
            count += 1
            reach = kid + 2 * e
    return count
```

```
# ---- follow-up 3: function ----
# Answer: this is the 1D k-center problem. Binary search the radius and
# test it with the greedy placement above. O(N log range).
def best_radius(kids, how_many):
    kids = sorted(kids)
    low = 0
    high = kids[-1] - kids[0]
    while low < high:
        mid = (low + high) // 2
        if sellers_needed(kids, mid) <= how_many:
            high = mid
        else:
```

```
            low = mid + 1
    return low


# ---- follow-up 3: tests ----
def test_follow_up_3():
    assert sellers_needed([1, 2, 3, 20], 1) == 2
    assert best_radius([1, 2, 3, 20], 2) == 1
    assert best_radius([5], 1) == 0


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

119 Find Maximum Envelope Nesting

HARD

Template fit: exactly the same sort and tails list

QUESTION

Envelopes are given as (width, height). One envelope fits inside another only when both its width and its height are strictly smaller. Return the largest number of envelopes that can be nested one inside the next (Russian dolls).

Function signature

def max_envelopes(envelopes: list) -> int

EXAMPLES

Example 1	
Input	`[(5, 4), (6, 4), (6, 7), (2, 3)]`
Output	`3`
Why	`(2, 3) fits in (5, 4), which fits in (6, 7): 3 nested.`

Example 2	
Input	`[(1, 1), (1, 1), (1, 1)]`
Output	`1`
Why	`Equal envelopes cannot go inside each other.`

FULL CODE | main question first, then every follow-up, tests at the end

```
# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

from bisect import bisect_left, bisect_right
```

```python
# ========================================================
# MAIN QUESTION
# ========================================================
# ---- main question: helper functions ----
def width_then_taller_first(envelope):
    # sort key: width ascending; for EQUAL widths the taller one first
    return (envelope[0], -envelope[1])


# ---- main question: main function ----
# Idea: sort by width ascending and, for equal widths, height descending.
# Now a strictly increasing run of heights is a valid nesting chain: two
# envelopes of the same width are in decreasing height order, so they can
# never both be in an increasing run. Find the longest strictly increasing
# run of heights (LIS) with a binary search in O(n log n): tails[k] is the
# smallest possible last height of a run of length k + 1.
def max_envelopes(envelopes):
    ordered = sorted(envelopes, key=width_then_taller_first)
    tails = []
    for width, height in ordered:
        i = bisect_left(tails, height)
        if i == len(tails):
            tails.append(height)           # longer than any run so far
        else:
            tails[i] = height              # a smaller ending for runs of length i + 1
    return len(tails)


# ---- main question: tests ----
def test_main_question():
    assert max_envelopes([[5, 4], [6, 4], [6, 7], [2, 3]]) == 3
    assert max_envelopes([]) == 0
    assert max_envelopes([[3, 3], [3, 3], [4, 4]]) == 2        # equal sizes do not nest (strict)
```

```python
# ========================================================
# FOLLOW-UP 1: Return the actual chain of envelopes
# ========================================================
# ---- follow-up 1: function ----
# Answer: remember, for each element, which element it extended (a parent
# link), and rebuild the chain from the last tail of the longest run.
# The sorting and the binary search stay the same.
def envelope_chain(envelopes):
    ordered = sorted(envelopes, key=width_then_taller_first)
    tails = []
    tail_index = []                        # the index in `ordered` of each tail
    previous = [-1] * len(ordered)
    for i in range(len(ordered)):
        height = ordered[i][1]
        k = bisect_left(tails, height)
        if k == len(tails):
            tails.append(height)
            tail_index.append(i)
        else:
            tails[k] = height
            tail_index[k] = i
        if k > 0:
            previous[i] = tail_index[k - 1]        # it extends the run that ended one shorter
    chain = []
    if tail_index:
        i = tail_index[-1]
        while i != -1:
            chain.append(ordered[i])
            i = previous[i]
    chain.reverse()
    return chain
```

```
# ---- follow-up 1: tests ----
def test_follow_up_1():
    chain = envelope_chain([[5, 4], [6, 4], [6, 7], [2, 3]])
    assert len(chain) == 3
    for i in range(len(chain) - 1):                    # every envelope fits strictly inside the next
        assert chain[i][0] < chain[i + 1][0] and chain[i][1] < chain[i + 1][1]
    assert envelope_chain([]) == []


# ============================================================
# FOLLOW-UP 2: Three dimensions (boxes)
# ============================================================
```

```
# ---- follow-up 2: function ----
# Answer: sorting plus LIS no longer reduces the problem. Use an O(n^2)
# DP (best[i] = longest chain ending at box i), or CDQ divide and conquer
# with a Fenwick tree for O(n log^2 n).
def max_boxes_3d(boxes):
    ordered = sorted(boxes)
    best = [1] * len(ordered)
    for i in range(len(ordered)):
        for j in range(i):
            a = ordered[j]
            b = ordered[i]
            if a[0] < b[0] and a[1] < b[1] and a[2] < b[2]:
                if best[j] + 1 > best[i]:
                    best[i] = best[j] + 1
    answer = 0
    for length in best:
        if length > answer:
            answer = length
    return answer


# ---- follow-up 2: tests ----
def test_follow_up_2():
    assert max_boxes_3d([(1, 1, 1), (2, 2, 2), (3, 1, 4), (3, 3, 3)]) == 3
    assert max_boxes_3d([]) == 0


# ============================================================
# FOLLOW-UP 3: Non-strict nesting (equal sizes allowed)
# ============================================================
# ---- follow-up 3: function ----
# Answer: sort by width AND height ascending, and use bisect_right (a
# longest NON-decreasing run) instead of bisect_left.
def max_nested_non_strict(envelopes):
    tails = []
    for width, height in sorted(envelopes):
        i = bisect_right(tails, height)
        if i == len(tails):
            tails.append(height)
        else:
            tails[i] = height
    return len(tails)


# ---- follow-up 3: tests ----
```

```
def test_follow_up_3():
    assert max_nested_non_strict([[3, 3], [3, 3], [4, 4]]) == 3


# ============================================================
```

```
# RUN EVERYTHING
# ======================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

120 Count of Smaller Numbers After Self

HARD

Template fit: go from the right, insort each number into a sorted list; the count is bisect_left of the number before inserting (a Fenwick tree is the fast version)

QUESTION

Given an integer array nums, return an array counts where counts[i] is the number of elements to the right of nums[i] that are strictly smaller than nums[i].

Function signature

def count_smaller_after(nums: list) -> list

EXAMPLES

Example 1	
Input	`[5, 2, 6, 1]`
Output	`[2, 1, 1, 0]`
Why	`5 has two smaller numbers to its right (2 and 1), 2 has one (1), 6 has one (1), 1 has none.`

Example 2	
Input	`[-1]`
Output	`[0]`
Why	`A single number has nothing to its right.`

Example 3	
Input	`[-1, -1]`
Output	`[0, 0]`
Why	`Equal numbers are not smaller.`

CONSTRAINTS

- 1 <= nums.length <= 10^5
- -10^4 <= nums[i] <= 10^4

FULL CODE | main question first, then every follow-up, tests at the end

```
# ======================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ======================================================

import random
```

```python
# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
class Fenwick:
    # a Fenwick tree (binary indexed tree): add(i) counts one more value with
    # rank i, prefix(i) tells how many counted values have a rank below i.
    # Both take O(log size).
    def __init__(self, size):
        self.size = size
        self.tree = [0] * (size + 1)

    def add(self, i):
        i += 1
        while i <= self.size:
            self.tree[i] += 1
            i += i & -i                  # jump to the next block that covers i

    def prefix(self, i):
        total = 0
        while i > 0:
            total += self.tree[i]
            i -= i & -i                  # jump to the previous block
        return total


def ranks(nums):
    # coordinate compression: give each distinct value a rank 0, 1, 2, ... by size
    rank = {}
    for i, v in enumerate(sorted(set(nums))):
        rank[v] = i
    return rank


# ---- main question: main function ----
```

```python
# Idea: compress the values to ranks and scan from the RIGHT. The answer
# for x is how many already-seen values have a smaller rank (one prefix
# query on the Fenwick tree); then x itself is added to the tree.
def count_smaller_after(nums):
    rank = ranks(nums)
    tree = Fenwick(len(rank))
    answers = []
    for x in reversed(nums):
        answers.append(tree.prefix(rank[x]))
        tree.add(rank[x])
    answers.reverse()
    return answers


# ---- main question: tests ----
def test_main_question():
    assert count_smaller_after([5, 2, 6, 1]) == [2, 1, 1, 0]
    assert count_smaller_after([-1]) == [0]
    assert count_smaller_after([-1, -1]) == [0, 0]
    assert count_smaller_after([]) == []


# ============================================================
# FOLLOW-UP 1: Do it with merge sort instead of a Fenwick tree
# ============================================================
# ---- follow-up 1: function ----
# Answer: merge-sort the INDEXES by value. While merging, whenever an item
# of the left half is placed, every item of the right half that was
```

```
# already placed is smaller than it and comes later in the array, so add
# that number to its answer. Same O(n log n), and no coordinate compression.
def count_smaller_merge(nums):
    counts = [0] * len(nums)
    order = list(range(len(nums)))

    def sort(lo, hi):
        if hi - lo <= 1:
            return
        mid = (lo + hi) // 2
        sort(lo, mid)
        sort(mid, hi)
        merged = []
        i = lo
        j = mid
        while i < mid or j < hi:
            take_left = False
```

```
            if j >= hi:
                take_left = True
            elif i < mid and nums[order[i]] <= nums[order[j]]:
                take_left = True
            if take_left:
                counts[order[i]] += j - mid        # right items already placed are smaller
                merged.append(order[i])
                i += 1
            else:
                merged.append(order[j])
                j += 1
        order[lo:hi] = merged

    sort(0, len(nums))
    return counts


# ---- follow-up 1: tests ----
def brute_smaller_after(nums):
    answers = []
    for i in range(len(nums)):
        count = 0
        for j in range(i + 1, len(nums)):
            if nums[j] < nums[i]:
                count += 1
        answers.append(count)
    return answers


def test_follow_up_1():
    assert count_smaller_merge([5, 2, 6, 1]) == [2, 1, 1, 0]
    random.seed(31)
    for round_number in range(300):
        nums = []
        for i in range(random.randint(0, 12)):
            nums.append(random.randint(-5, 5))
        assert count_smaller_merge(nums) == brute_smaller_after(nums)
        assert count_smaller_after(nums) == brute_smaller_after(nums)


# ========================================================
# FOLLOW-UP 2: Count larger numbers after self, or smaller numbers before self
# ========================================================
# ---- follow-up 2: functions ----
# Answer: larger after: ask for the part of the tree ABOVE the rank instead
```

```
# of below it. Smaller before: scan from the left instead of the right.
# Both are still O(n log n).
```

```
def count_larger_after(nums):
    rank = ranks(nums)
    tree = Fenwick(len(rank))
    answers = []
    seen = 0
    for x in reversed(nums):
        answers.append(seen - tree.prefix(rank[x] + 1))    # seen values minus those <= x
        tree.add(rank[x])
        seen += 1
    answers.reverse()
    return answers


def count_smaller_before(nums):
    rank = ranks(nums)
    tree = Fenwick(len(rank))
    answers = []
    for x in nums:
        answers.append(tree.prefix(rank[x]))
        tree.add(rank[x])
    return answers


# ---- follow-up 2: tests ----
def test_follow_up_2():
    sample = [5, 2, 6, 1]
    assert count_larger_after(sample) == [1, 1, 0, 0]
    assert count_smaller_before(sample) == [0, 0, 2, 0]
    random.seed(32)
    for round_number in range(300):
        nums = []
        for i in range(random.randint(0, 12)):
            nums.append(random.randint(-5, 5))
        larger = []
        smaller_before = []
        for i in range(len(nums)):
            bigger_after = 0
            for y in nums[i + 1:]:
                if y > nums[i]:
                    bigger_after += 1
            larger.append(bigger_after)
            before = 0
            for y in nums[:i]:
```

```
                if y < nums[i]:
                    before += 1
            smaller_before.append(before)
        assert count_larger_after(nums) == larger
        assert count_smaller_before(nums) == smaller_before


# ======================================================
# FOLLOW-UP 3: Count inversions in an array
# ======================================================
# ---- follow-up 3: function ----
# Answer: an inversion is a pair i < j with nums[i] > nums[j]. The
# answers of count_smaller_after add up to exactly that number.
def inversions(nums):
    total = 0
    for count in count_smaller_merge(nums):
        total += count
    return total


# ---- follow-up 3: tests ----
def test_follow_up_3():
```

```
    assert inversions([2, 4, 1, 3, 5]) == 3
    assert inversions([3, 2, 1]) == 3
    assert inversions([]) == 0


# ========================================================
# RUN EVERYTHING
# ========================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

GROUP 15 OF 43 | 8 QUESTIONS, ONE CODE TEMPLATE | 3 E 5 M

Prefix sums, Tier 1: precompute once, answer ranges and balance points in O(1)

121 Parity Alternating Subarray Query

EASY

Template fit: pre counts neighbouring pairs of equal parity; a range alternates when pre[r] - pre[l] == 0

QUESTION

Given an array nums, answer many queries alternates(l, r): do the elements nums[l], ..., nums[r] (both ends included) alternate in parity (odd, even, odd, ... or even, odd, even, ...)? Preprocess the array once so that every query takes O(1).

Function signature

class ParityQueries:
  def __init__(self, nums: list)
  def alternates(self, l: int, r: int) -> bool

EXAMPLES

Example 1	

Input	`p = ParityQueries([1, 2, 3, 5, 4])`
	`p.alternates(0, 2)`
	`p.alternates(0, 3)`
	`p.alternates(3, 4)`

Output	`p.alternates(0, 2) -> true`
	`p.alternates(0, 3) -> false`
	`p.alternates(3, 4) -> true`

Why	`1, 2, 3 alternates; the neighbours 3 and 5 are both odd, so any range containing both fails; 5, 4 alternates.`

CONSTRAINTS

- 1 <= nums.length <= 10^5
- 1 <= queries.length <= 10^5
- 0 <= l <= r < nums.length

FULL CODE | main question first, then every follow-up, tests at the end

```python
# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

import random


# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: main class ----
# Does the subarray nums[l..r] alternate between even and odd numbers?
# Idea: write down, for the first i neighbouring pairs, how many pairs have
# the SAME parity (bad[i]). The pairs inside nums[l..r] are the pairs
# l, l+1, ..., r-1, so the subarray alternates when bad[r] - bad[l] == 0.
# Building is O(n); every query is O(1).
class ParityQueries:
    def __init__(self, nums):
        self.bad = [0]
```

```
        for i in range(len(nums) - 1):
            same = 0
            if nums[i] % 2 == nums[i + 1] % 2:
                same = 1
            self.bad.append(self.bad[len(self.bad) - 1] + same)

    def alternates(self, l, r):
        return self.bad[r] - self.bad[l] == 0


# ---- main question: tests ----
def test_main_question():
    q = ParityQueries([1, 2, 3, 4, 5])
    assert q.alternates(0, 4) and q.alternates(1, 3)
    q = ParityQueries([1, 3, 2, 4])
    assert not q.alternates(0, 1)
    assert not q.alternates(0, 3)
    assert q.alternates(1, 2)
    assert q.alternates(2, 2)                          # a single element always alternates


# ========================================================
# FOLLOW-UP 1: The array also changes (point updates)
```

```
# ========================================================
# ---- follow-up 1: helper functions ----
class Fenwick:
    # a Fenwick tree: add to one position and ask for the sum of the first i
    # positions, both in O(log n). (i & -i is the lowest set bit of i.)
    def __init__(self, size):
        self.size = size
        self.tree = [0] * (size + 1)

    def add(self, i, delta):
        i += 1
        while i <= self.size:
            self.tree[i] += delta
            i += i & -i

    def prefix(self, i):
        total = 0
        while i > 0:
            total += self.tree[i]
            i -= i & -i
        return total


# ---- follow-up 1: function ----
# Answer: keep the "same parity" flags in a Fenwick tree. Changing one
# number changes at most two flags (the pairs on its left and on its
# right), each an O(log n) update; a query is two prefix sums.
class ParityWithUpdates:
    def __init__(self, nums):
        self.nums = list(nums)
        self.flags = [0] * max(len(nums) - 1, 0)
        self.tree = Fenwick(len(self.flags))
        for i in range(len(self.flags)):
            self.refresh(i)

    def refresh(self, i):
        # recompute the flag of the pair (i, i + 1)
        now = 0
        if self.nums[i] % 2 == self.nums[i + 1] % 2:
            now = 1
        self.tree.add(i, now - self.flags[i])
        self.flags[i] = now
```

```
    def update(self, i, value):
        self.nums[i] = value
```

```
        for pair in (i - 1, i):
            if 0 <= pair < len(self.flags):
                self.refresh(pair)

    def alternates(self, l, r):
        return self.tree.prefix(r) - self.tree.prefix(l) == 0


# ---- follow-up 1: tests ----
def test_follow_up_1():
    random.seed(11)
    for round_number in range(200):
        nums = []
        for i in range(random.randint(1, 10)):
            nums.append(random.randint(0, 9))
        live = ParityWithUpdates(nums)
        for step in range(20):
            if random.random() < 0.4:
                i = random.randrange(len(nums))
                nums[i] = random.randint(0, 9)
                live.update(i, nums[i])
            l = random.randrange(len(nums))
            r = random.randrange(l, len(nums))
            assert live.alternates(l, r) == ParityQueries(nums).alternates(l, r)


# =====================================================
# FOLLOW-UP 2: Find the longest alternating subarray
# =====================================================
# ---- follow-up 2: function ----
# Answer: one pass with a running length that restarts at 1 after a pair
# of equal parity. O(n) time, O(1) space.
def longest_alternating(nums):
    if not nums:
        return 0
    best = 1
    run = 1
    for i in range(1, len(nums)):
        if nums[i - 1] % 2 != nums[i] % 2:
            run += 1
        else:
            run = 1
        if run > best:
            best = run
    return best
```

```


# ---- follow-up 2: tests ----
def test_follow_up_2():
    assert longest_alternating([1, 3, 2, 4]) == 2
    assert longest_alternating([1, 2, 3, 4, 5]) == 5
    assert longest_alternating([]) == 0
    assert longest_alternating([7]) == 1


# =====================================================
# FOLLOW-UP 3: Offline queries, different property (for example all elements distinct)
# =====================================================
# Answer: a property that is NOT a sum of local pair flags needs another
# tool: sort the queries by their right end and sweep (offline), keeping the
# last position of every value in a Fenwick tree.
```

```
# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

122 Cake Balance Line

EASY

Template fit: the same balance test on the cake heights

QUESTION

A cake is described by a list heights , where heights[i] is the amount of cake sitting at position i (positions are left to right). You want to place a single vertical cut exactly at some position i so that the total amount of cake strictly to the left of the cut equals the total amount strictly to the right of the cut (the cake at position i itself is consumed by the cut and counts for neither side). Return the smallest index i where sum(heights[:i]) == sum(heights[i+1:]) , or -1 if no such position exists.

Function signature

def cake_balance_index(heights: list[int]) -> int:

    ...

Follow-up (as reported): Do it in one pass and O(1) extra space by tracking a running left sum against total - left - heights[i] .

EXAMPLES

Example 1	
Input	`heights = [1, 7, 3, 6, 5, 6]`
Output	`3`
Why	`Left of index 3 is 1+7+3=11; right of index 3 is 5+6=11.`

Example 2	
Input	`heights = [1, 2, 3]`
Output	`-1`
Why	`No cut position balances the two sides.`

Example 3	
Input	`heights = [2, 1, -1]`
Output	`0`
Why	`Left of index 0 is empty (0); right is 1+(-1)=0.`

CONSTRAINTS

- 1 <= len(heights) <= 10^5 .
- -10^4 <= heights[i] <= 10^4 (heights may be negative in this abstracted version).

FULL CODE | main question first, then every follow-up, tests at the end

```python
# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: main function ----
# Find a column of the cake whose left side weighs as much as its right
# side. Idea: the right side is total - left - heights[i], so keep a
# running left sum and compare in one pass.
def cake_balance_index(heights):
    total = 0
    for h in heights:
        total += h
    left = 0
    for i in range(len(heights)):
        if left == total - left - heights[i]:
            return i
        left += heights[i]
    return -1


# ---- main question: tests ----
def test_main_question():
    assert cake_balance_index([1, 7, 3, 6, 5, 6]) == 3
    assert cake_balance_index([1, 2, 3]) == -1
    assert cake_balance_index([2, 1, -1]) == 0


# ============================================================
# FOLLOW-UP 1: Which cut makes the two sides as equal as possible?
# ============================================================
# ---- follow-up 1: function ----
# Answer: the same scan, but track the smallest difference
# |left - right| and the first index that reaches it. A perfectly
# balanced cut wins with difference 0.
def closest_cut(heights):
    total = 0
    for h in heights:
        total += h
    left = 0
```

```python
    best_gap = None
    best_index = -1
    for i in range(len(heights)):
        gap = left - (total - left - heights[i])
        if gap < 0:
            gap = -gap
        if best_gap is None or gap < best_gap:
            best_gap = gap
            best_index = i
        left += heights[i]
    return best_index


# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert closest_cut([1, 7, 3, 6, 5, 6]) == 3
    assert closest_cut([5, 1]) == 0
    assert closest_cut([]) == -1
    for code in range(8 ** 6):                          # every list of 6 heights from -3..4: compare with add
ing up both sides
        h = []
```

```
        for i in range(6):
            h.append(code // 8 ** i % 8 - 3)
        best = None
        expected = -1
        for i in range(6):
            left = 0
            for j in range(i):
                left += h[j]
            right = 0
            for j in range(i + 1, 6):
                right += h[j]
            gap = abs(left - right)
            if best is None or gap < best:
                best = gap
                expected = i
        assert closest_cut(h) == expected


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")
```

```
if __name__ == "__main__":
    run_all_tests()
```

123 Find the Pivot Index (Balanced Split) in O(n) — EASY

Template fit: the template as written: left total equals total - left - x

QUESTION

Given an integer array nums , return the leftmost index i such that the sum of all elements strictly to the left of i equals the sum of all elements strictly to the right of i . The element at i itself is not counted on either side. For the first index, the left sum is 0 ; for the last index, the right sum is 0 . If no such index exists, return -1 . Your solution must run in O(n) time and O(1) extra space.

Function signature

def pivot_index(nums: list[int]) -> int:

    ...

Follow-up (as reported): If several pivots exist, return them all. Hint: keep scanning after the first match, maintaining the running left sum.

EXAMPLES

Example 1	
Input	`nums = [1, 7, 3, 6, 5, 6]`
Output	`3`
Why	`Left of index 3: 1+7+3 = 11. Right of index 3: 5+6 = 11.`

Example 2	
Input	`nums = [1, 2, 3]`
Output	`-1`
Why	`No index balances the two sides.`

QUESTION

Example 3	
Input	`nums = [2, 1, -1]`
Output	`0`
Why	`Left of index 0 is 0; right is 1 + (-1) = 0.`

CONSTRAINTS

- 1 <= len(nums) <= 10^4
- -1000 <= nums[i] <= 1000

FULL CODE | main question first, then every follow-up, tests at the end

```
# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: main function ----
# Idea: the sum on the right of index i is total - left - nums[i], so one
# pass with a running left sum is enough.
def pivot_index(nums):
    total = 0
    for x in nums:
        total += x
    left = 0
    for i in range(len(nums)):
        if left == total - left - nums[i]:
            return i
        left += nums[i]
    return -1


# ---- main question: tests ----
def test_main_question():
    assert pivot_index([1, 7, 3, 6, 5, 6]) == 3
    assert pivot_index([1, 2, 3]) == -1
    assert pivot_index([2, 1, -1]) == 0


# ============================================================
# FOLLOW-UP 1: Several pivots can exist: return all of them
# ============================================================
# ---- follow-up 1: function ----
# Answer: do not return at the first match; collect every index whose left
# sum equals its right sum while the scan goes on. Still O(n).
def all_pivots(nums):
    total = 0
    for x in nums:
        total += x
    left = 0
    found = []
    for i in range(len(nums)):
        if left == total - left - nums[i]:
            found.append(i)
        left += nums[i]
    return found
```

```python
# ---- follow-up 1: tests ----
def slow_pivots(nums):
    # check every index by adding up both sides from scratch
    found = []
    for i in range(len(nums)):
        left = 0
        for j in range(i):
            left += nums[j]
        right = 0
        for j in range(i + 1, len(nums)):
            right += nums[j]
        if left == right:
            found.append(i)
    return found


def test_follow_up_1():
    assert all_pivots([0, 0, 0]) == [0, 1, 2]
    for code in range(7 ** 6):                          # every array of 6 numbers from -3..3
        nums = []
        for i in range(6):
            nums.append(code // 7 ** i % 7 - 3)
        assert all_pivots(nums) == slow_pivots(nums)


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

124 Range Duplicate Check with Constant-Time Queries

MEDIUM

Template fit: next[i] = next occurrence of the same value; reach[l] = smallest next over l..end; the range l..r is all distinct when reach[l] > r

QUESTION

Preprocess an array once so that each query unique(l, r) answers in O(1) whether all elements from index l to index r (inclusive) are different.

Function signature

class RangeUnique:
  def __init__(self, a: list)
  def unique(self, l: int, r: int) -> bool

EXAMPLES

QUESTION

Example 1

Input	`ru = RangeUnique([1, 2, 3, 1, 4])`
	`ru.unique(0, 2)`
	`ru.unique(0, 3)`
	`ru.unique(1, 4)`

Output	`ru.unique(0, 2) -> true`
	`ru.unique(0, 3) -> false`
	`ru.unique(1, 4) -> true`

Why	Indexes 0..3 hold 1, 2, 3, 1: the value 1 repeats. Ranges 0..2 and 1..4 have no repeats.

FULL CODE | main question first, then every follow-up, tests at the end

```
# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ========================================================

INF = float("inf")


# ========================================================
# MAIN QUESTION
# ========================================================
# ---- main question: helper functions ----
def unique_with_set(a, l, r):
    # the easy version (say it first): put the range in a set
    seen = set()
    for i in range(l, r + 1):
        seen.add(a[i])
    return len(seen) == r - l + 1


def longest_unique_window(a):
    # stage 2: a sliding window with a count of every value in it
    count = {}
    left = 0
    best = 0
    for right in range(len(a)):
        count[a[right]] = count.get(a[right], 0) + 1
        while count[a[right]] > 1:              # shrink until the new value occurs only once
            count[a[left]] -= 1
            left += 1
        if right - left + 1 > best:
            best = right - left + 1
    return best


# ---- main question: main function ----
# Idea: for every index i find next[i], the next index holding the same
# value (infinity if none): scan from the right with a dictionary of the
# latest index of every value. Then reach[i] = the earliest "next repeat"
# among indices i, i+1, ... (a minimum from the right). The range [l, r]
# has no duplicate exactly when reach[l] > r. Build O(n), query O(1).
# The array can also change (follow-up 1): the simple answer is to rebuild
# in O(n) when reads dominate.
```

```
class RangeUnique:
    def __init__(self, a):
        self.a = list(a)
```

```
        self.rebuild()

    def rebuild(self):
        a = self.a
        n = len(a)
        next_same = [INF] * n
        latest = {}
        for i in range(n - 1, -1, -1):
            if a[i] in latest:
                next_same[i] = latest[a[i]]
            latest[a[i]] = i
        self.reach = [INF] * (n + 1)
        for i in range(n - 1, -1, -1):
            if next_same[i] < self.reach[i + 1]:
                self.reach[i] = next_same[i]
            else:
                self.reach[i] = self.reach[i + 1]

    def unique(self, l, r):
        return self.reach[l] > r

    def update(self, i, value):
        # follow-up 1: reads dominate, so a linear rebuild is fine
        self.a[i] = value
        self.rebuild()


# ---- main question: tests ----
def test_main_question():
    a = [1, 2, 3, 1, 4]
    q = RangeUnique(a)
    for l in range(len(a)):
        for r in range(l, len(a)):
            assert q.unique(l, r) == unique_with_set(a, l, r)
    assert q.unique(0, 2) and not q.unique(0, 3)
    assert longest_unique_window(a) == 4


# ============================================================
# FOLLOW-UP 1: The array changes between queries
# ============================================================
# Answer: RangeUnique.update changes the value and rebuilds the tables in
```

```
# O(n), which is fine when queries are much more common than updates. For
# many updates use a segment tree over "next repeat" with point updates.
# ---- follow-up 1: tests ----
def test_follow_up_1():
    q = RangeUnique([1, 2, 3, 1, 4])
    q.update(3, 9)                                  # an update can remove a duplicate ...
    assert q.unique(0, 4)
    q.update(0, 9)                                  # ... or create one
    assert not q.unique(0, 3)


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

125 Find a Horizontal Cut That Bisects Rectangle Area

MEDIUM

Template fit: events by height with a running width; accumulate area strip by strip until half the total, then solve inside that strip

QUESTION

Given many disjoint axis-aligned rectangles (x1, y1, x2, y2), return the y of the horizontal line that splits their combined area in half, so the area below the line equals the area above it. Return an exact fraction (no floating-point error).

Function signature

def bisecting_cut(rects: list) -> Fraction

EXAMPLES

Example 1	
Input	`[(0, 0, 2, 2)]`
Output	`1`
Why	`One 2x2 square: the middle line is y = 1.`

Example 2	
Input	`[(0, 0, 1, 1), (0, 2, 1, 4)]`
Output	`5/2`
Why	`Areas 1 and 2, total 3, so half is 1.5: all of the first rectangle (area 1) plus 0.5 of the second, which starts at y = 2: y = 5/2.`

FULL CODE | main question first, then every follow-up, tests at the end

```
# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

from fractions import Fraction


# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
# Rectangles are disjoint and written (x1, y1, x2, y2).
def total_area(rects):
    total = 0
    for x1, y1, x2, y2 in rects:
        total += (x2 - x1) * (y2 - y1)
    return total


def area_below(rects, y):
    # the exact area under the horizontal line at height y (used to check the answer)
    total = Fraction(0)
    for x1, y1, x2, y2 in rects:
        top = min(Fraction(y), y2)
        if top > y1:
            total += (x2 - x1) * (top - y1)
    return total


# ---- main question: main function ----
# Find the height of a horizontal line with exactly half of the total
# area below it.
# Idea: sweep upward through the heights where a rectangle starts (its
```

```
# width is added) or ends (its width is removed). Between two neighbouring
# event heights the covered width is constant, so the area grows linearly
# there. Find the strip where the running area reaches the wanted amount
# and interpolate inside it. `share` is the fraction of the area that
# must be below the cut (follow-up 4); exact Fractions are used, never floats.
def cut_height(rects, share=Fraction(1, 2)):
    wanted = share * total_area(rects)
    width_change = {}                                   # height -> width gained when the sweep passes it
    for x1, y1, x2, y2 in rects:
```

```
        width_change[y1] = width_change.get(y1, 0) + (x2 - x1)
        width_change[y2] = width_change.get(y2, 0) - (x2 - x1)
    heights = sorted(width_change)
    width = 0
    so_far = Fraction(0)
    for i in range(len(heights) - 1):
        low = heights[i]
        high = heights[i + 1]
        width += width_change[low]                      # inside this strip the covered width is constant
        gain = width * (high - low)
        if width > 0 and so_far + gain >= wanted:
            return low + (wanted - so_far) / width      # the cut is inside this strip: interpolate
        so_far += gain
    return Fraction(heights[len(heights) - 1])


# ---- main question: tests ----
def test_main_question():
    stacked = [(0, 0, 2, 1), (0, 1, 2, 2)]
    assert cut_height(stacked) == 1
    assert cut_height([(0, 0, 1, 4)]) == 2                          # one tall rectangle: halfway up
    assert cut_height([(0, 0, 1, 1), (0, 5, 1, 6)]) == 1            # a gap between rectangles: the cut sits at
the first top
    odd = [(0, 0, 3, 1), (0, 1, 1, 2)]                              # the cut falls inside a strip at a fraction
al height
    assert area_below(odd, cut_height(odd)) * 2 == total_area(odd)
    big = [(0, 0, 10 ** 9, 10 ** 9)]                                # large coordinates stay exact
    assert cut_height(big) == Fraction(10 ** 9, 2)


# ========================================================
# FOLLOW-UP 4: A cut that leaves a given fraction of the area below it, not exactly half
# ========================================================
# Answer: replace "half of the total" by share * total in the strip test
# (the share parameter); the interpolation inside the strip is the same.
# ---- follow-up 4: tests ----
def test_follow_up_4():
    stacked = [(0, 0, 2, 1), (0, 1, 2, 2)]
    quarter = cut_height(stacked, Fraction(1, 4))
    assert area_below(stacked, quarter) * 4 == total_area(stacked)


# ========================================================
# RUN EVERYTHING
# ========================================================
def run_all_tests():
    test_main_question()
```

```
    test_follow_up_4()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

126 Check pivot after removing one element

MEDIUM

Template fit: prefix sums plus first/last position maps: look up the value whose removal would balance both sides

QUESTION

A pivot index of an array is an index whose left-side sum equals its right-side sum (the pivot element itself is excluded). Remove exactly one element from the array, then ask whether the remaining array has a pivot index. Return one pair (removed index, pivot index), both as indexes in the original array, or None if no removal creates a pivot. Negative numbers and duplicates are allowed.

Function signature

def pivot_after_removal(a: list) -> tuple | None

EXAMPLES

Example 1	
Input	`[2, 1, -1, 3]`
Output	`(0, 3)`
Why	`Removing the 2 at index 0 leaves [1, -1, 3]; the 3 (original index 3) has left sum 1 + (-1) = 0 and nothing on its right.`

Example 2	
Input	`[1, 2, 3]`
Output	`null`
Why	`Whichever element is removed, no pivot appears.`

Example 3	
Input	`[0, 0]`
Output	`(0, 1)`
Why	`Remove either 0: the other one is a pivot of an array with one element.`

FULL CODE | main question first, then every follow-up, tests at the end

```
# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: main function ----
# Remove ONE element of the array, then is there a pivot: an element
# whose left sum equals its right sum? Return (removed index, pivot index)
# or None.
# Idea: let `prefix[j]` be the sum before index j and key[j] =
# 2 * prefix[j] + a[j].
#  - Pivot j AFTER the removed index i: the left sum is prefix[j] - a[i]
#    and the right sum is total - prefix[j] - a[j]; equal means
#    a[i] = key[j] - total. Scan j from the left, remembering the earlier values.
#  - Pivot j BEFORE the removed index i: left = prefix[j], right = total -
#    prefix[j] - a[j] - a[i]; equal means a[i] = total - key[j]. Scan j
#    from the right, remembering the later values.
# Two scans with a dictionary (value -> index): O(n).
def pivot_after_removal(a):
    n = len(a)
    total = 0
```

```
    for x in a:
        total += x
    prefix = [0]
    for x in a:
        prefix.append(prefix[len(prefix) - 1] + x)
    key = []
    for j in range(n):
        key.append(2 * prefix[j] + a[j])
    index_of = {}                                   # a value seen EARLIER -> its index
    for j in range(n):
        i = index_of.get(key[j] - total)
        if i is not None:
            return i, j
        index_of[a[j]] = j
    index_of = {}                                   # a value seen LATER -> its index
    for j in range(n - 1, -1, -1):
        i = index_of.get(total - key[j])
        if i is not None:
            return i, j
```

```
        index_of[a[j]] = j
    return None


# ---- main question: tests ----
def test_main_question():
    # [1,7,3,6,5,6] has a pivot at index 3 without removal, but removing one element never gives another
    assert pivot_after_removal([1, 7, 3, 6, 5, 6]) is None
    assert pivot_after_removal([1, 1, 1, 1]) == (1, 2)     # remove index 1: [1, 1, 1] has the middle pivot


# ========================================================
# FOLLOW-UP 1: Without any removal (LeetCode 724)
# ========================================================
# ---- follow-up 1: function ----
# Answer: one pass: the pivot is the index where the left sum equals the
# total minus the left sum minus the element itself. O(n) time, O(1) space.
def pivot_index(a):
    total = 0
    for x in a:
        total += x
    left = 0
    for i in range(len(a)):
        if left == total - left - a[i]:
            return i
        left += a[i]
    return -1


# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert pivot_index([1, 1, 1, 1]) == -1
    assert pivot_index([1, 7, 3, 6, 5, 6]) == 3
    assert pivot_index([1, 2, 3]) == -1


# ========================================================
# FOLLOW-UP 2: At most one removal (removal optional)
# ========================================================
# ---- follow-up 2: function ----
# Answer: true if pivot_index finds a pivot; otherwise run the removal test.
def has_pivot(a, allow_removal=True):
    if pivot_index(a) != -1:
        return True
    if not allow_removal:
```

```
        return False
    return pivot_after_removal(a) is not None


# ---- follow-up 2: tests ----
def test_follow_up_2():
    assert not has_pivot([1, 2, 3], allow_removal=False)
    assert has_pivot([1, 1, 1, 1])
    assert not has_pivot([1, 1, 1, 1], allow_removal=False)


# ========================================================
# FOLLOW-UP 3: Return which element to remove and the pivot
# ========================================================
# Answer: pivot_after_removal already returns (removed index, pivot index).


# ========================================================
# RUN EVERYTHING
# ========================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

127 Gold Chain Split into Two Equal Halves

MEDIUM

Template fit: prefix sums with first/last position maps: find two cut points that leave equal halves around one removed link

QUESTION

A gold chain is an array of segment weights. Remove exactly one segment, reconnect the rest, then cut the chain once so that the two parts have equal total weight. Return whether this is possible for some choice of the removed segment and the cut.

Function signature

def gold_chain(a: list) -> bool

EXAMPLES

Example 1	
Input	`[1, 2, 3, 4]`
Output	`true`
Why	`Remove the 4: [1, 2, 3] splits as [1, 2] and [3], both 3.`

Example 2	
Input	`[1, 1, 5]`
Output	`true`
Why	`Remove the 5: [1, 1] splits into 1 and 1.`

Example 3	
Input	`[1, 2, 4]`
Output	`false`
Why	`Removing 1 leaves 6 (halves of 3 need [2] | [4]: no), removing 2 leaves 5, removing 4 leaves 3: nothing works.`

FULL CODE | main question first, then every follow-up, tests at the end

```
# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ========================================================

# ========================================================
# MAIN QUESTION
# ========================================================
# ---- main question: helper functions ----
def prefix_sums(a):
    # P[j] = the sum of the first j segments
    P = [0]
    for x in a:
        P.append(P[len(P) - 1] + x)
    return P


# ---- main question: main function ----
# A chain of gold segments with weights a. Remove exactly ONE segment,
# then cut the rest once into two non-empty parts of equal weight. Can it
# be done?
# Idea: with prefix sums P (P[j] = the first j segments) removing segment r
# leaves total - a[r]; each half must weigh half of that. A cut BEFORE the
# removed segment (position j <= r) needs P[j] == half. A cut AFTER it
# (j > r) needs P[j] - a[r] == half. Instead of trying every cut, look up the
# FIRST position with a given prefix sum (for cuts before r) and the LAST
# position (for cuts after r) in two dictionaries: O(n) in total.
def gold_chain(a):
    n = len(a)
    if n < 3:
        return False
    P = prefix_sums(a)
    total = P[n]
    first = {}                              # prefix sum -> its first cut position
    last = {}                               # prefix sum -> its last cut position
    for j in range(1, n):                   # cuts sit between segments: positions 1 .. n - 1
        if P[j] not in first:
            first[P[j]] = j
        last[P[j]] = j
    for r in range(n):
        left_over = total - a[r]
        if left_over % 2 == 1:
            continue
```

```
        half = left_over // 2
        latest_cut_before = r               # the cut must leave a piece on the right, so j <= n - 2
        if n - 2 < latest_cut_before:
            latest_cut_before = n - 2
        if half in first and first[half] <= latest_cut_before:
            return True                     # cut before the removed segment
        if half + a[r] in last and last[half + a[r]] > r:
            return True                     # cut after it
    return False


# ---- main question: tests ----
def test_main_question():
    assert gold_chain([5, 1, 5])                # remove the middle 1, then cut 5 | 5
    assert gold_chain([2, 7, 2])
    assert not gold_chain([1, 2, 3])
    assert not gold_chain([1, 1])
    assert not gold_chain([1, 2, 4])
```

```
# ========================================================
# FOLLOW-UP 1: List every valid (removed segment, cut position) pair
# ========================================================
# ---- follow-up 1: function ----
# Answer: the answer itself can have quadratic size, so a plain double
# loop is fine: for each removed segment, walk along the rest and
# report every cut where the left part is half the total.
def valid_pairs(a):
    pairs = []
    for r in range(len(a)):
        rest = a[:r] + a[r + 1:]
        total = 0
        for x in rest:
            total += x
        running = 0
        for cut in range(1, len(rest)):          # cut after `cut` segments of the reconnected chain
            running += rest[cut - 1]
            if running * 2 == total:
                pairs.append((r, cut))
    return pairs


# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert (1, 1) in valid_pairs([1, 2, 1])
```

```
    for code in range(4 ** 5):                    # every chain of 5 segments with weights 1..4
        chain = []
        for i in range(5):
            chain.append(code // 4 ** i % 4 + 1)
        assert gold_chain(chain) == (len(valid_pairs(chain)) > 0)


# ========================================================
# FOLLOW-UP 2: The chain is a circle (cut anywhere)
# ========================================================
# Answer: a circle has no "before" and "after" the removed segment: after
# removing it, the pieces form a line again starting just after it, so
# rotate the array so the removed segment is last and reuse the same test.


# ========================================================
# FOLLOW-UP 3: Remove k segments instead of one
# ========================================================
# Answer: with prefix sums, removing a block of k neighbouring segments
# is a range sum, so the same lookups work for each block; removing k
# separate segments is a subset problem (much harder).


# ========================================================
# RUN EVERYTHING
# ========================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

128 Count the number of ways to split a string. MEDIUM

Template fit: positions of the ones; the answer multiplies the gap sizes around the k-th and 2k-th one (zero ones: choose two cut points)

QUESTION

A binary string s is split into three non-empty parts by two cuts. Count the ways in which the three parts contain the same number of 1s, and return the count modulo 1,000,000,007.

Function signature

def numWays(s: str) -> int

EXAMPLES

Example 1	
Input	`'10101'`
Output	`4`
Why	`Four ways to split '10101' so each part has one 1 (for example 1|010|1 and 10|10|1).`

Example 2	
Input	`'1001'`
Output	`0`
Why	`Two 1s cannot be divided into three equal groups.`

Example 3	
Input	`'0000'`
Output	`3`
Why	`No 1s: any two cuts work: choose 2 of the 3 gaps = 3 ways.`

FULL CODE | main question first, then every follow-up, tests at the end

```python
# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

MOD = 10 ** 9 + 7


# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
def choose(n, k):
    # the number of ways to pick k things out of n
    ways = 1
    for i in range(k):
        ways = ways * (n - i) // (i + 1)
    return ways


# ---- main question: main function ----
# Cut a binary string into `parts` non-empty pieces (3 in the classic
# question, LeetCode 1573) so that every piece has the same number of 1s.
# Count the ways.
# Idea: let `ones` be the positions of the 1s. If their number is not
# divisible by `parts` the answer is 0. With no 1s at all, any choice of
# parts - 1 cut positions among the n - 1 gaps works: choose(n - 1, parts - 1).
# Otherwise each piece holds `each` ones, and the cut after piece k can go
```

```
# anywhere in the run of zeros between the last 1 of that piece and the first
# 1 of the next: (the distance between those two 1s) choices. Multiply the
# choices of all cuts. (parts = m is follow-up 1.)
# For huge answers the result is kept modulo 10^9 + 7 after every
# multiplication (follow-up 3); mod=None gives the exact number.
def numWays(s, parts=3, mod=MOD):
    n = len(s)
    if n < parts:
        return 0
    ones = []
    for i in range(n):
        if s[i] == "1":
            ones.append(i)
    if len(ones) % parts != 0:
        return 0
```

```
    if len(ones) == 0:
        total = choose(n - 1, parts - 1)
        if mod:
            return total % mod
        return total
    each = len(ones) // parts
    answer = 1
    for cut in range(1, parts):
        gap = ones[cut * each] - ones[cut * each - 1]      # where the cut may go
        answer *= gap
        if mod:
            answer %= mod
    return answer


# ---- main question: tests ----
def test_main_question():
    assert numWays("10101") == 4
    assert numWays("1001") == 0
    assert numWays("0000") == 3
    assert numWays("100100010100110") == 12
    assert numWays("", parts=3) == 0


# ========================================================
# FOLLOW-UP 1: Split into m parts with equal ones
# ========================================================
# Answer: the same argument with `parts` = m: each of the m - 1 cuts sits in
# the gap after a multiple of k ones; multiply the gap sizes. With no ones it
# is choose(n - 1, m - 1).
# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert numWays("1100", parts=2) == 1
    assert numWays("0000", parts=2) == 3


# ========================================================
# FOLLOW-UP 2: Count ways to split into palindromes
# ========================================================
# ---- follow-up 2: function ----
# Answer: a different problem. dp[i] = the number of ways to cut the first
# i letters; add dp[j] for every palindrome s[j:i]. First build a table
# is_pal[i][j] ("s[i..j] is a palindrome") in O(n^2).
def count_palindrome_splits(s):
    n = len(s)
```

```
    is_pal = []
    for i in range(n):
        is_pal.append([False] * n)
    for i in range(n - 1, -1, -1):
```

```
        for j in range(i, n):
            inside_ok = j - i < 2 or is_pal[i + 1][j - 1]
            is_pal[i][j] = s[i] == s[j] and inside_ok
    dp = [1] + [0] * n
    for i in range(1, n + 1):
        for j in range(i):
            if is_pal[j][i - 1]:
                dp[i] += dp[j]
    return dp[n]


# ---- follow-up 2: tests ----
def test_follow_up_2():
    assert count_palindrome_splits("aab") == 2
    assert count_palindrome_splits("aaa") == 4
    assert count_palindrome_splits("") == 1


# ============================================================
# FOLLOW-UP 3: The numbers get huge
# ============================================================
# Answer: return the answer modulo 10^9 + 7 and reduce after every
# multiplication (the `mod` parameter); with fixed-width integers use
# 64-bit intermediate values. No big integers are needed: O(n).
# ---- follow-up 3: tests ----
def test_follow_up_3():
    big = "0" * 5000
    assert numWays(big) == choose(4999, 2) % MOD
    assert numWays(big, mod=None) == choose(4999, 2)


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    test_follow_up_3()
    print("all tests passed")
```

```
if __name__ == "__main__":
    run_all_tests()
```

GROUP 16 OF 43 | 8 QUESTIONS, ONE CODE TEMPLATE | 1 E 5 M 2 H

Sorted intervals: merge while they touch, then read the gaps or search

129 Check an Interval Against Existing Intervals for Conflicts — EASY

Template fit: bisect on the starts; a new booking conflicts if the previous one ends after its start or the next one starts before its end

QUESTION

Design a calendar of half-open intervals [start, end). book(start, end) books the interval and returns True when it does not overlap any interval booked so far, and returns False (booking nothing) when it does. Intervals that only touch, one ending exactly where the next starts, do not conflict.

Function signature

class Calendar:
    def book(self, start: int, end: int) -> bool

EXAMPLES

Example 1

Input	`c = Calendar()`
	`c.book(10, 20)`
	`c.book(15, 25)`
	`c.book(20, 30)`
	`c.book(5, 10)`
	`c.book(12, 14)`

Output	`c.book(10, 20) -> true`
	`c.book(15, 25) -> false`
	`c.book(20, 30) -> true`
	`c.book(5, 10) -> true`
	`c.book(12, 14) -> false`

Why	`[15, 25) overlaps [10, 20) and [12, 14) lies inside it, so both fail. [20, 30) and [5, 10) only touch the booked interval at an endpoint, so they succeed.`

FULL CODE | main question first, then every follow-up, tests at the end

```
# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ========================================================

# ========================================================
# MAIN QUESTION
# ========================================================
# ---- main question: helper functions ----
def first_greater(values, x):
    # binary search in a sorted list: the first index whose value is > x
    low = 0
    high = len(values)
    while low < high:
        middle = (low + high) // 2
        if values[middle] <= x:
            low = middle + 1
        else:
            high = middle
    return low


def conflicts(candidate, intervals):
```

```
    # the simple check against every booking: half-open [start, end), so
    # touching ends do NOT conflict
    start, end = candidate
    for a, b in intervals:
        if a < end and start < b:
            return True
    return False


# ---- main question: main class ----
# book(start, end) says whether the interval can be booked without
# overlapping an existing booking (and books it if so). Half-open intervals
# [start, end): a booking may start exactly when another ends.
# Idea: keep the bookings sorted by start. Then only the TWO neighbours of
# the new interval can conflict: the booking just before (it must end by the
# new start) and the booking just after (it must start at or after the new
# end). One binary search finds the place: O(log n) per check.
class Calendar:
    def __init__(self):
        self.starts = []
```

```
        self.ends = []

    def book(self, start, end):
        i = first_greater(self.starts, start)            # the new booking would go at position i
        if i > 0 and self.ends[i - 1] > start:
            return False                                 # the booking before runs into it
        if i < len(self.starts) and self.starts[i] < end:
            return False                                 # the booking after starts too early
        self.starts.insert(i, start)
        self.ends.insert(i, end)
        return True


# ---- main question: tests ----
def test_main_question():
    taken = [(10, 20), (30, 40)]
    assert conflicts((15, 25), taken)
    assert not conflicts((20, 30), taken)
    assert conflicts((19, 31), taken)
    calendar = Calendar()
    assert calendar.book(10, 20) and calendar.book(20, 30)
    assert not calendar.book(15, 25) and calendar.book(5, 10)


# ========================================================
# FOLLOW-UP 1: Allow double booking but never triple booking (LeetCode 731)
# ========================================================


# ========================================================
# FOLLOW-UP 2: Report the maximum number of overlapping bookings after each booking (LeetCode 732)
# ========================================================
# ---- follow-up 1: function ----
# Answer: keep a difference map (+1 at every start, -1 at every end) and
# sweep it in time order; the running total is the number of overlapping
# bookings. If the biggest total ever exceeds the limit (2 for LeetCode 731),
# undo the booking and reject it. O(n log n) per booking (a segment tree makes
# it faster). peak() is also the answer to LeetCode 732.
class DoubleBooking:
    def __init__(self, limit=2):
        self.delta = {}
        self.limit = limit

    def peak(self):
        # sweep the +1 / -1 changes in time order
```

```
        running = 0
        best = 0
        for t in sorted(self.delta):
            running += self.delta[t]
            if running > best:
                best = running
        return best

    def book(self, start, end):
        self.delta[start] = self.delta.get(start, 0) + 1
        self.delta[end] = self.delta.get(end, 0) - 1
        if self.peak() > self.limit:
            self.delta[start] -= 1                      # undo the booking and reject it
            self.delta[end] += 1
            return False
        return True


# ---- follow-up 1: tests ----
def test_follow_up_1():
    d = DoubleBooking()
    assert d.book(10, 20) and d.book(15, 25)            # double booking is allowed ...
    assert not d.book(17, 18)                           # ... triple booking is not
    assert d.peak() == 2
    one = DoubleBooking(limit=1)                        # limit 1 is the plain no-overlap calendar
    assert one.book(1, 5) and not one.book(4, 6)


# ========================================================
# FOLLOW-UP 3: Closed intervals instead of half-open
# ========================================================
# Answer: with closed intervals [start, end] two bookings that share an end
# DO conflict, so compare with <= instead of < (the previous booking must end
# before the new start; the next must start after the new end).


# ========================================================
# RUN EVERYTHING
# ========================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")
```

```
if __name__ == "__main__":
    run_all_tests()
```

130 Find Containing Range

MEDIUM

Template fit: merge once, keep the starts in a list, and bisect to find the one interval that could contain x

QUESTION

Given a collection of inclusive ranges [start, end] (overlapping or touching ranges are merged), find(x) returns the merged range that contains the number x, or None when no range contains it.

Function signature

class RangeIndex:
  def __init__(self, ranges: list)
  def find(self, x: int) -> tuple | None

EXAMPLES

QUESTION

Example 1

Input	`r = RangeIndex([(1, 3), (2, 6), (8, 10)])`
	`r.find(5)`
	`r.find(7)`
	`r.find(10)`

Output	`r.find(5) -> (1, 6)`
	`r.find(7) -> null`
	`r.find(10) -> (8, 10)`

Why	`[1, 3] and [2, 6] overlap and merge into [1, 6]; 7 falls in the gap between the merged ranges.`

FULL CODE | main question first, then every follow-up, tests at the end

```
# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ========================================================

# ========================================================
# MAIN QUESTION
# ========================================================
# ---- main question: helper functions ----
def first_at_least(values, x):
    # binary search in a sorted list: the first index whose value is >= x
    low = 0
    high = len(values)
    while low < high:
        middle = (low + high) // 2
        if values[middle] < x:
            low = middle + 1
        else:
            high = middle
    return low


def first_greater(values, x):
    # binary search in a sorted list: the first index whose value is > x
    low = 0
    high = len(values)
    while low < high:
        middle = (low + high) // 2
        if values[middle] <= x:
            low = middle + 1
        else:
            high = middle
    return low


# ---- main question: main class ----
# Given ranges [start, end] (closed), find(x) returns the MERGED range that
# contains x, or None.
# Idea: merge the ranges once (insert them one by one; an insert replaces
# every range it touches by one merged range). After that the merged ranges
# are sorted and disjoint, so the only candidate for x is the range with the
# largest start <= x (one binary search), and it contains x if its end is at
# least x. The merging is paid once.
```

```
class RangeIndex:
    def __init__(self, ranges=()):
        self.starts = []
```

```
        self.ends = []
        self.original = sorted(ranges)
        for start, end in self.original:
            self.insert(start, end)

    def insert(self, start, end):
        # follow-up 2: ranges can be inserted over time
        i = first_at_least(self.ends, start)              # the first range ending at or after the new start
 ...
        j = first_greater(self.starts, end)               # ... up to the last range starting at or before t
he new end
        if i < j:
            if self.starts[i] < start:
                start = self.starts[i]
            if self.ends[j - 1] > end:
                end = self.ends[j - 1]
        self.starts[i:j] = [start]                        # replace everything the new range touches
        self.ends[i:j] = [end]

    def find(self, x):
        i = first_greater(self.starts, x) - 1
        if i >= 0 and self.ends[i] >= x:
            return (self.starts[i], self.ends[i])
        return None

    def find_original(self, x):
        # follow-up 1: every ORIGINAL range that contains x (only those starting at or before x can)
        found = []
        for start, end in self.original:
            if start > x:
                break
            if end >= x:
                found.append((start, end))
        return found

    def find_many(self, queries):
        # follow-up 3: queries in bulk: sort the queries and sweep once along the ranges
        order = []
        for i in range(len(queries)):
            order.append((queries[i], i))
        order.sort()
        answers = [None] * len(queries)
        j = 0
        for x, i in order:
```

```
            while j < len(self.ends) and self.ends[j] < x:
                j += 1                                    # skip ranges that end before x
            if j < len(self.starts) and self.starts[j] <= x:
                answers[i] = (self.starts[j], self.ends[j])
        return answers


# ---- main question: tests ----
def test_main_question():
    index = RangeIndex([(1, 3), (2, 6), (8, 10), (15, 18)])
    assert index.find(4) == (1, 6)
    assert index.find(7) is None
    assert index.find(8) == (8, 10)
    assert index.find(0) is None and index.find(99) is None


# ========================================================
# FOLLOW-UP 1: Return every original range that contains x, not the merged one
# ========================================================
# Answer: keep the original ranges sorted by start; only ranges starting at
# or before x can contain x, and among them keep those whose end is at least
```

```
# x (find_original). With many overlaps use an interval tree: O(log n + k)
# per query for k results.
# ---- follow-up 1: tests ----
def test_follow_up_1():
    index = RangeIndex([(1, 3), (2, 6), (8, 10), (15, 18)])
    assert index.find_original(2) == [(1, 3), (2, 6)]


# =========================================================
# FOLLOW-UP 2: Ranges are inserted and deleted over time
# =========================================================
# Answer: insert() above splices a new range into the sorted lists
# (merging what it touches). Deleting needs the original ranges too (a
# merged range may have to split again), so keep a count per point of
# coverage, or an interval tree.
# ---- follow-up 2: tests ----
def test_follow_up_2():
    index = RangeIndex([(1, 3), (2, 6), (8, 10), (15, 18)])
    index.insert(6, 8)                                  # (6, 8) joins the first two blocks
    assert index.find(7) == (1, 10)
    assert index.find(12) is None


# =========================================================
```

```
# FOLLOW-UP 3: Queries are given in bulk
# =========================================================
# Answer: find_many sorts the queries and sweeps once along the ranges
# instead of one binary search per query: O((n + q) log q).
# ---- follow-up 3: tests ----
def test_follow_up_3():
    index = RangeIndex([(1, 3), (2, 6), (8, 10), (15, 18)])
    queries = [4, 20, 16, 9]
    expected = []
    for q in queries:
        expected.append(index.find(q))
    assert index.find_many(queries) == expected


# =========================================================
# RUN EVERYTHING
# =========================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

131 Union of Sorted Interval Lists

MEDIUM

Template fit: walk both sorted lists with two indices, always taking the smaller start next, then the same merge test

QUESTION

Two lists of intervals are each sorted and non-overlapping on their own. Return the union of both lists as sorted, non-overlapping intervals. Intervals that touch, such as (1, 3) and (3, 5), are merged unless touching=False.

Function signature

def union_two(a: list, b: list, touching=True) -> list

EXAMPLES

QUESTION

Example 1	
Input	[(1, 3), (6, 9)], [(2, 5), (10, 12)]
Output	[(1, 5), (6, 9), (10, 12)]
Why	(1, 3) and (2, 5) overlap and become (1, 5); (6, 9) and (10, 12) stay.

Example 2	
Input	[(1, 3)], [(3, 5)]
Output	[(1, 5)]
Why	(1, 3) and (3, 5) touch at 3, so they merge.

Example 3	
Input	[(1, 3)], [(3, 5)], touching = false
Output	[(1, 3), (3, 5)]
Why	With touching=False they only merge when they truly overlap.

FULL CODE | main question first, then every follow-up, tests at the end

```
# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ========================================================

import heapq

END = object()                                  # marks "this list has no more intervals"


# ========================================================
# MAIN QUESTION
# ========================================================
# ---- main question: helper functions ----
def merge_sorted(lists):
    # merge several sorted lists (or streams) into one sorted stream, one item
    # at a time: a heap holds the next item of every list, so memory is O(K)
    iterators = []
    for items in lists:
        iterators.append(iter(items))
    heap = []
    for i in range(len(iterators)):
        first = next(iterators[i], END)
        if first is not END:
            heap.append((first, i))
    heapq.heapify(heap)
    while heap:
        item, i = heapq.heappop(heap)
        yield item
        following = next(iterators[i], END)
        if following is not END:
            heapq.heappush(heap, (following, i))


def union_stream(lists, touching=True):
    # the union of sorted lists of (start, end), as a stream (follow-up 1).
    # Take the interval with the smaller start next; if it touches or overlaps
    # the block being built, extend that block, otherwise finish the block.
    current = None
```

```
    for start, end in merge_sorted(lists):
        if current is None:
            current = [start, end]
            continue
```

```
        if touching:
            joined = start <= current[1]           # closed intervals: [1, 3] and [3, 5] share the point 3
        else:
            joined = start < current[1]
        if joined:
            if end > current[1]:
                current[1] = end
        else:
            yield tuple(current)                   # finished one block: give it out now
            current = [start, end]
    if current is not None:
        yield tuple(current)


# ---- main question: main function ----
# Idea: a two-pointer merge: always take the interval with the smaller
# start next. If it touches or overlaps the last output interval (start <=
# its end), extend that one, otherwise start a new one.
def union_two(a, b, touching=True):
    return list(union_stream([a, b], touching))


# the same code with any number of lists: K lists, O(K) memory
def union_k(lists, touching=True):
    return list(union_stream(lists, touching))


# ---- main question: tests ----
def test_main_question():
    a = [(1, 3), (6, 8)]
    b = [(2, 4), (9, 10)]
    assert union_two(a, b) == [(1, 4), (6, 8), (9, 10)]
    assert union_two([(1, 3)], [(3, 5)]) == [(1, 5)]                    # closed intervals that touch merge
    assert union_two([(1, 3)], [(3, 5)], False) == [(1, 3), (3, 5)]     # half-open intervals that only tou
ch stay apart
    assert union_k([a, b, []]) == [(1, 4), (6, 8), (9, 10)]             # three lists, one of them empty


# ======================================================
# FOLLOW-UP 1: What if the lists are too big for memory?
# ======================================================
# Answer: read each list through an iterator (a file or a paged API).
# union_stream only holds one interval per list plus the block being built,
# so memory is O(K) and it can hand out results as it goes.
```

```
# ======================================================
# FOLLOW-UP 3: Do touching intervals merge?
# ======================================================
# Answer: the `touching` option: True merges intervals that share an end
# ([1, 3] and [3, 5]); False keeps them apart (half-open intervals).


# ======================================================
# FOLLOW-UP 2: What if one list may overlap itself?
# ======================================================


# ======================================================
# FOLLOW-UP 4: Each list may overlap itself or be unsorted
# ======================================================
```

```python
# ---- follow-up 2: function ----
# Answer: the merge already copes with overlapping intervals, because it
# folds every new interval into the block being built. For an UNSORTED list,
# sort it first (O(n log n)); then union the cleaned lists as before.
def normalise(intervals):
    ordered = sorted(intervals)
    return list(union_stream([ordered]))


def union_unsorted(a, b):
    return union_two(normalise(a), normalise(b))


# ---- follow-up 2: tests ----
def test_follow_up_2():
    assert union_unsorted([(5, 9), (1, 6)], [(8, 12)]) == [(1, 12)]     # a list that overlaps itself
    assert union_unsorted([(9, 10), (1, 2)], []) == [(1, 2), (9, 10)]   # an unsorted list


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_2()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

132 Range Module (Insert and Point Query) MEDIUM

Template fit: keep sorted lefts and rights; adding bisects to the first interval that touches and the first that starts after, and replaces that block by one merged interval

QUESTION

Design a data structure that tracks a set of half-open ranges [left, right) over the number line. add_range(left, right) adds the interval; overlapping or touching intervals are merged so that only disjoint, non-adjacent intervals remain. query_point(x) returns True when x lies inside a tracked interval; because the intervals are half-open, a point equal to a right endpoint is not inside.

Function signature

class RangeModule:
    def add_range(self, left: float, right: float) -> None
    def query_point(self, x: float) -> bool

EXAMPLES

Example 1

Input	`rm = RangeModule()`
	`rm.add_range(10, 20)`
	`rm.query_point(15)`
	`rm.query_point(20)`
	`rm.query_point(10)`

Output	`rm.query_point(15) -> true`
	`rm.query_point(20) -> false`
	`rm.query_point(10) -> true`

Why	`[10, 20) contains 10 and 15 but not 20.`

QUESTION

Example 2	

Input	`rm = RangeModule()`
	`rm.add_range(10, 20)`
	`rm.add_range(20, 30)`
	`rm.query_point(20)`
	`rm.query_point(25)`
	`rm.query_point(30)`

Output	`rm.query_point(20) -> true`
	`rm.query_point(25) -> true`
	`rm.query_point(30) -> false`

Why	`The touching ranges merge into [10, 30), so 20 is now covered; 30 still is not.`

Example 3	
Input	`rm = RangeModule()` `rm.query_point(5)`
Output	`rm.query_point(5) -> false`
Why	`Nothing has been added yet.`

CONSTRAINTS

- Endpoints and query points are integers or floats with -10^9 <= value <= 10^9 .
- For every add_range call, left < right .
- Up to 10^5 total operations.
- Touching intervals merge: add_range(1, 5) followed by add_range(5, 10) yields one interval [1,
- 10) .

FULL CODE | main question first, then every follow-up, tests at the end

```python
# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ========================================================

import random


# ========================================================
# MAIN QUESTION
# ========================================================
# ---- main question: helper functions ----
def first_at_least(values, x):
    # binary search in a sorted list: the first index whose value is >= x
    low = 0
    high = len(values)
    while low < high:
        middle = (low + high) // 2
        if values[middle] < x:
            low = middle + 1
        else:
            high = middle
    return low


def first_greater(values, x):
    # binary search in a sorted list: the first index whose value is > x
    low = 0
    high = len(values)
```

```
    while low < high:
        middle = (low + high) // 2
        if values[middle] <= x:
            low = middle + 1
        else:
            high = middle
    return low


# ---- main question: main class ----
# Ranges are half-open [left, right): add_range(left, right) adds one,
# query_point(x) says whether x is inside any added range.
# Idea: keep the ranges merged, sorted and disjoint (two lists, lefts and
# rights). Adding replaces every range the new one touches by one merged
```

```
# range (found with two binary searches). A query is one binary search for the
# only candidate: the range with the largest left <= x.
class RangeModule:
    def __init__(self):
        self.lefts = []
        self.rights = []

    def add_range(self, left, right):
        i = first_at_least(self.rights, left)              # the first range ending at or after the new left
        j = first_greater(self.lefts, right)               # the first range starting after the new right
        if i < j:
            if self.lefts[i] < left:
                left = self.lefts[i]
            if self.rights[j - 1] > right:
                right = self.rights[j - 1]
        self.lefts[i:j] = [left]
        self.rights[i:j] = [right]

    def query_point(self, x):
        i = first_greater(self.lefts, x) - 1
        return i >= 0 and x < self.rights[i]


# ---- main question: tests ----
def test_main_question():
    module = RangeModule()
    module.add_range(10, 20)
    assert module.query_point(15) and module.query_point(10) and not module.query_point(20)
    module.add_range(20, 30)
    assert module.query_point(20) and module.query_point(25) and not module.query_point(30)
    assert not RangeModule().query_point(5)


# ========================================================
# FOLLOW-UP 1: Add remove_range(left, right)
# ========================================================
# ---- follow-up 1: function ----
# Answer: find the block of ranges that overlap [left, right) with two
# binary searches. Keep the part of the first one to the left of the removal
# and the part of the last one to the right of it, and splice those
# remainders in place of the block.
class RemovableRangeModule(RangeModule):
    def remove_range(self, left, right):
        i = first_greater(self.rights, left)               # the first range that ends after `left`
        j = first_at_least(self.lefts, right)              # the first range that starts at or after `right`
```

```
        if i >= j:
            return                                         # nothing overlaps the removal
        new_lefts = []
        new_rights = []
        if self.lefts[i] < left:
```

```
            new_lefts.append(self.lefts[i])
            new_rights.append(left)
        if self.rights[j - 1] > right:
            new_lefts.append(right)
            new_rights.append(self.rights[j - 1])
        self.lefts[i:j] = new_lefts
        self.rights[i:j] = new_rights


# ---- follow-up 1: tests ----
def test_follow_up_1():
    random.seed(83)
    for round_number in range(300):                        # random adds and removes: compare with a set of p
oints
        module = RemovableRangeModule()
        points = set()
        for step in range(random.randint(1, 8)):
            a = random.randint(0, 20)
            b = a + random.randint(1, 6)
            if random.random() < 0.6:
                module.add_range(a, b)
                for p in range(a, b):
                    points.add(p)
            else:
                module.remove_range(a, b)
                for p in range(a, b):
                    points.discard(p)
        for p in range(-1, 30):
            assert module.query_point(p) == (p in points)


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
```

```
    run_all_tests()
```

133 Meetings Minus a Do-Not-Schedule Block

MEDIUM

Template fit: merge first, then for each merged meeting keep the part before the blocked window and the part after it

QUESTION

You are given a list of meetings , each a (start, end) interval with start < end , and one "do-not- schedule" (DNS) block (dns_start, dns_end) . Consider the set of all time points covered by at least one meeting. Return the smallest list of non-overlapping, sorted intervals that covers exactly those points except any point lying inside the DNS block (treat the DNS block as removing the open coverage on (dns_start, dns_end) ; equivalently split coverage at those bounds). Meetings may overlap and are given in any order. Return the result sorted by start time.

Function signature

def subtract_dns(meetings: list[tuple[int, int]],
dns: tuple[int, int]) -> list[tuple[int, int]]:
...

Follow-up (as reported): Generalize to a list of non-overlapping DNS blocks: merge the meetings first, then walk both sorted lists together subtracting each block.

QUESTION

EXAMPLES

Example 1	
Input	meetings = [(1,7),(5,10),(12,30),(22,40),(50,65)], dns = (20,35)
Output	[(1,10),(12,20),(35,40),(50,65)]
Why	Merged coverage is [1,10],[12,40],[50,65]. Removing (20,35) splits [12,40] into [12,20] and [35,40].

Example 2	
Input	meetings = [(0,10)], dns = (0, 10)
Output	[]
Why	The DNS block removes the entire covered range.

Example 3	
Input	meetings = [(0,5),(6,9)], dns = (100, 200)
Output	[(0,5),(6,9)]
Why	The DNS block covers nothing that was scheduled.

CONSTRAINTS

- 1 <= len(meetings) <= 10^5 .
- Coordinates are integers; 0 <= start < end <= 10^9 .

FULL CODE | main question first, then every follow-up, tests at the end

```
# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ========================================================

# ========================================================
# MAIN QUESTION
# ========================================================
# ---- main question: helper functions ----
def merge_intervals(meetings):
    # sort by start, then join the ones that overlap or touch
    merged = []
    for start, end in sorted(meetings):
        if merged and start <= merged[-1][1]:
            if end > merged[-1][1]:
                merged[-1] = (merged[-1][0], end)
        else:
            merged.append((start, end))
    return merged


# ---- main question: main function ----
# Remove a do-not-schedule block (low, high) from the meetings.
# Idea: merge the meetings first; then every merged meeting keeps at most
# two pieces: the part to the LEFT of the block and the part to the RIGHT of it.
def subtract_dns(meetings, dns):
    low, high = dns
    pieces = []
    for start, end in merge_intervals(meetings):
        if start < low:
            pieces.append((start, min(end, low)))          # the part before the block
        if end > high:
            pieces.append((max(start, high), end))         # the part after the block
```

```
    return pieces


# ---- main question: tests ----
def test_main_question():
    assert merge_intervals([(1, 3), (2, 6), (8, 10)]) == [(1, 6), (8, 10)]
    assert subtract_dns([(1, 5), (7, 12)], (4, 9)) == [(1, 4), (9, 12)]
    assert subtract_dns([(5, 6)], (4, 9)) == []                      # a meeting inside the block disappears
```

```
# ========================================================
# FOLLOW-UP 1: There are several do-not-schedule blocks. How do you subtract all of them?
# ========================================================
# ---- follow-up 1: function ----
# Answer: merge both the meetings and the blocks. For each merged meeting
# keep a moving start `current`; for every block that overlaps the meeting,
# keep the part of the meeting before the block, then jump past the block.
# What is left at the end is the last piece. O((n + b) log (n + b)).
def subtract_many(meetings, blocks):
    pieces = []
    merged_blocks = merge_intervals(blocks)
    for start, end in merge_intervals(meetings):
        current = start
        for low, high in merged_blocks:
            if high <= current or low >= end:
                continue                                          # this block does not overlap what is left
            if low > current:
                pieces.append((current, low))
            if high > current:
                current = high
        if current < end:
            pieces.append((current, end))
    return pieces


# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert subtract_many([(1, 10)], [(2, 3), (5, 6)]) == [(1, 2), (3, 5), (6, 10)]
    assert subtract_many([(1, 5), (7, 12)], [(4, 9)]) == subtract_dns([(1, 5), (7, 12)], (4, 9))


# ========================================================
# RUN EVERYTHING
# ========================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

134 Merge Intervals — MEDIUM

Template fit: the template as written

QUESTION

Merge Intervals. An interval is a representation of a range of values along a number line. For example, [1,3] represents all numbers between 1 and 3, inclusive. Given a collection of intervals, the task is to merge all overlapping intervals. For example, consider the intervals [1,3],[2,6],[8,10],[15,18]. The function should return merged intervals where overlapping intervals are combined. In this case, intervals [1,3] and [2,6] overlap, so they should be merged into [1,6]. Examples intervals = [[1,3], [2,6], [8,10], [15,18]] output: [[1,6], [8,10], [15,18]] intervals = [[1,4], [4,5]] output: [[1,5]] intervals = [[1,4], [2,3]] output: [[1,4]] intervals = [[1,2], [3,4], [5,6], [7,8]] output: [[1,2], [3,4], [5,6], [7,8]]The mergeIntervals function takes an array of intervals and merges all overlapping intervals into a single interval. The process involves sorting the intervals by their start times and then iterating through them to merge where necessary. Here is a step-by-step breakdown: Initial Check: If the input array is empty, the function immediately returns an empty array, as there are no intervals to merge. Sorting: The intervals are sorted by their start times. This ensures that intervals can be processed in a linear sweep, merging adjacent intervals when they overlap. Merging Intervals: Initialize a result list with the first interval. Then, iterate through the sorted intervals starting from the second interval. For each interval, compare it with the last interval in th

EXAMPLES

Example 1	
Input	`[[1, 3], [2, 6], [8, 10], [15, 18]]`
Output	`[[1, 6], [8, 10], [15, 18]]`
Why	`[1, 3] and [2, 6] overlap and merge into [1, 6].`

Example 2	
Input	`[[1, 4], [4, 5]]`
Output	`[[1, 5]]`
Why	`Intervals that touch at an endpoint are merged as well.`

Example 3	
Input	`[[8, 10], [1, 3], [2, 4]]`
Output	`[[1, 4], [8, 10]]`
Why	`Unsorted input is fine.`

FULL CODE | main question first, then every follow-up, tests at the end

```
# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ========================================================

import heapq

END = object()


# ========================================================
# MAIN QUESTION
# ========================================================
# ---- main question: main function ----
# Idea: sort by start, then sweep. If an interval starts at or before the
# end of the last output interval, extend that end (to the larger end);
# otherwise append it as a new interval. Because of the sort, nothing earlier
# can overlap the new interval. (touching_ints is follow-up 2.)
def merge_intervals(intervals, touching_ints=False):
    gap = 0
    if touching_ints:
```

```
        gap = 1                                  # [1, 2] and [3, 4] cover 1..4 for whole numbers
    merged = []
    for start, end in sorted(intervals):
        if merged and start <= merged[-1][1] + gap:
            if end > merged[-1][1]:
                merged[-1][1] = end
        else:
            merged.append([start, end])
    return merged


# ---- main question: tests ----
def test_main_question():
    assert merge_intervals([[1, 3], [2, 6], [8, 10], [15, 18]]) == [[1, 6], [8, 10], [15, 18]]
    assert merge_intervals([[1, 4], [4, 5]]) == [[1, 5]]
    assert merge_intervals([]) == []
    assert merge_intervals([[1, 2], [3, 4]]) == [[1, 2], [3, 4]]
    assert merge_intervals([[1, 2], [3, 4]], True) == [[1, 4]]       # touching whole numbers merge


# ========================================================
# FOLLOW-UP 1: Insert one interval into an already merged list (LeetCode 57)
```

```
# ========================================================
# ---- follow-up 1: function ----
# Answer: the list is already merged and sorted, so no sort is needed:
# copy the intervals that end before the new one starts, absorb the ones that
# overlap it, copy the rest. O(n).
def insert_interval(merged, new):
    result = []
    i = 0
    n = len(merged)
    while i < n and merged[i][1] < new[0]:
        result.append(merged[i])                 # 1) everything that ends before the new one starts
        i += 1
    start = new[0]
    end = new[1]
    while i < n and merged[i][0] <= end:          # 2) everything that overlaps it is absorbed
        if merged[i][0] < start:
            start = merged[i][0]
        if merged[i][1] > end:
            end = merged[i][1]
        i += 1
    result.append([start, end])
    while i < n:
        result.append(merged[i])                 # 3) the rest
        i += 1
    return result


# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert insert_interval([[1, 3], [6, 9]], [2, 5]) == [[1, 5], [6, 9]]
    assert insert_interval([], [5, 7]) == [[5, 7]]


# ========================================================
# FOLLOW-UP 2: Intervals of touching integers ([1, 2] and [3, 4]) should merge
# ========================================================
# Answer: the `touching_ints` option: two intervals merge when the next one
# starts at most 1 after the previous end.


# ========================================================
# FOLLOW-UP 3: Too many intervals for memory
# ========================================================
```

```
# ---- follow-up 3: function ----
# Answer: sort externally (an external merge sort) and stream the sweep:
```

```
# it only needs the interval being built in memory. Cost: an O(n log n)
# external sort with bounded memory, then an O(n) pass with O(1) memory.
def merge_stream(sorted_intervals):
    current = None
    for start, end in sorted_intervals:
        if current is not None and start <= current[1]:
            if end > current[1]:
                current[1] = end
        else:
            if current is not None:
                yield tuple(current)
            current = [start, end]
    if current is not None:
        yield tuple(current)


def merge_many_sorted(sources):
    # several files that are each already sorted: merge them with a heap, then sweep
    iterators = []
    for source in sources:
        iterators.append(iter(source))
    heap = []
    for i in range(len(iterators)):
        first = next(iterators[i], END)
        if first is not END:
            heap.append((first, i))
    heapq.heapify(heap)
    in_order = []
    while heap:
        item, i = heapq.heappop(heap)
        in_order.append(item)
        following = next(iterators[i], END)
        if following is not END:
            heapq.heappush(heap, (following, i))
    return list(merge_stream(in_order))


# ---- follow-up 3: tests ----
def test_follow_up_3():
    assert list(merge_stream([(1, 3), (2, 6), (8, 10)])) == [(1, 6), (8, 10)]
    assert merge_many_sorted([[(1, 3), (8, 9)], [(2, 4), (9, 12)]]) == [(1, 4), (8, 12)]


# ============================================================
# RUN EVERYTHING
```

```
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

135 Find safe travel intervals between planet influences HARD

Template fit: turn each planet into (centre - radius, centre + radius), merge, and read the gaps (including the infinite ends)

QUESTION

You are planning a space route along a one-dimensional line (the x-axis). You are given a list of planets. Each planet is represented by an integer pair ((c, r)):

- **c** is the coordinate of the planet's center on the x-axis.
- **r** is a non-negative integer radius of its gravitational influence. A point **x** on the x-axis is **unsafe** for a space traveler if it lies within the gravitational influence of at least one planet: [|x - c_i| \le r_i \quad \text{for some planet } i] Equivalently, each planet makes the interval ([c_i - r_i,\; c_i + r_i]) unsafe. Task:
- Consider the entire real line as the possible travel path.
- Compute all **maximal continuous intervals** on this line where the traveler is **safe**, i.e., points not covered by any planet's unsafe interval.
- Return these safe intervals as a sorted list of disjoint intervals. Representation details:
- Represent each safe interval as a pair **`(start, end)`** with **`start < end`**.
- Intervals may be unbounded on the left or right:
- Use **`-inf`** for an interval unbounded to the left (e.g., **`(-inf, a)`**).
- Use **`+inf`** for an interval unbounded to the right (e.g., **`(b, +inf)`**). You may assume the input list of planets can be in any order and may contain overlapping or nested gravitational ranges. Return the safe intervals in increasing order of **`start`**.

You are planning a route along the real x-axis. Each planet is given as a pair (c, r), where c is its center and r is a non-negative radius of gravitational influence. A point x is unsafe if it lies inside at least one planet's influence, meaning |x - c| <= r. So each planet creates an unsafe closed interval [c - r, c + r]. Your task is to return all maximal continuous safe intervals on the real line: regions not covered by any unsafe interval. Represent each safe interval as a tuple (start, end), where the interval means start < x < end. Because JSON/Python literals do not have a direct infinity literal, use None to represent an unbounded side:

- (None, a) means (-infinity, a)
- (b, None) means (b, +infinity)
- (None, None) means the entire real line Return the safe intervals in increasing order. Overlapping, nested, and touching unsafe intervals should be treated as one merged unsafe region.

Constraints
0 <= len(planets) <= 200000-10^9 <= c <= 10^90 <= r <= 10^9

EXAMPLES

```
Input: []
Expected Output: [(None, None)]
```

Explanation: With no planets, no point is unsafe, so the entire real line is safe.

```
Input: [(3, 1), (10, 2)]
Expected Output: [(None, 2), (4, 8), (12, None)]
Explanation: The unsafe intervals are [2, 4] and [8, 12]. Their complement is (-infinity, 2), (4, 8),
    and (12, +infinity).
```

Hints

First convert every planet (c, r) into the unsafe interval [c - r, c + r].Sort the unsafe intervals and merge all overlapping or touching ones. The safe intervals are the gaps before, between, and after the merged intervals.

FULL CODE | main question first, then every follow-up, tests at the end

```
# ====================================================
# HOW THIS FILE IS ORGANISED
```

```
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

INF = float("inf")


# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
def unsafe_blocks(planets):
    # planet (c, r) makes the stretch [c - r, c + r] unsafe; merge overlapping stretches
    spans = []
    for c, r in planets:
        spans.append((c - r, c + r))
    spans.sort()
    blocks = []
    for start, end in spans:
        if blocks and start <= blocks[-1][1]:            # sharing even one point joins two blocks
            if end > blocks[-1][1]:
                blocks[-1][1] = end
        else:
            blocks.append([start, end])
    return blocks


def gaps_between(blocks):
    # the open intervals between neighbouring blocks, and the two infinite ends
    if not blocks:
        return [(-INF, INF)]
    safe = [(-INF, blocks[0][0])]
    for i in range(len(blocks) - 1):
        safe.append((blocks[i][1], blocks[i + 1][0]))
    safe.append((blocks[len(blocks) - 1][1], INF))
    return safe


# ---- main question: main function ----
# Idea: sort the unsafe stretches by start and merge the ones that overlap or
# touch (they share a point, and that point is unsafe). The safe
# intervals are the gaps between the merged blocks, plus the two infinite ends.
```

```
def safe_intervals(planets):
    return gaps_between(unsafe_blocks(planets))


# ---- main question: tests ----
def test_main_question():
    planets = [(0, 2), (10, 1), (3, 1)]                  # unsafe: [-2, 4] and [9, 11]
    assert safe_intervals(planets) == [(-INF, -2), (4, 9), (11, INF)]
    assert safe_intervals([]) == [(-INF, INF)]
    assert safe_intervals([(1, 1), (3, 1)]) == [(-INF, 0), (4, INF)]      # touching blocks [0, 2] and [2, 4]
 merge


# ============================================================
# FOLLOW-UP 1: Only the part between two given positions matters (a route from A to B)
# ============================================================
# ---- follow-up 1: function ----
# Answer: cut every safe interval to [A, B] and keep the non-empty pieces.
def safe_between(planets, a, b):
    pieces = []
    for low, high in safe_intervals(planets):
        if a > low:
```

```
            low = a
        if b < high:
            high = b
        if low < high:
            pieces.append((low, high))
    return pieces


# ---- follow-up 1: tests ----
def test_follow_up_1():
    planets = [(0, 2), (10, 1), (3, 1)]
    assert safe_between(planets, 5, 20) == [(5, 9), (11, 20)]


# ========================================================
# FOLLOW-UP 2: Are the endpoints safe?
# ========================================================
# Answer: the unsafe stretches are CLOSED ([c - r, c + r]), so the endpoints
# of every gap are unsafe and the safe intervals are open (they leave out
# their ends). If the question says the stretches are open, the endpoints are
# safe and touching stretches no longer merge.


# ========================================================
```

```
# FOLLOW-UP 3: Planets arrive over time
# ========================================================
# ---- follow-up 3: function ----
# Answer: keep the merged unsafe blocks in a sorted list. A new planet
# absorbs every block it overlaps (like the stream-of-intervals question);
# the safe intervals are always the gaps between the blocks.
class LiveRoute:
    def __init__(self):
        self.blocks = []

    def add(self, c, r):
        start = c - r
        end = c + r
        kept = []
        for block_start, block_end in self.blocks:
            if block_end < start or block_start > end:
                kept.append([block_start, block_end])      # does not touch the new block
            else:
                if block_start < start:
                    start = block_start                    # overlaps: absorb it
                if block_end > end:
                    end = block_end
        kept.append([start, end])
        kept.sort()
        self.blocks = kept

    def safe(self):
        return gaps_between(self.blocks)


# ---- follow-up 3: tests ----
def test_follow_up_3():
    planets = [(0, 2), (10, 1), (3, 1)]
    live = LiveRoute()
    for c, r in planets:
        live.add(c, r)
    assert live.safe() == safe_intervals(planets)
    live.add(6, 3)                                         # a big planet at 6 swallows both blocks
    assert live.safe() == [(-INF, -2), (11, INF)]
```

```
# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
```

```
    test_main_question()
    test_follow_up_1()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

136 Implement a function that processes a stream of interval data. HARD

Template fit: the Range Module add: bisect, merge the block, splice in one interval; intervals() returns the pairs

QUESTION

Intervals arrive one at a time. add(start, end) receives a new interval, and intervals() returns the current list of disjoint intervals, sorted by start; intervals that overlap or touch are merged.

Function signature

class IntervalStream:
    def add(self, start: int, end: int) -> None
    def intervals(self) -> list

EXAMPLES

Example 1

Input	`s = IntervalStream()`
	`s.add(1, 3)`
	`s.add(6, 9)`
	`s.intervals()`
	`s.add(2, 5)`
	`s.intervals()`
	`s.add(5, 6)`
	`s.intervals()`

Output	`s.intervals() -> [(1, 3), (6, 9)]`
	`s.intervals() -> [(1, 5), (6, 9)]`
	`s.intervals() -> [(1, 9)]`

Why	`(2, 5) overlaps (1, 3) and extends it to (1, 5); (5, 6) touches (1, 5) at 5 and (6, 9) at 6, so everything merges into (1, 9).`

FULL CODE | main question first, then every follow-up, tests at the end

```
# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

import random


# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
```

```
def first_at_least(values, x):
    # binary search in a sorted list: the first index whose value is >= x
    low = 0
    high = len(values)
    while low < high:
        middle = (low + high) // 2
        if values[middle] < x:
            low = middle + 1
        else:
            high = middle
    return low


def first_greater(values, x):
    # binary search in a sorted list: the first index whose value is > x
    low = 0
    high = len(values)
    while low < high:
        middle = (low + high) // 2
        if values[middle] <= x:
            low = middle + 1
        else:
            high = middle
    return low


# ---- main question: main class ----
# Intervals arrive one at a time; after every add(start, end), intervals()
# must return the merged, sorted list.
# Idea: keep the merged intervals sorted by start (two lists: starts and
# ends). A new interval can only interact with the intervals found by two
```

```
# binary searches: the first interval that touches it and the first one
# completely to its right. Replace that slice by ONE merged interval.
# integer=True: [1, 3] and [4, 5] cover 1..5, so they merge (follow-up 2);
# integer=False: only overlapping intervals or a shared end merge.
# The sorted disjoint intervals also answer follow-up 3: is a point
# covered (contains) and how much is covered in total (total).
class IntervalStream:
    def __init__(self, integer=True):
        self.starts = []
        self.ends = []
        self.gap = 0                            # how far apart two intervals may be and still merge
        self.extra = 0                          # the length of [a, b] is b - a + extra
        if integer:
            self.gap = 1
            self.extra = 1
        self.covered = 0

    def length(self, start, end):
        return end - start + self.extra

    def add(self, start, end):
        if end < start:
            raise ValueError("bad interval")
        i = first_at_least(self.ends, start - self.gap)       # the first interval that touches the new one
        j = first_greater(self.starts, end + self.gap)        # the first one completely to its right
        for k in range(i, j):
            self.covered -= self.length(self.starts[k], self.ends[k])    # the swallowed intervals leave the
total
        if i < j:
            if self.starts[i] < start:
                start = self.starts[i]
            if self.ends[j - 1] > end:
                end = self.ends[j - 1]
        self.starts[i:j] = [start]
```

```
        self.ends[i:j] = [end]
        self.covered += self.length(start, end)              # and the merged one joins it

    def intervals(self):
        result = []
        for i in range(len(self.starts)):
            result.append((self.starts[i], self.ends[i]))
        return result

    def contains(self, point):
        # the interval with the largest start <= point must still reach the point
        i = first_greater(self.starts, point) - 1
        return i >= 0 and self.ends[i] >= point

    def total(self):
        return self.covered


def process(stream, integer=True):
    # feed a stream of (start, end) pairs; give the merged list after each add
    cover = IntervalStream(integer)
    steps = []
    for start, end in stream:
        cover.add(start, end)
        steps.append(cover.intervals())
    return steps


# ---- main question: tests ----
def test_main_question():
    steps = process([(1, 3), (8, 10), (4, 5), (6, 7)])
    assert steps[0] == [(1, 3)]
    assert steps[1] == [(1, 3), (8, 10)]
    assert steps[2] == [(1, 5), (8, 10)]                     # (4, 5) touches (1, 3) for whole numbers
    assert steps[3] == [(1, 10)]                             # (6, 7) touches both sides
    real = process([(1, 3), (4, 5), (3, 4)], False)
    assert real[1] == [(1, 3), (4, 5)]
    assert real[2] == [(1, 5)]


# ========================================================
# FOLLOW-UP 1: How do you keep adds fast for a very large number of intervals?
# ========================================================
# Answer: the lists give O(log n) searches but O(n) inserts (the lists
# shift). For very many intervals use a balanced search tree or a skip list,
# which find and replace a slice in O(log n).


# ========================================================
# FOLLOW-UP 2: Should touching intervals such as [1, 3] and [4, 5] merge?
# ========================================================
# Answer: it depends on whether the values are whole numbers (then they
# cover 1..5 and should merge) or real numbers (then there is a gap between
# 3 and 4). The `integer` option chooses; ask which one is meant.


# ========================================================
```

```
# FOLLOW-UP 3: Answer "is this point covered?" and "how much is covered in total?"
# ========================================================
# ---- follow-up 3: tests ----
# Answer: the sorted disjoint intervals already support both: a point is
# covered if the interval with the largest start at or before it ends at or
# after it (one binary search); the total length is kept up to date on
# every add.
```

```
def test_follow_up_3():
    box = IntervalStream()
    for pair in [(5, 8), (1, 2), (3, 4), (20, 21)]:
        box.add(pair[0], pair[1])
    assert box.intervals() == [(1, 8), (20, 21)]
    assert box.total() == 10
    assert box.contains(7) and box.contains(20)
    assert not box.contains(9) and not box.contains(0)
    random.seed(21)
    for round_number in range(200):                # random intervals: compare with a set of covered points
        box = IntervalStream()
        points = set()
        for i in range(random.randint(1, 8)):
            a = random.randint(0, 30)
            b = a + random.randint(0, 6)
            box.add(a, b)
            for p in range(a, b + 1):
                points.add(p)
            assert box.total() == len(points)
        for p in range(-1, 40):
            assert box.contains(p) == (p in points)


# ====================================================
# RUN EVERYTHING
# ====================================================
def run_all_tests():
    test_main_question()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

GROUP 17 OF 43 | 8 QUESTIONS, ONE CODE TEMPLATE | 7 M 1 H

Trie: nested dicts, one character per level

137 URL Request Handler Router — MEDIUM

Template fit: one trie level per URL segment; a node has literal children and one wildcard child; try the literal first

QUESTION

Design a request router. add(pattern) registers a URL pattern whose segments are either literal text or a placeholder written {name}; match(url) returns the registered pattern that handles the URL, or None. When a URL matches both a literal segment and a placeholder, the literal wins.

Function signature

class Router:
  def add(self, pattern: str) -> None
  def match(self, url: str) -> str | None

EXAMPLES

Example 1

Input	`r = Router()`
	`r.add('/users/{userId}/photo/{photoId}')`
	`r.add('/users/me/photo/{photoId}')`
	`r.match('/users/123/photo/456')`
	`r.match('/users/me/photo/9')`
	`r.match('/orders/1')`

Output	`r.match('/users/123/photo/456') -> '/users/{userId}/photo/{photoId}'`
	`r.match('/users/me/photo/9') -> '/users/me/photo/{photoId}'`
	`r.match('/orders/1') -> null`

Why	The placeholder pattern handles any user id, but for /users/me/... the literal 'me' pattern wins; an unknown path matches nothing.

FULL CODE | main question first, then every follow-up, tests at the end

```
# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
def segments(path):
    # "/users/7/photo" -> ["users", "7", "photo"]
    parts = []
    for piece in path.split("/"):
        if piece != "":
            parts.append(piece)
    return parts


class Node:
    # one node of the trie: one path segment per level
    def __init__(self):
        self.literal = {}          # fixed segment -> child node
        self.wild = None           # the child for a {placeholder} segment
        self.wild_name = None      # the placeholder's name, e.g. "userId"
```

```
        self.pattern = None        # the registered pattern that ends at this node
        self.rest = None           # the pattern with a trailing "*" (matches the rest of the path)


# ---- main question: main function ----
# Idea: store the patterns in a trie with one path SEGMENT per level.
# Every node has literal children plus at most one placeholder child, and
# the pattern is stored at its last node. To look up a URL, try the
# literal child first; if that whole branch fails, back up and try the
# placeholder child. So a literal segment wins over a placeholder.
class Router:
    def __init__(self):
        self.root = Node()

    def add(self, pattern):
        node = self.root
        for segment in segments(pattern):
            if segment == "*":                          # follow-up 2: a trailing * matches the rest of the p
ath
                node.rest = pattern
```

```
                return
            if segment.startswith("{") and segment.endswith("}"):
                if node.wild is None:
                    node.wild = Node()
                node.wild_name = segment[1:-1]
                node = node.wild
            else:
                if segment not in node.literal:
                    node.literal[segment] = Node()
                node = node.literal[segment]
        node.pattern = pattern

    def lookup(self, url):
        # returns (the matching pattern or None, the values of the placeholders)
        parts = segments(url)
        values = {}

        def walk(node, i):
            if i == len(parts):
                return node.pattern
            child = node.literal.get(parts[i])           # 1) the exact segment first
            if child is not None:
                found = walk(child, i + 1)
                if found is not None:
                    return found
            if node.wild is not None:                    # 2) then a placeholder, remembering its value
                values[node.wild_name] = parts[i]
                found = walk(node.wild, i + 1)
                if found is not None:
                    return found
                del values[node.wild_name]               # that branch failed: forget the value
            return node.rest                             # 3) finally a trailing wildcard

        pattern = walk(self.root, 0)
        return pattern, values

    def match(self, url):
        pattern, values = self.lookup(url)
        return pattern


# ---- main question: tests ----
def test_main_question():
    r = Router()
    r.add("/users/{userId}/photo/{photoId}")
```

```
    r.add("/users/me")
    assert r.match("/users/123/photo/456") == "/users/{userId}/photo/{photoId}"
    assert r.match("/users/me") == "/users/me"                      # a literal beats a placeholder
    assert r.match("/users/me/photo/9") == "/users/{userId}/photo/{photoId}"
    assert r.match("/nothing/here") is None
    assert r.match("/users") is None


# ============================================================
# FOLLOW-UP 1: Different HTTP methods per route
# ============================================================
# ---- follow-up 1: function ----
# Answer: keep one trie per method (a dictionary method -> Router), or
# treat the method as the first segment of the path. A lookup uses the
# method as its first step.
class MethodRouter:
    def __init__(self):
        self.routers = {}

    def add(self, pattern, method="GET"):
        if method not in self.routers:
            self.routers[method] = Router()
        self.routers[method].add(pattern)

    def match(self, url, method="GET"):
        if method not in self.routers:
            return None
        return self.routers[method].match(url)


# ---- follow-up 1: tests ----
def test_follow_up_1():
    r = MethodRouter()
    r.add("/orders/{id}", "POST")
    r.add("/orders/list")
    assert r.match("/orders/5", "POST") == "/orders/{id}"
    assert r.match("/orders/5") is None                             # a different method, a different route
    assert r.match("/orders/list") == "/orders/list"


# ============================================================
# FOLLOW-UP 2: A wildcard at the end that matches the rest of the path (/static/*)
# ============================================================
# ---- follow-up 2: tests ----
# Answer: Router.add stores a pattern that ends with "*" on the node
```

```
# BEFORE it (the `rest` field), and the lookup falls back to it when
# nothing more specific matched.
def test_follow_up_2():
    r = Router()
    r.add("/static/*")
    r.add("/static/logo.png")
    assert r.match("/static/css/app.css") == "/static/*"
    assert r.match("/static/logo.png") == "/static/logo.png"        # the more specific pattern wins


# ============================================================
# FOLLOW-UP 3: Return the extracted parameters (id = 7)
# ============================================================
# ---- follow-up 3: tests ----
# Answer: while walking, remember the value of every placeholder
# (and forget it again when a branch fails): Router.lookup returns the
# pattern together with those values.
def test_follow_up_3():
    r = Router()
    r.add("/users/{userId}/photo/{photoId}")
    r.add("/users/me")
```

```
    pattern, values = r.lookup("/users/123/photo/456")
    assert pattern == "/users/{userId}/photo/{photoId}"
    assert values == {"userId": "123", "photoId": "456"}
    assert r.lookup("/users/me") == ("/users/me", {})


# ======================================================
# RUN EVERYTHING
# ======================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

138 Trie Prefix Search / Autocomplete — MEDIUM

Template fit: walk to the prefix node; return the words under it (autocomplete)

QUESTION

Build a trie from a list of words. search_prefix(prefix) returns every word that starts with the prefix, sorted alphabetically (empty list when none).

Function signature

class Trie:
  def insert(self, word: str) -> None
  def search_prefix(self, prefix: str) -> list

EXAMPLES

Example 1

Input	`t = Trie(['car', 'card', 'care', 'cat', 'dog'])`
	`t.search_prefix('car')`
	`t.search_prefix('ca')`
	`t.search_prefix('x')`

Output	`t.search_prefix('car') -> ['car', 'card', 'care']`
	`t.search_prefix('ca') -> ['car', 'card', 'care', 'cat']`
	`t.search_prefix('x') -> []`

Why	`'car' is itself a word that starts with 'car'; the prefix 'x' matches nothing.`

FULL CODE | main question first, then every follow-up, tests at the end

```
# ======================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ======================================================

# ======================================================
# MAIN QUESTION
# ======================================================
# ---- main question: helper functions ----
class Node:
    def __init__(self):
        self.children = {}         # letter -> child node
```

```
        self.freq = 0              # how often the word ending here was inserted
        self.top = []              # the best (negative frequency, word) pairs below this node


# ---- main question: main function ----
# Idea: insert each word into the trie and mark its last node. For a
# query, walk the prefix (a missing step means no words), then collect
# every word below that node and sort them alphabetically.
class Trie:
    def __init__(self, k=3):
        self.root = Node()
        self.k = k                 # how many suggestions to keep per node (follow-up 1)

    def insert(self, word, freq=1):
        node = self.root
        path = [node]
        for ch in word:
            if ch not in node.children:
                node.children[ch] = Node()
            node = node.children[ch]
            path.append(node)
        node.freq += freq
        # follow-up 1: refresh the best-k list on every node of the path
        for n in path:
            kept = []
            for item in n.top:
                if item[1] != word:
                    kept.append(item)
            kept.append((-node.freq, word))
            kept.sort()                                    # most frequent first, then alphabetical
            n.top = kept[:self.k]
```

```
    def find(self, prefix):
        node = self.root
        for ch in prefix:
            if ch not in node.children:
                return None
            node = node.children[ch]
        return node

    def words_below(self, node, text):
        # every word at or below node (spelled `text` so far), in no particular order
        found = []
        stack = [(node, text)]
        while stack:
            current, spelled = stack.pop()
            if current.freq > 0:
                found.append(spelled)
            for ch in current.children:
                stack.append((current.children[ch], spelled + ch))
        return found

    def search_prefix(self, prefix):
        node = self.find(prefix)
        if node is None:
            return []
        return sorted(self.words_below(node, prefix))


# ---- main question: tests ----
def test_main_question():
    t = Trie(k=2)
    for word in ["apple", "app", "apricot", "banana"]:
        t.insert(word)
    assert t.search_prefix("ap") == ["app", "apple", "apricot"]
```

```
    assert t.search_prefix("x") == []
    assert t.search_prefix("") == ["app", "apple", "apricot", "banana"]


# ======================================================
# FOLLOW-UP 1: Return only the best K completions ranked by frequency
# ======================================================
# ---- follow-up 1: function ----
# Answer: every node keeps its best k words (see Trie.insert), so a query
# only has to walk the prefix and read ONE node: O(L + k) instead of
# searching the whole subtree.
```

```
def suggest(trie, prefix):
    node = trie.find(prefix)
    if node is None:
        return []
    words = []
    for negative_frequency, word in node.top:
        words.append(word)
    return words


# ---- follow-up 1: tests ----
def test_follow_up_1():
    t = Trie(k=2)
    for word in ["apple", "app", "apricot", "banana"]:
        t.insert(word)
    t.insert("app", 3)                                  # "app" was typed three more times
    assert suggest(t, "ap") == ["app", "apple"]
    assert suggest(t, "zzz") == []


# ======================================================
# FOLLOW-UP 2: Allow one wrong character (fuzzy match)
# ======================================================
# ---- follow-up 2: function ----
# Answer: walk the trie while counting the letters of the prefix that do
# not match; keep every branch with at most one mismatch. When the whole
# prefix has been read, collect the words below that node.
def fuzzy(trie, prefix):
    found = []
    stack = [(trie.root, "", 0)]
    while stack:
        node, text, wrong = stack.pop()
        if len(text) == len(prefix):
            found += trie.words_below(node, text)
            continue
        for ch in node.children:
            misses = wrong
            if ch != prefix[len(text)]:
                misses += 1
            if misses <= 1:
                stack.append((node.children[ch], text + ch, misses))
    return sorted(found)


# ---- follow-up 2: tests ----
```

```
def test_follow_up_2():
    t = Trie()
    for word in ["apple", "app", "apricot", "banana"]:
        t.insert(word)
    assert "banana" in fuzzy(t, "vana")                 # one wrong letter is allowed
    assert fuzzy(t, "zzzz") == []
    assert fuzzy(t, "app") == ["app", "apple", "apricot"]      # "apr" is one letter away from "app"
```

```
# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

139 Build Prefix Lookup with a Trie

MEDIUM

Template fit: build_trie, then walk to the prefix node and collect every word below it

QUESTION

Build a trie from a list of words. completions(prefix) returns every inserted word that starts with the prefix (including the prefix itself when it is a word), sorted alphabetically, or an empty list when none does.

Function signature

class PrefixTrie:
  def insert(self, word: str) -> None
  def completions(self, prefix: str) -> list

EXAMPLES

Example 1

Input	`t = PrefixTrie(['apple', 'app', 'apply', 'apt', 'bat'])`
	`t.completions('app')`
	`t.completions('ap')`
	`t.completions('z')`

Output	`t.completions('app') -> ['app', 'apple', 'apply']`
	`t.completions('ap') -> ['app', 'apple', 'apply', 'apt']`
	`t.completions('z') -> []`

Why	`'app' is itself a word and a prefix of two more; the prefix 'z' matches nothing.`

FULL CODE | main question first, then every follow-up, tests at the end

```
# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
class Node:
    def __init__(self):
        self.children = {}          # letter -> child node
        self.is_word = False        # does a word end here?
        self.count = 0              # how many different words pass through this node (follow-up 1)


# ---- main question: main function ----
# Idea: a trie stores the words in nested nodes, one letter per level.
```

```
# insert walks down creating missing nodes and marks the last one as a
# word end. For a prefix, walk down one letter at a time (a missing step
# means no completions), then go through everything below that node
# collecting the words.
class PrefixTrie:
    def __init__(self):
        self.root = Node()
        self.size = 0                                       # how many different words

    def find(self, text):
        # the node reached by following `text`, or None
        node = self.root
        for ch in text:
            if ch not in node.children:
                return None
            node = node.children[ch]
        return node

    def insert(self, word):
        existing = self.find(word)
        if existing is not None and existing.is_word:
            return                                          # a repeated word counts once
        node = self.root
        for ch in word:
            if ch not in node.children:
```

```
                node.children[ch] = Node()
            node = node.children[ch]
            node.count += 1
        node.is_word = True
        self.size += 1

    def words_below(self, node, text, limit):
        # the words at or below `node` in alphabetical order, at most `limit` of them.
        # A stack is used; the children are pushed in reverse order so the
        # smallest letter is taken first.
        found = []
        stack = [(node, text)]
        while stack and len(found) < limit:
            current, spelled = stack.pop()
            if current.is_word:
                found.append(spelled)
            for ch in sorted(current.children, reverse=True):
                stack.append((current.children[ch], spelled + ch))
        return found

    def completions(self, prefix):
        # every word that starts with the prefix, in alphabetical order
        node = self.find(prefix)
        if node is None:
            return []
        return self.words_below(node, prefix, self.size + 1)


# ---- main question: tests ----
def build(words):
    trie = PrefixTrie()
    for word in words:
        trie.insert(word)
    return trie


def test_main_question():
    t = build(["apple", "app", "apricot", "banana", "apple"])
    assert t.completions("ap") == ["app", "apple", "apricot"]
    assert t.completions("")[-1] == "banana"
```

```
    assert len(t.completions("")) == 4                # the repeated "apple" counts once
    assert t.completions("x") == []
    assert t.completions("apple") == ["apple"]
```

```
# ============================================================
# FOLLOW-UP 1: How many words start with this prefix?
# ============================================================
# ---- follow-up 1: function ----
# Answer: every node counts the words that pass through it (done in
# insert), so the answer is the counter of the node the prefix walk ends
# at: O(length of the prefix), no search below it.
def count_prefix(trie, prefix):
    if prefix == "":
        return trie.size
    node = trie.find(prefix)
    if node is None:
        return 0
    return node.count


# ---- follow-up 1: tests ----
def test_follow_up_1():
    t = build(["apple", "app", "apricot", "banana", "apple"])
    assert count_prefix(t, "ap") == 3
    assert count_prefix(t, "b") == 1
    assert count_prefix(t, "zz") == 0
    assert count_prefix(t, "") == 4


# ============================================================
# FOLLOW-UP 2: Return only the top 3 completions (by frequency or alphabetically)
# ============================================================
# ---- follow-up 2: function ----
# Answer: for the first k in alphabetical order, stop the walk after k
# words (words_below does, because children are visited in order). For
# the most frequent, store the best k words in every node (see the
# "Trie Prefix Search / Autocomplete" question).
def suggest(trie, prefix, k=3):
    node = trie.find(prefix)
    if node is None:
        return []
    return trie.words_below(node, prefix, k)


# ---- follow-up 2: tests ----
def test_follow_up_2():
    t = build(["apple", "app", "apricot", "banana"])
    assert suggest(t, "ap", 2) == ["app", "apple"]
    assert suggest(t, "ap") == ["app", "apple", "apricot"]
```

```
    assert suggest(t, "q") == []


# ============================================================
# FOLLOW-UP 3: Memory is large (many words with long common prefixes)
# ============================================================
# Answer: use a compressed (radix) trie that stores a whole stretch of
# letters on one edge, so long shared or unshared chains take one node
# instead of one node per letter.


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
```

```
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

140 Greedy Longest-Match Dictionary Tokenizer — MEDIUM

Template fit: the same longest-match walk, with a priority rule to choose between equal-length matches

QUESTION

Phone-screen coding round (R1). Tokenize a text against a dictionary of **`key:id`** entries using greedy longest-match; emit the matched id when a key matches, otherwise output the unmatched character as a literal. One follow-up on changing the tie-breaking / priority rule for which token wins. A second follow-up appears when the dictionary is very large: discuss how to keep the matcher memory- and lookup-efficient at scale.

Input: a text string and a dictionary mapping key (string) -> id.
Sweep the text left-to-right. At each position, find the longest dictionary key that matches a prefix of the remaining text; emit its id. If no key matches at the current position, emit the single character as a literal and advance by one.
Output the concatenation (or list) of ids and literal characters in order.
Follow-up: the interviewer changes the tie-breaking / priority rule for which key wins when multiple match (e.g., not longest but highest-priority by dictionary order). Adapt the data structure so the rule swap is cheap.
Follow-up: the dictionary becomes very large. Discuss how to keep memory and per-position lookup efficient — a compressed / radix trie (or sharding the key space) instead of holding every key in a flat map.

EXAMPLES

Example 1

Input	`t = Tokenizer({'ab': 1, 'abc': 2, 'c': 3})`
	`t.tokenize('abcab')`
	`t.tokenize('abx')`

Output	`t.tokenize('abcab') -> [2, 1]` `t.tokenize('abx') -> [1, 'x']`
Why	`At each position the longest dictionary key wins: 'abc' (not 'ab' followed by 'c'), then 'ab'. A character that starts no key is emitted as it is.`

FULL CODE | main question first, then every follow-up, tests at the end

```
# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: main function ----
# Idea: put every key in a trie with its id at the last node. From each
# text position walk down the trie while the next letter exists and
# remember the LAST (deepest) key that ended on the way: that is the
# longest match. Output its id and skip its length; if no key matches,
# output the character itself and move one position.
# priority(key) (follow-up 1) is an optional rule: a LOWER number wins over
# a longer key.
```

```
class Tokenizer:
    def __init__(self, table, priority=None):
        # table: key -> id
        self.priority = priority
        self.root = {}
        for key in table:
            node = self.root
            for ch in key:
                if ch not in node:
                    node[ch] = {}
                node = node[ch]
            node["$"] = (key, table[key])               # the end of a key stores the key and its id

    def better(self, new, old):
        # should the match `new` replace the match `old` found earlier?
        if old is None:
            return True
        if self.priority is None:
            return True                                 # deeper (longer) matches come later: keep them
        return self.priority(new[0]) < self.priority(old[0])

    def best_at(self, text, i):
        node = self.root
        best = None
        j = i
        while j < len(text) and text[j] in node:
            node = node[text[j]]
```

```
            j += 1
            if "$" in node and self.better(node["$"], best):
                best = node["$"]
        return best

    def tokenize(self, text):
        output = []
        i = 0
        while i < len(text):
            match = self.best_at(text, i)
            if match is None:
                output.append(text[i])                  # no key starts here: keep the character
                i += 1
            else:
                key, ident = match
                output.append(ident)
                i += len(key)
        return output


# a simpler version: one dictionary lookup for each possible length,
# longest first
def tokenize_map(text, table):
    longest = 0
    for key in table:
        if len(key) > longest:
            longest = len(key)
    output = []
    i = 0
    while i < len(text):
        size = longest
        if len(text) - i < size:
            size = len(text) - i
        matched = False
        while size > 0 and not matched:
            if text[i:i + size] in table:
                output.append(table[text[i:i + size]])
                i += size
```

```
                matched = True
            size -= 1
        if not matched:
            output.append(text[i])
            i += 1
    return output
```

```
# ---- main question: tests ----
TABLE = {"ab": 1, "abc": 2, "c": 3}


def test_main_question():
    t = Tokenizer(TABLE)
    assert t.tokenize("abcab") == [2, 1]                # "abc" is longer than "ab", so it wins
    assert t.tokenize("xyz") == ["x", "y", "z"]
    assert t.tokenize("") == []
    assert t.tokenize("cab") == [3, 1]
    assert tokenize_map("abcab", TABLE) == [2, 1]
    for text in ("abcab", "cabcabc", "xabcx", "", "ccc"):
        assert tokenize_map(text, TABLE) == t.tokenize(text)


# =========================================================
# FOLLOW-UP 1: The dictionary is huge
# =========================================================
# ---- follow-up 1: tests ----
# Answer: a flat map of every key wastes memory because keys share
# prefixes. A compressed (radix) trie stores each shared stretch once, or
# shard the keys by their first letters across machines. A different
# priority rule (the `priority` parameter) only changes `better`.
def test_follow_up_1():
    order = list(TABLE)
    by_order = Tokenizer(TABLE, priority=order.index)
    assert by_order.tokenize("abc") == [1, 3]           # "ab" was listed before "abc"


# =========================================================
# RUN EVERYTHING
# =========================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

141 Check if all substrings are dictionary words — MEDIUM

Template fit: for every start walk the trie; a substring of length 3 or more that leaves the trie or lacks END fails the check

QUESTION

You are given a string **`s`** (letters only) and access to an English dictionary **`dict`** (a set of valid words). Return **`true`** if **every contiguous substring** of **`s`** with length **`>= 3`** is a valid dictionary word (i.e., for all **`i, j`** with **`0 <= i <= j < n`** and **`j - i + 1 >= 3`**, **`s[i..j] in dict`**). Otherwise return **`false`**. Clarifications:

- Substring means **contiguous**.
- You may assume dictionary lookups are available (e.g., **`dict.contains(word)`**), or you can preprocess the dictionary. Example:
- If **`s = "cats"`**, you must check **`"cat"`**, **`"ats"`**, and **`"cats"`** (and any other substrings of length >= 3).

QUESTION

EXAMPLES

Example 1	
Input	'abcd', {'abc', 'abcd', 'bcd'}
Output	true
Why	Every substring of length 3 or more (abc, bcd, abcd) is in the dictionary.

Example 2	
Input	'abcd', {'abc', 'abcd'}
Output	false
Why	'bcd' is missing from the dictionary.

FULL CODE | main question first, then every follow-up, tests at the end

```
# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ========================================================

# ========================================================
# MAIN QUESTION
# ========================================================
# ---- main question: helper functions ----
def build_trie(words):
    # nested dictionaries, one letter per level; the key "$" marks a word end
    root = {}
    for word in words:
        node = root
        for ch in word:
            if ch not in node:
                node[ch] = {}
            node = node[ch]
        node["$"] = True
    return root


# ---- main question: main function ----
# Idea: every substring of length >= min_len (3) must be a word. For each
# start position i, walk down the trie along s[i:]. For the first
# min_len - 1 letters only walk; from depth min_len on, the node we are
# at must be marked as a word end. If the walk breaks (no word has this
# prefix) the answer is False when at least min_len letters remain from i,
# because then a substring of that length exists and is not a word.
def all_substrings_words(s, words, min_len=3):
    root = build_trie(words)
    for i in range(len(s)):
        node = root
        depth = 0
        for j in range(i, len(s)):
            ch = s[j]
            depth += 1
            if ch not in node:
                return len(s) - i < min_len
            node = node[ch]
            if depth >= min_len and "$" not in node:
                return False
    return True
```

```
# ---- main question: tests ----
WORDS = {"aaa", "aaaa", "aaaaa"}


def test_main_question():
    assert all_substrings_words("aaaaa", WORDS)
    assert not all_substrings_words("aab", WORDS)            # "aab" is a substring that is not a word
    assert all_substrings_words("ab", set())                 # shorter than 3 letters: nothing to check
    assert all_substrings_words("", WORDS)


# ========================================================
# FOLLOW-UP 1: Return the first failing substring
# ========================================================
# ---- follow-up 1: function ----
# Answer: the same idea, but return the substring itself instead of
# False, which is what a user wants to see. Check the SHORTER lengths
# first so the report is the smallest counterexample. (A length above
# the longest word fails at once.)
def first_failing(s, words, min_len=3):
    words = set(words)
    longest = 0
    for word in words:
        if len(word) > longest:
            longest = len(word)
    for length in range(min_len, len(s) + 1):
        if length > longest:
            return s[:length]
        for i in range(len(s) - length + 1):
            piece = s[i:i + length]
            if piece not in words:
                return piece
    return None


# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert first_failing("aaab", WORDS) == "aab"
    assert first_failing("aab", WORDS) == "aab"
    assert first_failing("aaaaa", WORDS) is None
    assert first_failing("aaaaaa", WORDS) == "aaaaaa"        # longer than every word
    for s in ("aaaaa", "aab", "aaab", "ab", "", "aaaaaa"):
        assert all_substrings_words(s, WORDS) == (first_failing(s, WORDS) is None)
```

```
# ========================================================
# FOLLOW-UP 2: The string is very long
# ========================================================
# Answer: a string of length n has about n^2 substrings, but they only
# need to be checked up to the length of the LONGEST word: a longer window
# fails at once. So at most n * (longest word) windows are walked.


# ========================================================
# FOLLOW-UP 3: The dictionary is huge and sharded
# ========================================================
# Answer: shard the dictionary (for example by the first letters) across
# machines and send each check to the shard that owns the prefix, or keep
# one compressed trie per shard.


# ========================================================
# RUN EVERYTHING
# ========================================================
```

```
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

142 Return dictionary words matching a prefix — MEDIUM

Template fit: no trie needed: sort the words and bisect to the first match, then read forward while startswith(prefix)

QUESTION

You are given a dictionary of words. Return all words that start with a given prefix. Matching is case-sensitive, duplicates in the dictionary are returned as often as they occur, and the result is an empty list when nothing matches. The index returns the matches in alphabetical order; limit (optional) stops after that many words.

Function signature

class PrefixIndex:
  def __init__(self, words: list)
  def matches(self, prefix: str, limit=None) -> list

EXAMPLES

Example 1

Input	`idx = PrefixIndex(['apple', 'app', 'Apple', 'apply', 'apple', 'bat'])`
	`idx.matches('app')`
	`idx.matches('app', limit=2)`
	`idx.matches('cat')`

Output	`idx.matches('app') -> ['app', 'apple', 'apple', 'apply']`
	`idx.matches('app', limit=2) -> ['app', 'apple']`
	`idx.matches('cat') -> []`

Why	`'Apple' starts with a capital letter so it does not match 'app'; 'apple' is listed twice because it is in the dictionary twice.`

FULL CODE | main question first, then every follow-up, tests at the end

```
# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

from bisect import bisect_left, bisect_right, insort


# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: main function ----
# Idea: sort the words once. All words with a given prefix then sit
# together in ONE block. Binary search the start of the block (the first
# word >= prefix) and the end (just after the last word that starts with
# the prefix, found by searching for the prefix followed by the biggest
# possible character). `limit` stops after that many words (follow-up 1).
class PrefixIndex:
    def __init__(self, words):
        self.words = sorted(words)
```

```
    def matches(self, prefix, limit=None):
        start = bisect_left(self.words, prefix)
        end = bisect_right(self.words, prefix + "\uffff")
        found = self.words[start:end]
        if limit is None:
            return found
        return found[:limit]

    # follow-up 2: words can be added while queries run. Keep the list
    # sorted by inserting at the right place (O(n) shifting, fast in practice).
    def add(self, word):
        insort(self.words, word)


# ---- main question: tests ----
def matches_scan(words, prefix):
    # the simple O(n * L) answer: look at every word
    found = []
    for word in words:
        if word.startswith(prefix):
            found.append(word)
    return found
```

```
WORDS = ["apple", "app", "apricot", "Apple", "banana", "app"]


def test_main_question():
    index = PrefixIndex(WORDS)
    assert index.matches("ap") == ["app", "app", "apple", "apricot"]     # duplicates stay; case matters
    assert index.matches("x") == []
    assert index.matches("") == sorted(WORDS)
    assert sorted(matches_scan(WORDS, "ap")) == index.matches("ap")


# ========================================================
# FOLLOW-UP 1: Only the first K matches (autocomplete)
# ========================================================
# ---- follow-up 1: tests ----
# Answer: the block is sorted, so the first K words of the block are the
# first K alphabetically: the `limit` parameter just cuts the block.
def test_follow_up_1():
    index = PrefixIndex(WORDS)
    assert index.matches("ap", limit=2) == ["app", "app"]


# ========================================================
# FOLLOW-UP 2: Words are added while queries run
# ========================================================
# ---- follow-up 2: tests ----
# Answer: keep the list sorted with insort (add above), or use a balanced
# tree; a trie also handles inserts in O(length of the word).
def test_follow_up_2():
    index = PrefixIndex(WORDS)
    index.add("apex")
    assert index.matches("ap") == ["apex", "app", "app", "apple", "apricot"]


# ========================================================
# FOLLOW-UP 3: Case-insensitive matching
# ========================================================
# ---- follow-up 3: function ----
# Answer: store (folded word, original word) pairs sorted by the folded
# word and search with the folded prefix; show the ORIGINAL words.
```

```
# casefold() is the proper way to ignore case.
class CaseInsensitiveIndex:
    def __init__(self, words):
        self.items = []
        for word in words:
            self.items.append((word.casefold(), word))
        self.items.sort()

    def matches(self, prefix):
        key = prefix.casefold()
        i = bisect_left(self.items, (key,))
        found = []
        while i < len(self.items) and self.items[i][0].startswith(key):
            found.append(self.items[i][1])
            i += 1
        return found


# ---- follow-up 3: tests ----
def test_follow_up_3():
    assert CaseInsensitiveIndex(WORDS).matches("APP") == ["app", "app", "Apple", "apple"]


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

143 Implement Trie Insert and Exact-Word Search — MEDIUM

Template fit: insert is build_trie; search is walk and then END in node

QUESTION

Implement a trie with insert(word), search(word) and startsWith(prefix). search is true only when the complete word was inserted earlier (a longer stored word that merely starts with it does not count); startsWith is true when some inserted word begins with the prefix.

Function signature

class Trie:
  def insert(self, word: str) -> None
  def search(self, word: str) -> bool
  def startsWith(self, prefix: str) -> bool

EXAMPLES

QUESTION

Example 1

Input	`t = Trie()`
	`t.insert('apple')`
	`t.search('apple')`
	`t.search('app')`
	`t.startsWith('app')`
	`t.insert('app')`
	`t.search('app')`

Output	`t.search('apple') -> true`
	`t.search('app') -> false`
	`t.startsWith('app') -> true`
	`t.search('app') -> true`

Why	`'app' is only a prefix of 'apple' until it is inserted on its own.`

FULL CODE | main question first, then every follow-up, tests at the end

```
# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
class Node:
    def __init__(self):
        self.children = {}          # letter -> child node
        self.end = False            # does a word end here?


# ---- main question: main function ----
# Idea: each node has children by letter and an "a word ends here" flag.
# insert creates the missing nodes and marks the last one. search walks
# the word and also needs the flag at the end; startsWith only needs the
# walk to succeed.
class Trie:
    def __init__(self):
        self.root = Node()

    def walk(self, text):
        # follow the letters of text; None if the path breaks
        node = self.root
        for ch in text:
            if ch not in node.children:
                return None
            node = node.children[ch]
        return node

    def insert(self, word):
        node = self.root
        for ch in word:
            if ch not in node.children:
                node.children[ch] = Node()
            node = node.children[ch]
        node.end = True

    def search(self, word):
        node = self.walk(word)
```

```
        return node is not None and node.end        # the whole word must have been inserted

    def startsWith(self, prefix):
        return self.walk(prefix) is not None


# ---- main question: tests ----
def test_main_question():
    t = Trie()
    t.insert("apple")
    assert t.search("apple")
    assert not t.search("app")                      # a shared prefix is not a match for search
    assert t.startsWith("app")
    assert not t.startsWith("b")
    t.insert("app")
    assert t.search("app")


# ========================================================
# FOLLOW-UP 1: Support "." as a wildcard in search (LeetCode 211)
# ========================================================
# ---- follow-up 1: function ----
# Answer: on ".", try every child at that level and recurse; any other
# letter follows its single child. The worst case is O(26^L) for a
# pattern of dots on a full trie, so mention the bound.
def search_wild(trie, pattern):
    def go(node, i):
        if i == len(pattern):
            return node.end
        ch = pattern[i]
        if ch == ".":
            for child in node.children.values():
                if go(child, i + 1):
                    return True
            return False
        if ch not in node.children:
            return False
        return go(node.children[ch], i + 1)

    return go(trie.root, 0)


# ---- follow-up 1: tests ----
def test_follow_up_1():
    t = Trie()
    for word in ("bad", "dad", "mad"):
        t.insert(word)
    assert search_wild(t, "b.d") and search_wild(t, ".ad")
    assert not search_wild(t, "b..d")
    assert search_wild(t, "...")
    assert not search_wild(t, "pad")


# ========================================================
# FOLLOW-UP 2: Delete a word
# ========================================================
# ---- follow-up 2: function ----
# Answer: remember the nodes on the path to the word, unmark the last
# one, then walk back up and remove every node that is no longer a word
# end and has no children (nothing else needs it).
def delete(trie, word):
    path = [trie.root]
    for ch in word:
        child = path[-1].children.get(ch)
        if child is None:
            return False                            # the word is not there
```

```
        path.append(child)
    if not path[-1].end:
        return False
    path[-1].end = False
    for i in range(len(word), 0, -1):
        node = path[i]
        if node.end or len(node.children) > 0:
            break                                   # this node is still needed
        del path[i - 1].children[word[i - 1]]
    return True


# ---- follow-up 2: tests ----
def test_follow_up_2():
    t = Trie()
    t.insert("apple")
    t.insert("app")
    assert delete(t, "apple")                       # the shorter word that shares the path stays
    assert t.search("app") and not t.search("apple")
    assert not delete(t, "apple")
    assert delete(t, "app") and not t.startsWith("a")


# ============================================================
```

```
# FOLLOW-UP 3: Unicode or large alphabets
# ============================================================
# Answer: the children are a dictionary keyed by character, so any
# character works as it is. For a huge alphabet or very long words use a
# compressed trie, or hash each node's children into a shared table.


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

144 Implement Longest-Match Text Replacement — HARD

Template fit: at each text position walk the trie as far as it goes and remember the last END; replace the longest match and jump past it

QUESTION

You are given a text string and a dictionary of replacement entries. Each dictionary entry has the form **`token:id`**, where **`token`** is a string pattern and **`id`** is the value that should be emitted when that token is matched. Implement a function that scans the text from left to right and produces an output sequence using greedy longest-prefix matching:

1. At the current text position, find all dictionary tokens that match the substring starting at that position.
2. If at least one token matches, choose the longest matching token, append its corresponding **`id`** to the output, and advance by the length of that token.
3. If no token matches, append the original character at the current position to the output and advance by one character.
4. Continue until the entire text has been processed. Example:

QUESTION

```
text = "abcdxyz" dictionary = { "ab": "1", "abc": "2", "xy": "3" }
```

Output:

```
["2", "d", "3", "z"]
```

Explanation: At index **0**, both **"ab"** and **"abc"** match, so choose the longest token **"abc"** and output **"2"**. Then **"d"** has no match, so output it literally. Then **"xy"** matches and outputs **"3"**. Finally, **"z"** is output literally. Follow-up: How would you adapt your design if the matching priority changed from longest match to a configurable priority rule, such as dictionary insertion order, highest score, or a custom comparator?

Part 1: Longest-Match Text ReplacementGiven a text string and a dictionary that maps token strings to replacement id strings, scan the text from left to right. At each position:

1. Find every token that matches the substring starting at that position.
2. If at least one token matches, choose the longest matching token, append its replacement id to the output, and advance by the token length.
3. If no token matches, append the original character at that position as a one-character string and advance by 1. Return the final output as a list of strings.

Constraints

0 <= len(text) <= 1000000 <= len(dictionary) <= 10000Each token in dictionary is non-empty.The sum of all token lengths is at most 100000.Matching is case-sensitive.

EXAMPLES

```
Input: ("abcdxyz", {"ab": "1", "abc": "2", "xy": "3"})
Expected Output: ["2", "d", "3", "z"]
```

Explanation: Both 'ab' and 'abc' match at index 0, so the longer token 'abc' is chosen.

```
Input: ("aaaaa", {"a": "X", "aa": "Y"})
Expected Output: ["Y", "Y", "X"]
```

Explanation: Greedy longest matching picks 'aa' twice, then the final 'a'.

Hints

A trie lets you test all token prefixes starting at one position without checking every token from scratch.While walking forward from a position, remember the last trie node that represented a complete token.

Part 2: Configurable-Priority Text ReplacementImplement the same left-to-right text replacement process, but the match-selection rule is configurable. You are given: - a text string - a list of replacement entries in insertion order - a priority rule Each replacement entry is a tuple: (token, replacement_id, score). At each text position, find all entries whose token matches the substring starting there. If one or more entries match, choose exactly one using the selected priority rule:

- 'longest': choose the longest matching token; if tied, choose the earlier entry in the list - 'insertion': choose the earliest matching entry in the input list - 'score': choose the matching entry with the highest score; if tied, choose the longer token; if still tied, choose the earlier entry in the list Append the chosen replacement_id to the output and advance by the chosen token length. If nothing matches, append the original character and advance by 1. Return the final output as a list of strings.

Constraints

0 <= len(text) <= 1000000 <= len(entries) <= 10000Each token is non-empty.Tokens in entries are unique.The sum of all token lengths i

FULL CODE | main question first, then every follow-up, tests at the end

```
# ====================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
```

```
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ========================================================

# ========================================================
# MAIN QUESTION
# ========================================================
# ---- main question: main function ----
# Idea: put the tokens in a trie (a tree with one letter per level); the
# last node of a token stores the token and its id. At a text position,
# walk down the trie along the text; every node on the way that stores
# an id is a token that matches here. Take the LONGEST one, output its id,
# and move on by its length. If nothing matches, output the character
# itself and move on by one.
class Replacer:
    def __init__(self, mapping):
        # mapping: token -> id
        self.root = {}
        for token in mapping:
            node = self.root
            for ch in token:
                if ch not in node:
                    node[ch] = {}
                node = node[ch]
            node["$"] = (token, mapping[token])

    def candidates(self, text, i):
        # every token that matches at position i, shortest first (all the word
        # ends along the trie path, not only the last one)
        found = []
        node = self.root
        j = i
        while j < len(text) and text[j] in node:
            node = node[text[j]]
            j += 1
            if "$" in node:
                found.append(node["$"])
        return found

    def replace(self, text, choose=None, word_boundary=False):
        # `choose(candidates)` picks the winner (follow-up 1); by default
        # the longest token. word_boundary=True accepts only whole-word
```

```
        # matches (follow-up 3).
        output = []
        i = 0
        while i < len(text):
            found = self.candidates(text, i)
            if word_boundary:
                whole = []
                for token, ident in found:
                    if is_whole_word(text, i, len(token)):
                        whole.append((token, ident))
                found = whole
            if found:
                if choose is None:
                    token, ident = found[-1]
                else:
                    token, ident = choose(found)
                output.append(ident)
                i += len(token)
            else:
                output.append(text[i])                # nothing matches here: keep the character
                i += 1
        return output
```

```
def is_whole_word(text, start, size):
    # no letter or digit touches the match on either side
    before_ok = start == 0 or not text[start - 1].isalnum()
    end = start + size
    after_ok = end == len(text) or not text[end].isalnum()
    return before_ok and after_ok


# ---- main question: tests ----
TABLE = {"ab": 1, "abc": 2, "c": 3}


def test_main_question():
    r = Replacer(TABLE)
    assert r.replace("abcab") == [2, 1]               # "abc" beats "ab" at position 0
    assert r.replace("xyz") == ["x", "y", "z"]
    assert r.replace("") == []
    assert r.replace("cab") == [3, 1]


# =====================================================
```

```
# FOLLOW-UP 1: The priority is not longest match: insertion order, a score, or a custom comparator
# =====================================================
# ---- follow-up 1: tests ----
# Answer: collect EVERY token that matches at the position (the trie
# already gives all of them along the path) and choose with a separate
# policy function. The trie finds the candidates; the policy decides.
def first_inserted(candidates):
    order = list(TABLE)
    best = candidates[0]
    for candidate in candidates:
        if order.index(candidate[0]) < order.index(best[0]):
            best = candidate
    return best


def test_follow_up_1():
    r = Replacer(TABLE)
    assert r.replace("abc", first_inserted) == [1, 3]          # "ab" was inserted before "abc"


# =====================================================
# FOLLOW-UP 2: The text is huge or the dictionary changes often
# =====================================================
# Answer: the trie walk reads each text position once, so a huge text can
# be processed as a stream. For a changing dictionary, inserting or
# removing a token only changes one path of the trie.


# =====================================================
# FOLLOW-UP 3: Overlapping matches or word boundaries
# =====================================================
# ---- follow-up 3: tests ----
# Answer: word boundaries: accept a match only when no letter touches it
# on either side (the word_boundary flag). Overlapping matches: do not
# skip the matched text; output every candidate for each position.
def test_follow_up_3():
    words = Replacer({"cat": "C"})
    got = words.replace("cat concat cat", None, True)
    assert got[0] == "C" and got[-1] == "C"
    assert "C" not in got[1:-1]


# =====================================================
```

```
# RUN EVERYTHING
# ======================================================
```

```
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

GROUP 18 OF 43 | 8 QUESTIONS, ONE CODE TEMPLATE | 1 E 6 M 1 H

A dict for lookup plus an ordered container for who is next (lazy deletion)

145 Implement Batched Undo/Redo Layer — EASY

Template fit: two stacks (undo, redo) of batches; a batch records (old value, new value) per property; undo applies the old values and moves the batch

QUESTION

You are implementing a simplified document layer for a design tool. The layer stores properties as a **`map<string, string>`**. Implement a class that supports the following operations:

- **`beginBatch()`**: start a batch of edits.
- **`setProperty(key, value)`**: set or overwrite a property inside the current batch.
- **`endBatch()`**: commit the batch as one undoable action.
- **`undo()`**: revert the most recently committed batch.
- **`redo()`**: reapply the most recently undone batch. Requirements:

1. A batch may update the same key multiple times.
2. **`undo()`** must restore the exact document state from before that batch.
3. **`redo()`** must restore the exact document state after that batch.
4. If a property did not exist before the batch, then **`undo()`** should remove that property.
5. If **`undo()`** has been called and then a new batch is committed, the redo history must be cleared.
6. Follow-up: make the history memory-efficient. Instead of storing every intermediate edit in a batch, compact the batch so that each key appears at most once in history while still preserving enough information for correct **`undo()`** and **`redo()`**. A useful compact record for one key is:

```
Change { string key; string value; // final value after the batch string old_value; // value before the ba
tch bool has_old_value; // whether the key existed before the batch }
```

Explain how you would build and use such a compact history so that repeated updates to the same key inside one batch do not waste memory.

Part 1: Implement Batched Undo/Redo for Document PropertiesYou are given a sequence of operations for an initially empty document represented as a map<string, string>. A batch starts with ('begin',), contains zero or more ('set', key, value) operations, and ends with ('end',). Each committed batch must be undoable and redoable as one unit. Operation ('snapshot',) asks you to append the current document to the answer; snapshots may appear while a batch is open and should reflect the live document state. If a key is set multiple times in one batch, undo must restore the value from before the batch began. If a key did not exist before the batch, undo must remove it. If a new non-empty batch is committed after an undo, the redo history is cleared. If undo or redo is impossible, do nothing. If end closes an empty batch, treat it as a no-op.
Constraints
0 <= len(operations) <= 200000The total number of ('set', key, value) operations is at most 200000Keys and values are strings of length 1 to 50Batches are not nestedIf a batch contains no set operations, committing it does not create history

EXAMPLES

```
Input: [('begin',), ('set', 'color', 'red'), ('end',), ('snapshot',), ('undo',), ('snapshot',),
    ('redo',), ('snapshot',)]
Expected Output: [{'color': 'red'}, {}, {'color': 'red'}]
```

Explanation: A single batch is committed, undone, and redone.

QUESTION

```
Input: [('begin',), ('set', 'a', '1'), ('end',), ('begin',), ('set', 'a', '2'), ('set', 'a', '3'),
    ('set', 'b', '9'), ('end',), ('snapshot',), ('undo',), ('snapshot',), ('redo',), ('snapshot',)]
Expected Output: [{'a': '3', 'b': '9'}, {'a': '1'}, {'a': '3', 'b': '9'}]
```

Explanation: The second batch updates 'a' twice and adds 'b'. Undo must restore the exact pre-batch state.

Hints

For each key touched in the current batch, record its old value only the first time that key is edited in the batch.Use two stacks: one for committed batches that can be undone, and one for undone batches that can be redone. Clear the redo stack when a new non-empty batch is committed.

Part 2: Compact a Batch into Memory-Efficient Change RecordsYou are given the document state before a batch and the raw setProperty edits that happened inside that batch. Multiple edits may target the same key. Build a compact history so that each key appears at most once, while still storing enough informa

FULL CODE | main question first, then every follow-up, tests at the end

```python
# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ========================================================

from collections import deque


# ========================================================
# MAIN QUESTION
# ========================================================
# ---- main question: helper functions ----
class Change:
    # one key changed inside one batch: its value before and after
    def __init__(self, key, old, had_old, new):
        self.key = key
        self.old = old              # the value before the batch (if it had one)
        self.had_old = had_old      # did the key exist before the batch?
        self.new = new              # the value at the end of the batch


def merge_batch(parent, child):
    # a nested batch folds into its parent: keep the parent's OLD value
    # when the key was already touched there, but take the newest value
    for key in child:
        if key in parent:
            parent[key].new = child[key].new
        else:
            parent[key] = child[key]


# ---- main question: main function ----
# Idea: while a batch is open, remember for every key touched the value
# it had BEFORE the batch (recorded only the first time the key is
# touched) and its latest value. end_batch puts the batch on the undo
# stack as ONE action and clears the redo stack. Undo restores the old
# values; redo applies the new ones.
# `history` limits how many batches are remembered (follow-up 1): a deque
# with a maximum length drops the oldest batch by itself.
# `open_batches` is a stack, so batches can be nested (follow-up 2).
class Layer:
    def __init__(self, history=None):
        self.props = {}
```

```
        self.undo_stack = deque(maxlen=history)       # committed batches, newest last
        self.redo_stack = []
        self.open_batches = []                        # each open batch: key -> Change

    def begin_batch(self):
        self.open_batches.append({})

    def set_property(self, key, value):
        if len(self.open_batches) == 0:               # outside a batch: it is a batch of one change
            self.begin_batch()
            self.set_property(key, value)
            self.end_batch()
            return
        batch = self.open_batches[-1]
        if key in batch:
            batch[key].new = value                    # touched again: keep the ORIGINAL old value
        else:
            batch[key] = Change(key, self.props.get(key), key in self.props, value)
        self.props[key] = value

    def end_batch(self):
        batch = self.open_batches.pop()
        if len(self.open_batches) > 0:
            merge_batch(self.open_batches[-1], batch)     # a nested batch folds into its parent
        elif len(batch) > 0:
            self.undo_stack.append(list(batch.values()))  # only the OUTERMOST batch is one undoable action
            self.redo_stack.clear()                       # a new action clears the redo history

    def undo(self):
        if len(self.undo_stack) == 0:
            return False
        changes = self.undo_stack.pop()
        for change in changes:
            if change.had_old:
                self.props[change.key] = change.old
            else:
                del self.props[change.key]                # the key did not exist before: remove it
        self.redo_stack.append(changes)
        return True

    def redo(self):
        if len(self.redo_stack) == 0:
            return False
        changes = self.redo_stack.pop()
        for change in changes:
```

```
            self.props[change.key] = change.new
        self.undo_stack.append(changes)
        return True


# ---- main question: tests ----
def test_main_question():
    layer = Layer()
    layer.begin_batch()
    layer.set_property("color", "red")
    layer.set_property("color", "blue")
    layer.set_property("size", "4")
    layer.end_batch()
    assert layer.props == {"color": "blue", "size": "4"}
    assert len(layer.undo_stack[0]) == 2              # each key is stored once, however many edits
    layer.undo()
    assert layer.props == {}
    layer.redo()
    assert layer.props == {"color": "blue", "size": "4"}
    layer.begin_batch()
    layer.set_property("color", "green")
```

```
    layer.end_batch()
    layer.undo()
    assert layer.props["color"] == "blue"             # the old value comes back, not the first value of the
batch
    layer.set_property("x", "1")
    assert not layer.redo()                           # a new batch clears the redo history


# ============================================================
# FOLLOW-UP 1: Limit the history to the last N batches
# ============================================================


# ============================================================
# FOLLOW-UP 4: Limit the history to the last N batches
# ============================================================
# ---- follow-up 1: tests ----
# Answer: keep the undo stack in a deque with maxlen N so the oldest
# batch is dropped automatically (the `history` parameter). Redo is not
# affected.
def test_follow_up_1():
    small = Layer(history=2)
    for i in range(4):
        small.set_property("k", str(i))
    assert len(small.undo_stack) == 2
```

```
    small.undo()
    small.undo()
    assert not small.undo() and small.props["k"] == "1"


# ============================================================
# FOLLOW-UP 2: Nested batches
# ============================================================
# ---- follow-up 2: tests ----
# Answer: keep a STACK of open batches and merge a child batch into its
# parent when it ends (keeping the parent's old value if the key was
# already touched). Only the outermost end_batch commits an undoable action.
def test_follow_up_2():
    layer = Layer()
    layer.begin_batch()
    layer.begin_batch()
    layer.set_property("y", "2")
    layer.end_batch()
    layer.set_property("z", "3")
    layer.end_batch()
    layer.undo()                                      # one undo removes both properties
    assert "y" not in layer.props
    assert "z" not in layer.props


# ============================================================
# FOLLOW-UP 3: Undo of individual setProperty calls, not whole batches
# ============================================================
# Answer: make every set_property its own batch (call begin_batch and
# end_batch around each one), or record one Change per call instead of
# one per key.


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
```

```
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

146 Design a restaurant waitlist system

MEDIUM

Template fit: a segment tree of party sizes gives the first entry that fits in O(log n); the dict maps ids to slots

QUESTION

You are implementing the waitlist system for a restaurant. Parties arrive over time, can cancel, and get seated when a table becomes available.

Rules

- Each party has a unique **`partyId`**, a **`name`**, and a **`size`** (number of people).
- Parties are seated in **arrival order**, but only if they **fit** the available table.
- When a table of capacity **`c`** becomes available, you must seat the **earliest-arrived** party whose **`size <= c`**.
- If no party fits, the table remains unused for that event.

Operations to support Design a

data structure / class that supports the following operations efficiently:

1. **`addParty(name, size) -> partyId`**

- Adds a new party to the end of the waitlist and returns its id.

2. **`cancelParty(partyId) -> bool`**

- Removes the party from the waitlist if present.
- Returns **`true`** if removed, otherwise **`false`**.

3. **`seatTable(capacity) -> partyId or null`**

- Finds and removes the earliest party with **`size <= capacity`**.
- Returns that party's id, or **`null`** if nobody fits.

4. **`getPosition(partyId) -> int`**

- Returns how many parties are ahead of this party in the current waitlist order (0-based).
- If the party is not in the waitlist, return **`-1`**.

Constraints

- Up to **`2 * 10^5`** operations.
- Party size is a small positive integer (e.g., 1–20).
- Aim for better-than-linear time per operation (especially for **`seatTable`** and **`getPosition`**).

Example

(one possible interaction) - addParty("A", 4) -> id1 - addParty("B", 2) -> id2 - seatTable(2) -> id2 (B fits and is earliest among those that fit) - seatTable(4) -> id1

EXAMPLES

QUESTION

Example 1

Input	`w = Waitlist()`
	`w.add(1, 'ann', 4)`
	`w.add(2, 'bob', 2)`
	`w.add(3, 'cy', 3)`
	`w.seat(3)`
	`w.cancel(3)`
	`w.seat(3)`
	`w.seat(4)`

Output	`w.seat(3) -> 2`
	`w.cancel(3) -> true`
	`w.seat(3) -> null`
	`w.seat(4) -> 1`

Why	A table for 3 takes the earliest party that fits: bob (2), not ann (4). After cy cancels, nobody left fits a table for 3; a table for 4 seats ann.

FULL CODE | main question first, then every follow-up, tests at the end

```python
# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ========================================================

from bisect import bisect_right, insort
from collections import deque

INF = float("inf")


# ========================================================
# MAIN QUESTION
# ========================================================
# ---- main question: main function ----
# Idea: seat the EARLIEST party that fits the table. Each arriving party
# gets the next position (its arrival order). A segment tree over these
# positions stores, for each range, the SMALLEST party size waiting in it
# (infinity for empty or already seated positions). To seat a table of
# capacity c: if the smallest size overall is above c nobody fits;
# otherwise go down the tree, always LEFT first when the left half holds a
# party that fits, so we reach the earliest one.
class Waitlist:
    def __init__(self, capacity=1024):
        self.cap = capacity                          # the most parties that can ever join
        self.tree = [INF] * (2 * capacity)           # leaves are tree[cap .. 2 * cap - 1]
        self.count = 0                               # how many parties have joined so far
        self.slot = {}                               # party id -> its position
        self.who = {}                                # position -> party id

    def update(self, position, value):
        i = position + self.cap
        self.tree[i] = value
        while i > 1:
            i //= 2
            left = self.tree[2 * i]
            right = self.tree[2 * i + 1]
            if left < right:
                self.tree[i] = left
            else:
                self.tree[i] = right
```

```
    def add(self, party_id, name, size):
```

```
        self.slot[party_id] = self.count
        self.who[self.count] = party_id
        self.update(self.count, size)
        self.count += 1

    def cancel(self, party_id):
        if party_id not in self.slot:
            return False
        position = self.slot[party_id]
        if self.tree[position + self.cap] == INF:
            return False                            # already seated or cancelled
        self.update(position, INF)
        return True

    def seat(self, capacity):
        if self.tree[1] > capacity:
            return None                             # no waiting party is small enough
        i = 1
        while i < self.cap:
            if self.tree[2 * i] <= capacity:
                i = 2 * i                           # the left half holds a party that fits
            else:
                i = 2 * i + 1
        self.update(i - self.cap, INF)
        return self.who[i - self.cap]


# ---- main question: tests ----
def test_main_question():
    w = Waitlist(8)
    w.add("p1", "Ann", 4)
    w.add("p2", "Bob", 2)
    w.add("p3", "Cy", 6)
    assert w.seat(3) == "p2"                        # a table for 3: the earliest party that fits is Bob
    assert w.cancel("p1") and not w.cancel("p1")
    assert w.seat(6) == "p3"                        # Ann cancelled, so Cy gets the table for 6
    assert w.seat(10) is None


# ========================================================
# FOLLOW-UP 1: Parties that wait too long should be seated first (starvation)
# ========================================================
# Answer: a party that waits long enough gets priority: either age the
# parties (compare arrival time plus a bonus for waiting) or give every
# party a deadline and seat the ones past their deadline first.
```

```



# ========================================================
# FOLLOW-UP 2: Seat the best-fitting party, not the earliest
# ========================================================


# ========================================================
# FOLLOW-UP 4: Seat the best-fitting party (the largest that fits), not the earliest
# ========================================================
# ---- follow-up 2: function ----
# Answer: a different policy. Group the waiting parties by size in queues
# (arrival order inside a size) and keep the sizes in a sorted list. The
# best fit for a table of capacity c is the LARGEST size that is at most c,
# found with a binary search; take the oldest party of that size.
class BestFit:
    def __init__(self):
        self.queues = {}                            # size -> parties of that size, oldest first
```

```
        self.sizes = []                                 # the sizes that have waiting parties, sorted

    def add(self, party_id, size):
        if size not in self.queues:
            self.queues[size] = deque()
        if len(self.queues[size]) == 0:
            insort(self.sizes, size)
        self.queues[size].append(party_id)

    def seat(self, capacity):
        k = bisect_right(self.sizes, capacity)       # sizes[:k] are the sizes that fit
        if k == 0:
            return None
        size = self.sizes[k - 1]
        party = self.queues[size].popleft()
        if len(self.queues[size]) == 0:
            self.sizes.pop(k - 1)
        return party


# ---- follow-up 2: tests ----
def test_follow_up_2():
    b = BestFit()
    for party_id, size in (("a", 2), ("b", 4), ("c", 4), ("d", 6)):
        b.add(party_id, size)
    assert b.seat(5) == "b"                          # a table for 5: the biggest fit is size 4, oldest first
    assert b.seat(5) == "c"
```

```
    assert b.seat(5) == "a"
    assert b.seat(1) is None


# ========================================================
# FOLLOW-UP 3: Estimate the waiting time of a party
# ========================================================
# Answer: use the history of how long tables stay occupied (an average
# per table size) and count how many parties are ahead of this one that
# compete for the same tables; the estimate is that count times the
# average turnover time per table.


# ========================================================
# RUN EVERYTHING
# ========================================================
def run_all_tests():
    test_main_question()
    test_follow_up_2()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

147 Implement Python's itertools.tee

MEDIUM

Template fit: one deque per consumer; whoever runs out pulls the next value from the source and appends it to every queue

QUESTION

Implement itertools.tee: given an iterable and n, return n independent iterators that each produce the same values from the one source. The source must be read only once, and a value must be kept only until every iterator has taken it.

Function signature

def tee_queues(iterable, n=2) -> tuple

EXAMPLES

QUESTION

Example 1

Input	`a, b = tee_queues([1, 2, 3])`
	`next(a)`
	`next(a)`
	`next(b)`
	`list(b)`
	`list(a)`

Output	`next(a) -> 1`
	`next(a) -> 2`
	`next(b) -> 1`
	`list(b) -> [2, 3]`
	`list(a) -> [3]`

Why	Iterator a runs ahead by two values; b still sees every value from the start, and neither disturbs the other.

FULL CODE | main question first, then every follow-up, tests at the end

```
# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

import threading
from collections import deque


# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: main function ----
# Idea: one waiting line (deque) of values per consumer. When a
# consumer's line is empty it pulls the next value from the source,
# gives it to EVERY consumer's line, and then takes its own copy. Each
# consumer is a generator, so values are read from the source only when
# somebody asks for them (it works on endless sources too).
def tee_queues(iterable, n=2):
    source = iter(iterable)
    queues = []
    for i in range(n):
        queues.append(deque())

    def consumer(mine):
        while True:
            if len(mine) == 0:
                try:
                    value = next(source)
                except StopIteration:
                    return
                for q in queues:
                    q.append(value)              # give the new value to everybody
            yield mine.popleft()

    consumers = []
    for q in queues:
        consumers.append(consumer(q))
    return tuple(consumers)


# ---- main question: tests ----
def test_main_question():
```

```
    a, b = tee_queues([1, 2, 3])
    assert next(a) == 1
    assert next(a) == 2
    assert next(b) == 1
    assert list(b) == [2, 3]
    assert list(a) == [3]
    x, y, z = tee_queues("abc", 3)
    assert list(x) == list(y) == list(z) == list("abc")


# ======================================================
# FOLLOW-UP 1: Memory grows when one consumer stalls. Is there a way to free items automatically?
# ======================================================
# ---- follow-up 1: helper functions ----
class Cell:
    # one link of the shared list: a value (once it has been read from the source) and the next link
    def __init__(self):
        self.filled = False
        self.value = None
        self.next = None


# ---- follow-up 1: function ----
# Answer: use ONE shared singly linked list with a pointer per consumer.
# The only references to old cells are the consumers' pointers, so Python's
# garbage collector frees the cells behind the slowest consumer by itself,
# with no bookkeeping.
def tee_linked(iterable, n=2):
    source = iter(iterable)

    def consumer(cell):
        while True:
            if not cell.filled:                    # the first reader to arrive pulls the value
                try:
                    cell.value = next(source)
                except StopIteration:
                    return
                cell.filled = True
                cell.next = Cell()
            value = cell.value
            cell = cell.next
            yield value

    head = Cell()                                  # every reader starts at the same first cell
    consumers = []
```

```
    for i in range(n):
        consumers.append(consumer(head))
    return tuple(consumers)


# ---- follow-up 1: tests ----
def count_forever():
    i = 0
    while True:
        yield i
        i += 1


def test_follow_up_1():
    a, b = tee_linked(range(5))
    assert next(a) == 0 and next(a) == 1               # readers are independent, at any speed
    assert list(b) == [0, 1, 2, 3, 4]
    assert list(a) == [2, 3, 4]
    e1, e2 = tee_linked(count_forever())               # works on an endless source
    assert [next(e1), next(e1), next(e1)] == [0, 1, 2]
    assert next(e2) == 0
```

```
# ============================================================
# FOLLOW-UP 2: Make it safe for several threads
# ============================================================
# ---- follow-up 2: function ----
# Answer: pull from the source under ONE lock, and fill the cell inside
# it. Without the lock two consumers could pull two values at the same
# time and hand them out in different orders.
def tee_threadsafe(iterable, n=2):
    source = iter(iterable)
    lock = threading.Lock()

    def consumer(cell):
        while True:
            with lock:
                if not cell.filled:
                    try:
                        cell.value = next(source)
                    except StopIteration:
                        return
                    cell.filled = True
                    cell.next = Cell()
                value = cell.value
```

```
                cell = cell.next
            yield value

    head = Cell()
    consumers = []
    for i in range(n):
        consumers.append(consumer(head))
    return tuple(consumers)


# ---- follow-up 2: tests ----
def test_follow_up_2():
    results = []

    def read_all(it):
        results.append(list(it))

    a, b, c = tee_threadsafe(range(1000), 3)
    workers = []
    for it in (a, b, c):
        worker = threading.Thread(target=read_all, args=(it,))
        workers.append(worker)
        worker.start()
    for worker in workers:
        worker.join()
    assert len(results) == 3
    for values in results:
        assert values == list(range(1000))             # every thread sees all values in order


# ============================================================
# FOLLOW-UP 3: Why not just copy the iterator?
# ============================================================
# Answer: most iterators (generators, files, network streams) cannot be
# copied, and a copy would run the source's side effects twice or share
# its position. tee reads the source ONCE and buffers what the slower
# consumers still need.


# ============================================================
# RUN EVERYTHING
```

```
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
```

```
    test_follow_up_2()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

148 Store With Cheapest-Item Removal — MEDIUM

Template fit: the order is a heap keyed by (price, timestamp); remove_cheapest pops it

QUESTION

Design a Store that holds items. Each item has a string value , an integer price in cents, and an integer timestamp . Support: add(value, price, timestamp) : insert an item. remove_cheapest() : remove and return the value of the item with the smallest price. If several items share the smallest price, remove the one with the smallest timestamp . Return None if the store is empty. A heap keyed by (price, timestamp) gives O(log n) per operation.

Function signature

```
class Store:
    def __init__(self) -> None:
        ...
    def add(self, value: str, price: int, timestamp: int) -> None:
        ...
    def remove_cheapest(self) -> str | None:
        """Remove and return the value of the lowest-priced item (ties broken by
earliest timestamp)."""
        ...
```

Follow-up (as reported): Improve remove_cheapest from O(log n) to O(1) amortized using the price bound: keep a length-10001 array indexed by price (each slot a queue/count of items) and track the smallest non-empty price index seen so far.

EXAMPLES

Example 1	
Input	`add("a",5,1); add("b",2,2); add("c",8,3); remove_cheapest()`
Output	`"b"`
Why	`"b" has the smallest price (2 cents).`

Example 2	
Input	`add("x",7,10); add("y",7,4); remove_cheapest()`
Output	`"y"`
Why	`Prices tie at 7; "y" wins on the earlier timestamp (4 < 10).`

Example 3	
Input	`remove_cheapest() # on an empty store`
Output	`None`

CONSTRAINTS

- Prices are integers with 1 <= price <= 10000 (i.e. 1 cent up to $100.00).

QUESTION

- Up to 10^6 operations.

FULL CODE | main question first, then every follow-up, tests at the end

```
# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ========================================================

import heapq


# ========================================================
# MAIN QUESTION
# ========================================================
# ---- main question: main function ----
# Idea: a min-heap of (price, timestamp, value). The smallest tuple is
# always on top: the cheapest item, and among equal prices the one with
# the earliest timestamp.
class Store:
    def __init__(self):
        self.heap = []

    def add(self, value, price, timestamp):
        heapq.heappush(self.heap, (price, timestamp, value))

    def remove_cheapest(self):
        if len(self.heap) == 0:
            return None
        price, timestamp, value = heapq.heappop(self.heap)
        return value


# ---- main question: tests ----
def test_main_question():
    s = Store()
    s.add("a", 5, 1)
    s.add("b", 2, 2)
    s.add("c", 8, 3)
    assert s.remove_cheapest() == "b"
    s = Store()
    s.add("x", 7, 10)
    s.add("y", 7, 4)
    assert s.remove_cheapest() == "y"                # the same price: the earlier timestamp
    assert Store().remove_cheapest() is None
```

```
# ========================================================
# FOLLOW-UP 1: Prices are integers from 0 to 10000. Can removal be O(1) amortized?
# ========================================================
# ---- follow-up 1: function ----
# Answer: use an array with one bucket per price (10001 of them), each a
# small heap ordered by timestamp, and a pointer `low` to the lowest price
# that might still have items. Removal moves the pointer up past empty
# buckets; the pointer only moves down when a cheaper item is added, so
# the pointer work is O(1) amortized for a fixed range of prices.
class BucketStore:
    MAX_PRICE = 10000

    def __init__(self):
        self.buckets = []
        for price in range(self.MAX_PRICE + 1):
            self.buckets.append([])
```

```
        self.low = self.MAX_PRICE + 1

    def add(self, value, price, timestamp):
        heapq.heappush(self.buckets[price], (timestamp, value))
        if price < self.low:
            self.low = price

    def remove_cheapest(self):
        while self.low <= self.MAX_PRICE and len(self.buckets[self.low]) == 0:
            self.low += 1
        if self.low > self.MAX_PRICE:
            return None
        timestamp, value = heapq.heappop(self.buckets[self.low])
        return value


# ---- follow-up 1: tests ----
def test_follow_up_1():
    s = BucketStore()
    s.add("a", 5, 1)
    s.add("b", 2, 2)
    s.add("c", 8, 3)
    assert s.remove_cheapest() == "b"
    s.add("d", 5, 0)
    assert s.remove_cheapest() == "d"               # price 5 twice: the earlier timestamp
    assert s.remove_cheapest() == "a"
    assert s.remove_cheapest() == "c"
    assert s.remove_cheapest() is None
```

```
# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

149 Round-Robin Job Scheduler — MEDIUM

Template fit: a deque of [job, time left]; pop, run for min(quantum, left); push back at the end if unfinished, else record the finish time

QUESTION

The third question was "not really leetcode style" but was a job-scheduling problem solved with a round- robin approach. Reconstruct it as follows. You are given a list of jobs, each with a unique string id and a positive integer burst (the number of time units of CPU it needs). All jobs are available at time 0 and are placed in a ready queue in the given order. The CPU runs jobs round-robin with a fixed time quantum : it takes the job at the front of the queue, runs it for min(quantum, remaining) time units, and then — if the job still has work left — puts it at the back of the queue; otherwise the job completes at that moment. Return the list of (id, completion_time) pairs in the order the jobs complete.

Function signature

def round_robin(jobs: list[tuple[str, int]], quantum: int) -> list[tuple[str, int]]:
    """Simulate round-robin scheduling; return (id, completion_time) pairs in completion order."""
    ...

QUESTION

Follow-up (as reported): Report note ("a constraint was changed"): add per-job arrival times so a job only enters the queue once the clock reaches its arrival. Hint: keep a pointer over arrival-sorted jobs and admit them each time the clock advances.

EXAMPLES

Example 1	
Input	`jobs = [("A", 4), ("B", 3), ("C", 2)], quantum = 2`
Output	`[("C", 6), ("A", 8), ("B", 9)]`
Why	`t=0-2 A (rem 2), t=2-4 B (rem 1), t=4-6 C completes at 6, t=6-8 A completes at 8, t=8-9 B completes at 9.`

Example 2	
Input	`jobs = [("X", 5)], quantum = 2`
Output	`[("X", 5)]`
Why	`One job, run in slices 2+2+1, finishing at time 5.`

Example 3	
Input	`jobs = [("P", 1), ("Q", 1)], quantum = 3`
Output	`[("P", 1), ("Q", 2)]`
Why	`Each finishes within a single quantum: P at 1, Q at 2.`

CONSTRAINTS

- 1 <= len(jobs) <= 10^4
- 1 <= burst <= 10^6 , 1 <= quantum <= 10^6
- Job ids are unique, non-empty strings.

FULL CODE | main question first, then every follow-up, tests at the end

```
# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ========================================================

from collections import deque
import random


# ========================================================
# MAIN QUESTION
# ========================================================
# ---- main question: main function ----
# Idea: a queue of jobs. Take the job at the front, run it for one time
# slice (at most `quantum`), and if it is not finished put it at the
# BACK of the queue. Record the time each job finishes.
# jobs is a list of (id, burst time); the result is a list of
# (id, finish time) in the order the jobs finish.
def round_robin(jobs, quantum):
    queue = deque()
    for job_id, burst in jobs:
        queue.append([job_id, burst])
    clock = 0
    finished = []
    while queue:
        job = queue.popleft()
        run = quantum
```

```
        if job[1] < run:
            run = job[1]
        clock += run
        job[1] -= run
        if job[1] > 0:
            queue.append(job)                       # not finished: back of the line
        else:
            finished.append((job[0], clock))
    return finished


# ---- main question: tests ----
def test_main_question():
    assert round_robin([("A", 4), ("B", 3), ("C", 2)], 2) == [("C", 6), ("A", 8), ("B", 9)]
    assert round_robin([("X", 5)], 2) == [("X", 5)]
    assert round_robin([("P", 1), ("Q", 1)], 3) == [("P", 1), ("Q", 2)]
```

```
    assert round_robin([], 2) == []


# ========================================================
# FOLLOW-UP 1: Each job has an arrival time. How do you change it?
# ========================================================
# ---- follow-up 1: helper functions ----
def arrival_of(job):
    return job[2]                                   # a job is (id, burst, arrival)


# ---- follow-up 1: function ----
# Answer: sort the jobs by arrival. Before every pick, admit all jobs whose
# arrival time has been reached; if the line is empty, jump the clock to
# the next arrival. A job that used its slice but is not finished goes to
# the back AFTER the jobs that arrived while it was running.
def round_robin_arrivals(jobs, quantum):
    pending = deque(sorted(jobs, key=arrival_of))
    queue = deque()
    clock = 0
    finished = []
    while pending or queue:
        if not queue and pending[0][2] > clock:
            clock = pending[0][2]                   # nothing to run: wait for the next arrival
        while pending and pending[0][2] <= clock:
            job_id, burst, arrival = pending.popleft()
            queue.append([job_id, burst])
        job = queue.popleft()
        run = quantum
        if job[1] < run:
            run = job[1]
        clock += run
        job[1] -= run
        while pending and pending[0][2] <= clock:    # jobs that arrived while this one ran line up before it
            job_id, burst, arrival = pending.popleft()
            queue.append([job_id, burst])
        if job[1] > 0:
            queue.append(job)
        else:
            finished.append((job[0], clock))
    return finished


# ---- follow-up 1: tests ----
def test_follow_up_1():
```

```
    random.seed(3)
    for round_number in range(300):
        jobs = []
```

```
        for i in range(random.randint(1, 6)):
            jobs.append((str(i), random.randint(1, 7)))
        quantum = random.randint(1, 4)
        with_arrivals = []
        for job_id, burst in jobs:
            with_arrivals.append((job_id, burst, 0))
        assert round_robin_arrivals(with_arrivals, quantum) == round_robin(jobs, quantum)   # all arrive at t
ime 0
    assert round_robin_arrivals([("A", 3, 0), ("B", 1, 5)], 2) == [("A", 3), ("B", 6)]


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

150 Aggregate FizzBuzz Offer Matches

MEDIUM

Template fit: a deque of offers per type; a new offer matches against the front of the opposite queue, partial quantities stay at the front

QUESTION

You are given a list of offers in priority order (earliest = highest priority). Each offer is a dict with keys type (either "Fizz" or "Buzz"), name (a string), and quantity (a positive integer). Fizz offers match against Buzz offers on a first-come, first-served basis, and offers may be partially fulfilled. Process the offers in order, maintaining a FIFO queue of unmatched Fizz offers and a FIFO queue of unmatched Buzz offers. When a new offer arrives, match it against the front of the opposite queue repeatedly, each time consuming min(remaining quantities) and recording a match (fizz_name, buzz_name, matched_quantity) ; drop an offer from the queue once its quantity hits 0. If the arriving offer still has quantity left when the opposite queue empties, enqueue it in its own queue. Aggregate matches so the same (fizz_name, buzz_name) pair is reported once with its summed quantity. Return the aggregated matches, in the order each pair was first produced, formatted as: "FizzBuzz! <quantity> <FizzName>:<BuzzName>" .

Function signature

def fizzbuzz_matches(offers: list[dict]) -> list[str]:

...

EXAMPLES

Example 1	
Input	`offers = [ {"type": "Fizz", "name": "A", "quantity": 5}, {"type": "Buzz", "name": "X", "quantity": 3}, {"type": "Buzz", "name": "Y", "quantity": 4}, ]`
Output	`["FizzBuzz! 3 A:X", "FizzBuzz! 2 A:Y"]`
Why	A(5) waits. X(3) matches A -> (A,X,3), A now 2. Y(4) matches A's remaining 2 -> (A,Y,2); Y's leftover 2 waits with no Fizz to match.

Example 2	
Input	`offers = [ {"type": "Fizz", "name": "A", "quantity": 3}, {"type": "Buzz", "name": "X", "quantity": 1}, {"type": "Buzz", "name": "X", "quantity": 2}, ]`
Output	`["FizzBuzz! 3 A:X"]`
Why	A matches the first X for 1 and the second X for 2. Both are the pair (A, X), so they aggregate to a single "FizzBuzz! 3 A:X".

QUESTION

CONSTRAINTS

- 0 <= len(offers) <= 10**4 ; quantities are positive integers.

FULL CODE | main question first, then every follow-up, tests at the end

```
# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ========================================================

from collections import deque
import random


# ========================================================
# MAIN QUESTION
# ========================================================
# ---- main question: helper functions ----
def other_side(kind):
    if kind == "Fizz":
        return "Buzz"
    return "Fizz"


def match_offers(offers):
    # Each offer is {"type": "Fizz" or "Buzz", "name": ..., "quantity": ...}.
    # Offers are handled in order. A new offer is matched against the
    # OPPOSITE side's waiting line, front first, for as long as it has
    # quantity left and the line is not empty; the rest of the offer
    # waits in its own line. Returns the matched totals per
    # (fizz name, buzz name) pair and the two waiting lines.
    queues = {"Fizz": deque(), "Buzz": deque()}       # each entry is [name, quantity left]
    totals = {}
    for offer in offers:
        kind = offer["type"]
        mine = queues[kind]
        other = queues[other_side(kind)]
        left = offer["quantity"]
        while left > 0 and len(other) > 0:
            front = other[0]
            matched = left
            if front[1] < matched:
                matched = front[1]
            if kind == "Fizz":
                pair = (offer["name"], front[0])
            else:
                pair = (front[0], offer["name"])
            totals[pair] = totals.get(pair, 0) + matched
```

```
            left -= matched
            front[1] -= matched
            if front[1] == 0:
                other.popleft()
        if left > 0:
            mine.append([offer["name"], left])
    return totals, queues


# ---- main question: main function ----
# Idea: two FIFO queues (one per side) and an ordered total per pair of
# names. Matching an offer against the front of the opposite queue until
# the offer or the queue is used up is match_offers above; this
# function turns the totals into the lines "FizzBuzz! quantity fizz:buzz".
```

```
def fizzbuzz_matches(offers):
    totals, queues = match_offers(offers)
    lines = []
    for fizz_name, buzz_name in totals:
        lines.append("FizzBuzz! " + str(totals[(fizz_name, buzz_name)]) + " " + fizz_name + ":" + buzz_name)
    return lines


# ---- main question: tests ----
def test_main_question():
    offers = [{"type": "Fizz", "name": "A", "quantity": 5},
              {"type": "Buzz", "name": "X", "quantity": 3},
              {"type": "Buzz", "name": "Y", "quantity": 4}]
    assert fizzbuzz_matches(offers) == ["FizzBuzz! 3 A:X", "FizzBuzz! 2 A:Y"]
    offers2 = [{"type": "Fizz", "name": "A", "quantity": 3},
               {"type": "Buzz", "name": "X", "quantity": 1},
               {"type": "Buzz", "name": "X", "quantity": 2}]
    assert fizzbuzz_matches(offers2) == ["FizzBuzz! 3 A:X"]      # the same pair is added up
    assert fizzbuzz_matches([]) == []


# ========================================================
# FOLLOW-UP 1: After processing, report which offers were not fully matched
# ========================================================
# ---- follow-up 1: function ----
# Answer: the two waiting lines ARE the state: whatever is left in them at
# the end is the unmatched quantity of each offer, in priority order.
# Return both lines.
def unmatched_after(offers):
    totals, queues = match_offers(offers)
    result = {}
```

```
    for side in queues:
        result[side] = []
        for name, quantity in queues[side]:
            result[side].append((name, quantity))
    return result


# ---- follow-up 1: tests ----
def test_follow_up_1():
    offers = [{"type": "Fizz", "name": "A", "quantity": 5},
              {"type": "Buzz", "name": "X", "quantity": 3}]
    assert unmatched_after(offers) == {"Fizz": [("A", 2)], "Buzz": []}
    random.seed(63)
    for round_number in range(400):                  # matched + unmatched must add up to what was offered
        offers = []
        for i in range(random.randint(0, 8)):
            offers.append({"type": random.choice(["Fizz", "Buzz"]),
                           "name": random.choice("ABCDE"),
                           "quantity": random.randint(1, 5)})
        left = unmatched_after(offers)
        matched = 0
        for line in fizzbuzz_matches(offers):
            matched += int(line.split()[1])
        offered = {"Fizz": 0, "Buzz": 0}
        for offer in offers:
            offered[offer["type"]] += offer["quantity"]
        for side in ("Fizz", "Buzz"):
            unmatched = 0
            for name, quantity in left[side]:
                unmatched += quantity
            assert unmatched == offered[side] - matched


# ========================================================
```

```
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

151 Restaurant Waitlist: join / delete / find_first_match — MEDIUM

Template fit: a deque per party size with tickets; delete only removes the name from the dict; find_first_match skips dead tickets

QUESTION

Design a restaurant waitlist. join(name, size) adds a party with its size at the back. delete(name) removes a party (it may be anywhere in the queue). find_first_match(table) returns the name of the party that joined earliest among those whose size fits the table (size at most table), or None.

Function signature

class Waitlist:
    def join(self, name: str, size: int) -> None
    def delete(self, name: str) -> None
    def find_first_match(self, table: int) -> str | None

EXAMPLES

Example 1

Input	`w = Waitlist()`
	`w.join('ann', 4)`
	`w.join('bob', 2)`
	`w.join('cy', 3)`
	`w.find_first_match(3)`
	`w.delete('bob')`
	`w.find_first_match(3)`
	`w.find_first_match(1)`

Output	`w.find_first_match(3) -> 'bob'`
	`w.find_first_match(3) -> 'cy'`
	`w.find_first_match(1) -> null`

Why	A table for 3 skips ann (4 is too big) and takes bob, who joined before cy. After bob leaves it is cy. No party fits a table for 1.

FULL CODE | main question first, then every follow-up, tests at the end

```
# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

from collections import deque


# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: main function ----
# Idea: one queue per party size, plus a dictionary of the people who
```

```
# are still waiting. join puts (arrival number, name) in the queue of its
# size. delete only removes the person from the dictionary ("lazy
# deletion": the old entry stays in its queue and is skipped later).
# find_first_match looks at the front of every queue whose size fits the
# table (after removing deleted people from the front) and returns the
# earliest arrival among them. It does not seat anybody.
class Waitlist:
    def __init__(self):
        self.counter = 0                          # arrival number: lower means earlier
        self.waiting = {}                         # name -> arrival number (people still waiting)
        self.queues = {}                          # party size -> deque of (arrival number, name)

    def join(self, name, size):
        self.counter += 1
        self.waiting[name] = self.counter
        if size not in self.queues:
            self.queues[size] = deque()
        self.queues[size].append((self.counter, name))

    def delete(self, name):
        if name in self.waiting:
            del self.waiting[name]

    def front(self, queue):
        # drop deleted (or re-joined) people from the front, then look at the first one
        while queue:
            number, name = queue[0]
            if self.waiting.get(name) == number:
                return queue[0]
            queue.popleft()
        return None

    def find_first_match(self, table):
        best = None
        for size in self.queues:
            if size > table:
                continue                          # this party is too big for the table
            head = self.front(self.queues[size])
            if head is not None and (best is None or head[0] < best[0]):
                best = head
        if best is None:
            return None
        return best[1]


# ---- main question: tests ----
def test_main_question():
    w = Waitlist()
    w.join("Alice", 2)
    w.join("Bob", 4)
    w.join("Carol", 6)
    assert w.find_first_match(3) == "Alice"
    assert w.find_first_match(5) == "Alice"       # Alice is still the earliest who fits 5
    assert w.find_first_match(5) == "Alice"       # finding does not remove anyone
    w.delete("Alice")
    assert w.find_first_match(5) == "Bob"
    assert w.find_first_match(1) is None
    w.join("Dan", 4)
    w.delete("Bob")
    assert w.find_first_match(4) == "Dan"         # the same size: arrival order decides
    w.delete("Nobody")                            # deleting someone who is not there is harmless


# ================================================
# FOLLOW-UP 1: What would you cache to speed up repeated finds?
```

```
# ============================================================
# Answer: cache the answer per table size and clear (or update) it when a
# party joins or is deleted; or keep, for each size, a pointer to its
# current front so repeated finds do not skip deleted entries again.


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
```

```
    test_main_question()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

152 Implement LRU Cache. HARD

Template fit: dict key -> node plus a doubly linked list; every get/put moves the node to the front; evict from the back (or use OrderedDict.move_to_end)

QUESTION

Your program keeps re-reading the same data from disk, so you want an in-memory cache that holds only the n most recently used items (a least-recently-used cache). Write a class LRUCache(n) with get(key), which returns the stored value or -1, and put(key, value), which stores a value and, when the cache is full, first removes the item that was used least recently. Both get and put count as a use. Both must run in O(1).

Function signature

class LRUCache:
  def __init__(self, n: int)
  def get(self, key) -> int
  def put(self, key, value) -> None

EXAMPLES

Example 1

Input	`c = LRUCache(2)`
	`c.put(1, 1)`
	`c.put(2, 2)`
	`c.get(1)`
	`c.put(3, 3)`
	`c.get(2)`
	`c.put(4, 4)`
	`c.get(1)`
	`c.get(3)`
	`c.get(4)`

Output	`c.get(1) -> 1`
	`c.get(2) -> -1`
	`c.get(1) -> -1`
	`c.get(3) -> 3`
	`c.get(4) -> 4`

Why	`get(1) makes key 1 the most recent, so put(3, 3) evicts key 2; then put(4, 4) evicts key 1 (older than 3).`

FULL CODE | main question first, then every follow-up, tests at the end

```python
# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ========================================================

import threading
import time
from collections import OrderedDict


# ========================================================
# MAIN QUESTION
# ========================================================
# ---- main question: helper functions ----
class Node:
    # one entry of the doubly linked list
    def __init__(self, key=None, value=None):
        self.key = key
        self.value = value
        self.prev = None
        self.next = None


# ---- main question: main function ----
# Idea: a dictionary maps each key to its node in a doubly linked list
# that is ordered by recency (most recently used at the FRONT). Two
# dummy nodes, `head` and `tail`, sit at the ends so no operation has
# to treat an empty list or the ends specially. get and put are O(1).
class LRUCache:
    def __init__(self, n):
        self.capacity = n
        self.nodes = {}                                 # key -> node
        self.head = Node()
        self.tail = Node()
        self.head.next = self.tail
        self.tail.prev = self.head

    def unlink(self, node):
        node.prev.next = node.next
        node.next.prev = node.prev

    def push_front(self, node):
        node.next = self.head.next
```

```python
        node.prev = self.head
        self.head.next.prev = node
        self.head.next = node

    def get(self, key):
        if key not in self.nodes:
            return -1
        node = self.nodes[key]
        self.unlink(node)                               # reading makes it the most recently used
        self.push_front(node)
        return node.value

    def put(self, key, value):
        if key in self.nodes:
            node = self.nodes[key]
            node.value = value
            self.unlink(node)
            self.push_front(node)
            return
        if len(self.nodes) == self.capacity:
            oldest = self.tail.prev                     # full: evict the least recently used (at the back)
```

```
            self.unlink(oldest)
            del self.nodes[oldest.key]
        node = Node(key, value)
        self.nodes[key] = node
        self.push_front(node)


# ---- main question: tests ----
def test_main_question():
    c = LRUCache(2)
    c.put(1, 1)
    c.put(2, 2)
    assert c.get(1) == 1
    c.put(3, 3)                                    # 2 is now the least recently used, so 3 evicts it
    assert c.get(2) == -1 and c.get(3) == 3
    c.put(1, 10)
    assert c.get(1) == 10


# ======================================================
# FOLLOW-UP 1: Many threads share the cache
# ======================================================
# ---- follow-up 1: function ----
# Answer: put get and put behind ONE lock (even get is a write, because it
```

```
# moves the node). For more throughput shard the cache by the hash of the
# key, each shard with its own lock and its own capacity.
class SafeLRU:
    def __init__(self, n):
        self.cache = LRUCache(n)
        self.lock = threading.Lock()

    def get(self, key):
        with self.lock:
            return self.cache.get(key)

    def put(self, key, value):
        with self.lock:
            self.cache.put(key, value)


# ---- follow-up 1: tests ----
def test_follow_up_1():
    s = SafeLRU(1)
    s.put("a", 1)
    s.put("b", 2)
    assert s.get("a") == -1 and s.get("b") == 2
    workers = []

    def hammer(start):
        for i in range(start, start + 300):
            s.put(i, i)
            s.get(i)

    for start in (0, 1000, 2000):
        worker = threading.Thread(target=hammer, args=(start,))
        workers.append(worker)
        worker.start()
    for worker in workers:
        worker.join()
    assert len(s.cache.nodes) == 1                 # the capacity held


# ======================================================
# FOLLOW-UP 2: Entries expire after a time to live (TTL)
# ======================================================
```

```
# ---- follow-up 2: function ----
# Answer: store an expiry time with each entry and check it in get: a
# stale entry counts as missing and is removed. (Optionally also sweep the
# old end of the list on put.) The recency order and the O(1) costs stay
```

```
# the same. `clock` is a function that returns the current time, so tests
# can use a fake one.
class TTLCache(LRUCache):
    def __init__(self, n, ttl, clock=time.monotonic):
        LRUCache.__init__(self, n)
        self.ttl = ttl
        self.clock = clock
        self.expiry = {}                            # key -> the time it expires

    def get(self, key):
        if key in self.nodes and self.expiry[key] <= self.clock():
            self.unlink(self.nodes[key])            # stale: remove it
            del self.nodes[key]
            del self.expiry[key]
        return LRUCache.get(self, key)

    def put(self, key, value):
        LRUCache.put(self, key, value)
        self.expiry[key] = self.clock() + self.ttl
        for k in list(self.expiry):                 # forget the expiry of keys that were evicted
            if k not in self.nodes:
                del self.expiry[k]


# ---- follow-up 2: tests ----
class FakeClock:
    def __init__(self):
        self.now = 0

    def __call__(self):
        return self.now


def test_follow_up_2():
    clock = FakeClock()
    t = TTLCache(2, 10, clock)
    t.put("x", 1)
    assert t.get("x") == 1
    clock.now = 11
    assert t.get("x") == -1                         # expired
    t.put("y", 2)
    assert t.get("y") == 2


# ========================================================
```

```
# FOLLOW-UP 3: Least frequently used (LFU) instead of least recently used
# ========================================================
# ---- follow-up 3: function ----
# Answer: keep a use count per key and "buckets": for each count, the keys
# with that count in order of use (an OrderedDict), plus a pointer to the
# lowest count in use. get and put move a key to the next bucket in O(1);
# eviction takes the OLDEST key of the lowest bucket (LeetCode 460).
class LFUCache:
    def __init__(self, n):
        self.capacity = n
        self.value = {}                             # key -> value
        self.freq = {}                              # key -> how often it was used
        self.buckets = {}                           # count -> keys with that count, oldest first
        self.lowest = 0
```

```
    def bucket(self, count):
        if count not in self.buckets:
            self.buckets[count] = OrderedDict()
        return self.buckets[count]

    def touch(self, key):
        count = self.freq[key]
        del self.bucket(count)[key]
        if len(self.bucket(count)) == 0 and self.lowest == count:
            self.lowest += 1                        # the lowest bucket is empty now
        self.freq[key] = count + 1
        self.bucket(count + 1)[key] = None

    def get(self, key):
        if key not in self.value:
            return -1
        self.touch(key)
        return self.value[key]

    def put(self, key, value):
        if self.capacity == 0:
            return
        if key in self.value:
            self.value[key] = value
            self.touch(key)
            return
        if len(self.value) == self.capacity:
            oldest, nothing = self.bucket(self.lowest).popitem(last=False)
            del self.value[oldest]
            del self.freq[oldest]
```

```
        self.value[key] = value
        self.freq[key] = 1
        self.bucket(1)[key] = None
        self.lowest = 1


# ---- follow-up 3: tests ----
def test_follow_up_3():
    f = LFUCache(2)
    f.put(1, 1)
    f.put(2, 2)
    assert f.get(1) == 1
    f.put(3, 3)                                     # 2 was used least often, so it goes
    assert f.get(2) == -1 and f.get(3) == 3
    f.put(4, 4)                                     # 1 and 3 were used twice: the older one (1) goes
    assert f.get(1) == -1 and f.get(3) == 3 and f.get(4) == 4


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

GROUP 19 OF 43 | 7 QUESTIONS, ONE CODE TEMPLATE | 2 E 4 M 1 H

Rolling state: the next state comes only from the previous one

153 Climbing Stairs

EASY

Template fit: state = the last two counts: a, b = b, a + b, repeated n - 1 times

QUESTION

Given an integer n, representing the number of steps in a staircase, write a function climbStairs that returns the number of distinct ways to climb to the top of the staircase. Each time you can either climb 1 step or 2 steps. You may assume that n is a non-negative integer. Examples Input: n = 1 Output: 1 Explanation: There is only 1 way to climb a staircase with 1 step.

Input: n = 2 Output: 2 Explanation: There are 2 ways to climb a staircase with 2 steps:

1. Climb 1 step + 1 step 2. Climb 2 steps directly Input: n = 3 Output: 3 Explanation: There are 3 ways to climb a staircase with 3 steps:

1. Climb 1 step + 1 step + 1 step 2. Climb 1 step + 2 steps 3. Climb 2 steps + 1 step Input: n = 4 Output: 5 Explanation: There are 5 ways to climb a staircase with 4 steps:

1. Climb 1 step + 1 step + 1 step + 1 step 2. Climb 1 step + 1 step + 2 steps 3. Climb 1 step + 2 steps + 1 step 4. Climb 2 steps + 1 step + 1 step 5. Climb 2 steps + 2 stepsChallenge 1Can you come up with a solution that has a O(n) time complexity?Challenge 2Can you come up with a solution that has a O(1) space complexity?Solution 1: Dynamic Programming Approach In this solution, we use dynamic programming to compute the number of distinct ways to climb n stairs. The idea is to build up the solution incrementally by solving subproblems. We initialize a list ways where ways[i] represents the number of ways to reach the ith step. We start by setting the base cases: ways[0] = 1 (1 way to stay at the ground) ways[1] =

EXAMPLES

Example 1	
Input	**2**
Output	**2**
Why	**Two ways to climb 2 steps: 1+1 or 2.**

Example 2	
Input	**3**
Output	**3**
Why	**Three ways to climb 3 steps: 1+1+1, 1+2, 2+1.**

Example 3	
Input	**5**
Output	**8**
Why	**Eight ways for 5 steps.**

FULL CODE | main question first, then every follow-up, tests at the end

```
# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ========================================================

# ========================================================
```

```python
# MAIN QUESTION
# =====================================================
# ---- main question: main function ----
# Idea: the ways to reach step n are the ways to reach step n - 1 (then
# one step) plus the ways to reach step n - 2 (then two steps). Keep just
# the last two numbers: (a, b) becomes (b, a + b). Start with
# ways(0) = ways(1) = 1. O(n) time, O(1) space, no table, no recursion.
def climb(n):
    a = 1
    b = 1
    for i in range(n - 1):
        a, b = b, a + b
    if n >= 1:
        return b
    return 1


# ---- main question: tests ----
def test_main_question():
    answers = []
    for n in range(1, 6):
        answers.append(climb(n))
    assert answers == [1, 2, 3, 5, 8]
    assert climb(0) == 1


# =====================================================
# FOLLOW-UP 1: n is huge (10^18)
# =====================================================
# ---- follow-up 1: helper functions ----
def mat_mult(x, y, mod):
    # multiply two 2 x 2 matrices, keeping every number below `mod`
    out = [[0, 0], [0, 0]]
    for i in range(2):
        for j in range(2):
            total = 0
            for k in range(2):
                total += x[i][k] * y[k][j]
            out[i][j] = total % mod
    return out


# ---- follow-up 1: function ----
# Answer: raise the matrix [[1, 1], [1, 0]] to the n-th power by
# repeated squaring: O(log n) multiplications (fast doubling is the same
# idea). Take every product modulo a prime if the answer is huge.
def climb_huge(n, mod=10 ** 9 + 7):
    result = [[1, 0], [0, 1]]                           # the identity matrix
    base = [[1, 1], [1, 0]]
    power = n
    while power > 0:
        if power % 2 == 1:
            result = mat_mult(result, base, mod)
        base = mat_mult(base, base, mod)
        power = power // 2
    return result[0][0] % mod                           # result = [[F(n+1), F(n)], [F(n), F(n-1)]]


# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert climb_huge(5) == 8 and climb_huge(1) == 1
    assert 0 <= climb_huge(10 ** 18) < 10 ** 9 + 7
    assert climb_huge(50) == climb(50) % (10 ** 9 + 7)
```

```
# ============================================================
# FOLLOW-UP 2: You can take 1 to k steps (or a given set of step sizes)
# ============================================================
# ---- follow-up 2: function ----
# Answer: ways[i] = the sum of ways[i - s] over every allowed step size s.
# For steps 1..k keep a sliding-window sum instead, which makes it O(n).
def climb_steps(n, steps):
    ways = [1] + [0] * n
    for i in range(1, n + 1):
        for s in steps:
            if i >= s:
                ways[i] += ways[i - s]
    return ways[n]


def climb_up_to_k(n, k):
    ways = [1] + [0] * n
```

```
    window = 1                                          # the sum of the last k values of ways
    for i in range(1, n + 1):
        ways[i] = window
        window += ways[i]
        if i - k >= 0:
            window -= ways[i - k]
    return ways[n]


# ---- follow-up 2: tests ----
def test_follow_up_2():
    assert climb_steps(5, (1, 2)) == 8
    assert climb_steps(4, (1, 3)) == 3
    assert climb_up_to_k(5, 2) == 8
    assert climb_up_to_k(4, 3) == 7


# ============================================================
# FOLLOW-UP 3: Some steps are broken (cannot be stood on)
# ============================================================
# ---- follow-up 3: function ----
# Answer: set ways[i] = 0 for a broken step while filling the table;
# everything else stays the same.
def climb_broken(n, broken):
    bad = set(broken)
    ways = [1] + [0] * n
    for i in range(1, n + 1):
        if i in bad:
            continue
        two_back = 0
        if i > 1:
            two_back = ways[i - 2]
        ways[i] = ways[i - 1] + two_back
    return ways[n]


# ---- follow-up 3: tests ----
def test_follow_up_3():
    assert climb_broken(5, [3]) == 2
    assert climb_broken(3, [1, 2]) == 0


# ============================================================
# RUN EVERYTHING
# ============================================================
```

```
def run_all_tests():
    test_main_question()
```

```
    test_follow_up_1()
    test_follow_up_2()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

154 Fibonacci Numbers — EASY

Template fit: the same pair update starting from (0, 1)

QUESTION

Write a function fib(n) that returns the nth Fibonacci number. The Fibonacci sequence is defined as F(n) = F(n-1) + F(n-2) with F(0) = 1 and F(1) = 1, which produces the following pattern: 1, 1, 2, 3, 5, 8, 13 ... Examples Pythonfib(0)

=> 1 fib(1)

=> 1 fib(2)

=> 2 fib(3)

=> 3 ... fib(10)

=> 89 fib(20)

=> 10946Hint 1The Fibonacci sequence

is defined recursively since it refers to itself.Hint 2We can start with a recursive implementation and optimize it.Hint 3Each call to F(n) has two subproblems: F(n-1) and F(n-2). Can we memoize these calculations?Challenge 1Does your solution work in O(n) time?This question is a classic example of how dynamic programming, or memoization, can help us solve problems faster by reducing repetitive computations. In this first version, we recursively call the fib function twice, and then we call it twice more in every subsequent recursive call, so the total number of operations expands to O(2^n)! To improve the runtime of our program, let's save our prior calculations in a dictionary called memo and pass it into our function. Now, if we've already found fib(n) before, our function will use this stored value instead of recursing again. Time Complexity: The time complexity of the memoized version is O(n). This is because each Fibonacci number from 0 to n is computed only once and then stored in the memo dictionary. The recursive calls will now simply look up the value in the dictionary if it has already been computed, reducing redundant

EXAMPLES

Example 1	
Input	`0`
Output	`1`
Why	`F(0) = 1 and F(1) = 1, as defined in this question.`

Example 2	
Input	`5`
Output	`8`
Why	`1, 1, 2, 3, 5, 8: F(5) = 8.`

Example 3	
Input	`10`
Output	`89`
Why	`F(10) = 89.`

FULL CODE | main question first, then every follow-up, tests at the end

```python
# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ========================================================

# ========================================================
# MAIN QUESTION
# ========================================================
# ---- main question: main functions ----
# This question numbers the sequence 1, 1, 2, 3, 5, 8 ...: fib(0) = fib(1) = 1.
# The definition (slow: about 1.6 ** n calls; say it first):
def fib_slow(n):
    if n < 2:
        return 1
    return fib_slow(n - 1) + fib_slow(n - 2)


# Idea: keep only the previous two values and move forward n times.
# O(n) time and O(1) memory (memoising the recursion gives the same time
# with O(n) memory).
def fib(n):
    a = 1
    b = 1
    for i in range(n):
        a, b = b, a + b
    return a


# ---- main question: tests ----
def test_main_question():
    first = []
    for n in range(7):
        first.append(fib(n))
    assert first == [1, 1, 2, 3, 5, 8, 13]
    assert fib(10) == 89 and fib(20) == 10946
    assert fib_slow(10) == 89


# ========================================================
# FOLLOW-UP 1: n is up to 10^18 and the answer is needed modulo a prime
# ========================================================
# ---- follow-up 1: helper functions ----
MOD = 10 ** 9 + 7
```

```python
def pair(n, mod):
    # fast doubling. Returns (F(n), F(n + 1)) of the standard sequence 0, 1, 1, 2, 3 ...
    # using F(2k) = F(k) * (2 * F(k + 1) - F(k)) and F(2k + 1) = F(k)^2 + F(k + 1)^2.
    if n == 0:
        return 0, 1
    a, b = pair(n // 2, mod)                # F(k) and F(k + 1) for k = n // 2
    c = a * (2 * b - a) % mod               # F(2k)
    d = (a * a + b * b) % mod               # F(2k + 1)
    if n % 2 == 1:
        return d, (c + d) % mod
    return c, d


# ---- follow-up 1: function ----
# Answer: fast doubling (or matrix power) with every multiplication
# reduced modulo the prime: O(log n) steps with small numbers. Our
# fib(n) is the standard F(n + 1).
def fib_mod(n, mod=MOD):
    return pair(n + 1, mod)[0] % mod
```

```
# ---- follow-up 1: tests ----
def test_follow_up_1():
    for n in range(200):
        assert fib_mod(n, MOD) == fib(n) % MOD
        big = 10 ** 18 + 9
        assert fib_mod(n, big) == fib(n) % big
    assert 0 <= fib_mod(10 ** 18) < MOD                    # a huge n finishes at once


# ============================================================
# FOLLOW-UP 2: Why is the recursive version slow, and how do you fix it without changing the structure?
# ============================================================
# ---- follow-up 2: function ----
# Answer: it solves the same smaller problems again and again (the call
# tree has about 1.6 ** n nodes). Remember each answer in a dictionary
# (memoisation): every value is computed once, so O(n) time and O(n)
# memory. The loop with two variables brings the memory down to O(1).
def fib_memo(n, memo=None):
    if memo is None:
        memo = {}
    if n < 2:
        return 1
    if n not in memo:
        memo[n] = fib_memo(n - 1, memo) + fib_memo(n - 2, memo)
    return memo[n]


# ---- follow-up 2: tests ----
def test_follow_up_2():
    assert fib_memo(20) == 10946
    assert fib(0) == 1 and fib(1) == 1
    assert fib_memo(90) == fib(90)


# ============================================================
# FOLLOW-UP 3: Closed form (Binet's formula)
# ============================================================
# ---- follow-up 3: function ----
# Answer: F(n) = round(phi^n / sqrt(5)) with phi = (1 + sqrt(5)) / 2 is
# exact only while floating point has enough precision (up to about n =
# 70); after that the last digits are wrong, so use the integer methods.
def fib_binet(n):
    root5 = 5 ** 0.5
    phi = (1 + root5) / 2
    return round(phi ** (n + 1) / root5)


# ---- follow-up 3: tests ----
def test_follow_up_3():
    for n in range(60):
        assert fib_binet(n) == fib(n)                      # exact for a while ...
    wrong = []
    for n in range(60, 200):
        if fib_binet(n) != fib(n):
            wrong.append(n)
    assert wrong and wrong[0] < 100                        # ... then floating point fails


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
```

```
    test_follow_up_1()
    test_follow_up_2()
    test_follow_up_3()
    print("all tests passed")
```

```
if __name__ == "__main__":
    run_all_tests()
```

155 Maximize sum without choosing adjacent elements — MEDIUM

Template fit: exactly the house robber update on the array

QUESTION

Given a list of non-negative numbers, choose some of them so that no two chosen numbers are neighbours in the list, and return the largest possible sum of the chosen numbers.

Function signature

def rob_line(nums: list) -> int

EXAMPLES

Example 1	
Input	`[2, 7, 9, 3, 1]`
Output	`12`
Why	`Choose 2, 9 and 1 (positions 0, 2, 4): 12.`

Example 2	
Input	`[5]`
Output	`5`
Why	`One number: take it.`

Example 3	
Input	`[]`
Output	`0`
Why	`Nothing to choose: 0.`

FULL CODE | main question first, then every follow-up, tests at the end

```
# ====================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ====================================================

# ====================================================
# MAIN QUESTION
# ====================================================
# ---- main question: helper functions ----
def bigger(x, y):
    if x > y:
        return x
    return y
```

```python
# ---- main question: main function ----
# Idea: keep two running numbers: `take` = the best total whose LAST
# chosen element is the current one, `skip` = the best total that does
# not use the current one. For the next number x:
#   new take = skip + x   (the previous element must have been skipped)
#   new skip = the bigger of take and skip.
def rob_line(nums):
    take = 0
    skip = 0
    for x in nums:
        new_take = skip + x
        new_skip = bigger(take, skip)
        take = new_take
        skip = new_skip
    return bigger(take, skip)


# ---- main question: tests ----
def test_main_question():
    assert rob_line([2, 7, 9, 3, 1]) == 12
    assert rob_line([]) == 0
    assert rob_line([5]) == 5


# ======================================================
# FOLLOW-UP 1: The houses form a circle (LeetCode 213)
# ======================================================
# ---- follow-up 1: function ----
```

```python
# Answer: the first and last house are neighbours now, so you cannot
# take both: the answer is the better of "ignore the first house" and
# "ignore the last house", each solved with the line version.
def rob_circle(nums):
    if len(nums) == 1:
        return nums[0]
    return bigger(rob_line(nums[1:]), rob_line(nums[:-1]))


# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert rob_circle([2, 3, 2]) == 3
    assert rob_circle([1, 2, 3, 1]) == 4
    assert rob_circle([4]) == 4


# ======================================================
# FOLLOW-UP 2: Return which houses to rob
# ======================================================
# ---- follow-up 2: function ----
# Answer: keep the whole table dp[i] = the best total from house i to the
# end (filled from the right), then walk forward: take house i when
# nums[i] + dp[i + 2] equals dp[i] (and jump two houses), otherwise move on one.
def rob_with_choice(nums):
    n = len(nums)
    dp = [0] * (n + 2)
    for i in range(n - 1, -1, -1):
        dp[i] = bigger(dp[i + 1], nums[i] + dp[i + 2])
    chosen = []
    i = 0
    while i < n:
        if nums[i] > 0 and nums[i] + dp[i + 2] == dp[i]:
            chosen.append(i)
            i += 2
        else:
            i += 1
    return dp[0], chosen
```

```python
# ---- follow-up 2: tests ----
def test_follow_up_2():
    total, houses = rob_with_choice([2, 7, 9, 3, 1])
    assert total == 12 and houses == [0, 2, 4]
```

```python
# ============================================================
# FOLLOW-UP 3: Robbing a house also alerts the next k houses
# ============================================================
# ---- follow-up 3: function ----
# Answer: dp[i] = the bigger of dp[i + 1] and nums[i] + dp[i + k + 1]
# (taking a house blocks the next k). The O(1)-space trick would need a
# window of k + 1 numbers, so simply keep the table.
def rob_gap(nums, k):
    n = len(nums)
    dp = [0] * (n + k + 2)
    for i in range(n - 1, -1, -1):
        dp[i] = bigger(dp[i + 1], nums[i] + dp[i + k + 1])
    return dp[0]


# ---- follow-up 3: tests ----
def test_follow_up_3():
    assert rob_gap([2, 7, 9, 3, 1], 1) == 12              # k = 1 is the classic problem
    assert rob_gap([5, 1, 1, 5, 1, 1, 5], 2) == 15        # blocking two houses after each pick


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

156 Given n houses in a line with money in each, find the maximum amount a robber can steal, without stealing from two adjacent houses. MEDIUM

Template fit: state = (best if you take this house, best if you skip it): take, skip = skip + x, max(take, skip)

QUESTION

Houses stand in a line and house i holds nums[i] money. A robber cannot rob two houses that are next to each other. Return the largest total amount he can steal.

Function signature

def rob_line(nums: list) -> int

EXAMPLES

Example 1	
Input	`[1, 2, 3, 1]`
Output	`4`
Why	`Rob houses 0 and 2: 1 + 3 = 4.`

QUESTION

Example 2	
Input	[2, 7, 9, 3, 1]
Output	12
Why	Rob houses 0, 2 and 4: 2 + 9 + 1 = 12.

FULL CODE | main question first, then every follow-up, tests at the end

```
# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ========================================================

# ========================================================
# MAIN QUESTION
# ========================================================
# ---- main question: helper functions ----
def bigger(x, y):
    if x > y:
        return x
    return y


# ---- main question: main function ----
# Idea: two running numbers. `take` = the best total when the current
# house IS robbed, `skip` = the best total when it is not. For the next
# house with money x: new take = skip + x (the house before must have
# been skipped), new skip = the bigger of take and skip.
def rob_line(nums):
    take = 0
    skip = 0
    for x in nums:
        new_take = skip + x
        new_skip = bigger(take, skip)
        take = new_take
        skip = new_skip
    return bigger(take, skip)


# ---- main question: tests ----
def test_main_question():
    assert rob_line([1, 2, 3, 1]) == 4
    assert rob_line([2, 7, 9, 3, 1]) == 12
    assert rob_line([]) == 0


# ========================================================
# FOLLOW-UP 1: The houses form a circle (LeetCode 213)
# ========================================================
# ---- follow-up 1: function ----
# Answer: the first and last houses touch, so you cannot rob both: the
```

```
# answer is the better of "ignore the first house" and "ignore the last
# house", each solved with the line version.
def rob_circle(nums):
    if len(nums) == 1:
        return nums[0]
    return bigger(rob_line(nums[1:]), rob_line(nums[:-1]))


# ---- follow-up 1: tests ----
```

```python
def test_follow_up_1():
    assert rob_circle([2, 3, 2]) == 3
    assert rob_circle([1, 2, 3, 1]) == 4
    assert rob_circle([5]) == 5


# ========================================================
# FOLLOW-UP 2: Return which houses to rob
# ========================================================
# ---- follow-up 2: function ----
# Answer: keep dp[i] = the best total from house i to the end (filled
# from the right). Walk forward: rob house i when nums[i] + dp[i + 2]
# equals dp[i] (then jump two houses), otherwise move on one.
def rob_with_choice(nums):
    n = len(nums)
    dp = [0] * (n + 2)
    for i in range(n - 1, -1, -1):
        dp[i] = bigger(dp[i + 1], nums[i] + dp[i + 2])
    houses = []
    i = 0
    while i < n:
        if nums[i] > 0 and nums[i] + dp[i + 2] == dp[i]:
            houses.append(i)
            i += 2
        else:
            i += 1
    return dp[0], houses


# ---- follow-up 2: tests ----
def test_follow_up_2():
    total, houses = rob_with_choice([2, 7, 9, 3, 1])
    assert total == 12 and houses == [0, 2, 4]


# ========================================================
```

```python
# FOLLOW-UP 3: Robbing a house also alerts the next k houses
# ========================================================
# ---- follow-up 3: function ----
# Answer: dp[i] = the bigger of dp[i + 1] and nums[i] + dp[i + k + 1]
# (a robbed house alerts the next k). The O(1)-space version needs a
# window of k + 1 numbers, or just keep the O(n) table.
def rob_alarm(nums, k):
    n = len(nums)
    dp = [0] * (n + k + 2)
    for i in range(n - 1, -1, -1):
        dp[i] = bigger(dp[i + 1], nums[i] + dp[i + k + 1])
    return dp[0]


# ---- follow-up 3: tests ----
def test_follow_up_3():
    assert rob_alarm([2, 7, 9, 3, 1], 1) == 12
    assert rob_alarm([5, 1, 1, 5, 1, 1, 5], 2) == 15


# ========================================================
# RUN EVERYTHING
# ========================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    test_follow_up_3()
    print("all tests passed")
```

```
if __name__ == "__main__":
    run_all_tests()
```

157 Paint House II — MEDIUM

Template fit: state = a whole row of costs, one per colour; new[j] = cost[j] + the smallest previous value that is not colour j (keep the two smallest)

QUESTION

There are n houses in a row, each to be painted one of k colors. costs[i][j] is the cost to paint house i with color j . No two adjacent houses may share the same color. Return the minimum total cost to paint all houses.

Function signature

def min_cost_paint_houses(costs: list[list[int]]) -> int:

    ...

Follow-up (as reported): Reduce the per-row transition from O(k) per cell to O(1) amortized by tracking the two smallest values of the previous row.

EXAMPLES

Example 1	
Input	`costs = [[1, 5, 3], [2, 9, 4]]`
Output	`5`
Why	`Paint house 0 color 0 (1) and house 1 color 2 (4) -> 5; or house 0 color 2 (3) and house 1 color 0 (2) -> 5.`

Example 2	
Input	`costs = [[1, 3], [2, 4]]`
Output	`5`
Why	`1 (house0 color0) + 4 (house1 color1) = 5, the cheapest valid pairing.`

Example 3	
Input	`costs = [[7]]`
Output	`7`
Why	`A single house, one color.`

CONSTRAINTS

- costs is an n x k grid, 0 <= n , k >= 1 (and k >= 2 whenever n >= 2 , so a valid painting exists).
- Costs are non-negative integers. Return 0 for n == 0 .
- Aim for O(n*k) time.

FULL CODE | main question first, then every follow-up, tests at the end

```
# ====================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ====================================================

import random


# ====================================================
```

```
# MAIN QUESTION
# =====================================================
# ---- main question: helper functions ----
def two_smallest(values):
    # (index of the smallest, the smallest value, the second smallest value)
    first = 0
    for i in range(len(values)):
        if values[i] < values[first]:
            first = i
    second_value = values[first]                  # with one colour there is no second choice
    found_second = False
    for i in range(len(values)):
        if i != first and (not found_second or values[i] < second_value):
            second_value = values[i]
            found_second = True
    return first, values[first], second_value


# ---- main question: main function ----
# Every house is painted one of k colours; neighbouring houses must
# differ; costs[i][j] is the cost of colour j for house i. Find the
# cheapest total cost.
# Idea: the best total so far for a house painted with colour j is its
# cost plus the cheapest total of the previous house in a DIFFERENT
# colour. That is the smallest previous total, unless the smallest is
# colour j itself: then take the second smallest. So only the two
# smallest values of the previous row are needed: O(n * k).
def min_cost_paint_houses(costs):
    previous = [0] * len(costs[0])
    for row in costs:
        first, smallest, second_smallest = two_smallest(previous)
        current = []
        for j in range(len(row)):
            if j == first:
```

```
                current.append(row[j] + second_smallest)
            else:
                current.append(row[j] + smallest)
        previous = current
    return min(previous)


# ---- main question: tests ----
def test_main_question():
    assert min_cost_paint_houses([[1, 5, 3], [2, 9, 4]]) == 5
    assert min_cost_paint_houses([[1, 3], [2, 4]]) == 5
    assert min_cost_paint_houses([[7]]) == 7


# =====================================================
# FOLLOW-UP 1: Return which colour each house gets, not just the cost
# =====================================================
# ---- follow-up 1: function ----
# Answer: store the whole table of totals (O(n * k) memory), then walk
# back from the cheapest last cell: for house i, the colour of house i - 1
# is a different colour whose total equals this house's total minus this
# house's own cost.
def paint_plan(costs):
    n = len(costs)
    k = len(costs[0])
    rows = []
    previous = [0] * k
    for row in costs:
        current = []
        for j in range(k):
            cheapest = None
```

```
            for x in range(k):
                if x != j and (cheapest is None or previous[x] < cheapest):
                    cheapest = previous[x]
            if cheapest is None:
                cheapest = 0                        # only one colour: nothing before to avoid
            current.append(row[j] + cheapest)
        rows.append(current)
        previous = current
    colour = 0
    for j in range(k):
        if rows[n - 1][j] < rows[n - 1][colour]:
            colour = j
    total = rows[n - 1][colour]
    plan = [colour]
```

```
    for i in range(n - 1, 0, -1):
        needed = rows[i][colour] - costs[i][colour]
        for x in range(k):
            if x != colour and rows[i - 1][x] == needed:
                colour = x
                break
        plan.append(colour)
    plan.reverse()
    return total, plan


# ---- follow-up 1: tests ----
def test_follow_up_1():
    random.seed(68)
    for round_number in range(300):
        n = random.randint(1, 5)
        k = random.randint(2, 4)
        costs = []
        for i in range(n):
            row = []
            for j in range(k):
                row.append(random.randint(1, 9))
            costs.append(row)
        total, plan = paint_plan(costs)
        for i in range(n - 1):
            assert plan[i] != plan[i + 1]                   # neighbours differ
        paid = 0
        for i in range(n):
            paid += costs[i][plan[i]]
        assert paid == total == min_cost_paint_houses(costs)


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

158 Write SQL code to publish the Fibonacci series. MEDIUM

Template fit: WITH RECURSIVE keeps (n, a, b); every new row is (n + 1, b, a + b): the same rolling pair, written as a query

QUESTION

Write a SQL query that publishes the first N Fibonacci numbers (0, 1, 1, 2, 3, 5, ...), one per row and in order, where N is given as the parameter :limit. Use a recursive common table expression.

Function signature

-- result: one column named fibonacci, N rows

EXAMPLES

Example 1	
Input	`:limit = 8`
Output	`0, 1, 1, 2, 3, 5, 8, 13`
Why	`Eight rows: each number is the sum of the two before it.`

Example 2	
Input	`:limit = 1`
Output	`0`
Why	`A single row.`

FULL CODE | main question first, then every follow-up, tests at the end

```
# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

import sqlite3


# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
# A recursive query: every new row is computed from the previous row.
FIB_SQL = """
WITH RECURSIVE fib(n, a, b) AS (
    SELECT 1, 0, 1
    UNION ALL
    SELECT n + 1, b, a + b FROM fib WHERE n < ?
)
SELECT a FROM fib
"""


def run_query(sql, parameters):
    # run one query on a fresh in-memory database and return the first column of every row
    db = sqlite3.connect(":memory:")
    rows = db.execute(sql, parameters).fetchall()
    db.close()
    column = []
    for row in rows:
        column.append(row[0])
    return column


def fibonacci_python(count):
```

```
    # the same sequence in Python, used to check the SQL
    a = 0
    b = 1
    terms = []
    for i in range(count):
        terms.append(a)
        a, b = b, a + b
    return terms
```

```


# ---- main question: main function ----
# Idea: a recursive CTE that carries the three values (n, a, b). The
# first row is (1, 0, 1): the first term is 0 and the next pair is (0, 1).
# Every recursive step makes (n + 1, b, a + b) while n is below the limit,
# so column a holds the Fibonacci numbers.
# The limit is a QUERY PARAMETER (the ? above), never glued into the SQL
# text, and the recursion runs inside the database: one round trip for any n
# (this is also follow-up 4).
def fibonacci(count):
    return run_query(FIB_SQL, (count,))


# ---- main question: tests ----
def test_main_question():
    assert fibonacci(10) == [0, 1, 1, 2, 3, 5, 8, 13, 21, 34]
    assert fibonacci(10) == fibonacci_python(10)
    assert fibonacci(1) == [0]


# =============================================================
# FOLLOW-UP 1: Only the even terms, or the sum of the first n terms
# =============================================================
# ---- follow-up 1: function ----
# Answer: keep the recursive query and filter or add up in an outer query:
# WHERE a % 2 = 0 for the even terms, SUM(a) for the total.
def even_terms(count):
    sql = "SELECT a FROM (" + FIB_SQL + ") WHERE a % 2 = 0"
    return run_query(sql, (count,))


def total(count):
    sql = "SELECT SUM(a) FROM (" + FIB_SQL + ")"
    return run_query(sql, (count,))[0]


# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert even_terms(10) == [0, 2, 8, 34]
    assert total(10) == 88                                  # 0+1+1+2+3+5+8+13+21+34


# =============================================================
# FOLLOW-UP 2: Which database?
```

```
# =============================================================
# Answer: the recursive WITH works in PostgreSQL, MySQL 8, SQLite and
# SQL Server (which leaves out the word RECURSIVE); Oracle can also use
# CONNECT BY. Say which one you assume.


# =============================================================
# FOLLOW-UP 3: Large n overflows
# =============================================================
# Answer: a 64-bit column is exact up to the 92nd term. Beyond that use
# a big-number type (NUMERIC), or compute modulo a prime.
```

```python
# ---- follow-up 3: tests ----
def test_follow_up_3():
    assert fibonacci(80) == fibonacci_python(80)          # 80 terms are still exact in a 64-bit column


# =========================================================
# RUN EVERYTHING
# =========================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

159 Constrained Dice Roll Sequences — HARD

Template fit: state = a table [value][run length]; a different value starts a run of 1, the same value extends its run (up to the limit)

QUESTION

You roll a single 6-faced die exactly n times. You are given an array roll_max of length 6, where roll_max[i] is the maximum number of times the value i + 1 may appear consecutively. That is, you may never roll the same value more than roll_max[i] times in a row. Return the number of distinct sequences of n rolls that respect every consecutive-count limit. Because the answer can be very large, return it modulo 10**9 + 7 .

Function signature

def count_dice_sequences(n: int, roll_max: list[int]) -> int:

    ...

Follow-up (as reported): The interviewer's actual twist used two dice per turn (a pair rolled together, with the consecutive limit applied to each die independently). Extend the state to track the run lengths of both dice and count the joint sequences.

EXAMPLES

Example 1	
Input	`n = 2, roll_max = [1, 1, 2, 2, 2, 3]`
Output	`34`
Why	`There are 6*6 = 36 sequences of length 2. Two of them are forbidden: rolling value 1 twice, and rolling value 2 twice (both have roll_max = 1). 36 - 2 = 34.`

Example 2	
Input	`n = 2, roll_max = [1, 1, 1, 1, 1, 1]`
Output	`30`
Why	`No value may repeat consecutively, so the 6 sequences "value v then v" are removed: 36 - 6 = 30.`

Example 3	
Input	`n = 3, roll_max = [1, 1, 1, 2, 2, 3]`
Output	`181`
Why	`Hand-verified against the memoized DP over (rolls_used, last_value, run_length).`

CONSTRAINTS

- 1 <= n <= 5000 roll_max has exactly 6 entries, each 1 <= roll_max[i] <= 15 .

QUESTION

- A naive brute force over all 6**n sequences is far too slow; you must memoize over (rolls_used, last_value, consecutive_count_of_last_value) .

FULL CODE | main question first, then every follow-up, tests at the end

```
# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ========================================================

# ========================================================
# MAIN QUESTION
# ========================================================
# ---- main question: helper functions ----
MOD = 10 ** 9 + 7


def total_of(dp):
    # the sum of every number in the table
    total = 0
    for row in dp:
        for count in row:
            total += count
    return total


# ---- main question: main function ----
# n rolls of a six-sided die; the value v+1 may appear at most roll_max[v]
# times IN A ROW. Count the valid sequences (modulo 10^9 + 7).
# Idea (DP over the last value and how long it has been repeated):
# dp[v][r] = the number of valid sequences so far that end with value v
# repeated r times in a row. For the next roll:
#   - a DIFFERENT value than before starts a run of length 1:
#     (all sequences) - (those that already end in v);
#   - the SAME value extends its run by one (only while r < roll_max[v]).
def count_dice_sequences(n, roll_max):
    dp = []
    for v in range(6):
        row = [0] * (roll_max[v] + 1)
        row[1] = 1                                      # one roll: every value once
        dp.append(row)
    for roll in range(n - 1):
        everything = total_of(dp) % MOD
        new = []
        for v in range(6):
            new.append([0] * (roll_max[v] + 1))
        for v in range(6):
            ending_in_v = 0
            for count in dp[v]:
                ending_in_v += count
            new[v][1] = (everything - ending_in_v) % MOD
            for r in range(1, roll_max[v]):
                new[v][r + 1] = dp[v][r]
        dp = new
    return total_of(dp) % MOD


# ---- main question: tests ----
def test_main_question():
    assert count_dice_sequences(2, [1, 1, 2, 2, 2, 3]) == 34
    assert count_dice_sequences(2, [1, 1, 1, 1, 1, 1]) == 30
    assert count_dice_sequences(3, [1, 1, 1, 2, 2, 3]) == 181
```

```
    assert count_dice_sequences(1, [1, 1, 1, 1, 1, 1]) == 6


# ========================================================
# FOLLOW-UP 1: Each turn rolls a pair of dice together, and the limit applies to each die independently. How
many joint sequences are there?
# ========================================================
# ---- follow-up 1: function ----
# Answer: the two dice never influence each other, so a joint sequence is
# one valid sequence for each die: the answer is the single-die count
# SQUARED (modulo 10^9 + 7).
def count_two_dice(n, roll_max):
    one = count_dice_sequences(n, roll_max)
    return one * one % MOD


# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert count_two_dice(2, [1, 1, 2, 2, 2, 3]) == 34 * 34
    assert 0 <= count_dice_sequences(500, [15] * 6) < MOD


# ========================================================
# RUN EVERYTHING
# ========================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
```

```
    run_all_tests()
```

GROUP 20 OF 43 | 7 QUESTIONS, ONE CODE TEMPLATE | 4 E 3 M

Two pointers: move i and j toward each other (or along two lists)

160 Reverse a linked list. EASY

Template fit: three pointers prev, current, next: point current.next back at prev, then move all three one step

QUESTION

Given the head of a singly linked list, reverse the list and return its head. Implement a function reverseList that takes the head of the linked list as input and returns the new head of the reversed list. Refer to the ListNode class/struct and do not comment it out. Example Input: 1 -> 2 -> 3 -> 4 Output: 4 -> 3 -> 2 -> 1Challenge 1Can you come up with a solution with a time complexity of O(n)?Challenge 2Can you come up with a solution with a space complexity of O(1)?

EXAMPLES

Example 1	
Input	`head = 1 -> 2 -> 3`
Output	`vals(reverse_list(head)) -> [3, 2, 1]`
Why	`1 -> 2 -> 3 becomes 3 -> 2 -> 1.`

FULL CODE | main question first, then every follow-up, tests at the end

```
# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next


def make_list(values):
    # [1, 2, 3] -> 1 -> 2 -> 3
    dummy = ListNode()
    tail = dummy
    for v in values:
        tail.next = ListNode(v)
        tail = tail.next
    return dummy.next


def to_list(node):
    values = []
    while node:
        values.append(node.val)
        node = node.next
    return values


# ---- main question: main function ----
# Idea: walk with `prev` and `node`. Remember node.next FIRST (the step
# people forget), point node back at prev, then move both forward. When
# node is None, prev is the new head.
```

```
def reverse_list(head):
    prev = None
    node = head
    while node:
        following = node.next
        node.next = prev
        prev = node
        node = following
    return prev


# ---- main question: tests ----
def test_main_question():
    assert to_list(reverse_list(make_list([1, 2, 3, 4]))) == [4, 3, 2, 1]
    assert reverse_list(None) is None
    assert to_list(reverse_list(make_list([7]))) == [7]


# ============================================================
# FOLLOW-UP 1: Do it recursively
# ============================================================
# ---- follow-up 1: function ----
# Answer: handle one node, then hand the rest to the same function: save
# the next node, point this one back at `prev`, and recurse with this
# node as the new prev. The call stack is O(n): mention the depth limit.
def reverse_recursive(head, prev=None):
    if head is None:
        return prev
    following = head.next
    head.next = prev
    return reverse_recursive(following, head)


# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert to_list(reverse_recursive(make_list([1, 2, 3]))) == [3, 2, 1]


# ============================================================
# FOLLOW-UP 2: Reverse only positions left to right (LeetCode 92)
# ============================================================
# ---- follow-up 2: function ----
# Answer: walk to the node before the segment, reverse right - left + 1
# nodes with the same loop, then reconnect: the old first node of the
# segment must point to what came after the segment, and the node before
# the segment points to the new first node.
def reverse_between(head, left, right):
    dummy = ListNode(0, head)
    before = dummy
    for i in range(left - 1):
        before = before.next
    prev = None
    node = before.next
    for i in range(right - left + 1):
        following = node.next
        node.next = prev
        prev = node
        node = following
    before.next.next = node                          # the old first node points past the segment
    before.next = prev
    return dummy.next


# ---- follow-up 2: tests ----
```

```
def test_follow_up_2():
    assert to_list(reverse_between(make_list([1, 2, 3, 4, 5]), 2, 4)) == [1, 4, 3, 2, 5]
    assert to_list(reverse_between(make_list([1, 2]), 1, 2)) == [2, 1]


# ======================================================
# FOLLOW-UP 3: Reverse in groups of k and leave a short last group alone (LeetCode 25)
# ======================================================
# ---- follow-up 3: function ----
# Answer: check that k nodes exist before reversing a group, reverse them
# with the same loop, reconnect both ends, and move on to the old first
# node of the group (now its last).
def reverse_k_group(head, k):
    dummy = ListNode(0, head)
    group_prev = dummy
    while True:
        end = group_prev                              # is there a full group of k nodes?
        for i in range(k):
            end = end.next
            if end is None:
                return dummy.next
        first = group_prev.next
        after = end.next
        prev = after
        node = first
        while node is not after:
            following = node.next
            node.next = prev
            prev = node
            node = following
        group_prev.next = prev
        group_prev = first
```

```
# ---- follow-up 3: tests ----
def test_follow_up_3():
    assert to_list(reverse_k_group(make_list([1, 2, 3, 4, 5]), 2)) == [2, 1, 4, 3, 5]
    assert to_list(reverse_k_group(make_list([1, 2, 3]), 3)) == [3, 2, 1]
    assert to_list(reverse_k_group(make_list([1]), 2)) == [1]


# ======================================================
# RUN EVERYTHING
# ======================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

161 Sort a binary array in linear time. EASY

Template fit: left skips 0s, right skips 1s; when left sits on a 1 and right on a 0, swap them

QUESTION

Sort an array that contains only 0s and 1s in place, in one pass over the array (O(n) time) and with O(1) extra space.

Function signature

def sort_binary(arr: list) -> list

EXAMPLES

Example 1	
Input	`[1, 0, 1, 0, 0, 1]`
Output	`[0, 0, 0, 1, 1, 1]`
Why	`All the 0s move to the front.`

Example 2	
Input	`[0, 0, 1, 1]`
Output	`[0, 0, 1, 1]`
Why	`Already sorted arrays stay as they are.`

Example 3	
Input	`[1]`
Output	`[1]`
Why	`A single value.`

FULL CODE | main question first, then every follow-up, tests at the end

```
# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: main function ----
# Idea: two pointers. Zeros collect on the left, ones on the right.
# Move lo past a 0, move hi back past a 1; when lo sits on a 1 and hi on
# a 0, swap them. Every step moves a pointer: O(n), in place.
def sort_binary(arr):
    lo = 0
    hi = len(arr) - 1
    while lo < hi:
        if arr[lo] == 0:
            lo += 1
        elif arr[hi] == 1:
            hi -= 1
        else:
            arr[lo], arr[hi] = arr[hi], arr[lo]       # a 1 on the left and a 0 on the right
            lo += 1
            hi -= 1
    return arr


# the same idea for records that only have a 0/1 key: swapping keeps the
# objects themselves (counting would lose them). Not stable.
```

```
def sort_records(items, key):
    lo = 0
    hi = len(items) - 1
    while lo < hi:
        if key(items[lo]) == 0:
            lo += 1
        elif key(items[hi]) == 1:
            hi -= 1
        else:
            items[lo], items[hi] = items[hi], items[lo]
            lo += 1
            hi -= 1
    return items
```

```
# the same idea for 0, 1 and 2 (Dutch national flag)
def sort_three(arr):
    low = 0              # arr[:low] are all 0
    mid = 0              # arr[low:mid] are all 1
    high = len(arr)      # arr[high:] are all 2; arr[mid:high] is not looked at yet
    while mid < high:
        if arr[mid] == 0:
            arr[low], arr[mid] = arr[mid], arr[low]
            low += 1
            mid += 1
        elif arr[mid] == 2:
            high -= 1
            arr[mid], arr[high] = arr[high], arr[mid]
        else:
            mid += 1
    return arr


# ---- main question: tests ----
def test_main_question():
    assert sort_binary([1, 0, 1, 0, 1, 0]) == [0, 0, 0, 1, 1, 1]
    assert sort_binary([]) == []
    assert sort_binary([1]) == [1]
    assert sort_binary([1, 1, 0]) == [0, 1, 1]
    assert sort_three([2, 0, 1, 2, 0, 1]) == [0, 0, 1, 1, 2, 2]
    assert sort_three([2, 2, 0]) == [0, 2, 2]


# ======================================================
# FOLLOW-UP 1: Why not just count zeros and fill?
# ======================================================
# Answer: it works for bare numbers, but it cannot sort RECORDS that only
# have a 0 or 1 key, because rewriting the array from a count loses the
# objects. The swapping version keeps them.


# ======================================================
# FOLLOW-UP 2: Count the zeros and rebuild (also O(n))
# ======================================================
# ---- follow-up 2: function ----
# Answer: one pass to count the zeros, then write that many 0s and the
# rest 1s.
def sort_by_counting(arr):
    zeros = 0
```

```
    for x in arr:
        if x == 0:
            zeros += 1
    return [0] * zeros + [1] * (len(arr) - zeros)
```

```
# ---- follow-up 2: tests ----
def second(person):
    return person[1]


def test_follow_up_2():
    for arr in ([0, 0], [1, 1], [1, 0], [0, 1, 0, 0, 1]):
        assert sort_binary(arr[:]) == sorted(arr)
        assert sort_by_counting(arr) == sorted(arr)
    people = [("ann", 1), ("bob", 0), ("cy", 1), ("di", 0)]
    done = sort_records(people[:], second)
    assert [second(p) for p in done] == [0, 0, 1, 1]
    assert sorted(done) == sorted(people)             # no record was lost


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_2()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

162 Is this a valid palindrome? EASY

Template fit: compare the two ends; skip characters that are not letters or digits; stop when the pointers cross

QUESTION

Given a string s, return true if it is a palindrome, and false otherwise. A string is considered a palindrome if, after converting all uppercase letters into lowercase letters and removing all non-alphanumeric characters, it reads the same forward and backward. Write a function isPalindrome that implements this logic. Examples Input: s = "H123!@321h" Output: true Input: s = "race a car" Output: false Constraints 1 <= s.length <= 2 * 10^5 The input string s can include any printable ASCII characters. Challenge 1Can you come up with a solution that achieves O(n) time complexity?Challenge 2Can you come up with a solution that achieves O(1) space complexity?Solution 1: Filtering and Reversing Approach This solution processes the input string by first filtering out non-alphanumeric characters and converting all remaining characters to lowercase. It then checks if this filtered string reads the same forward and backward. Filtering: Iterate through the string and collect alphanumeric characters while converting them to lowercase. Reversing and Comparing: Create a reversed version of the filtered string and compare it with the original filtered string to check for palindrome properties. Time Complexity: The time complexity is O(n), where n is the length of the input string. This is because filtering the characters and creating the reversed string both require iterating through the input once. Space Complexity: The space complexity is O(n) for storing the filtered characters and their

EXAMPLES

Example 1	
Input	`'A man, a plan, a canal: Panama'`
Output	`true`
Why	`Ignoring punctuation, spaces and case it reads 'amanaplanacanalpanama'.`

Example 2	
Input	`'race a car'`
Output	`false`
Why	`'raceacar' is not a palindrome.`

QUESTION

Example 3	
Input	`' '`
Output	`true`
Why	`Nothing but a space: nothing to compare, so it is a palindrome.`

FULL CODE | main question first, then every follow-up, tests at the end

```
# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ========================================================

import unicodedata


# ========================================================
# MAIN QUESTION
# ========================================================
# ---- main question: main function ----
# Idea: two pointers, one at each end, moving toward each other. Skip
# characters that are not letters or digits and compare the lowercase
# forms. The first mismatch ends it; nothing is copied.
def is_palindrome(s):
    lo = 0
    hi = len(s) - 1
    while lo < hi:
        if not s[lo].isalnum():
            lo += 1
        elif not s[hi].isalnum():
            hi -= 1
        elif s[lo].lower() != s[hi].lower():
            return False
        else:
            lo += 1
            hi -= 1
    return True


# ---- main question: tests ----
def test_main_question():
    assert is_palindrome("A man, a plan, a canal: Panama")
    assert not is_palindrome("race a car")
    assert is_palindrome(" ")
    assert is_palindrome("")


# ========================================================
# FOLLOW-UP 1: You may delete at most one character (LeetCode 680)
# ========================================================
# ---- follow-up 1: helper functions ----
```

```
def letters_only(s):
    # the lowercase letters and digits of s, as a list
    chars = []
    for ch in s:
        if ch.isalnum():
            chars.append(ch.lower())
    return chars
```

```
def is_straight(chars, lo, hi):
    # is chars[lo..hi] a palindrome?
    while lo < hi:
        if chars[lo] != chars[hi]:
            return False
        lo += 1
        hi -= 1
    return True


# ---- follow-up 1: function ----
# Answer: move inward until the first mismatch. Then the answer is
# whether skipping the LEFT character or skipping the RIGHT character
# leaves a palindrome: two more linear checks, so O(n) overall.
def almost_palindrome(s):
    chars = letters_only(s)
    lo = 0
    hi = len(chars) - 1
    while lo < hi:
        if chars[lo] != chars[hi]:
            if is_straight(chars, lo + 1, hi):
                return True
            return is_straight(chars, lo, hi - 1)
        lo += 1
        hi -= 1
    return True


# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert almost_palindrome("abca")
    assert almost_palindrome("aba")
    assert not almost_palindrome("abc")


# ========================================================
```

```
# FOLLOW-UP 2: Unicode, accents, or locale-specific case
# ========================================================
# ---- follow-up 2: functions ----
# Answer: normalise first (unicodedata.normalize splits "e with an accent"
# into "e" plus an accent mark; drop the marks; casefold() is the proper
# way to ignore case) and decide what counts as a letter. isalnum() covers
# many scripts, but combining marks need the normalisation.
def clean_unicode(s):
    text = unicodedata.normalize("NFKD", s)
    chars = []
    for ch in text:
        if not unicodedata.combining(ch):
            chars.append(ch)
    folded = "".join(chars).casefold()
    result = []
    for ch in folded:
        if ch.isalnum():
            result.append(ch)
    return result


def is_palindrome_unicode(s):
    chars = clean_unicode(s)
    return is_straight(chars, 0, len(chars) - 1)


# ---- follow-up 2: tests ----
def test_follow_up_2():
    assert is_palindrome_unicode("Été")
```

```
    assert is_palindrome_unicode("\u00c1a")             # "Aa" with an accent on the first A
    assert not is_palindrome("\u00c1a")                 # the plain version sees two different letters
    assert not is_palindrome_unicode("abc")


# ============================================================
# FOLLOW-UP 3: Longest palindromic substring
# ============================================================
# ---- follow-up 3: function ----
# Answer: expand around each of the 2n - 1 centres (n letters and n - 1
# gaps) while both ends match: O(n^2). Manacher's algorithm does it in
# O(n). The two-pointer comparison is the building block.
def longest_palindromic_substring(s):
    best = ""
    for centre in range(2 * len(s) - 1):
        i = centre // 2
```

```
        j = i + centre % 2
        while i >= 0 and j < len(s) and s[i] == s[j]:
            i -= 1
            j += 1
        if j - i - 1 > len(best):
            best = s[i + 1:j]
    return best


# ---- follow-up 3: tests ----
def test_follow_up_3():
    assert longest_palindromic_substring("babad") in ("bab", "aba")
    assert longest_palindromic_substring("cbbd") == "bb"
    assert longest_palindromic_substring("") == ""


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

163 Reverse a Sentence

EASY

Template fit: swap from both ends to reverse the whole array, then reverse every word the same way

QUESTION

You are given an array of characters arrthat consists of sequences of characters separated by space characters. Each space-delimited sequence of characters defines a word. Implement a function reverseWords that reverses the order of the words in the array in the most efficient manner. Explain your solution and analyze its time and space complexities. Example input: arr = ['p', 'e', 'r', 'f', 'e', 'c', 't', ' ', 'm', 'a', 'k', 'e', 's', ' ', 'p', 'r', 'a', 'c', 't', 'i', 'c', 'e'] output: ['p', 'r', 'a', 'c', 't', 'i', 'c', 'e', ' ', 'm', 'a', 'k', 'e', 's', ' ', 'p', 'e', 'r', 'f', 'e', 'c', 't']Hint 1Even if your language of choice enables this, don't be tempted to use standard functions. Consider how inefficient it is to separately create a string, split the words, and reverse the split.Challenge 1As a general rule, if you're tempted to use a standard or library function, be sure to explain that function's time and space complexity before committing to that solution. You may find a more efficient way to solve the problem yourself. Oftentimes in coding interviews, this is the whole point of the exercise.Challenge 2How have you handled edge cases? For example, empty arrays, arrays with nothing but spaces, arrays with single words only, or with multiple spaces between words, etc.Challenge 3Were you able to achieve a linear time complexity and a constant space complexity?Challenge 4Bonus: how might the mirrorReverse function be implemented with a single index?One possible s

EXAMPLES

Example 1	
Input	`arr = ['t','h','e',' ','s','k','y',' ','i','s',' ','b','l','u','e']`
Output	`''.join(reverse_words(list('the sky is blue'))) -> 'blue is sky the'`
Why	**The words come out in reverse order while each word keeps its letters in order (the array is reversed in place).**

FULL CODE | main question first, then every follow-up, tests at the end

```
# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

import random


# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
def reverse_range(arr, lo, hi):
    # reverse arr[lo..hi] in place with two pointers
    while lo < hi:
        arr[lo], arr[hi] = arr[hi], arr[lo]
        lo += 1
        hi -= 1


def find_words(arr):
    # the (start, end) of every word: a run of characters without a space
    words = []
    i = 0
    while i < len(arr):
        if arr[i] == " ":
            i += 1
            continue
        j = i
        while j + 1 < len(arr) and arr[j + 1] != " ":
            j += 1
        words.append((i, j))
        i = j + 1
    return words
```

```
# ---- main question: main function ----
# Idea: reverse the WHOLE array first (the words are now in the right
# order but spelled backwards), then reverse each word back. Everything
# happens inside the same array: O(n) time, O(1) extra space. Spaces stay
# where the reversal puts them.
def reverse_words(arr):
    reverse_range(arr, 0, len(arr) - 1)
```

```
    for start, end in find_words(arr):
        reverse_range(arr, start, end)
    return arr


# ---- main question: tests ----
def test_main_question():
    assert "".join(reverse_words(list("perfect makes practice"))) == "practice makes perfect"
    assert reverse_words([]) == []
    assert reverse_words(list("one")) == list("one")
    assert "".join(reverse_words(list(" a  bc "))) == " bc  a "


# =====================================================
# FOLLOW-UP 1: The input may have leading, trailing or repeated spaces
# =====================================================
# ---- follow-up 1: function ----
# Answer: a split-based version already tidies them; the in-place version
# keeps them, so say which you want. To tidy in place, add a pass with a
# second pointer that WRITES the words with single spaces, then cut the
# array to the new length.
def normalise_spaces(arr):
    write = 0
    for read in range(len(arr)):
        if arr[read] != " ":
            arr[write] = arr[read]
            write += 1
        elif write > 0 and arr[write - 1] != " ":
            arr[write] = " "
            write += 1
    if write > 0 and arr[write - 1] == " ":
        write -= 1                                  # no trailing space
    del arr[write:]
    return arr


# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert "".join(normalise_spaces(list("  a   bc  "))) == "a bc"
    assert normalise_spaces(list("   ")) == []
    assert normalise_spaces(list("x")) == ["x"]
    tidy = reverse_words(normalise_spaces(list("  one   two three ")))
    assert "".join(tidy) == "three two one"
    random.seed(11)
    for round_number in range(300):                 # compare with the split-based answer
```

```
        s = ""
        for i in range(random.randint(0, 12)):
            s += random.choice("ab c")
        parts = s.split(" ")
        parts.reverse()
        assert "".join(reverse_words(list(s))) == " ".join(parts)


# =====================================================
# FOLLOW-UP 2: Reverse the letters of each word but keep the word order (LeetCode 557)
```

```python
# ============================================================
# ---- follow-up 2: function ----
# Answer: skip the first full reversal and reverse each word in place.
def reverse_each_word(arr):
    for start, end in find_words(arr):
        reverse_range(arr, start, end)
    return arr


# ---- follow-up 2: tests ----
def test_follow_up_2():
    assert "".join(reverse_each_word(list("let's code"))) == "s'tel edoc"


# ============================================================
# FOLLOW-UP 3: The sentence is a linked list of characters
# ============================================================
# Answer: reverse the whole list (the pointer flip), then reverse each
# word's nodes, finding each word by walking to the next space. Same
# idea, pointers instead of indices.


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

164 Find median of merged RLE arrays

MEDIUM

Template fit: walk both sorted run lists, always taking the smaller value, adding its count until you pass the middle position

QUESTION

Two sorted arrays are stored run-length encoded as (value, count) pairs in increasing order of value. Return the median of all their elements together, without expanding the runs (a count can be huge).

Function signature

def median_rle(a: list, b: list) -> float

EXAMPLES

Example 1	
Input	`[(1, 2), (5, 1)], [(3, 3)]`
Output	`3`
Why	`Combined sequence 1, 1, 3, 3, 3, 5: the two middle elements are 3 and 3.`

Example 2	
Input	`[(1, 1)], [(2, 1), (9, 1)]`
Output	`2`
Why	`Combined sequence 1, 2, 9: the middle one is 2.`

QUESTION

Example 3	
Input	`[(1, 1000000)], [(5, 1000000)]`
Output	3
Why	Counts may be enormous: a million 1s and a million 5s have median 3.

FULL CODE | main question first, then every follow-up, tests at the end

```
# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ========================================================

from bisect import bisect_right


# ========================================================
# MAIN QUESTION
# ========================================================
# ---- main question: helper functions ----
def total_count(runs):
    # how many elements the run-length list stands for
    total = 0
    for value, count in runs:
        total += count
    return total


def kth_smallest(runs_a, runs_b, k):
    # the element of (zero-based) rank k in the merged order, found by
    # walking the runs of the two sorted lists; nothing is expanded
    i = 0
    j = 0
    seen = 0
    while True:
        take_a = j >= len(runs_b)                    # b is used up: take from a
        if not take_a and i < len(runs_a):
            take_a = runs_a[i][0] <= runs_b[j][0]
        if take_a:
            value, count = runs_a[i]
        else:
            value, count = runs_b[j]
        seen += count
        if seen > k:
            return value
        if take_a:
            i += 1
        else:
            j += 1
```

```
# ---- main question: main function ----
# Idea: let N be the total number of elements. For odd N the median has
# rank N // 2; for even N it is the average of the elements of rank
# N // 2 - 1 and N // 2. Walk the two run lists in sorted order and stop
# at those ranks. a and b are [(value, count), ...] sorted by value.
def median_rle(a, b):
    total = total_count(a) + total_count(b)
    if total % 2 == 1:
        return kth_smallest(a, b, total // 2)
    low = kth_smallest(a, b, total // 2 - 1)
```

```
    high = kth_smallest(a, b, total // 2)
    return (low + high) / 2


# ---- main question: tests ----
A = [(1, 3), (4, 1)]
B = [(2, 2), (5, 2)]


def test_main_question():
    assert median_rle(A, B) == 2                    # merged: 1 1 1 2 2 4 5 5
    assert median_rle([(7, 6)], [(1, 1)]) == 7      # an odd total
    big = [(1, 10 ** 12)]
    assert median_rle(big, [(3, 10 ** 12)]) == 2    # huge counts are never expanded


# =====================================================
# FOLLOW-UP 1: K arrays instead of two
# =====================================================
# ---- follow-up 1: function ----
# Answer: put all the runs of all arrays together in value order (sort
# them, or merge with a heap of the K heads, O(R log K)) and walk to the
# two middle ranks. R is the total number of runs.
def median_k_arrays(arrays):
    runs = []
    for array in arrays:
        for run in array:
            runs.append(run)
    runs.sort()
    total = total_count(runs)

    def kth(k):
        seen = 0
        for value, count in runs:
            seen += count
            if seen > k:
                return value

    if total % 2 == 1:
        return kth(total // 2)
    return (kth(total // 2 - 1) + kth(total // 2)) / 2


# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert median_k_arrays([A, B]) == 2
    assert median_k_arrays([A, B, [(0, 1), (10, 1)]]) == 2


# =====================================================
# FOLLOW-UP 2: The arrays are huge and sorted, and you want O(log) time
# =====================================================


# =====================================================
# FOLLOW-UP 3: Runs are not sorted by value
# =====================================================
# ---- follow-up 2: function ----
# Answer: binary search on the VALUE. For a candidate value x count how
# many elements are <= x in each array (a binary search on the running
# totals of the counts), and find the smallest value that has more than k
# elements at or below it. O(log(range) * log(runs)). (If the runs are
# not sorted by value, sort them first: that costs O(R log R).)
def median_by_value(arrays):
    prepared = []                                   # (values, running totals of the counts) per array
```

```
    for runs in arrays:
        values = []
        sums = []
        running = 0
        for value, count in runs:
            running += count
            values.append(value)
            sums.append(running)
        prepared.append((values, sums))

    def count_at_most(x):
        total = 0
        for values, sums in prepared:
            i = bisect_right(values, x)
            if i > 0:
                total += sums[i - 1]
        return total

    def kth(k):
        lo = -(10 ** 18)
        hi = 10 ** 18
        while lo < hi:
            mid = (lo + hi) // 2
            if count_at_most(mid) > k:                # mid already has more than k elements below it
                hi = mid
            else:
                lo = mid + 1
        return lo

    n = count_at_most(10 ** 18)
    if n % 2 == 1:
        return kth(n // 2)
    return (kth(n // 2 - 1) + kth(n // 2)) / 2


# ---- follow-up 2: tests ----
def test_follow_up_2():
    assert median_by_value([A, B]) == 2
    three = [A, B, [(0, 1), (10, 1)]]
    assert median_by_value(three) == median_k_arrays(three)


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

165 Run-Length Vector Storage + Dot Product

MEDIUM

Template fit: one pointer per run list; multiply the values and take min(remaining counts) of the two current runs

QUESTION

Two vectors of the same length are stored run-length encoded as (value, count) pairs. Compute their dot product without expanding them to full arrays.

Function signature

def rle_dot(a: list, b: list) -> int

EXAMPLES

Example 1	
Input	`[(3, 3), (0, 3), (4, 2)], [(1, 4), (2, 4)]`
Output	`25`
Why	`A = [3,3,3,0,0,0,4,4], B = [1,1,1,1,2,2,2,2]: 3*1*3 + 0 + 4*2*2 = 9 + 16 = 25.`

Example 2	
Input	`[(2, 1)], [(5, 1)]`
Output	`10`
Why	`One element each: 2 * 5.`

FULL CODE | main question first, then every follow-up, tests at the end

```
# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

from bisect import bisect_right


# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
def rle_encode(values):
    # [3, 3, 3, 0, 0] -> [(3, 3), (0, 2)]: each run is (value, how many in a row)
    runs = []
    for v in values:
        if runs and runs[-1][0] == v:
            runs[-1] = (v, runs[-1][1] + 1)
        else:
            runs.append((v, 1))
    return runs


def rle_decode(runs):
    values = []
    for v, count in runs:
        for i in range(count):
            values.append(v)
    return values


# ---- main question: main function ----
# Idea: two pointers over the RUNS (never expand them). Look at what is
# left of the current run of a and of b, multiply the two values by the
# overlap, then move on in whichever run is used up (or in both).
```

```
def rle_dot(a, b):
    i = 0
    j = 0
    left_a = 0
    left_b = 0
    if a:
        left_a = a[0][1]
    if b:
        left_b = b[0][1]
    total = 0
    while i < len(a) and j < len(b):
        take = left_a                       # both runs overlap for min(left_a, left_b) positions
        if left_b < take:
            take = left_b
        total += a[i][0] * b[j][0] * take
        left_a -= take
        left_b -= take
        if left_a == 0:
            i += 1
            if i < len(a):
                left_a = a[i][1]
        if left_b == 0:
            j += 1
            if j < len(b):
                left_b = b[j][1]
    return total


# ---- main question: tests ----
A = [3, 3, 3, 0, 0, 0, 4, 4]
B = [1, 1, 1, 1, 2, 2, 2, 2]


def test_main_question():
    ra = rle_encode(A)
    rb = rle_encode(B)
    assert ra == [(3, 3), (0, 3), (4, 2)]
    assert rle_decode(ra) == A
    assert rle_dot(ra, rb) == 25                        # 3*1*3 + 0 + 4*2*2
    assert rle_dot([], rb) == 0
    dense = 0
    for i in range(len(A)):
        dense += A[i] * B[i]
    assert dense == 25


# ============================================================
# FOLLOW-UP 1: What if one vector is mostly zeros and stored sparsely, and the other as runs?
# ============================================================


# ============================================================
# FOLLOW-UP 3: One vector is a sparse map (index to value) and the other is run-length encoded
```

```
# ============================================================
# ---- follow-up 1: function ----
# Answer: loop over the SMALL side (the sparse non-zero indices) and, for
# each index, binary-search the run list for the value there, using the
# running totals of the run lengths: O(s log r) and nothing is expanded.
def sparse_dot_rle(sparse, runs):
    # sparse is {index: value}; ends[k] is where run k stops
    ends = []
    position = 0
    for value, count in runs:
        position += count
```

```
        ends.append(position)
    total = 0
    for index in sparse:
        k = bisect_right(ends, index)             # the run that contains this index
        if k < len(runs):
            total += sparse[index] * runs[k][0]
    return total


# ---- follow-up 1: tests ----
def test_follow_up_1():
    sparse = {0: 2, 7: 5}                          # non-zeros at index 0 and 7
    assert sparse_dot_rle(sparse, rle_encode(B)) == 2 * 1 + 5 * 2


# ======================================================
# FOLLOW-UP 2: How should the data be updated if you set element i?
# ======================================================
# ---- follow-up 2: function ----
# Answer: find the run that contains index i, split it into at most three
# runs (before, the new value, after), then merge equal neighbours.
def set_at(runs, index, value):
    pieces = []
    position = 0
    for v, count in runs:
        if position <= index < position + count:
            pieces.append((v, index - position))
            pieces.append((value, 1))
            pieces.append((v, position + count - index - 1))
        else:
            pieces.append((v, count))
        position += count
    merged = []
    for v, count in pieces:
```

```
        if count == 0:
            continue
        if merged and merged[-1][0] == v:
            merged[-1] = (v, merged[-1][1] + count)
        else:
            merged.append((v, count))
    return merged


# ---- follow-up 2: tests ----
def test_follow_up_2():
    ra = rle_encode(A)
    assert set_at(ra, 4, 3) == [(3, 3), (0, 1), (3, 1), (0, 1), (4, 2)]
    assert set_at([(1, 3)], 1, 1) == [(1, 3)]


# ======================================================
# RUN EVERYTHING
# ======================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

166 Count Unique Triplets That Sum to Zero

MEDIUM

Template fit: sort, fix the first number, then run the pair template on the rest; skip equal neighbours so each triple counts once

QUESTION

Given an integer array nums , count how many unique triplets (nums[i], nums[j], nums[k]) with three distinct indices i < j < k satisfy nums[i] + nums[j] + nums[k] == 0 . Two triplets are considered the same if they consist of the same multiset of values, regardless of the indices they came from. Return the number of distinct value-triplets (not the number of index combinations).

Function signature

def count_zero_sum_triplets(nums: list[int]) -> int:

    ...

Follow-up (as reported): Return the actual list of unique triplets instead of the count. Hint: sort the array, fix one element, and use two pointers while skipping duplicate values.

EXAMPLES

Example 1	
Input	`nums = [-1, 0, 1, 2, -1, -4]`
Output	`2`
Why	`The distinct value-triplets are [-1, -1, 2] and [-1, 0, 1]. Even though [-1, 0, 1] can be formed with different index sets, it is counted once.`

Example 2	
Input	`nums = [0, 0, 0, 0]`
Output	`1`
Why	`The only triplet is [0, 0, 0].`

Example 3	
Input	`nums = [1, 2, 3]`
Output	`0`
Why	`No three elements sum to 0.`

CONSTRAINTS

- 0 <= len(nums) <= 3000
- -10^5 <= nums[i] <= 10^5
- The array may contain duplicates.
- If len(nums) < 3 , the answer is 0 .

FULL CODE | main question first, then every follow-up, tests at the end

```
# ====================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ====================================================

import random


# ====================================================
# MAIN QUESTION
# ====================================================
# ---- main question: main function ----
# Idea: sort. Fix the smallest value nums[i] and sweep two pointers
```

```python
# (left, right) inward over the rest: a sum below 0 needs a bigger left
# value, a sum above 0 needs a smaller right value. Skip repeated values
# so each triplet is counted once.
def count_zero_sum_triplets(nums):
    nums = sorted(nums)
    count = 0
    for i in range(len(nums) - 2):
        if i > 0 and nums[i] == nums[i - 1]:
            continue                                # the same smallest value again
        left = i + 1
        right = len(nums) - 1
        while left < right:
            total = nums[i] + nums[left] + nums[right]
            if total < 0:
                left += 1
            elif total > 0:
                right -= 1
            else:
                count += 1
                left += 1
                right -= 1
                while left < right and nums[left] == nums[left - 1]:
                    left += 1                       # skip a repeated middle value
    return count


# ---- main question: tests ----
def test_main_question():
    assert count_zero_sum_triplets([-1, 0, 1, 2, -1, -4]) == 2
    assert count_zero_sum_triplets([0, 0, 0, 0]) == 1
```

```python
    assert count_zero_sum_triplets([1, 2, 3]) == 0
    assert count_zero_sum_triplets([]) == 0


# ======================================================
# FOLLOW-UP 1: Return the unique triplets themselves
# ======================================================
# ---- follow-up 1: function ----
# Answer: the same sweep, but on every hit append the triplet
# [nums[i], nums[left], nums[right]] instead of counting. The output can
# have O(n^2) triplets, which dominates the cost.
def zero_sum_triplets(nums):
    nums = sorted(nums)
    found = []
    for i in range(len(nums) - 2):
        if i > 0 and nums[i] == nums[i - 1]:
            continue
        left = i + 1
        right = len(nums) - 1
        while left < right:
            total = nums[i] + nums[left] + nums[right]
            if total < 0:
                left += 1
            elif total > 0:
                right -= 1
            else:
                found.append([nums[i], nums[left], nums[right]])
                left += 1
                right -= 1
                while left < right and nums[left] == nums[left - 1]:
                    left += 1
    return found


# ---- follow-up 1: tests ----
```

```
def brute_force_triplets(nums):
    # try every triple (slow, obviously right) and keep the distinct sorted ones
    seen = set()
    for i in range(len(nums)):
        for j in range(i + 1, len(nums)):
            for k in range(j + 1, len(nums)):
                if nums[i] + nums[j] + nums[k] == 0:
                    seen.add(tuple(sorted([nums[i], nums[j], nums[k]])))
    return seen
```

```
def test_follow_up_1():
    assert zero_sum_triplets([-1, 0, 1, 2, -1, -4]) == [[-1, -1, 2], [-1, 0, 1]]
    random.seed(15)
    for round_number in range(300):
        nums = []
        for i in range(random.randint(0, 9)):
            nums.append(random.randint(-4, 4))
        triplets = zero_sum_triplets(nums)
        assert len(triplets) == count_zero_sum_triplets(nums)
        assert set(tuple(t) for t in triplets) == brute_force_triplets(nums)


# =======================================================
# RUN EVERYTHING
# =======================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

Walk two strings at once with two indices

167 Find the First Occurrence of a Substring — EASY

Template fit: try every start i; inside, j walks the needle; a full match returns i

QUESTION

Reconstructing the "find substring in array" item as substring search: given two strings haystack and needle , return the index of the first occurrence of needle in haystack , or -1 if needle is not part of haystack .

Function signature

def str_str(haystack: str, needle: str) -> int:
    """Return the index of the first occurrence of needle in haystack, or -1."""
    ...

Follow-up (as reported): Achieve O(n + m) worst-case time with the Knuth-Morris-Pratt (KMP) failure-function approach instead of the naive O(n * m) scan.

EXAMPLES

Example 1	
Input	`haystack = "sadbutsad", needle = "sad"`
Output	`0`
Why	`"sad" occurs at indices 0 and 6; the first is index 0.`

Example 2	
Input	`haystack = "leetcode", needle = "leeto"`
Output	`-1`
Why	`"leeto" does not occur in "leetcode".`

Example 3	
Input	`haystack = "abc", needle = "c"`
Output	`2`

CONSTRAINTS

- 1 <= len(haystack), len(needle) <= 10^4 haystack and needle consist of only lowercase English letters.
- If needle is the empty string it is considered to occur at index 0 (though the constraints above make it non-empty).

FULL CODE | main question first, then every follow-up, tests at the end

```
# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ========================================================

import random


# ========================================================
# MAIN QUESTION
# ========================================================
# ---- main question: main function ----
# Idea: try every start position i of the haystack; compare the needle
# letter by letter and stop at the first mismatch. The first start where
```

```python
# all letters match is the answer; -1 if there is none.
def str_str(haystack, needle):
    n = len(haystack)
    m = len(needle)
    for i in range(n - m + 1):
        j = 0
        while j < m and haystack[i + j] == needle[j]:
            j += 1
        if j == m:
            return i
    return -1


# ---- main question: tests ----
def test_main_question():
    assert str_str("sadbutsad", "sad") == 0
    assert str_str("leetcode", "leeto") == -1
    assert str_str("abc", "c") == 2
    assert str_str("abc", "") == 0


# ============================================================
# FOLLOW-UP 1: Make it O(n + m) in the worst case
# ============================================================
# ---- follow-up 1: function ----
# Answer: Knuth-Morris-Pratt. table[i] is the length of the longest proper
# prefix of needle[:i + 1] that is also its suffix. When a letter fails
# to match, fall back to table[j - 1] instead of starting again, so the
# haystack is never re-read: O(n + m).
```

```python
def str_str_kmp(haystack, needle):
    if needle == "":
        return 0
    table = [0] * len(needle)
    j = 0
    for i in range(1, len(needle)):                # build the table from the needle itself
        while j > 0 and needle[i] != needle[j]:
            j = table[j - 1]
        if needle[i] == needle[j]:
            j += 1
        table[i] = j
    j = 0                                          # how many needle letters match right now
    for i in range(len(haystack)):
        while j > 0 and haystack[i] != needle[j]:
            j = table[j - 1]
        if haystack[i] == needle[j]:
            j += 1
        if j == len(needle):
            return i - j + 1
    return -1


# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert str_str_kmp("sadbutsad", "sad") == 0
    assert str_str_kmp("leetcode", "leeto") == -1
    assert str_str_kmp("abc", "") == 0
    random.seed(10)
    for round_number in range(500):                # both versions must agree
        haystack = ""
        for i in range(random.randint(0, 10)):
            haystack += random.choice("ab")
        needle = ""
        for i in range(random.randint(0, 4)):
            needle += random.choice("ab")
        assert str_str(haystack, needle) == str_str_kmp(haystack, needle)
```

```
# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")
```

```
if __name__ == "__main__":
    run_all_tests()
```

168 Determine if two words are sorted in lexicographic order. EASY

Template fit: walk both words; at the first differing letter compare the letters' ranks in the given alphabet; if one word ends first, it must be the shorter

QUESTION

Words are ordered using a custom alphabet: rank maps each letter to its position in that alphabet. Return whether word a comes before word b (or is equal to it). The first letter where the words differ decides; if one word is a prefix of the other, the shorter one comes first.

Function signature

def in_order(a: str, b: str, rank: dict) -> bool

EXAMPLES

Example 1

Input	`alphabet = 'hlabcdefgijkmnopqrstuvwxyz' # 'h' comes first, then 'l', then a, b, c ...`
	`rank = {c: i for i, c in enumerate(alphabet)}`
	`in_order('hello', 'leetcode', rank)`
	`in_order('word', 'world', rank)`
	`in_order('app', 'apple', rank)`

Output	`in_order('hello', 'leetcode', rank) -> true`
	`in_order('word', 'world', rank) -> false`
	`in_order('app', 'apple', rank) -> true`

Why	`'h' ranks before 'l', so hello comes first. word vs world differ at d versus l, and d ranks after l, so the order is wrong. A prefix always comes first.`

FULL CODE | main question first, then every follow-up, tests at the end

```
# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

import unicodedata
import random

ALPHABET = "abcdefghijklmnopqrstuvwxyz"


# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
```

```
def make_rank(order=ALPHABET):
    # the position of every letter in the alphabet: {"a": 0, "b": 1, ...}
    rank = {}
    for i in range(len(order)):
        rank[order[i]] = i
    return rank


def rank_of(rank, ch):
    # a character that is not in the alphabet is ranked after all the known ones
    if ch in rank:
        return rank[ch]
    return len(rank) + ord(ch)


# ---- main question: main function ----
# Idea: walk both words with one index. The first position where the
# letters differ decides: compare the two letters by their rank. If one
# word is a prefix of the other, the SHORTER one comes first ("app" is
# before "apple", not the other way round).
def in_order(a, b, rank):
    i = 0
    while i < len(a) and i < len(b):
        if a[i] != b[i]:
            return rank_of(rank, a[i]) < rank_of(rank, b[i])
        i += 1
    return len(a) <= len(b)                      # the prefix rule (and equal words are in order)
```

```
# ---- main question: tests ----
def test_main_question():
    rank = make_rank()
    assert in_order("apple", "banana", rank)
    assert not in_order("banana", "apple", rank)
    assert in_order("app", "apple", rank)           # the prefix rule
    assert not in_order("apple", "app", rank)
    assert in_order("same", "same", rank)
    assert in_order("", "a", rank)
    random.seed(10)
    for round_number in range(500):                 # with the normal alphabet it is plain string comparison
        a = ""
        for i in range(random.randint(0, 4)):
            a += random.choice("abc")
        b = ""
        for i in range(random.randint(0, 4)):
            b += random.choice("abc")
        assert in_order(a, b, rank) == (a <= b)


# ========================================================
# FOLLOW-UP 1: What about uppercase, accents, or case-insensitive order?
# ========================================================


# ========================================================
# FOLLOW-UP 3: Case-insensitive or accented comparison
# ========================================================
# ---- follow-up 1: function ----
# Answer: normalise both words first (casefold ignores case; the unicode
# "NFKD" form splits an accented letter into the letter plus an accent mark,
# which is then dropped), or give the rank map entries for the extra characters.
def fold(word):
    text = unicodedata.normalize("NFKD", word.casefold())
    result = ""
    for ch in text:
        if not unicodedata.combining(ch):
```

```
            result += ch
    return result


# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert fold("Great") == "great"
```

```
    assert fold("\u00e9clair") == "eclair"          # an accented e becomes a plain e


# ========================================================
# FOLLOW-UP 2: A whole list of words in an alien alphabet (LeetCode 953)
# ========================================================


# ========================================================
# FOLLOW-UP 4: Prefix rule
# ========================================================
# ---- follow-up 2: function ----
# Answer: a list is sorted when every ADJACENT pair is in order. Build the
# rank table once. The prefix rule (a longer word must not come before
# its own prefix) is already inside in_order.
def words_sorted(words, order=ALPHABET, fold_case=False):
    rank = make_rank(order)
    if fold_case:
        folded = []
        for w in words:
            folded.append(fold(w))
        words = folded
    for i in range(len(words) - 1):
        if not in_order(words[i], words[i + 1], rank):
            return False
    return True


# ---- follow-up 2: tests ----
def test_follow_up_2():
    alien = "hlabcdefgijkmnopqrstuvwxyz"
    assert words_sorted(["hello", "leetcode"], alien)
    alien = "worldabcefghijkmnpqstuvxyz"
    assert not words_sorted(["word", "world", "row"], alien)
    assert not words_sorted(["apple", "app"], ALPHABET)
    assert words_sorted([], ALPHABET)
    assert words_sorted(["x"], ALPHABET)
    assert not words_sorted(["A", "b"], ALPHABET)         # case only matters when asked for
    assert words_sorted(["A", "b"], fold_case=True)
    assert words_sorted(["\u00e9clair", "ecole"], fold_case=True)


# ========================================================
# RUN EVERYTHING
# ========================================================
```

```
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

169 Return all unique substrings of a given string. EASY

Template fit: sort all suffixes; the unique substrings are all prefixes minus the shared prefixes of neighbouring suffixes

QUESTION

Return every distinct substring of a string, sorted alphabetically (a substring is a contiguous piece, the whole string and single letters included).

Function signature

def unique_substrings(s: str) -> list

EXAMPLES

Example 1	
Input	`'abc'`
Output	`['a', 'ab', 'abc', 'b', 'bc', 'c']`
Why	`Six different substrings of 'abc'.`

Example 2	
Input	`'aaa'`
Output	`['a', 'aa', 'aaa']`
Why	`'a' appears three times but is listed once.`

FULL CODE | main question first, then every follow-up, tests at the end

```
# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

import random


# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: main function ----
# Idea (the simplest): cut out every slice s[i:j] and put it in a set (a
# set keeps each text only once), then sort. That is O(n^3): fine for a
# short string; the follow-ups make it faster.
def unique_substrings(s):
    seen = set()
    for i in range(len(s)):
        for j in range(i + 1, len(s) + 1):
            seen.add(s[i:j])
    return sorted(seen)


# ---- main question: tests ----
def test_main_question():
    assert unique_substrings("abc") == ["a", "ab", "abc", "b", "bc", "c"]
    assert unique_substrings("aaa") == ["a", "aa", "aaa"]
    assert unique_substrings("") == []


# ============================================================
# FOLLOW-UP 1: List them without building duplicates first
# ============================================================
# ---- follow-up 1: function ----
# Answer: insert every suffix into a trie (a tree with one character per
```

```
# level). A substring is NEW exactly when its trie node is created while
# inserting. O(n^2) time and nodes, and the substrings come out once each.
def trie_substrings(s):
    found = []
    root = {}
    for start in range(len(s)):
        node = root
        for end in range(start, len(s)):
```

```
            ch = s[end]
            if ch not in node:
                node[ch] = {}
                found.append(s[start:end + 1])      # a new node: a substring we have not seen
            node = node[ch]
    return found


# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert sorted(trie_substrings("aaa")) == ["a", "aa", "aaa"]


# =======================================================
# FOLLOW-UP 2: Return the k-th lexicographic distinct substring, or the longest repeated substring
# =======================================================
# ---- follow-up 2: helper functions ----
def suffix_array(s):
    # the start positions of all suffixes, in the sorted order of the suffixes
    pairs = []
    for i in range(len(s)):
        pairs.append((s[i:], i))
    pairs.sort()
    starts = []
    for suffix, i in pairs:
        starts.append(i)
    return starts


def lcp_array(s, sa):
    # lcp[i] = how many letters the suffix sa[i] shares with the one before it
    # in sorted order (Kasai's method: O(n) in total, because the shared
    # length drops by at most one from one start to the next)
    n = len(s)
    rank = [0] * n
    for i in range(n):
        rank[sa[i]] = i
    lcp = [0] * n
    h = 0
    for start in range(n):
        if rank[start] > 0:
            before = sa[rank[start] - 1]
            while start + h < n and before + h < n and s[start + h] == s[before + h]:
                h += 1
            lcp[rank[start]] = h
```

```
            if h > 0:
                h -= 1
        else:
            h = 0
    return lcp


# ---- follow-up 2: functions ----
# Answer: both use the suffix array with the LCP array. Walking the sorted
# suffixes, each suffix brings (its length - its LCP) NEW substrings, so
# the k-th one is found by subtracting those counts. The longest repeated
```

```
# substring is the largest LCP value.
def kth_substring(s, k):
    sa = suffix_array(s)
    lcp = lcp_array(s, sa)
    for i in range(len(sa)):
        fresh = len(s) - sa[i] - lcp[i]
        if k <= fresh:
            return s[sa[i]:sa[i] + lcp[i] + k]
        k -= fresh
    return None


def longest_repeated(s):
    if s == "":
        return ""
    sa = suffix_array(s)
    lcp = lcp_array(s, sa)
    best = 0
    for i in range(len(lcp)):
        if lcp[i] > lcp[best]:
            best = i
    return s[sa[best]:sa[best] + lcp[best]]


# ---- follow-up 2: tests ----
def test_follow_up_2():
    assert kth_substring("abc", 4) == "b"
    assert kth_substring("abc", 7) is None
    assert longest_repeated("banana") == "ana"
    assert longest_repeated("abcd") == ""
    random.seed(12)
    for round_number in range(200):
        s = ""
        for i in range(random.randint(0, 9)):
```

```
            s += random.choice("abc")
        everything = unique_substrings(s)
        for k in range(1, len(everything) + 1):
            assert kth_substring(s, k) == everything[k - 1]


# ========================================================
# FOLLOW-UP 3: The string has a million characters
# ========================================================
# ---- follow-up 3: function ----
# Answer: quadratic is out. Build a suffix AUTOMATON in O(n): every state
# stands for a group of substrings, and the number of distinct substrings
# is the sum of (length[v] - length[link[v]]) over its states. (Or use a
# suffix array built in O(n log n).)
def count_distinct(s):
    length = [0]                    # the longest substring of each state
    link = [-1]                     # the suffix link of each state
    transitions = [{}]              # state -> {letter: next state}
    last = 0
    for ch in s:
        cur = len(length)
        length.append(length[last] + 1)
        link.append(0)
        transitions.append({})
        p = last
        while p != -1 and ch not in transitions[p]:
            transitions[p][ch] = cur
            p = link[p]
        if p != -1:
            q = transitions[p][ch]
            if length[p] + 1 == length[q]:
```

```
                link[cur] = q
            else:
                clone = len(length)                # split q: a copy with the shorter length
                length.append(length[p] + 1)
                link.append(link[q])
                transitions.append(dict(transitions[q]))
                while p != -1 and transitions[p].get(ch) == q:
                    transitions[p][ch] = clone
                    p = link[p]
                link[q] = clone
                link[cur] = clone
        last = cur
    total = 0
    for v in range(1, len(length)):
```

```
        total += length[v] - length[link[v]]
    return total


# ---- follow-up 3: tests ----
def test_follow_up_3():
    assert count_distinct("aaa") == 3
    assert count_distinct("") == 0
    random.seed(13)
    for round_number in range(300):
        s = ""
        for i in range(random.randint(0, 9)):
            s += random.choice("abc")
        everything = unique_substrings(s)
        assert count_distinct(s) == len(everything)
        assert sorted(trie_substrings(s)) == everything


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

170 Can board states be transformed? MEDIUM

Template fit: skip the X squares in both strings; the remaining letters must match in order, and L may only move left, R only right

QUESTION

A board is a string of 'L', 'R', '#' and 'X'. 'X' is an empty square. An 'L' piece may move left into an empty square, an 'R' piece may move right into an empty square, pieces never jump over each other, and '#' (a wall piece) never moves. Return whether start can be turned into end by any number of moves.

Function signature

def can_transform(start: str, end: str) -> bool

EXAMPLES

QUESTION

Example 1	
Input	'RXXLRXRXL', 'XRLXXRRLX'
Output	true
Why	L moves left and R moves right while keeping their order: this classic board is reachable.

Example 2	
Input	'XL', 'LX'
Output	true
Why	L can move left.

Example 3	
Input	'LX', 'XL'
Output	false
Why	L cannot move to the right.

Example 4	
Input	'#X', 'X#'
Output	false
Why	A wall never moves.

FULL CODE | main question first, then every follow-up, tests at the end

```
# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ========================================================

# ========================================================
# MAIN QUESTION
# ========================================================
# ---- main question: helper functions ----
def pieces(board):
    # the symbols that are not blank ("X"), each with its position
    result = []
    for i in range(len(board)):
        if board[i] != "X":
            result.append((board[i], i))
    return result


# ---- main question: main function ----
# Idea: "L" can only move left, "R" only right, "X" is a blank cell and
# "#" is a wall that never moves. Pieces never pass each other, so the
# ORDER of the non-blank symbols is fixed. Take the pieces of both boards
# and compare them one by one (two pointers): the same symbol; an L may
# only end up further left (or in place), an R further right; a wall must
# stay exactly where it is. (Walls are also never jumped over, because
# the order of the pieces cannot change.)
def can_transform(start, end):
    if len(start) != len(end):
        return False
    a = pieces(start)
    b = pieces(end)
```

```
    if len(a) != len(b):
        return False
    for k in range(len(a)):
        symbol_a, i = a[k]
        symbol_b, j = b[k]
        if symbol_a != symbol_b:
            return False
        if symbol_a == "L" and i < j:
            return False                    # an L cannot move right
        if symbol_a == "R" and i > j:
            return False                    # an R cannot move left
        if symbol_a == "#" and i != j:
            return False                    # a wall cannot move
    return True


# ---- main question: tests ----
def test_main_question():
    assert can_transform("RXXLRXRXL", "XRLXXRRLX")
    assert not can_transform("X", "L")
    assert can_transform("XXXL", "XXLX")
    assert not can_transform("XR", "RX")              # R cannot move left
    assert not can_transform("LX", "XL")              # L cannot move right
    assert not can_transform("RL", "LR")              # pieces keep their order


# ========================================================
# FOLLOW-UP 1: Return the minimum number of moves
# ========================================================
# ---- follow-up 1: function ----
# Answer: pieces only have to keep their order, so each piece moves on its
# own from its start index to its end index: the minimum number of moves
# is the sum of the index differences (when the transformation is possible).
def min_moves(start, end):
    if not can_transform(start, end):
        return None
    a = pieces(start)
    b = pieces(end)
    total = 0
    for k in range(len(a)):
        difference = a[k][1] - b[k][1]
        if difference < 0:
            difference = -difference
        total += difference
    return total


# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert min_moves("RXXLRXRXL", "XRLXXRRLX") == 4
    assert min_moves("RXX", "XXR") == 2
    assert min_moves("RX", "XR") == 1
    assert min_moves("LL", "RR") is None


# ========================================================
# FOLLOW-UP 2: Add walls that cannot be crossed (next question)
# ========================================================
# ---- follow-up 2: tests ----
# Answer: a wall "#" is just a piece that must stay where it is; since
# pieces never pass each other nothing crosses it (can_transform above).
def test_follow_up_2():
    assert can_transform("R#X", "R#X")
    assert not can_transform("R#X", "X#R")            # R cannot jump over the wall
```

```
# ============================================================
# FOLLOW-UP 3: Pieces can move one step per round and several move at once
# ============================================================
# ---- follow-up 3: function ----
# Answer: if pieces never block each other, the number of rounds is the
# longest single trip. With blocking, simulate the rounds or use each
# piece's arrival time together with the order constraint.
def rounds_needed(start, end):
    if not can_transform(start, end):
        return None
    a = pieces(start)
    b = pieces(end)
    longest = 0
    for k in range(len(a)):
        trip = a[k][1] - b[k][1]
        if trip < 0:
            trip = -trip
        if trip > longest:
            longest = trip
    return longest


# ---- follow-up 3: tests ----
def test_follow_up_3():
    assert rounds_needed("RXX", "XXR") == 2
    assert rounds_needed("LL", "RR") is None


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    test_follow_up_3()
```

```
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

171 Decide string transform with directional tokens and walls

MEDIUM

Template fit: the same walk with dots as blanks and the movement rule from the statement (walls block)

QUESTION

Two equal-length strings start and target use the characters 'L', 'R', '.' (an empty slot) and 'X' (an immovable wall). An 'L' token can move left into empty slots, an 'R' token can move right into empty slots, tokens can never pass through each other or through a wall, and 'X' never moves. Return whether start can be turned into target by any number of moves.

Function signature

def can_transform(start: str, end: str) -> bool

EXAMPLES

QUESTION

Example 1	
Input	'R..L', '.RL.'
Output	true
Why	R moves one step right and L one step left: both stay in order.

Example 2	
Input	'L.', '.L'
Output	false
Why	L can only move left, never right.

Example 3	
Input	'X.', '.X'
Output	false
Why	A wall cannot move.

FULL CODE | main question first, then every follow-up, tests at the end

```
# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ========================================================

import itertools
from collections import deque


# ========================================================
# MAIN QUESTION
# ========================================================
# ---- main question: helper functions ----
def pieces(board):
    # everything except the empty "." slots, each with its position
    result = []
    for i in range(len(board)):
        if board[i] != ".":
            result.append((board[i], i))
    return result


# ---- main question: main function ----
# Idea: "L" moves left, "R" moves right, "X" is a wall that never
# moves, "." is an empty slot. Pieces never pass each other, so the order
# of the non-dot symbols is fixed. Walk both strings skipping the dots:
# the next symbol must be equal; a wall must be at the same index; an L
# must have start index >= end index; an R must have start index <= end index.
def can_transform(start, end):
    if len(start) != len(end):
        return False
    a = pieces(start)
    b = pieces(end)
    if len(a) != len(b):
        return False
    for k in range(len(a)):
        symbol_a, i = a[k]
        symbol_b, j = b[k]
        if symbol_a != symbol_b:
```

```
            return False
        if symbol_a == "X" and i != j:
            return False
        if symbol_a == "L" and i < j:
```

```
            return False
        if symbol_a == "R" and i > j:
            return False
    return True


# ---- main question: tests ----
def test_main_question():
    assert can_transform("R.L", ".RL") is True
    assert not can_transform("RL.", "LR.")
    assert can_transform("R.X.L", ".RX.L") is True         # a wall cannot move and splits the board
    assert not can_transform("R.X", "X.R")


# ========================================================
# FOLLOW-UP 1: Segments between walls are independent
# ========================================================
# Answer: yes: a wall never moves and nothing crosses it, so each
# segment between two walls can be solved on its own. can_transform does
# the same job in one pass because walls are compared by position.


# ========================================================
# FOLLOW-UP 2: Return the moves or the minimum number of steps
# ========================================================


# ========================================================
# FOLLOW-UP 4: Return the minimum number of moves
# ========================================================
# ---- follow-up 4: function ----
# Answer: when the transformation is possible every piece moves on its
# own from its start index to its end index, so the minimum is the sum of
# the index differences (checked below against a breadth-first search).
def min_moves(start, end):
    if not can_transform(start, end):
        return None
    a = pieces(start)
    b = pieces(end)
    total = 0
    for k in range(len(a)):
        difference = a[k][1] - b[k][1]
        if difference < 0:
            difference = -difference
        total += difference
```

```
    return total


# ---- follow-up 4: tests ----
def one_move_away(board):
    # the boards you get by moving one piece one step: an R into a dot on
    # its right, or an L into a dot on its left
    result = []
    for i in range(len(board)):
        if board[i] == "R" and i + 1 < len(board) and board[i + 1] == ".":
            result.append(board[:i] + ".R" + board[i + 2:])
        if board[i] == "L" and i > 0 and board[i - 1] == ".":
            result.append(board[:i - 1] + "L." + board[i + 1:])
    return result
```

```
def bfs_distances(start):
    # the number of moves from start to every board it can reach
    distance = {start: 0}
    queue = deque([start])
    while queue:
        board = queue.popleft()
        for nxt in one_move_away(board):
            if nxt not in distance:
                distance[nxt] = distance[board] + 1
                queue.append(nxt)
    return distance


def test_follow_up_4():
    assert min_moves("R..L", ".RL.") == 2
    assert min_moves("L.", ".L") is None
    for n in range(1, 5):                                  # every board of up to 4 cells, against a real sea
rch
        boards = []
        for symbols in itertools.product("LR.X", repeat=n):
            boards.append("".join(symbols))
        for start in boards:
            reach = bfs_distances(start)
            for end in boards:
                assert can_transform(start, end) == (end in reach)
                if end in reach:
                    assert min_moves(start, end) == reach[end]


# =====================================================
```

```
# FOLLOW-UP 3: The board is 2D
# =====================================================
# Answer: the order argument no longer holds in two dimensions (pieces
# can go around each other), so it becomes a search problem (BFS over
# board states) or a flow / matching problem.


# =====================================================
# RUN EVERYTHING
# =====================================================
def run_all_tests():
    test_main_question()
    test_follow_up_4()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

172 Determine if two sentences are similar. MEDIUM

Template fit: zip the two sentences; each pair of words must be equal or listed as similar (store pairs in both directions)

QUESTION

Determine if Two Sentences Are Similar. Two sentences are similar if they have the same length and each pair of corresponding words in the two sentences is similar. The similarity between words is defined by the provided list of similar word pairs. A word is always similar to itself. For example, if we have the list of similar word pairs as [("great", "good"), ("acting","drama"), ("skills","talent")], then the sentences "You have great acting skills" and "You have good drama talent" are similar. Examples sentence1 = ["Let's", "code", "in", "Python"] sentence2 = ["Let's", "program", "in", "Python"] similarPairs = [("code", "program"),] output: true sentence1 = ["I", "love", "to", "play", "football"] sentence2 = ["I", "love", "playing", "soccer"] similarPairs = [("play", "playing"), ("football", "soccer")] output: false, different sentence lengths sentence1 = ["Do", "you", "like", "coffee"] sentence2 = ["Do", "you", "love", "coffee"] similarPairs = [("like", "enjoy"), ("coffee", "tea"),] output: false

"like" is not similar to

"love" based on the given pairs. sentence1 = ["I", "really", "love", "leetcode", "and", "apples"] sentence2 = ["I", "so", "like", "codesignal", "and", "oranges"] similarPairs = [("very", "so"), ("love", "adore"), ("really", "very"), ("leetcode", "codesignal"), ("apples", "oranges"), ("like", "adore"),] output: trueThe provided solution checks if two given sentences are similar by using a graph-based approach. It constructs a graph where words are c

EXAMPLES

Example 1	
Input	`['great', 'acting', 'skills'], ['fine', 'drama', 'talent'], [['great', 'fine'], ['acting', 'drama'], ['skills', 'talent']]`
Output	`true`
Why	`Each word is paired with a similar word.`

Example 2	
Input	`['a'], ['a', 'b'], []`
Output	`false`
Why	`Sentences of different lengths are never similar.`

FULL CODE | main question first, then every follow-up, tests at the end

```
# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ========================================================

# ========================================================
# MAIN QUESTION
# ========================================================
# ---- main question: main function ----
# Idea: put every similar pair in a set, in both directions. Two sentences
# are similar when they have the same number of words and every word
# pair is equal or listed. This is NOT transitive: good ~ fine and
# fine ~ okay do not make good ~ okay.
def similar_direct(s1, s2, pairs):
    if len(s1) != len(s2):
        return False
    allowed = set()
    for a, b in pairs:
        allowed.add((a, b))
        allowed.add((b, a))
    for i in range(len(s1)):
        if s1[i] != s2[i] and (s1[i], s2[i]) not in allowed:
            return False
    return True
```

```
# ---- main question: tests ----
def test_main_question():
    pairs = [("great", "good"), ("acting", "drama"), ("skills", "talent")]
    a = "You have great acting skills".split()
    b = "You have good drama talent".split()
    assert similar_direct(a, b, pairs)
    assert not similar_direct(a, b[:-1], pairs)
    assert not similar_direct(["a"], ["b"], [])
    assert similar_direct(["a"], ["a"], [])
    chain = [("good", "fine"), ("fine", "okay")]
    assert not similar_direct(["good"], ["okay"], chain)         # not transitive


# =====================================================
# FOLLOW-UP 1: Similarity is transitive (a ~ b and b ~ c means a ~ c)
# =====================================================
# ---- follow-up 1: helper functions ----
```

```
class DisjointSet:
    # union-find over words: words in the same group are similar
    def __init__(self):
        self.parent = {}

    def find(self, x):
        if x not in self.parent:
            self.parent[x] = x
        root = x
        while self.parent[root] != root:
            root = self.parent[root]
        while self.parent[x] != root:              # path compression: point everything on the way at the roo
t
            following = self.parent[x]
            self.parent[x] = root
            x = following
        return root

    def union(self, a, b):
        self.parent[self.find(a)] = self.find(b)


# ---- follow-up 1: function ----
# Answer: union-find over the words: union each listed pair; two words are
# similar when they are equal or have the same root. Path compression
# keeps it near O(n + P).
def similar_transitive(s1, s2, pairs):
    if len(s1) != len(s2):
        return False
    groups = DisjointSet()
    for a, b in pairs:
        groups.union(a, b)
    for i in range(len(s1)):
        if s1[i] != s2[i] and groups.find(s1[i]) != groups.find(s2[i]):
            return False
    return True


# ---- follow-up 1: tests ----
def test_follow_up_1():
    chain = [("good", "fine"), ("fine", "okay")]
    assert similar_transitive(["good"], ["okay"], chain)
    assert not similar_transitive(["good"], ["bad"], chain)


# =====================================================
```

```
# FOLLOW-UP 2: One sentence is the other with a sentence of words inserted somewhere (LeetCode 1813)
# ============================================================
# ---- follow-up 2: function ----
# Answer: match the longest common PREFIX and the longest common SUFFIX of
# the two word lists; the shorter list must be covered by prefix plus
# suffix (they must not overlap).
def similar_insertion(s1, s2):
    if len(s1) > len(s2):
        s1, s2 = s2, s1                          # s1 is the shorter sentence
    head = 0
    while head < len(s1) and s1[head] == s2[head]:
        head += 1
    tail = 0
    room = len(s1) - head
    while tail < room and s1[-1 - tail] == s2[-1 - tail]:
        tail += 1
    return head + tail == len(s1)


# ---- follow-up 2: tests ----
def test_follow_up_2():
    assert similar_insertion("My name is Haley".split(), "My Haley".split())
    assert similar_insertion(["of"], ["A", "lot", "of"])
    assert similar_insertion("Eating right now".split(), ["Eating"])
    assert not similar_insertion(["a", "b"], ["c", "d"])


# ============================================================
# FOLLOW-UP 3: Case should not matter, or words can have synonyms with scores
# ============================================================
# ---- follow-up 3: function ----
# Answer: normalise the words with casefold() before comparing or hashing.
# For synonyms with scores, build a graph with weighted edges and ask
# "is there a path where every edge is above the threshold?" (the same
# union-find idea over the strong edges only).
def normalise(words):
    result = []
    for w in words:
        result.append(w.casefold())
    return result


# ---- follow-up 3: tests ----
def test_follow_up_3():
    assert normalise(["Great", "ACTING"]) == ["great", "acting"]
```

```
    assert similar_direct(normalise(["Great"]), normalise(["great"]), [])


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

173 Find Dictionary Matches for a Jammed String HARD

Template fit: compress every word to (letter, run length); a query matches when the letters agree and each run is equal or stretched to at least 3

QUESTION

A word was typed on a keyboard where some keys got stuck, so a letter may be repeated. Given a dictionary of words, matches(query) returns the dictionary words the query could be a jammed version of: the runs of equal letters line up with the word's runs, and each run of the query is either exactly as long as in the word or longer and at least min_stretch (default 3) letters long.

Function signature

class JammedIndex:
    def __init__(self, words: list)
    def matches(self, query: str, min_stretch: int = 3) -> list

EXAMPLES

Example 1

Input	`idx = JammedIndex(['hello', 'help', 'hel'])`
	`idx.matches('heeello')`
	`idx.matches('hello')`
	`idx.matches('heello')`
	`idx.matches('helllo')`

Output	`idx.matches('heeello') -> ['hello']`
	`idx.matches('hello') -> ['hello']`
	`idx.matches('heello') -> []`
	`idx.matches('helllo') -> ['hello']`

Why	'heeello' stretches the e to 3 letters, which is allowed. 'heello' only stretches it to 2 (shorter than min_stretch), and 'helllo' stretches the l run from 2 to 3 letters, which is allowed.

FULL CODE | main question first, then every follow-up, tests at the end

```
# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
def runs(word):
    # "heello" -> [("h", 1), ("e", 2), ("l", 2), ("o", 1)]: each letter with how many times in a row
    result = []
    for ch in word:
        if result and result[-1][0] == ch:
            result[-1] = (ch, result[-1][1] + 1)
        else:
            result.append((ch, 1))
    return result


def run_letters(word):
    # "hello" -> "helo": the letters of the runs, repeats collapsed
    letters = ""
    for ch, count in runs(word):
        letters += ch
    return letters
```

```
def run_counts(word):
    counts = []
    for ch, count in runs(word):
        counts.append(count)
    return counts


def stretch_fits(query_count, word_count, min_stretch):
    # a run matches when it is the same length, or when the query stretched
    # it to at least min_stretch letters
    return query_count == word_count or (query_count > word_count and query_count >= min_stretch)


# ---- main question: main function ----
# Idea: group the dictionary words by their run letters ("hello" -> "helo").
```

```
# A query can only match words in its own group, so compare the run
# lengths only there. Build the index once and reuse it for many queries.
# min_stretch is the shortest stretch that is allowed (follow-up 1: use 1
# for "any run may be longer").
class JammedIndex:
    def __init__(self, words):
        self.groups = {}
        for word in words:
            letters = run_letters(word)
            if letters not in self.groups:
                self.groups[letters] = []
            self.groups[letters].append((word, run_counts(word)))

    def matches(self, query, min_stretch=3):
        letters = run_letters(query)
        counts = run_counts(query)
        found = []
        for word, word_counts in self.groups.get(letters, []):
            all_fit = True
            for i in range(len(counts)):
                if not stretch_fits(counts[i], word_counts[i], min_stretch):
                    all_fit = False
            if all_fit:
                found.append(word)
        return sorted(found)


# ---- main question: tests ----
def test_main_question():
    index = JammedIndex(["hello", "help", "hi", "hip"])
    assert index.matches("heeeelloooo") == ["hello"]        # e stretched to 4 and o to 3
    assert index.matches("hello") == ["hello"]
    assert index.matches("heello") == []                    # a stretch of only 2 is not enough
    assert index.matches("hiiii") == ["hi"]
    assert index.matches("zzz") == []
    assert runs("aabccc") == [("a", 2), ("b", 1), ("c", 3)]


# ========================================================
# FOLLOW-UP 1: A different stretch rule: any run may be longer, no minimum
# ========================================================
# ---- follow-up 1: tests ----
# Answer: only the comparison changes; the index is the same. That is why
# min_stretch is a parameter: min_stretch=1 means any longer run is fine.
def test_follow_up_1():
```

```
    index = JammedIndex(["hello", "help", "hi", "hip"])
    assert index.matches("heello", min_stretch=2) == ["hello"]
    assert index.matches("hiip", min_stretch=1) == ["hip"]
```

```python
# ============================================================
# FOLLOW-UP 2: Millions of dictionary words and many queries
# ============================================================
# Answer: the index already does the heavy work once. For millions of
# words keep the index on disk or sharded by the run letters, and only
# load the one group a query needs.


# ============================================================
# FOLLOW-UP 3: The jammed string may also be missing letters
# ============================================================
# Answer: then run letters can differ, so the exact-group lookup is not
# enough. Use a trie over the words and walk it with the query, allowing
# a bounded number of skipped letters (edit distance style).


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

Simulate a machine: a class with state and one branch per kind of event

174 Mario-Style Score Tracker (OOP) EASY

Template fit: the template as written: an event name decides how score, lives and size change

QUESTION

Design a class that tracks the state of a platformer character across a sequence of events. The character has a score (starts at 0), a lives count (starts at 3), and a big flag (starts False). Process events with these rules: "coin" : add 100 to the score. "mushroom" : add 1000 to the score and set big = True . "enemy" : add 200 to the score (the character stomps an enemy). "damage" : if big is True , set big = False (no life lost); otherwise decrease lives by 1. Score is unchanged. The character is alive while lives > 0 . Once lives reaches 0, the game is over and any further events are ignored. Implement the class and a method to run a batch of events.

Function signature

```
class MarioGame:
    def __init__(self) -> None:
        """score=0, lives=3, big=False."""
        ...
    def process(self, event: str) -> None:
        """Apply one event, ignoring it if the game is already over."""
        ...
    def run(self, events: list[str]) -> None:
        """Apply a sequence of events in order."""
        ...
    def get_score(self) -> int:
        ...
    def is_alive(self) -> bool:
        ...
```

EXAMPLES

Example 1	
Input	`g = MarioGame(); g.run(["coin", "mushroom", "enemy", "damage", "damage"]); (g.get_score(), g.is_alive())`
Output	`(1300, True)`
Why	`coin -> 100; mushroom -> 1100, big=True; enemy -> 1300; damage -> big=False (no life lost); damage -> small, lives 3->2. Score 1300, still alive.`

Example 2	
Input	`g = MarioGame(); g.run(["damage", "damage", "damage", "coin"]); (g.get_score(), g.is_alive())`
Output	`(0, False)`
Why	`Three small-state hits drop lives 3->0 (game over). The trailing "coin" is ignored.`

CONSTRAINTS

- Event strings are always one of the four listed above.

FULL CODE | main question first, then every follow-up, tests at the end

```
# ====================================================
# HOW THIS FILE IS ORGANISED
```

```
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

import random


# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: main function ----
# Idea: a small class with the state (score, lives, big or small) and ONE
# method that owns the rules, one branch per kind of event. The game ends
# when the lives reach zero; events after that are ignored.
class MarioGame:
    def __init__(self):
        self.score = 0
        self.lives = 3
        self.big = False

    def process(self, event):
        if self.lives == 0:
            return                                  # the game is over
        if event == "coin":
            self.score += 100
        elif event == "mushroom":
            self.score += 1000
            self.big = True
        elif event == "enemy":
            self.score += 200
        elif event == "damage":
            if self.big:
                self.big = False                    # damage first costs the mushroom power-up
            else:
                self.lives -= 1

    def run(self, events):
        for event in events:
            self.process(event)

    def get_score(self):
        return self.score
```

```
    def is_alive(self):
        return self.lives > 0


# ---- main question: tests ----
def test_main_question():
    g = MarioGame()
    g.run(["coin", "mushroom", "enemy", "damage", "damage"])
    assert (g.get_score(), g.is_alive()) == (1300, True)
    g = MarioGame()
    g.run(["damage", "damage", "damage", "coin"])
    assert (g.get_score(), g.is_alive()) == (0, False)      # the coin after game over is ignored


# ============================================================
# FOLLOW-UP 1: Add undo() to take back the last event
# ============================================================
# ---- follow-up 1: function ----
# Answer: before applying an event push a snapshot (score, lives, big) on a
# stack; undo pops the last snapshot and restores the three fields. Events
# that are ignored after the game is over must NOT push a snapshot.
```

```
class UndoMarioGame(MarioGame):
    def __init__(self):
        MarioGame.__init__(self)
        self.history = []

    def process(self, event):
        if self.lives == 0:
            return
        self.history.append((self.score, self.lives, self.big))
        MarioGame.process(self, event)

    def undo(self):
        if len(self.history) > 0:
            self.score, self.lives, self.big = self.history.pop()


# ---- follow-up 1: tests ----
def test_follow_up_1():
    random.seed(60)
    for round_number in range(300):
        events = []
        for i in range(random.randint(0, 12)):
            events.append(random.choice(["coin", "mushroom", "enemy", "damage"]))
```

```
        game = UndoMarioGame()
        game.run(events)
        for i in range(len(events)):
            game.undo()
        assert (game.get_score(), game.lives, game.big) == (0, 3, False)       # undoing everything restores
the start


# ========================================================
# RUN EVERYTHING
# ========================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

175 Tic-Tac-Toe System with K Players and 3-in-a-Row Win

MEDIUM

Template fit: store the board; after each move count the run through the new cell in four directions and compare with the target length

QUESTION

Design Tic-Tac-Toe for k players on an n x n board. Players move in turn (1, 2, ..., k, then 1 again). move(player, r, c) places a mark and returns 'player p wins' as soon as that player has 3 marks in a straight line (row, column or diagonal), 'draw' when the board is full, and 'continue' otherwise. A move out of turn, off the board or onto a taken cell raises an error.

Function signature

class TicTacToe:
  def __init__(self, n: int, k: int, target: int = 3)
  def move(self, player: int, r: int, c: int) -> str

EXAMPLES

QUESTION

Example 1

Input	`g = TicTacToe(3, 2)`
	`g.move(1, 0, 0)`
	`g.move(2, 1, 0)`
	`g.move(1, 0, 1)`
	`g.move(2, 1, 1)`
	`g.move(1, 0, 2)`

Output	`g.move(1, 0, 0) -> 'continue'`
	`g.move(2, 1, 0) -> 'continue'`
	`g.move(1, 0, 1) -> 'continue'`
	`g.move(2, 1, 1) -> 'continue'`
	`g.move(1, 0, 2) -> 'player 1 wins'`

Why	Player 1 takes the whole top row while player 2 has two in the middle row: player 1 wins.

Example 2

Input	`g = TicTacToe(3, 3)`
	`g.move(1, 0, 0)`
	`g.move(2, 0, 1)`
	`g.move(3, 0, 2)`
	`g.move(1, 1, 1)`

Output	`g.move(1, 0, 0) -> 'continue'`
	`g.move(2, 0, 1) -> 'continue'`
	`g.move(3, 0, 2) -> 'continue'`
	`g.move(1, 1, 1) -> 'continue'`

Why	With three players the turns go 1, 2, 3, 1; nobody has three in a line yet.

FULL CODE | main question first, then every follow-up, tests at the end

```python
# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ========================================================

# ========================================================
# MAIN QUESTION
# ========================================================
# ---- main question: helper functions ----
DIRECTIONS = ((0, 1), (1, 0), (1, 1), (1, -1))        # right, down, and the two diagonals


# ---- main question: main function ----
# Idea: place the mark, then check only the four lines THROUGH it. For each
# direction walk outward both ways while the cells hold the same player and
# add the two counts plus one for the new mark. A run of at least `target`
# (3 by default) wins at once. Players take turns 1, 2, ..., k, 1, ...
# `target` is the winning length w (follow-up 1): the check costs O(w)
# per move, whatever the board size.
class TicTacToe:
    def __init__(self, n, k, target=3):
        self.n = n
        self.k = k
        self.target = target
        self.board = []
        for r in range(n):
            self.board.append([0] * n)              # 0 means an empty cell
        self.moves = 0
```

```
        self.over = False

    def inside(self, r, c):
        return 0 <= r < self.n and 0 <= c < self.n

    def run_length(self, r, c, dr, dc):
        player = self.board[r][c]
        run = 1
        for sign in (1, -1):
            a = r + sign * dr
            b = c + sign * dc
            while self.inside(a, b) and self.board[a][b] == player:
                run += 1
                a += sign * dr
                b += sign * dc
```

```
        return run

    def move(self, player, r, c):
        turn = self.moves % self.k + 1
        if self.over or player != turn:
            raise ValueError("not this player's turn")
        if not self.inside(r, c) or self.board[r][c] != 0:
            raise ValueError("illegal cell")
        self.board[r][c] = player
        self.moves += 1
        for dr, dc in DIRECTIONS:
            if self.run_length(r, c, dr, dc) >= self.target:
                self.over = True
                return "player " + str(player) + " wins"
        if self.moves == self.n * self.n:
            self.over = True
            return "draw"
        return "continue"


# ---- main question: tests ----
def play(game, plan):
    results = []
    for player, r, c in plan:
        results.append(game.move(player, r, c))
    return results


def test_main_question():
    results = play(TicTacToe(3, 2), [(1, 0, 0), (2, 1, 0), (1, 0, 1), (2, 1, 1), (1, 0, 2)])
    assert results[:-1] == ["continue"] * 4
    assert results[-1] == "player 1 wins"
    sequence = [(1, 0, 0), (2, 0, 3), (3, 3, 3), (1, 1, 0), (2, 1, 3), (3, 3, 0), (1, 2, 1), (2, 2, 3)]
    assert play(TicTacToe(4, 3), sequence)[-1] == "player 2 wins"      # three players: player 2 wins down co
lumn 3
    for bad in ((2, 0, 0), (1, 5, 5)):                                  # a wrong turn and an outside cell ar
e rejected
        try:
            TicTacToe(3, 2).move(bad[0], bad[1], bad[2])
            raise AssertionError(bad)
        except ValueError:
            pass
    g = TicTacToe(3, 2)                                                 # a full board with no winner is a dr
aw
    draw = [(1, 0, 0), (2, 0, 1), (1, 0, 2), (2, 1, 1), (1, 1, 0), (2, 1, 2), (1, 2, 1), (2, 2, 0), (1, 2, 2)
]
    assert play(g, draw)[-1] == "draw"
```

```python
# ========================================================
# FOLLOW-UP 1: What changes when the winning length is a parameter w?
# ========================================================
# ---- follow-up 1: tests ----
# Answer: use w instead of 3 in the run check (the `target` parameter).
# Each of the four directions walks at most w - 1 cells each way, so a move
# costs O(w), and the board size no longer matters: that is the point
# of checking only around the new mark.
def test_follow_up_1():
    game = TicTacToe(5, 2, target=4)
    plan = [(1, 0, 0), (2, 1, 0), (1, 0, 1), (2, 1, 1), (1, 0, 2), (2, 1, 2)]
    assert play(game, plan) == ["continue"] * 6                          # three in a row is not enough now
    assert game.move(1, 0, 3) == "player 1 wins"                         # four is


# ========================================================
# RUN EVERYTHING
# ========================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

176 Design Tic-Tac-Toe With K Players

MEDIUM

Template fit: per player Counters for rows, columns and both diagonals; a move wins when a counter reaches n

QUESTION

Design and implement a Tic-Tac-Toe game engine. You are given:

- **k** players, each with a unique player ID or mark.
- An **n x n** board, initially empty.
- Players take turns placing their own mark on an empty cell. Unlike classic Tic-Tac-Toe, a player wins as soon as they form **3 consecutive marks** in a straight line, regardless of how large **n** is. A straight line may be horizontal, vertical, diagonal down-right, or diagonal down-left. Implement a class or set of functions that supports making moves and returns the game status after each move, such as:
- **`"Game Over"`** if a move is attempted after the game has ended.
- **`"<player> won"`** when the latest move causes that player to have 3 consecutive marks.
- A non-terminal status such as **`"Continue"`** if the game should keep going. Your implementation should handle invalid moves, such as placing a mark outside the board or on an occupied cell. Discuss the time and space complexity of your approach, especially for large **n**.

Design a Tic-Tac-Toe game engine for k players on an n x n board. The board starts empty, and players must play in cyclic order: player 1, player 2, ..., player k, then back to player 1. A player wins immediately when their latest valid move creates 3 consecutive marks in a straight line. A straight line may be horizontal, vertical, diagonal down-right, or diagonal down-left. For this problem, implement a function that processes a list of move attempts and returns the status after each attempt. Each move is given as [player, row, col] using 0-based indexing. Return one status string per move:

- "Game Over" if a move is attempted after someone has already won.
- "Invalid move" if the move is not legal. A move is invalid if: - the player ID is not in [1, k] - it is not that player's turn - the cell is outside the board - the cell is already occupied - "<player> won" if the move is valid and causes that player to have 3 consecutive marks - "Continue" if the move is valid and the game should continue Important: an invalid move does not change the board and does not advance the turn. You do not need to detect draws. If no one wins, valid moves simply return "Continue".

QUESTION

Constraints

1 <= n <= 10^91 <= k <= 10^50 <= len(moves) <= 2 * 10^5Each move is a list of three integers: [player, row, col]Row and column in the input may be outside the board and should then be treated as an invalid moveYour approach should be efficient for large n, so avoid scanning the whole board after every move

EXAMPLES

```
Input: (3, 2, [])
Expected Output: []
```

Explanation: No moves are made, so there are no statuses to report.

```
Input: (5, 2, [[1, 0, 0], [2, 1, 0], [1, 0, 1], [2, 1, 1], [1, 0, 2], [2, 2, 2]])
Expected Output: ['Continue', 'Continue', 'Continue', 'Continue', '1 won', 'Game Over']
```

Explanation: Player 1 forms three consecutive marks horizontally at (0,0), (0,1), and (0,2). Any later move returns 'Game Over'.

Hints

Only the latest valid move can create a new 3-in-a-row, so you never need to rescan the entire board.For large boards, store only occupied cells in a hash map keyed by (row, col), then check the 4 line directions around the new move.

FULL CODE | main question first, then every follow-up, tests at the end

```python
# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ========================================================

# ========================================================
# MAIN QUESTION
# ========================================================
# ---- main question: helper functions ----
DIRECTIONS = ((0, 1), (1, 0), (1, 1), (1, -1))        # right, down, and the two diagonals


# ---- main question: main function ----
# Idea: only the four lines through the NEW mark can create a win. After
# a move, walk outward in both directions of each line while the cells hold
# the same player; the run length is those two counts plus the new mark.
# Players take turns 1, 2, ..., k, 1, ... An invalid move (wrong player,
# wrong turn, outside the board, taken cell) changes nothing.
# `target` is how many in a row win (follow-up 1); cells are stored in a
# dictionary, so only taken cells use memory (follow-up 3: huge board).
class TicTacToe:
    def __init__(self, n, k, target=3):
        self.n = n
        self.k = k
        self.target = target
        self.cells = {}                               # (row, column) -> the player who took it
        self.history = []                             # the moves so far, for undo
        self.winner = None

    def turn(self):
        return len(self.history) % self.k + 1

    def run_length(self, r, c, dr, dc, player):
        # the number of marks of `player` in a line through (r, c)
        run = 1
        for sign in (1, -1):
            a = r + sign * dr
            b = c + sign * dc
```

```
            while self.cells.get((a, b)) == player:
                run += 1
                a += sign * dr
                b += sign * dc
        return run
```

```
    def move(self, player, r, c):
        if self.winner is not None:
            return "Game Over"
        if player < 1 or player > self.k or player != self.turn():
            return "Invalid move"
        if not (0 <= r < self.n and 0 <= c < self.n) or (r, c) in self.cells:
            return "Invalid move"
        self.cells[(r, c)] = player
        self.history.append((r, c))
        for dr, dc in DIRECTIONS:
            if self.run_length(r, c, dr, dc, player) >= self.target:
                self.winner = player
                return str(player) + " won"
        return "Continue"

    # follow-up 2: take the last move back (this also reopens a finished game)
    def undo(self):
        if len(self.history) == 0:
            return False
        del self.cells[self.history.pop()]
        self.winner = None
        return True


# ---- main question: tests ----
def process(n, k, moves):
    game = TicTacToe(n, k)
    results = []
    for move in moves:
        results.append(game.move(move[0], move[1], move[2]))
    return results


MOVES = [[1, 0, 0], [2, 1, 0], [1, 0, 1], [2, 1, 1], [1, 0, 2], [2, 2, 2]]


def test_main_question():
    assert process(3, 2, MOVES) == ["Continue", "Continue", "Continue", "Continue", "1 won", "Game Over"]
    bad = "Invalid move"
    ok = "Continue"
    got = process(3, 2, [[2, 0, 0], [1, 5, 5], [1, 0, 0], [2, 0, 0], [3, 1, 1], [2, 1, 1]])
    assert got == [bad, bad, ok, bad, bad, ok]          # invalid moves do not use up the turn
    big = TicTacToe(10 ** 9, 2)                          # a huge board costs nothing: only taken cells are s
tored
    assert big.move(1, 10 ** 9 - 1, 5) == "Continue"
```

```


# ============================================================
# FOLLOW-UP 1: Win with m in a row, not the full line
# ============================================================
# ---- follow-up 1: tests ----
# Answer: counters per row and column no longer work, because a run can sit
# anywhere in a line. After each move look along the four directions from
# the new cell and count same-player cells in both directions (the `target`
# parameter): O(m) per move, whatever the board size.
def test_follow_up_1():
    game = TicTacToe(5, 2, target=4)
    for player, r, c in [(1, 0, 0), (2, 1, 0), (1, 0, 1), (2, 1, 1), (1, 0, 2), (2, 1, 2)]:
```

```
        assert game.move(player, r, c) == "Continue"
    assert game.move(1, 0, 3) == "1 won"                  # four in a row


# =====================================================
# FOLLOW-UP 2: Undo the last move
# =====================================================
# ---- follow-up 2: tests ----
# Answer: remember the moves in a list; undo removes the last cell and
# clears the winner (TicTacToe.undo above).
def test_follow_up_2():
    game = TicTacToe(3, 2)
    for move in MOVES[:5]:
        game.move(move[0], move[1], move[2])
    assert game.winner == 1
    assert game.undo() and game.winner is None
    assert game.move(1, 0, 2) == "1 won"


# =====================================================
# FOLLOW-UP 3: The board is huge or unbounded
# =====================================================
# Answer: the cells are in a dictionary, so only taken cells use memory,
# and a win is checked locally around the new mark; neither depends on n.


# =====================================================
# RUN EVERYTHING
# =====================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
```

```
    test_follow_up_2()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

177 Infinite Deck Card Duel

MEDIUM

Template fit: an infinite deck is just a random draw; play rounds, compare the highest cards, and decide the champion by the lead in wins

QUESTION

Design an InfiniteDeck that can be drawn from forever: each call to draw() returns a uniformly random integer card value in [1, 13] (cards are never exhausted). Make the deck seedable so games are reproducible. Then simulate a duel between two players. In each round, each player draws hand_size cards. The round winner is the player holding the single highest card that round; if both players' highest cards are equal, the round is a draw (nobody scores). Play num_rounds rounds and tally round wins per player. Finally, declare a champion: the player is champion only if they lead the other in round wins by at least 2; otherwise there is no champion. Implement: InfiniteDeck.draw() returning the next card. round_winner(hand1, hand2) -> 1 , 2 , or 0 (0 meaning draw), based on the highest card in each hand. decide_champion(wins1, wins2) -> 1 , 2 , or 0 (0 meaning no champion), applying the "lead by at least 2" rule. play_game(...) tying it together (uses the deck for randomness).

QUESTION

Function signature

```
import random
class InfiniteDeck:
    def __init__(self, seed: int | None = None) -> None:
        """Create a deck backed by a (optionally seeded) RNG."""
        ...
    def draw(self) -> int:
        """Return a uniformly random card value in [1, 13]."""
        ...
def round_winner(hand1: list[int], hand2: list[int]) -> int:
    """Return 1 if player 1's max card is higher, 2 if player 2's is higher, 0 on a
tie."""
    ...
def decide_champion(wins1: int, wins2: int) -> int:
    """Return the champion (1 or 2) only if they lead by >= 2 round wins, else 0."""
    ...
def play_game(deck: InfiniteDeck, hand_size: int = 5, num_rounds: int = 100) -> int:
    """Play the duel and return the champion via decide_champion, or 0 if none."""
    ...
```

Follow-up (as reported): Extend to n players: replace the two win counters with a dictionary keyed by player id and pick the highest card across all hands each round. Introduce multiple independent InfiniteDeck instances (or one shared deck with per-card metadata) to support probabilistic house rules.

EXAMPLES

Example 1	
Input	`round_winner([3, 7, 2], [7, 1, 9])`
Output	`2`
Why	`Player 2's highest card (9) beats player 1's highest (7).`

Example 2	
Input	`round_winner([5, 5], [5, 4])`
Output	`0`
Why	`Both players' highest card is 5, so the round is a draw.`

CONSTRAINTS

- 1 <= hand_size <= 20 , 1 <= num_rounds <= 100000 , card values in [1, 13] .

FULL CODE | main question first, then every follow-up, tests at the end

```
# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ========================================================

import random


# ========================================================
# MAIN QUESTION
# ========================================================
# ---- main question: helper functions ----
```

```
class InfiniteDeck:
    # an endless deck: every draw is a random card from 1 to 13. The deck
    # owns its random generator, so a seed makes a game repeatable.
    def __init__(self, seed=None):
        self.rng = random.Random(seed)

    def draw(self):
        return self.rng.randint(1, 13)


def best_card(hand):
    best = hand[0]
    for card in hand:
        if card > best:
            best = card
    return best


# ---- main question: main functions ----
# The rules are plain functions of hands and win counts.
# A round: each player draws a hand; the player whose BEST card is higher
# wins the round; equal best cards give nobody a point.
def round_winner(hand1, hand2):
    best1 = best_card(hand1)
    best2 = best_card(hand2)
    if best1 == best2:
        return 0
    if best1 > best2:
        return 1
    return 2
```

```
# The champion is the player who leads by at least 2 rounds, else nobody (0).
def decide_champion(wins1, wins2):
    if wins1 - wins2 >= 2:
        return 1
    if wins2 - wins1 >= 2:
        return 2
    return 0


def play_game(deck, hand_size=5, num_rounds=100):
    wins = {1: 0, 2: 0}
    for round_number in range(num_rounds):
        hand1 = []
        hand2 = []
        for i in range(hand_size):
            hand1.append(deck.draw())
        for i in range(hand_size):
            hand2.append(deck.draw())
        winner = round_winner(hand1, hand2)
        if winner != 0:
            wins[winner] += 1
    return decide_champion(wins[1], wins[2])


# ---- main question: tests ----
def test_main_question():
    assert round_winner([3, 7, 2], [7, 1, 9]) == 2
    assert round_winner([5, 5], [5, 4]) == 0
    assert decide_champion(51, 49) == 1
    assert decide_champion(50, 49) == 0
    assert decide_champion(10, 13) == 2
    random.seed(28)
    for seed in range(40):                          # the same seed gives the same game
        hand = random.randint(1, 4)
```

```
        rounds = random.randint(1, 12)
        assert play_game(InfiniteDeck(seed), hand, rounds) == play_game(InfiniteDeck(seed), hand, rounds)


# ======================================================
# FOLLOW-UP 1: Extend it to n players
# ======================================================
# ---- follow-up 1: function ----
# Answer: replace the two counters with a dictionary keyed by player. Each
# round every player draws a hand; the round goes to the player whose
```

```
# best card is strictly the highest, and a tie for the top gives nobody a
# point. The champion leads the second best by at least 2 rounds.
def play_game_n(deck, players, hand_size=5, num_rounds=100):
    wins = {}
    for p in range(1, players + 1):
        wins[p] = 0
    for round_number in range(num_rounds):
        best = {}
        for p in wins:
            hand = []
            for i in range(hand_size):
                hand.append(deck.draw())
            best[p] = best_card(hand)
        top = max(best.values())
        leaders = []
        for p in best:
            if best[p] == top:
                leaders.append(p)
        if len(leaders) == 1:
            wins[leaders[0]] += 1
    counts = sorted(wins.values(), reverse=True)
    if len(counts) > 1 and counts[0] - counts[1] >= 2:
        for p in wins:
            if wins[p] == counts[0]:
                return p
    return 0


# ---- follow-up 1: tests ----
def test_follow_up_1():
    for seed in range(30):
        assert play_game_n(InfiniteDeck(seed), 2, 3, 30) in (0, 1, 2)
        assert play_game_n(InfiniteDeck(seed), 4, 3, 30) in (0, 1, 2, 3, 4)
    assert play_game_n(InfiniteDeck(1), 3, 3, 0) == 0                # no rounds: no champion


# ======================================================
# RUN EVERYTHING
# ======================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")
```

```
if __name__ == "__main__":
    run_all_tests()
```

178 Implement iterator merging lists with filtering MEDIUM

Template fit: keep a position per list and a turn counter; advance() looks for the next unblocked, unseen value and stores it for next()/has_next()

QUESTION

Design an iterator over several lists that takes one element from each list in turn (a zigzag), skipping any value in the blocked set and any value that was already returned. hasNext() says whether another value is available and next() returns it.

Function signature

class FilteredZigzag:
  def __init__(self, lists: list, blocked=())
  def hasNext(self) -> bool
  def next(self) -> int

EXAMPLES

Example 1

Input	`it = FilteredZigzag([[1, 2, 3], [4, 5, 6, 7], [8, 9]], blocked=[2, 5])`
	`out = []`
	`while it.hasNext():`
	`out.append(it.next())`
	`out`

Output	`out -> [1, 4, 8, 9, 3, 6, 7]`
Why	`Order of visit: 1, 4, 8, then 2 (blocked), 5 (blocked), 9, then 3, 6, then 7. The blocked values never appear.`

Example 2

Input	`it = FilteredZigzag([[1, 1], [1, 2]])`
	`out = []`
	`while it.hasNext():`
	`out.append(it.next())`
	`out`

Output	`out -> [1, 2]`
Why	`Repeated values are returned only once.`

FULL CODE | main question first, then every follow-up, tests at the end

```
# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ========================================================

import heapq


# ========================================================
# MAIN QUESTION
# ========================================================
# ---- main question: main function ----
# Idea: take one item from each list in turn (list 0, list 1, ..., back
# to list 0), skipping ids that are blocked and ids that were already
# returned. Keep one item of LOOKAHEAD ready, so hasNext() only has to
# check whether there is one. advance() goes round and round until it
# finds the next good item, or gives up after a whole round in which every
# list was empty.
class FilteredZigzag:
    def __init__(self, lists, blocked=()):
```

```
        self.lists = lists
        self.blocked = set(blocked)
        self.position = [0] * len(lists)            # how far we have read in each list
        self.turn = 0                               # whose turn it is
        self.seen = set()                           # the ids already returned
        self.ahead = None                           # the next item to return
        self.advance()

    def advance(self):
        self.ahead = None
        empty_in_a_row = 0
        while empty_in_a_row < len(self.lists):
            i = self.turn
            self.turn = (self.turn + 1) % len(self.lists)
            if self.position[i] >= len(self.lists[i]):
                empty_in_a_row += 1                 # this list has nothing left
                continue
            value = self.lists[i][self.position[i]]
            self.position[i] += 1
            empty_in_a_row = 0
            if value not in self.blocked and value not in self.seen:
                self.ahead = value
                return
```

```
    def hasNext(self):
        return self.ahead is not None

    def next(self):
        value = self.ahead
        self.seen.add(value)
        self.advance()
        return value


# ---- main question: tests ----
def test_main_question():
    it = FilteredZigzag([[1, 2, 3], [2, 4], [5]], blocked=[4])
    values = []
    while it.hasNext():
        values.append(it.next())
    assert values == [1, 2, 5, 3]                   # then 2 is seen already and 4 is blocked
    assert not FilteredZigzag([[], []]).hasNext()
    assert not FilteredZigzag([]).hasNext()


# ========================================================
# FOLLOW-UP 1: The lists are sorted and the output must stay sorted
# ========================================================
# ---- follow-up 1: function ----
# Answer: replace the round robin by a k-way merge with a heap
# (heapq.merge). Equal values come out next to each other, so drop a value
# equal to the previous output instead of keeping a `seen` set: memory
# O(k). A generator hands the values out one at a time.
def merge_sorted(lists, blocked=()):
    blocked = set(blocked)
    last = None
    started = False
    for value in heapq.merge(*lists):
        if value in blocked:
            continue
        if started and value == last:
            continue                                # the same value as the one just returned
        started = True
        last = value
        yield value
```

```
# ---- follow-up 1: tests ----
```

```
def test_follow_up_1():
    assert list(merge_sorted([[1, 3, 5], [1, 2, 5]], blocked=[3])) == [1, 2, 5]
    assert list(merge_sorted([[], []])) == []


# ========================================================
# FOLLOW-UP 2: Read the lists one after another, not interleaved
# ========================================================
# ---- follow-up 2: function ----
# Answer: go through the lists one after another (a generator); the same
# blocked and seen filtering applies. A generator plus one item of lookahead
# gives the hasNext / next behaviour.
def read_in_turn(lists, blocked=()):
    blocked = set(blocked)
    seen = set()
    for one_list in lists:
        for value in one_list:
            if value not in blocked and value not in seen:
                seen.add(value)
                yield value


# ---- follow-up 2: tests ----
def test_follow_up_2():
    assert list(read_in_turn([[1, 2], [2, 3]], [1])) == [2, 3]


# ========================================================
# FOLLOW-UP 3: The seen set grows without bound
# ========================================================
# Answer: for sorted input (follow-up 1) only the previous value is
# needed. Otherwise keep the seen ids in a bounded structure: an LRU set
# if repeats are close together, or a Bloom filter if a small chance of a
# wrongly skipped id is acceptable.


# ========================================================
# RUN EVERYTHING
# ========================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    print("all tests passed")
```

```
if __name__ == "__main__":
    run_all_tests()
```

179 Design an elevator control system

MEDIUM

Template fit: each car keeps floor, direction and stops; a request goes to the car with the lowest cost; step() moves every car one floor

QUESTION

Design the control logic for a multi-elevator building. A Request(floor, direction) is a call from a floor (direction +1 for up, -1 for down). Each Elevator keeps its current floor, direction and a set of stops. cost(request) estimates how expensive it is for this car to take the request (the distance, plus a detour penalty when the car is heading away); the dispatcher gives the request to the cheapest car. step() moves the car one floor towards its stops and returns the floor it served, if any.

Function signature

class Elevator:
  def __init__(self, car_id, floor=0)
  def cost(self, req: Request) -> float
  def step(self) -> int | None

EXAMPLES

Example 1	

Input	`e = Elevator(1, floor=0)`
	`e.stops.update({3, 5})`
	`[e.step() for _ in range(6)]`

Output	`[e.step() for _ in range(6)] -> [null, null, null, 3, null, 5]`
Why	The car climbs one floor per step and stops at floors 3 and 5 (served when it is on the floor at the start of a step).

Example 2	

Input	`idle = Elevator(2, floor=7)`
	`busy = Elevator(3, floor=2)`
	`busy.direction = 1`
	`busy.stops.update({9})`
	`idle.cost(Request(4, -1))`
	`busy.cost(Request(4, 1))`
	`busy.cost(Request(4, -1))`

Output	`idle.cost(Request(4, -1)) -> 3`
	`busy.cost(Request(4, 1)) -> 2`
	`busy.cost(Request(4, -1)) -> 14`

Why	An idle car costs its distance (3). A car already going up that will pass floor 4 going the same way costs its distance (2); going the other way it must finish its run first, so it costs much more.

FULL CODE | main question first, then every follow-up, tests at the end

```python
# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ========================================================

INF = float("inf")


# ========================================================
# MAIN QUESTION
# ========================================================
# ---- main question: helper functions ----
class Request:
    def __init__(self, floor, direction):
```

```
        self.floor = floor
        self.direction = direction            # +1 for a call going up, -1 for going down


# ---- main question: main function ----
# Idea: every car is a small state machine. It keeps a set of floors to
# stop at and a direction (SCAN, like a real elevator: keep going the same
# way while there are stops ahead, then reverse). step() is one tick: serve
# a stop on the current floor, then move one floor. cost(req) says how
# bad it would be to give this car a new call; a Controller (below) gives
# each call to the cheapest car.
class Elevator:
    def __init__(self, car_id, floor=0):
        self.id = car_id
        self.floor = floor
        self.direction = 0                    # 0 = idle, +1 = going up, -1 = going down
        self.stops = set()
        self.in_service = True

    def cost(self, req):
        if not self.in_service:
            return INF                        # a broken car never gets calls
        distance = abs(self.floor - req.floor)
        if self.direction == 0:
            return distance                   # idle: just the distance
        gap = req.floor - self.floor
        on_the_way = gap * self.direction >= 0        # the car is moving toward the floor ...
        if on_the_way and req.direction == self.direction:
            return distance                           # ... and the call goes the same way
```

```
        highest = max(self.stops)                     # otherwise the car finishes its sweep first
        lowest = min(self.stops)
        return distance + 2 * (highest - lowest + 1) + 10

    def step(self):
        served = None
        if self.floor in self.stops:
            self.stops.discard(self.floor)
            served = self.floor
        if len(self.stops) == 0:
            self.direction = 0
            return served
        stops_above = False
        stops_below = False
        for stop in self.stops:
            if stop > self.floor:
                stops_above = True
            if stop < self.floor:
                stops_below = True
        if (self.direction >= 0 and stops_above) or (self.direction < 0 and not stops_below):
            self.direction = 1                # keep the direction while there are stops ahead
        else:
            self.direction = -1
        self.floor += self.direction
        return served


# ---- main question: tests ----
def test_main_question():
    car = Elevator(1, floor=0)
    car.stops.update({3, 5})
    assert [car.step(), car.step(), car.step(), car.step(), car.step(), car.step()] == [None, None, None, 3,
None, 5]
    assert car.direction == 0                         # nothing left: idle
    idle = Elevator(2, floor=4)
    assert idle.cost(Request(7, 1)) == 3
```

```
    idle.in_service = False
    assert idle.cost(Request(7, 1)) == INF


# =====================================================
# FOLLOW-UP 1: Peak traffic (morning lobby rush, evening exodus)
# =====================================================


# =====================================================
```

```
# FOLLOW-UP 4: Morning rush: idle cars should wait at the lobby
# =====================================================
# ---- follow-up 1: function ----
# Answer: add modes. In the morning rush park idle cars in the lobby and
# let them go up with a full load; in the evening do the opposite (spread
# idle cars over the upper floors). The Controller assigns calls to the
# cheapest car, fails over a broken car (follow-up 3), and parks idle cars.
class Controller:
    def __init__(self, cars, lobby=0):
        self.cars = cars
        self.lobby = lobby

    def assign(self, req):
        best_car = None
        best_cost = INF
        for car in self.cars:
            c = car.cost(req)
            if c < best_cost:
                best_cost = c
                best_car = car
        if best_car is None:
            return None                         # every car is out of service
        best_car.stops.add(req.floor)
        return best_car.id

    def tick(self):
        served = {}
        for car in self.cars:
            served[car.id] = car.step()
        return served

    def fail(self, car_id):
        # a broken car: take it out of service and hand its stops to the others
        for car in self.cars:
            if car.id == car_id:
                car.in_service = False
                orphans = car.stops
                car.stops = set()
                for floor in orphans:
                    self.assign(Request(floor, 0))

    def park_idle(self):
        # morning rush: idle cars wait in the lobby
        for car in self.cars:
            if car.in_service and len(car.stops) == 0:
```

```
                car.stops.add(self.lobby)


# ---- follow-up 1: tests ----
def test_follow_up_1():
    near = Elevator(1, floor=2)
    far = Elevator(2, floor=9)
    controller = Controller([near, far])
    assert controller.assign(Request(3, 1)) == 1       # the nearer car takes the call
```

```
    assert 3 in near.stops
    far.stops.add(5)
    controller.fail(2)                                  # the far car breaks: its stop moves to the other car
    assert 5 in near.stops and not far.in_service
    controller.park_idle()
    assert 0 not in far.stops                           # a broken car is not parked
    assert controller.assign(Request(1, 1)) == 1


# ============================================================
# FOLLOW-UP 2: Fairness and starvation
# ============================================================
# Answer: a call must not wait forever because new, cheaper calls keep
# arriving. Age the calls (the cost goes down the longer a call waits) or
# give every call a deadline after which it jumps the queue.


# ============================================================
# FOLLOW-UP 3: Safety and fault tolerance
# ============================================================
# Answer: take a failed car out of service and hand its stops to the other
# cars (Controller.fail), never move with the doors open, keep a hard
# limit on the load, and make the controller restart safely from the state
# of each car.


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
```

```
    run_all_tests()
```

180 Simulate In-Place Cellular State Updates

HARD

Template fit: encode old and new state in two bits of each cell so every cell is computed from the old state; shift at the end

QUESTION

Conway's Game of Life on a grid of 0 (dead) and 1 (alive) cells. A live cell with 2 or 3 live neighbours (of its 8 neighbours) stays alive, a dead cell with exactly 3 live neighbours becomes alive, and every other cell is dead in the next generation. All cells change at the same time. Do it in place with O(1) extra space; with wrap=True the grid wraps around its edges.

Function signature

def step_in_place(board: list, wrap=False) -> list

EXAMPLES

Example 1	
Input	`[[0, 1, 0], [0, 1, 0], [0, 1, 0]]`
Output	`[[0, 0, 0], [1, 1, 1], [0, 0, 0]]`
Why	`The vertical blinker turns into a horizontal one.`

QUESTION

Example 2	
Input	`[[1, 1], [1, 1]]`
Output	`[[1, 1], [1, 1]]`
Why	A 2x2 block is stable: every cell has 3 live neighbours.

Example 3	
Input	`[[0, 0, 0], [0, 1, 0], [0, 0, 0]]`
Output	`[[0, 0, 0], [0, 0, 0], [0, 0, 0]]`
Why	A lonely cell dies of isolation.

FULL CODE | main question first, then every follow-up, tests at the end

```
# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ========================================================

# ========================================================
# MAIN QUESTION
# ========================================================
# ---- main question: helper functions ----
AROUND = [(-1, -1), (-1, 0), (-1, 1), (0, -1), (0, 1), (1, -1), (1, 0), (1, 1)]     # the 8 neighbours


def live_neighbors(board, r, c, wrap=False):
    rows = len(board)
    cols = len(board[0])
    count = 0
    for dr, dc in AROUND:
        a = r + dr
        b = c + dc
        if wrap:
            a = a % rows                                # a torus: the edges are glued together
            b = b % cols
        elif not (0 <= a < rows and 0 <= b < cols):
            continue
        count += board[a][b] & 1                        # bit 0 is the OLD state, even for cells already update
d
    return count


# ---- main question: main function ----
# Idea (Game of Life without a second board): store TWO bits in every
# cell. Bit 0 is the current state, bit 1 will hold the NEXT state.
# Pass 1: count the live neighbours of each cell from bit 0 only
# (value & 1), apply the rules (3 neighbours -> alive; a live cell with 2
# -> stays alive) and write the result into bit 1. Pass 2: shift every cell
# right by one so the new state becomes bit 0.
def step_in_place(board, wrap=False):
    rows = len(board)
    cols = len(board[0])
    for r in range(rows):
        for c in range(cols):
            live = live_neighbors(board, r, c, wrap)
            alive = board[r][c] & 1
            if live == 3 or (alive == 1 and live == 2):
```

```
                board[r][c] |= 2                    # bit 1 stores the NEW state
    for r in range(rows):
        for c in range(cols):
            board[r][c] >>= 1                       # the new state moves into bit 0
    return board


# ---- main question: tests ----
def test_main_question():
    assert step_in_place([[0, 1, 0], [0, 1, 0], [0, 1, 0]]) == [[0, 0, 0], [1, 1, 1], [0, 0, 0]]
    assert step_in_place([[1, 1], [1, 1]]) == [[1, 1], [1, 1]]
    assert step_in_place([[0, 0, 0], [0, 1, 0], [0, 0, 0]]) == [[0, 0, 0], [0, 0, 0], [0, 0, 0]]


# ========================================================
# FOLLOW-UP 1: The board is infinite, or huge and mostly empty
# ========================================================
# ---- follow-up 1: function ----
# Answer: store only the LIVE cells in a set. Count, for every cell next
# to a live cell, how many live neighbours it has; the next live cells are
# those with 3, or with 2 when they are alive now.
def step_sparse(live):
    counts = {}
    for r, c in live:
        for dr, dc in AROUND:
            cell = (r + dr, c + dc)
            counts[cell] = counts.get(cell, 0) + 1
    result = set()
    for cell in counts:
        if counts[cell] == 3 or (counts[cell] == 2 and cell in live):
            result.add(cell)
    return result


# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert step_sparse({(1, 0), (1, 1), (1, 2)}) == {(0, 1), (1, 1), (2, 1)}       # a blinker flips
    assert step_sparse({(0, 0), (0, 1), (1, 0), (1, 1)}) == {(0, 0), (0, 1), (1, 0), (1, 1)}


# ========================================================
# FOLLOW-UP 2: Why is bit 0 used when counting neighbors?
# ========================================================
# Answer: while the board is being updated, some cells already carry
# their NEW state in bit 1. Counting with value & 1 reads only bit 0, the
# OLD state, so every cell is judged by the same generation. Without the
# mask a cell's new bit would be counted by its neighbours too.


# ========================================================
# FOLLOW-UP 3: Cells wrap around the edges (a torus)
# ========================================================
# ---- follow-up 3: tests ----
# Answer: take the neighbour's row and column modulo the board size (the
# `wrap` parameter above).
def test_follow_up_3():
    block = [[1, 1, 0, 0], [0, 0, 0, 0], [0, 0, 0, 0], [1, 1, 0, 0]]
    copy_for_wrap = [row[:] for row in block]
    assert step_in_place(copy_for_wrap, wrap=True) == block        # rows 3 and 0 touch: one 2 x 2 block, a s
till life
    assert step_in_place(block, wrap=False) == [[0] * 4 for i in range(4)]     # without wrapping both pairs
die


# ========================================================
# RUN EVERYTHING
```

```
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

GROUP 23 OF 43 | 6 QUESTIONS, ONE CODE TEMPLATE | 6 M

Top-K by frequency: Counter plus nsmallest

181 Top-K Frequent in Large Logs

MEDIUM

Template fit: count the first word of every line (the speaker); return the names of the k most frequent

QUESTION

A chat log is a list of lines, each starting with the speaker's name ('A says hi'). Return the k speakers who appear most often; ties are broken alphabetically by name.

Function signature

def top_k_speakers(chat_log: list, k: int) -> list

EXAMPLES

Example 1	
Input	`['A says hi', 'B says hi', 'A says bye', 'C says hi', 'A says yo'], 2`
Output	`['A', 'B']`
Why	`A spoke 3 times, B and C once each; B comes before C alphabetically.`

Example 2	
Input	`['A says hi', 'B says hi', 'A says bye', 'C says hi', 'A says yo'], 3`
Output	`['A', 'B', 'C']`
Why	`Asking for 3 returns everyone, most frequent first.`

FULL CODE | main question first, then every follow-up, tests at the end

```
# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

import heapq


# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
def speaker(line):
    # "A says hi" -> "A"
    return line.split(" ", 1)[0]


def count_speakers(chat_log):
    counts = {}
    for line in chat_log:
        name = speaker(line)
        counts[name] = counts.get(name, 0) + 1
    return counts


def rank_of(pair):
    # pair = (name, count): more messages first, ties by name
    return (-pair[1], pair[0])
```

```
# ---- main question: main function ----
# Idea: take the speaker out of each line and count them in a dictionary.
# heapq.nsmallest keeps a heap of only k entries, so it never sorts all d
# distinct users: O(n + d log k). Ties are broken by name so the output is
# always the same.
def top_k_speakers(chat_log, k):
    counts = count_speakers(chat_log)
    best = heapq.nsmallest(k, counts.items(), key=rank_of)
    names = []
    for name, count in best:
        names.append(name)
    return names
```

```
# ---- main question: tests ----
def test_main_question():
    log = ["A says hi", "B says hi", "A says bye", "C says hi", "A says yo"]
    assert top_k_speakers(log, 2) == ["A", "B"]
    assert top_k_speakers(log, 5) == ["A", "B", "C"]
    assert top_k_speakers([], 3) == []


# =====================================================
# FOLLOW-UP 1: How do you do it across many machines when the log does not fit on one?
# =====================================================
# ---- follow-up 1: function ----
# Answer: count on each machine (shard). Adding up the FULL counts of
# every shard gives the exact answer. Sending only each shard's local
# top-K is cheaper but can be wrong: a user who is second on every shard
# can be first overall.
def merge_shards(shard_logs, k):
    total = {}
    for log in shard_logs:
        counts = count_speakers(log)
        for name in counts:
            total[name] = total.get(name, 0) + counts[name]
    best = heapq.nsmallest(k, total.items(), key=rank_of)
    names = []
    for name, count in best:
        names.append(name)
    return names


# ---- follow-up 1: tests ----
def test_follow_up_1():
    shard1 = ["A says x", "B says x"]
    shard2 = ["B says y", "B says z"]                    # two shards that each rank someone else first
    assert merge_shards([shard1, shard2], 1) == ["B"]


# =====================================================
# FOLLOW-UP 2: What if the log arrives as an unbounded stream?
# =====================================================


# =====================================================
# FOLLOW-UP 4: The log is a stream and memory is limited
# =====================================================
```

```
# ---- follow-up 2: function ----
# Answer: keep a Misra-Gries summary with m counters: a user with more
# than n / (m + 1) of the n messages is guaranteed to stay in it. When a
# new user arrives and there is no room, EVERY counter pays one. A second
# pass (or a count-min sketch) can then give the real counts of the survivors.
class MisraGries:
```

```
    def __init__(self, m):
        self.m = m
        self.counts = {}

    def add(self, user):
        if user in self.counts:
            self.counts[user] += 1
        elif len(self.counts) < self.m:
            self.counts[user] = 1
        else:
            for u in list(self.counts):          # no room: every counter pays one
                self.counts[u] -= 1
                if self.counts[u] == 0:
                    del self.counts[u]

    def candidates(self):
        return set(self.counts)


# ---- follow-up 2: tests ----
def test_follow_up_2():
    summary = MisraGries(2)
    for user in "AAABBC":
        summary.add(user)
    assert "A" in summary.candidates()                  # A has 3 of 6 messages (more than 6 / 3), so it mus
t stay


# =============================================================
# FOLLOW-UP 3: Which approach would you choose and why?
# =============================================================
# Answer: for a log that fits in memory, count exactly and keep a size-k
# heap (simple, exact). For a huge log, use shards for an exact answer, or
# the Misra-Gries summary when memory is the limit and a small error is
# acceptable.


# =============================================================
# RUN EVERYTHING
# =============================================================
```

```
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

182 Rearrange Tasks With Cooldown — MEDIUM

*Template fit: only the biggest count matters: answer = max(len(tasks), (biggest - 1) * (n + 1) + how many tasks share the biggest count)*

QUESTION

You are given a list of tasks (letters; the same letter means the same kind of task). Every task takes one unit of time, and two tasks of the same kind must have at least n other tasks or idle units between them (the cooldown). The CPU may stay idle. Return the least number of time units needed to finish all the tasks.

Function signature

def least_interval(tasks: list, n: int) -> int

EXAMPLES

QUESTION

Example 1	
Input	['A', 'A', 'A', 'B', 'B', 'B'], 2
Output	8
Why	A B idle A B idle A B: three A's and three B's with a cooldown of 2 take 8 units.

Example 2	
Input	['A', 'A', 'A', 'B', 'B', 'B'], 0
Output	6
Why	No cooldown: just run the 6 tasks back to back.

Example 3	
Input	['A', 'B', 'C'], 5
Output	3
Why	All tasks differ, so no cooldown is ever needed.

FULL CODE | main question first, then every follow-up, tests at the end

```
# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ========================================================

import heapq
from collections import deque


# ========================================================
# MAIN QUESTION
# ========================================================
# ---- main question: helper functions ----
def count_tasks(tasks):
    counts = {}
    for task in tasks:
        counts[task] = counts.get(task, 0) + 1
    return counts


# ---- main question: main function ----
# The same task must be at least n slots apart. How many time slots
# (including idle ones) are needed?
# Idea (closed form): let `most` be the highest frequency and `tied` the
# number of tasks that have that frequency. The most frequent task needs
# (most - 1) full frames of n + 1 slots plus one last row holding every
# tied task: (most - 1) * (n + 1) + tied. If there are so many different
# tasks that no idle slot is needed, the answer is just the number of tasks.
def least_interval(tasks, n):
    if not tasks:
        return 0
    counts = count_tasks(tasks)
    most = 0
    for count in counts.values():
        if count > most:
            most = count
    tied = 0
    for count in counts.values():
        if count == most:
```

```
            tied += 1
    frame_length = (most - 1) * (n + 1) + tied
    if frame_length > len(tasks):
        return frame_length
```

```
    return len(tasks)


# ---- main question: tests ----
def test_main_question():
    assert least_interval(list("AAABBB"), 2) == 8
    assert least_interval(list("AAABBB"), 0) == 6
    assert least_interval([], 3) == 0
    assert least_interval(list("ABCDEFG"), 2) == 7        # many different tasks: no idle time needed


# ========================================================
# FOLLOW-UP 1: Print an actual schedule, not just the time
# ========================================================
# ---- follow-up 1: function ----
# Answer: a heap plus a cooldown queue. At every time step run the
# available task with the most copies left (a max-heap: push negative
# counts), then park it for n steps; when nothing is available write
# "idle". Tasks come back to the heap when their cooldown is over.
def schedule(tasks, n):
    heap = []
    counts = count_tasks(tasks)
    for task in counts:
        heap.append((-counts[task], task))
    heapq.heapify(heap)
    cooling = deque()                           # (time it is ready again, minus copies left, task)
    plan = []
    now = 0
    while heap or cooling:
        while cooling and cooling[0][0] <= now:
            ready_time, left, task = cooling.popleft()
            heapq.heappush(heap, (left, task))
        if heap:
            left, task = heapq.heappop(heap)    # the task with the most copies left
            plan.append(task)
            if left + 1 < 0:
                cooling.append((now + n + 1, left + 1, task))
        else:
            plan.append("idle")
        now += 1
    return plan


# ---- follow-up 1: tests ----
def is_valid(plan, n):
```

```
    # equal tasks must be more than n slots apart
    last = {}
    for i in range(len(plan)):
        task = plan[i]
        if task == "idle":
            continue
        if task in last and i - last[task] <= n:
            return False
        last[task] = i
    return True


def test_follow_up_1():
    plan = schedule(list("AAABBB"), 2)
    assert len(plan) == 8 and is_valid(plan, 2)
```

```
    idle = 0
    for slot in plan:
        if slot == "idle":
            idle += 1
    assert idle == 2
    for tasks, n in ((list("AAAABBC"), 3), (list("AB"), 5)):
        plan = schedule(tasks, n)
        assert is_valid(plan, n)
        assert len(plan) == least_interval(tasks, n)


# ============================================================
# FOLLOW-UP 2: Different cooldowns per task
# ============================================================
# Answer: remember for each task when it may run again (its own cooldown)
# instead of one n; the heap-and-queue schedule works unchanged.


# ============================================================
# FOLLOW-UP 3: Tasks have priorities or deadlines
# ============================================================
# Answer: change what the heap orders by: (priority or deadline first,
# then the remaining copies). The cooldown queue stays the same.


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
```

```
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

183 Implement frequency + distance top■K queries

MEDIUM

Template fit: count the point tuples; the tie-break is distance from the origin, then x, then y

QUESTION

You will implement solutions for two coding interview questions.

Question

1: Return the top-K points by frequency (with distance tie-break) You are given an array **`points`** of 2D integer points, where each point is represented as **`[x, y]`**. The array may contain duplicates (the same **`[x, y]`** can appear multiple times). Define:

- **`freq(p)`** = number of times point **p** appears in the array.
- **`dist(p)`** = squared Euclidean distance to the origin, **`x^2 + y^2`**. Return **k distinct points** sorted by the following priority:

1. Higher frequency first (**`freq`** descending)
2. If frequencies tie, smaller distance first (**`dist`** ascending)
3. If still tied, sort by **`x`** ascending, then **`y`** ascending

Input

- **`points`**: list of length **n**, each element **`[x, y]`**
- **`k`**: integer, **`1 <= k <= (# of distinct points)`**

QUESTION

Output

- A list of **k** distinct points following the ranking above.

Constraints (typical interview assumptions)

- **`1 <= n <= 2 * 10^5`**
- **`-10^4 <= x, y <= 10^4`** ---

Question

2: Time-based key-value store with “closest timestamp” lookup Design a data structure supporting the following operations:

- **`set(key, value, timestamp)`**: store the string **`value`** for string **`key`** at integer **`timestamp`**.
- Assume timestamps for the same **`key`** are inserted in **non-decreasing** order.
- **`getClosest(key, timestamp) -> value or ""`**:
- If **`key`** has no stored values, return **`""`**.
- Otherwise, return the **`value`** whose stored timestamp is **closest** to the query **`timestamp`**.
- If there is a tie (equal distance to two timestamps), return the value with the **smaller** timestamp.

Example

If for key **`"a"`** you stored timestamps **`[2, 10, 15]`** with values **`["x", "y", "z"]`**:

- **`getClosest("a", 11)`** returns **`"y"`** (10 is closest)
- **`getClosest("a", 14)`** returns **`"z"`** (15 is closest)
- **`getClosest("a", 12)`** returns **`"y"`** (10 and 15 are equally far; choose smaller timestamp)

What to discuss

- Time and space complexity of your approach - How you would improve a “naive” solution (e.g., linear scan) to a more optimal one

Constraints (typical interview assumptions)

- Up to **`2 * 10^5`** total operations - Keys and values are strings; timestamps are integers within typical 32-bit range

Part 1: Top-K Points by Frequency with Distance Tie-BreakGiven a list of 2D integer points, where duplicate points may appear, return the top **k** distinct points ranked by these rules: (1) higher frequency first, (2) if frequencies tie, smaller squared distance to the origin first, and (3) if still tied, smaller **x** first, then smaller **y**. The result must contain exactly **k** distinct points in ranked order.

Constraints

1 <= len(points) <= 2 * 10^5-10^4 <= x, y <= 10^41 <= k <= number of distinct points

EXAMPLES

```
Input: ([[1,1],[2,2],[2,2],[1,1],[2,2],[3,3]], 2)
Expected Output: [[2, 2], [1, 1]]
Explanation: Point [2,2] appears 3 times and [1,1] appears 2 times, so they are the top 2 distinct
    points.
Input: ([[1,2],[0,2],[1,2],[2,1],[0,2],[2,1],[3,0]], 3)
Expected Output: [[0, 2], [1, 2], [2, 1]]
Explanation: The points [0,2], [1,2], and [2,1] each appear twice. Their squared distances are 4, 5, and
    5, so [0,2] comes first. Between [1,2] and [2,1], x is smaller for [1,2].
```

Hints

First count how many times each point appears. After that, each distinct point only needs to be ranked once.Python can sort tuples directly, so think about building a sort key like **`(-frequency, distance, x, y)`**.

Part 2: Time-Based Key-Value Store with Closest Timestamp QueriesProcess a sequence of operations for a time-based key-value store. Each operation is either **`['set', key, value, timestamp]`** or **`['getClosest', key, timestamp]`**. For every **`getClosest`** operation, return the stored value whose timestamp is closest to the query tim

FULL CODE | main question first, then every follow-up, tests at the end

```
# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ========================================================

import heapq


# ========================================================
# MAIN QUESTION
# ========================================================
# ---- main question: helper functions ----
def count_points(points):
    # {(x, y): how many times the point appears}
    counts = {}
    for x, y in points:
        counts[(x, y)] = counts.get((x, y), 0) + 1
    return counts


def ranked_points(counts):
    # one row per point: (minus its count, squared distance from the origin, x, y).
    # A smaller row is better: more frequent first, then closer to the origin, then by x, y.
    # (Squared distances need no square root and are exact for whole numbers.)
    rows = []
    for (x, y), count in counts.items():
        rows.append((-count, x * x + y * y, x, y))
    return rows


# ---- main question: main functions ----
# Q1: the k best points: most frequent, then closest to the origin.
# Idea: count the points, make one ranking row per point, and keep a heap
# of the k smallest rows (heapq.nsmallest).
def top_points(points, k):
    best = heapq.nsmallest(k, ranked_points(count_points(points)))
    result = []
    for minus_count, distance, x, y in best:
        result.append([x, y])
    return result


# Q2: a store of values by key and time.
```

```
def first_not_before(items, ts):
    # binary search: the first index whose timestamp is >= ts
    low = 0
    high = len(items)
    while low < high:
        middle = (low + high) // 2
        if items[middle][0] < ts:
            low = middle + 1
        else:
            high = middle
    return low


def first_after(items, ts):
    # binary search: the first index whose timestamp is > ts
    low = 0
    high = len(items)
    while low < high:
        middle = (low + high) // 2
        if items[middle][0] <= ts:
```

```
            low = middle + 1
        else:
            high = middle
    return low


class TimeStore:
    # set(key, value, ts) stores a value; get_closest(key, ts) returns the value
    # stored at the time closest to ts (the earlier time if two are equally
    # close). Each key keeps its (timestamp, value) pairs in order of time,
    # even when they arrive out of order (follow-up 2).
    def __init__(self):
        self.data = {}

    def set(self, key, value, ts):
        if key not in self.data:
            self.data[key] = []
        items = self.data[key]
        position = 0
        while position < len(items) and items[position] <= (ts, value):
            position += 1                           # insert so the list stays sorted
        items.insert(position, (ts, value))

    def get_closest(self, key, ts):
        items = self.data.get(key)
```

```
        if not items:
            return ""
        i = first_not_before(items, ts)            # the first time at or after ts; the one before it is i -
1
        options = []
        if i > 0:
            options.append(items[i - 1])
        if i < len(items):
            options.append(items[i])
        best = options[0]
        for item in options:
            gap = item[0] - ts
            if gap < 0:
                gap = -gap
            best_gap = best[0] - ts
            if best_gap < 0:
                best_gap = -best_gap
            if gap < best_gap or (gap == best_gap and item[0] < best[0]):
                best = item
        return best[1]

    def get_at_or_before(self, key, ts):
        # LeetCode 981: the latest value stored at a time not after ts
        items = self.data.get(key, [])
        i = first_after(items, ts) - 1
        if i >= 0:
            return items[i][1]
        return ""


# ---- main question: tests ----
POINTS = [[1, 1], [1, 1], [2, 0], [0, 2], [2, 0], [3, 3]]


def test_main_question():
    assert top_points(POINTS, 2) == [[1, 1], [2, 0]]      # (1,1) and (2,0) both appear twice; (1,1) is close
r
    assert top_points(POINTS, 4) == [[1, 1], [2, 0], [0, 2], [3, 3]]
    assert top_points([[5, 5]], 1) == [[5, 5]]
    store = TimeStore()
```

```
    for ts, value in ((2, "x"), (10, "y"), (15, "z")):
        store.set("a", value, ts)
    assert store.get_closest("a", 11) == "y"
    assert store.get_closest("a", 14) == "z"
    assert store.get_closest("a", 12) == "y"              # a tie between 10 and 15 from 12.5 ... the earlier
 time wins
    assert store.get_closest("b", 1) == ""
    assert store.get_closest("a", 100) == "z"
```

```
    assert store.get_closest("a", -5) == "x"


# ====================================================
# FOLLOW-UP 1: Q1 with points arriving over time
# ====================================================
# ---- follow-up 1: function ----
# Answer: keep the counts as points arrive, so nothing is recounted;
# compute the top K on demand with a heap: O(d log k) per query (or keep a
# sorted structure updated on every add if queries are far more common).
class TopPointsStream:
    def __init__(self):
        self.counts = {}

    def add(self, x, y):
        self.counts[(x, y)] = self.counts.get((x, y), 0) + 1

    def top_k(self, k):
        best = heapq.nsmallest(k, ranked_points(self.counts))
        result = []
        for minus_count, distance, x, y in best:
            result.append([x, y])
        return result


# ---- follow-up 1: tests ----
def test_follow_up_1():
    live = TopPointsStream()
    for x, y in POINTS:
        live.add(x, y)
    assert live.top_k(2) == [[1, 1], [2, 0]]
    live.add(3, 3)
    live.add(3, 3)
    live.add(3, 3)
    assert live.top_k(1) == [[3, 3]]                      # (3, 3) is now the most frequent point


# ====================================================
# FOLLOW-UP 2: Q2 with timestamps arriving out of order
# ====================================================
# Answer: TimeStore.set inserts each pair at its sorted place, so late
# arrivals are fine (O(n) to insert into a list, O(log n) with a balanced tree).
# ---- follow-up 2: tests ----
def test_follow_up_2():
    store = TimeStore()
```

```
    for ts, value in ((2, "x"), (10, "y"), (15, "z")):
        store.set("a", value, ts)
    store.set("a", "late", 5)
    assert store.get_closest("a", 6) == "late"


# ====================================================
# FOLLOW-UP 3: Q2 returns the closest value at or before the timestamp (LeetCode 981)
# ====================================================
# Answer: get_at_or_before binary-searches the last stored time that is not
# after the asked time.
```

```python
# ---- follow-up 3: tests ----
def test_follow_up_3():
    store = TimeStore()
    for ts, value in ((2, "x"), (10, "y"), (15, "z")):
        store.set("a", value, ts)
    assert store.get_at_or_before("a", 14) == "y"
    assert store.get_at_or_before("a", 1) == ""
    assert store.get_at_or_before("a", 15) == "z"


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

184 Solve chat, grid paths, and car rentals

MEDIUM

Template fit: the top-K part is the template as written; the other parts are a rolling-column path count and a greedy car assignment

QUESTION

You are given three independent programming tasks. For each, describe and implement an efficient algorithm, and be prepared to analyze its time and space complexity. ---

Q1.

Find top-K most active chat users You are given a log of chat messages from a messaging application.
Input:

- An array **`messages`** of length **`n`**, where each element **`messages[i]`** is the **`user_id`** (string or integer) of the sender of the i-th message.
- An integer **`k`** (1 <= k <= number of distinct users).

Output:

- Return a list of **`k`** user IDs corresponding to the users who have sent the most messages.

Details & requirements:

- The "activity" of a user is defined as the number of messages they have sent (i.e., their frequency in **`messages`**).
- If there are fewer than **`k`** distinct users, return all distinct users.
- If multiple users have the same frequency, you may break ties arbitrarily, or (for determinism) sort tied users by **`user_id`** in ascending order.
- Aim for an algorithm that is more efficient than sorting all users by count when **`k`** is much smaller than the number of distinct users. Design and implement a function, for example:
- **`List<UserId> topKActiveUsers(List<UserId> messages, int k)`** Also explain:
- At least two different approaches (e.g., based on sorting, heaps, or selection algorithms).
- The time and space complexity of each approach. ---

Q2.

Count paths in a 2D grid with constrained moves You have a 2D grid with **`m`** rows and **`n`** columns. Index rows from top to bottom as **`0..m-1`** and columns from left to right as **`0..n-1`**.

QUESTION

- The **start** cell is the bottom-left corner: **`(m-1, 0)`**.
- The **end** cell is the bottom-right corner: **`(m-1, n-1)`**. From any cell **`(r, c)`**, you may move **only** to the next column to the right, in one of up to three ways (if within bounds):
- Right-up: **`(r-1, c+1)`**
- Right: **`(r, c+1)`**
- Right-down: **`(r+1, c+1)`** You cannot move left or stay in the same column. All cells are passable (no obstacles).

Task:

- Given integers **`m`** and **`n`**, compute the total number of distinct paths from the start cell **`(m-1, 0)`** to the end cell **`(m-1, n-1)`** using only the allowed moves.

Requirements:

- Implement a function such as **`long countPaths(int m, int n)`**.
- The result may be large; assume it fits in a 64-bit integer, or, if you prefer, you may return the count modulo a large prime (e.g., 1,000,000,007) — specify which convention you are using.
- First design a straightforward dynamic programming solution using O(m·n) time and O(m·n) space.
- Then describe how to optimize the space usage (e.g., to O(m)) while keeping the same time complexity, and explain why the optimization is correct. ---

Q3.

Minimum cars needed for rentals and assignment You work at a car rental company. You are given a list of rental bookings, each with a pickup and return time.

Input:

- An array of **`R`** rental records. Each record has:
- **`rental_id`**: a unique identifier (e.g., integer or string).
- **`pickup_time`**: an integer start time.
- **`return_time`**: an integer end time, with **`pickup_time < return_time`**. Assume times are on a single timeline (e.g., minutes from some epoch). A car can be reassigned immediately after it is returned, i.e., if one rental ends at time **`t`** and another starts at time **`t`**, the same car can serve both.

Tasks: 1.

Capacity: Compute the **minimum number of cars** required so that all rentals can be fulfilled without time conflicts on any single car. 2.

Assignment: Produce one valid assignment of rentals to cars that uses exactly that minimum number of cars.

Output:

- An integer **`C`** = minimal number of cars needed.
- A mapping from each **`rental_id`** to a **`car_id`** in the range **`1..C`**, such that no two rentals assigned to the same **`car_id`** overlap in time.

Requirements & clarifications:

- Overlap definition: two rentals with intervals **`[s1, e1)`** and **`[s2, e2)`** overlap if their time intervals intersect with non-empty duration. Using half-open intervals, if **`e1 == s2`** they do **not** over

EXAMPLES

Example 1	
Input	**`['ann', 'bob', 'ann', 'cy', 'bob', 'ann'], 2`**
Output	**`['ann', 'bob']`**
Why	**`Task 1: ann sent 3 messages, bob 2, cy 1; the top 2 are ann and bob.`**

QUESTION

Example 2	
Input	3, 3
Output	2
Why	Task 2: paths over 3 columns of a 3-row grid that start and end in the bottom row, moving up-right, right or down-right.

Example 3	
Input	[(1, 0, 5), (2, 1, 4), (3, 5, 8)]
Output	(2, {1: 0, 2: 1, 3: 1})
Why	Task 3: rentals are (id, pickup, return). Rental 3 starts at 5, after rental 2 returned its car at 4, so it reuses that car: 2 cars in total.

FULL CODE | main question first, then every follow-up, tests at the end

```
# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ========================================================

import heapq


# ========================================================
# MAIN QUESTION
# ========================================================
# ---- main question: helper functions ----
def count_items(items):
    # how many times each item appears: {item: count}
    counts = {}
    for item in items:
        counts[item] = counts.get(item, 0) + 1
    return counts


def rank_of(pair):
    # pair = (user, count): more messages first, ties by the smaller user id
    return (-pair[1], pair[0])


# ---- main question: main functions ----
# Q1: the k most active users in a chat log.
# Idea: count the messages of every user in a dictionary, then keep a heap
# of only the best k (heapq.nsmallest with a ranking key). That beats
# sorting all d users when k is small: O(n + d log k). Ties are broken by
# the smaller user id so the answer is always the same.
def top_k_users(messages, k):
    counts = count_items(messages)
    best = heapq.nsmallest(k, counts.items(), key=rank_of)
    users = []
    for user, count in best:
        users.append(user)
    return users


# the second approach for Q1: sort every user, O(d log d)
def top_k_users_sorted(messages, k):
    counts = count_items(messages)
```

```
    pairs = []
    for user in counts:
        pairs.append((-counts[user], user))
    pairs.sort()
    users = []
    for minus_count, user in pairs[:k]:
        users.append(user)
    return users


# Q2: count the paths through a grid with m rows and n columns. Start
# at the bottom-left cell (m - 1, 0), end at the bottom-right cell
# (m - 1, n - 1); each move goes right, right-and-up or right-and-down.
# Cells in `blocked` cannot be used (follow-up 4); mod keeps the numbers small.
# Idea: ways[r] = the number of paths in row r of the current column. One
# column is enough: O(m) memory.
def count_paths(m, n, blocked=(), mod=None):
    wall = set(blocked)
    ways = [0] * m
    ways[m - 1] = 1
    for c in range(1, n):
        following = [0] * m
        for r in range(m):
            if (r, c) in wall:
                continue                        # a blocked cell contributes no paths
            total = ways[r]
            if r > 0:
                total += ways[r - 1]
            if r + 1 < m:
                total += ways[r + 1]
            if mod:
                total = total % mod
            following[r] = total
        ways = following
    return ways[m - 1]


# Q3: rentals are (pickup, return) times. How many cars are needed?
# Idea: sort the rentals by pickup time and keep a min-heap of the return
# times of the cars that are out. If the earliest car is back by the
# pickup, reuse it; otherwise a new car is needed.
def min_cars(rentals):
    out = []
    for pickup, back in sorted(rentals):
        if out and out[0] <= pickup:
            heapq.heapreplace(out, back)        # a car came back in time: reuse it
        else:
            heapq.heappush(out, back)
    return len(out)


# ---- main question: tests ----
def test_main_question():
    log = ["a", "b", "a", "c", "a", "b"]
    assert top_k_users(log, 2) == ["a", "b"]
    assert top_k_users_sorted(log, 5) == ["a", "b", "c"]
    assert count_paths(3, 3) == 2                       # a 3 x 3 grid has 2 paths
    assert count_paths(1, 5) == 1
    assert count_paths(3, 1) == 1
    assert count_paths(4, 5) == 9
    assert min_cars([(1, 4), (2, 5), (4, 6), (5, 8)]) == 2
    assert min_cars([]) == 0


# ====================================================
# FOLLOW-UP 1: Q1: what if the log is a stream and memory is limited?
```

```
# ============================================================
# Answer: keep an approximate summary with a fixed number of counters
# (Misra-Gries or Space-Saving: see the "Top-K Frequent in Large Logs"
# question) and confirm the real counts of the survivors in a second pass.


# ============================================================
# FOLLOW-UP 2: Q2: what changes if there are obstacles or the path may also go straight up and down?
# ============================================================


# ============================================================
# FOLLOW-UP 4: Q2 with obstacles
# ============================================================
# Answer: count_paths already takes `blocked` cells (they get 0 paths). If
# a path may also go straight up or down inside a column, the cells of one
# column depend on each other, so count with one more dimension: how many
# vertical moves are still allowed in this column.
# ---- follow-up 2: tests ----
def test_follow_up_2():
    assert count_paths(3, 3, blocked=[(2, 1)]) == 1       # an obstacle in the middle of the bottom row
    modulus = 10 ** 9 + 7
    assert count_paths(5, 30, mod=modulus) < modulus
```

```
# ============================================================
# FOLLOW-UP 3: Q3: how do you answer "is one more rental possible" online?
# ============================================================
# ---- follow-up 3: function ----
# Answer: keep the heap of return times between calls. A new rental fits
# without a new car when some car has returned by its pickup time: take out
# every return time that is not after the pickup, then see if there is room.
class RentalDesk:
    def __init__(self, fleet):
        self.fleet = fleet                      # how many cars exist
        self.out = []                           # return times of the cars that are out

    def rent(self, pickup, back):
        while self.out and self.out[0] <= pickup:
            heapq.heappop(self.out)             # that car is back
        if len(self.out) >= self.fleet:
            return False
        heapq.heappush(self.out, back)
        return True


# ---- follow-up 3: tests ----
def test_follow_up_3():
    desk = RentalDesk(1)
    assert desk.rent(1, 4) and not desk.rent(2, 3)
    assert desk.rent(4, 6)


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_2()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

185 Top-K Chatters by Word Count

MEDIUM

Template fit: count len(body.split()) per speaker instead of lines; ties by name

QUESTION

You are given a chat transcript as a list of message lines. Each line has the form "Name: message text" , where everything before the first ": " is the author's name and everything after it is the message body. Count the number of words each person typed across the entire transcript (a word is a maximal run of non- whitespace characters in the body). A person may appear on many lines; sum their words. Return the k people who typed the most words, as a list of (name, word_count) tuples sorted by word_count descending. Break ties by name in ascending (lexicographic) order.

Function signature

def top_k_chatters(lines: list[str], k: int) -> list[tuple[str, int]]:

    ...

Follow-up (as reported): The interviewer's real ask was to implement the parsing inside a class alongside stubbed API methods. Wrap this in a ChatAnalyzer class that ingests lines incrementally via add_line(line) and answers top_k(k) on demand.

EXAMPLES

Example 1	
Input	`lines = ["Alice: hello there friend", "Bob: hi", "Alice: how are you"], k = 1`
Output	`[("Alice", 6)]`
Why	`Alice typed 3 + 3 = 6 words, Bob typed 1. Top 1 is Alice.`

Example 2	
Input	`lines = ["Ann: a b", "Bob: c d", "Cara: e"], k = 2`
Output	`[("Ann", 2), ("Bob", 2)]`
Why	`Ann and Bob both have 2 words; tie broken alphabetically, so Ann precedes Bob. Cara (1 word) is excluded.`

Example 3	
Input	`lines = [], k = 3`
Output	`[]`

CONSTRAINTS

- 0 <= len(lines) <= 10^5
- Every line contains exactly one ": " separating name from body; the name is non-empty and contains no ": " .
- The body may be empty (0 words).
- 1 <= k ; if k exceeds the number of distinct people, return all of them.

FULL CODE | main question first, then every follow-up, tests at the end

```
# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ========================================================

import heapq
import random


# ========================================================
# MAIN QUESTION
# ========================================================
```

```python
# ---- main question: helper functions ----
def rank_of(pair):
    # pair = (name, word count): more words first, ties by name
    return (-pair[1], pair[0])


def words_in(line):
    # "Alice: hello there friend" -> ("Alice", 3)
    name, body = line.split(": ", 1)
    return name, len(body.split())


# ---- main question: main function ----
# Idea: add up the words written by every author in a dictionary, then
# pick the k best with a heap ranked by (-words, name).
def top_k_chatters(lines, k):
    counts = {}
    for line in lines:
        name, words = words_in(line)
        counts[name] = counts.get(name, 0) + words
    return heapq.nsmallest(k, counts.items(), key=rank_of)


# ---- main question: tests ----
def test_main_question():
    lines = ["Alice: hello there friend", "Bob: hi", "Alice: how are you"]
    assert top_k_chatters(lines, 1) == [("Alice", 6)]
    assert top_k_chatters(["Ann: a b", "Bob: c d", "Cara: e"], 2) == [("Ann", 2), ("Bob", 2)]
    assert top_k_chatters([], 3) == []
```

```python
# ========================================================
# FOLLOW-UP 1: Wrap it in a ChatAnalyzer class that takes lines one at a time and answers top_k on demand
# ========================================================
# ---- follow-up 1: function ----
# Answer: keep the counts as the state; add_line parses the line and
# updates them in O(words); top_k runs the selection. If top_k is called
# far more often than add_line, keep a sorted structure (or a heap with
# lazy updates) instead of recomputing.
class ChatAnalyzer:
    def __init__(self):
        self.counts = {}

    def add_line(self, line):
        name, words = words_in(line)
        self.counts[name] = self.counts.get(name, 0) + words

    def top_k(self, k):
        return heapq.nsmallest(k, self.counts.items(), key=rank_of)


# ---- follow-up 1: tests ----
def test_follow_up_1():
    random.seed(18)
    names = ["Zed", "amy", "Bo", "Cy"]
    for round_number in range(300):
        lines = []
        for i in range(random.randint(0, 8)):
            body = []
            for j in range(random.randint(1, 4)):
                body.append("w" * random.randint(1, 3))
            lines.append(random.choice(names) + ": " + " ".join(body))
        k = random.randint(1, 5)
        chat = ChatAnalyzer()
        for line in lines:
            chat.add_line(line)
```

```
        assert chat.top_k(k) == top_k_chatters(lines, k)


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")
```

```
if __name__ == "__main__":
    run_all_tests()
```

186 Find top/bottom-k words in list or stream

MEDIUM

Template fit: the same Counter; run nsmallest once with -count and once with +count for the bottom-k

QUESTION

You are given words (strings) either as a finite list or as an unbounded stream. 1)
List version: Given an array **`words`** and an integer **k**, return: - the **k** most frequent distinct words, and - the **k** least frequent distinct words. Specify how you break ties (e.g., lexicographically, or any order). 2)
Stream version (online): Design a data structure that supports:

- **`ingest(word)`**: process the next word in the stream - **`topK(k)`**: return the **k** most frequent distinct words seen so far - **`bottomK(k)`**: return the **k** least frequent distinct words seen so far Discuss time/space complexity and how you would implement these operations efficiently. 3)

Variant (discuss/design): Extend the stream design to support returning the **first k** and/or **last k** words that are currently **non-repeating** (i.e., words with frequency exactly 1) according to stream arrival order.

Part 1: Top and Bottom K Frequent Words in a ListGiven an array of strings words and a non-negative integer k, return two lists: the k most frequent distinct words and the k least frequent distinct words. Use deterministic tie-breaking: if two words have the same frequency, the lexicographically smaller word comes first. If k is larger than the number of distinct words, return all distinct words.
Constraints
0 <= len(words) <= 2 * 10^50 <= k <= 2 * 10^5Each word is a non-empty stringComparison is case-sensitive unless otherwise stated

EXAMPLES

```
Input: (['i', 'love', 'leetcode', 'i', 'love', 'coding'], 2)
Expected Output: [['i', 'love'], ['coding', 'leetcode']]
Explanation: `i` and `love` both appear twice, so lexicographical order gives `['i', 'love']`. `coding`
    and `leetcode` both appear once, so lexicographical order gives `['coding', 'leetcode']`.
Input: (['apple', 'banana', 'apple', 'cherry', 'banana', 'date'], 3)
Expected Output: [['apple', 'banana', 'cherry'], ['cherry', 'date', 'apple']]
```

Explanation: Frequencies are: apple=2, banana=2, cherry=1, date=1. Top 3 are ordered by frequency descending, then lexicographically: **`apple`**, **`banana`**, **`cherry`**. Bottom 3 are ordered by frequency ascending, then lexicographically: **`cherry`**, **`date`**, **`apple`**.

QUESTION

Hints

Count how many times each distinct word appears before deciding the answer.You can sort the distinct words twice: once by (-frequency, word) and once by (frequency, word).

Part 2: Online Top/Bottom K Words from a StreamYou are given a sequence of stream operations. Each operation is a 2-element list of strings: ['ingest', word], ['topK', str(k)], or ['bottomK', str(k)]. Process the operations in order. For each query, return the k most frequent or k least frequent distinct words seen so far. Ties are broken lexicographically ascending. If k is larger than the number of distinct words seen so far, return all distinct words.

Constraints

1 <= len(operations) <= 2 * 10^5Each operation is one of ['ingest', word], ['topK', str(k)], or ['bottomK', str(k)]0 <= k <= 2 * 10^5Queries may appear before any word is ingested

Examples

Input: [['ingest', 'apple'], ['ingest', 'banana'], ['ingest', 'apple'], ['topK', '2'], ['bottomK', '2']]

Expected Output: [['apple', 'banana'], ['banana', 'apple']]

Explanation: After the ingests, counts are apple=2 and banana=1. topK(2) returns the most frequent first, while bottomK(2) returns the least frequent first.

Input: [['topK', '3'], ['ingest', 'dog'], ['ingest', 'cat'], ['ingest', 'cat'], ['topK', '2'], ['bottomK', '2'], ['topK', '5']]

Expected Output: [[], ['cat', 'dog'], ['dog', 'cat'], ['cat', 'dog']]

Explanation: The first query happens before any words are seen, so it returns []. Later, cat has frequency 2 and dog has frequency 1.

Hints

Keep an exact frequency map for all words seen so far.A max-heap and a min-heap with lazy deletion let you support online updates without fully resorting all words

FULL CODE | main question first, then every follow-up, tests at the end

```
# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ========================================================

import heapq


# ========================================================
# MAIN QUESTION
# ========================================================
# ---- main question: helper functions ----
def count_words(words):
    counts = {}
    for word in words:
        counts[word] = counts.get(word, 0) + 1
    return counts


def names_of(pairs):
    names = []
    for first, second in pairs:
        names.append(second)
    return names


# ---- main question: main function ----
# Idea: one pass counts the words. A heap of size k then picks the k most
# frequent (rows (-count, word): the smallest rows win) and the k least
# frequent (rows (count, word)), each in O(d log k) without sorting
# everything. Ties are broken alphabetically.
```

```
def top_bottom(words, k):
    counts = count_words(words)
    top_rows = []
    bottom_rows = []
    for word in counts:
        top_rows.append((-counts[word], word))
        bottom_rows.append((counts[word], word))
    top = names_of(heapq.nsmallest(k, top_rows))
    bottom = names_of(heapq.nsmallest(k, bottom_rows))
    return top, bottom
```

```
# ---- main question: tests ----
def test_main_question():
    words = "a b a c b a d d".split()
    top, bottom = top_bottom(words, 2)
    assert top == ["a", "b"] and bottom == ["c", "b"]


# ========================================================
# FOLLOW-UP 1: The stream is too big to count every distinct word
# ========================================================
# ---- follow-up 1: function ----
# Answer: the Space-Saving (or Misra-Gries) summary with m counters. A word
# that appears more than n / m times is guaranteed to be tracked, and its
# stored count is an upper bound. When a new word arrives and there is no
# room, it replaces the word with the smallest counter and inherits that
# count + 1.
class SpaceSaving:
    def __init__(self, m):
        self.m = m
        self.counts = {}

    def add(self, word):
        if word in self.counts:
            self.counts[word] += 1
        elif len(self.counts) < self.m:
            self.counts[word] = 1
        else:
            smallest = None
            for tracked in self.counts:
                if smallest is None or self.counts[tracked] < self.counts[smallest]:
                    smallest = tracked
            self.counts[word] = self.counts.pop(smallest) + 1

    def top_k(self, k):
        rows = []
        for word in self.counts:
            rows.append((-self.counts[word], word))
        rows.sort()
        return names_of(rows[:k])


# ---- follow-up 1: tests ----
def test_follow_up_1():
    summary = SpaceSaving(2)
    for word in ["x", "x", "x", "y", "z", "x", "x"]:
```

```
        summary.add(word)
    assert summary.top_k(1) == ["x"]


# ========================================================
# FOLLOW-UP 2: topK must be O(k), not O(d log k), because it is called all the time
# ========================================================
# ---- follow-up 2: function ----
```

```
# Answer: keep the words in BUCKETS by count (like the LFU cache): the most
# frequent words are in the highest bucket. Adding a word moves it up one
# bucket in O(1); top_k only reads the highest buckets.
class FreqBuckets:
    def __init__(self):
        self.count = {}                         # word -> its count
        self.bucket = {}                        # count -> the words with exactly that count
        self.high = 0                           # the highest count so far

    def add(self, word):
        c = self.count.get(word, 0)
        if c > 0:
            del self.bucket[c][word]            # leave the old bucket
            if not self.bucket[c]:
                del self.bucket[c]
        self.count[word] = c + 1
        if c + 1 not in self.bucket:
            self.bucket[c + 1] = {}
        self.bucket[c + 1][word] = True
        if c + 1 > self.high:
            self.high = c + 1

    def top_k(self, k):
        result = []
        level = self.high
        while level > 0 and len(result) < k:
            if level in self.bucket:
                result += sorted(self.bucket[level])   # inside one bucket: alphabetical
            level -= 1
        return result[:k]

    def bottom_k(self, k):
        result = []
        for level in sorted(self.bucket):
            result += sorted(self.bucket[level])
            if len(result) >= k:
                break
        return result[:k]


# ---- follow-up 2: tests ----
def test_follow_up_2():
    buckets = FreqBuckets()
    for word in "a b a c b a d d".split():
        buckets.add(word)
    assert buckets.top_k(2) == ["a", "b"]
    assert buckets.bottom_k(2) == ["c", "b"]
    assert buckets.top_k(10) == ["a", "b", "d", "c"]


# ======================================================
# FOLLOW-UP 3: Case and punctuation
# ======================================================
# ---- follow-up 3: function ----
# Answer: clean the text before counting (lowercase, remove punctuation)
# and say that you did; the counting and ranking code does not change.
def normalise(text):
    cleaned = ""
    for ch in text.lower():
        if ch.isalnum():
            cleaned += ch
    return cleaned


# ---- follow-up 3: tests ----
```

```
def test_follow_up_3():
    assert normalise("Hello, World!") == "helloworld"


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

Digits and characters: peel one at a time and carry

187 Caesar Cipher: Recover Key, Encrypt, and Decrypt — EASY

Template fit: per letter: (ord(ch) - 97 + k) % 26 + 97; the key comes from one letter pair; decrypt uses -k

QUESTION

A message was scrambled with a Caesar cipher: every lowercase letter was shifted forward by the same secret amount k (0 <= k < 26) with wraparound (z +1 -> a); all non-lowercase characters were left unchanged. You are given one example pair — a known plaintext and its corresponding ciphertext — from which you must recover the secret shift, and then implement the cipher in both directions. Specifically implement: find_shift(plaintext, ciphertext) -> int : return the shift k (0..25) that maps plaintext to ciphertext . The two strings have equal length and contain at least one lowercase letter that pins down k . encrypt(text, k) -> str : shift each lowercase letter forward by k with wraparound; leave other characters unchanged. decrypt(text, k) -> str : the inverse of encrypt — shift each lowercase letter backward by k with wraparound.

Function signature

```
def find_shift(plaintext: str, ciphertext: str) -> int:
    ...
def encrypt(text: str, k: int) -> str:
    ...
def decrypt(text: str, k: int) -> str:
    ...
```

Follow-up (as reported): Given only a ciphertext (no key and no plaintext), how would you break the cipher? Discuss trying all 26 shifts (brute force) or English letter-frequency analysis.

EXAMPLES

Example 1	
Input	`find_shift("abc", "def")`
Output	`3`
Why	`'a'->'d', 'b'->'e', 'c'->'f' is a uniform forward shift of 3.`

Example 2	
Input	`encrypt("hello world", 3)`
Output	`"khoor zruog"`
Why	`Each letter moves forward 3 ('h'->'k', 'o'->'r', 'w'->'z'); the space is unchanged.`

Example 3	
Input	`decrypt("khoor zruog", 3)`
Output	`"hello world"`
Why	`decrypt inverts encrypt, so the original message is recovered.`

Example 4	
Input	`encrypt("xyz", 3)`
Output	`"abc"`
Why	`Wraparound: 'x'->'a', 'y'->'b', 'z'->'c'.`

CONSTRAINTS

- 1 <= len(text) <= 10^5 . text may contain lowercase letters, spaces, digits, and punctuation; only lowercase letters a – z are shifted.

QUESTION

- For find_shift , len(plaintext) == len(ciphertext) and they align character by character.

FULL CODE | main question first, then every follow-up, tests at the end

```
# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ========================================================

# ========================================================
# MAIN QUESTION
# ========================================================
# ---- main question: main functions ----
# A Caesar cipher moves every lowercase letter k places along the
# alphabet (wrapping z to a). Other characters stay as they are.

# Idea: one letter pair is enough to recover the key: the key is how far
# the ciphertext letter is from the plaintext letter, modulo 26.
def find_shift(plaintext, ciphertext):
    for i in range(len(plaintext)):
        if "a" <= plaintext[i] <= "z":
            return (ord(ciphertext[i]) - ord(plaintext[i])) % 26
    return 0


# The same helper shifts forward (encrypt) or backward (decrypt, shift by -k).
def shift_text(text, k):
    result = ""
    for ch in text:
        if "a" <= ch <= "z":
            position = ord(ch) - ord("a")                    # 0 for "a" ... 25 for "z"
            ch = chr((position + k) % 26 + ord("a"))
        result += ch
    return result


def encrypt(text, k):
    return shift_text(text, k)


def decrypt(text, k):
    return shift_text(text, -k)


# ---- main question: tests ----
def test_main_question():
    assert find_shift("abc", "def") == 3
```

```
    assert encrypt("hello world", 3) == "khoor zruog"
    assert decrypt("khoor zruog", 3) == "hello world"
    assert encrypt("xyz", 3) == "abc"
    assert encrypt("a1 b!", 2) == "c1 d!"


# ========================================================
# FOLLOW-UP 1: You only have a ciphertext. How do you break it?
# ========================================================
# ---- follow-up 1: function ----
# Answer: try all 26 keys and score each decrypted text by how English it
# looks: give every common letter (e, t, a, o, i, n, s, h, r, d, l, u) a
# weight (e the biggest) and add up the weights of the letters. The key
# with the best score wins. Longer texts make the letter-frequency signal
# reliable.
```

```
def break_caesar(ciphertext):
    common = "etaoinshrdlu"
    weight = {}
    for i in range(len(common)):
        weight[common[i]] = 12 - i
    best_key = 0
    best_score = -1
    for k in range(26):
        text = decrypt(ciphertext, k)
        score = 0
        for ch in text:
            if ch in weight:
                score += weight[ch]
        if score > best_score:
            best_score = score
            best_key = k
    return best_key


# ---- follow-up 1: tests ----
def test_follow_up_1():
    sample = "the quick brown fox jumps over the lazy dog "
    sample += "while the sun sets over the quiet hills "
    sample += "and the rivers run to the sea"
    for k in range(26):
        assert decrypt(encrypt(sample, k), k) == sample
        assert break_caesar(encrypt(sample, k)) == k


# ============================================================
```

```
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

188 Add two big integers from digit lists

MEDIUM

Template fit: the template as written

QUESTION

You are given two non-empty singly linked lists (or arrays) representing two non-negative integers. The digits are stored in **reverse order**, and each node contains a single digit from 0 to 9. Your task is to add the two numbers and return the sum as a new linked list, also in reverse order. You may assume:

- Each list has at least one node.
- The integers do not contain leading zeros, except the number 0 itself.

Example

Input:

- List 1: 2 -> 4 -> 3 (represents the integer 342)
- List 2: 5 -> 6 -> 4 (represents the integer 465)

Output:

- 7 -> 0 -> 8 (represents the integer 807, since 342 + 465 = 807)

QUESTION

Another example

Input:

- List 1: 9 -> 9 -> 9 (represents 999)
- List 2: 1 (represents 1)

Output:

- 0 -> 0 -> 0 -> 1 (represents 1000)

Constraints

- Let n be the length of the first list and m be the length of the second list.
- 1 <= n, m <= 10^5.
- Each node value is an integer in the range [0, 9].

Task: Describe an algorithm to perform this addition and construct the resulting linked list in O(n + m) time and O(1) additional space (excluding the space for the output list).

You are given two non-empty arrays of digits representing two non-negative integers. The digits are stored in **reverse order** (least significant digit first), and each element is a single digit from 0 to 9. Add the two numbers and return the sum as a new array of digits, also in reverse order.

Assumptions

- Each array has at least one element.
- The integers do not contain leading zeros, except the number 0 itself.

Example 1

```
Input: l1 = [2, 4, 3], l2 = [5, 6, 4] (342 + 465)
Output: [7, 0, 8] (807)
```

Example 2

```
Input: l1 = [9, 9, 9], l2 = [1] (999 + 1)
Output: [0, 0, 0, 1] (1000)
```

Implement **`add_two_numbers(l1, l2)`** (camelCase **`addTwoNumbers`** in Java/C++/JS) returning the digit array of the sum in reverse order. Aim for O(n + m) time and O(1) extra space beyond the output.

Constraints

1 <= n, m <= 10^5 where n = len(l1), m = len(l2)Each element is an integer in the range [0, 9]Digits are stored in reverse order (least significant digit first)No leading zeros except for the number 0 itself

EXAMPLES

```
Input: ([2, 4, 3], [5, 6, 4])
Expected Output: [7, 0, 8]
Explanation: 342 + 465 = 807, stored reversed as [7, 0, 8].
Input: ([9, 9, 9], [1])
Expected Output: [0, 0, 0, 1]
```

Explanation: 999 + 1 = 1000; carry propagates through all three digits and creates a new leading digit.

Hints

Walk both arrays from index 0 (the least significant digit) toward the end, just like grade-school column addition.Maintain a single carry variable. At each position, sum the available digits plus the carry; append (sum % 10) and set carry = sum // 10.When one array runs out, keep going with just the carry and the remaining longer array. Don't forget a final carry can add one extra digit (e.g. 999 + 1 -> [0,0,0,1]).

FULL CODE | main question first, then every follow-up, tests at the end

```python
# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ========================================================

# ========================================================
# MAIN QUESTION
# ========================================================
# ---- main question: main function ----
# Idea: the numbers are lists of digits with the MOST significant digit
# first. Walk both lists from the right with indices i and j. At each
# step add a[i] (or 0 when a is used up), b[j] (or 0) and the carry,
# keep total % 10 as the digit, and carry total // 10. Continue while
# anything is left, including a final carry. Reverse at the end.
def add_digits(a, b):
    i = len(a) - 1
    j = len(b) - 1
    carry = 0
    digits = []
    while i >= 0 or j >= 0 or carry > 0:
        total = carry
        if i >= 0:
            total += a[i]
        if j >= 0:
            total += b[j]
        digits.append(total % 10)
        carry = total // 10
        i -= 1
        j -= 1
    if len(digits) == 0:
        return [0]
    digits.reverse()
    return digits


# ---- main question: tests ----
def test_main_question():
    assert add_digits([9, 9], [1]) == [1, 0, 0]
    assert add_digits([1, 2, 3], [4, 5]) == [1, 6, 8]
    assert add_digits([9, 9, 9], [1]) == [1, 0, 0, 0]
    assert add_digits([0], [0]) == [0]
```

```python
# ========================================================
# FOLLOW-UP 1: The digits are stored least significant first in a linked list
# ========================================================
# ---- follow-up 1: helper functions ----
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next


def make_list(digits):
    # [2, 4, 3] -> 2 -> 4 -> 3 (least significant digit first)
    dummy = ListNode()
    tail = dummy
    for d in digits:
        tail.next = ListNode(d)
        tail = tail.next
    return dummy.next


def to_digits(node):
    digits = []
```

```
    while node:
        digits.append(node.val)
        node = node.next
    return digits


# ---- follow-up 1: function ----
# Answer: the head is already the LOW end, so walk both lists together
# with the same carry loop and build the result list as you go. No
# reversal is needed (LeetCode 2).
def add_lists(a, b):
    dummy = ListNode()
    tail = dummy
    carry = 0
    while a or b or carry > 0:
        total = carry
        if a:
            total += a.val
            a = a.next
        if b:
            total += b.val
            b = b.next
        tail.next = ListNode(total % 10)
        tail = tail.next
        carry = total // 10
    return dummy.next


# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert to_digits(add_lists(make_list([2, 4, 3]), make_list([5, 6, 4]))) == [7, 0, 8]     # 342 + 465 = 80
7
    assert to_digits(add_lists(make_list([9, 9]), make_list([1]))) == [0, 0, 1]              # 99 + 1 = 100


# ========================================================
# FOLLOW-UP 2: Multiply instead of add
# ========================================================
# ---- follow-up 2: function ----
# Answer: schoolbook multiplication: the product of the digits a[i] and
# b[j] lands at position i + j + 1, and its carry spills to position
# i + j. O(m * n); Karatsuba or FFT for very long numbers.
def multiply_digits(a, b):
    result = [0] * (len(a) + len(b))
    for i in range(len(a) - 1, -1, -1):
        for j in range(len(b) - 1, -1, -1):
            total = a[i] * b[j] + result[i + j + 1]
            result[i + j + 1] = total % 10
            result[i + j] += total // 10
    while len(result) > 1 and result[0] == 0:
        result.pop(0)                                  # no leading zeros
    return result


# ---- follow-up 2: tests ----
def test_follow_up_2():
    assert multiply_digits([1, 2, 3], [4, 5]) == [5, 5, 3, 5]
    assert multiply_digits([0], [9, 9]) == [0]


# ========================================================
# FOLLOW-UP 3: Subtract, or numbers in another base
# ========================================================
# ---- follow-up 3: function ----
# Answer: subtraction BORROWS instead of carrying, and needs the larger
```

```
# number first (compare lengths, then digits). Another base replaces 10
# by the base in % and //.
def subtract_digits(a, b):
    # a - b where a >= b, most significant digit first
    i = len(a) - 1
    j = len(b) - 1
    borrow = 0
    digits = []
    while i >= 0:
        d = a[i] - borrow
        if j >= 0:
            d -= b[j]
        if d < 0:
            d += 10
            borrow = 1
        else:
            borrow = 0
        digits.append(d)
        i -= 1
        j -= 1
    while len(digits) > 1 and digits[-1] == 0:
        digits.pop()                                    # drop leading zeros
    digits.reverse()
    return digits


# ---- follow-up 3: tests ----
def test_follow_up_3():
    assert subtract_digits([1, 0, 0], [1]) == [9, 9]
    assert subtract_digits([5, 0], [5, 0]) == [0]


# ======================================================
# RUN EVERYTHING
# ======================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

189 Solve several coding interview problems — MEDIUM

Template fit: the big-number addition on strings, with int(ch) per digit; the other parts are list-meeting and overlap sweeps

QUESTION

Three small tasks. (1) add_strings(a, b): add two non-negative integers given as digit strings and return the sum as a string, without converting the whole numbers. (2) intersection(head_a, head_b): two singly linked lists may share a common tail; return the first shared node, or None. (3) overlap_depth(rects): rectangles (x1, y1, x2, y2) include their left and bottom edges but not their right and top edges; return the largest number of rectangles that cover a single point.

Function signature

def add_strings(a: str, b: str) -> str
def intersection(head_a, head_b)
def overlap_depth(rects: list) -> int

EXAMPLES

QUESTION

Example 1	
Input	'999', '1'
Output	'1000'
Why	A carry runs through every digit: 999 + 1 = 1000.

Example 2	
Input	'456', '77'
Output	'533'
Why	456 + 77 = 533.

Example 3	

Input	class N:
	def __init__(self, val, next=None):
	self.val, self.next = val, next
	shared = N(8, N(9))
	a = N(1, N(2, shared))
	b = N(5, shared)
	intersection(a, b).val

Output	intersection(a, b).val -> 8
Why	List a is 1, 2, 8, 9 and list b is 5, 8, 9: they join at the node holding 8.

Example 4	
Input	[(0, 0, 4, 4), (2, 2, 6, 6), (3, 3, 5, 5)]
Output	3
Why	The point (3, 3) lies inside all three rectangles.

Example 5	
Input	[(0, 0, 2, 2), (2, 0, 4, 2)]
Output	1
Why	Rectangles that only share an edge do not overlap.

FULL CODE | main question first, then every follow-up, tests at the end

```
# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next


# ---- main question: main functions ----
# PART 1: add two big numbers kept as digit strings.
# Idea: walk both strings from the right, add the two digits and the
```

```
# carry, keep total % 10 and carry total // 10; after the loop add a final
# carry digit if there is one; reverse the digits at the end.
def add_strings(a, b):
    i = len(a) - 1
    j = len(b) - 1
    carry = 0
    digits = []
    while i >= 0 or j >= 0 or carry > 0:
        total = carry
        if i >= 0:
            total += int(a[i])
        if j >= 0:
            total += int(b[j])
        digits.append(str(total % 10))
        carry = total // 10
        i -= 1
        j -= 1
    digits.reverse()
    return "".join(digits)


# PART 2: where do two linked lists meet?
# Idea: each pointer walks its own list and then continues on the OTHER
# list. Both walk the same total length (m + n), so they arrive at the
# meeting node at the same moment (or both reach None together).
```

```
def intersection(head_a, head_b):
    a = head_a
    b = head_b
    while a is not b:
        if a is None:
            a = head_b
        else:
            a = a.next
        if b is None:
            b = head_a
        else:
            b = b.next
    return a


# PART 3: the deepest overlap of rectangles. rects are (x1, y1, x2, y2),
# half-open: the largest number of rectangles that share one point.
# Idea: the best point can be a rectangle's lower-left corner, so test
# those corners and count the rectangles that contain each.
def overlap_depth(rects):
    best = 0
    for x, y, x2, y2 in rects:
        inside = 0
        for r in rects:
            if r[0] <= x < r[2] and r[1] <= y < r[3]:
                inside += 1
        if inside > best:
            best = inside
    return best


# ---- main question: tests ----
def test_main_question():
    assert add_strings("999", "1") == "1000"
    assert add_strings("0", "0") == "0"
    assert add_strings("123456789123456789", "987654321") == str(123456789123456789 + 987654321)
    shared = ListNode(8, ListNode(9))
    a = ListNode(1, ListNode(2, shared))
    b = ListNode(5, shared)
    assert intersection(a, b) is shared
```

```
    assert intersection(ListNode(1), ListNode(2)) is None           # no meeting: both end at None
    assert overlap_depth([(0, 0, 4, 4), (2, 2, 6, 6), (3, 3, 5, 5)]) == 3
    assert overlap_depth([]) == 0
```

```
# ============================================================
# FOLLOW-UP 1: Ancestors in a tree with parent pointers, not a linked list
# ============================================================
# Answer: the same idea. The "lowest common ancestor" of two nodes is
# where their paths up to the root meet: walk each pointer up to the root
# and then restart at the other node (like intersection above).


# ============================================================
# FOLLOW-UP 2: Add two big numbers stored as linked lists
# ============================================================


# ============================================================
# FOLLOW-UP 4: Add two big numbers stored as linked lists (least significant digit first)
# ============================================================
# ---- follow-up 2: function ----
# Answer: if the digits are stored least significant FIRST, walk both
# lists together with a carry and build the result as you go (LeetCode 2):
# O(max(m, n)). If the most significant digit comes first, reverse the
# lists first or use stacks.
def add_lists(a, b):
    dummy = ListNode()
    tail = dummy
    carry = 0
    while a or b or carry > 0:
        total = carry
        if a:
            total += a.val
            a = a.next
        if b:
            total += b.val
            b = b.next
        tail.next = ListNode(total % 10)
        tail = tail.next
        carry = total // 10
    return dummy.next


# ---- follow-up 2: tests ----
def to_list(node):
    values = []
    while node:
        values.append(node.val)
        node = node.next
```

```
    return values


def test_follow_up_2():
    x = ListNode(2, ListNode(4, ListNode(3)))
    y = ListNode(5, ListNode(6, ListNode(4)))
    assert to_list(add_lists(x, y)) == [7, 0, 8]                # 342 + 465 = 807 with digits reversed


# ============================================================
# FOLLOW-UP 3: Count overlapping rectangles instead of intervals
# ============================================================
# ---- follow-up 3: function ----
# Answer: counting overlapping PAIRS is O(n^2) by checking every pair. The
# deepest overlap in O(n log n) needs a sweep over x with a segment tree
```

```python
# over y (range add, range max).
def overlapping_pairs(rects):
    count = 0
    for i in range(len(rects)):
        for j in range(i + 1, len(rects)):
            a = rects[i]
            b = rects[j]
            if a[0] < b[2] and b[0] < a[2] and a[1] < b[3] and b[1] < a[3]:
                count += 1
    return count


# ---- follow-up 3: tests ----
def test_follow_up_3():
    assert overlapping_pairs([(0, 0, 4, 4), (2, 2, 6, 6), (3, 3, 5, 5)]) == 3
    assert overlapping_pairs([(0, 0, 1, 1), (2, 2, 3, 3)]) == 0


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_2()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

190 Check Whether Digits Are Monotone Increasing

MEDIUM

Template fit: peel digits from the right; each digit must not exceed the previous one you saw

QUESTION

Return true when the decimal digits of a non-negative integer never decrease from left to right (repeated digits are allowed). Zero and single-digit numbers qualify.

Function signature

def is_monotone_digits(n: int) -> bool

EXAMPLES

Example 1	
Input	`1234`
Output	`true`
Why	`1, 2, 3, 4 only goes up.`

Example 2	
Input	`1223`
Output	`true`
Why	`Equal neighbours are allowed.`

Example 3	
Input	`332`
Output	`false`
Why	`3, 3, 2 goes down at the end.`

QUESTION

Example 4	
Input	0
Output	true
Why	Zero qualifies.

FULL CODE | main question first, then every follow-up, tests at the end

```
# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ========================================================

# ========================================================
# MAIN QUESTION
# ========================================================
# ---- main question: helper functions ----
def is_monotone_string(n):
    # the same test on the text of the number (used to check the main function)
    text = str(n)
    for i in range(len(text) - 1):
        if text[i] > text[i + 1]:
            return False
    return True


# ---- main question: main function ----
# Idea: take the last digit with n % 10, then drop it with n // 10, and so
# on. Moving from right to left, each digit just read must not be bigger
# than the digit read before it (the one to its right). Zero and a
# single digit are monotone.
def is_monotone_digits(n):
    previous = 9                              # nothing is bigger than 9
    while n > 0:
        digit = n % 10
        if digit > previous:
            return False                      # a digit bigger than the one to its right
        previous = digit
        n //= 10
    return True


# ---- main question: tests ----
def test_main_question():
    assert is_monotone_digits(0) and is_monotone_digits(7)
    assert is_monotone_digits(1223)
    assert is_monotone_digits(999)
    assert not is_monotone_digits(332)
    assert not is_monotone_digits(10)
    for n in range(2000):
        assert is_monotone_digits(n) == is_monotone_string(n)
```

```
# ========================================================
# FOLLOW-UP 1: Strictly increasing digits
# ========================================================
# ---- follow-up 1: function ----
# Answer: use >= instead of > when comparing with the previous digit, and
# start `previous` at 10 so that a single digit passes.
def is_strictly_monotone(n):
```

```
    previous = 10
    while n > 0:
        digit = n % 10
        if digit >= previous:
            return False
        previous = digit
        n //= 10
    return True


# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert is_strictly_monotone(1234)
    assert not is_strictly_monotone(1224)
    assert is_strictly_monotone(5)


# =========================================================
# FOLLOW-UP 2: Negative numbers or numbers in another base
# =========================================================
# ---- follow-up 2: function ----
# Answer: decide what "monotone" means for the sign (usually apply it to
# the absolute value). For another base b, use b instead of 10 in % and //.
def is_monotone_base(n, base=10):
    previous = base - 1
    while n > 0:
        digit = n % base
        if digit > previous:
            return False
        previous = digit
        n //= base
    return True


# ---- follow-up 2: tests ----
def test_follow_up_2():
```

```
    assert is_monotone_base(3, 2)              # binary 11
    assert not is_monotone_base(2, 2)          # binary 10


# =========================================================
# FOLLOW-UP 3: Find the next monotone number after n
# =========================================================
# ---- follow-up 3: function ----
# Answer: take n + 1, find its first descent from the LEFT and copy that
# digit over the rest (332 becomes 333).
def next_monotone(n):
    digits = list(str(n + 1))
    for i in range(len(digits) - 1):
        if digits[i] > digits[i + 1]:
            for j in range(i + 1, len(digits)):
                digits[j] = digits[i]
            break
    return int("".join(digits))


# ---- follow-up 3: tests ----
def test_follow_up_3():
    assert next_monotone(332) == 333
    assert next_monotone(9) == 11
    for n in range(2000):
        following = n + 1
        while not is_monotone_string(following):
            following += 1
        assert next_monotone(n) == following
```

```python
# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

191 Check Digits and Combine Ranges

HARD

Template fit: palindrome number: reverse only the second half with n % 10 and n // 10 and compare with the first half

QUESTION

You are asked to solve two independent coding tasks.

Task 1:

Check Whether an Integer Reads the Same Backward Given an integer **`x`**, return **`true`** if its decimal representation reads the same from left to right and from right to left; otherwise return **`false`**. Requirements and edge cases:

- Negative integers should return **`false`** because of the leading minus sign.
- You may assume **`x`** fits in a signed 32-bit integer.
- Try to solve it without converting the integer to a string. Example:

```
Input: x = 121
Output: true
```

```
Input: x = -121
Output: false
```

```
Input: x = 10
Output: false
```

Task 2:

Combine Overlapping Ranges Given an array of intervals **`intervals`**, where each interval is represented as **`[start, end]`**, combine all overlapping intervals and return an array of non-overlapping intervals that covers exactly the same ranges. Requirements and edge cases:

- Intervals may be provided in any order.
- Two intervals overlap if they share at least one point. For example, **`[1, 4]`** and **`[4, 5]`** should be combined into **`[1, 5]`**.
- Return the intervals in increasing order by start value. Example:

```
Input: intervals = [[1,3],[2,6],[8,10],[15,18]]
Output: [[1,6],[8,10],[15,18]]
```

```
Input: intervals = [[1,4],[4,5]]
Output: [[1,5]]
```

QUESTION

Part 1: Check Whether an Integer Reads the Same BackwardGiven an integer x, return True if its decimal representation reads the same from left to right and from right to left. Otherwise, return False. Negative integers are not palindromes because of the leading minus sign. Try to solve the problem using arithmetic operations instead of converting the integer to a string.

Constraints-2^31 <= x <= 2^31 - 1Negative integers must return FalseAn arithmetic-based solution is preferred over string conversion

EXAMPLES

```
Input: 121
Expected Output: True
```

Explanation: 121 reads the same forward and backward.

```
Input: -121
Expected Output: False
```

Explanation: Negative numbers are not palindromes.

Hints

A negative number can never be a palindrome because the minus sign only appears on the left.Try reversing only the second half of the digits instead of the whole number.

Part 2: Combine Overlapping RangesGiven a list of intervals, where each interval is represented as [start, end], merge all overlapping intervals and return a new list of non-overlapping intervals that covers exactly the same ranges. Intervals may appear in any order. Two intervals overlap if they share at least one point, so [1, 4] and [4, 5] should be merged into [1, 5]. Return the result sorted by starting value.

Constraints

0 <= len(intervals) <= 10^4-10^9 <= start <= end <= 10^9Intervals may be unsorted

Examples

Input: [[1, 3], [2, 6], [8, 10], [15, 18]]

Expected Output: [[1, 6], [8, 10], [15, 18]]

Explanation: The first two intervals overlap and merge into [1, 6].

Input: [[1, 4], [4, 5]]

Expected Output: [[1, 5]]

Explanation: The intervals share the point 4, so they overlap.

Hints

Sort the intervals by their start value before trying to merge them.Keep track of the last interval in the output. If the next interval overlaps, extend the end; otherwise, start a new interval.

FULL CODE | main question first, then every follow-up, tests at the end

```
# ======================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ======================================================

# ======================================================
# MAIN QUESTION
# ======================================================
# ---- main question: main functions ----
# PART 1: is the number a palindrome, without turning it into text?
# Idea: negative numbers and positive numbers ending in 0 can never be
# palindromes. Build the REVERSED SECOND HALF of the number digit by digit
# and stop when it catches up with the first half. An odd number of digits
# has a middle digit, which is dropped from the reversed half.
# (base is the number base; follow-up 3 asks for another one.)
```

```python
def is_palindrome(x, base=10):
    if x < 0 or (x % base == 0 and x != 0):
        return False
    reversed_half = 0
    while x > reversed_half:
        reversed_half = reversed_half * base + x % base
        x //= base
    return x == reversed_half or x == reversed_half // base


# PART 2: combine overlapping ranges.
# Idea: sort by start so overlapping ranges are neighbours. A range that
# starts before (or exactly where) the last merged one ends is absorbed.
# With adjacent=True, ranges that merely touch ([1, 2] and [3, 4])
# are merged too (follow-up 1).
def merge_ranges(intervals, adjacent=False):
    gap = 0
    if adjacent:
        gap = 1
    merged = []
    for start, end in sorted(intervals):
        if merged and start <= merged[-1][1] + gap:
            if end > merged[-1][1]:
                merged[-1][1] = end
        else:
            merged.append([start, end])
    return merged
```

```python
# ---- main question: tests ----
def test_main_question():
    assert is_palindrome(121) and is_palindrome(0)
    assert not is_palindrome(-121)
    assert not is_palindrome(10)
    assert is_palindrome(1221) and not is_palindrome(123)
    assert is_palindrome(5, 2) and not is_palindrome(6, 2)       # binary 101 and 110
    assert merge_ranges([[1, 3], [2, 6], [8, 10], [15, 18]]) == [[1, 6], [8, 10], [15, 18]]
    assert merge_ranges([[1, 4], [4, 5]]) == [[1, 5]]            # sharing one point counts
    assert merge_ranges([[8, 10], [1, 3]]) == [[1, 3], [8, 10]]  # any input order
    assert merge_ranges([]) == []


# ======================================================
# FOLLOW-UP 1: Adjacent integer ranges ([1, 2] and [3, 4]) should merge too
# ======================================================
# ---- follow-up 1: tests ----
# Answer: allow a gap of 1 when comparing (the `adjacent` parameter above).
def test_follow_up_1():
    touching = [[1, 2], [3, 4]]
    assert merge_ranges(touching) == touching
    assert merge_ranges(touching, True) == [[1, 4]]


# ======================================================
# FOLLOW-UP 2: Summarise a sorted list of integers as ranges (LeetCode 228)
# ======================================================
# ---- follow-up 2: function ----
# Answer: one pass: extend the current range while the next number is
# exactly one more, otherwise start a new range. The same "extend or
# start new" shape as the merge.
def summarize_ranges(nums):
    ranges = []
    for x in nums:
        if ranges and x == ranges[-1][1] + 1:
            ranges[-1][1] = x
        else:
```

```
            ranges.append([x, x])
    return ranges


# ---- follow-up 2: tests ----
def test_follow_up_2():
    assert summarize_ranges([0, 1, 2, 4, 5, 7]) == [[0, 2], [4, 5], [7, 7]]
```

```
    assert summarize_ranges([]) == []


# ======================================================
# FOLLOW-UP 3: Palindrome in another base, or as a linked list
# ======================================================
# Answer: another base: pass `base` (already supported). Linked list:
# find the middle with a slow and a fast pointer, reverse the second
# half, and compare the two halves node by node.


# ======================================================
# RUN EVERYTHING
# ======================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

192 Permutation Sequence — HARD

Template fit: factorial number system: divide k - 1 by (n - 1)!, (n - 2)!, ... to pick each digit from the list of unused digits

QUESTION

The set [1, 2, ..., n] has n! distinct permutations. Listing them in strictly increasing (lexicographic) order and numbering them from 1 , return the k -th permutation as a string. Aim for an O(n) -ish construction using the factorial number system rather than generating all permutations.

Function signature

def get_permutation(n: int, k: int) -> str:

...

Follow-up (as reported): A classmate asked whether a brute-force "generate all and index" is acceptable; discuss why the factorial-base method avoids O(n!) .

EXAMPLES

Example 1	
Input	n = 3, k = 3
Output	"213"
Why	Order is 123, 132, 213, 231, 312, 321; the 3rd is "213".

Example 2	
Input	n = 4, k = 9
Output	"2314"
Why	The 9th permutation of 1234 in lexicographic order is "2314".

QUESTION

Example 3	
Input	`n = 3, k = 1`
Output	`"123"`
Why	`The first permutation is the sorted sequence.`

CONSTRAINTS

- 1 <= n <= 9
- 1 <= k <= n!

FULL CODE | main question first, then every follow-up, tests at the end

```
# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ========================================================

# ========================================================
# MAIN QUESTION
# ========================================================
# ---- main question: helper functions ----
def factorial(n):
    # 1 * 2 * ... * n  (the number of orderings of n different things)
    result = 1
    for i in range(2, n + 1):
        result *= i
    return result


# ---- main question: main function ----
# Idea (the factorial number system): the permutations of 1..n are in
# blocks. The first digit decides a block of (n - 1)! permutations, so
# k // (n - 1)! says which unused digit comes first; then k % (n - 1)!
# is the position inside that block, and the same step repeats on the
# remaining digits. k starts at 1, so subtract 1 first.
def get_permutation(n, k):
    digits = []
    for d in range(1, n + 1):
        digits.append(d)
    k -= 1
    answer = ""
    for left in range(n, 0, -1):
        block = factorial(left - 1)
        chosen = digits.pop(k // block)
        answer += str(chosen)
        k %= block
    return answer


# ---- main question: tests ----
def test_main_question():
    assert get_permutation(3, 3) == "213"
    assert get_permutation(4, 9) == "2314"
    assert get_permutation(3, 1) == "123"
    assert get_permutation(3, 6) == "321"
```

```
# ========================================================
# FOLLOW-UP 1: Given a permutation, what is its position k in the list?
# ========================================================
```

```python
# ---- follow-up 1: function ----
# Answer: run the idea backwards. For every position, count how many
# UNUSED digits are smaller than the current one and multiply by the
# factorial of the digits that remain after it; add everything up and add
# one at the end.
def permutation_rank(perm):
    unused = []
    for ch in perm:
        unused.append(int(ch))
    unused.sort()
    rank = 0
    for i in range(len(perm)):
        smaller = unused.index(int(perm[i]))          # unused digits smaller than this one
        rank += smaller * factorial(len(perm) - i - 1)
        unused.pop(smaller)
    return rank + 1


# ---- follow-up 1: tests ----
def test_follow_up_1():
    for n in range(1, 7):
        step = factorial(n) // 40
        if step < 1:
            step = 1
        for k in range(1, factorial(n) + 1, step):
            assert permutation_rank(get_permutation(n, k)) == k


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

GROUP 25 OF 43 | 6 QUESTIONS, ONE CODE TEMPLATE | 4 M 2 H

Look at the input once and keep only a small summary (and count your calls)

193 Two-Day Log Intersection under Memory Limit

MEDIUM

Template fit: dict object -> set of event codes for each day; keep the objects that appear on both days with at least two distinct codes

QUESTION

PhD intern coding round 1: two log files with **`(timestamp, obj_id, client_id)`** entries. Find **`obj_id`**s that (a) appear in both files and (b) have at least two distinct **`client_id`**s. Follow-up imposes a memory limit, expecting external sort + two-pointer.

Input: two log files; each line is (timestamp, obj_id, client_id).
Find every obj_id such that:

(a) appears in both files, and
(b) is associated with at least 2 distinct client_ids (aggregated across both files).

Follow-up 1: complexity of set-intersection on the file contents.
Follow-up 2: memory cannot hold either file fully — design a streaming solution.

EXAMPLES

```
File A:  (t1, X, c1), (t2, X, c2), (t3, Y, c1)
File B:  (t4, X, c1), (t5, Z, c2)
-> Only X is in both files; X has 2 distinct clients {c1, c2}.
Output: [X]
```

FULL CODE | main question first, then every follow-up, tests at the end

```python
# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: main function ----
# Idea (everything fits in memory): for each day build a dictionary
# object -> set of clients. Keep the objects that appear on BOTH days and
# whose clients, over the two days together, number at least 2.
# A row is (timestamp, object id, client id).
def log_objects(day1, day2):
    clients1 = {}
    clients2 = {}
    for timestamp, obj, client in day1:
        if obj not in clients1:
            clients1[obj] = set()
        clients1[obj].add(client)
    for timestamp, obj, client in day2:
        if obj not in clients2:
            clients2[obj] = set()
        clients2[obj].add(client)
    result = []
    for obj in clients1:
        if obj in clients2 and len(clients1[obj] | clients2[obj]) >= 2:
            result.append(obj)
    return sorted(result)
```

```
# ---- main question: tests ----
DAY1 = [(1, "X", "c1"), (2, "X", "c2"), (3, "Y", "c1")]
DAY2 = [(4, "X", "c1"), (5, "Z", "c2")]


def test_main_question():
    assert log_objects(DAY1, DAY2) == ["X"]
    one = [(1, "A", "c1")]
    assert log_objects(one, one) == []                          # one client in total is not enough
    assert log_objects([(1, "A", "c1")], [(2, "A", "c2")]) == ["A"]     # clients may be split across the two
 days
    assert log_objects([], DAY2) == []
```

```
# ============================================================
# FOLLOW-UP 1: Why prefer sort and two pointers to a streaming hash map?
# ============================================================
# ---- follow-up 1: helper functions ----
def object_of(row):
    return row[1]


def group_rows(rows):
    # rows sorted by object -> a list of (object, set of its clients),
    # produced one object at a time
    groups = []
    current = None
    clients = set()
    for timestamp, obj, client in rows:
        if obj != current:
            if current is not None:
                groups.append((current, clients))
            current = obj
            clients = set()
        clients.add(client)
    if current is not None:
        groups.append((current, clients))
    return groups


# ---- follow-up 1: function ----
# Answer: a hash map needs memory proportional to the number of DISTINCT
# objects (and clients), which is exactly what the memory limit forbids.
# If both files are sorted by object (an external sort), they can be
# merged like two sorted lists, and only ONE object's clients are in memory
# at a time.
def log_objects_sorted(rows1, rows2):
    groups1 = group_rows(rows1)
    groups2 = group_rows(rows2)
    result = []
    i = 0
    j = 0
    while i < len(groups1) and j < len(groups2):
        if groups1[i][0] == groups2[j][0]:
            if len(groups1[i][1] | groups2[j][1]) >= 2:
                result.append(groups1[i][0])
            i += 1
            j += 1
```

```
        elif groups1[i][0] < groups2[j][0]:
            i += 1
        else:
            j += 1
    return result
```

```python
# ---- follow-up 1: tests ----
def test_follow_up_1():
    sorted1 = sorted(DAY1, key=object_of)
    sorted2 = sorted(DAY2, key=object_of)
    assert log_objects_sorted(sorted1, sorted2) == ["X"]
    one = [(1, "A", "c1")]
    assert log_objects_sorted(one, one) == []
    assert log_objects_sorted([(1, "A", "c1")], [(2, "A", "c2")]) == ["A"]


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

194 Find shared objects across two log files — MEDIUM

Template fit: load only the smaller file's object ids into a set, then stream the big file and keep ids found in the set

QUESTION

You are given two large log files representing activity on two different days. Each line of each log has three fields:

- **`timestamp`**: a time value (you may treat it as an opaque string or integer)
- **`obj_id`**: identifier of an object (string or integer)
- **`client_id`**: identifier of a client (string or integer) The logs are not necessarily sorted by any field. Define an object as **interesting** if:

1. Its **`obj_id`** appears at least once in day 1's log **and** at least once in day 2's log; and 2. Across all of its appearances in both days combined, it is associated with at least **two distinct `client_id`** values.

Tasks

1. Design and implement a function that, given file handles (or streaming iterators) for the two log files, returns the set of all such interesting **`obj_id`**s.
2. For your initial, straightforward in-memory solution (you may assume both logs fit in memory), state the time and space complexity in terms of the total number of log records.
3. Follow-up (memory constrained): Now assume the logs are very large and **cannot** both fit into memory at once.

- You may still sequentially scan each file and you may perform external sorting.
- Describe how you would modify your approach to work under this memory constraint.
- Analyze the time and space complexity of this memory-efficient solution.

Return sorted obj_ids that appear in both day logs and have at least two distinct client_ids across both days combined.
Constraints
Rows are [timestamp, obj_id, client_id] or dicts with those fields

EXAMPLES

```
Input: ([['t1', 'o1', 'c1'], ['t2', 'o2', 'c1']], [['t3', 'o1', 'c2'], ['t4', 'o2', 'c1']])
Expected Output: ['o1']
Input: ([], [['t', 'o', 'c']])
Expected Output: []
```

QUESTION

Hints

Track which days each object appears in and its set of clients.

FULL CODE | main question first, then every follow-up, tests at the end

```
# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ========================================================

import zlib


# ========================================================
# MAIN QUESTION
# ========================================================
# ---- main question: helper functions ----
def object_of(row):
    return row[1]                                       # a row is [timestamp, object id, client id]


# ---- main question: main function ----
# Idea: an object is "shared" when it appears on BOTH days and was seen by
# at least 2 different clients over the two days. Go through every row once
# and remember, for each object, which days it appeared on and which
# clients saw it. Each line is read once; memory depends on the number of
# distinct objects.
def shared_objects(day1, day2):
    days = {}                                           # object -> the set of days it appeared on
    clients = {}                                        # object -> the set of clients that saw it
    for number, rows in ((1, day1), (2, day2)):
        for timestamp, obj, client in rows:
            if obj not in days:
                days[obj] = set()
                clients[obj] = set()
            days[obj].add(number)
            clients[obj].add(client)
    result = []
    for obj in days:
        if len(days[obj]) == 2 and len(clients[obj]) >= 2:
            result.append(obj)
    return sorted(result)


# the other way: sort both days by object, then merge them like two sorted
# lists, holding the clients of only ONE object at a time
def shared_objects_sorted(day1, day2):
    a = sorted(day1, key=object_of)
```

```
    b = sorted(day2, key=object_of)
    i = 0
    j = 0
    result = []
    while i < len(a) and j < len(b):
        if a[i][1] < b[j][1]:
            i += 1
        elif a[i][1] > b[j][1]:
            j += 1
        else:
            obj = a[i][1]
            clients = set()
            while i < len(a) and a[i][1] == obj:
```

```
                clients.add(a[i][2])
                i += 1
            while j < len(b) and b[j][1] == obj:
                clients.add(b[j][2])
                j += 1
            if len(clients) >= 2:
                result.append(obj)
    return result


# ---- main question: tests ----
D1 = [["t1", "o1", "c1"], ["t2", "o2", "c1"]]
D2 = [["t3", "o1", "c2"], ["t4", "o2", "c1"]]


def test_main_question():
    assert shared_objects(D1, D2) == ["o1"]
    assert shared_objects([], [["t", "o", "c"]]) == []
    assert shared_objects_sorted(D1, D2) == ["o1"]


# =========================================================
# FOLLOW-UP 1: Both files are too big for memory
# =========================================================
# ---- follow-up 1: helper functions ----
def bucket_of(obj, buckets):
    # a STABLE hash: Python's own hash() changes between runs
    return zlib.crc32(str(obj).encode()) % buckets


# ---- follow-up 1: function ----
# Answer: hash-partition both files into B buckets (on disk) by the hash
```

```
# of the object id, then process one pair of buckets at a time with the
# in-memory join. All rows of an object land in the same bucket. Choose B
# so that one bucket of the smaller file fits in memory.
def shared_objects_partitioned(day1, day2, buckets=4):
    parts1 = []
    parts2 = []
    for b in range(buckets):
        parts1.append([])
        parts2.append([])
    for row in day1:
        parts1[bucket_of(row[1], buckets)].append(row)
    for row in day2:
        parts2[bucket_of(row[1], buckets)].append(row)
    result = []
    for b in range(buckets):
        result += shared_objects(parts1[b], parts2[b])
    return sorted(result)


# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert shared_objects_partitioned(D1, D2, 3) == ["o1"]
    many1 = []
    many2 = []
    for i in range(40):
        many1.append([i, "o" + str(i % 7), "c" + str(i % 3)])
        many2.append([i, "o" + str(i % 5), "c" + str(i % 2)])
    expected = shared_objects(many1, many2)
    assert shared_objects_partitioned(many1, many2, 5) == expected
    assert shared_objects_sorted(many1, many2) == expected


# =========================================================
```

```
# FOLLOW-UP 2: Want objects seen on both days within an hour of each other
# ========================================================
# Answer: sort each day's rows by (object, timestamp) and merge the two
# lists with two pointers; for one object keep a pointer into the other
# day's timestamps and move it while the gap is more than an hour.


# ========================================================
# FOLLOW-UP 3: A tiny memory budget and an approximate answer is fine
# ========================================================
# ---- follow-up 3: function ----
# Answer: build a Bloom filter from one file's object ids (a bit array and a
```

```
# few hash functions) and test the other file's ids against it. It may say
# "yes" for a few objects that were never added (a small false-positive
# rate) but never "no" for one that was. Verify the few hits exactly in a
# second pass if needed.
class Bloom:
    def __init__(self, bits=1 << 16, hashes=3):
        self.bits = bytearray(bits)
        self.hashes = hashes

    def spots(self, key):
        places = []
        for seed in range(self.hashes):
            text = (str(seed) + ":" + str(key)).encode()
            places.append(zlib.crc32(text) % len(self.bits))
        return places

    def add(self, key):
        for spot in self.spots(key):
            self.bits[spot] = 1

    def maybe(self, key):
        for spot in self.spots(key):
            if self.bits[spot] == 0:
                return False
        return True


# ---- follow-up 3: tests ----
def test_follow_up_3():
    bloom = Bloom()
    for row in D1:
        bloom.add(row[1])
    assert bloom.maybe("o1") and bloom.maybe("o2")
    assert not bloom.maybe("o999")


# ========================================================
# RUN EVERYTHING
# ========================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_3()
    print("all tests passed")
```

```
if __name__ == "__main__":
    run_all_tests()
```

195 Streaming Points: Emit K Within Distance

MEDIUM

Template fit: keep the points in a sorted list; each new point is inserted with bisect and only windows around it are re-checked

QUESTION

Positions (floats) arrive one at a time. Whenever k stored points fit inside a span of d (largest minus smallest is at most d), output those k points and remove them. add(x) returns the groups emitted because of this arrival (a list of lists).

Function signature

class PointStream:
    def __init__(self, k: int, d: float)
    def add(self, x: float) -> list

EXAMPLES

Example 1

Input	`s = PointStream(3, 2.0)`
	`s.add(1.0)`
	`s.add(5.0)`
	`s.add(2.0)`
	`s.add(1.5)`

Output	`s.add(1.0) -> []`
	`s.add(5.0) -> []`
	`s.add(2.0) -> []`
	`s.add(1.5) -> [[1, 1.5, 2]]`

Why	After 1, 5, 2 no three points fit in a span of 2 (1 and 5 are too far apart); adding 1.5 gives 1, 1.5, 2 with span 1, which is emitted and removed.

FULL CODE | main question first, then every follow-up, tests at the end

```
# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

from bisect import bisect_left, insort


# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: main function ----
# Idea: points arrive one at a time. As soon as there are k waiting points
# that fit inside a stretch of length d (the span of the group =
# largest - smallest <= d), emit them as a group and forget them. Keep
# the waiting points in a SORTED list (insort = binary search plus insert).
# Before the new point no such group existed, so a new group must contain the
# new point: only the (at most k) windows of k neighbours that include its
# position need to be checked.
class PointStream:
    def __init__(self, k, d):
        self.k = k
        self.d = d
        self.points = []                        # the waiting points, sorted

    def window_start(self, pos):
        # the first window of k neighbouring points that includes position
        # pos and fits in a span of d, or None
        first = pos - self.k + 1
        if first < 0:
            first = 0
```

```
        last = len(self.points) - self.k
        if pos < last:
            last = pos
        for i in range(first, last + 1):
            span = self.points[i + self.k - 1] - self.points[i]
            if span <= self.d:
                return i
        return None

    def add(self, x):
        insort(self.points, x)
        groups = []
```

```
        pos = bisect_left(self.points, x)
        while True:
            i = self.window_start(pos)
            if i is None:
                return groups
            groups.append(self.points[i:i + self.k])      # emit the group ...
            del self.points[i:i + self.k]                 # ... and remove it so it is emitted only once
            pos = i


# ---- main question: tests ----
def test_main_question():
    s = PointStream(3, 1.0)
    assert s.add(0.5) == []
    assert s.add(5.0) == []
    assert s.add(1.0) == []
    assert s.add(1.4) == [[0.5, 1.0, 1.4]]              # the span is 0.9 <= 1.0
    assert s.points == [5.0]                            # the far point is still waiting


# ======================================================
# FOLLOW-UP 1: What if "within d" means every pair, and points are in 2D?
# ======================================================


# ======================================================
# FOLLOW-UP 4: The condition is "every pair within d", not "span at most d"
# ======================================================
# ---- follow-up 4: function ----
# Answer: on a line they are the same: the largest distance between two
# points is max - min, so checking the span is enough (verified below
# against checking all pairs). In 2D they differ: "every pair within d"
# becomes a clique problem (and a grid index finds the candidates).
def all_pairs_within(group, d):
    for a in group:
        for b in group:
            if abs(a - b) > d:
                return False
    return True


# ---- follow-up 4: tests ----
def test_follow_up_4():
    s = PointStream(3, 1.0)
    s.add(0.5)
```

```
    s.add(1.0)
    groups = s.add(1.4)
    for group in groups:
        assert all_pairs_within(group, 1.0)
        assert max(group) - min(group) <= 1.0           # on a line, span and pair checks agree
    assert not all_pairs_within([0.0, 0.6, 1.2], 1.0)
    t = PointStream(2, 0.5)                             # k = 2 groups close neighbours as they appear
```

```
    assert t.add(1.0) == []
    assert t.add(1.4) == [[1.0, 1.4]]


# ========================================================
# FOLLOW-UP 2: Floating-point coordinates
# ========================================================
# Answer: compare with a small tolerance (span <= d + 1e-9), or use
# integers (scale and round) or exact fractions when exactness matters.


# ========================================================
# FOLLOW-UP 3: Several groups qualify at once
# ========================================================
# Answer: add() already loops: after emitting a group it checks the
# windows again from the same position, so every group that the new
# point completes is emitted.


# ========================================================
# RUN EVERYTHING
# ========================================================
def run_all_tests():
    test_main_question()
    test_follow_up_4()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

196 Minimize calls to find all bad test pairs

MEDIUM

Template fit: the same group testing: put a whole group in one call and split only the groups that fail

QUESTION

You have (n) atomic tests (t_1, t_2, \dots, t_n). You are given access to a black-box function:

```
bool runTest(set<Test> S)
```

which behaves as follows:

- **`runTest(S) == True`** if the subset (S) contains **no** incompatible (bad) pair of tests.
- **`runTest(S) == False`** if the subset (S) contains **at least one** incompatible (bad) pair of tests. There is some unknown set of unordered **bad pairs** ((t_i, t_j)), where (i\ne j). You do **not** know in advance which pairs are bad or how many bad pairs there are. Your goal is to discover **all** bad pairs while making as few calls to **`runTest`** as possible. You may adaptively choose each subset (S) based on the results of previous calls.

Tasks

1. Design an algorithm that, using only calls to **`runTest`**, finds **all** bad pairs ((t_i, t_j)).
2. Analyze the number of calls to **`runTest`** your algorithm requires, as a function of:

- (n): the total number of tests, and - (k): the total number of bad pairs.

3. Argue why your algorithm is correct (i.e., why it will eventually find every bad pair and not report any good pair as bad). You do not need to provide full production code, but describe the algorithm clearly enough that it could be implemented.

QUESTION

You have n atomic tests numbered 0 to n - 1. Some unknown unordered pairs of tests are incompatible, called bad pairs. A black-box oracle runTest(S) returns True if the subset S contains no bad pair, and False if S contains at least one bad pair. In this non-interactive version, you are given bad_pairs only so your code can simulate the oracle. Implement the following deterministic oracle-discovery strategy and return both the bad pairs it discovers and the number of oracle calls it makes. Strategy:

1. Process vertices from right to left: v = n - 1, n - 2, ..., 0.
2. Maintain all bad pairs already discovered among vertices greater than the current v.
3. For the suffix vertices v + 1 through n - 1, greedily partition them into color classes such that no discovered bad pair appears inside one color class. Since all suffix-suffix bad pairs have already been discovered, every color class is truly compatible internally.
4. For each color class C, find all bad partners of v inside C using recursive group testing:

- Call runTest({v} union C).
- If the oracle returns True, v has no bad pair with any test in C.
- If C has size 1 and the oracle returns False, that single pair is bad.
- Otherwise split C into two halves and recurse on both halves. Return all discovered bad pairs sorted lexicographically, where each pair is represented as [i, j] with i < j, along with the exact number of runTest calls made by this strategy.

Constraints
0 <= n <= 5000 <= len(bad_pairs) <= n * (n - 1) / 2Each bad pair contains two distinct test indices in the range [0, n - 1]bad_pairs may be given in any endpoint order

EXAMPLES

```
Input: (0, [])
Expected Output: ([], 0)
```

Explanation: There are no tests and therefore no oracle calls are needed.

```
Input: (4, [])
Expected Output: ([], 3)
```

Explanation: For v = 2, 1, and 0, the algorithm queries the compatible suffix once and finds no bad pairs. v = 3 has an empty suffix.

Hints

If a candidate set C is internally compatible, then runTest({v} union C) tells you whether v conflicts with at least one element of C.Process tests from right to left so that when handling v, all bad pairs inside the suffix v + 1 ... n - 1 are already known and can be used to build compatible groups.

FULL CODE | main question first, then every follow-up, tests at the end

```python
# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ========================================================

# ========================================================
# MAIN QUESTION
# ========================================================
# ---- main question: helper functions ----
# run_test(set_of_tests) is the expensive oracle: it returns True when
# the tests in the set pass TOGETHER (no bad pair inside it).

def shortest_failing_prefix(items, extra, run_test):
    # binary search the smallest prefix of `items` that fails together
```

```
    # with the tests in `extra`; its LAST item is a partner of something
    low = 1
    high = len(items)
    while low < high:
        mid = (low + high) // 2
        if run_test(set(items[:mid]) | extra):
            low = mid + 1                          # still passes: the culprit is later
        else:
            high = mid
    return items[low - 1]


def partners(group, t, run_test):
    # every member of the group that is bad together with t
    rest = list(group)
    found = []
    while len(rest) > 0 and not run_test(set(rest) | {t}):
        x = shortest_failing_prefix(rest, {t}, run_test)
        found.append(x)
        rest.remove(x)                             # take it out and test the others again
    return found


# ---- main question: main function ----
# Idea: keep GROUPS of tests that pass together. For each new test t: ask
# each group for its partners with t (one call tells us when there are
# none; otherwise a binary search finds each partner in about log n
# calls). t joins the first group in which it has no partner, or starts a
# new group. Returns the set of bad pairs (smaller test, bigger test).
```

```
def find_bad_pairs(tests, run_test):
    groups = []
    pairs = set()
    for t in tests:
        placed = False
        for group in groups:
            found = partners(group, t, run_test)
            for x in found:
                if x < t:
                    pairs.add((x, t))
                else:
                    pairs.add((t, x))
            if len(found) == 0 and not placed:
                group.append(t)
                placed = True
        if not placed:
            groups.append([t])
    return pairs


# ---- main question: tests ----
class Oracle:
    # a pretend run_test that knows the bad pairs and counts its calls
    def __init__(self, bad):
        self.bad = bad
        self.calls = 0

    def __call__(self, subset):
        self.calls += 1
        ordered = sorted(subset)
        for i in range(len(ordered)):
            for j in range(i + 1, len(ordered)):
                if (ordered[i], ordered[j]) in self.bad:
                    return False
        return True
```

```
def test_main_question():
    bad = {(3, 17), (5, 29)}
    oracle = Oracle(bad)
    assert find_bad_pairs(list(range(30)), oracle) == bad
    assert oracle.calls < 435 // 2                      # far fewer than the 435 pairs
    oracle = Oracle(set())
    assert find_bad_pairs(list(range(30)), oracle) == set()
    assert oracle.calls <= 30                           # no bad pairs: about one call per test
```

```
# ========================================================
# FOLLOW-UP 1: How many calls does it need, and can you prove a lower bound?
# ========================================================
# Answer: with p bad pairs about p * log2(n) calls plus about n calls for
# the checks that find nothing. A lower bound: there are about n^2 / 2
# possible pairs, each call gives one yes/no answer, so at least
# log2(n^2 / 2), about 2 * log2(n) calls are needed even for ONE bad pair.


# ========================================================
# FOLLOW-UP 2: Only one bad pair exists
# ========================================================


# ========================================================
# FOLLOW-UP 4: Only one bad pair exists
# ========================================================
# ---- follow-up 2: function ----
# Answer: binary search the shortest failing prefix of the tests (its last
# test belongs to the pair), then binary search the EARLIER tests for the
# partner: about 2 * log2(n) calls instead of n * (n - 1) / 2.
def one_bad_pair(tests, run_test):
    if run_test(set(tests)):
        return None                                     # everything passes: no bad pair
    b = shortest_failing_prefix(tests, set(), run_test)
    earlier = tests[:tests.index(b)]
    a = shortest_failing_prefix(earlier, {b}, run_test)
    return (a, b)


# ---- follow-up 2: tests ----
def test_follow_up_2():
    oracle = Oracle({(10, 40)})
    assert one_bad_pair(list(range(64)), oracle) == (10, 40)
    assert oracle.calls <= 16                           # about 2 * log2(64) = 12, far below 2016 pairs
    assert one_bad_pair(list(range(10)), Oracle(set())) is None


# ========================================================
# FOLLOW-UP 3: runTest is expensive but can run in parallel
# ========================================================
# Answer: ask several questions at once: test many groups in parallel
# (for example every group against the new test t at the same time), and
```

```
# split a failing group into several parts instead of two. You use more
# calls in total but fewer rounds one after another.


# ========================================================
# RUN EVERYTHING
# ========================================================
def run_all_tests():
    test_main_question()
    test_follow_up_2()
```

```
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

197 Find All Bad Pairs Using runTest Oracle

HARD

Template fit: for each test, binary search the prefix of earlier tests where runTest first fails; that test is a bad partner; repeat until none is left

QUESTION

PhD intern coding round 2: given a black-box oracle **`runTest(S)`** that returns **`true`** iff a subset **`S`** contains no bad pair, identify all bad pairs across N test items with as few oracle calls as possible.

Items t_1, ..., t_N are given.
Oracle: runTest(S: Set[Item]) -> bool returns true iff S contains zero bad pairs; otherwise false.
Output: every bad pair (t_i, t_j).
Minimize total oracle calls.

EXAMPLES

```
N=4, bad pairs = {(1,2)}.
runTest({1,2,3,4}) -> false -> at least one bad pair somewhere.
runTest({1,3,4})   -> true  -> 2 is in every bad pair containing items from this set.
... binary search to isolate.
```

FULL CODE | main question first, then every follow-up, tests at the end

```
# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
# run_test(set_of_items) is the expensive oracle: it returns True when the
# items in the set pass TOGETHER (no bad pair inside it).

def first_partner(group, t, run_test):
    # binary search the shortest prefix of the group that fails together
    # with t; its LAST member is a partner of t
    low = 1
    high = len(group)
    while low < high:
        mid = (low + high) // 2
        if run_test(set(group[:mid]) | {t}):
            low = mid + 1
        else:
            high = mid
    return group[low - 1]


def partners(group, t, run_test):
    # all members of the group that are bad together with t
    rest = list(group)
    found = []
    while len(rest) > 0 and not run_test(set(rest) | {t}):
        x = first_partner(rest, t, run_test)
        found.append(x)
```

```
        rest.remove(x)                              # take it out and test the others again
    return found


# ---- main question: main function ----
# Idea: keep groups of items that pass together (no bad pair inside a
# group). A new item t is checked against every group: one call says
# "no partner here", otherwise a binary search finds each partner. t joins
# the first group in which it has no partner, or starts a new group.
# Returns the set of bad pairs (smaller item, bigger item).
```

```
def find_bad_pairs(items, run_test):
    groups = []
    pairs = set()
    for t in items:
        placed = False
        for group in groups:
            found = partners(group, t, run_test)
            for x in found:
                if x < t:
                    pairs.add((x, t))
                else:
                    pairs.add((t, x))
            if len(found) == 0 and not placed:
                group.append(t)
                placed = True
        if not placed:
            groups.append([t])
    return pairs


# ---- main question: tests ----
class Oracle:
    # a pretend run_test that knows the bad pairs and counts its calls
    def __init__(self, bad):
        self.bad = bad
        self.calls = 0

    def __call__(self, subset):
        self.calls += 1
        ordered = sorted(subset)
        for i in range(len(ordered)):
            for j in range(i + 1, len(ordered)):
                if (ordered[i], ordered[j]) in self.bad:
                    return False
        return True


def test_main_question():
    bad = {(3, 17), (5, 29)}
    oracle = Oracle(bad)
    assert find_bad_pairs(list(range(30)), oracle) == bad
    assert oracle.calls < 435 // 2                      # far fewer than the 435 possible pairs
    oracle = Oracle(set())
    assert find_bad_pairs(list(range(30)), oracle) == set()
    assert oracle.calls <= 30                           # no bad pairs: about one call per item
```

```
    assert find_bad_pairs([1, 2], Oracle({(1, 2)})) == {(1, 2)}


# ======================================================
# FOLLOW-UP 1: How many calls are needed in the best case and the worst case?
# ======================================================
# Answer: best case, no bad pair: about one call per item (n calls), because
# each new item passes with the one group. Worst case, many bad pairs: each
# bad pair costs about log2(n) calls to find, plus the calls that prove a
```

```
# group has no more partners; with p bad pairs about p * log2(n) + n calls,
# and in the extreme (every pair bad) about n^2 / 2 calls.


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

198 Find maximum value in huge text file

HARD

Template fit: the template as written; the file is never loaded as a whole

QUESTION

A text file is too large to load into memory (tens or hundreds of gigabytes). Each line holds one signed 64-bit integer. Return the largest value by reading the file sequentially, holding only a small amount in memory.

Function signature

def max_in_file(path: str) -> int

EXAMPLES

Example 1

Input	**nums.txt contains the lines:** **5** **-3** **12** **7**
Output	**max_in_file(path) -> 12**
Why	**Only the current line and the best value so far are in memory, so the file size does not matter.**

FULL CODE | main question first, then every follow-up, tests at the end

```
# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

import heapq
import os
import tempfile


# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: main function ----
# Idea: never load the file. Read it line by line (the file object reads
# in buffered chunks) and keep ONE variable: the maximum so far. It
# starts as None, not 0, so that a file of only negative numbers, or of one
# line, gives the right answer. Blank lines are ignored (state this
# contract). An empty file has no maximum: None.
def max_in_file(path):
```

```
    best = None
    with open(path) as f:
        for line in f:
            text = line.strip()
            if text == "":
                continue
            value = int(text)
            if best is None or value > best:
                best = value
    return best


# ---- main question: tests ----
def write_file(lines):
    # a small test file; returns its path
    f = tempfile.NamedTemporaryFile("w", delete=False)
    f.write("".join(lines))
    f.close()
    return f.name


def test_main_question():
    p = write_file(["5\n", "-3\n", "\n", "99\n", "12"])
```

```
    assert max_in_file(p) == 99
    q = write_file(["-7\n", "-2\n", "-9\n"])
    assert max_in_file(q) == -2                          # all negative numbers
    big = write_file([str(2 ** 63 - 1) + "\n", str(-2 ** 63) + "\n"])
    assert max_in_file(big) == 2 ** 63 - 1               # extreme 64-bit values
    e = write_file([])
    assert max_in_file(e) is None                        # an empty file has no maximum
    for name in (p, q, big, e):
        os.remove(name)


# ======================================================
# FOLLOW-UP 1: Edge cases to mention
# ======================================================
# Answer: an empty file, blank or malformed lines (skip or fail? say which),
# one line only, ALL numbers negative (do not start the maximum at 0),
# numbers bigger than 64 bits, a last line without a newline, and a file
# that is being written while you read it.


# ======================================================
# FOLLOW-UP 2: How do you scale to many machines and handle failures?
# ======================================================


# ======================================================
# FOLLOW-UP 3: Is it I/O-bound or CPU-bound, and why does that matter?
# ======================================================
# Answer: reading a file and doing one comparison per line is I/O-BOUND:
# the disk or network is the limit, not the processor. So more threads on
# one machine do not help; more machines (each reading its own part of
# the data) do. Tasks are idempotent, so a failed worker is simply retried.


# ======================================================
# FOLLOW-UP 4: The file has billions of lines and one machine is too slow
# ======================================================
# ---- follow-up 4: helper functions ----
def max_in_range(path, start, end):
    # one worker: the maximum of the lines that START inside [start, end)
    best = None
    with open(path, "rb") as f:
```

```
        if start > 0:
            f.seek(start - 1)
            f.readline()                # we may be in the middle of a line: the worker before owns it
```

```
        while f.tell() < end:
            line = f.readline()
            if not line:
                break
            if line.strip() != b"":
                value = int(line)
                if best is None or value > best:
                    best = value
    return best


# ---- follow-up 4: function ----
# Answer: split the file by BYTE offsets across the machines (each one
# skips to the next newline), compute a local maximum on each, and take
# the maximum of those few numbers. Here the "machines" are just
# loops over the byte ranges.
def parallel_max(path, workers=4):
    size = os.path.getsize(path)
    best = None
    for i in range(workers):
        start = size * i // workers
        end = size * (i + 1) // workers
        local = max_in_range(path, start, end)
        if local is not None and (best is None or local > best):
            best = local
    return best


# ---- follow-up 4: tests ----
def test_follow_up_4():
    p = write_file(["5\n", "-3\n", "\n", "99\n", "12"])
    assert parallel_max(p, 3) == 99
    e = write_file([])
    assert parallel_max(e) is None
    numbers = []
    lines = []
    for i in range(2000):
        number = (i * 7919) % 10007 - 5000
        numbers.append(number)
        lines.append(str(number) + "\n")
    r = write_file(lines)
    for workers in (1, 2, 5, 8):                          # every worker count agrees
        assert parallel_max(r, workers) == max(numbers)
    for name in (p, e, r):
        os.remove(name)
```

```


# ======================================================
# FOLLOW-UP 5: Now return the K largest values instead of only the maximum
# ======================================================
# ---- follow-up 5: function ----
# Answer: keep a MIN-heap of size K while streaming: push a value while
# the heap is not full, otherwise replace the root (the smallest of the K
# best) when the new value is larger. Memory O(K), time O(n log K).
# For a parallel version, merge the per-worker heaps.
def top_k_in_file(path, k):
    heap = []
    with open(path) as f:
        for line in f:
            text = line.strip()
            if text == "":
```

```
                continue
            value = int(text)
            if len(heap) < k:
                heapq.heappush(heap, value)
            elif value > heap[0]:
                heapq.heapreplace(heap, value)
    return sorted(heap, reverse=True)


# ---- follow-up 5: tests ----
def test_follow_up_5():
    p = write_file(["5\n", "-3\n", "\n", "99\n", "12"])
    assert top_k_in_file(p, 2) == [99, 12]
    assert top_k_in_file(p, 10) == [99, 12, 5, -3]
    os.remove(p)


# =============================================================
# RUN EVERYTHING
# =============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_4()
    test_follow_up_5()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

GROUP 26 OF 43 | 5 QUESTIONS, ONE CODE TEMPLATE | 3 M 2 H

Grid DP: each cell from its neighbours above and to the left

199 Grid Path Count with Diagonal Moves and Checkpoints

MEDIUM

Template fit: advance column by column keeping the number of ways in each row; new[r] = old[r-1] + old[r] + old[r+1]; zero out rows that miss a checkpoint

QUESTION

Count the paths in an n x m grid from the bottom-left cell (n - 1, 0) to the bottom-right cell (n - 1, m - 1). From (r, c) you may move up-right (r - 1, c + 1), right (r, c + 1) or down-right (r + 1, c + 1) and must stay inside the grid. Optionally you are given checkpoint cells that every path has to pass through; with ordered=True they must be visited in the order given.

Function signature

def count_paths(n: int, m: int, checkpoints=(), ordered=False) -> int

EXAMPLES

Example 1	
Input	3, 3
Output	2
Why	Two paths: through (1, 1) and then down-right, or through (2, 1) and then right.

Example 2	
Input	3, 3, checkpoints = [(1, 1)]
Output	1
Why	The path must also pass through (1, 1), the middle cell.

Example 3	
Input	3, 4, checkpoints = [(1, 1), (2, 2)], ordered = true
Output	1
Why	Two checkpoints in different columns, visited in column order.

FULL CODE | main question first, then every follow-up, tests at the end

```
# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
def step(ways):
    # the next column from the current column only: a path in row r came
    # from row r - 1, r or r + 1 of the column before (up-right, right, down-right)
    n = len(ways)
    following = []
    for r in range(n):
        total = ways[r]
        if r > 0:
            total += ways[r - 1]
        if r + 1 < n:
            total += ways[r + 1]
```

```
        following.append(total)
    return following


def keep_row(ways, row):
    # a checkpoint forces the path through one row of this column
    kept = []
    for r in range(len(ways)):
        if r == row:
            kept.append(ways[r])
        else:
            kept.append(0)
    return kept


def checkpoint_rows(checkpoints, ordered):
    # column -> the row the path must be in; None if that is impossible
    rows = {}
    for r, c in checkpoints:
        if c in rows and rows[c] != r:
            return None                          # two different checkpoints in one column: no path can visit
 both
        rows[c] = r
    if len(rows) != len(checkpoints):
```

```
        return None                              # the same checkpoint listed twice
    if ordered:
        for i in range(len(checkpoints) - 1):
            if checkpoints[i][1] > checkpoints[i + 1][1]:
                return None                      # ordered checkpoints must be listed left to right
    return rows


# ---- main question: main function ----
# A grid with n rows and m columns. A path starts at the bottom-left
# cell, ends at the bottom-right cell and each step moves to the next
# column in the same row or one row up or down (a diagonal). Count the
# paths that pass through all the checkpoints (row, column).
# Idea: ways[r] = the number of paths that are in row r at the current
# column. The next column is ways'[r] = ways[r - 1] + ways[r] + ways[r + 1].
# At a checkpoint column keep only the checkpoint's row. One rolling
# column is enough: O(n) memory. (Two checkpoints in one column: the
# answer is 0, since a path is in exactly one row per column: follow-up 1.)
def count_paths(n, m, checkpoints=(), ordered=False):
    rows = checkpoint_rows(list(checkpoints), ordered)
    if rows is None:
        return 0
    ways = [0] * n
    ways[n - 1] = 1
    if 0 in rows:
        ways = keep_row(ways, rows[0])
    for c in range(1, m):
        ways = step(ways)
        if c in rows:
            ways = keep_row(ways, rows[c])
    return ways[n - 1]


# ---- main question: tests ----
def count_paths_table(n, m):
    # the whole table ways[c][r], to check the rolling version against
    ways = []
    for c in range(m):
        ways.append([0] * n)
    ways[0][n - 1] = 1
    for c in range(1, m):
```

```
        for r in range(n):
            for previous_row in (r - 1, r, r + 1):
                if 0 <= previous_row < n:
                    ways[c][r] += ways[c - 1][previous_row]
```

```
    return ways[m - 1][n - 1]


def test_main_question():
    assert count_paths(3, 3) == 2
    assert count_paths_table(3, 3) == 2
    assert count_paths(1, 5) == 1
    assert count_paths(3, 1) == 1
    assert count_paths_table(4, 6) == count_paths(4, 6)
    assert count_paths(3, 3, [(1, 1)]) == 1                    # force the middle column through row 1
    assert count_paths(3, 4, [(1, 1), (1, 2)], True) == 1      # ordered checkpoints: left to right
    assert count_paths(3, 4, [(1, 2), (1, 1)], True) == 0


# ========================================================
# FOLLOW-UP 1: Two checkpoints share a column
# ========================================================
# Answer: no path can visit both (a path is in exactly one row per
# column), so the answer is 0; checkpoint_rows returns None before the DP
# even starts. Checkpoints at the start or end cell are fine: they only
# filter the rows of the first or the last column.
# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert count_paths(3, 3, [(0, 1), (1, 1)]) == 0


# ========================================================
# RUN EVERYTHING
# ========================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

200 Longest Same-Color Grid Sequence

MEDIUM

Template fit: keep four run lengths per cell (horizontal, vertical, two diagonals) from the previous row and the left cell; the answer is the largest

QUESTION

Given a grid of colours (letters such as R, G, B), return the length of the longest run of one colour along a straight line: horizontal, vertical, or diagonal (in either diagonal direction).

Function signature

def longest_same_color_line(g: list) -> int

EXAMPLES

Example 1	
Input	`['RRRG', 'RRBB', 'GRRR', 'BGBR']`
Output	`4`
Why	`The main diagonal is R, R, R, R from corner to corner: 4. No other line is longer.`

QUESTION

Example 2	
Input	['RGB', 'BGR', 'RGB']
Output	3
Why	The longest line is the vertical run of three G in the middle column.

Example 3	
Input	['RR', 'RR']
Output	2
Why	A single colour fills the grid: the longest line is a full row or column.

FULL CODE | main question first, then every follow-up, tests at the end

```
# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: main function ----
# Idea: the longest straight line of one colour (across, down, or along a
# diagonal). Keep, for the PREVIOUS row, the length of the run that ends
# at each column going straight down, down-left and down-right, and a
# running number for the run across the current row. A cell extends the
# run of the cell above it (or above-left / above-right) if the colour
# matches, otherwise it starts a new run of 1.
def longest_same_color_line(g):
    if not g or not g[0]:
        return 0
    n = len(g[0])
    best = 0
    down = [0] * n                          # runs ending in the previous row ...
    down_left = [0] * n                     # (coming from the above-right cell)
    down_right = [0] * n                    # (coming from the above-left cell)
    for i in range(len(g)):
        row = g[i]
        down_now = [1] * n
        left_now = [1] * n
        right_now = [1] * n
        across = 0
        for j in range(n):
            color = row[j]
            if j > 0 and row[j - 1] == color:
                across += 1                 # continue the run across this row
            else:
                across = 1
            if i > 0:
                if g[i - 1][j] == color:
                    down_now[j] = down[j] + 1
                if j > 0 and g[i - 1][j - 1] == color:
                    right_now[j] = down_right[j - 1] + 1
                if j + 1 < n and g[i - 1][j + 1] == color:
                    left_now[j] = down_left[j + 1] + 1
            for length in (across, down_now[j], left_now[j], right_now[j]):
                if length > best:
                    best = length
```

```
        down = down_now
        down_left = left_now
        down_right = right_now
    return best


# ---- main question: tests ----
def test_main_question():
    assert longest_same_color_line(["RRRG", "RRBB", "GRRR", "BGBR"]) == 4
    assert longest_same_color_line([]) == 0
    assert longest_same_color_line(["R", "R", "R"]) == 3              # one column
    assert longest_same_color_line(["RGR", "GRG"]) == 2               # alternating colours
    assert longest_same_color_line(["RRR", "GGR"]) == 3


# ======================================================
# FOLLOW-UP 1: The line may also bend (a connected region)
# ======================================================
# ---- follow-up 1: function ----
# Answer: that is a different problem: the largest CONNECTED region of one
# colour, found with a flood fill (DFS or BFS) or union-find in O(m * n).
# The run-length DP only handles straight lines because each run depends
# on exactly one earlier cell.
def largest_region(g):
    seen = set()
    best = 0
    for r in range(len(g)):
        for c in range(len(g[0])):
            if (r, c) in seen:
                continue
            seen.add((r, c))
            stack = [(r, c)]
            size = 0
            while stack:
                x, y = stack.pop()
                size += 1
                for a, b in ((x + 1, y), (x - 1, y), (x, y + 1), (x, y - 1)):
                    if a < 0 or b < 0 or a >= len(g) or b >= len(g[0]):
                        continue
                    if (a, b) not in seen and g[a][b] == g[r][c]:
                        seen.add((a, b))
                        stack.append((a, b))
            if size > best:
                best = size
    return best


# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert largest_region(["RR", "GR"]) == 3
    assert largest_region(["RRR", "GGR"]) == 4              # a bent region is longer than any straight line


# ======================================================
# RUN EVERYTHING
# ======================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

201 Count Same-Color Squares in Matrix

MEDIUM

Template fit: the template as written: a square ends here if the three neighbours match; its size is 1 + min of their sizes

QUESTION

LeetCode 221 variant — count maximal same-color square sub-grids. DP on each cell as the bottom-right corner.

Input: m × n grid of colored cells.
Output: count of square sub-grids whose cells are all the same color (or maximum side length, depending on prompt).
DP: dp[i][j] = 1 + min(dp[i-1][j], dp[i][j-1], dp[i-1][j-1]) if grid[i][j] == grid[i-1][j] == grid[i][j-1] == grid[i-1][j-1] else 1.
Sum dp[i][j] across the grid for the count variant; max for the side variant.

EXAMPLES

The plain "count all-1 squares" DP generalizes to same-color directly.

FULL CODE | main question first, then every follow-up, tests at the end

```
# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
def square_sides(g):
    # side[i][j] = the side of the biggest same-colour square whose bottom-right corner is (i, j)
    sides = []
    for i in range(len(g)):
        row_sides = [1] * len(g[i])
        for j in range(len(g[i])):
            c = g[i][j]
            if i > 0 and j > 0 and g[i - 1][j] == c and g[i][j - 1] == c and g[i - 1][j - 1] == c:
                smallest = sides[i - 1][j]                # all three neighbours must match this cell
                if row_sides[j - 1] < smallest:
                    smallest = row_sides[j - 1]
                if sides[i - 1][j - 1] < smallest:
                    smallest = sides[i - 1][j - 1]
                row_sides[j] = 1 + smallest
        sides.append(row_sides)
    return sides


# ---- main question: main function ----
# Idea: a cell's square side is 1 + the smallest side of its top, left
# and top-left neighbours when all three have its colour, otherwise 1.
# Every cell is the bottom-right corner of one square per size up to its side.
def count_same_color_squares(g):
    total = 0
    for row_sides in square_sides(g):
        for side in row_sides:
            total += side
    return total


# ---- main question: tests ----
def test_main_question():
    assert count_same_color_squares(["RRR", "RRR", "RRR"]) == 14     # a 3 x 3 block: 9 + 4 + 1
    assert count_same_color_squares(["RRR", "RGR", "RRR"]) == 9      # a different colour inside blocks the b
igger squares
```

```
    assert count_same_color_squares([]) == 0
```

```python
# ============================================================
# FOLLOW-UP 1: Return only the largest side instead of the count
# ============================================================
# ---- follow-up 1: function ----
# Answer: keep the same table and take the maximum instead of adding.
# The common bug is to check only the diagonal neighbour: the cells above
# and to the left must match as well, or a different colour can hide
# inside the square.
def largest_side(g):
    best = 0
    for row_sides in square_sides(g):
        for side in row_sides:
            if side > best:
                best = side
    return best


# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert largest_side(["RRR", "RRR", "RRR"]) == 3
    assert largest_side(["RRR", "RGR", "RRR"]) == 1
    assert largest_side([]) == 0


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

202 Find the Longest Straight Run of One Color

HARD

Template fit: the same four run lengths; extend a run when the neighbour in that direction has the same colour

QUESTION

Given a rectangular grid of colors, return the length of the longest consecutive sequence of cells with the same color along a horizontal, vertical, or diagonal straight line. A sequence cannot change direction or skip a cell. Both diagonal orientations count. The length is the number of cells in the sequence.

Input

- **`grid`**: an array of strings of equal length. Each character represents one cell's color.

Output

Return the maximum length as an integer.

Constraints

and Edge Cases - For this practice version, the grid contains at least one row and one column, with at most **`1000000`** cells in total.

- Colors are uppercase English letters; equal letters mean equal colors.
- A one-cell grid has answer **`1`**.
- Diagonal lines may run down-right or down-left. Reading the same line in reverse does not change its length.
- A connected region that bends is not necessarily one valid sequence.

Example

QUESTION

1

```
grid = ["RRRG", "RRBB", "GRRR", "BGBR"] output = 4
```

The main diagonal contains four consecutive **R** cells.

Example

2

```
grid = ["RBG", "BGR", "GRB"] output = 3
```

The down-left diagonal from the top-right cell contains three **G** cells. No horizontal or vertical sequence has that length.

EXAMPLES

Example 1	
Input	`['RRRG', 'RRBB', 'GRRR', 'BGBR']`
Output	`4`
Why	`Four R cells lie on the main diagonal.`

Example 2	
Input	`['RGB', 'BGR', 'RGB']`
Output	`3`
Why	`Three G cells in the middle column.`

FULL CODE | main question first, then every follow-up, tests at the end

```python
# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ========================================================

# ========================================================
# MAIN QUESTION
# ========================================================
# ---- main question: helper functions ----
DIRECTIONS = ((0, 1), (1, 0), (1, 1), (1, -1))        # right, down, down-right, down-left


def all_runs(grid):
    # every maximal straight run of one colour as (length, start cell, direction).
    # A run is walked only from its FIRST cell (the cell before it in that
    # direction has another colour or does not exist), so the total work is
    # linear in the number of cells.
    rows = len(grid)
    cols = len(grid[0])
    found = []
    for r in range(rows):
        for c in range(cols):
            for dr, dc in DIRECTIONS:
                before_r = r - dr
                before_c = c - dc
                if 0 <= before_r < rows and 0 <= before_c < cols:
                    if grid[before_r][before_c] == grid[r][c]:
                        continue                        # not the first cell of its run
                length = 0
                a = r
                b = c
                while 0 <= a < rows and 0 <= b < cols and grid[a][b] == grid[r][c]:
```

```
                    length += 1
                    a += dr
                    b += dc
                found.append((length, (r, c), (dr, dc)))
    return found


# ---- main question: main function ----
# Idea: look for the longest straight line (across, down, or along
# either diagonal) made of one colour. Walk each run once from its first
# cell and keep the longest.
```

```
def longest_run(grid):
    if not grid or not grid[0]:
        return 0
    best = 0
    for length, start, direction in all_runs(grid):
        if length > best:
            best = length
    return best


# ---- main question: tests ----
GRID = ["RRRG", "RRBB", "GRRR", "BGBR"]


def test_main_question():
    assert longest_run(GRID) == 4
    assert longest_run(["RBG", "BGR", "GRB"]) == 3
    assert longest_run(["R"]) == 1
    assert longest_run([]) == 0


# ======================================================
# FOLLOW-UP 1: Return where the run is, not only its length
# ======================================================
# ---- follow-up 1: function ----
# Answer: all_runs already starts a walk only at a cell that begins a run,
# which keeps the work linear; just keep the start cell and direction of
# the best run.
def longest_run_where(grid):
    if not grid or not grid[0]:
        return 0, None, None
    best = None
    for run in all_runs(grid):
        if best is None or run[0] > best[0]:
            best = run
    return best


# ---- follow-up 1: tests ----
def test_follow_up_1():
    length, start, direction = longest_run_where(GRID)
    assert length == 4
    r, c = start
    for i in range(length):                          # walk the run and check the colour
        assert GRID[r + i * direction[0]][c + i * direction[1]] == GRID[r][c]
```

```


# ======================================================
# FOLLOW-UP 2: Require the run to be at least K and count how many such runs exist
# ======================================================
# ---- follow-up 2: function ----
# Answer: count a run ONCE, at its first cell, when its length is at
# least K (all_runs already lists each run once). Counting at every cell
```

```
# would count a long run many times.
def count_runs_at_least(grid, k):
    if not grid or not grid[0]:
        return 0
    count = 0
    for length, start, direction in all_runs(grid):
        if length >= k:
            count += 1
    return count


# ---- follow-up 2: tests ----
def test_follow_up_2():
    assert count_runs_at_least(GRID, 3) == 4         # runs of length 3 or more are counted once each


# ============================================================
# FOLLOW-UP 3: The grid is huge and arrives row by row
# ============================================================
# ---- follow-up 3: function ----
# Answer: the run length ending at a cell depends only on the PREVIOUS
# row, so keep just the run lengths of the last row (across, down,
# down-right, down-left): O(width) memory for any number of rows.
def longest_run_rows(row_stream):
    best = 0
    previous = None
    down = []
    diagonal_right = []
    diagonal_left = []
    for row in row_stream:
        n = len(row)
        across_now = [1] * n
        down_now = [1] * n
        right_now = [1] * n
        left_now = [1] * n
        for c in range(n):
            if c > 0 and row[c] == row[c - 1]:
                across_now[c] = across_now[c - 1] + 1
            if previous is not None:
                if row[c] == previous[c]:
                    down_now[c] = down[c] + 1
                if c > 0 and row[c] == previous[c - 1]:
                    right_now[c] = diagonal_right[c - 1] + 1
                if c + 1 < n and row[c] == previous[c + 1]:
                    left_now[c] = diagonal_left[c + 1] + 1
            for length in (across_now[c], down_now[c], right_now[c], left_now[c]):
                if length > best:
                    best = length
        previous = row
        down = down_now
        diagonal_right = right_now
        diagonal_left = left_now
    return best


# ---- follow-up 3: tests ----
def test_follow_up_3():
    assert longest_run_rows(iter(GRID)) == 4
    assert longest_run_rows(["RBG", "BGR", "GRB"]) == 3


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
```

```
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

203 Count same-color squares in a character grid

HARD

Template fit: exactly the same recurrence on characters

QUESTION

You are given a 2D grid (matrix) of characters. Each character represents a color: cells with the same character are considered the same color. Formally:

- The grid has **`m`** rows and **`n`** columns.
- **`grid[i][j]`** is an alphabetic character (e.g., **`'a'`**–**`'z'`** or **`'A'`**–**`'Z'`**). A **square** in this grid is defined as a contiguous **`k × k`** submatrix aligned with the grid axes, for some integer **`k >= 1`**. A square is **monochromatic** if **all** of its cells contain the **same** character (i.e., the same color). Task:
- Count and return the total number of monochromatic squares in the grid.
- Count all sizes of squares (1×1, 2×2, ..., up to the largest possible that fits in the grid). Example (just for clarity, not necessarily exhaustive): If the grid is:

```
a a b a a b b b b
```

Then some monochromatic squares include:

- All 1×1 cells (each single cell is a monochromatic square).
- The 2×2 square in the top-left corner formed by **`'a'`**. Your function should return the total count of such monochromatic squares for a given input grid.

You are given a 2D grid of characters, where each character represents a color. A square is a contiguous k × k submatrix aligned with the grid axes, for some k >= 1. A square is monochromatic if all of its cells contain the same character. Return the total number of monochromatic squares of all possible sizes and positions in the grid. Every 1 × 1 cell is considered a monochromatic square.
Constraints
0 <= len(grid) <= 1000If grid is non-empty, 0 <= len(grid[0]) <= 1000All rows in grid have the same lengthgrid[i][j] is an alphabetic characterThe total number of cells is at most 1,000,000

EXAMPLES

```
Input: ([])
Expected Output: 0
```

Explanation: An empty grid contains no squares.

```
Input: (["aab", "aab", "bbb"])
Expected Output: 10
```

Explanation: There are 9 single-cell squares and one 2x2 monochromatic square of 'a' in the top-left corner.

Hints

Try defining dp[i][j] as the side length of the largest monochromatic square whose bottom-right corner is cell (i, j).A square larger than 1 can end at (i, j) only if the current cell matches its top, left, and top-left neighboring cells.

FULL CODE | main question first, then every follow-up, tests at the end

```
# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ========================================================

# ========================================================
# MAIN QUESTION
# ========================================================
# ---- main question: helper functions ----
def square_table(grid):
    # side[i][j] = the side of the biggest same-colour square whose
    # BOTTOM-RIGHT corner is (i, j)
    sides = []
    for i in range(len(grid)):
        row_sides = [1] * len(grid[i])
        for j in range(len(grid[i])):
            ch = grid[i][j]
            if i > 0 and j > 0 and grid[i - 1][j] == ch and grid[i][j - 1] == ch and grid[i - 1][j - 1] == ch
:
                # the cells above, to the left and diagonally must match too
                smallest = sides[i - 1][j]
                if row_sides[j - 1] < smallest:
                    smallest = row_sides[j - 1]
                if sides[i - 1][j - 1] < smallest:
                    smallest = sides[i - 1][j - 1]
                row_sides[j] = 1 + smallest
        sides.append(row_sides)
    return sides


# ---- main question: main function ----
# Idea: if a cell matches its top, left and top-left neighbours its square
# side is 1 + the smallest of their sides; otherwise it is 1. A cell with
# side s is the corner of s squares (sizes 1..s), so adding up all the
# sides counts every square exactly once, at its bottom-right corner.
def count_same_color_squares(grid):
    total = 0
    for row_sides in square_table(grid):
        for side in row_sides:
            total += side
    return total


# ---- main question: tests ----
```

```
def test_main_question():
    assert count_same_color_squares(["aab", "aab", "bbb"]) == 10      # 9 single cells plus the 2 x 2 block o
f a's
    assert count_same_color_squares(["zzz", "zzz", "zzz"]) == 14      # 9 + 4 + 1
    assert count_same_color_squares(["a"]) == 1


# ========================================================
# FOLLOW-UP 1: Only the largest square (LeetCode 221)
# ========================================================
# ---- follow-up 1: function ----
# Answer: keep the biggest number of the same table instead of the sum.
def largest_square(grid):
    best = 0
    for row_sides in square_table(grid):
        for side in row_sides:
            if side > best:
                best = side
    return best
```

```
# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert largest_square(["aab", "aab", "bbb"]) == 2


# ========================================================
# FOLLOW-UP 2: Allow at most one different cell inside a square
# ========================================================
# Answer: keep two tables per cell: the biggest square with no mistake and
# the biggest with one mistake (a different cell inside); the second one
# can also grow from a neighbour's "no mistake" square when this cell is
# the one that differs.


# ========================================================
# FOLLOW-UP 3: Count only squares of colour c, or only side >= 2
# ========================================================
# ---- follow-up 3: function ----
# Answer: filter while adding up: count only cells of colour c, and for
# squares of side at least m add side - m + 1 per cell (the sizes m..side).
def count_of_color(grid, color, min_side=1):
    total = 0
    sides = square_table(grid)
    for i in range(len(grid)):
        for j in range(len(grid[i])):
            if grid[i][j] == color and sides[i][j] >= min_side:
                total += sides[i][j] - min_side + 1
    return total


# ---- follow-up 3: tests ----
def test_follow_up_3():
    g = ["aab", "aab", "bbb"]
    assert count_of_color(g, "a") == 5
    assert count_of_color(g, "b") == 5
    full = ["zzz", "zzz", "zzz"]
    assert count_of_color(full, "z", 2) == 5                # 4 squares of side 2 and 1 of side 3
    assert count_of_color(full, "q") == 0


# ========================================================
# RUN EVERYTHING
# ========================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

GROUP 27 OF 43 | 5 QUESTIONS, ONE CODE TEMPLATE | 5 M

Nested structures: a stack of contexts, or recursion with a cache

204 Decode String — MEDIUM

Template fit: the template as written

QUESTION

An encoded string uses k[text] to mean that text is repeated k times; the brackets can be nested. Return the decoded string. (k is a positive integer, possibly with several digits; the input is always valid.)

Function signature

def decode(s: str) -> str

EXAMPLES

Example 1	
Input	`'3[a]2[bc]'`
Output	`'aaabcbc'`
Why	`3 copies of a, then 2 copies of bc.`

Example 2	
Input	`'2[abc]3[cd]ef'`
Output	`'abcabccdcdcdef'`
Why	`Plain letters after the brackets are kept as they are.`

Example 3	
Input	`'3[a2[c]]'`
Output	`'accaccacc'`
Why	`Nested: the inner 2[c] first (cc), then 3 copies of acc.`

FULL CODE | main question first, then every follow-up, tests at the end

```
# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: main function ----
# Idea: a stack of "frames", one for each "[" that is still open. On "["
# push the text built so far together with the repeat count just read, and
# start a fresh text for the inside. On "]" pop the frame and set
# text = saved text + (inside text repeated count times).
def decode(s):
    stack = []
    current = ""
    number = 0
    for ch in s:
        if ch.isdigit():
            number = number * 10 + int(ch)
        elif ch == "[":
            stack.append((current, number))
```

```
            current = ""
            number = 0
        elif ch == "]":
            saved, times = stack.pop()
            current = saved + current * times
        else:
            current += ch
    return current


# ---- main question: tests ----
def test_main_question():
    assert decode("3[a]2[bc]") == "aaabcbc"
    assert decode("2[abc]3[cd]ef") == "abcabccdcdcdef"
    assert decode("3[a2[c]]") == "accaccacc"
    assert decode("") == ""
    assert decode("abc") == "abc"


# ======================================================
# FOLLOW-UP 1: How would you do it recursively, and what is the risk?
```

```
# ======================================================
# ---- follow-up 1: function ----
# Answer: let a function read text up to the matching "]" and call itself
# when it meets a "[". It is shorter to write, but the nesting depth is
# the recursion depth, so very deep input can hit the recursion limit.
def decode_recursive(s):
    def read(i):
        # read from position i until a "]" or the end; return (text, position)
        text = ""
        number = 0
        while i < len(s) and s[i] != "]":
            ch = s[i]
            if ch.isdigit():
                number = number * 10 + int(ch)
                i += 1
            elif ch == "[":
                inner, i = read(i + 1)
                text += inner * number
                number = 0
                i += 1                              # skip the "]"
            else:
                text += ch
                i += 1
        return text, i

    return read(0)[0]


# ---- follow-up 1: tests ----
def test_follow_up_1():
    for sample in ("3[a]2[bc]", "2[abc]3[cd]ef", "3[a2[c]]", "", "10[x]"):
        assert decode_recursive(sample) == decode(sample)


# ======================================================
# FOLLOW-UP 2: What if the decoded string is enormous?
# ======================================================


# ======================================================
# FOLLOW-UP 3: The decoded string would be enormous
# ======================================================
# ---- follow-up 3: helper functions ----
def parse(s):
```

```
    # the nested form: a list whose items are single characters or
    # (repeat count, list of items). Bad input raises ValueError (follow-up 5).
    stack = []
    current = []
    number = 0
    for ch in s:
        if ch.isdigit():
            number = number * 10 + int(ch)
        elif ch == "[":
            if number == 0:
                raise ValueError("[ needs a count")
            stack.append((current, number))
            current = []
            number = 0
        elif ch == "]":
            if len(stack) == 0:
                raise ValueError("unmatched ]")
            saved, times = stack.pop()
            saved.append((times, current))
            current = saved
        else:
            if number != 0:
                raise ValueError("count without [")
            current.append(ch)
    if len(stack) > 0:
        raise ValueError("missing ]")
    return current


# ---- follow-up 3: function ----
# Answer: compute the LENGTH from the nested form without building the
# string: a character counts 1, and a repeat counts (times * length of
# the inside). The length can be found for strings that could never fit
# in memory.
def length(items):
    total = 0
    for item in items:
        if isinstance(item, str):
            total += 1
        else:
            total += item[0] * length(item[1])
    return total


# ---- follow-up 3: tests ----
def test_follow_up_3():
    assert length(parse("3[a2[c]]")) == 9
    big = parse("999[999[999[ab]]]")
    assert length(big) == 2 * 999 ** 3                  # measured, never built


# ========================================================
# FOLLOW-UP 4: Return the character at position k only
# ========================================================
# ---- follow-up 4: function ----
# Answer: walk the nested form with the lengths. For a repeat, if k falls
# inside it, which copy does not matter: look at position k modulo the
# length of the inside (LeetCode 880 style).
def char_at(items, k):
    for item in items:
        if isinstance(item, str):
            if k == 0:
                return item
            k -= 1
```

```
        else:
            times, inner = item
            inner_length = length(inner)
            if k < times * inner_length:
                return char_at(inner, k % inner_length)
            k -= times * inner_length
    return None


# ---- follow-up 4: tests ----
def test_follow_up_4():
    items = parse("3[a2[c]]")
    letters = []
    for i in range(9):
        letters.append(char_at(items, i))
    assert "".join(letters) == decode("3[a2[c]]")
    assert char_at(items, 9) is None


# ============================================================
# FOLLOW-UP 5: Invalid input (missing bracket, count with no bracket)
# ============================================================
# ---- follow-up 5: tests ----
# Answer: parse() above checks while reading: "3[a" (missing "]"), "a]"
# (unmatched "]"), "[a]" (no count) and "3a" (a count with no "[") all
# raise ValueError.
def test_follow_up_5():
```

```
    for bad in ("3[a", "a]", "[a]", "3a"):
        try:
            parse(bad)
            raise AssertionError(bad)
        except ValueError:
            pass


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_3()
    test_follow_up_4()
    test_follow_up_5()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

205 Template Variable Expansion with Follow-Ups

MEDIUM

Template fit: resolve(name) with a cache and a 'being resolved' set; meeting a name that is being resolved is a cycle

QUESTION

Replace **%KEY%** placeholders using a key-value map whose values may contain other placeholders. Clarify malformed delimiters and literal-percent escaping, flatten nested values, and detect cyclic dependencies before expansion.

QUESTION

Input: a key-value map and a string containing placeholders delimited by %, such as %USER%. Replace every placeholder with the corresponding value.
Map values can themselves contain placeholders, so resolve or flatten dependencies before expanding the final string. Define behavior for malformed delimiters such as unmatched percent signs, for a literal % in output, and for nested-looking forms before coding; the prompt intentionally leaves these contracts open.
Follow-up: detect cyclic dependencies among map values. Model keys as a dependency graph and use indegrees plus a queue to determine whether a complete topological resolution is possible.

EXAMPLES

```
Given {USER: "admin", HOME: "/%USER%/home"}:
```

Input: I am %USER% My home is %HOME% Output: I am admin My home is /admin/home

FULL CODE | main question first, then every follow-up, tests at the end

```python
# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ========================================================

from collections import deque


# ========================================================
# MAIN QUESTION
# ========================================================
# ---- main question: helper functions ----
def tokens(s):
    # split the text into ("text", character) and ("var", name) pieces:
    # "%%" is one literal percent sign, "%NAME%" is a variable
    pieces = []
    i = 0
    while i < len(s):
        if s[i] != "%":
            pieces.append(("text", s[i]))
            i += 1
        elif s[i:i + 2] == "%%":
            pieces.append(("text", "%"))
            i += 2
        else:
            end = s.find("%", i + 1)
            if end < 0:
                raise ValueError("unmatched %")
            pieces.append(("var", s[i + 1:end]))
            i = end + 1
    return pieces


# ---- main question: main function ----
# Idea: scan the text once: "%%" becomes "%", "%NAME%" becomes the
# resolved value of NAME. A value may itself hold placeholders, so resolve
# it with the same scan first, remember the result (memo), and keep a
# "visiting" set to catch a name that needs itself.
def expand_template(text, mapping):
    done = {}
    visiting = set()

    def resolve(name):
```

```
        if name in done:
            return done[name]
```

```
        if name in visiting:
            raise ValueError("cycle at " + name)
        if name not in mapping:
            raise KeyError(name)
        visiting.add(name)
        done[name] = scan(mapping[name])
        visiting.discard(name)
        return done[name]

    def scan(s):
        result = ""
        for kind, piece in tokens(s):
            if kind == "var":
                result += resolve(piece)
            else:
                result += piece
        return result

    return scan(text)


# ---- main question: tests ----
MAPPING = {"USER": "admin", "HOME": "/%USER%/home"}


def test_main_question():
    assert expand_template("I am %USER% My home is %HOME%", MAPPING) == "I am admin My home is /admin/home"
    assert expand_template("100%% sure", {}) == "100% sure"


# ========================================================
# FOLLOW-UP 1: Detect cycles before expanding
# ========================================================
# ---- follow-up 1: function ----
# Answer: model every key as a vertex with an edge from each key its value
# mentions; count how many unresolved keys each key waits for and run
# Kahn's algorithm (take keys that wait for nothing, lower the counts of
# the keys that needed them). If fewer vertices than keys come out, the
# rest sit on a cycle and cannot be resolved.
def has_cycle(mapping):
    waits_for = {}                                  # key -> the keys it mentions (that exist)
    for key in mapping:
        mentioned = set()
```

```
        for kind, piece in tokens(mapping[key]):
            if kind == "var" and piece in mapping:
                mentioned.add(piece)
        waits_for[key] = mentioned
    users = {}                                      # key -> the keys whose values mention it
    for key in mapping:
        users[key] = []
    for key in waits_for:
        for other in waits_for[key]:
            users[other].append(key)
    unmet = {}
    queue = deque()
    for key in waits_for:
        unmet[key] = len(waits_for[key])
        if unmet[key] == 0:
            queue.append(key)
    resolved = 0
    while queue:
        key = queue.popleft()
        resolved += 1
        for user in users[key]:
            unmet[user] -= 1
```

```
            if unmet[user] == 0:
                queue.append(user)
    return resolved != len(mapping)                  # keys left over sit on a cycle


# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert not has_cycle(MAPPING)
    loop = {"A": "%B%", "B": "%A%"}
    assert has_cycle(loop)
    assert has_cycle({"A": "x %A% y"})               # a key that needs itself
    for bad, error in (("a % b", ValueError), ("%NOPE%", KeyError)):
        try:
            expand_template(bad, {})
            raise AssertionError("should have failed")
        except error:
            pass
    try:
        expand_template("%A%", loop)
        raise AssertionError("cycle not caught")
    except ValueError:
        pass
```

```
# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

206 Expand Recursive Key-Value Placeholders in a Template

MEDIUM

Template fit: regex-find {key}, expand each key recursively with a cache and a visiting set to stop cycles

QUESTION

Expand %KEY% placeholders in a template. A placeholder is replaced by its value from the mapping, and placeholders inside the values are expanded too, recursively. An unknown key is left exactly as written. A cycle (A needs B needs A) raises an error.

Function signature

def expand_template(template: str, mapping: dict) -> str

EXAMPLES

Example 1	
Input	`'%GREETING%, welcome!', {'NAME': 'Ada', 'GREETING': 'Hello %NAME%'}`
Output	`'Hello Ada, welcome!'`
Why	`%GREETING% expands to 'Hello %NAME%', and %NAME% is expanded in turn.`

Example 2	
Input	`'Hi %WHO%', {}`
Output	`'Hi %WHO%'`
Why	`An unknown key is kept as written.`

QUESTION

Example 3	
Input	`'%A%', {'A': '%B%', 'B': '%A%'}`
Output	`raises ValueError: cycle at A`
Why	`A needs B and B needs A: a cycle is reported instead of looping forever.`

FULL CODE | main question first, then every follow-up, tests at the end

```
# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ========================================================

# ========================================================
# MAIN QUESTION
# ========================================================
# ---- main question: helper functions ----
def is_word(name):
    # a key is made of letters, digits and underscores (and is not empty)
    if name == "":
        return False
    for ch in name:
        if not (ch.isalnum() or ch == "_"):
            return False
    return True


# ---- main question: main function ----
# Idea: expand(key) replaces every placeholder in the key's value by the
# expansion of the key it names, remembers each result (a cache, so a
# key used many times is expanded once), and keeps a "visiting" set so
# that a key that needs itself (a cycle) raises an error. Unknown keys
# stay as written (follow-up 2 can also empty them or raise an error).
# "%%" is a literal percent sign (follow-up 1): output one "%" and skip
# both characters. `limit` stops an expansion that explodes (follow-up 3).
def expand_template(template, mapping, unknown="keep", limit=10 ** 6):
    cache = {}
    visiting = set()

    def expand_text(text):
        result = ""
        i = 0
        while i < len(text):
            if text[i] != "%":
                result += text[i]
                i += 1
            elif text[i + 1:i + 2] == "%":
                result += "%"                          # "%%" is a literal percent sign
                i += 2
            else:
                end = text.find("%", i + 1)
```

```
                name = ""
                if end != -1:
                    name = text[i + 1:end]
                if not is_word(name):
                    result += "%"                      # not a placeholder: keep the percent sign
                    i += 1
                elif name in mapping:
                    result += expand_key(name)
                    i = end + 1
```

```
                else:
                    if unknown == "error":
                        raise KeyError(name)
                    if unknown == "keep":
                        result += text[i:end + 1]      # leave it exactly as written
                    i = end + 1                        # (unknown == "empty": it simply disappears)
        if len(result) > limit:
            raise ValueError("expansion too large")
        return result

    def expand_key(key):
        if key in cache:
            return cache[key]
        if key in visiting:
            raise ValueError("cycle at " + key)
        visiting.add(key)
        cache[key] = expand_text(mapping[key])
        visiting.discard(key)
        return cache[key]

    return expand_text(template)


# ---- main question: tests ----
MAPPING = {"USER": "admin", "HOME": "/%USER%/home"}


def test_main_question():
    assert expand_template("I am %USER% My home is %HOME%", MAPPING) == "I am admin My home is /admin/home"
    assert expand_template("a %X% b", MAPPING) == "a %X% b"           # an unknown key stays as written
    assert expand_template("Hi %WHO%", {}) == "Hi %WHO%"
    try:
        expand_template("%A%", {"A": "%B%", "B": "%A%"})              # a cycle is reported
        raise AssertionError("cycle not caught")
    except ValueError:
        pass
```

```


# ========================================================
# FOLLOW-UP 1: A literal percent sign must be possible
# ========================================================
# ---- follow-up 1: tests ----
# Answer: choose an escape such as "%%" and handle it in the same scan:
# when the text has "%%" output one "%" and skip both characters,
# otherwise try to read a placeholder. Test percent signs next to
# placeholders.
def test_follow_up_1():
    assert expand_template("100%% of %USER%", MAPPING) == "100% of admin"
    assert expand_template("%%USER%%", MAPPING) == "%USER%"
    assert expand_template("50%", MAPPING) == "50%"


# ========================================================
# FOLLOW-UP 2: An unknown key should be an error, or replaced by an empty string
# ========================================================
# ---- follow-up 2: tests ----
# Answer: a parameter `unknown`: "keep" (leave the text), "empty" (replace
# by nothing) or "error" (raise KeyError). Only the branch for an unknown
# key changes.
def test_follow_up_2():
    assert expand_template("a %X% b", MAPPING, unknown="empty") == "a  b"
    try:
        expand_template("%X%", MAPPING, unknown="error")
        raise AssertionError("missing key not reported")
    except KeyError:
```

```
        pass


# ======================================================
# FOLLOW-UP 3: Values are huge and expansion explodes
# ======================================================
# ---- follow-up 3: tests ----
# Answer: cap the size of every expansion (the `limit` parameter) and stop
# with an error as soon as it is exceeded. Here every key doubles the
# previous one, so the full text would have 2^30 characters.
def test_follow_up_3():
    boom = {"K0": "ab"}
    for i in range(1, 30):
        boom["K" + str(i)] = "%K" + str(i - 1) + "%%K" + str(i - 1) + "%"
    try:
        expand_template("%K29%", boom, limit=1000)
```

```
        raise AssertionError("explosion not stopped")
    except ValueError:
        pass


# ======================================================
# RUN EVERYTHING
# ======================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

207 Nested List Weight Sum (LC 339) with Follow-Ups

MEDIUM

*Template fit: depth counter: '[' adds 1, ']' subtracts 1; each number adds value * depth (read digits and a minus sign as you go)*

QUESTION

A nested list holds integers and further lists. The weight of an integer is its depth: 1 at the top level, 2 inside one inner list, 3 inside two, and so on. Return the sum of every integer times its depth. A second version reads the nested list from a string like '[1,[4,[6]]]'.

Function signature

def weight_sum(nested: list) -> int
def weight_sum_string(s: str) -> int

EXAMPLES

Example 1	
Input	`[3, 8, [2, 14], [2, [91]]]`
Output	`320`
Why	`3 + 8 at depth 1, 2 + 14 and 2 at depth 2, and 91 at depth 3: 11 + 32 + 4 + 273 = 320.`

QUESTION

Example 2	
Input	'[1,[4,[6]]]'
Output	27
Why	The string version of [1, [4, [6]]]: 1*1 + 4*2 + 6*3 = 27.

FULL CODE | main question first, then every follow-up, tests at the end

```
# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ========================================================

# ========================================================
# MAIN QUESTION
# ========================================================
# ---- main question: helper functions ----
def with_depths(nested):
    # every integer together with its depth (an integer in the top-level list
    # has depth 1). A stack of (item, depth) is used instead of recursion, so
    # very deep input cannot overflow the call stack.
    pairs = []
    stack = [(nested, 0)]
    while stack:
        item, depth = stack.pop()
        if isinstance(item, list):
            for child in item:
                stack.append((child, depth + 1))
        else:
            pairs.append((item, depth))
    return pairs


# ---- main question: main functions ----
# Idea (LeetCode 339): every integer counts as value * depth. Collect the
# integers with their depths and add value * depth up.
def weight_sum(nested):
    total = 0
    for value, depth in with_depths(nested):
        total += value * depth
    return total


# The same sum straight from the TEXT of the nested list, "[1,[4,[6]]]",
# in one pass with O(1) extra space. An integer is weighted by the number
# of "[" that are currently open (the depth). Digits are collected into a
# number, which is added when a non-digit character ends it. The extra "]"
# at the end flushes a number that finishes the text.
def weight_sum_string(s):
    depth = 0
    total = 0
```

```
    number = 0
    sign = 1
    reading = False
    for ch in s + "]":
        if ch.isdigit():
            number = number * 10 + int(ch)
            reading = True
            continue
        if reading:
```

```
            total += sign * number * depth
            number = 0
            sign = 1
            reading = False
        if ch == "[":
            depth += 1
        elif ch == "]":
            depth -= 1
        elif ch == "-":
            sign = -1
    return total


# ---- main question: tests ----
def test_main_question():
    assert weight_sum([3, 8, [2, 14], [2, [91]]]) == 320          # 3 + 8 + 2 * (2 + 14) + 2 * 2 + 3 * 91
    assert weight_sum([]) == 0
    assert weight_sum_string("[1,[4,[6]]]") == 27
    assert weight_sum_string("[-1,[2]]") == 3
    assert weight_sum_string("[[]]") == 0


# ========================================================
# FOLLOW-UP 1: When does the recursive (DFS) version become unsafe?
# ========================================================
# ---- follow-up 1: function ----
# Answer: when the nesting is very deep (thousands of levels): every level
# is one function call, and Python stops at about 1000 with a
# RecursionError. The stack-based with_depths above has no such limit.
def weight_sum_recursive(nested, depth=1):
    total = 0
    for item in nested:
        if isinstance(item, list):
            total += weight_sum_recursive(item, depth + 1)
        else:
            total += item * depth
    return total


# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert weight_sum_recursive([3, 8, [2, 14], [2, [91]]]) == 320
    deep = [1]
    for i in range(4999):
        deep = [deep]                                  # 5000 levels deep
    assert weight_sum(deep) == 5000
    try:
        weight_sum_recursive(deep)
        raise AssertionError("the recursive version should run out of stack")
    except RecursionError:
        pass


# ========================================================
# FOLLOW-UP 2: What is the inverse version (deeper integers weigh less)?
# ========================================================


# ========================================================
# FOLLOW-UP 3: The inverse version (shallow integers weigh less, LeetCode 364)
# ========================================================
# ---- follow-up 2: function ----
# Answer (LeetCode 364): the deepest integers weigh 1 and the top-level
# ones weigh the most: weight = deepest depth + 1 - depth. Find the
# deepest depth first, then add value * weight. (A level-by-level pass that
```

```
# adds a running sum once per level gives the same number.)
def inverse_weight_sum(nested):
    pairs = with_depths(nested)
    if len(pairs) == 0:
        return 0
    deepest = 0
    for value, depth in pairs:
        if depth > deepest:
            deepest = depth
    total = 0
    for value, depth in pairs:
        total += value * (deepest + 1 - depth)
    return total


# ---- follow-up 2: tests ----
```

```
def test_follow_up_2():
    assert inverse_weight_sum([[1, 1], 2, [1, 1]]) == 8           # LeetCode 364 example
    assert inverse_weight_sum([1, [4, [6]]]) == 17
    assert inverse_weight_sum([]) == 0


# ========================================================
# RUN EVERYTHING
# ========================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

208 Nested Function Expression Evaluator

MEDIUM

Template fit: tokenise, then a recursive parse(i): a number returns itself; a name reads its arguments until ')' and applies the function

QUESTION

Evaluate an expression made of integers and nested function calls: add and mul (any number of arguments), sub and pow (two), div (integer division truncating toward zero), min and max (any number). Return the integer result.

Function signature

def evaluate(expr: str) -> int

EXAMPLES

Example 1	
Input	`'add(1, sub(1, 0))'`
Output	`2`
Why	`sub(1, 0) is 1, so add(1, 1) = 2.`

Example 2	
Input	`'add(2, mul(3, pow(4, 5)))'`
Output	`3074`
Why	`pow(4, 5) = 1024, times 3 = 3072, plus 2 = 3074.`

QUESTION

Example 3	
Input	`'max(3, div(-7, 2), min(5, 9))'`
Output	`5`
Why	`div(-7, 2) truncates toward zero to -3; max of 3, -3 and 5 is 5.`

FULL CODE | main question first, then every follow-up, tests at the end

```
# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ========================================================

# ========================================================
# MAIN QUESTION
# ========================================================
# ---- main question: helper functions ----
# The expression looks like  add(1, mul(2, x))  : a function name, then
# arguments (numbers, variables, or more calls) in brackets, separated by commas.

def add_all(*numbers):
    total = 0
    for x in numbers:
        total += x
    return total


def multiply_all(*numbers):
    total = 1
    for x in numbers:
        total *= x
    return total


def smallest(*numbers):
    best = numbers[0]
    for x in numbers:
        if x < best:
            best = x
    return best


def largest(*numbers):
    best = numbers[0]
    for x in numbers:
        if x > best:
            best = x
    return best


def subtract(a, b):
```

```
    return a - b


def power(a, b):
    return a ** b


def divide(a, b):
    # integer division that TRUNCATES toward zero (follow-up 3 explains why)
```

```
    if b == 0:
        raise ZeroDivisionError("division by zero")
    return int(a / b)


FUNCTIONS = {"add": add_all, "sub": subtract, "mul": multiply_all, "pow": power,
             "div": divide, "min": smallest, "max": largest}


def tokenize(expr):
    # split the text into numbers (a "-" right before digits is a sign),
    # names, and the characters ( ) ,
    tokens = []
    i = 0
    while i < len(expr):
        ch = expr[i]
        if ch == " ":
            i += 1
        elif ch in "(),":
            tokens.append(ch)
            i += 1
        elif ch.isdigit() or (ch == "-" and i + 1 < len(expr) and expr[i + 1].isdigit()):
            j = i + 1
            while j < len(expr) and expr[j].isdigit():
                j += 1
            tokens.append(expr[i:j])
            i = j
        elif ch.isalpha() or ch == "_":
            j = i + 1
            while j < len(expr) and (expr[j].isalnum() or expr[j] == "_"):
                j += 1
            tokens.append(expr[i:j])
            i = j
        else:
            raise ValueError("unexpected character " + ch)
    return tokens
```

```
def is_number(token):
    return token.lstrip("-").isdigit()


# ---- main question: main function ----
# Idea: tokenize, then "recursive descent": parse(i) reads ONE value that
# starts at token i and returns (value, index after it). A number is
# itself; a name not followed by "(" is a variable (looked up in env,
# follow-up 4); a name followed by "(" is a call: read the arguments one
# by one (each is again a value) until ")", then apply the function.
def evaluate(expr, env=None):
    if env is None:
        env = {}
    tokens = tokenize(expr)

    def parse(i):
        token = tokens[i]
        if is_number(token):
            return int(token), i + 1
        if i + 1 >= len(tokens) or tokens[i + 1] != "(":
            if token not in env:
                raise ValueError("undefined " + token)
            return env[token], i + 1
        if token not in FUNCTIONS:
            raise ValueError("unknown function " + token)
        i += 2                                      # skip the name and "("
        args = []
```

```
        while tokens[i] != ")":
            value, i = parse(i)
            args.append(value)
            if tokens[i] == ",":
                i += 1
        return FUNCTIONS[token](*args), i + 1

    value, end = parse(0)
    if end != len(tokens):
        raise ValueError("trailing tokens")
    return value


# ---- main question: tests ----
def test_main_question():
    assert evaluate("add(1, sub(1, 0))") == 2
```

```
    assert evaluate("add(2, mul(3, pow(4, 5)))") == 3074
    assert evaluate("add(1, 2, 3, 4)") == 10                  # functions take any number of arguments
    assert evaluate("max(3, min(9, 4), 1)") == 4
    assert evaluate("add(x, mul(x, 2))", {"x": 5}) == 15      # variables come from an environment
    assert evaluate("div(-7, 2)") == -3                       # division truncates toward zero
    for bad in ("foo(1)", "add(1", "add(1) 2", "y"):
        try:
            evaluate(bad)
            raise AssertionError(bad)
        except (ValueError, IndexError):
            pass


# ========================================================
# FOLLOW-UP 1: What if the nesting is very deep?
# ========================================================
# Answer: the recursive version uses one Python call per level and stops at
# about 1000 levels (RecursionError). Use the iterative version below.


# ========================================================
# FOLLOW-UP 2: How do you make it iterative?
# ========================================================
# ---- follow-up 2: function ----
# Answer: use two stacks: one with the names of the calls that are still
# open, and one with the lists of arguments evaluated so far. A number is
# added to the top argument list; "(" opens a new list for the name just
# pushed; ")" applies the function to the top list and adds the result to
# the list below it.
def evaluate_iterative(expr):
    names = []
    arg_lists = [[]]
    for token in tokenize(expr):
        if is_number(token):
            arg_lists[-1].append(int(token))
        elif token == "(":
            arg_lists.append([])
        elif token == ")":
            result = FUNCTIONS[names.pop()](*arg_lists.pop())
            arg_lists[-1].append(result)
        elif token != ",":
            names.append(token)
    return arg_lists[0][0]
```

```
# ---- follow-up 2: tests ----
def test_follow_up_2():
    assert evaluate_iterative("add(2, mul(3, pow(4, 5)))") == 3074
    deep = "add(1, " * 3000 + "0" + ")" * 3000            # 3000 levels deep
```

```
    assert evaluate_iterative(deep) == 3000
    try:
        evaluate(deep)
        raise AssertionError("the recursive version should run out of stack")
    except RecursionError:
        pass


# =====================================================
# FOLLOW-UP 3: What about division?
# =====================================================
# Answer: state the contract: integers or floats, TRUNCATE toward zero or
# floor (Python's // floors, which differs for negative numbers), and what
# happens on division by zero. divide() above truncates toward zero and
# raises ZeroDivisionError.
# ---- follow-up 3: tests ----
def test_follow_up_3():
    assert divide(7, 2) == 3
    assert divide(-7, 2) == -3                 # 7 // 2 would give -4 for -7 // 2
    try:
        divide(1, 0)
        raise AssertionError("division by zero")
    except ZeroDivisionError:
        pass


# =====================================================
# FOLLOW-UP 4: Variables
# =====================================================
# Answer: already supported: a name that is not followed by "(" is looked
# up in `env`, and an unknown variable is an error.


# =====================================================
# RUN EVERYTHING
# =====================================================
def run_all_tests():
    test_main_question()
    test_follow_up_2()
    test_follow_up_3()
    print("all tests passed")
```

```
if __name__ == "__main__":
    run_all_tests()
```

GROUP 28 OF 43 | 5 QUESTIONS, ONE CODE TEMPLATE | 5 M

Frontier search (BFS or DFS), Tier 2: free steps, early exit, neighbours worked out on the fly

209 Shortest Path Nodes and Escaping a Pursuer

MEDIUM

Template fit: run it from a, from t and from b; a node is on a shortest a-to-t path when da + dt == da[t]; compare with the pursuer's distance db

QUESTION

In an unweighted, undirected graph (an adjacency dict) Alice is at node a and wants to reach the target t. Return the length (number of edges) of the shortest path from a to t found with BFS. The follow-ups ask for every node that lies on some shortest path, and whether Alice can reach t before a pursuer Bob.

Function signature

def shortest_length(adj: dict, a, t) -> int

EXAMPLES

Example 1	
Input	`{0: [1, 2], 1: [0, 3], 2: [0, 3], 3: [1, 2]}, 0, 3`
Output	`2`
Why	`0 -> 1 -> 3 and 0 -> 2 -> 3 both take 2 edges.`

Example 2	
Input	`{0: []}, 0, 0`
Output	`0`
Why	`A start that is the target needs 0 edges.`

FULL CODE | main question first, then every follow-up, tests at the end

```
# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

from collections import deque

FAR = 10 ** 9


# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
def bfs(adj, s):
    # distance from s to every node it can reach
    dist = {s: 0}
    queue = deque([s])
    while queue:
        u = queue.popleft()
        for v in adj[u]:
            if v not in dist:
                dist[v] = dist[u] + 1
                queue.append(v)
    return dist
```

```
# ---- main question: main function ----
# Idea: BFS from Alice (a) gives the length of the shortest path to t.
def shortest_length(adj, a, t):
    return bfs(adj, a).get(t)


# the nodes that lie on some shortest path a -> t
def nodes_on_shortest_paths(adj, a, t):
    da = bfs(adj, a)
    dt = bfs(adj, t)
    result = set()
    if t not in da:
        return result
    for v in da:
        # on a shortest path exactly when the two distances add up to the full length
        if v in dt and da[v] + dt[v] == da[t]:
```

```
            result.add(v)
    return result


# ---- main question: tests ----
def test_main_question():
    # a square 0-1-3 and 0-2-3 plus a tail 3-4
    adj = {0: [1, 2], 1: [0, 3], 2: [0, 3], 3: [1, 2, 4], 4: [3]}
    assert shortest_length(adj, 0, 4) == 3
    assert nodes_on_shortest_paths(adj, 0, 4) == {0, 1, 2, 3, 4}
    assert shortest_length(adj, 0, 9) is None


# ========================================================
# FOLLOW-UP 1: What if Bob can also meet Alice in the middle of an edge?
# ========================================================
# ---- follow-up 1: helper functions ----
def step_is_safe(u, v, da, db, meet_on_edge):
    # may Alice walk from u to v? Bob must reach v strictly after Alice does.
    if db.get(v, FAR) <= da[v]:
        return False
    if meet_on_edge:
        # they would swap places inside the edge u - v
        if db.get(v, FAR) == da[u] and db.get(u, FAR) == da[v]:
            return False
    return True


# ---- follow-up 1: function ----
# Answer: Alice only needs to follow shortest-path layers, and every
# step u -> v must be safe. With meet_on_edge=True a swap on an edge
# counts as being caught.
def can_escape(adj, a, t, b, meet_on_edge=False):
    da = bfs(adj, a)
    db = bfs(adj, b)
    if t not in da:
        return False
    if db.get(a, FAR) <= 0:                  # Bob stands on Alice's start
        return False
    on_path = nodes_on_shortest_paths(adj, a, t)
    seen = {a}
    frontier = [a]
    while frontier:
        next_frontier = []
        for u in frontier:
```

```
            for v in adj[u]:
                if v not in on_path or v in seen or da[v] != da[u] + 1:
```

```
                    continue
                if step_is_safe(u, v, da, db, meet_on_edge):
                    seen.add(v)
                    next_frontier.append(v)
        frontier = next_frontier
    return t in seen


# ---- follow-up 1: tests ----
def test_follow_up_1():
    adj = {0: [1, 2], 1: [0, 3], 2: [0, 3], 3: [1, 2, 4], 4: [3]}
    assert not can_escape(adj, 0, 4, 4)              # Bob waiting at the target catches Alice there
    chain = {0: [1], 1: [0, 2], 2: [1, 3], 3: [2]}
    assert not can_escape(chain, 0, 3, 3)            # Bob starts at the target and reaches it first
    assert can_escape(chain, 1, 3, 0)                # Bob starts behind Alice: she stays ahead


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

210 Radio Broadcast Reachability — MEDIUM

Template fit: neighbors(u) = every radio within the radius, worked out with squared distances; goal = the target radio, answer = whether it was reached

QUESTION

You are given a list of radios placed on a 2D plane, each described by its (x, y) coordinates. Every radio can broadcast to any other radio within a fixed Euclidean distance radius (inclusive). A message can travel from one radio to another either directly (within radius) or through a chain of intermediate radios that each relay it to a next radio within radius . Given the radios, the broadcast radius , a source index, and a target index, determine whether a message originating at source can reach target through zero or more hops. Return True or False .

Function signature

```
from typing import List, Tuple
def can_reach(radios: List[Tuple[float, float]],
              radius: float,
              source: int,
              target: int) -> bool:
    ...
```

Follow-up (as reported): Model the radios as nodes and connect any two within radius , then run BFS (or DFS / union-find) from source . To avoid floating-point issues, compare squared distances against radius * radius .

EXAMPLES

Example 1	
Input	`radios = [(0, 0), (1, 0), (2, 0), (5, 0)], radius = 1.5, source = 0, target = 2`
Output	`True`
Why	`0->1 (dist 1) and 1->2 (dist 1) are each within 1.5, so 2 is reachable.`

QUESTION

Example 2	
Input	`radios = [(0, 0), (1, 0), (2, 0), (5, 0)], radius = 1.5, source = 0, target = 3`
Output	`False`
Why	`Radio 3 is 3 units from radio 2 (> 1.5) and farther from the rest; no chain of hops reaches it.`

Example 3	
Input	`radios = [(0, 0), (3, 4)], radius = 5.0, source = 0, target = 1`
Output	`True`
Why	`The Euclidean distance is exactly 5.0, which is within radius.`

CONSTRAINTS

- 1 <= len(radios) <= 2000
- Coordinates are integers or floats with -10^4 <= x, y <= 10^4 . radius >= 0 .
- 0 <= source, target < len(radios) .
- Two radios a and b are directly connected when sqrt((ax-bx)^2 + (ay-by)^2) <= radius .
- If source == target , the answer is True .

FULL CODE | main question first, then every follow-up, tests at the end

```
# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

from collections import deque


# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
def squared_distance(p, q):
    # compare squared distances so no square root is needed
    dx = p[0] - q[0]
    dy = p[1] - q[1]
    return dx * dx + dy * dy


# ---- main question: main function ----
# Idea: the radios are the nodes of an implicit graph: two radios are
# neighbours when they are within `radius`. BFS from the source radio;
# every radio enters the queue once.
def can_reach(radios, radius, source, target):
    limit = radius * radius
    seen = [False] * len(radios)
    seen[source] = True
    queue = deque([source])
    while queue:
        u = queue.popleft()
        if u == target:
            return True
        for v in range(len(radios)):
            if not seen[v] and squared_distance(radios[u], radios[v]) <= limit:
                seen[v] = True
                queue.append(v)
    return False
```

```
# ---- main question: tests ----
def test_main_question():
    line = [(0, 0), (1, 0), (2, 0), (5, 0)]
    assert can_reach(line, 1.5, 0, 2) is True
```

```
    assert can_reach(line, 1.5, 0, 3) is False
    assert can_reach([(0, 0), (3, 4)], 5.0, 0, 1) is True
    assert can_reach(line, 1, 3, 3) is True


# ======================================================
# FOLLOW-UP 1: Thousands of source/target queries on the same radios
# ======================================================
# ---- follow-up 1: helper functions ----
class RadioNetwork:
    # build the connected groups once with union-find, then every query
    # is two finds: radios in one group can reach each other
    def __init__(self, radios, radius):
        n = len(radios)
        self.parent = list(range(n))
        for i in range(n):
            for j in range(i + 1, n):
                if squared_distance(radios[i], radios[j]) <= radius * radius:
                    self.parent[self.find(i)] = self.find(j)

    def find(self, a):
        while self.parent[a] != a:
            self.parent[a] = self.parent[self.parent[a]]
            a = self.parent[a]
        return a

    def can_reach(self, source, target):
        return self.find(source) == self.find(target)


# ---- follow-up 1: tests ----
def test_follow_up_1():
    line = [(0, 0), (1, 0), (2, 0), (5, 0)]
    network = RadioNetwork(line, 1.5)
    assert network.can_reach(0, 2) is True
    assert network.can_reach(0, 3) is False
    assert network.can_reach(3, 3) is True


# ======================================================
# RUN EVERYTHING
# ======================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
```

```
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

211 Minimum Direction Violations in a Directed Graph

MEDIUM

Template fit: an edge in its own direction costs w = 0, the same edge walked backwards costs w = 1; free steps use appendleft

QUESTION

A directed graph has n nodes. Walking along an edge in its own direction is free; walking against it costs 1 (a direction violation). Return the minimum total cost of a walk from start to end, or -1 if end cannot be reached at all.

Function signature

def min_violations(n: int, edges: list, start: int, end: int) -> int

EXAMPLES

Example 1	
Input	`5, [[0, 1], [2, 1], [2, 3], [4, 3]], 0, 4`
Output	`2`
Why	`Path 0 -> 1 (free), 1 -> 2 against edge 2 -> 1 (cost 1), 2 -> 3 (free), 3 -> 4 against edge 4 -> 3 (cost 1): total 2.`

Example 2	
Input	`3, [[0, 1]], 0, 2`
Output	`-1`
Why	`Node 2 has no edge at all, so it cannot be reached.`

FULL CODE | main question first, then every follow-up, tests at the end

```
# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

from collections import deque
import heapq

INF = float("inf")


# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
def build_graph(n, edges):
    # every edge u -> v is also usable backwards: forward costs 0, backward costs 1
    graph = []
    for i in range(n):
        graph.append([])
    for u, v in edges:
        graph[u].append((v, 0))
        graph[v].append((u, 1))
    return graph


# ---- main question: main function ----
# Idea: with every cost either 0 or 1 a deque is enough (0-1 BFS):
# a free step goes to the FRONT of the deque, a cost-1 step to the BACK,
# so cells are always taken out in order of distance.
def min_violations(n, edges, start, end):
    graph = build_graph(n, edges)
    dist = [INF] * n
    dist[start] = 0
```

```
    dq = deque([start])
    while dq:
        u = dq.popleft()
        for v, w in graph[u]:
            if dist[u] + w < dist[v]:
                dist[v] = dist[u] + w
                if w == 0:
                    dq.appendleft(v)
                else:
```

```
                    dq.append(v)
    if dist[end] == INF:
        return -1
    return dist[end]


# ---- main question: tests ----
def test_main_question():
    edges = [(0, 1), (2, 1), (2, 3), (4, 3)]
    assert min_violations(5, edges, 0, 4) == 2        # 0 -> 1 -> 2 -> 3 -> 4 reverses two edges
    assert min_violations(5, edges, 0, 0) == 0
    assert min_violations(3, [(0, 1)], 0, 2) == -1


# =======================================================
# FOLLOW-UP 1: Why is 0-1 BFS correct, and how does it compare with Dijkstra?
# =======================================================
# Answer: the deque always holds distances d and d + 1 only, in order,
# so each node is settled in distance order exactly like Dijkstra, but
# with O(1) deque operations instead of O(log n) heap operations: O(V + E).


# =======================================================
# FOLLOW-UP 2: What if edge costs can be arbitrary non-negative numbers?
# =======================================================


# =======================================================
# FOLLOW-UP 3: Edge costs are arbitrary non-negative numbers (different for each direction)
# =======================================================
# ---- follow-up 3: function ----
# Answer: use Dijkstra with a heap on the same doubled graph and give
# each direction its own cost. 0-1 BFS only works because the costs are 0 or 1.
def cheapest_route(n, edges, start, end):
    # edges are (u, v, forward_cost, backward_cost)
    graph = []
    for i in range(n):
        graph.append([])
    for u, v, forward, backward in edges:
        graph[u].append((v, forward))
        graph[v].append((u, backward))
    dist = [INF] * n
    dist[start] = 0
    heap = [(0, start)]
    while heap:
```

```
        d, u = heapq.heappop(heap)
        if d > dist[u]:
            continue                        # an outdated heap entry
        for v, w in graph[u]:
            if d + w < dist[v]:
                dist[v] = d + w
                heapq.heappush(heap, (d + w, v))
    if dist[end] == INF:
        return -1
    return dist[end]
```

```
# ---- follow-up 3: tests ----
def test_follow_up_3():
    edges = [(0, 1), (2, 1), (2, 3), (4, 3)]
    costed = []
    for u, v in edges:
        costed.append((u, v, 0, 1))                  # the same graph, every reversal costs 1
    assert cheapest_route(5, costed, 0, 4) == 2
    assert cheapest_route(2, [(0, 1, 5, 2)], 0, 1) == 5
    assert cheapest_route(2, [(0, 1, 5, 2)], 1, 0) == 2


# ======================================================
# RUN EVERYTHING
# ======================================================
def run_all_tests():
    test_main_question()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

212 Compute shortest paths with blocked nodes

MEDIUM

Template fit: neighbors skip every blocked node; if the source itself is blocked the answer is all -1; unreachable nodes stay -1

QUESTION

Given a graph with nodes and edges and a designated source node s, compute the shortest distance from s to every other node. Some nodes are inaccessible (blocked) and cannot be traversed or reached; treat them as removed from the graph. Return a distance for each node where unreachable nodes (including blocked ones) are -1. Describe your algorithm, data structures, and time/space complexity, and note any differences for directed vs. undirected graphs.

Given n nodes labeled 0..n-1, edges, a source, and blocked nodes, return shortest unweighted distances from the source. Blocked and unreachable nodes must be -1. Set directed=True to treat edges as directed.
Constraints
Nodes are labeled 0 through n-1Edges are unweighted

EXAMPLES

```
Input: (5, [[0, 1], [1, 2], [0, 3], [3, 4]], 0, [3], False)
Expected Output: [0, 1, 2, -1, -1]
Input: (4, [[0, 1], [1, 2], [2, 3]], 0, [], True)
Expected Output: [0, 1, 2, 3]
```

Hints

Treat blocked nodes as removed before BFS.For directed graphs, only add the given edge direction.

FULL CODE | main question first, then every follow-up, tests at the end

```
# ======================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ======================================================

from collections import deque
import heapq

INF = float("inf")
```

```python
# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
def make_adjacency(n, edges, directed):
    # neighbour lists; an undirected edge is stored in both directions
    adj = []
    for i in range(n):
        adj.append([])
    for edge in edges:
        u, v = edge[0], edge[1]
        weight = 1
        if len(edge) == 3:
            weight = edge[2]
        adj[u].append((v, weight))
        if not directed:
            adj[v].append((u, weight))
    return adj


# ---- main question: main function ----
# Idea: a BFS that never enters a blocked node. If the source itself is
# blocked everything is -1; unreachable and blocked nodes also get -1.
def blocked_distances(n, edges, source, blocked, directed=False):
    if source in blocked:
        return [-1] * n
    adj = make_adjacency(n, edges, directed)
    dist = [-1] * n
    dist[source] = 0
    queue = deque([source])
    while queue:
        u = queue.popleft()
        for v, w in adj[u]:
            if v not in blocked and dist[v] == -1:
                dist[v] = dist[u] + 1
                queue.append(v)
    return dist


# ---- main question: tests ----
def test_main_question():
    edges = [(0, 1), (1, 2), (0, 3), (3, 2), (2, 4)]
    assert blocked_distances(5, edges, 0, []) == [0, 1, 2, 1, 3]
    assert blocked_distances(5, edges, 0, [1])[2] == 2                # blocking 1 forces the route through
3
    assert blocked_distances(5, edges, 0, [2]) == [0, 1, -1, 1, -1]    # blocking 2 cuts node 4 off
    assert blocked_distances(3, edges[:2], 0, [0]) == [-1, -1, -1]     # the source is blocked
    path = [(0, 1), (1, 2)]
    assert blocked_distances(3, path, 2, [], True) == [-1, -1, 0]      # directed edges go one way only


# ============================================================
# FOLLOW-UP 1: The edges have weights
# ============================================================


# ============================================================
# FOLLOW-UP 2: Directed versus undirected
# ============================================================
# ---- follow-up 1: function ----
# Answer: BFS is no longer correct with weights; use Dijkstra with a
# heap and skip blocked nodes the same way: O((V + E) log V).
# Directed and undirected differ only in whether the reverse edge is
```

```
# added (the `directed` flag of make_adjacency).
def blocked_distances_weighted(n, edges, source, blocked, directed=False):
    if source in blocked:
        return [-1] * n
    adj = make_adjacency(n, edges, directed)
    dist = [INF] * n
    dist[source] = 0
    heap = [(0, source)]
    while heap:
        d, u = heapq.heappop(heap)
        if d > dist[u]:
            continue
        for v, w in adj[u]:
            if v not in blocked and d + w < dist[v]:
                dist[v] = d + w
```

```
                heapq.heappush(heap, (d + w, v))
    answer = []
    for d in dist:
        if d == INF:
            answer.append(-1)
        else:
            answer.append(d)
    return answer


# ---- follow-up 1: tests ----
def test_follow_up_1():
    weighted = [(0, 1, 5), (1, 2, 1), (0, 2, 10)]
    assert blocked_distances_weighted(3, weighted, 0, []) == [0, 5, 6]
    assert blocked_distances_weighted(3, weighted, 0, [1])[2] == 10


# ======================================================
# FOLLOW-UP 3: Nodes get blocked over time and you must keep distances up to date
# ======================================================
# Answer: blocking a node can only make distances larger. Recompute
# the BFS from the source after each change, or only for the nodes
# whose shortest path went through the blocked node.


# ======================================================
# RUN EVERYTHING
# ======================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

213 Shortest Directed Cycle Through a Node

MEDIUM

Template fit: start = v; the goal is an edge that leads back to v, so return dist[u] + 1 the moment a neighbour equals v

QUESTION

Given a directed graph with n nodes and a list of edges (a, b), return the length (number of edges) of the shortest cycle that passes through node v, or -1 if v is on no cycle.

Function signature

def shortest_cycle_through(n: int, edges: list, v: int) -> int

EXAMPLES

QUESTION

Example 1	
Input	3, [(0, 1), (1, 2), (2, 0)], 0
Output	3
Why	0 -> 1 -> 2 -> 0 is a cycle of 3 edges.

Example 2	
Input	3, [(0, 1), (1, 2)], 0
Output	-1
Why	No edge leads back to 0.

Example 3	
Input	3, [(0, 1), (1, 0), (1, 2), (2, 0)], 0
Output	2
Why	A shorter cycle 0 -> 1 -> 0 exists next to the longer one.

FULL CODE | main question first, then every follow-up, tests at the end

```
# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ========================================================

from collections import deque
import heapq

INF = float("inf")


# ========================================================
# MAIN QUESTION
# ========================================================
# ---- main question: helper functions ----
def make_graph(n, edges):
    # adjacency lists of (neighbour, cost); an edge may be (a, b) or (a, b, cost)
    adj = []
    for i in range(n):
        adj.append([])
    for edge in edges:
        a, b = edge[0], edge[1]
        cost = 1
        if len(edge) == 3:
            cost = edge[2]
        adj[a].append((b, cost))
    return adj


# ---- main question: main function ----
# Idea: a directed cycle through v is a path v -> u followed by an edge
# u -> v. BFS reaches nodes in order of distance, so the first edge we
# see that leads back to v closes the shortest cycle: dist[u] + 1.
def shortest_cycle_through(n, edges, v):
    adj = make_graph(n, edges)
    dist = {v: 0}
    queue = deque([v])
    while queue:
        u = queue.popleft()
```

```
        for w, cost in adj[u]:
            if w == v:
                return dist[u] + 1
            if w not in dist:
```

```
                dist[w] = dist[u] + 1
                queue.append(w)
    return -1


# ---- main question: tests ----
def test_main_question():
    ring = [(0, 1), (1, 2), (2, 0)]
    assert shortest_cycle_through(3, ring, 0) == 3
    assert shortest_cycle_through(3, [(0, 1), (1, 2)], 0) == -1
    assert shortest_cycle_through(2, [(0, 0)], 0) == 1       # a self loop is a cycle of length 1


# ========================================================
# FOLLOW-UP 1: How do you find the shortest cycle in the whole graph?
# ========================================================


# ========================================================
# FOLLOW-UP 3: The shortest cycle anywhere in the graph (the girth)
# ========================================================
# ---- follow-up 1: function ----
# Answer: run the same BFS from every node and take the smallest
# result: O(V * (V + E)), fine up to a few thousand nodes. (For an
# undirected graph, any edge that is not a tree edge closes a cycle of
# length dist[u] + dist[w] + 1: the LeetCode 2608 version.)
def girth(n, edges):
    best = -1
    for v in range(n):
        size = shortest_cycle_through(n, edges, v)
        if size > 0 and (best == -1 or size < best):
            best = size
    return best


# ---- follow-up 1: tests ----
def test_follow_up_1():
    ring = [(0, 1), (1, 2), (2, 0)]
    both = ring + [(1, 2), (2, 1)]                  # a shorter cycle exists: 1 <-> 2
    assert girth(3, both) == 2
    assert girth(3, [(0, 1), (1, 2)]) == -1


# ========================================================
# FOLLOW-UP 2: What if edges have positive weights?
```

```
# ========================================================
# ---- follow-up 2: function ----
# Answer: replace BFS by Dijkstra from v and close the cycle at the
# cheapest dist[u] + weight(u, v): O((V + E) log V) with a heap. With
# negative weights use Bellman-Ford and handle negative cycles separately.
def weighted_cycle_through(n, edges, v):
    adj = make_graph(n, edges)
    dist = {v: 0}
    heap = [(0, v)]
    best = INF
    while heap:
        d, u = heapq.heappop(heap)
        if d > dist.get(u, INF):
            continue
        for w, cost in adj[u]:
```

```
            if w == v:
                if d + cost < best:
                    best = d + cost
            elif d + cost < dist.get(w, INF):
                dist[w] = d + cost
                heapq.heappush(heap, (d + cost, w))
    if best == INF:
        return -1
    return best


# ---- follow-up 2: tests ----
def test_follow_up_2():
    weighted = [(0, 1, 1), (1, 0, 2), (0, 2, 1), (2, 0, 9)]
    assert weighted_cycle_through(3, weighted, 0) == 3     # the cheap way back is 0 -> 1 -> 0


# =============================================================
# RUN EVERYTHING
# =============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

GROUP 29 OF 43 | 5 QUESTIONS, ONE CODE TEMPLATE | 5 M

Topological sort (Kahn) and cycle detection

214 Course Scheduling and Dependency Variants

MEDIUM

Template fit: build the vertex set from the edges too; return the order or None

QUESTION

Topological-ordering family. Variants: course ordering with validity checks and a nested-list extension; merging two arrays whose internal order defines precedence (return false on a conflict cycle); and a damaged-node dependency-path variant that minimizes broken nodes on a route.

Course-ordering variant

Work through the course-ordering problem, including validity checks, complexity analysis, and test cases.
Extend the result to a nested-list format. The grouping rule is unspecified; clarify what each inner list represents before implementing the extension.

Damaged-node and path variant

Remove damaged nodes and their downstream effects, then find a valid path.
A follow-up minimizes broken nodes on a required route. Clarify the graph direction, path endpoints, and meaning of broken nodes before choosing an algorithm.

Merge-two-ordered-sequences variant

Input: two integer arrays a and b. The order of elements inside each array is an ordering constraint — a = [1, 3, 5] means 1 must come before 3, and 3 before 5.
Determine whether the two arrays can be merged into one sequence that preserves the relative order of both. Return false if the constraints conflict; otherwise return true (or any valid merged sequence).

EXAMPLES

Merge-two-ordered-sequences

```
a = [1, 2, 3]
b = [2, 4, 5]
-> true   (one valid merged order: [1, 2, 3, 4, 5])

a = [1, 6, 4]
b = [4, 1]
-> false  (a requires 1 -> 6 -> 4, b requires 4 -> 1: contradiction)
```

FULL CODE | main question first, then every follow-up, tests at the end

```python
# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

from collections import deque


# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
def edges_from_sequences(sequences):
    # each neighbouring pair in a sequence is a rule: the first comes before the second
    edges = []
    for seq in sequences:
        for i in range(len(seq) - 1):
```

```
            edges.append((seq[i], seq[i + 1]))
    return edges


# ---- main question: main function ----
# Idea (Kahn's algorithm): an edge (u, v) means u must come before v.
# Count how many edges point into each vertex (indegree), keep a queue
# of vertices with indegree 0, take one, add it to the order and lower
# the indegree of the vertices after it. If not every vertex was taken
# there is a cycle and no order exists (None).
def topological_order(vertices, edges):
    indegree = {}
    after = {}
    for v in vertices:
        indegree[v] = 0
        after[v] = []
    for u, v in edges:
        for x in (u, v):
            if x not in indegree:
                indegree[x] = 0                 # a vertex that appears only in an edge
                after[x] = []
        after[u].append(v)
        indegree[v] += 1
    queue = deque()
    for v in indegree:
        if indegree[v] == 0:
```

```
            queue.append(v)
    order = []
    while queue:
        u = queue.popleft()
        order.append(u)
        for v in after[u]:
            indegree[v] -= 1
            if indegree[v] == 0:
                queue.append(v)
    if len(order) == len(indegree):
        return order
    return None


# the same idea merges two sequences: shared values are the same vertex
def merge_two_orders(a, b):
    vertices = set(a) | set(b)
    return topological_order(vertices, edges_from_sequences([a, b]))


def can_merge(a, b):
    # is there an order that keeps both sequences in order?
    return merge_two_orders(a, b) is not None


# ---- main question: tests ----
def test_main_question():
    order = topological_order(range(4), [(0, 1), (1, 2)])    # course 3 has no rule at all
    assert len(order) == 4
    assert order.index(0) < order.index(1) < order.index(2)
    assert topological_order(range(2), [(0, 1), (1, 0)]) is None      # a cycle has no order
    assert can_merge([1, 2, 3], [2, 4, 5])
    assert merge_two_orders([1, 2, 3], [2, 4, 5]) is not None
    assert not can_merge([1, 6, 4], [4, 1])
    assert not can_merge([1, 2, 1], [])                      # a repeated value is a conflict with itself


# ================================================
# FOLLOW-UP 1: How do you check that an order is valid?
```

```
# ============================================================
# ---- follow-up 1: function ----
# Answer: write down each vertex's position in the order, check every
# vertex appears exactly once, then check that for every edge (u, v)
# the position of u is smaller than the position of v.
def is_valid_order(order, vertices, edges):
    if len(order) != len(set(vertices)):
        return False
    if set(order) != set(vertices):
        return False
    position = {}
    for i in range(len(order)):
        position[order[i]] = i
    for u, v in edges:
        if position[u] >= position[v]:
            return False                        # a prerequisite comes too late
    return True


# ---- follow-up 1: tests ----
def test_follow_up_1():
    courses = range(4)
    rules = [(0, 1), (1, 2)]
    order = topological_order(courses, rules)
    assert is_valid_order(order, courses, rules)
    assert not is_valid_order([1, 0, 2, 3], courses, rules)     # a swapped order breaks a rule
    assert not is_valid_order([0, 1, 2], courses, rules)        # a vertex is missing


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

215 Validate Race Results (Cycle Detection) MEDIUM

Template fit: edge = winner -> loser; the results are consistent exactly when there is no cycle

QUESTION

You are given a list of pairwise race results as (winner, loser) string pairs, meaning winner beat loser . The results are consistent if they can all be true simultaneously under a strict ranking — equivalently, if the directed graph (edge winner -> loser) contains no cycle. Return True if the results are valid (acyclic) and False otherwise.

Function signature

def results_are_valid(results: list[tuple[str, str]]) -> bool:

...

Follow-up (as reported): Recursive DFS can hit Python's recursion limit on long chains; convert to an iterative DFS with an explicit stack, or use Kahn's topological-sort algorithm.

EXAMPLES

QUESTION

Example 1	
Input	`results = [("A","B"), ("B","C")]`
Output	`True`
Why	`A beat B beat C — a consistent ranking A > B > C, no cycle.`

Example 2	
Input	`results = [("A","B"), ("B","C"), ("C","A")]`
Output	`False`
Why	`A > B > C > A is a contradiction (a cycle).`

Example 3	
Input	`results = []`
Output	`True`
Why	`No results impose no contradiction.`

CONSTRAINTS

- 0 <= len(results) <= 10^5 .
- Player names are non-empty strings; duplicate result pairs may appear.
- An empty result list is valid (True).

FULL CODE | main question first, then every follow-up, tests at the end

```
# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ========================================================

from collections import deque


# ========================================================
# MAIN QUESTION
# ========================================================
# ---- main question: helper functions ----
def build_graph(results):
    # results are pairs (winner, loser); graph[winner] = the people they beat
    graph = {}
    for winner, loser in results:
        if winner not in graph:
            graph[winner] = []
        if loser not in graph:
            graph[loser] = []
        graph[winner].append(loser)
    return graph


# ---- main question: main function ----
# Idea (Kahn's algorithm): results are consistent when nobody beats
# somebody who (through a chain) beat them, i.e. there is no cycle.
# Repeatedly take people nobody is left above; everybody gets taken
# exactly when there is no cycle.
def results_are_valid(results):
    graph = build_graph(set(results))              # a repeated result counts once
    beaten_by = {}
    for person in graph:
```

```
        beaten_by[person] = 0
    for winner in graph:
        for loser in graph[winner]:
            beaten_by[loser] += 1
    queue = deque()
    for person in graph:
        if beaten_by[person] == 0:
            queue.append(person)
    removed = 0
    while queue:
```

```
        u = queue.popleft()
        removed += 1
        for v in graph[u]:
            beaten_by[v] -= 1
            if beaten_by[v] == 0:
                queue.append(v)
    return removed == len(graph)


# ---- main question: tests ----
def test_main_question():
    assert results_are_valid([("A", "B"), ("B", "C")]) is True
    assert results_are_valid([("A", "B"), ("B", "C"), ("C", "A")]) is False
    assert results_are_valid([]) is True
    assert results_are_valid([("A", "A")]) is False


# ============================================================
# FOLLOW-UP 1: Recursive DFS hits Python's recursion limit on long chains. What do you do?
# ============================================================
# ---- follow-up 1: function ----
# Answer: use Kahn's algorithm (already a loop), or the three-colour DFS
# with your own stack so the depth is limited by memory, not by the
# interpreter. state: missing = not visited, 1 = on the current path,
# 2 = finished; meeting a person who is on the current path is a cycle.
def results_are_valid_dfs(results):
    graph = build_graph(results)
    state = {}
    for start in graph:
        if start in state:
            continue
        state[start] = 1
        stack = [start]
        next_index = {start: 0}                    # which child of each person to look at next
        while stack:
            u = stack[-1]
            i = next_index[u]
            if i < len(graph[u]):
                next_index[u] = i + 1
                v = graph[u][i]
                if state.get(v) == 1:
                    return False
                if v not in state:
                    state[v] = 1
                    next_index[v] = 0
```

```
                    stack.append(v)
            else:
                state[u] = 2
                stack.pop()
    return True


# ---- follow-up 1: tests ----
def test_follow_up_1():
```

```
    long_chain = []
    for i in range(5000):
        long_chain.append((str(i), str(i + 1)))
    assert results_are_valid(long_chain) and results_are_valid_dfs(long_chain)
    assert not results_are_valid_dfs([("A", "B"), ("B", "A")])


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

216 Solve tree, graph, sliding-window problems

MEDIUM

Template fit: the course-schedule part is can-finish: len(order) == n; the zigzag and window parts are separate small loops

QUESTION

Question

LeetCode 103. Binary Tree Zigzag Level Order Traversal – given a binary tree root, return its zigzag level-order traversal. LeetCode 207. Course Schedule – given course prerequisites, determine if all courses can be finished by detecting cycles with topological sort. LeetCode 340. Longest Substring with At Most K Distinct Characters – given string s and integer k, return the length of the longest substring containing at most k distinct characters.
https://leetcode.com/problems/binary-tree-zigzag-level-order-traversal/description/
https://leetcode.com/problems/course-schedule/description/
https://leetcode.com/problems/longest-substring-with-at-most-k-distinct-characters/description/

Part 1: Binary Tree Zigzag Level Order TraversalYou are given a binary tree in level-order array form, where each element is either an integer or None for a missing child. Return the zigzag level-order traversal of the tree's node values. In a zigzag traversal, the first level is read from left to right, the second from right to left, the third from left to right, and so on.

Constraints

0 <= len(root) <= 2000Each non-None node value is an integer in the range [-10^4, 10^4]The input list represents a valid binary tree in level-order form

EXAMPLES

```
Input: ([3, 9, 20, None, None, 15, 7],)
Expected Output: [[3], [20, 9], [15, 7]]
```

Explanation: Level 1 is read left-to-right, level 2 right-to-left, and level 3 left-to-right.

```
Input: ([1],)
Expected Output: [[1]]
```

Explanation: A single-node tree has only one level.

QUESTION

Hints

A breadth-first search naturally processes the tree one level at a time.You can collect each level left-to-right, then reverse every other level before appending it to the answer.

Part 2: Course ScheduleThere are num_courses courses labeled from 0 to num_courses - 1. You are given a list of prerequisite pairs [a, b], meaning you must take course b before course a. Determine whether it is possible to finish all courses. This is equivalent to checking whether the prerequisite graph contains a cycle.

Constraints

1 <= num_courses <= 10^50 <= len(prerequisites) <= 2 * 10^5Each prerequisite pair has length 20 <= course, prerequisite < num_courses

Examples

Input: (2, [[1, 0]])

Expected Output: True

Explanation: Course 0 can be taken first, then course 1.

Input: (2, [[1, 0], [0, 1]])

Expected Output: False

Explanation: The two courses depend on each other, forming a cycle.

Hints

Think of courses as nodes in a directed graph and prerequisites as directed edges.If you repeatedly take courses with indegree 0 and cannot process all nodes, a cycle exists.

Part 3: Longest Substring with At Most K Distinct CharactersGiven a string s and an integer k, return the length of the longest substring that contains at most k distinct characters.

Constraints

0 <= len(s) <= 2 * 10^50 <= k <= len(s)s may contain any standard characters

Examples

Input: ("eceba", 2)

Expected Output: 3

Explanation: The longest valid substring is 'ece', which has length 3.

Input: ("aa", 1)

Expected Output: 2

Explanation: The whole string contains only one distinct character.

Hints

A sliding window can maintain a valid substring while expanding to the right.Keep character counts in the current window, and shrink from the left whenever the number of distinct characters exceeds k.

FULL CODE | main question first, then every follow-up, tests at the end

```
# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

from collections import deque


# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
```

```
        self.val = val
        self.left = left
        self.right = right


def build_tree(values):
    # level-order list, None for a missing child: [3, 9, 20, None, None, 15, 7]
    if not values:
        return None
    root = TreeNode(values[0])
    queue = deque([root])
    i = 1
    while queue and i < len(values):
        node = queue.popleft()
        if i < len(values) and values[i] is not None:
            node.left = TreeNode(values[i])
            queue.append(node.left)
        i += 1
        if i < len(values) and values[i] is not None:
            node.right = TreeNode(values[i])
            queue.append(node.right)
        i += 1
    return root


# ---- main question: main functions ----
# PART 1: zigzag level order of a binary tree.
# Idea: go level by level with a queue; reverse every second level.
def zigzag(root):
```

```
    result = []
    if root is None:
        return result
    queue = deque([root])
    left_to_right = True
    while queue:
        level = []
        for i in range(len(queue)):              # exactly the nodes of this level
            node = queue.popleft()
            level.append(node.val)
            if node.left:
                queue.append(node.left)
            if node.right:
                queue.append(node.right)
        if not left_to_right:
            level.reverse()
        result.append(level)
        left_to_right = not left_to_right
    return result


# PART 2: can all courses be finished? prereqs are pairs (a, b): take b before a.
# Idea (Kahn's algorithm): count the unmet prerequisites of each course;
# take a course with none, lower the count of the courses that needed it,
# and repeat. If every course gets taken there is no cycle.
def can_finish(n, prereqs):
    needs = []
    for i in range(n):
        needs.append([])
    unmet = [0] * n
    for a, b in prereqs:
        needs[b].append(a)                       # b unlocks a
        unmet[a] += 1
    queue = deque()
    for c in range(n):
        if unmet[c] == 0:
```

```
            queue.append(c)
    taken = 0
    while queue:
        c = queue.popleft()
        taken += 1
        for nxt in needs[c]:
            unmet[nxt] -= 1
            if unmet[nxt] == 0:
                queue.append(nxt)
```

```
    return taken == n


# PART 3: longest substring with at most k distinct letters.
# Idea: sliding window with a count of each letter inside it; when
# there are too many distinct letters, shrink from the left.
def longest_at_most_k(s, k):
    count = {}
    left = 0
    best = 0
    for right in range(len(s)):
        ch = s[right]
        count[ch] = count.get(ch, 0) + 1
        while len(count) > k:
            count[s[left]] -= 1
            if count[s[left]] == 0:
                del count[s[left]]
            left += 1
        if right - left + 1 > best:
            best = right - left + 1
    return best


# ---- main question: tests ----
def test_main_question():
    tree = build_tree([3, 9, 20, None, None, 15, 7])
    assert zigzag(tree) == [[3], [20, 9], [15, 7]]
    assert zigzag(None) == []
    assert can_finish(2, [(1, 0)])
    assert not can_finish(2, [(1, 0), (0, 1)])
    assert longest_at_most_k("eceba", 2) == 3
    assert longest_at_most_k("aa", 1) == 2
    assert longest_at_most_k("abc", 0) == 0


# ============================================================
# FOLLOW-UP 1: Course Schedule II: return an order in which to take the courses
# ============================================================
# ---- follow-up 1: function ----
# Answer: the same Kahn loop, but write down each course as it is
# taken. If fewer than n courses were taken there is a cycle: return [].
def course_order(n, prereqs):
    needs = []
    for i in range(n):
        needs.append([])
```

```
    unmet = [0] * n
    for a, b in prereqs:
        needs[b].append(a)
        unmet[a] += 1
    queue = deque()
    for c in range(n):
        if unmet[c] == 0:
            queue.append(c)
    order = []
    while queue:
```

```
        c = queue.popleft()
        order.append(c)
        for nxt in needs[c]:
            unmet[nxt] -= 1
            if unmet[nxt] == 0:
                queue.append(nxt)
    if len(order) == n:
        return order
    return []


# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert course_order(4, [(1, 0), (2, 1), (3, 2)]) == [0, 1, 2, 3]
    assert course_order(2, [(0, 1), (1, 0)]) == []


# ============================================================
# FOLLOW-UP 2: Longest substring with exactly K distinct characters
# ============================================================
# ---- follow-up 2: helper functions ----
def count_at_most_k(s, k):
    # how many substrings have at most k distinct letters?
    if k < 1:
        return 0
    count = {}
    left = 0
    total = 0
    for right in range(len(s)):
        ch = s[right]
        count[ch] = count.get(ch, 0) + 1
        while len(count) > k:
            count[s[left]] -= 1
            if count[s[left]] == 0:
                del count[s[left]]
```

```
            left += 1
        total += right - left + 1                # every substring ending at `right` and starting at left..r
ight
    return total


# ---- follow-up 2: function ----
# Answer: "exactly k" = "at most k" minus "at most k - 1". (Or keep one
# window and record lengths only when the distinct count is exactly k,
# which needs more care when shrinking.)
def count_exactly_k(s, k):
    return count_at_most_k(s, k) - count_at_most_k(s, k - 1)


# ---- follow-up 2: tests ----
def test_follow_up_2():
    assert count_exactly_k("abab", 2) == 6
    assert count_at_most_k("abab", 1) == 4


# ============================================================
# FOLLOW-UP 3: Zigzag on an n-ary tree, or the right-to-left variant
# ============================================================
# Answer: for an n-ary tree loop over node.children instead of left and
# right. For the variant that starts right-to-left, begin with
# left_to_right = False in zigzag().


# ============================================================
# RUN EVERYTHING
```

```
# ====================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

217 Validate course catalog dependencies — MEDIUM

Template fit: first reject prerequisites naming courses that do not exist; then the same peel; report the missing names or the cycle

QUESTION

You are given a set of course ids and prerequisite pairs (u, v), meaning course v requires course u. The catalog is valid when no prerequisite mentions a course that does not exist and there is no cycle. Return (True, [], order) with a possible study order when valid; (False, missing, None) listing the unknown course ids; or (False, [], cycle) with one cycle that was found.

Function signature

def validate_catalog(courses: set, prereqs: list) -> tuple

EXAMPLES

Example 1	
Input	{1, 2, 3, 4}, [(1, 2), (2, 3), (1, 3)]
Output	(true, [], [1, 4, 2, 3])
Why	Course 1 comes first, 4 is unrelated, then 2 and 3.

Example 2	
Input	{1, 2}, [(1, 9)]
Output	(false, [9], null)
Why	Course 9 is not in the catalog.

Example 3	
Input	{1, 2}, [(1, 2), (2, 1)]
Output	(false, [], [1, 2, 1])
Why	1 requires 2 and 2 requires 1: a cycle.

FULL CODE | main question first, then every follow-up, tests at the end

```
# ====================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ====================================================

from collections import deque


# ====================================================
# MAIN QUESTION
# ====================================================
# ---- main question: helper functions ----
```

```
def find_cycle(known, prereqs, unmet):
    # `unmet` are the counts left after Kahn's algorithm. Every leftover
    # course still has a leftover prerequisite, so walking backwards along
    # prerequisites must eventually repeat a course: that repeat closes a cycle.
    left = set()
    for course in known:
        if unmet[course] > 0:
            left.add(course)
    before = {}                                 # course -> its prerequisites that are leftovers
    for u, v in prereqs:
        if u in left and v in left:
            if v not in before:
                before[v] = []
            before[v].append(u)
    x = min(left, key=str)
    walk = []
    position = {}
    while x not in position:
        position[x] = len(walk)
        walk.append(x)
        x = before[x][0]
    cycle = walk[position[x]:] + [x]
    cycle.reverse()                             # we walked against the edges: list prerequisites first
    return cycle


# ---- main question: main function ----
# Idea: 1) report ids that are used but not in the course set. 2) Run
# Kahn's algorithm: repeatedly take courses with no remaining
# prerequisite, building an order. 3) If courses are left over there is
# a cycle: find one in the leftovers.
```

```
# prereqs are pairs (u, v): course v requires course u. Returns
# (ok, missing ids, order or one cycle).
def validate_catalog(courses, prereqs):
    known = set(courses)
    missing = set()
    for u, v in prereqs:
        if u not in known:
            missing.add(u)
        if v not in known:
            missing.add(v)
    if missing:
        return False, sorted(missing, key=str), None
    unlocks = {}
    unmet = {}
    for c in known:
        unlocks[c] = []
        unmet[c] = 0
    for u, v in prereqs:
        unlocks[u].append(v)
        unmet[v] += 1
    ready = []
    for c in known:
        if unmet[c] == 0:
            ready.append(c)
    queue = deque(sorted(ready, key=str))
    order = []
    while queue:
        c = queue.popleft()
        order.append(c)
        for nxt in unlocks[c]:
            unmet[nxt] -= 1
            if unmet[nxt] == 0:
                queue.append(nxt)
    if len(order) == len(known):
```

```
        return True, [], order
    return False, [], find_cycle(known, prereqs, unmet)


# ---- main question: tests ----
COURSES = ["a", "b", "c", "d"]
RULES = [("a", "b"), ("b", "c"), ("a", "d")]


def test_main_question():
    ok, missing, order = validate_catalog(COURSES, RULES)
```

```
    assert ok and order[0] == "a"
    assert order.index("b") < order.index("c")
    assert validate_catalog(COURSES, RULES + [("zzz", "a")]) == (False, ["zzz"], None)   # an unknown id
    ok, missing, cycle = validate_catalog(COURSES, RULES + [("c", "b")])                 # a cycle b -> c ->
b
    assert not ok and cycle[0] == cycle[-1]
    assert set(cycle) == {"b", "c"}
    assert validate_catalog([], []) == (True, [], [])


# ======================================================
# FOLLOW-UP 1: Return all missing course references and one cycle
# ======================================================
# Answer: validate_catalog above already returns every unknown id (sorted)
# when there are any, and otherwise a cycle: a list of courses that starts
# and ends with the same course.


# ======================================================
# FOLLOW-UP 2: Group courses into semesters (the earliest round each can be taken)
# ======================================================
# ---- follow-up 2: function ----
# Answer: Kahn's algorithm level by level: all courses with no remaining
# prerequisite are round 1; remove them and the next set is round 2, and
# so on. The number of rounds is the longest chain of prerequisites.
def semesters(courses, prereqs):
    unlocks = {}
    unmet = {}
    for c in courses:
        unlocks[c] = []
        unmet[c] = 0
    for u, v in prereqs:
        unlocks[u].append(v)
        unmet[v] += 1
    level = []
    for c in courses:
        if unmet[c] == 0:
            level.append(c)
    level.sort()
    rounds = []
    while level:
        rounds.append(level)
        following = []
        for c in level:
            for v in unlocks[c]:
                unmet[v] -= 1
```

```
                if unmet[v] == 0:
                    following.append(v)
        following.sort()
        level = following
    return rounds
```

```python
# ---- follow-up 2: tests ----
def test_follow_up_2():
    assert semesters(COURSES, RULES) == [["a"], ["b", "d"], ["c"]]


# ========================================================
# FOLLOW-UP 3: Find all cycles, not just one
# ========================================================
# Answer: compute the strongly connected components (Tarjan or
# Kosaraju). Every component with more than one course (or one course
# with an edge to itself) is a group of cycles to report.


# ========================================================
# RUN EVERYTHING
# ========================================================
def run_all_tests():
    test_main_question()
    test_follow_up_2()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

218 Course Schedule

MEDIUM

Template fit: edge = prerequisite -> course; return the order, or an empty list when there is a cycle

QUESTION

You are given numCourses and a list of prerequisites, where each element in prerequisites[i] = [a_i, b_i] means you must complete course b_i before taking course a_i. Write a function canFinish that returns True if it's possible to finish all courses, and False if it's not. You need to complete all the courses for the result to be True. Assume there are no duplicate prerequisites, and numCourses is a non-negative number. Examples Input: numCourses = 3, prerequisites = [[2, 1], [1, 0]]
Output: True Explanation: You can complete course 0 first, then course 1, and finally course 2.
Input: numCourses = 4, prerequisites = [[3, 2], [2, 1], [1, 0], [0, 3]]
Output: False Explanation: The prerequisites form a cycle, making it impossible to finish all courses.
Input: numCourses = 5, prerequisites = [[4, 2], [3, 1], [2, 0]]
Output: True Explanation: You can complete the courses in order from course 0 to course 4.Hint 1Try modeling the courses as a graph, with edges representing prerequisites. Consider how you can detect cycles to determine if all courses can be completed.

EXAMPLES

Example 1	
Input	`2, [[1, 0]]`
Output	`true`
Why	`Course 1 requires course 0: finishing both is possible.`

Example 2	
Input	`2, [[1, 0], [0, 1]]`
Output	`false`
Why	`Course 1 requires 0 and course 0 requires 1: a cycle.`

QUESTION

Example 3	
Input	2, [[1, 0]]
Output	[0, 1]
Why	A valid order: take 0 before 1.

FULL CODE | main question first, then every follow-up, tests at the end

```
# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ========================================================

from collections import deque


# ========================================================
# MAIN QUESTION
# ========================================================
# ---- main question: helper functions ----
def build(n, prereqs):
    # prereqs (a, b): course b must be taken before course a.
    # unlocks[b] = the courses that need b; unmet[a] = how many prerequisites a still waits for
    unlocks = []
    for i in range(n):
        unlocks.append([])
    unmet = [0] * n
    for a, b in prereqs:
        unlocks[b].append(a)
        unmet[a] += 1
    return unlocks, unmet


# ---- main question: main function ----
# Idea (Kahn's algorithm): queue every course with no unmet
# prerequisite. Take one, lower the count of the courses that needed
# it, and queue those that reach zero. If all n courses get taken
# there is no cycle.
def can_finish(n, prereqs):
    unlocks, unmet = build(n, prereqs)
    queue = deque()
    for c in range(n):
        if unmet[c] == 0:
            queue.append(c)
    taken = 0
    while queue:
        c = queue.popleft()
        taken += 1
        for nxt in unlocks[c]:
            unmet[nxt] -= 1
            if unmet[nxt] == 0:
```

```
                queue.append(nxt)
    return taken == n


# ---- main question: tests ----
DIAMOND = [(1, 0), (2, 0), (3, 1), (3, 2)]


def test_main_question():
```

```
    assert can_finish(2, [(1, 0)])
    assert not can_finish(2, [(1, 0), (0, 1)])
    assert can_finish(4, DIAMOND)
    assert can_finish(3, []) and can_finish(0, [])


# ========================================================
# FOLLOW-UP 1: Return the order of courses (LeetCode 210)
# ========================================================
# ---- follow-up 1: function ----
# Answer: the same loop, writing down each course as it is taken; if
# fewer than n were taken return []. O(V + E).
def find_order(n, prereqs):
    unlocks, unmet = build(n, prereqs)
    queue = deque()
    for c in range(n):
        if unmet[c] == 0:
            queue.append(c)
    order = []
    while queue:
        c = queue.popleft()
        order.append(c)
        for nxt in unlocks[c]:
            unmet[nxt] -= 1
            if unmet[nxt] == 0:
                queue.append(nxt)
    if len(order) == n:
        return order
    return []                                       # a cycle blocks the rest


# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert find_order(4, DIAMOND)[0] == 0
    assert find_order(2, [(1, 0), (0, 1)]) == []
```

```
# ========================================================
# FOLLOW-UP 2: The minimum number of semesters if you can take any number of courses per semester
# ========================================================
# ---- follow-up 2: function ----
# Answer: Kahn's algorithm level by level: each level is one semester.
# The number of levels is the longest chain of prerequisites; -1 if a
# cycle stops some courses from ever being taken.
def min_semesters(n, prereqs):
    unlocks, unmet = build(n, prereqs)
    level = []
    for c in range(n):
        if unmet[c] == 0:
            level.append(c)
    semesters = 0
    taken = 0
    while level:
        semesters += 1
        taken += len(level)
        following = []
        for c in level:
            for nxt in unlocks[c]:
                unmet[nxt] -= 1
                if unmet[nxt] == 0:
                    following.append(nxt)
        level = following
    if taken == n:
        return semesters
    return -1
```

```
# ---- follow-up 2: tests ----
def test_follow_up_2():
    assert min_semesters(4, DIAMOND) == 3
    assert min_semesters(2, [(1, 0), (0, 1)]) == -1
    assert min_semesters(3, []) == 1


# ========================================================
# FOLLOW-UP 3: Report an actual cycle
# ========================================================
# ---- follow-up 3: function ----
# Answer: after Kahn's algorithm the courses that are left all still
# wait for a prerequisite that is also left over. Walk backwards along
# prerequisites inside that set until a course repeats: that is a cycle.
```

```
def find_cycle(n, prereqs):
    unlocks, unmet = build(n, prereqs)
    queue = deque()
    for c in range(n):
        if unmet[c] == 0:
            queue.append(c)
    while queue:
        c = queue.popleft()
        for nxt in unlocks[c]:
            unmet[nxt] -= 1
            if unmet[nxt] == 0:
                queue.append(nxt)
    left = set()
    for c in range(n):
        if unmet[c] > 0:
            left.add(c)
    if not left:
        return []
    before = {}                                  # leftover course -> a leftover prerequisite
    for a, b in prereqs:
        if a in left and b in left:
            before[a] = b
    x = min(left)
    walk = []
    position = {}
    while x not in position:
        position[x] = len(walk)
        walk.append(x)
        x = before[x]
    return walk[position[x]:]


# ---- follow-up 3: tests ----
def test_follow_up_3():
    cycle = find_cycle(4, [(1, 0), (2, 1), (1, 2), (3, 2)])
    assert set(cycle) == {1, 2}
    assert find_cycle(2, [(1, 0)]) == []


# ========================================================
# RUN EVERYTHING
# ========================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
```

```
    test_follow_up_2()
    test_follow_up_3()
    print("all tests passed")
```

```
if __name__ == "__main__":
    run_all_tests()
```

GROUP 30 OF 43 | 5 QUESTIONS, ONE CODE TEMPLATE | 4 M 1 H

Counting pairs by looking up each partner in a Counter or a bucket

219 Progressive Array Partitioning — MEDIUM

Template fit: Counter of the values; stage 1 asks whether all the counts are different: len(set(counts.values())) == len(counts)

QUESTION

The problem is given in stages; stage 1 is: group the equal values of an array together. Return true when every group has a different size (no two values occur the same number of times). The function takes the occurrence count of each value as a dict, so counting stays separate from the rule and later stages can change only the rule.

Function signature

def stage1(counts: dict) -> bool

counts maps value -> how

many times it occurs

EXAMPLES

Example 1

Input	`from collections import Counter`
	`nums = [1, 2, 3, 4, 2, 3, 3, 4, 4, 4]`
	`stage1(Counter(nums))`

Output	`stage1(Counter(nums)) -> true`
Why	`The groups have sizes 1, 2, 3, 4: all different, so true.`

Example 2

Input	`from collections import Counter`
	`nums = [1, 2, 3]`
	`stage1(Counter(nums))`

Output	`stage1(Counter(nums)) -> false`
Why	`Every value occurs once, so all three groups have size 1: false.`

FULL CODE | main question first, then every follow-up, tests at the end

```
# ====================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ====================================================

# ====================================================
# MAIN QUESTION
# ====================================================
# ---- main question: helper functions ----
def count_values(nums):
    # {value: how many times it occurs}
    counts = {}
    for x in nums:
        counts[x] = counts.get(x, 0) + 1
    return counts


# ---- main question: main function ----
# Stage 1. `counts` maps value -> how many times it occurs. The array can be
# split as the stage asks exactly when EVERY value occurs a different number
# of times (no two values share a count).
```

```python
# Idea: the counts have no repeats exactly when the SET of counts has as many
# elements as there are distinct values. One pass to count, one over the counts.
def stage1(counts):
    return len(set(counts.values())) == len(counts)


# ---- main question: tests ----
def check(stage, nums):
    # each stage is one small function, so a new rule replaces one branch, not the whole pipeline
    if stage == 1:
        return stage1(count_values(nums))
    return stage2_candidate(nums)


def test_main_question():
    assert check(1, [1, 2, 3, 4, 2, 3, 3, 4, 4, 4])
    assert not check(1, [1, 2, 3])


# ========================================================
# FOLLOW-UP 1: Stage 2 arrives. What do you do?
# ========================================================
```

```python
# ---- follow-up 1: function ----
# Answer: do not guess. Write down what the second example pair implies and
# check any candidate rule against BOTH examples. One reading of stage 2
# (LeetCode 659): split the array into runs of CONSECUTIVE values, each at
# least min_len long. Greedy: take the values in order; put x on a run that
# ended at x - 1 if there is one, otherwise start a new run x, x + 1, x + 2.
def stage2_candidate(nums, min_len=3):
    left = count_values(nums)                    # copies of each value not yet placed
    ends = {}                                    # value -> how many runs currently end at it
    for x in nums:
        if left.get(x, 0) == 0:
            continue
        left[x] -= 1
        if ends.get(x - 1, 0) > 0:
            ends[x - 1] -= 1                     # extend a run that ended at x - 1
            ends[x] = ends.get(x, 0) + 1
        elif left.get(x + 1, 0) > 0 and left.get(x + 2, 0) > 0:
            left[x + 1] -= 1                     # start a new run x, x + 1, x + 2
            left[x + 2] -= 1
            ends[x + 2] = ends.get(x + 2, 0) + 1
        else:
            return False
    return True


# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert check(2, [1, 2, 2, 3, 3, 4, 4, 5, 5, 6])      # the stage 2 example the candidate gets right
    # the second stage-2 example says False, but this reading says True: the rule is incomplete, so ask
    assert check(2, [4, 5, 6, 7, 7, 8, 9]) is True


# ========================================================
# FOLLOW-UP 2: Keep the code adaptable between stages
# ========================================================
# Answer: make each stage ONE small function over the same shared helper
# (count_values), and pick the stage in one place (check): a new rule then
# replaces one branch instead of the whole pipeline.


# ========================================================
# RUN EVERYTHING
# ========================================================
```

```
def run_all_tests():
    test_main_question()
```

```
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

220 K-diff Pairs in an Array

MEDIUM

Template fit: the template as written: partner = v + k; for k == 0 count values that appear twice

QUESTION

Given an integer array nums and an integer k , return the number of unique k-diff pairs in the array. A k-diff pair is a pair of integers (nums[i], nums[j]) where: i != j , and abs(nums[i] - nums[j]) == k . Two k-diff pairs (a, b) and (c, d) are considered the same (counted once) if they consist of the same pair of values. Count each distinct value pair only once.

Function signature

```
def count_k_diff_pairs(nums: list[int], k: int) -> int:
    """Return the number of unique value-pairs with absolute difference k."""
    ...
```

EXAMPLES

Example 1	
Input	`nums = [3, 1, 4, 1, 5], k = 2`
Output	`2`
Why	`The unique 2-diff value pairs are (1, 3) and (3, 5).`

Example 2	
Input	`nums = [1, 2, 3, 4, 5], k = 1`
Output	`4`
Why	`(1,2), (2,3), (3,4), (4,5).`

Example 3	
Input	`nums = [1, 3, 1, 5, 4], k = 0`
Output	`1`
Why	`With k = 0 count values that appear at least twice; only 1 does.`

CONSTRAINTS

- 1 <= len(nums) <= 10^4
- -10^7 <= nums[i] <= 10^7
- 0 <= k <= 10^7

FULL CODE | main question first, then every follow-up, tests at the end

```
# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================
```

```python
import random


# ====================================================
# MAIN QUESTION
# ====================================================
# ---- main question: main function ----
# Count the DISTINCT pairs of values (x, x + k) that both appear in the array.
# Idea: count the values in a dictionary. A value v starts a pair when
# v + k is also present; for k = 0 the same value must appear at least twice.
def count_k_diff_pairs(nums, k):
    if k < 0:
        return 0
    counts = {}
    for x in nums:
        counts[x] = counts.get(x, 0) + 1
    pairs = 0
    for v in counts:
        if k == 0:
            if counts[v] >= 2:
                pairs += 1
        elif v + k in counts:
            pairs += 1
    return pairs


# ---- main question: tests ----
def test_main_question():
    assert count_k_diff_pairs([3, 1, 4, 1, 5], 2) == 2
    assert count_k_diff_pairs([1, 2, 3, 4, 5], 1) == 4
    assert count_k_diff_pairs([1, 3, 1, 5, 4], 0) == 1


# ====================================================
# FOLLOW-UP 1: Do it without a hash map
# ====================================================
# ---- follow-up 1: function ----
# Answer: sort, then use two pointers: move the right pointer on when the
```

```python
# difference is too small and the left pointer when it is too big; when the
# difference equals k count it and skip duplicate values on the left side.
# O(n log n) time, O(1) extra space.
def count_k_diff_sorted(nums, k):
    nums = sorted(nums)
    left = 0
    right = 1
    pairs = 0
    while right < len(nums) and left < len(nums):
        difference = nums[right] - nums[left]
        if left == right or difference < k:
            right += 1
        elif difference > k:
            left += 1
        else:
            pairs += 1
            left += 1
            while left < len(nums) and nums[left] == nums[left - 1]:
                left += 1                           # the same value cannot start another pair
    return pairs


# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert count_k_diff_sorted([3, 1, 4, 1, 5], 2) == 2
    assert count_k_diff_sorted([1, 3, 1, 5, 4], 0) == 1
    random.seed(8)
```

```
    for round_number in range(300):
        nums = []
        for i in range(random.randint(0, 9)):
            nums.append(random.randint(0, 6))
        k = random.randint(0, 3)
        assert count_k_diff_sorted(nums, k) == count_k_diff_pairs(nums, k)


# =====================================================
# RUN EVERYTHING
# =====================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
```

```
    run_all_tests()
```

221 Count Axis-Aligned Squares Among Grid Points

MEDIUM

Template fit: a set of points; two opposite corners fix the other two, so look both up; every square is found twice, divide by 2

QUESTION

The interviewer described a diagram of squares and rectangles and asked which data structure to use and how to count the squares. Framed concretely: you are given a list of distinct integer lattice points in the plane. Count the number of axis-aligned squares whose four corners all appear in points . A square is axis-aligned when its sides are parallel to the x- and y-axes. Return the count of such squares.

Function signature

from typing import List, Tuple
def count_axis_aligned_squares(points: List[Tuple[int, int]]) -> int:
    """Return the number of axis-aligned squares whose 4 corners are all in points."""
    ...

Follow-up (as reported): A cleaner O(n^2) approach: put all points in a set, then for every pair that could be two opposite corners of a square (equal absolute x- and y-difference), check the other two corners; each square is found via both its diagonals, so divide the total by 2.

EXAMPLES

Example 1	
Input	`points = [(0, 0), (0, 1), (1, 0), (1, 1)]`
Output	`1`
Why	`The four points form exactly one unit square.`

Example 2	
Input	`points = [(0,0),(0,1),(1,0),(1,1),(0,2),(1,2),(2,0),(2,1),(2,2)]`
Output	`5`
Why	`A 3x3 grid of points has four 1x1 squares and one 2x2 square.`

QUESTION

Example 3	
Input	`points = [(0, 0), (2, 0), (0, 1), (2, 1)]`
Output	`0`
Why	`These form a 2x1 rectangle, which is not a square.`

CONSTRAINTS

- 0 <= len(points) <= 2000 ; all points are distinct.
- Each point is (x, y) with -10^9 <= x, y <= 10^9 .
- A square must have positive side length (four distinct corners); degenerate zero-size "squares" do not count.
- Only axis-aligned squares count (tilted squares are ignored).

FULL CODE | main question first, then every follow-up, tests at the end

```
# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ========================================================

# ========================================================
# MAIN QUESTION
# ========================================================
# ---- main question: main function ----
# Count the squares with sides parallel to the axes whose four corners are
# all among the points.
# Idea: two OPPOSITE corners decide the other two, so one set lookup per pair
# is enough: for points (x1, y1) and (x2, y2) with |x1 - x2| == |y1 - y2| != 0,
# the other corners are (x1, y2) and (x2, y1). Every square has two diagonals,
# so it is found twice: halve the total.
def count_axis_aligned_squares(points):
    present = set(points)
    total = 0
    for i in range(len(points)):
        x1, y1 = points[i]
        for j in range(i + 1, len(points)):
            x2, y2 = points[j]
            if abs(x1 - x2) == abs(y1 - y2) and x1 != x2:
                if (x1, y2) in present and (x2, y1) in present:
                    total += 1
    return total // 2


# ---- main question: tests ----
def test_main_question():
    assert count_axis_aligned_squares([(0, 0), (0, 1), (1, 0), (1, 1)]) == 1
    grid = []
    for x in range(3):
        for y in range(3):
            grid.append((x, y))
    assert count_axis_aligned_squares(grid) == 5               # four 1 x 1 squares and one 2 x 2
    assert count_axis_aligned_squares([(0, 0), (2, 0), (0, 1), (2, 1)]) == 0
    assert count_axis_aligned_squares([]) == 0


# ========================================================
# FOLLOW-UP 1: Points may repeat. Count every distinct choice of four points
# ========================================================
```

```
# ---- follow-up 1: function ----
# Answer: keep a COUNT per point. For each pair of distinct diagonal points,
```

```python
# multiply the number of copies of all four corners; halve the total
# because each square has two diagonals.
def count_squares_with_duplicates(points):
    copies = {}
    for point in points:
        copies[point] = copies.get(point, 0) + 1
    distinct = list(copies)
    total = 0
    for i in range(len(distinct)):
        x1, y1 = distinct[i]
        for j in range(i + 1, len(distinct)):
            x2, y2 = distinct[j]
            if abs(x1 - x2) == abs(y1 - y2) and x1 != x2:
                total += copies[(x1, y1)] * copies[(x2, y2)] * copies.get((x1, y2), 0) * copies.get((x2, y1),
 0)
    return total // 2


# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert count_squares_with_duplicates([(0, 0), (0, 1), (1, 0), (1, 1)]) == 1
    assert count_squares_with_duplicates([(0, 0), (0, 0), (0, 1), (1, 0), (1, 1)]) == 2     # two copies of o
ne corner


# ======================================================
# RUN EVERYTHING
# ======================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

222 Sum of Two Squares in Two Ways

MEDIUM

*Template fit: enumerate every pair (a, b) with a <= b once, add a*a + b*b to a Counter, answer = sums counted exactly twice*

QUESTION

Given an integer n, find all integers s < n that can be written as a sum of two positive squares in exactly two different ways (the order of the two squares does not matter). For example 50 = 1^2 + 7^2 = 5^2 + 5^2. Return them in increasing order.

Function signature

def two_squares_two_ways(n: int) -> list

EXAMPLES

Example 1	
Input	`100`
Output	`[50, 65, 85]`
Why	`50 = 1 + 49 = 25 + 25 and 65 = 1 + 64 = 16 + 49 both have exactly two ways.`

Example 2	
Input	`50`
Output	`[]`
Why	`Nothing below 50 qualifies.`

CONSTRAINTS

QUESTION

- 1 <= n <= 2^31 - 1

FULL CODE | main question first, then every follow-up, tests at the end

```
# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ========================================================

# ========================================================
# MAIN QUESTION
# ========================================================
# ---- main question: main function ----
# Find every number s < n that can be written as a sum of two positive
# squares a^2 + b^2 (with a <= b) in EXACTLY two different ways.
# Idea: loop over every pair a <= b with a^2 + b^2 < n (about pi * n / 8
# pairs) and count how often each sum appears in a dictionary. The answer is
# the sorted list of the sums whose count is exactly 2.
# Follow-up 1: exactly k ways: compare with k instead of 2 (for k = 3 the
# smallest is 325). Follow-up 2: zero=True starts a at 0, so perfect squares
# like 25 = 0^2 + 5^2 gain a way: check the examples to see which convention the
# question uses (here: positive only).
def two_squares_two_ways(n, k=2, zero=False):
    ways = {}
    a = 1
    if zero:
        a = 0
    while 2 * a * a < n:
        b = a
        while a * a + b * b < n:
            total = a * a + b * b
            ways[total] = ways.get(total, 0) + 1
            b += 1
        a += 1
    sums = []
    for total in ways:
        if ways[total] == k and total > 0:
            sums.append(total)
    sums.sort()
    return sums


# ---- main question: tests ----
def test_main_question():
    assert two_squares_two_ways(100) == [50, 65, 85]
    assert two_squares_two_ways(10) == []
    assert 25 not in two_squares_two_ways(100)
```

```
# ========================================================
# FOLLOW-UP 1: Exactly k ways instead of two
# ========================================================
# Answer: the parameter k: change the comparison from == 2 to == k; the
# counting loop is the same.
# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert two_squares_two_ways(1000, 3)[0] == 325            # 325 = 1 + 324 = 36 + 289 = 100 + 225


# ========================================================
# FOLLOW-UP 2: Allow zero (0^2 + b^2)
# ========================================================
```

```
# Answer: start a at 0 (the parameter zero=True): then perfect squares like
# 25 gain a representation, so some answers appear or disappear.
# ---- follow-up 2: tests ----
def test_follow_up_2():
    assert 25 in two_squares_two_ways(100, 2, zero=True)


# ============================================================
# FOLLOW-UP 3: n is huge (10^12) and you only need to test one number
# ============================================================
# ---- follow-up 3: helper functions ----
def integer_sqrt(m):
    # the biggest whole number r with r * r <= m (binary search)
    low = 0
    high = m + 1
    while low + 1 < high:
        middle = (low + high) // 2
        if middle * middle <= m:
            low = middle
        else:
            high = middle
    return low


def factorise(m):
    # {prime: exponent}; trial division is fine up to about 10^12
    factors = {}
    p = 2
    while p * p <= m:
        while m % p == 0:
```

```
            factors[p] = factors.get(p, 0) + 1
            m = m // p
        p += 1
    if m > 1:
        factors[m] = factors.get(m, 0) + 1
    return factors


# ---- follow-up 3: function ----
# Answer: factorise the number. The number of ways to write it as a sum of
# two squares depends only on the primes that are 1 mod 4 (Jacobi): a prime
# that is 3 mod 4 with an odd exponent makes it impossible, and with
# B = the product of (exponent + 1) over the primes 1 mod 4 there are 4B ordered
# signed pairs (x, y). Take away the special shapes a == 0 (m is a square) and
# a == b (m is twice a square); every other unordered pair is counted 8 times.
def representations(m, zero=False):
    if m <= 0:
        return 0
    boost = 1
    factors = factorise(m)
    for p in factors:
        if p % 4 == 3 and factors[p] % 2 == 1:
            return 0
        if p % 4 == 1:
            boost *= factors[p] + 1
    is_square = integer_sqrt(m) ** 2 == m
    half = m % 2 == 0 and integer_sqrt(m // 2) ** 2 == m // 2
    special = 0
    if is_square:
        special += 4
    if half:
        special += 4
    result = (4 * boost - special) // 8
    if half:
        result += 1                                    # the pair a == b
```

```
    if zero and is_square:
        result += 1                                  # the pair 0, sqrt(m)
    return result


# ---- follow-up 3: tests ----
def test_follow_up_3():
    assert representations(50) == 2
    assert representations(325) == 3
    assert representations(3) == 0
```

```
    assert representations(25) == 1
    assert representations(25, zero=True) == 2
    assert representations(5 ** 5 * 13 * 17 * 29) > 10        # a 10 digit number in a moment
    small = two_squares_two_ways(100)
    for m in range(1, 100):
        assert (m in small) == (representations(m) == 2)      # the formula agrees with the loops


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

223 Maximum-Area Rectangle from Planar Points

HARD

Template fit: bucket every pair of points by (sum of coordinates, squared length); two pairs in one bucket are the diagonals of a rectangle

QUESTION

Given 2D points, find the maximum area of a rectangle whose four corners are input points; sides need not be parallel to the axes. Candidates compare it to LC 963 (Minimum Area Rectangle II) but maximizing. Interviewers ask to improve beyond an O(N^3) approach.

Input: a list of 2D coordinate points.
Return the maximum area of any rectangle whose four vertices are all input points.
Rectangle sides do not need to be parallel to the coordinate axes.
Follow-up pressure: start from an O(N^3) idea, then optimize toward a better complexity.

EXAMPLES

Example 1	
Input	`[(1, 1), (1, 3), (3, 1), (3, 3)]`
Output	`4`
Why	`The four points form a 2 x 2 axis-aligned square.`

Example 2	
Input	`[(0, 1), (1, 0), (2, 1), (1, 2)]`
Output	`2`
Why	`A tilted rectangle: a square with diagonal 2 has area 2.`

QUESTION

Example 3	
Input	`[(0, 0), (1, 0), (0, 1)]`
Output	`0`
Why	Three points never make a rectangle.

FULL CODE | main question first, then every follow-up, tests at the end

```
# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ========================================================

# ========================================================
# MAIN QUESTION
# ========================================================
# ---- main question: helper functions ----
def cross(o, p, q):
    # twice the signed area of the triangle o, p, q
    ax = p[0] - o[0]
    ay = p[1] - o[1]
    bx = q[0] - o[0]
    by = q[1] - o[1]
    return ax * by - ay * bx


def all_rectangle_areas(points):
    # the area of every rectangle that the points form (the sides need not be
    # parallel to the axes). Idea: the two diagonals of a rectangle have the
    # SAME middle and the SAME length, so put every pair of points in a bucket
    # keyed by (sum of x, sum of y, squared length); any two pairs in one bucket
    # are the diagonals of a rectangle.
    unique = []
    seen = set()
    for x, y in points:
        if (x, y) not in seen:                          # duplicates cannot help
            seen.add((x, y))
            unique.append((x, y))
    buckets = {}
    for i in range(len(unique)):
        for j in range(i + 1, len(unique)):
            p = unique[i]
            q = unique[j]
            key = (p[0] + q[0], p[1] + q[1], (p[0] - q[0]) ** 2 + (p[1] - q[1]) ** 2)
            if key not in buckets:
                buckets[key] = []
            buckets[key].append((p, q))
    areas = []
    for pairs in buckets.values():
        for a in range(len(pairs)):
            for b in range(a + 1, len(pairs)):
```

```
                p, q = pairs[a]
                r, s = pairs[b]
                area = cross(p, r, s)                   # the two sides leaving p are p-r and p-s: their cross
 product is the rectangle's area
                if area < 0:
                    area = -area
                areas.append(area)
    return areas
```

```
# ---- main question: main function ----
# The biggest rectangle (any angle) with corners among the points, or 0.
def max_rect_area(points):
    best = 0
    for area in all_rectangle_areas(points):
        if area > best:
            best = area
    return best


# the related variant: the SMALLEST area (LeetCode 963)
def min_rect_area(points):
    smallest = 0
    for area in all_rectangle_areas(points):
        if smallest == 0 or area < smallest:
            smallest = area
    return smallest


# ---- main question: tests ----
def test_main_question():
    square = [(0, 0), (1, 1), (0, 2), (-1, 1)]                # a tilted square of area 2
    assert max_rect_area(square) == 2
    rect = [(0, 0), (2, 0), (0, 1), (2, 1), (5, 5)]           # a 2 x 1 rectangle plus a stray point
    assert max_rect_area(rect) == 2
    assert max_rect_area([(0, 0), (1, 1), (2, 2)]) == 0       # collinear points make no rectangle
    two = rect + [(0, 3), (2, 3)]
    assert min_rect_area(two) == 2                            # the minimum version picks the small one
    assert max_rect_area(two) == 6


# ============================================================
# FOLLOW-UP 1: Can you do better than O(n^2) pairs?
# ============================================================
# Answer: not in general. The answer can depend on every pair of points, and
# even deciding whether ANY rectangle exists is believed to need about n^2
```

```
# work, so the bucket method is the practical optimum.


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

GROUP 31 OF 43 | 5 QUESTIONS, ONE CODE TEMPLATE | 3 M 2 H

Tree DP: build a parent order once, then add up from the leaves

224 Compute File System Total Size — MEDIUM

Template fit: add each file's size while walking down from the root with a stack; a shared directory is counted once (seen set)

QUESTION

A file system is a tree: File(size) is a file and Dir(*children) is a directory holding files and directories. Return the total size of all files. A directory object can be shared (it appears under more than one parent); its files are counted once.

Function signature

def total_size(root) -> int

EXAMPLES

Example 1	
Input	`root = Dir(Dir(File(10), File(20)), File(5), Dir(File(30)))` `total_size(root)`
Output	`total_size(root) -> 65`
Why	`10 + 20 + 5 + 30 = 65.`

Example 2	

Input	`shared = Dir(File(7))`
	`root = Dir(shared, shared, File(1))`
	`total_size(root)`

Output	`total_size(root) -> 8`
Why	`The same directory appears twice but its 7 is counted once: 7 + 1 = 8.`

FULL CODE | main question first, then every follow-up, tests at the end

```
# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
class File:
    def __init__(self, size):
        self.size = size
        self.parent = None


class Dir:
    def __init__(self, *children):
        self.children = list(children)
        self.parent = None
        self.cached = None                      # the total size; None means "needs recomputing"
        for child in children:
            child.parent = self


# ---- main question: main function ----
# Idea: an explicit stack removes the recursion limit, and a set of the
```

```
# nodes already seen makes each one count once, so a directory that is
# shared (hard links, follow-up 2) is not counted twice and a cycle
# cannot loop forever. count_once=False counts a shared file once per path.
def total_size(root, count_once=True):
    total = 0
    seen = set()
    stack = [root]
    while stack:
        node = stack.pop()
        if count_once:
            if node in seen:
                continue
            seen.add(node)
        if isinstance(node, File):
            total += node.size
        else:
            for child in node.children:
```

```
                stack.append(child)
    return total


# ---- main question: tests ----
def example_tree():
    inner = Dir(File(10), File(20))
    return inner, Dir(inner, File(5), Dir(File(30)))


def test_main_question():
    inner, root = example_tree()
    assert total_size(root) == 65


# =====================================================
# FOLLOW-UP 1: The tree is queried often and files change
# =====================================================
# ---- follow-up 1: helper functions ----
def size_of(node):
    # a file's size, or a directory's cached total
    if isinstance(node, File):
        return node.size
    return node.cached


def cached_total(root):
    # fill in the cached total of every directory that needs it, children first
    order = []
    stack = [root]
    while stack:
        node = stack.pop()
        order.append(node)
        if isinstance(node, Dir) and node.cached is None:
            for child in node.children:
                stack.append(child)
    for i in range(len(order) - 1, -1, -1):       # children before parents
        node = order[i]
        if isinstance(node, Dir) and node.cached is None:
            total = 0
            for child in node.children:
                total += size_of(child)
            node.cached = total
    return root.cached
```

```
# ---- follow-up 1: function ----
# Answer: cache every directory's total. When a file changes, clear the
```

```
# cache only of its ancestors (follow the parent links); the next query
# recomputes just those. An update costs O(depth).
def resize(file, size):
    file.size = size
    node = file.parent
    while node is not None:
        node.cached = None
        node = node.parent


# ---- follow-up 1: tests ----
def test_follow_up_1():
    inner, root = example_tree()
    assert cached_total(root) == 65
    resize(inner.children[0], 15)                  # only the ancestors are recomputed
    assert root.cached is None and inner.cached is None
    assert cached_total(root) == 70


# ========================================================
# FOLLOW-UP 2: Hard links: the same file appears under two directories
# ========================================================
# Answer: count a file once per identity (the `seen` set, the default) if
# the question asks for disk usage, but once per path if it asks for the
# sum of the sizes shown. Ask which one is meant.
# ---- follow-up 2: tests ----
def test_follow_up_2():
    shared = File(7)
    both = Dir(Dir(shared), Dir(shared))
    assert total_size(both) == 7
    assert total_size(both, count_once=False) == 14


# ========================================================
# FOLLOW-UP 3: Find the largest directory
# ========================================================
# ---- follow-up 3: function ----
# Answer: the totals come from the same post-order pass, so the largest
# directory is just the maximum. For the top K directories, sort the
# totals and take K (or keep a heap of size K: O(n log K)).
def directory_totals(root):
    cached_total(root)
```

```
    totals = []
    stack = [root]
    while stack:
        node = stack.pop()
        if isinstance(node, Dir):
            totals.append(node.cached)
            for child in node.children:
                stack.append(child)
    return totals


def top_k_directories(root, k):
    totals = directory_totals(root)
    totals.sort(reverse=True)
    return totals[:k]


# ---- follow-up 3: tests ----
def test_follow_up_3():
    inner, root = example_tree()
    assert top_k_directories(root, 2) == [65, 30]      # 65 for the root, 30 for inner (or the last Dir)
    assert directory_totals(root).count(30) == 2       # inner (10 + 20) and Dir(File(30))
```

```
# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

225 Minimum-Cost Tree Disconnection

MEDIUM

Template fit: backwards sweep with two numbers per node: the best cost when the node is cut off, and the cost if it is free

QUESTION

A rooted tree is given as directed edges (parent, child, cost) with node 0 as the root. Cut edges so that no path from the root to any original leaf remains. The price of a cut set is the sum of the costs of the edges cut (costs may be negative). Return the minimum total price.

Function signature

def disconnect_cost(n: int, edges: list) -> int

EXAMPLES

Example 1	
Input	`3, [(0, 1, 3), (0, 2, 2)]`
Output	`5`
Why	`Two leaves hang off the root: both edges must be cut, 3 + 2 = 5.`

Example 2	
Input	`3, [(0, 1, 4), (1, 2, 1)]`
Output	`1`
Why	`A chain 0 - 1 - 2: cutting the cheaper edge 1 -> 2 (cost 1) already separates the leaf.`

Example 3	
Input	`3, [(0, 1, -1), (1, 2, -1)]`
Output	`-2`
Why	`Negative costs: cutting both edges costs -2, which beats cutting only one.`

FULL CODE | main question first, then every follow-up, tests at the end

```
# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

INF = float("inf")
```

```
# ========================================================
# MAIN QUESTION
# ========================================================
# ---- main question: helper functions ----
def tree_costs(n, edges):
    # edges are (parent, child, cost); node 0 is the root.
    # best[v] = the cheapest set of cuts that separates v from every leaf below it
    # free[v] = the sum of all NEGATIVE costs below v (cutting those too only helps)
    kids = []
    for i in range(n):
        kids.append([])
    for u, v, w in edges:
        kids[u].append((v, w))
    order = [0]                                  # parents before children (breadth first)
    for u in order:                              # (a loop, not recursion: safe on a long chain)
        for v, w in kids[u]:
            order.append(v)
    best = [0] * n
    free = [0] * n
    for i in range(len(order) - 1, -1, -1):      # children before parents
        v = order[i]
        for c, w in kids[v]:
            cut = w + free[c]                    # option 1: cut this edge (and the negatives below)
            if kids[c]:
                keep = best[c]                   # option 2: keep it and cut deeper
            else:
                keep = INF                       # a leaf has nothing deeper to cut
            if cut < keep:
                best[v] += cut
            else:
                best[v] += keep
            if w < 0:
                free[v] += w
            free[v] += free[c]
    return kids, best, free
```

```
# ---- main question: main function ----
# Idea: for a child c reached by an edge of weight w there are two
# options: cut that edge (pay w, plus the negative edges you can still
# cut below c) or keep it and cut deeper (best(c)), which is impossible
# if c is a leaf. Add up the cheaper option over all children of the root.
# For non-negative costs this is simply: a leaf child costs w, any other
# child costs min(w, best(child)).
def disconnect_cost(n, edges):
    kids, best, free = tree_costs(n, edges)
    return best[0]


# ---- main question: tests ----
def test_main_question():
    negative = [(0, 1, -1), (1, 2, -1)]                  # the chain A -(-1)-> B -(-1)-> C
    assert disconnect_cost(3, negative) == -2
    chain = [(0, 1, 5), (1, 2, 3)]
    assert disconnect_cost(3, chain) == 3                # non-negative costs: cut the cheaper edge
    assert disconnect_cost(1, []) == 0                   # a single node needs no cut
    big = []
    for i in range(5000):
        big.append((i, i + 1, 1))
    assert disconnect_cost(5001, big) == 1               # a long chain does not overflow the stack


# ========================================================
# FOLLOW-UP 1: Weights are non-negative
# ========================================================
```

```
# Answer: then free(c) is always 0 and the rule becomes: best(v) = the sum
# over children of (w if the child is a leaf, else min(w, best(child))).


# ======================================================
# FOLLOW-UP 2: The tree is a long chain and recursion overflows
# ======================================================
# Answer: tree_costs handles the nodes in reverse breadth-first order
# (children before parents) and stores best and free in arrays, so there is
# no recursion at all. Space O(n).


# ======================================================
# FOLLOW-UP 3: Return which edges to cut
# ======================================================
```

```
# ---- follow-up 3: function ----
# Answer: at each child remember which option was cheaper, then walk
# down from the root collecting the cut edges. When an edge is cut, the
# negative edges below it are cut as well.
def edges_to_cut(n, edges):
    kids, best, free = tree_costs(n, edges)
    cuts = []
    stack = [0]
    while stack:
        v = stack.pop()
        for c, w in kids[v]:
            if kids[c]:
                keep = best[c]
            else:
                keep = INF
            if w + free[c] <= keep:
                cuts.append((v, c))                    # cut here ...
                below = [c]
                while below:                           # ... plus every negative edge below c
                    x = below.pop()
                    for y, wy in kids[x]:
                        if wy < 0:
                            cuts.append((x, y))
                        below.append(y)
            else:
                stack.append(c)
    return cuts


# ---- follow-up 3: tests ----
def test_follow_up_3():
    assert sorted(edges_to_cut(3, [(0, 1, -1), (1, 2, -1)])) == [(0, 1), (1, 2)]
    assert edges_to_cut(3, [(0, 1, 5), (1, 2, 3)]) == [(1, 2)]


# ======================================================
# RUN EVERYTHING
# ======================================================
def run_all_tests():
    test_main_question()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
```

```
    run_all_tests()
```

226	**Given a large n-ary tree where the root node can pass information to one child per iteration, and in subsequent iterations, nodes can pass information to one child while the root continues with others, find the minimum number of iterations to reach all nodes.**	MEDIUM

Template fit: t[v] = max over children (rank among children sorted by time, longest first, + t[child]); read t at the root

QUESTION

Information starts at the root of a tree. In every iteration a node that already has the information can pass it to one of its children (the tree maps each node to its children as (child, cost) pairs; in the basic problem every cost is 1 iteration). Return the minimum number of iterations until every node has the information.

Function signature

def min_rounds(tree: dict, root) -> int

EXAMPLES

Example 1	
Input	`{'r': [('a', 1), ('b', 1), ('c', 1)], 'a': [('d', 1)]}, 'r'`
Output	`3`
Why	`The root informs a, b and c in iterations 1, 2, 3, while a informs d in iteration 2: everyone knows after 3 iterations.`

Example 2	
Input	`{'r': [('a', 1)], 'a': [('b', 1)]}, 'r'`
Output	`2`
Why	`A path r - a - b needs one iteration per edge.`

FULL CODE | main question first, then every follow-up, tests at the end

```
# ====================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ====================================================

# ====================================================
# MAIN QUESTION
# ====================================================
# ---- main question: helper functions ----
def unit_tree(children):
    # turn {"r": ["a", "b"]} into the form the question uses: every call costs 1 round
    tree = {}
    for node in children:
        tree[node] = []
        for child in children[node]:
            tree[node].append((child, 1))
    return tree


def finish_times(tree, root):
    # time[v] = rounds until v and everything below it has the news, once v has it.
    # v calls its children one after another; the call to the i-th child costs the
    # calls made before it, and that child then needs time[child] more rounds.
    # Best order: the child that needs the LONGEST first.
    order = []
    stack = [root]
    while stack:                                    # parents before children
        node = stack.pop()
        order.append(node)
```

```
        for child, cost in tree.get(node, []):
            stack.append(child)
    time = {}
    plan = {}
    for i in range(len(order) - 1, -1, -1):        # children before parents
        node = order[i]
        kids = []
        for index, pair in enumerate(tree.get(node, [])):
            child, cost = pair
            kids.append((-time[child], index, child, cost))   # sorts the slowest child first
        kids.sort()
        spent = 0
        worst = 0
        order_of_calls = []
```

```
        for slowness, index, child, cost in kids:
            spent += cost                          # the call to this child is finished at `spent`
            if spent + time[child] > worst:
                worst = spent + time[child]
            order_of_calls.append(child)
        time[node] = worst
        plan[node] = order_of_calls
    return time, plan


# ---- main question: main function ----
# Idea: let T(v) be the rounds needed to inform the subtree of v once v
# knows. v informs its children one per round, so serve the child with
# the BIGGEST T first (the biggest tail should start earliest) and
# T(v) = the latest finish over its children. Answer: T(root).
def min_rounds(tree, root):
    time, plan = finish_times(tree, root)
    return time[root]


# ---- main question: tests ----
def test_main_question():
    star = unit_tree({"r": ["a", "b", "c"]})             # a root with three leaves needs 3 rounds
    assert min_rounds(star, "r") == 3
    assert min_rounds({}, "r") == 0
    deep = unit_tree({"r": ["a", "b"], "b": ["c", "d"]})
    assert min_rounds(deep, "r") == 3                     # serve the child with the deeper subtree first
    assert finish_times(deep, "r")[1]["r"] == ["b", "a"]
    example = {"r": [("a", 1), ("b", 1), ("c", 1)], "a": [("d", 1)]}
    assert min_rounds(example, "r") == 3
    chain = {}
    for i in range(20000):
        chain[i] = [(i + 1, 1)]
    assert min_rounds(chain, 0) == 20000                  # a very deep tree needs no recursion


# ======================================================
# FOLLOW-UP 1: Why is it optimal to serve the child with the largest subtree time first?
# ======================================================
# Answer: an exchange argument. If a slow child is served after a fast
# one, swapping them never makes the slow child finish later, and the fast
# child still finishes no later than the slow child's old finish.


# ======================================================
```

```
# FOLLOW-UP 2: What if each edge has a different transmission cost?
# ======================================================


# ======================================================
```

```python
# FOLLOW-UP 3: Each call takes a different time per child
# ========================================================
# ---- follow-up 2: tests ----
# Answer: finish_times already adds the cost of each call: the child in
# position i can only start after the costs of the calls before it, so
# T(v) = max over i of (cost of calls up to child i + T(child i)). The
# best order is still "largest T first" (Jackson's rule).
def test_follow_up_2():
    costly = {"r": [("a", 5), ("b", 1)], "b": [("c", 1)]}
    assert min_rounds(costly, "r") == 6


# ========================================================
# RUN EVERYTHING
# ========================================================
def run_all_tests():
    test_main_question()
    test_follow_up_2()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

227 Compute distance-sum from every tree node

HARD

Template fit: down[v] = sum of distances to v's subtree; then a forward sweep: ans[child] = ans[parent] - size[child] + (n - size[child])

QUESTION

A tree has n nodes numbered 0 to n - 1 and n - 1 edges (u, v). For every node, return the sum of the distances from that node to all the other nodes.

Function signature

def distance_sums(n: int, edges: list) -> list

EXAMPLES

Example 1	
Input	`4, [(0, 1), (0, 2), (2, 3)]`
Output	`[4, 6, 4, 6]`
Why	`Node 0: 1 + 1 + 2 = 4. Node 1 is a leaf: 1 + 2 + 3 = 6. Node 2: 1 + 2 + 1 = 4. Node 3: 1 + 2 + 3 = 6.`

Example 2	
Input	`3, [(0, 1), (1, 2)]`
Output	`[3, 2, 3]`
Why	`A path of three nodes: the middle one is closest to the others.`

FULL CODE | main question first, then every follow-up, tests at the end

```python
# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ========================================================

# ========================================================
# MAIN QUESTION
# ========================================================
```

```python
# ---- main question: helper functions ----
def build_adjacency(n, edges):
    # an edge is [u, v] or [u, v, weight]; neighbours are (node, weight)
    adj = []
    for i in range(n):
        adj.append([])
    for e in edges:
        u, v = e[0], e[1]
        w = 1
        if len(e) > 2:
            w = e[2]
        adj[u].append((v, w))
        adj[v].append((u, w))
    return adj


def distances_from(adj, s):
    # distance from s to every node (in a tree one walk is enough)
    dist = {s: 0}
    stack = [s]
    while stack:
        u = stack.pop()
        for v, w in adj[u]:
            if v not in dist:
                dist[v] = dist[u] + w
                stack.append(v)
    return dist


# ---- main question: main function ----
# Idea (re-rooting): root the tree at 0 and compute size[v] (the number
# of nodes below v) and the answer for the root: the sum of its
# distances. Moving the "root" from a parent p across an edge of weight w
# to its child u makes the size[u] nodes below u closer by w and the
# other n - size[u] nodes farther by w:
```

```python
#   answer[u] = answer[p] + w * (n - 2 * size[u]).
# Edges may carry a weight (follow-up 2); an edge without one costs 1.
def distance_sums(n, edges):
    adj = build_adjacency(n, edges)
    parent = [-1] * n
    up = [0] * n                                    # weight of the edge to the parent
    order = [0]                                     # parents before children
    for u in order:
        for v, w in adj[u]:
            if v != parent[u]:
                parent[v] = u
                up[v] = w
                order.append(v)
    size = [1] * n
    for i in range(n - 1, -1, -1):                  # children before parents
        u = order[i]
        if parent[u] >= 0:
            size[parent[u]] += size[u]
    answer = [0] * n
    for d in distances_from(adj, 0).values():
        answer[0] += d                              # the root's answer is a plain sum of distances
    for u in order[1:]:
        answer[u] = answer[parent[u]] + up[u] * (n - 2 * size[u])
    return answer


# ---- main question: tests ----
EDGES = [[0, 1], [0, 2], [2, 3], [2, 4], [2, 5]]
```

```
def test_main_question():
    assert distance_sums(6, EDGES) == [8, 12, 6, 10, 10, 10]
    assert distance_sums(1, []) == [0]


# =========================================================
# FOLLOW-UP 1: Find the node with the smallest distance sum (the tree's centroid)
# =========================================================
# ---- follow-up 1: function ----
# Answer: it is the position of the smallest number in the answer list
# (the centroid, which you can also find by walking toward the heavier
# subtree).
def best_center(n, edges):
    sums = distance_sums(n, edges)
    best = 0
```

```
    for v in range(n):
        if sums[v] < sums[best]:
            best = v
    return best


# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert best_center(6, EDGES) == 2


# =========================================================
# FOLLOW-UP 2: The tree has weighted edges
# =========================================================
# Answer: distance_sums already multiplies the shift by the weight of
# the edge it crosses.
# ---- follow-up 2: tests ----
def test_follow_up_2():
    weighted = [[0, 1, 2], [1, 2, 3]]               # the path 0 -(2)- 1 -(3)- 2
    assert distance_sums(3, weighted) == [7, 5, 8]


# =========================================================
# FOLLOW-UP 3: Maximum distance (eccentricity) from each node instead of the sum
# =========================================================
# ---- follow-up 3: helper functions ----
def farthest_node(dist):
    # the node with the largest distance in a {node: distance} table
    far = None
    for node in dist:
        if far is None or dist[node] > dist[far]:
            far = node
    return far


# ---- follow-up 3: function ----
# Answer: the farthest node from ANY node is one end of the tree's
# diameter. Find both ends with two walks, measure from each end, and the
# answer for a node is the larger of its two distances. O(n).
def eccentricities(n, edges):
    adj = build_adjacency(n, edges)
    from_start = distances_from(adj, 0)
    end_a = farthest_node(from_start)
    from_a = distances_from(adj, end_a)
    end_b = farthest_node(from_a)
```

```
    from_b = distances_from(adj, end_b)
    result = []
    for v in range(n):
        if from_a[v] > from_b[v]:
```

```
            result.append(from_a[v])
        else:
            result.append(from_b[v])
    return result


# ---- follow-up 3: tests ----
def test_follow_up_3():
    assert eccentricities(6, EDGES) == [2, 3, 2, 3, 3, 3]


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

228 Tree Distance Sum (Re-root DP) HARD

Template fit: exactly the same two sweeps (this is the re-rooting version of the question)

QUESTION

Onsite coding round 1: given an undirected tree on N nodes, return for every node the sum of distances to all other nodes. Equivalent to LC 834. Interviewer expects baseline O(n²) DFS followed by the O(n) re-root DP.

Input: undirected tree with N nodes and N-1 edges.
Output: array ans where ans[u] is the sum of shortest-path distances from u to every other node.
Walk through baseline (BFS/DFS from every node, O(n²)) before optimizing.
Final solution should be O(n) via two-pass re-root DP.

EXAMPLES

```
Edges: [[0,1],[0,2],[2,3],[2,4],[2,5]]
ans:   [8, 12, 6, 10, 10, 10]
```

FULL CODE | main question first, then every follow-up, tests at the end

```
# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
def build_adjacency(n, edges):
    adj = []
    for i in range(n):
        adj.append([])
    for a, b in edges:
        adj[a].append(b)
        adj[b].append(a)
```

```
    return adj


# ---- main question: main function ----
# Idea: two passes. Root the tree at 0 and list the nodes parents-first
# (a loop, no recursion). Going through that list backwards compute
# size[u] (nodes in u's subtree) and inside[u] (the sum of distances
# from u to the nodes of its own subtree). For the root that is already
# the answer. Going forwards, moving the root from p to its child u makes
# the size[u] nodes below 1 closer and the other n - size[u] nodes 1
# farther: answer[u] = answer[p] + n - 2 * size[u].
def tree_distance_sums(n, edges):
    if n == 0:
        return []
    adj = build_adjacency(n, edges)
    parent = [-1] * n
    order = [0]
    for u in order:
        for v in adj[u]:
            if v != parent[u]:
                parent[v] = u
                order.append(v)
    size = [1] * n
    inside = [0] * n
    for i in range(n - 1, -1, -1):                # children before parents
        u = order[i]
        p = parent[u]
```

```
        if p >= 0:
            size[p] += size[u]
            inside[p] += inside[u] + size[u]       # every node below u is one step farther from p
    answer = [0] * n
    answer[0] = inside[0]
    for u in order[1:]:
        answer[u] = answer[parent[u]] + n - 2 * size[u]
    return answer


# ---- main question: tests ----
def test_main_question():
    edges = [[0, 1], [0, 2], [2, 3], [2, 4], [2, 5]]
    assert tree_distance_sums(6, edges) == [8, 12, 6, 10, 10, 10]
    assert tree_distance_sums(1, []) == [0]
    assert tree_distance_sums(0, []) == []
    n = 100000                                     # a chain of 100000 nodes: no recursion trouble
    chain = []
    for i in range(1, n):
        chain.append((i, i - 1))
    assert tree_distance_sums(n, chain)[0] == n * (n - 1) // 2


# =========================================================
# FOLLOW-UP 1: Why is the re-root formula correct?
# =========================================================
# Answer: when the root moves across the edge p - u, every node in u's
# subtree (size[u] nodes) gets one step closer and every other node
# (n - size[u] nodes) gets one step farther, so the sum changes by
# (n - size[u]) - size[u] = n - 2 * size[u].


# =========================================================
# RUN EVERYTHING
# =========================================================
def run_all_tests():
    test_main_question()
    print("all tests passed")
```

```
if __name__ == "__main__":
    run_all_tests()
```

GROUP 32 OF 43 | 5 QUESTIONS, ONE CODE TEMPLATE | 5 M

Counting or reaching an amount: the knapsack loop

229 Maximize Shelf Space (Unbounded Knapsack) MEDIUM

Template fit: reachable[t] = any(reachable[t - size]); the answer is the largest reachable total up to the capacity

QUESTION

A shelf has a fixed integer capacity capacity . You have boxes of various integer sizes given in sizes , and you may use any number of each box, including duplicates (each size is available infinitely). Choose boxes so their total size does not exceed capacity , maximizing the total size used. Return that maximum total.

Function signature

def max_shelf_space(capacity: int, sizes: list[int]) -> int:

...

EXAMPLES

Example 1	
Input	`capacity = 10, sizes = [3, 5, 7]`
Output	`10`
Why	`3 + 7 = 10 exactly fills the shelf.`

Example 2	
Input	`capacity = 11, sizes = [5, 3]`
Output	`11`
Why	`5 + 3 + 3 = 11.`

Example 3	
Input	`capacity = 7, sizes = [4, 5]`
Output	`5`
Why	`Options within 7 are 4 or 5 (two boxes would be >= 8). The best is 5.`

CONSTRAINTS

- 0 <= capacity <= 10**4 .
- 1 <= len(sizes) <= 100 , each size is a positive integer.

FULL CODE | main question first, then every follow-up, tests at the end

```python
# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ========================================================

# ========================================================
# MAIN QUESTION
# ========================================================
# ---- main question: main function ----
# Idea: reachable[c] says "the boxes can fill exactly c units of shelf".
# 0 is reachable, and c is reachable if taking away one box (size s <= c)
# leaves a reachable amount. A box size may be used many times (unbounded).
# The answer is the biggest reachable amount up to the capacity.
def max_shelf_space(capacity, sizes):
    reachable = [True] + [False] * capacity
```

```
    for c in range(1, capacity + 1):
        for s in sizes:
            if s <= c and reachable[c - s]:
                reachable[c] = True
    best = 0
    for c in range(capacity + 1):
        if reachable[c]:
            best = c
    return best


# ---- main question: tests ----
def test_main_question():
    assert max_shelf_space(10, [3, 5, 7]) == 10
    assert max_shelf_space(11, [5, 3]) == 11
    assert max_shelf_space(7, [4, 5]) == 5
    assert max_shelf_space(3, [5]) == 0


# ========================================================
# FOLLOW-UP 1: Each box size is only available once
# ========================================================
# ---- follow-up 1: function ----
# Answer: that is the 0/1 knapsack (subset sum). Take the boxes one at a
# time and go through the amounts DOWNWARD, so a box is not used twice:
# reachable[c] becomes true when reachable[c - s] is.
def max_shelf_space_once(capacity, sizes):
    reachable = [True] + [False] * capacity
```

```
    for s in sizes:
        for c in range(capacity, s - 1, -1):
            if reachable[c - s]:
                reachable[c] = True
    best = 0
    for c in range(capacity + 1):
        if reachable[c]:
            best = c
    return best


# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert max_shelf_space_once(10, [3, 5, 7]) == 10        # 3 + 7
    assert max_shelf_space_once(11, [5, 3]) == 8            # each box once: 5 + 3
    assert max_shelf_space_once(3, [5]) == 0


# ========================================================
# RUN EVERYTHING
# ========================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

230 Coin Change

MEDIUM

Template fit: fewest coins: start with infinity, best[t] = min(best[t], best[t - c] + 1); -1 when it stays infinity

QUESTION

You are given an integer array coins representing different coin denominations and an integer amount representing a total amount of money. Write a function coinChange that returns the fewest number of coins needed to make up that amount. If that amount cannot be made up by any combination of the coins, return -1. You may assume that you have an infinite number of each kind of coin. Examples PythonInput: coins = [1, 2, 5], amount = 11 Output: 3 Explanation: The minimum number of coins needed is 3 (11 = 5 + 5 + 1).
Input: coins = [4, 5], amount = 7 Output: -1 Explanation: It is not possible to make up the amount 7 with coins of denominations 4 and 5.Solution 1: Brute-Force approach The naive approach to solving the coin change problem involves a recursive strategy to explore all possible combinations of coins to find the minimum number needed to make up the target amount. It starts by recursively trying each coin denomination to reduce the target amount. For each coin, the algorithm subtracts the coin's value from the current amount and solves the reduced problem. If the amount becomes zero, it indicates that no more coins are needed, and the function returns zero. If the amount becomes negative, it signals that this path is not feasible, so the function returns infinity or a large number. The algorithm combines the results from each recursive call with the current coin to determine the minimum number of coins required. Finally, after exploring all possibilities, it returns the

EXAMPLES

Example 1	
Input	`[1, 2, 5], 11`
Output	`3`
Why	`5 + 5 + 1 = 11: 3 coins.`

Example 2	
Input	`[2], 3`
Output	`-1`
Why	`A coin of 2 can never make 3.`

Example 3	
Input	`[1], 0`
Output	`0`
Why	`Zero needs no coins.`

FULL CODE | main question first, then every follow-up, tests at the end

```
# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

import random

INF = float("inf")


# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: main function ----
# Idea (bottom-up table): best[a] = the fewest coins that make exactly a.
```

```
# best[0] = 0, and for a = 1, 2, ... try every coin c <= a: best[a] =
# 1 + best[a - c] when that is better. Every amount is solved once.
# Answer best[amount], or -1 if it stayed infinite.
def coin_change(coins, amount):
    best = [0] + [INF] * amount
    for a in range(1, amount + 1):
        for c in coins:
            if c <= a and best[a - c] + 1 < best[a]:
                best[a] = best[a - c] + 1
    if best[amount] == INF:
        return -1
    return best[amount]


# ---- main question: tests ----
def test_main_question():
    assert coin_change([1, 2, 5], 11) == 3
    assert coin_change([2], 3) == -1
    assert coin_change([1], 0) == 0


# ============================================================
# FOLLOW-UP 1: Count the number of combinations that make the amount (LeetCode 518)
# ============================================================
# ---- follow-up 1: function ----
# Answer: put the COINS in the outer loop and the amounts in the inner
# loop, adding ways[a - c]. That order counts each combination once; with
# the loops the other way round you would count every order separately.
```

```
def count_ways(coins, amount):
    ways = [1] + [0] * amount
    for c in coins:
        for a in range(c, amount + 1):
            ways[a] += ways[a - c]
    return ways[amount]


# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert count_ways([1, 2, 5], 5) == 4
    assert count_ways([2], 3) == 0
    assert count_ways([10], 10) == 1
    assert count_ways([1, 2], 0) == 1


# ============================================================
# FOLLOW-UP 2: Return which coins are used
# ============================================================
# ---- follow-up 2: function ----
# Answer: remember the last coin chosen for every amount, then walk back
# from the target.
def coins_used(coins, amount):
    best = [0] + [INF] * amount
    last = [0] * (amount + 1)
    for a in range(1, amount + 1):
        for c in coins:
            if c <= a and best[a - c] + 1 < best[a]:
                best[a] = best[a - c] + 1
                last[a] = c
    if best[amount] == INF:
        return None
    used = []
    a = amount
    while a > 0:
        used.append(last[a])
        a -= last[a]
```

```
    used.sort(reverse=True)
    return used


# ---- follow-up 2: tests ----
def test_follow_up_2():
    assert coins_used([1, 2, 5], 11) == [5, 5, 1]
    assert coins_used([2], 3) is None
```

```
    random.seed(8)
    for round_number in range(300):
        coins = random.sample(range(1, 9), random.randint(1, 3))
        amount = random.randint(0, 25)
        used = coins_used(coins, amount)
        if used is not None:
            assert sum(used) == amount and len(used) == coin_change(coins, amount)


# ========================================================
# FOLLOW-UP 3: Does greedy work?
# ========================================================
# ---- follow-up 3: function ----
# Answer: only for special coin systems (like US coins). For general
# coins it can be wrong (coins 1, 3, 4 and amount 6). Mention the
# counterexample, then give the DP.
def greedy(coins, amount):
    used = []
    for c in sorted(coins, reverse=True):           # always take the biggest coin that still fits
        while amount >= c:
            amount -= c
            used.append(c)
    if amount == 0:
        return used
    return None


# ---- follow-up 3: tests ----
def test_follow_up_3():
    assert greedy([1, 3, 4], 6) == [4, 1, 1]            # greedy: 3 coins ...
    assert coin_change([1, 3, 4], 6) == 2               # ... but 3 + 3 needs only 2
    assert coins_used([1, 3, 4], 6) == [3, 3]


# ========================================================
# RUN EVERYTHING
# ========================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    test_follow_up_3()
    print("all tests passed")
```

```
if __name__ == "__main__":
    run_all_tests()
```

231 Partition an array into two sub-arrays with equal sum. MEDIUM

Template fit: subset sum to total / 2, written with a bitset: reach |= reach << x

QUESTION

Given an array of integers, determine if it can be partitioned into two subsets such that the sum of the elements in each subset is equal. A partition is considered valid if it divides the array into two subsets where both subsets have the same sum. Examples Input: [2, 4, 11, 5]
Output: True Explanation: The array can be partitioned into two subsets [2, 4, 5] and [11] with equal sum.
Input: [2, 7, 3, 1]
Output: False Explanation: The array cannot be partitioned into two subsets with equal sums.

EXAMPLES

Example 1	
Input	`[1, 5, 11, 5]`
Output	`true`
Why	`[1, 5, 5] and [11] both add up to 11.`

Example 2	
Input	`[1, 2, 3, 5]`
Output	`false`
Why	`The total is 11, which is odd.`

FULL CODE | main question first, then every follow-up, tests at the end

```python
# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: main function ----
# Idea: the array can be split into two parts with the same sum exactly
# when some subset adds up to half of the total. reachable[s] means "some
# subset adds up to s"; for each number x go through s from high to low
# (so x is used once) and mark reachable[s] when reachable[s - x] is true.
def can_partition(nums):
    total = 0
    for x in nums:
        total += x
    if total % 2 == 1:
        return False                                # an odd total cannot be split evenly
    half = total // 2
    reachable = [True] + [False] * half
    for x in nums:
        for s in range(half, x - 1, -1):
            if reachable[s - x]:
                reachable[s] = True
    return reachable[half]


# ---- main question: tests ----
def test_main_question():
    assert can_partition([2, 4, 11, 5]) is True           # 2 + 4 + 5 = 11 = the other part
    assert can_partition([1, 2, 5]) is False
    assert can_partition([]) is True
```

```
    assert can_partition([1]) is False


# ======================================================
# FOLLOW-UP 1: Return the two subsets
# ======================================================
# ---- follow-up 1: function ----
# Answer: for every reachable sum remember the number of the item that
# FIRST reached it. Read off the subset for half the total by walking
# back (sum, minus that item, ...) and give the rest to the other side.
```

```
def partition_subsets(nums):
    total = 0
    for x in nums:
        total += x
    if total % 2 == 1:
        return None
    half = total // 2
    first_item = {0: -1}                                # sum -> the item that first reached it
    for i in range(len(nums)):
        for s in list(first_item):                      # a copy: each item is used once
            if s + nums[i] <= half and s + nums[i] not in first_item:
                first_item[s + nums[i]] = i
    if half not in first_item:
        return None
    chosen = []
    s = half
    while s > 0:
        i = first_item[s]
        chosen.append(i)
        s -= nums[i]
    left = []
    right = []
    for i in range(len(nums)):
        if i in chosen:
            left.append(nums[i])
        else:
            right.append(nums[i])
    return left, right


# ---- follow-up 1: tests ----
def test_follow_up_1():
    left, right = partition_subsets([1, 5, 11, 5])
    assert sum(left) == sum(right) == 11
    assert sorted(left + right) == [1, 5, 5, 11]
    assert partition_subsets([1, 2, 5]) is None


# ======================================================
# FOLLOW-UP 2: Minimise the difference between the two parts (LeetCode 1049)
# ======================================================
# ---- follow-up 2: function ----
# Answer: find every reachable subset sum up to half the total and take
# the biggest one, s; the best split has difference total - 2 * s.
def min_difference(nums):
```

```
    total = 0
    for x in nums:
        total += x
    half = total // 2
    reachable = [True] + [False] * half
    for x in nums:
        for s in range(half, x - 1, -1):
            if reachable[s - x]:
                reachable[s] = True
```

```
    best = 0
    for s in range(half + 1):
        if reachable[s]:
            best = s
    return total - 2 * best


# ---- follow-up 2: tests ----
def test_follow_up_2():
    assert min_difference([1, 6, 11, 5]) == 1
    assert min_difference([2, 4, 11, 5]) == 0


# ========================================================
# FOLLOW-UP 3: The parts must be contiguous sub-arrays (a split point)
# ========================================================
# ---- follow-up 3: function ----
# Answer: then only one cut position can work: add up a running prefix
# and check whether it ever equals half the total. O(n) time, O(1) space,
# no DP.
def can_split_contiguous(nums):
    total = 0
    for x in nums:
        total += x
    if total % 2 == 1:
        return False
    prefix = 0
    for x in nums:
        prefix += x
        if prefix * 2 == total:
            return True
    return False


# ---- follow-up 3: tests ----
def test_follow_up_3():
```

```
    assert can_split_contiguous([1, 2, 3, 6])
    assert not can_split_contiguous([1, 5, 11, 5])


# ========================================================
# RUN EVERYTHING
# ========================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

232 Explain how to find a target sum in an array. MEDIUM

Template fit: turn +/- signs into a subset sum of (total + S) / 2, items used once (sweep downwards)

QUESTION

Determine the Number of Ways to Assign Symbols to Reach a Target Sum. You are given a list of non-negative integers, a1, a2, ..., an, and a target sum S. Your task is to assign each integer in the list a symbol + or - and calculate how many different ways there are to arrange these symbols such that the sum of the integers equals S after applying the assigned symbols. For example, if you have the list [1, 1, 1, 1, 1] and the target sum 3, one way to reach the target sum is to assign + to three 1s and - to two 1s, like so: +1 +1 +1 +1 -1. Another way could be +1 -1 +1 +1 +1. You need to return the total number of unique arrangements that meet the target sum S. Examples nums = [1, 1, 1, 1, 1] targetSum = 3 output: 5 nums = [1, 2, 1] targetSum = 2 output: 2 nums = [1, 2, 7, 1, 5, 3] targetSum = 8 output: -1 (Explanation: Since the total sum of the numbers plus the target is odd, it's not possible to reach the target sum.)The original problem asks for the number of ways to assign symbols (+ or -) to a list of numbers such that their sum equals a target sum S. This problem can be converted into a problem of finding subsets of the given numbers whose sum equals (totalSum + S) / 2. This is because if we can find subsets whose sum equals (totalSum + S) / 2, the remaining numbers will naturally form the other subset that sums to (totalSum - S) / 2, which effectively gives us the desired configuration of symbols. NoteWhy (totalSum + S) / 2?Given a list of numbers nums=[a1,a2,...,an]\te

EXAMPLES

Example 1	
Input	`[1, 1, 1, 1, 1], 3`
Output	`5`
Why	`Put + or - in front of every number; five ways to reach 3 with five 1s.`

Example 2	
Input	`[1], 1`
Output	`1`
Why	`+1 is the only way.`

Example 3	
Input	`[1, 2], 5`
Output	`-1`
Why	`No sign choice gives 5.`

FULL CODE | main question first, then every follow-up, tests at the end

```
# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ========================================================

import random


# ========================================================
# MAIN QUESTION
# ========================================================
# ---- main question: main function ----
# Put a + or - sign in front of every number so that they add up to
# `target`; count the ways (-1 if the target cannot be reached at all).
# Idea: the numbers with a PLUS sign add up to P, the others to total - P,
# so P - (total - P) = target, i.e. P = (total + target) / 2. The question
```

```python
# becomes "how many subsets add up to P?". ways[s] = the number of subsets
# with sum s; for each number x update s from high to low (so each number is
# used once): ways[s] += ways[s - x].
def count_signs(nums, target):
    total = 0
    for x in nums:
        total += x
    if abs(target) > total or (total + target) % 2 == 1:
        return -1
    goal = (total + target) // 2
    ways = [1] + [0] * goal
    for x in nums:
        for s in range(goal, x - 1, -1):
            ways[s] += ways[s - x]
    return ways[goal]


# ---- main question: tests ----
def test_main_question():
    assert count_signs([1, 1, 1, 1, 1], 3) == 5
    assert count_signs([1, 2, 1], 2) == 2
    assert count_signs([1, 2, 7, 1, 5, 3], 8) == -1
    assert count_signs([1], 5) == -1
    assert count_signs([0, 0, 1], 1) == 4
    assert count_signs([], 0) == 1
```

```python
# ======================================================
# FOLLOW-UP 1: The question was really "find two numbers that add up to the target" (Two Sum)
# ======================================================
# ---- follow-up 1: function ----
# Answer: one pass with a dictionary from value to index: for each x look
# for target - x among the earlier numbers. O(n) time and space. With a
# SORTED array use two pointers instead: O(1) space.
def two_sum(nums, target):
    seen = {}
    for i in range(len(nums)):
        if target - nums[i] in seen:
            return seen[target - nums[i]], i
        seen[nums[i]] = i
    return None


def two_sum_sorted(nums, target):
    low = 0
    high = len(nums) - 1
    while low < high:
        total = nums[low] + nums[high]
        if total == target:
            return low, high
        if total < target:
            low += 1
        else:
            high -= 1
    return None


# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert two_sum([2, 7, 11, 15], 9) == (0, 1)
    assert two_sum([1, 2], 7) is None
    assert two_sum_sorted([1, 3, 4, 8], 11) == (1, 3)


# ======================================================
# FOLLOW-UP 2: Only the existence of a subset with the target sum
```

```
# ========================================================
# ---- follow-up 2: function ----
# Answer: the same subset-sum table with True/False instead of counts:
# reachable[s] says "some subset adds up to s".
def subset_exists(nums, goal):
    reachable = [True] + [False] * goal
    for x in nums:
        for s in range(goal, x - 1, -1):
            if reachable[s - x]:
                reachable[s] = True
    return reachable[goal]


# ---- follow-up 2: tests ----
def test_follow_up_2():
    assert subset_exists([3, 34, 4, 12, 5, 2], 9)
    assert not subset_exists([3, 34, 4], 8)


# ========================================================
# FOLLOW-UP 3: Negative numbers or a huge sum
# ========================================================
# ---- follow-up 3: function ----
# Answer: with negative numbers a table indexed by the sum no longer
# works. Use a dictionary from "reachable sum" to "number of ways": it
# holds only the sums that really occur, so a huge target costs nothing.
def count_signs_dict(nums, target):
    ways = {0: 1}
    for x in nums:
        following = {}
        for s in ways:
            for new_sum in (s + x, s - x):
                following[new_sum] = following.get(new_sum, 0) + ways[s]
        ways = following
    return ways.get(target, 0)


# ---- follow-up 3: tests ----
def test_follow_up_3():
    assert count_signs_dict([1, 1, 1, 1, 1], 3) == 5
    assert count_signs_dict([-1, 2, 1], 2) == 2            # negative numbers work with the dictionary
    random.seed(3)
    for round_number in range(300):
        nums = []
        for i in range(random.randint(0, 7)):
            nums.append(random.randint(0, 4))
        target = random.randint(-8, 12)
        expected = count_signs_dict(nums, target)
        got = count_signs(nums, target)
        assert got == expected or (got == -1 and expected == 0)
```

```
# ========================================================
# RUN EVERYTHING
# ========================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

233 Reverse Coin Change

MEDIUM

Template fit: given the ways table, find the coin values: the first amount whose count is wrong must be a coin; add it and rebuild

QUESTION

In the coin change counting problem, dp[i] is the number of ways to make amount i from a set of coins (dp[0] = 1). In the reverse problem you are given the dp array and must find a set of coins that produces it, or None if no coin set does.

Function signature

def coins_from_dp(dp: list) -> list | None

EXAMPLES

Example 1	
Input	`[1, 1, 2, 2, 3, 3]`
Output	`[1, 2]`
Why	`Coins 1 and 2 make 1, 1, 2, 2, 3, 3 ways for the amounts 0 to 5.`

Example 2	
Input	`[1, 0, 1, 0, 1]`
Output	`[2]`
Why	`Only the coin 2 can make 2 and 4 but not 1 and 3.`

Example 3	
Input	`[1, 2, 3]`
Output	`null`
Why	`No coin set can produce these counts.`

CONSTRAINTS

- 1 <= dp.length <= 1000
- dp[0] = 1

FULL CODE | main question first, then every follow-up, tests at the end

```
# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ========================================================

import random


# ========================================================
# MAIN QUESTION
# ========================================================
# ---- main question: helper functions ----
def ways_table(coins, n):
    # the FORWARD direction: dp[a] = the number of combinations of the coins that make a.
    # (coins in the outer loop: each combination counts once, not once per order)
    dp = [1] + [0] * n
    for c in coins:
        for a in range(c, n + 1):
            dp[a] += dp[a - c]
    return dp


# ---- main question: main function ----
```

```python
# The usual coin-change table dp[a] counts the combinations of coins that
# make amount a. Here you get the table and must find the coins.
# Idea: scan the amounts upward and keep the table built from the coins
# found so far (`built`). At the first amount where the real dp is
# exactly ONE more than what we built, there must be a new coin of that
# value (it adds the single way "just that coin"). Any other difference is
# impossible. At the end the rebuilt table must equal the input (this is
# also follow-up 1: the array might not come from any coins).
def coins_from_dp(dp):
    if len(dp) == 0 or dp[0] != 1:
        return None                                 # dp[0] must be 1: one way to make 0
    n = len(dp) - 1
    built = [1] + [0] * n
    coins = []
    for a in range(1, n + 1):
        if dp[a] == built[a]:
            continue                                # the coins found so far already explain amount a
        if dp[a] != built[a] + 1:
            return None                             # needs more than one new way: impossible
        coins.append(a)
        for x in range(a, n + 1):
            built[x] += built[x - a]                # add the new coin to the table
    if built == list(dp):
        return coins
    return None


# ---- main question: tests ----
def test_main_question():
    assert coins_from_dp([1, 1, 2, 2, 3]) == [1, 2]
    assert coins_from_dp([1, 1, 1, 2, 2, 2, 3]) == [1, 3]
    assert coins_from_dp([1]) == []
    assert coins_from_dp([]) is None
    assert coins_from_dp([2, 1]) is None            # not a table: dp[0] must be 1
    assert coins_from_dp([1, 3]) is None            # an amount that jumps by more than one new way
    assert coins_from_dp([1, 1, 2, 2, 5]) is None  # looks fine early, but the whole table does not match


# ========================================================
# FOLLOW-UP 1: The array might not come from any set of coins
# ========================================================
# Answer: coins_from_dp already returns None when dp[0] is not 1, when
# an amount needs more than one new way, or when the table rebuilt from
# the coins differs from the input: the three checks above.


# ========================================================
# FOLLOW-UP 2: Order of the coins matters (the dp counts permutations, not combinations)
# ========================================================
# ---- follow-up 2: helper functions ----
def ordered_table(coins, n):
    # forward: dp[a] = the sum of dp[a - c] over the coins c (every order counts)
    dp = [1] + [0] * n
    for a in range(1, n + 1):
        for c in coins:
            if c <= a:
                dp[a] += dp[a - c]
    return dp


# ---- follow-up 2: function ----
# Answer: that is a different recurrence, dp[a] = sum of dp[a - c] over the
# coins. Invert it the same way: at each amount, what the known coins
# already explain is the sum of dp[a - c]; if dp[a] is exactly one more,
# a is a new coin. Ask which kind of counting is meant.
```

```python
def coins_from_ordered(dp):
    if len(dp) == 0 or dp[0] != 1:
        return None
    n = len(dp) - 1
    coins = []
    for a in range(1, n + 1):
        known = 0
        for c in coins:
            known += dp[a - c]
        if dp[a] == known + 1:
            coins.append(a)
        elif dp[a] != known:
            return None
    if ordered_table(coins, n) == list(dp):
        return coins
    return None


# ---- follow-up 2: tests ----
def test_follow_up_2():
    assert coins_from_ordered(ordered_table([1, 2], 6)) == [1, 2]
    assert coins_from_ordered([1, 1, 2, 3, 5]) == [1, 2]


# =======================================================
# FOLLOW-UP 3: Each coin may be used at most once
# =======================================================
# ---- follow-up 3: helper functions ----
def once_table(coins, n):
    # forward, each coin at most once: go through the amounts DOWNWARD
    dp = [1] + [0] * n
    for c in coins:
        for a in range(n, c - 1, -1):
            dp[a] += dp[a - c]
    return dp


# ---- follow-up 3: function ----
# Answer: the forward step visits the amounts downward. The inverse
# still scans upward adding a coin when the table falls short by one, but
# the table update must use the same 0/1 (downward) form.
def coins_from_once(dp):
    if len(dp) == 0 or dp[0] != 1:
        return None
    n = len(dp) - 1
```

```python
    built = [1] + [0] * n
    coins = []
    for a in range(1, n + 1):
        if dp[a] == built[a]:
            continue
        if dp[a] != built[a] + 1:
            return None
        coins.append(a)
        for x in range(n, a - 1, -1):
            built[x] += built[x - a]
    if built == list(dp):
        return coins
    return None


# ---- follow-up 3: tests ----
def test_follow_up_3():
    assert coins_from_once(once_table([1, 2, 3], 6)) == [1, 2, 3]
    random.seed(1)
    for round_number in range(300):                # random coin sets: all three inverses find them again
        coins = sorted(random.sample(range(1, 10), random.randint(0, 4)))
```

```
        n = random.randint(0, 14)
        small = []
        for c in coins:
            if c <= n:
                small.append(c)
        assert coins_from_dp(ways_table(coins, n)) == small
        assert coins_from_ordered(ordered_table(coins, n)) == small
        assert coins_from_once(once_table(coins, n)) == small
        junk = [1]
        for i in range(n):
            junk.append(random.randint(0, 3))
        found = coins_from_dp(junk)
        if found is not None:
            assert ways_table(found, n) == junk


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_2()
    test_follow_up_3()
    print("all tests passed")
```

```
if __name__ == "__main__":
    run_all_tests()
```

GROUP 33 OF 43 | 5 QUESTIONS, ONE CODE TEMPLATE | 4 M 1 H

Prefix sums, Tier 2: running total plus a dict of earlier totals

234 Maximum-Sum Subarray With Equal Endpoints — MEDIUM

Template fit: per value keep the earliest start with the smallest total before it; candidate = total now - that earlier total

QUESTION

Given an integer array nums , choose a (contiguous) subarray whose first and last elements are equal and whose sum is as large as possible. A single element is a valid subarray (its first and last elements coincide). Return the chosen subarray itself (as a list). If several subarrays tie for the maximum sum, return the one with the smallest starting index; if they still tie, return the shortest.

Function signature

def max_sum_equal_endpoints(nums: list[int]) -> list[int]:

    ...

Follow-up (as reported): Achieve O(n): for each value, you only ever care about pairing the earliest occurrence (with the best prefix sum) against later occurrences.

EXAMPLES

Example 1	
Input	`nums = [1, 3, -2, 4, 1]`
Output	`[1, 3, -2, 4, 1]`
Why	`The value 1 appears at indices 0 and 4; that whole window sums to 7, beating any single element.`

Example 2	
Input	`nums = [5, -10, 5]`
Output	`[5]`
Why	`The window from the first 5 to the last 5 sums to 0, which is worse than taking a single 5 (sum 5).`

Example 3	
Input	`nums = [-3, -1, -2]`
Output	`[-1]`
Why	`All values are distinct, so only single elements qualify; -1 is the largest.`

CONSTRAINTS

- 1 <= len(nums) <= 10^5 .
- -10^4 <= nums[i] <= 10^4 .

FULL CODE | main question first, then every follow-up, tests at the end

```
# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ========================================================

# ========================================================
# MAIN QUESTION
# ========================================================
# ---- main question: main function ----
```

```python
# Find the subarray with the biggest sum whose first and last elements are
# equal. Ties: the smaller start, then the shorter one. Return the subarray.
# Idea: with prefix sums, the sum of nums[i..j] is prefix[j+1] - prefix[i].
# For an end j with value x, the best start is an earlier index with the
# SAME value x and the smallest prefix[i] (a start at j itself is allowed:
# a single element). Remember, per value, the best start so far.
def max_sum_equal_endpoints(nums):
    prefix = [0]
    for x in nums:
        prefix.append(prefix[len(prefix) - 1] + x)
    best_start = {}                                     # value -> the start index with the smallest prefix
    best = None                                         # (minus the sum, start, length): smaller is better
    for j in range(len(nums)):
        x = nums[j]
        if x not in best_start or prefix[j] < prefix[best_start[x]]:
            best_start[x] = j
        i = best_start[x]
        key = (-(prefix[j + 1] - prefix[i]), i, j - i)
        if best is None or key < best:
            best = key
    if best is None:
        return []
    return nums[best[1]:best[1] + best[2] + 1]


# ---- main question: tests ----
def test_main_question():
    assert max_sum_equal_endpoints([1, 3, -2, 4, 1]) == [1, 3, -2, 4, 1]
    assert max_sum_equal_endpoints([5, -10, 5]) == [5]
    assert max_sum_equal_endpoints([-3, -1, -2]) == [-1]
    assert max_sum_equal_endpoints([]) == []


# ========================================================
```

```python
# FOLLOW-UP 1: Return only the best sum
# ========================================================
# ---- follow-up 1: function ----
# Answer: drop the tie-break bookkeeping: the same sweep, remembering for
# every value the smallest prefix sum seen before an element of that value.
def max_equal_ends_sum(nums):
    smallest = {}                                       # value -> the smallest prefix sum before an element
 of that value
    best = None
    total = 0
    for x in nums:
        if x not in smallest or total < smallest[x]:
            smallest[x] = total
        total += x
        candidate = total - smallest[x]
        if best is None or candidate > best:
            best = candidate
    return best


# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert max_equal_ends_sum([1, 3, -2, 4, 1]) == 7
    assert max_equal_ends_sum([]) is None
    for nums in ([5, -10, 5], [-3, -1, -2], [2, 2, -1, 2]):
        found = max_sum_equal_endpoints(nums)
        total = 0
        for x in found:
            total += x
        assert max_equal_ends_sum(nums) == total
```

```python
# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

235 Maximum Subarray Sum With a Minimum Length Constraint

MEDIUM

Template fit: the smallest earlier total is only allowed from min_len positions back, so update it one step late

QUESTION

This is the classic "maximum subarray" problem with the extra constraint the report mentioned ("Max sub array question with a weird constraint"). Given an integer array nums and an integer min_len , find the contiguous subarray (containing at least min_len elements) that has the largest sum, and return that sum.

Function signature

def max_subarray_min_len(nums: list[int], min_len: int) -> int:
    """Return the maximum sum over all contiguous subarrays of length >= min_len."""
    ...

Follow-up (as reported): The report said each question had a follow-up where a constraint was changed. Now bound the subarray length to the range [min_len, max_len] . Hint: maintain a sliding window of prefix-sum minima with a monotonic deque.

EXAMPLES

Example 1	
Input	`nums = [1, -2, 3, 4, -1], min_len = 2`
Output	`7`
Why	`The subarray [3, 4] (length 2) has sum 7, the largest among all subarrays of length >= 2.`

Example 2	
Input	`nums = [-1, -2, -3], min_len = 2`
Output	`-3`
Why	`Every element is negative. The best length->=2 subarray is [-1, -2] with sum -3.`

Example 3	
Input	`nums = [5, 4, -1, 7, 8], min_len = 1`
Output	`23`
Why	`With min_len = 1 this reduces to ordinary Kadane; the whole array sums to 23.`

CONSTRAINTS

- 1 <= min_len <= len(nums) <= 10^5
- -10^4 <= nums[i] <= 10^4
- The array is guaranteed to be non-empty and min_len never exceeds len(nums) , so a valid subarray always exists.
- Values may be negative; the answer can therefore be negative.

FULL CODE | main question first, then every follow-up, tests at the end

```
# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

from collections import deque


# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
def prefix_sums(nums):
    # prefix[j] = the sum of the first j numbers
    prefix = [0]
    for x in nums:
        prefix.append(prefix[len(prefix) - 1] + x)
    return prefix


# ---- main question: main function ----
# Idea: the sum of nums[i..end-1] is prefix[end] - prefix[i]. For each end,
# the best start is the SMALLEST prefix sum among the starts that leave at
# least min_len elements (i <= end - min_len). Sweep the end forward and
# keep that running minimum.
def max_subarray_min_len(nums, min_len):
    prefix = prefix_sums(nums)
    best = None
    smallest = None
    for end in range(min_len, len(nums) + 1):
        allowed_start = prefix[end - min_len]            # the newest start that is allowed now
        if smallest is None or allowed_start < smallest:
            smallest = allowed_start
        candidate = prefix[end] - smallest
        if best is None or candidate > best:
            best = candidate
    return best


# ---- main question: tests ----
def test_main_question():
    assert max_subarray_min_len([1, -2, 3, 4, -1], 2) == 7
    assert max_subarray_min_len([-1, -2, -3], 2) == -3
```

```
    assert max_subarray_min_len([5, 4, -1, 7, 8], 1) == 23


# ============================================================
# FOLLOW-UP 1: Now the length must also be at most max_len. How does it change?
# ============================================================
# ---- follow-up 1: function ----
# Answer: the allowed start positions now form a sliding window
# [end - max_len, end - min_len]. Keep the window in a deque of start
# positions whose prefix sums increase from front to back, so the front is
# always the smallest: drop the back entries that are not smaller than the
# newcomer, and drop the front entry when it falls out of the window.
def max_subarray_len_range(nums, min_len, max_len):
    prefix = prefix_sums(nums)
    window = deque()
    best = None
    for end in range(min_len, len(nums) + 1):
        new_start = end - min_len                        # the start that just became allowed
        while window and prefix[window[len(window) - 1]] >= prefix[new_start]:
            window.pop()
```

```
        window.append(new_start)
        while window[0] < end - max_len:                # this start would make the subarray too long
            window.popleft()
        candidate = prefix[end] - prefix[window[0]]
        if best is None or candidate > best:
            best = candidate
    return best


# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert max_subarray_len_range([1, -2, 3, 4, -1], 2, 3) == 7
    for code in range(5 ** 6):                          # every array of 6 numbers from -2..2, every length r
ange
        nums = []
        for i in range(6):
            nums.append(code // 5 ** i % 5 - 2)
        for low in range(1, 7):
            for high in range(low, 7):
                slow = None
                for i in range(6):
                    for j in range(i + low, min(6, i + high) + 1):
                        total = 0
                        for x in nums[i:j]:
                            total += x
                        if slow is None or total > slow:
```

```
                            slow = total
                assert max_subarray_len_range(nums, low, high) == slow


# =====================================================
# RUN EVERYTHING
# =====================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

236 Solve array-sum and city-community problems

MEDIUM

Template fit: count subarrays summing to k with the template as written; the second task is union-find over building cells

QUESTION

Two tasks. (1) count_subarrays(nums, k): count the contiguous subarrays whose elements add up to k. (2) communities(grid): a grid has 1 for a building and 0 for empty land; buildings that touch up, down, left or right belong to the same community. Return the number of communities.

Function signature

def count_subarrays(nums: list, k: int) -> int
def communities(grid: list) -> int

EXAMPLES

Example 1	
Input	`[1, 1, 1], 2`
Output	`2`
Why	`[1, 1] appears twice (positions 0-1 and 1-2).`

QUESTION

Example 2	
Input	[1, 2, 3], 3
Output	2
Why	[1, 2] and [3] both add up to 3.

Example 3	
Input	[[1, 1, 0], [0, 0, 1], [1, 0, 1]]
Output	3
Why	Top-left pair, the single building at the right of the middle row joined to the one below it, and the bottom-left: 3 communities.

FULL CODE | main question first, then every follow-up, tests at the end

```
# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
STEPS = ((1, 0), (-1, 0), (0, 1), (0, -1))


# ---- main question: main functions ----
# PART 1: count the subarrays whose sum is exactly k.
# Idea: walk along the array with the running (prefix) sum. A subarray
# that ENDS here has sum k when some EARLIER prefix sum equals
# (prefix - k). Keep a dictionary: prefix sum -> how many times it occurred.
def count_subarrays(nums, k):
    seen = {0: 1}                          # the empty prefix
    prefix = 0
    count = 0
    for x in nums:
        prefix += x
        count += seen.get(prefix - k, 0)
        seen[prefix] = seen.get(prefix, 0) + 1
    return count


# PART 2: count the communities: groups of neighbouring buildings (1) in a grid.
# Idea: scan the grid; an unseen building starts a new community, and a
# flood fill marks all the buildings joined to it.
def communities(grid):
    seen = set()
    count = 0
    for r in range(len(grid)):
        for c in range(len(grid[0])):
            if grid[r][c] != 1 or (r, c) in seen:
                continue
            count += 1
            seen.add((r, c))
            stack = [(r, c)]
            while stack:
                x, y = stack.pop()
```

```
                for dx, dy in STEPS:
                    a = x + dx
                    b = y + dy
                    if a < 0 or a >= len(grid) or b < 0 or b >= len(grid[0]):
                        continue
                    if grid[a][b] == 1 and (a, b) not in seen:
                        seen.add((a, b))
                        stack.append((a, b))
    return count


# ---- main question: tests ----
def test_main_question():
    assert count_subarrays([1, 1, 1], 2) == 2
    assert count_subarrays([1, -1, 0], 0) == 3
    assert count_subarrays([], 0) == 0
    assert communities([[1, 1, 0], [0, 0, 1], [1, 0, 1]]) == 3


# ========================================================
# FOLLOW-UP 1: The numbers are all positive
# ========================================================
# ---- follow-up 1: function ----
# Answer: a sliding window works with O(1) memory: move the right end
# forward and move the left end forward while the sum is too big. With
# negative numbers the window is not monotone, which is why prefix sums are
# needed there.
def count_positive(nums, k):
    count = 0
    left = 0
    total = 0
    for right in range(len(nums)):
        total += nums[right]
        while total > k and left <= right:
            total -= nums[left]
            left += 1
        if total == k:
            count += 1
    return count


# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert count_positive([1, 2, 3, 2, 1], 3) == 3
```

```


# ========================================================
# FOLLOW-UP 2: Longest subarray with sum K
# ========================================================
# ---- follow-up 2: function ----
# Answer: store the FIRST index of every prefix sum instead of a count,
# and take the longest distance back to the prefix (prefix - k).
def longest_subarray(nums, k):
    first = {0: -1}
    prefix = 0
    best = 0
    for i in range(len(nums)):
        prefix += nums[i]
        if prefix - k in first and i - first[prefix - k] > best:
            best = i - first[prefix - k]
        if prefix not in first:
            first[prefix] = i
    return best


# ---- follow-up 2: tests ----
def test_follow_up_2():
```

```
    assert longest_subarray([1, -1, 5, -2, 3], 3) == 4
    assert longest_subarray([1, 2], 9) == 0


# ====================================================
# FOLLOW-UP 3: Buildings are added over time and you report the number of communities after each addition
# ====================================================
# ---- follow-up 3: function ----
# Answer: union-find with a counter: +1 for the new building, -1 for every
# union with a different neighbouring community (LeetCode 305).
def communities_online(rows, cols, additions):
    parent = {}
    count = 0
    counts = []

    def find(x):
        while parent[x] != x:
            parent[x] = parent[parent[x]]
            x = parent[x]
        return x

    for r, c in additions:
        if (r, c) not in parent:
```

```
            parent[(r, c)] = (r, c)
            count += 1
            for dr, dc in STEPS:
                neighbour = (r + dr, c + dc)
                if neighbour in parent:
                    a = find((r, c))
                    b = find(neighbour)
                    if a != b:
                        parent[a] = b
                        count -= 1                  # two communities became one
        counts.append(count)
    return counts


# ---- follow-up 3: tests ----
def test_follow_up_3():
    additions = [(0, 0), (0, 1), (1, 2), (2, 2), (1, 1)]
    assert communities_online(3, 3, additions) == [1, 1, 2, 2, 1]


# ====================================================
# RUN EVERYTHING
# ====================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

237 Find the maximum subarray sum. MEDIUM

Template fit: the dict is just one number: the smallest earlier total; the best subarray ends here with total - smallest (Kadane is the same idea)

QUESTION

Given an array of integers nums, write a function maxSubarraySum to find the maximum sum of a contiguous subarray within the array and return that maximum sum. The subarray must be contiguous, meaning that the elements must appear consecutively in the original array. Examples Input: nums = [2, 3, -2, 4]
Output: 7 Explanation: Maximum sum is 2 + 3 + (-2) + 4 = 7.
Input: nums = [1, -1, -5, -4]
Output: 1 Explanation: The maximum sum is 1, which is the single element with the highest value.Challenge 1Can you come up with a solution with a O(n) time complexity?Challenge 2Can you come up with a solution with a O(1) space complexity?

EXAMPLES

Example 1	
Input	`[-2, 1, -3, 4, -1, 2, 1, -5, 4]`
Output	`6`
Why	`[4, -1, 2, 1] has the largest sum, 6.`

Example 2	
Input	`[1]`
Output	`1`
Why	`A single element.`

Example 3	
Input	`[5, 4, -1, 7, 8]`
Output	`23`
Why	`All numbers are positive: take them all.`

FULL CODE | main question first, then every follow-up, tests at the end

```
# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
def bigger(x, y):
    if x > y:
        return x
    return y


# ---- main question: main function ----
# Idea (Kadane): `current` is the best sum of a subarray that ENDS at the
# current element: either the element alone, or the element added to the
# previous `current`. `best` is the biggest `current` seen.
def max_subarray(nums):
    current = nums[0]
    best = nums[0]
    for i in range(1, len(nums)):
```

```
        current = bigger(nums[i], current + nums[i])
        best = bigger(best, current)
    return best


# ---- main question: tests ----
MIXED = [-2, 1, -3, 4, -1, 2, 1, -5, 4]


def test_main_question():
    assert max_subarray([2, 3, -2, 4]) == 7
    assert max_subarray([-3, -1, -2]) == -1
    assert max_subarray([5]) == 5
    assert max_subarray(MIXED) == 6


# ========================================================
# FOLLOW-UP 1: Return the subarray itself
# ========================================================
# ---- follow-up 1: function ----
```

```
# Answer: remember where the current run started: when we start fresh at
# i, set start = i; when `current` beats `best`, record (start, i). Same
# single pass.
def max_subarray_range(nums):
    current = nums[0]
    start = 0
    best = nums[0]
    span = (0, 0)
    for i in range(1, len(nums)):
        if nums[i] > current + nums[i]:
            current = nums[i]
            start = i
        else:
            current += nums[i]
        if current > best:
            best = current
            span = (start, i)
    return best, span


# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert max_subarray_range(MIXED) == (6, (3, 6))


# ========================================================
# FOLLOW-UP 2: The array is circular (LeetCode 918)
# ========================================================
# ---- follow-up 2: function ----
# Answer: the best circular subarray either does not wrap around (an
# ordinary one) or wraps, and then it is the WHOLE array minus the worst
# ordinary subarray (the minimum one). Take the bigger of the two, but if
# every number is negative the "wrap" would be empty, so use the ordinary one.
def max_circular(nums):
    best = max_subarray(nums)
    total = 0
    negated = []
    for x in nums:
        total += x
        negated.append(-x)
    worst = -max_subarray(negated)               # the smallest subarray sum
    if total == worst:
        return best
    return bigger(best, total - worst)
```

```
# ---- follow-up 2: tests ----
def test_follow_up_2():
    assert max_circular([5, -3, 5]) == 10
    assert max_circular([-3, -2, -3]) == -2
    assert max_circular([1, -2, 3, -2]) == 3


# ========================================================
# FOLLOW-UP 3: Divide and conquer instead of Kadane
# ========================================================
# ---- follow-up 3: function ----
# Answer: split the array in the middle. The best subarray lies entirely
# in the left half, entirely in the right half, or CROSSES the middle: the
# best sum ending at the middle on the left plus the best sum starting at
# the middle on the right. T(n) = 2 T(n/2) + O(n) = O(n log n); useful when
# the data is spread over several machines.
def max_subarray_dc(nums, low=0, high=None):
    if high is None:
        high = len(nums)
    if high - low == 1:
        return nums[low]
    middle = (low + high) // 2
    left_best = None
    running = 0
    for i in range(middle - 1, low - 1, -1):
        running += nums[i]
        if left_best is None or running > left_best:
            left_best = running
    right_best = None
    running = 0
    for i in range(middle, high):
        running += nums[i]
        if right_best is None or running > right_best:
            right_best = running
    return bigger(bigger(max_subarray_dc(nums, low, middle), max_subarray_dc(nums, middle, high)),
                  left_best + right_best)


# ---- follow-up 3: tests ----
def test_follow_up_3():
    for arr in ([2, 3, -2, 4], [-1, -2], [1, -1, 1, -1, 5], MIXED):
        assert max_subarray_dc(arr) == max_subarray(arr)
```

```
# ========================================================
# RUN EVERYTHING
# ========================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

238 Given an integer array nums and an integer k, return true if nums has a subarray of at least two elements whose sum is a multiple of k. HARD

Template fit: key = total % k; store the first index; a hit at least 2 positions back means a subarray whose sum is a multiple of k

QUESTION

Determine if a given array contains a subarray of at least two elements whose sum is a multiple of a specified number k. An array is considered to have a "good subarray" if there exists at least one subarray (consisting of two or more elements) such that the sum of the elements in this subarray is a multiple of k. For example, the array [23, 2, 4, 7] with k = 6 has a "good subarray" ([2, 4]), as the sum 6 is a multiple of k = 6. The array [5, 0, 0, 0] with k = 3 also has a "good subarray" ([0, 0]), since the sum is 0, and 0 is a multiple of 3. Examples nums = [23, 2, 4, 7], k = 6 output: true nums = [5, 0, 0, 0], k = 3 output: true nums = null, k = 1 output: false In these examples, the output is true if there exists a "good subarray" in the given array nums for the specified k, and false otherwise.Challenge 1Can you come up with a solution with a time complexity of O(n)?Challenge 2Can you come up with a solution with a space complexity of O(n)?

EXAMPLES

Example 1	
Input	[23, 2, 4, 6, 7], 6
Output	true
Why	[2, 4] adds up to 6, a multiple of 6.

Example 2	
Input	[23, 2, 6, 4, 7], 13
Output	false
Why	No subarray of at least two elements adds up to a multiple of 13.

Example 3	
Input	[23, 2, 6, 4, 7], 6
Output	true
Why	[23, 2, 6, 4, 7] adds up to 42, a multiple of 6.

FULL CODE | main question first, then every follow-up, tests at the end

```
# ====================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ====================================================

# ====================================================
# MAIN QUESTION
# ====================================================
# ---- main question: helper functions ----
def find_good_subarray(nums, k):
    # the (start, end) indexes of a subarray of at least 2 elements whose
    # sum is a multiple of k, or None.
    # Idea: prefix sums MODULO k. Two equal remainders at indexes at least 2
    # apart mean the elements between them add up to a multiple of k. Keep
    # only the FIRST index of each remainder: a later one only gives a
    # shorter block. (k == 0 means "equal to zero": compare the sums themselves.)
    first = {0: -1}                        # remainder -> the first index where it occurred
    total = 0
    for i in range(len(nums)):
        total += nums[i]
        if k != 0:
            key = total % k
```

```
        else:
            key = total
        if key not in first:
            first[key] = i
        elif i - first[key] >= 2:
            return first[key] + 1, i
    return None


# ---- main question: main function ----
def has_good_subarray(nums, k):
    return find_good_subarray(nums, k) is not None


# ---- main question: tests ----
def test_main_question():
    assert has_good_subarray([23, 2, 4, 7], 6)
    assert has_good_subarray([23, 2, 6, 4, 7], 6)
    assert not has_good_subarray([23, 2, 6, 4, 7], 13)
    assert find_good_subarray([23, 2, 4, 7], 6) == (1, 2)
    assert not has_good_subarray([6], 6)                # a single item is not enough
```

```
    assert has_good_subarray([0, 0], 5)
    assert not has_good_subarray([0, 1, 0], 0)
    assert has_good_subarray([0, 0], 0)
    assert has_good_subarray([5, -5], 7)
    assert has_good_subarray([1, 2], 10 ** 9) is False


# ========================================================
# FOLLOW-UP 1: Count the subarrays whose sum is a multiple of k (any length, LeetCode 974)
# ========================================================
# ---- follow-up 1: function ----
# Answer: count how many times each remainder has occurred, and add that
# count every time a remainder repeats (each earlier equal remainder is one
# more subarray). In Python % is never negative; in other languages use
# ((s % k) + k) % k. O(n).
def count_multiples(nums, k):
    seen = {0: 1}
    total = 0
    answer = 0
    for x in nums:
        total += x
        key = total % k
        answer += seen.get(key, 0)
        seen[key] = seen.get(key, 0) + 1
    return answer


# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert count_multiples([4, 5, 0, -2, -3, 1], 5) == 7


# ========================================================
# FOLLOW-UP 2: Why store only the first index of each remainder?
# ========================================================
# Answer: the later we meet the same remainder again, the shorter the
# block between them. The FIRST index gives the longest block, and the
# only thing we need is "at least 2 apart".


# ========================================================
# FOLLOW-UP 3: The values can be negative or k huge
# ========================================================
# Answer: Python's % gives a non-negative remainder even for negative sums;
```

```
# in other languages normalise it. A huge k costs nothing: only the
```

```
# remainders that occur are stored in the dictionary.


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

GROUP 34 OF 43 | 5 QUESTIONS, ONE CODE TEMPLATE | 1 E 3 M 1 H

Group by a canonical key

239 **Given an array of strings, return the words that can be typed using letters of the alphabet on only one row of American keyboard.** **EASY**

Template fit: key = the keyboard row of each letter; a word qualifies when its letters map to one row

QUESTION

Given an array of words, return those that can be typed using letters from a single row of an American keyboard (the rows are qwertyuiop, asdfghjkl and zxcvbnm). Ignore upper and lower case and keep the input order.

Function signature

def findWords(words: list) -> list

EXAMPLES

Example 1	
Input	`['Hello', 'Alaska', 'Dad', 'Peace']`
Output	`['Alaska', 'Dad']`
Why	`'Alaska' and 'Dad' use only the middle row; 'Hello' and 'Peace' mix rows.`

Example 2	
Input	`['omk']`
Output	`[]`
Why	`'omk' mixes the top and bottom rows.`

Example 3	
Input	`['adsdf', 'sfd']`
Output	`['adsdf', 'sfd']`
Why	`Both words stay on one row.`

FULL CODE | main question first, then every follow-up, tests at the end

```
# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
QWERTY = ["qwertyuiop", "asdfghjkl", "zxcvbnm"]


def row_map(rows):
    # letter -> the number of its row; built once and reused. Change `rows`
    # for another keyboard layout (follow-up 1).
    row_of = {}
    for i in range(len(rows)):
        for ch in rows[i]:
            row_of[ch] = i
    return row_of
```

```
# ---- main question: main function ----
# Which words can be typed with the letters of ONE keyboard row?
# Idea: build a dictionary from letter to row number (one entry per letter).
# A word qualifies if all of its (lowercased, so "Alaska" works) letters are
# on the same row. A letter that is on no row makes the word invalid.
def findWords(words, rows=QWERTY):
    row_of = row_map(rows)
    result = []
    for word in words:
        letters = word.lower()
        valid = True
        used_rows = set()
        for ch in letters:
            if ch not in row_of:
                valid = False                          # a letter on no row
                break
            used_rows.add(row_of[ch])
        if valid and len(used_rows) <= 1:
            result.append(word)
    return result
```

```
# ---- main question: tests ----
def test_main_question():
    assert findWords(["Hello", "Alaska", "Dad", "Peace"]) == ["Alaska", "Dad"]
    assert findWords([]) == []
    assert findWords(["", "a1"]) == [""]


# ========================================================
# FOLLOW-UP 1: What if the keyboard layout is a parameter (Dvorak, AZERTY, a phone keypad)?
# ========================================================


# ========================================================
# FOLLOW-UP 2: Other layouts (Dvorak, AZERTY) or a phone keypad
# ========================================================


# ========================================================
# FOLLOW-UP 3: A different keyboard layout
# ========================================================
# Answer: pass the rows as a parameter (rows=...) and build the
# letter-to-row map from it: a phone keypad or Dvorak works the same way.
# ---- follow-up 1: tests ----
def test_follow_up_1():
    keypad = ["abc", "def", "ghi"]
    assert findWords(["bad", "fed", "cab"], keypad) == ["fed", "cab"]
    dvorak = ["pyfgcrl", "aoeuidhtns", "qjkxbmwvz"]
    assert findWords(["tent"], dvorak) == ["tent"]


# ========================================================
# RUN EVERYTHING
# ========================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

240 Check if all substrings are anagrams of words

MEDIUM

Template fit: key = the tuple of letter counts; keep the keys of the allowed words in a set and test every substring of length 3 or more against it

QUESTION

You are given a string **`s`** (letters only) and access to an English dictionary **`dict`** (a set of valid words). Return **`true`** if **every contiguous substring** of **`s`** with length **`>= 3`** can be **rearranged (anagrammed)** into a valid dictionary word. Formally, for every substring **`t = s[i..j]`** with **`|t| >= 3`**, there exists some word **`w in dict`** such that **`w`** is a permutation of **`t`** (same multiset of characters). Otherwise return **`false`**. Clarifications:

- Substring means **contiguous**. - "Can be rearranged into a word" means the substring and the dictionary word have exactly the same character counts. Example:
- If **`t = "act"`**, it passes if the dictionary contains any of **`{ "act", "cat", "tac" }`**.

EXAMPLES

Example 1	
Input	`'abca', ['abc', 'cab', 'bca', 'abca']`
Output	`true`
Why	`Every substring of length 3 or more of 'abca' is an anagram of a dictionary word.`

Example 2	
Input	`'abca', ['abc']`
Output	`false`
Why	`'bca' is an anagram of 'abc' but 'abca' has no match.`

FULL CODE | main question first, then every follow-up, tests at the end

```
# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ========================================================

# ========================================================
# MAIN QUESTION
# ========================================================
# ---- main question: helper functions ----
ALPHABET = "abcdefghijklmnopqrstuvwxyz"


def count_vector(text):
    # the number of times each letter a..z occurs, as a tuple (the "signature")
    counts = [0] * len(ALPHABET)
    for ch in text:
        counts[ord(ch) - ord("a")] += 1
    return tuple(counts)


# ---- main question: main function ----
# Is EVERY substring of s (of length at least min_len) an anagram of some
# dictionary word?
# Idea: precompute the set of letter-count signatures of the dictionary
# words. For each start index grow the substring one letter at a time and
# update its count vector in O(1); once the length is at least min_len, look
# its signature up in the set.
def all_anagram_words(s, words, min_len=3):
    signatures = set()
    for word in words:
        signatures.add(count_vector(word))
```

```
    for i in range(len(s)):
        counts = [0] * len(ALPHABET)
        for j in range(i, len(s)):
            counts[ord(s[j]) - ord("a")] += 1
            if j - i + 1 >= min_len and tuple(counts) not in signatures:
                return False
    return True


# ---- main question: tests ----
WORDS = {"cat", "act", "tac", "tact", "cats"}
```

```
def test_main_question():
    assert all_anagram_words("cat", WORDS)
    assert all_anagram_words("tca", WORDS)
    assert not all_anagram_words("tcaz", WORDS)
    assert all_anagram_words("ab", set())                    # no substring is long enough to need a word


# ========================================================
# FOLLOW-UP 1: A bigger alphabet or Unicode
# ========================================================
# ---- follow-up 1: function ----
# Answer: replace the fixed 26-slot vector by a dictionary of counts and use
# a sorted tuple of (letter, count) pairs as the signature (or a hash of it).
def signature_of(counts):
    pairs = []
    for letter in counts:
        if counts[letter] > 0:
            pairs.append((letter, counts[letter]))
    pairs.sort()
    return tuple(pairs)


def all_anagram_words_any_alphabet(s, words, min_len=3):
    signatures = set()
    for word in words:
        counts = {}
        for ch in word:
            counts[ch] = counts.get(ch, 0) + 1
        signatures.add(signature_of(counts))
    for i in range(len(s)):
        counts = {}
        for j in range(i, len(s)):
            counts[s[j]] = counts.get(s[j], 0) + 1
            if j - i + 1 >= min_len and signature_of(counts) not in signatures:
                return False
    return True


# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert all_anagram_words_any_alphabet("cat", WORDS)
    assert not all_anagram_words_any_alphabet("tcaz", WORDS)
```

```
# ========================================================
# FOLLOW-UP 2: Only substrings of lengths that exist in the dictionary need checking
# ========================================================
# Answer: if no dictionary word has length L, the first substring of length
# L fails at once; so only the lengths that occur among the words (and are at
# least min_len) can ever succeed. first_failing below uses this.


# ========================================================
```

```
# FOLLOW-UP 3: Subsets of the letters (any sub-multiset of a word) instead of a full permutation
# ============================================================
# Answer: precompute, for every word, the signatures of ALL its sub-multisets
# (2^length of them for short words) and put them all in the set.


# ============================================================
# FOLLOW-UP 4: Report the first failing substring instead of True or False
# ============================================================
# ---- follow-up 4: helper functions ----
def sorted_letters(text):
    # the letters in sorted order: equal for anagrams
    return "".join(sorted(text))


# ---- follow-up 4: function ----
# Answer: test the substrings by LENGTH, shortest first, so the report is a
# smallest counterexample. A set of sorted-letter signatures makes each test one
# lookup after sorting the substring's letters.
def first_failing(s, words, min_len=3):
    signatures = set()
    lengths = set()
    for word in words:
        signatures.add(sorted_letters(word))
        lengths.add(len(word))
    for length in range(min_len, len(s) + 1):
        if length not in lengths:
            return s[:length]                                   # no word has this length: the first window fa
ils
        for i in range(len(s) - length + 1):
            piece = s[i:i + length]
            if sorted_letters(piece) not in signatures:
                return piece
    return None


# ---- follow-up 4: tests ----
```

```
def test_follow_up_4():
    assert first_failing("cat", WORDS) is None
    assert first_failing("catz", WORDS) == "atz"                # "cat" is fine; "atz" is the first substring
that is not
    assert first_failing("aaaa", {"aaa"}) == "aaaa"


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_4()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

241 Group anagrams — MEDIUM

Template fit: key = the sorted letters (or a tuple of 26 letter counts)

QUESTION

Given a list of words, group together the words that are anagrams of each other (the same letters in a different order). Return the groups in any order.

Function signature

def group_by_sorting(words: list) -> list

EXAMPLES

Example 1	
Input	`['eat', 'tea', 'tan', 'ate', 'nat', 'bat']`
Output	`[['eat', 'tea', 'ate'], ['tan', 'nat'], ['bat']]`
Why	`eat, tea and ate use the same letters; tan and nat; bat is alone.`

Example 2	
Input	`['']`
Output	`[['']]`
Why	`An empty string is its own group.`

Example 3	
Input	`['a']`
Output	`[['a']]`
Why	`One word.`

FULL CODE | main question first, then every follow-up, tests at the end

```
# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

import unicodedata


# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: main function ----
# Idea: anagrams have the SAME letters in sorted order, so sort the letters
# of each word and use that string as the key of a dictionary; words with the
# same key are anagrams. One pass, one dictionary (groups come out in the
# order their first word appears).
def group_by_sorting(words):
    groups = {}
    for word in words:
        key = "".join(sorted(word))
        if key not in groups:
            groups[key] = []
        groups[key].append(word)
    return list(groups.values())


# ---- main question: tests ----
WORDS = ["eat", "tea", "tan", "ate", "nat", "bat"]
```

```
GROUPED = [["eat", "tea", "ate"], ["tan", "nat"], ["bat"]]


def test_main_question():
    assert group_by_sorting(WORDS) == GROUPED
    assert group_by_sorting([]) == []
    assert group_by_sorting([""]) == [[""]]


# ============================================================
# FOLLOW-UP 1: Make the key O(L) instead of O(L log L)
# ============================================================
# ---- follow-up 1: function ----
# Answer: use a tuple of 26 letter COUNTS as the key instead of sorting
# the letters: O(L) per word, O(n * L) in total. For a big alphabet use the
```

```
# counts of the letters that occur (a sorted tuple of (letter, count) pairs).
def group_by_counts(words):
    groups = {}
    for word in words:
        counts = [0] * 26
        for ch in word:
            counts[ord(ch) - ord("a")] += 1
        key = tuple(counts)
        if key not in groups:
            groups[key] = []
        groups[key].append(word)
    return list(groups.values())


# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert group_by_counts(WORDS) == GROUPED


# ============================================================
# FOLLOW-UP 2: Case-insensitive, Unicode, or ignore spaces and punctuation
# ============================================================
# ---- follow-up 2: function ----
# Answer: normalise first (lowercase with casefold, remove accents with Unicode
# normalisation, drop everything that is not a letter) and then compute the key.
# Say what counts as a letter.
def normalise(word):
    text = unicodedata.normalize("NFKD", word).casefold()
    letters = ""
    for ch in text:
        if ch.isalpha():
            letters += ch
    return letters


def group_normalised(words):
    groups = {}
    for word in words:
        key = "".join(sorted(normalise(word)))
        if key not in groups:
            groups[key] = []
        groups[key].append(word)
    return list(groups.values())
```

```
# ---- follow-up 2: tests ----
def test_follow_up_2():
    mixed = ["Listen", "Silent", "enlist", "a gentleman"]
    assert group_normalised(mixed)[0] == ["Listen", "Silent", "enlist"]
    assert normalise("Caf\u00e9!") == "cafe"
```

```
# ========================================================
# FOLLOW-UP 3: The words are in many files
# ========================================================
# ---- follow-up 3: function ----
# Answer: compute the key on each machine and send every word to the machine
# given by a HASH of its key, so a whole anagram group lands on one worker, which
# then groups locally (a MapReduce shuffle).
def shard_of(word, workers):
    key = "".join(sorted(word))
    total = 0
    for ch in key:
        total += ord(ch)
    return total % workers


# ---- follow-up 3: tests ----
def test_follow_up_3():
    assert shard_of("eat", 8) == shard_of("tea", 8)           # anagrams always go to the same worker


# ========================================================
# RUN EVERYTHING
# ========================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

242 Given an array of strings, return groups of "buddy strings," where the strings have the same length and the distance between corresponding characters is the same. MEDIUM

Template fit: key = the differences between neighbouring letters, modulo 26 (a shifted word has the same differences)

QUESTION

Two strings are buddy strings when they have the same length and the distance between each pair of neighbouring letters is the same (the alphabet wraps around after z), so one is the other shifted by a constant. Group the strings into buddy groups and return the groups in any order.

Function signature

def group_buddies(words: list) -> list

EXAMPLES

Example 1	
Input	`['abc', 'bcd', 'acf', 'xyz', 'az', 'ba', 'a', 'z']`
Output	`[['abc', 'bcd', 'xyz'], ['acf'], ['az', 'ba'], ['a', 'z']]`
Why	`'abc', 'bcd' and 'xyz' all step +1, +1; 'az' and 'ba' both step +25 (a to z wraps); single letters 'a' and 'z' form a group.`

FULL CODE | main question first, then every follow-up, tests at the end

```
# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

import random


# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
def shift_key(word):
    # the gaps between neighbouring letters, wrapping z to a (mod 26).
    # Words that are shifts of each other share this key.
    gaps = []
    for i in range(len(word) - 1):
        gaps.append((ord(word[i + 1]) - ord(word[i])) % 26)
    return tuple(gaps)


def canonical(groups):
    # the groups sorted, so two answers can be compared
    result = []
    for group in groups:
        result.append(sorted(group))
    result.sort()
    return result


# ---- main question: main function ----
# Group the strings that are "buddies": the same length, and every letter
# is shifted by the same amount (a -> b, b -> c, ...; z wraps to a).
# Idea: two strings are buddies exactly when the GAPS between neighbouring
# letters (mod 26) are the same, so use the gaps (and the length) as the key of a
# dictionary and put the words with the same key together.
def group_buddies(words, key=shift_key):
    groups = {}
    for word in words:
        word_key = (len(word), key(word))           # words of different length never share a group
        if word_key not in groups:
            groups[word_key] = []
        groups[word_key].append(word)
    return list(groups.values())


# ---- main question: tests ----
WORDS = ["abc", "bcd", "acef", "xyz", "az", "ba", "a", "z"]


def test_main_question():
    assert canonical(group_buddies(WORDS)) == [["a", "z"], ["abc", "bcd", "xyz"], ["acef"], ["az", "ba"]]
    assert group_buddies([]) == []


# ============================================================
# FOLLOW-UP 1: What if shifting is not cyclic (z does not wrap to a)?
# ============================================================


# ============================================================
# FOLLOW-UP 3: Shifting does not wrap around (z stays z)
# ============================================================
# ---- follow-up 1: function ----
```

```
# Answer: drop the "% 26" and use the raw gaps as the key. Equal raw gaps
# mean one fixed whole-number shift, which is always legal here because both
# words are already made of letters, so no extra check is needed.
def raw_key(word):
    gaps = []
    for i in range(len(word) - 1):
        gaps.append(ord(word[i + 1]) - ord(word[i]))
    return tuple(gaps)


# ---- follow-up 1: tests ----
def test_follow_up_1():
    plain = canonical(group_buddies(WORDS, raw_key))
    assert ["az", "ba"] not in plain                # without wrap-around "az" and "ba" are different
    assert ["abc", "bcd", "xyz"] in plain


# ========================================================
# FOLLOW-UP 2: How would you do this with strings that are too many for one machine?
# ========================================================
# ---- follow-up 2: function ----
# Answer: every machine computes the keys for its own slice of the words;
# each (key, word) pair is sent to the machine number hash(key) % machines, so
# all buddies meet on the same machine, and each machine groups what it
```

```
# receives. Only the key travels, so the shuffle is one pass over the data.
def group_distributed(words, machines, key=shift_key):
    inbox = []
    for m in range(machines):
        inbox.append([])
    for m in range(machines):
        for word in words[m::machines]:             # the slice of machine m
            word_key = (len(word), key(word))
            inbox[hash(word_key) % machines].append((word_key, word))
    result = []
    for mail in inbox:
        local = {}
        for word_key, word in mail:
            if word_key not in local:
                local[word_key] = []
            local[word_key].append(word)
        result += list(local.values())
    return result


# ---- follow-up 2: tests ----
def test_follow_up_2():
    random.seed(26)
    for round_number in range(200):
        words = []
        for i in range(random.randint(0, 10)):
            word = ""
            for j in range(random.randint(1, 3)):
                word += random.choice("abyz")
            words.append(word)
        groups = group_buddies(words)
        total = 0
        for group in groups:
            total += len(group)
        assert total == len(words)
        assert canonical(group_distributed(words, random.randint(1, 4))) == canonical(groups)


# ========================================================
# RUN EVERYTHING
# ========================================================
```

```
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
```

```
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

243 Given two strings and two allowed operations, swapping positions of letters and converting all occurrences of one character to another, can the first string be transformed into the second? HARD

Template fit: same letters used, and the sorted list of letter frequencies must match: compare set(s) == set(t) and the sorted Counter values

QUESTION

Two strings are 'close' if you can turn one into the other with these operations, used any number of times: (1) swap the positions of any two letters; (2) take two letters that occur in the string and swap all of their occurrences (every a becomes b and every b becomes a). Return whether the first string can be transformed into the second.

Function signature

def close_strings(s: str, t: str) -> bool

EXAMPLES

Example 1	
Input	`'abc', 'bca'`
Output	`true`
Why	`Swapping positions turns 'abc' into 'bca'.`

Example 2	
Input	`'a', 'aa'`
Output	`false`
Why	`The lengths differ.`

Example 3	
Input	`'cabbba', 'abbccc'`
Output	`true`
Why	`Use operation 1 to reorder, then operation 2 to swap the letters a and b, and c and ... until the counts match.`

Example 4	
Input	`'uau', 'ssx'`
Output	`false`
Why	`'u' appears in the first string but not the second, so they cannot be close.`

FULL CODE | main question first, then every follow-up, tests at the end

```
# ================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
```

```
#   At the bottom, run_all_tests() calls every test function in this order.
# ========================================================

ALPHABET = 26


# ========================================================
# MAIN QUESTION
# ========================================================
# ---- main question: helper functions ----
def count_letters(text):
    counts = {}
    for ch in text:
        counts[ch] = counts.get(ch, 0) + 1
    return counts


def can_group(have, want):
    # can the counts in `have` be grouped (each used once) so that the group
    # sums are exactly the counts in `want`? Letters can merge, never split.
    # Backtracking: put each count (biggest first) into a bin of `want`.
    if sum(have) != sum(want):
        return False
    have = sorted(have, reverse=True)
    room = list(want)                                   # how much each bin can still take

    def place(i):
        if i == len(have):
            for r in room:
                if r != 0:
                    return False
            return True
        tried = set()
        for j in range(len(room)):
            if room[j] >= have[i] and room[j] not in tried:    # equal bins behave the same: try each size on
ce
                tried.add(room[j])
                room[j] -= have[i]
                if place(i + 1):
                    return True
                room[j] += have[i]
        return False
```

```
    return place(0)


# ---- main question: main function ----
# Two operations: swap the positions of two letters, and convert ALL
# occurrences of one letter into another letter. Can s become t?
# Idea: swaps make the positions irrelevant, so only the letter COUNTS
# matter. Converting a letter can only rename it or merge it into another one. So
# if the counts already match, yes; otherwise the counts of s must be
# groupable into the counts of t (can_group). If both strings already use all
# 26 letters there is no spare letter left to rename safely.
def close_strings(s, t, alphabet=ALPHABET):
    if len(s) != len(t):
        return False
    count_s = count_letters(s)
    count_t = count_letters(t)
    if count_s == count_t:
        return True
    if not can_group(list(count_s.values()), list(count_t.values())):
        return False
    if len(count_s) == len(count_t) == alphabet:
        return False
```

```
    return True


# ---- main question: tests ----
def test_main_question():
    assert close_strings("abc", "bca")
    assert not close_strings("a", "aa")
    assert close_strings("cabbba", "abbccc")


# =========================================================
# FOLLOW-UP 1: What if the two operations are combined (one-directional conversion plus swaps)?
# =========================================================
# Answer: that is close_strings above: swaps make the positions irrelevant,
# and the conversion works on the letter counts.


# =========================================================
# FOLLOW-UP 2: Why does a full 26-letter target make the one-directional version impossible?
# =========================================================
# Answer: converting a letter to another merges two letters into one, and
# to break a cycle of conversions (a -> b, b -> a) you need a SPARE letter as a
```

```
# temporary home. If the target uses all 26 letters there is none.


# =========================================================
# FOLLOW-UP 3: Only position swaps are allowed
# =========================================================
# ---- follow-up 3: function ----
# Answer: then s can become t exactly when they are anagrams: compare the
# letter counts. O(n).
def swaps_only(s, t):
    return count_letters(s) == count_letters(t)


# ---- follow-up 3: tests ----
def test_follow_up_3():
    assert swaps_only("listen", "silent")
    assert not swaps_only("ab", "aa")


# =========================================================
# FOLLOW-UP 4: Conversion is one-directional (every a becomes b) and there are no position swaps (LeetCode 11
53)
# =========================================================
# ---- follow-up 4: function ----
# Answer: each source letter must map to ONE target letter; and if the
# strings differ and the target already uses all 26 letters there is no spare
# letter to break mapping cycles, so it is impossible.
def convert_only(s, t, alphabet=ALPHABET):
    if len(s) != len(t):
        return False
    if s == t:
        return True
    mapping = {}
    for i in range(len(s)):
        if s[i] in mapping and mapping[s[i]] != t[i]:
            return False                                # one letter would need two targets
        mapping[s[i]] = t[i]
    return len(set(t)) < alphabet


# ---- follow-up 4: tests ----
def test_follow_up_4():
    assert convert_only("aabcc", "ccdee")
```

```
    assert not convert_only("leetcode", "codeleet")
```

```
# ============================================================
# FOLLOW-UP 5: Return the sequence of operations, or the minimum number
# ============================================================
# ---- follow-up 5: function ----
# Answer: for swaps only, with all letters different, the fewest swaps is
# n minus the number of CYCLES of the permutation that puts s into the order
# of t. swap_sequence writes the swaps down: for each position put the right
# letter there, one swap per misplaced letter.
def min_swaps(s, t):
    where = {}
    for i in range(len(t)):
        where[t[i]] = i
    seen = set()
    cycles = 0
    for start in range(len(s)):
        if start not in seen:
            cycles += 1
            i = start
            while i not in seen:
                seen.add(i)
                i = where[s[i]]
    return len(s) - cycles


def swap_sequence(s, t):
    current = list(s)
    where = {}
    for i in range(len(current)):
        where[current[i]] = i
    steps = []
    for i in range(len(t)):
        if current[i] != t[i]:
            j = where[t[i]]
            where[current[i]] = j
            where[t[i]] = i
            current[i], current[j] = current[j], current[i]
            steps.append((i, j))
    return steps


# ---- follow-up 5: tests ----
def test_follow_up_5():
    assert min_swaps("abc", "bca") == 2
    assert len(swap_sequence("abc", "bca")) == 2
    assert swap_sequence("abc", "abc") == []
```

```
# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_3()
    test_follow_up_4()
    test_follow_up_5()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

GROUP 35 OF 43 | 5 QUESTIONS, ONE CODE TEMPLATE | 1 E 2 M 2 H

K-way merge: a heap of the next item from each source

244 Find k pairs with smallest sums — EASY

Template fit: seed with (a[i] + b[0], i, 0); the successor of pair (i, j) is (i, j + 1)

QUESTION

You are given two **sorted** integer arrays **`arr1`** and **`arr2`** (each sorted in non-decreasing order), and an integer **`k`**. Consider all possible pairs formed by taking one element from **`arr1`** and one element from **`arr2`**. For indices **`i`** and **`j`**, the pair is **`(arr1[i], arr2[j])`** with sum **`arr1[i] + arr2[j]`**. Your task is to return the **`k`** **pairs `(arr1[i], arr2[j])`** with the smallest sums. If there are fewer than **`k`** total possible pairs, return all possible pairs. You may return the pairs in any order. ---

Function Signature

(one possible form)

```
kSmallestPairs(arr1: List[int], arr2: List[int], k: int) -> List[Tuple[int, int]]
```

Example

```
arr1 = [1, 7, 11] arr2 = [2, 4, 6] k = 3
```

All possible pairs and their sums are:

- (1, 2) -> 3 - (1, 4) -> 5 - (1, 6) -> 7 - (7, 2) -> 9 - (7, 4) -> 11 - (7, 6) -> 13 - (11, 2) -> 13 - (11, 4) -> 15 - (11, 6) -> 17
  The 3 pairs with the smallest sums are:

```
[(1, 2), (1, 4), (1, 6)]
```

(The order among these returned pairs does not matter.) Explain an efficient algorithm to solve this problem and analyze its time and space complexity. You may assume the arrays are large, so a solution that explicitly generates and sorts all **`len(arr1) * len(arr2)`** pairs is too slow.

EXAMPLES

Example 1	
Input	`[1, 7, 11], [2, 4, 6], 3`
Output	`[(1, 2), (1, 4), (1, 6)]`
Why	`Pairs (one from each array) in order of their sum.`

Example 2	
Input	`[1, 1, 2], [1, 2, 3], 2`
Output	`[(1, 1), (1, 1)]`
Why	`Equal numbers give equal pairs.`

FULL CODE | main question first, then every follow-up, tests at the end

```
# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ========================================================

import heapq
```

```python
# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: main function ----
# a and b are sorted. Find the k pairs (x from a, y from b) with the
# smallest sums x + y.
# Idea: think of a table with a[i] + b[j]; every row i is sorted. A
# min-heap holds the best UNUSED pair of every row. Start with the first
# pair of each row (only the first k rows can matter). Take the smallest
# out, record the pair, and offer the next pair of that row, (i, j + 1).
# After k pops you have the k smallest sums.
def k_smallest_pairs(a, b, k):
    if not a or not b or k <= 0:
        return []
    heap = []
    for i in range(min(k, len(a))):
        heap.append((a[i] + b[0], i, 0))
    heapq.heapify(heap)
    pairs = []
    while heap and len(pairs) < k:
        total, i, j = heapq.heappop(heap)
        pairs.append((a[i], b[j]))
        if j + 1 < len(b):
            heapq.heappush(heap, (a[i] + b[j + 1], i, j + 1))
    return pairs


# ---- main question: tests ----
def test_main_question():
    a = [1, 7, 11]
    b = [2, 4, 6]
    assert k_smallest_pairs(a, b, 3) == [(1, 2), (1, 4), (1, 6)]
    assert k_smallest_pairs(a, b, 4)[3] == (7, 2)
    assert k_smallest_pairs([1, 1, 2], [1, 2, 3], 2) == [(1, 1), (1, 1)]
    assert k_smallest_pairs([], [1], 3) == []
```

```python
    assert len(k_smallest_pairs([1], [1, 2], 10)) == 2     # k bigger than the number of pairs: all of them


# ============================================================
# FOLLOW-UP 1: The arrays are not sorted
# ============================================================
# ---- follow-up 1: function ----
# Answer: sort both first (O(m log m + n log n)): the heap method needs
# sorted rows.
def k_smallest_pairs_unsorted(a, b, k):
    return k_smallest_pairs(sorted(a), sorted(b), k)


# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert k_smallest_pairs_unsorted([7, 1], [4, 2], 2) == [(1, 2), (1, 4)]


# ============================================================
# FOLLOW-UP 2: Only the k-th smallest sum, with huge arrays
# ============================================================
# ---- follow-up 2: helper functions ----
def count_at_most(a, b, limit):
    # how many pairs have a sum <= limit? Two pointers: as x grows, fewer
    # values of b fit, so the pointer into b only moves down.
    count = 0
    j = len(b) - 1
    for x in a:
        while j >= 0 and x + b[j] > limit:
```

```
            j -= 1
        count += j + 1
    return count


# ---- follow-up 2: function ----
# Answer: binary search the SUM itself. For a guessed sum, count how many
# pairs are at most that big (O(m + n) with two pointers); the k-th smallest
# is the smallest sum whose count is at least k. O((m + n) log range)
# time and O(1) space: no heap.
def kth_smallest_sum(a, b, k):
    low = a[0] + b[0]
    high = a[len(a) - 1] + b[len(b) - 1]
    while low < high:
        middle = (low + high) // 2
        if count_at_most(a, b, middle) >= k:
```

```
            high = middle
        else:
            low = middle + 1
    return low


# ---- follow-up 2: tests ----
def test_follow_up_2():
    a = [1, 7, 11]
    b = [2, 4, 6]
    sums = []
    for x in a:
        for y in b:
            sums.append(x + y)
    sums.sort()
    for k in range(1, 10):
        assert kth_smallest_sum(a, b, k) == sums[k - 1]    # the 4th smallest sum is 9 (7 + 2)


# ======================================================
# FOLLOW-UP 3: Three arrays, or k pairs with the largest sums
# ======================================================
# ---- follow-up 3: function ----
# Answer: for the LARGEST sums run the same code on the negated values
# (the biggest numbers become the smallest). For three arrays use a heap
# of triples (i, j, l) and push the three neighbours of the popped one,
# with a set of visited triples.
def k_largest_pairs(a, b, k):
    negated_a = []
    for x in sorted(a, reverse=True):
        negated_a.append(-x)
    negated_b = []
    for y in sorted(b, reverse=True):
        negated_b.append(-y)
    pairs = []
    for x, y in k_smallest_pairs(negated_a, negated_b, k):
        pairs.append((-x, -y))
    return pairs


# ---- follow-up 3: tests ----
def test_follow_up_3():
    assert k_largest_pairs([1, 7, 11], [2, 4, 6], 2) == [(11, 6), (11, 4)]
```

```
# ======================================================
# RUN EVERYTHING
# ======================================================
def run_all_tests():
```

```
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

245 K Largest Elements Across Sorted Lists

MEDIUM

Template fit: seed with the LAST item of each list (negated); the successor is the item before it; reverse the result

QUESTION

You are given a list of inner lists, where each inner list is sorted in ascending order. Return the k largest elements taken from the union of all the inner lists, and return them sorted in ascending order. Duplicate values are kept (an element counts once per position it occupies). The interviewer first accepted a simple solution, then asked for an optimized one.

Function signature

```
from typing import List
def k_largest_sorted(lists: List[List[int]], k: int) -> List[int]:
    """Return the k largest elements across all (individually ascending) inner
    lists, in ascending sorted order."""
    ...
```

Follow-up (as reported): Optimize beyond concatenate-and-sort. Hint: because each list is already sorted, seed a max-heap with the last element of every non-empty list and pop k times, pushing the predecessor within a list after each pop; collect the popped values and reverse them.

EXAMPLES

Example 1	
Input	`lists = [[1, 4, 7], [2, 5, 8], [3, 6, 9]], k = 4`
Output	`[6, 7, 8, 9]`
Why	`The four largest values overall are 9, 8, 7, 6, returned ascending.`

Example 2	
Input	`lists = [[10, 20, 30]], k = 2`
Output	`[20, 30]`

Example 3	
Input	`lists = [[5], [1, 3]], k = 5`
Output	`[1, 3, 5]`
Why	`k exceeds the total count, so all elements are returned sorted.`

Example 4	
Input	`lists = [[1, 2], [3, 4]], k = 0`
Output	`[]`

CONSTRAINTS

- Let m be the number of inner lists and T the total number of elements across all of them.
- 0 <= m <= 10^4 , 0 <= T <= 10^6 .
- Each element is an integer in [-10^9, 10^9] .

QUESTION

- 0 <= k . If k == 0 return [] . If k >= T , return every element sorted ascending.
- Some inner lists may be empty.

FULL CODE | main question first, then every follow-up, tests at the end

```
# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ========================================================

import heapq
import random


# ========================================================
# MAIN QUESTION
# ========================================================
# ---- main question: main function ----
# Each list is sorted (smallest first). Return the k largest numbers of
# all lists together, in increasing order.
# Idea: the next biggest number is always at the END of one of the lists.
# A max-heap (push negatives) holds the last unused element of every list.
# Take the biggest out, then offer the element just before it in its list.
def k_largest_sorted(lists, k):
    heap = []
    for i in range(len(lists)):
        if lists[i]:
            last = len(lists[i]) - 1
            heap.append((-lists[i][last], i, last))
    heapq.heapify(heap)
    biggest_first = []
    while heap and len(biggest_first) < k:
        negative, i, j = heapq.heappop(heap)
        biggest_first.append(-negative)
        if j > 0:
            heapq.heappush(heap, (-lists[i][j - 1], i, j - 1))
    biggest_first.reverse()
    return biggest_first


# ---- main question: tests ----
def test_main_question():
    assert k_largest_sorted([[1, 4, 7], [2, 5, 8], [3, 6, 9]], 4) == [6, 7, 8, 9]
    assert k_largest_sorted([[10, 20, 30]], 2) == [20, 30]
    assert k_largest_sorted([[5], [1, 3]], 5) == [1, 3, 5]
    assert k_largest_sorted([[1, 2], [3, 4]], 0) == []
```

```
# ========================================================
# FOLLOW-UP 1: The lists are not sorted, or arrive as streams. What changes?
# ========================================================
# ---- follow-up 1: function ----
# Answer: keep a MIN-heap of the k largest values seen so far: push every
# value, and when the heap holds more than k, drop the smallest. One pass,
# O(N log k) time and O(k) memory.
def k_largest_stream(lists, k):
    heap = []
    for numbers in lists:
        for x in numbers:
            if len(heap) < k:
                heapq.heappush(heap, x)
            elif k > 0 and x > heap[0]:
```

```
                heapq.heapreplace(heap, x)                # drop the smallest of the k kept
    return sorted(heap)


# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert k_largest_stream([[4, 1, 7], [8, 2, 5], [3, 9, 6]], 4) == [6, 7, 8, 9]
    random.seed(77)
    for round_number in range(200):
        lists = []
        for i in range(random.randint(0, 4)):
            numbers = []
            for j in range(random.randint(0, 5)):
                numbers.append(random.randint(0, 20))
            lists.append(sorted(numbers))
        k = random.randint(0, 6)
        everything = []
        for numbers in lists:
            everything += numbers
        everything.sort()
        start = len(everything) - k
        if start < 0:
            start = 0
        expected = everything[start:]
        if k == 0:
            expected = []
        assert k_largest_sorted(lists, k) == expected
        assert k_largest_stream(lists, k) == expected


# ======================================================
```

```
# RUN EVERYTHING
# ======================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

246 Sort a nearly sorted (or K sorted) array. MEDIUM

Template fit: keep a heap of the first k + 1 items; pop the smallest and push the next input item (heapreplace)

QUESTION

In a k-sorted array every element is at most k positions away from where it would be in the sorted array. Sort such an array in O(n log k) time (better than a full sort when k is small).

Function signature

def sort_k_sorted(arr: list, k: int) -> list

EXAMPLES

Example 1	
Input	`[6, 5, 3, 2, 8, 10, 9], 3`
Output	`[2, 3, 5, 6, 8, 9, 10]`
Why	Every number is within 3 positions of its place in the sorted order.

QUESTION

Example 2	
Input	`[2, 1, 4, 3], 1`
Output	`[1, 2, 3, 4]`
Why	`k = 1: neighbours may be swapped.`

FULL CODE | main question first, then every follow-up, tests at the end

```
# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ========================================================

import heapq
import random


# ========================================================
# MAIN QUESTION
# ========================================================
# ---- main question: main function ----
# Every element is at most k places away from where it belongs when the
# array is sorted. Idea: the smallest of the next k + 1 elements must be
# the next one of the answer. Keep a min-heap of k elements; for every new
# element push it and take out the smallest (heappushpop does both). At the
# end empty the heap. O(n log k).
def sort_k_sorted(arr, k):
    heap = arr[:k]
    heapq.heapify(heap)
    result = []
    for x in arr[k:]:
        result.append(heapq.heappushpop(heap, x))      # push x, then pop the smallest
    while heap:
        result.append(heapq.heappop(heap))
    return result


# ---- main question: tests ----
def test_main_question():
    nearly = [6, 5, 3, 2, 8, 10, 9]
    assert sort_k_sorted(nearly, 3) == sorted(nearly)
    assert sort_k_sorted([], 2) == []
    assert sort_k_sorted([3, 1, 2], 5) == [1, 2, 3]
    assert sort_k_sorted([1, 2, 3], 0) == [1, 2, 3]


# ========================================================
# FOLLOW-UP 1: K is not given. How do you find it?
# ========================================================
# ---- follow-up 1: function ----
# Answer: sort the positions by value (equal values keep their order) and
```

```
# take the biggest distance between a position and the place it ends up
# in; that is the smallest valid k. O(n log n), so it is only worth it if you
# will sort many similar arrays.
def find_k(arr):
    pairs = []
    for position in range(len(arr)):
        pairs.append((arr[position], position))
    pairs.sort()                                       # now pairs[i] says which original position belongs at
 place i
```

```
    biggest = 0
    for place in range(len(pairs)):
        distance = pairs[place][1] - place
        if distance < 0:
            distance = -distance
        if distance > biggest:
            biggest = distance
    return biggest


# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert find_k([1, 2, 3]) == 0
    assert find_k([2, 1, 3]) == 1
    assert find_k([6, 5, 3, 2, 8, 10, 9]) == 3
    assert find_k([]) == 0


# =======================================================
# FOLLOW-UP 2: The data is a stream you cannot hold in memory
# =======================================================
# ---- follow-up 2: function ----
# Answer: the heap is already a streaming algorithm: after the heap holds
# more than k elements, each new arrival lets one sorted element go out.
# As a generator it uses O(k) memory.
def stream_sort(items, k):
    heap = []
    for x in items:
        heapq.heappush(heap, x)
        if len(heap) > k:
            yield heapq.heappop(heap)
    while heap:
        yield heapq.heappop(heap)


# ---- follow-up 2: tests ----
def test_follow_up_2():
```

```
    nearly = [6, 5, 3, 2, 8, 10, 9]
    assert list(stream_sort(iter(nearly), 3)) == sorted(nearly)


# =======================================================
# FOLLOW-UP 3: K is close to n
# =======================================================
# ---- follow-up 3: function ----
# Answer: then O(n log k) is close to O(n log n) and the heap has no
# advantage: use the ordinary sort. Say where the crossover is.
def sort_any(arr, k):
    if k >= len(arr) // 2:
        return sorted(arr)
    return sort_k_sorted(arr, k)


# ---- follow-up 3: tests ----
def test_follow_up_3():
    random.seed(19)
    for round_number in range(300):
        arr = []
        for i in range(random.randint(0, 9)):
            arr.append(random.randint(0, 9))
        k = find_k(arr)
        assert sort_k_sorted(arr, k) == sorted(arr)      # k is valid
        assert sort_any(arr, k) == sorted(arr)
    for round_number in range(300):                      # different values: compare with the plain definitio
n
```

```
        arr = random.sample(range(30), random.randint(2, 8))
        place = {}
        for i in range(len(arr)):
            place[sorted(arr)[i]] = i
        farthest = 0
        for i in range(len(arr)):
            distance = abs(i - place[arr[i]])
            if distance > farthest:
                farthest = distance
        assert find_k(arr) == farthest


# ======================================================
# RUN EVERYTHING
# ======================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
```

```
    test_follow_up_2()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

247 Merge k sorted linked lists. HARD

Template fit: seed with each list head (value, list number, node); the successor of a node is node.next

QUESTION

You are given k linked lists, each sorted in ascending order. Merge them into one sorted linked list and return its head. (In the examples the linked lists are written as arrays.)

Function signature

def merge_k_heap(lists: list) -> ListNode

EXAMPLES

Example 1	
Input	`lists = [[1, 4, 5], [1, 3, 4], [2, 6]]`
Output	`_to_list(merge_k_heap([mk([1, 4, 5]), mk([1, 3, 4]), mk([2, 6])])) -> [1, 1, 2, 3, 4, 4, 5, 6]`
Why	The three sorted lists merge into one sorted list.

Example 2	
Input	`lists = []`
Output	`_to_list(merge_k_heap([])) -> []`
Why	No lists: an empty result.

FULL CODE | main question first, then every follow-up, tests at the end

```
# ======================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ======================================================
```

```
import heapq

END = object()                                  # marks "this stream has no more items"


# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next


def make(values):
    # a linked list from a Python list
    dummy = ListNode()
    tail = dummy
    for v in values:
        tail.next = ListNode(v)
        tail = tail.next
    return dummy.next


def to_list(node):
    values = []
    while node:
        values.append(node.val)
        node = node.next
    return values


def merge_two(a, b):
    # merge two sorted linked lists into one
    dummy = ListNode()
    tail = dummy
    while a and b:
```

```
        if a.val <= b.val:
            tail.next = a
            a = a.next
        else:
            tail.next = b
            b = b.next
        tail = tail.next
    if a:
        tail.next = a
    else:
        tail.next = b
    return dummy.next


# ---- main question: main function ----
# Idea: a min-heap holds the CURRENT head of every list as (value, list
# number, node). Take the smallest out, add it to the result and put
# its successor in the heap. The list number breaks ties so two nodes are
# never compared. O(N log k) for N nodes in k lists.
def merge_k_heap(lists):
    heap = []
    for i in range(len(lists)):
        if lists[i]:
            heap.append((lists[i].val, i, lists[i]))
    heapq.heapify(heap)
    dummy = ListNode()
```

```
    tail = dummy
    while heap:
        value, i, node = heapq.heappop(heap)
        tail.next = node
        tail = node
        if node.next:
            heapq.heappush(heap, (node.next.val, i, node.next))
    return dummy.next


# ---- main question: tests ----
DATA = [[1, 4, 5], [1, 3, 4], [2, 6]]
WANTED = [1, 1, 2, 3, 4, 4, 5, 6]


def build_all():
    lists = []
    for values in DATA:
        lists.append(make(values))
```

```
    return lists


def test_main_question():
    assert to_list(merge_k_heap(build_all())) == WANTED
    assert merge_k_heap([]) is None
    assert to_list(merge_k_heap([None, make([1])])) == [1]
    assert to_list(merge_two(make([1, 3]), make([2]))) == [1, 2, 3]


# =========================================================
# FOLLOW-UP 1: Merge the lists in pairs instead of using a heap
# =========================================================
# ---- follow-up 1: function ----
# Answer: repeatedly merge neighbouring pairs of lists with the two-list
# merge. There are log k rounds and each touches all N nodes: O(N log k)
# time and no heap.
def merge_k_pairs(lists):
    lists = [node for node in lists if node]
    if not lists:
        return None
    while len(lists) > 1:
        merged = []
        for i in range(0, len(lists), 2):
            if i + 1 < len(lists):
                merged.append(merge_two(lists[i], lists[i + 1]))
            else:
                merged.append(lists[i])
        lists = merged
    return lists[0]


# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert to_list(merge_k_pairs(build_all())) == WANTED
    assert merge_k_pairs([None]) is None


# =========================================================
# FOLLOW-UP 2: The lists are too large for memory (files)
# =========================================================
# ---- follow-up 2: function ----
# Answer: the heap approach already works on streams: it holds just one
# item per list. Read each input through an iterator and write the output
# as you go. This is the merge step of an external sort.
```

```
def merge_streams(iterators):
    heap = []
    for i in range(len(iterators)):
        first = next(iterators[i], END)
        if first is not END:
            heap.append((first, i))
    heapq.heapify(heap)
    while heap:
        value, i = heapq.heappop(heap)
        yield value
        following = next(iterators[i], END)
        if following is not END:
            heapq.heappush(heap, (following, i))


# ---- follow-up 2: tests ----
def test_follow_up_2():
    iterators = []
    for values in DATA:
        iterators.append(iter(values))
    assert list(merge_streams(iterators)) == WANTED


# =====================================================
# RUN EVERYTHING
# =====================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

248 Return top-k subset sums in descending order

HARD

Template fit: heap of (sum, index); each popped state pushes two successors: add the next item, or swap the last item for the next

QUESTION

Given a list of integers (possibly negative), return the k largest subset sums in descending order. The empty subset counts and has sum 0; different subsets with the same sum are listed separately.

Function signature

def top_k_sums(nums: list, k: int) -> list

EXAMPLES

Example 1	
Input	`[1, 2, 3], 4`
Output	`[6, 5, 4, 3]`
Why	`The eight subset sums are 6, 5, 4, 3, 3, 2, 1, 0; the top four are 6, 5, 4, 3.`

Example 2	
Input	`[3, -2, 1], 3`
Output	`[4, 3, 2]`
Why	`With a negative number: 3 + 1 = 4, then 3, then 3 + (-2) + 1 = 2.`

FULL CODE | main question first, then every follow-up, tests at the end

```python
# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ========================================================

import heapq


# ========================================================
# MAIN QUESTION
# ========================================================
# ---- main question: helper functions ----
def sorted_costs(nums):
    # |x| of every number, smallest first
    costs = []
    for x in nums:
        if x < 0:
            costs.append(-x)
        else:
            costs.append(x)
    costs.sort()
    return costs


# ---- main question: main function ----
# Return the k biggest sums of subsets of nums (the empty subset counts).
# Idea: the best subset takes every positive number. Any other subset
# is the best one with some numbers "flipped" (a positive one left out or a
# negative one added), and flipping x costs |x|. So the k best subsets are
# the k CHEAPEST sets of flips. Generate the flip sets in order of cost with a
# heap over the sorted costs: from a set whose last flipped item is i, the
# two successors are (a) add item i + 1, (b) replace item i by item i + 1.
# Neither is ever cheaper than the set itself, so the heap yields them in order.
def top_k_sums(nums, k):
    base = 0
    for x in nums:
        if x > 0:
            base += x
    costs = sorted_costs(nums)
    flip_costs = [0]                                    # the best subset costs 0 flips
    heap = []
    if costs:
        heap.append((costs[0], 0))                      # (total cost, index of the last flipped item)
    while heap and len(flip_costs) < k:
        cost, i = heapq.heappop(heap)
        flip_costs.append(cost)
        if i + 1 < len(costs):
            heapq.heappush(heap, (cost + costs[i + 1], i + 1))                  # add the next item
            heapq.heappush(heap, (cost - costs[i] + costs[i + 1], i + 1))       # move the last item up by o
ne
    sums = []
    for cost in flip_costs[:k]:
        sums.append(base - cost)
    return sums


# ---- main question: tests ----
def test_main_question():
    assert top_k_sums([3, -1, 2, -4], 4) == [5, 4, 3, 2]
    assert top_k_sums([], 3) == [0]
    assert top_k_sums([5, -2, 7, 1], 3) == [13, 12, 11]


# ========================================================
```

```python
# FOLLOW-UP 1: Return the subsets themselves, not only the sums
# ======================================================
# ---- follow-up 1: function ----
# Answer: do not copy lists into the heap. Store each state as a link
# (last flipped item, the state it came from) and rebuild a subset by
# walking the links only for the k answers. Every push stays O(1).
def top_k_subsets(nums, k):
    order = []                                          # item numbers from the cheapest to the most expensive
    for i in range(len(nums)):
        order.append(i)
    order.sort(key=lambda i: abs(nums[i]))              # (the sort key is the size of the number)
    costs = []
    for i in order:
        costs.append(abs(nums[i]))
    best = set()
    base = 0
    for i in range(len(nums)):
        if nums[i] > 0:
            best.add(i)
            base += nums[i]

    def subset_of(chain):
        flipped = set()
        while chain is not None:
            flipped.add(order[chain[0]])
            chain = chain[1]
        chosen = []
        for i in range(len(nums)):
            if (i in best) != (i in flipped):           # the best subset with the flipped items changed
                chosen.append(nums[i])
        return sorted(chosen)

    results = [(base, subset_of(None))]
    heap = []
    if costs:
        heap.append((costs[0], 0, (0, None)))           # (cost, last item, the chain of flipped items)
    while heap and len(results) < k:
        cost, i, chain = heapq.heappop(heap)
        results.append((base - cost, subset_of(chain)))
        if i + 1 < len(costs):
            heapq.heappush(heap, (cost + costs[i + 1], i + 1, (i + 1, chain)))
            heapq.heappush(heap, (cost - costs[i] + costs[i + 1], i + 1, (i + 1, chain[1])))
    return results[:k]


# ---- follow-up 1: tests ----
def test_follow_up_1():
    results = top_k_subsets([3, -1, 2, -4], 4)
    totals = []
    for total, subset in results:
        assert sum(subset) == total                     # the subsets add up to the sums
        totals.append(total)
    assert totals == [5, 4, 3, 2]
    assert results[0][1] == [2, 3]


# ======================================================
# FOLLOW-UP 2: k can be as large as 2^n
# ======================================================
# Answer: the heap holds at most k entries and costs O(k log k), which is
# fine for k up to millions but not for 2^n. For huge k, binary-search the
# sum: count how many subsets have a sum at least s (meet in the middle),
# and find the k-th value directly.
```

```
# ============================================================
# FOLLOW-UP 3: Limit subsets to at most m elements
# ============================================================
# Answer: keep the number of elements in each heap state and skip a
# successor that would use more than m elements.
```

```
# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

GROUP 36 OF 43 | 5 QUESTIONS, ONE CODE TEMPLATE | 1 E 4 M

Tree recursion, Tier 2: pass a rule down, return a node or a yes/no up

249 Next Birthday (BST In-Order Successor with Wraparound) EASY

Template fit: descend(): go left after a hit (a closer candidate may exist), right otherwise; if no candidate exists wrap around to the leftmost node

QUESTION

This is the "alarm / next-birthday" problem: from a set of birthdays stored in a binary search tree, find the next one that will occur. Each birthday is stored as an integer day-of-year in [1, 365] , and the tree is a valid BST keyed on that value. Given the root of the BST and an integer today , return the next upcoming birthday: the smallest key that is strictly greater than today . If no key is strictly greater than today , the calendar wraps around, so return the smallest key in the whole tree. If the tree is empty, return None .

Function signature

```
from typing import Optional
class TreeNode:
    def __init__(self, val: int, left: "Optional[TreeNode]" = None,
                 right: "Optional[TreeNode]" = None) -> None:
        self.val = val
        self.left = left
        self.right = right
def next_birthday(root: Optional[TreeNode], today: int) -> Optional[int]:
    """Return the smallest key strictly greater than today, wrapping around
    to the smallest key if none is greater. Return None if the tree is empty."""
    ...
```

Follow-up (as reported): If the input were an unsorted binary tree (not a BST), extract all values, sort them, and binary-search for the successor. Hint: bisect_right .

EXAMPLES

Example 1	
Input	root = above, today = 25
Output	30
Why	Smallest key strictly greater than 25 is 30.

Example 2	
Input	root = above, today = 40
Output	10
Why	No key > 40, so wrap around to the smallest key, 10.

Example 3	
Input	root = above, today = 5
Output	10
Why	Smallest key greater than 5 is 10.

Example 4	
Input	root = None, today = 100
Output	None

CONSTRAINTS

QUESTION

- 0 <= number of nodes <= 10^5 ; all node values are distinct and in [1, 365] .
- 1 <= today <= 365 .
- Aim for O(h) time where h is the tree height (do not sort all nodes if you can avoid it).

FULL CODE | main question first, then every follow-up, tests at the end

```
# ======================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ======================================================

# ======================================================
# MAIN QUESTION
# ======================================================
# ---- main question: helper functions ----
class TreeNode:
    def __init__(self, val, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right


# ---- main question: main function ----
# Idea: the birthdays are in a binary SEARCH tree. Walk down once: a
# value above today is a candidate (the answer might be even closer in
# its left subtree), a value at or below today sends us right. If no
# value is above today, wrap around to the smallest value (the leftmost node).
def next_birthday(root, today):
    if root is None:
        return None
    best = None
    node = root
    while node:
        if node.val > today:
            best = node.val
            node = node.left
        else:
            node = node.right
    if best is not None:
        return best
    node = root
    while node.left:
        node = node.left
    return node.val


# ---- main question: tests ----
def example_tree():
    return TreeNode(20, TreeNode(10), TreeNode(30, None, TreeNode(40)))
```

```


def test_main_question():
    root = example_tree()
    assert next_birthday(root, 25) == 30
    assert next_birthday(root, 40) == 10                    # nothing later: wrap around
    assert next_birthday(root, 5) == 10
    assert next_birthday(None, 100) is None


# ======================================================
# FOLLOW-UP 1: The tree is not a BST. What now?
```

```
# ============================================================
# ---- follow-up 1: function ----
# Answer: the ordering can no longer be used, so look at every value:
# the best one is the smallest value above today, or the smallest value
# overall if there is none. O(n) per question. For many questions on the
# same tree, sort the values once and binary-search (bisect) for the
# first value above today: O(log n) per question.
def next_birthday_unsorted(root, today):
    if root is None:
        return None
    smallest = None
    next_one = None
    stack = [root]
    while stack:
        node = stack.pop()
        if smallest is None or node.val < smallest:
            smallest = node.val
        if node.val > today and (next_one is None or node.val < next_one):
            next_one = node.val
        for child in (node.left, node.right):
            if child:
                stack.append(child)
    if next_one is not None:
        return next_one
    return smallest


# ---- follow-up 1: tests ----
def test_follow_up_1():
    messy = TreeNode(30, TreeNode(40), TreeNode(10, TreeNode(20)))     # not in search-tree order
    assert next_birthday_unsorted(messy, 25) == 30
    assert next_birthday_unsorted(messy, 40) == 10
    assert next_birthday_unsorted(example_tree(), 25) == next_birthday(example_tree(), 25)
```

```
    assert next_birthday_unsorted(None, 1) is None


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

250 Lowest Common Ancestor

MEDIUM

Template fit: dfs returns the node if it is p or q; a node that gets an answer from both sides is the ancestor

QUESTION

Given the root of a binary tree and two of its nodes p and q, find their lowest common ancestor: the lowest node that has both p and q as descendants (a node can be a descendant of itself).

Function signature

def lowestCommonAncestor_tree(root: BNode, p: BNode, q: BNode) -> BNode

EXAMPLES

QUESTION

Example 1

Input	`root = [3, [5, [6], [2]], [1]] (value, left, right)`
	`first query: p = 5, q = 1`
	`second query: p = 5, q = 6`

Output	`lowestCommonAncestor_tree(root, p, q).val -> 3` `lowestCommonAncestor_tree(root, p, p.left).val -> 5`
Why	`5 and 1 are in different subtrees of 3, so 3 is their lowest common ancestor. 5 is an ancestor of 6, and a node may be its own ancestor, so the answer is 5.`

CONSTRAINTS

- Number of nodes in tree: [2, 10^5]
- -10^9 <= Node.val <= 10^9
- All node values are unique
- p != q
- p and q exist in the tree

FULL CODE | main question first, then every follow-up, tests at the end

```
# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ========================================================

import random


# ========================================================
# MAIN QUESTION
# ========================================================
# ---- main question: helper functions ----
class Node:
    def __init__(self, val, children=None):
        # two children for a binary tree, any number for an n-ary tree
        self.val = val
        self.children = []
        if children:
            self.children = children


# ---- main question: main function ----
# Idea: one recursive pass. find(node) answers "where did I meet p or q
# in this subtree?": the node itself if it is p or q, otherwise what the
# children report. If TWO children each found something, this node is
# the lowest common ancestor; if only one did, pass that result up.
# The same code works for an n-ary tree (follow-up 1): it just loops
# over every child.
def lca(root, p, q):
    def find(node):
        if node is None:
            return None
        if node is p or node is q:
            return node
        hits = []
        for child in node.children:
            found = find(child)
            if found is not None:
                hits.append(found)
        if len(hits) >= 2:
```

```
            return node
        if len(hits) == 1:
            return hits[0]
```

```
        return None

    return find(root)


# ---- main question: tests ----
def example_tree():
    six, four = Node(6), Node(4)
    two = Node(2, [Node(7), four])
    five = Node(5, [six, two])
    one = Node(1, [Node(0), Node(8)])
    return Node(3, [five, one]), five, one, six, four


def test_main_question():
    root, five, one, six, four = example_tree()
    assert lca(root, six, four) is five
    assert lca(root, five, four) is five          # a node can be its own ancestor
    assert lca(root, six, one) is root
    kids = [Node(0), Node(1), Node(2)]            # an n-ary tree: three children under one node
    top = Node("top", [Node("mid", kids), Node("side")])
    assert lca(top, kids[0], kids[2]).val == "mid"


# ========================================================
# FOLLOW-UP 1: The tree is n-ary
# ========================================================
# Answer: nothing to change: lca() above already collects the hits of
# every child; two or more hits make this node the answer.


# ========================================================
# FOLLOW-UP 2: A very deep tree, or no recursion allowed
# ========================================================
# ---- follow-up 2: helper functions ----
def parents_of(root):
    # one walk with a stack (no recursion) that remembers every node's parent
    parent = {root: None}
    stack = [root]
    while stack:
        node = stack.pop()
        for child in node.children:
            parent[child] = node
            stack.append(child)
    return parent
```

```


# ---- follow-up 2: function ----
# Answer: record every node's parent, then climb from p collecting its
# ancestors, and climb from q until you meet one of them: O(n) time and
# space, no recursion.
def lca_iterative(root, p, q):
    parent = parents_of(root)
    above = set()
    node = p
    while node:
        above.add(node)
        node = parent[node]
    node = q
    while node not in above:
        node = parent[node]
```

```
    return node


# ---- follow-up 2: tests ----
def test_follow_up_2():
    root, five, one, six, four = example_tree()
    assert lca_iterative(root, six, four) is five
    chain = Node(0)                                  # a chain of 5000 nodes: no recursion limit
    last = chain
    for i in range(1, 5000):
        child = Node(i)
        last.children.append(child)
        last = child
    assert lca_iterative(chain, last, chain) is chain
    random.seed(3)
    for round_number in range(100):                  # random trees: both versions must agree
        nodes = [Node(0)]
        for i in range(1, random.randint(1, 10)):
            child = Node(i)
            random.choice(nodes).children.append(child)
            nodes.append(child)
        p = random.choice(nodes)
        q = random.choice(nodes)
        assert lca(nodes[0], p, q) is lca_iterative(nodes[0], p, q)


# ========================================================
# FOLLOW-UP 3: p or q might not be in the tree
# ========================================================
```

```
# ---- follow-up 3: function ----
# Answer: the plain recursion would return the one that does exist,
# which is wrong. First check that both nodes are really in the tree
# (they are in the parent map), and return None if not.
def lca_checked(root, p, q):
    parent = parents_of(root)
    if p not in parent or q not in parent:
        return None
    return lca_iterative(root, p, q)


# ---- follow-up 3: tests ----
def test_follow_up_3():
    root, five, one, six, four = example_tree()
    stranger = Node(404)
    assert lca_checked(root, six, stranger) is None
    assert lca_checked(root, six, four) is five


# ========================================================
# RUN EVERYTHING
# ========================================================
def run_all_tests():
    test_main_question()
    test_follow_up_2()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

251 Determine if a given binary tree is a binary search tree (BST). MEDIUM

Template fit: check(): pass lo and hi bounds down; fail when a value leaves its allowed window

QUESTION

Given the root of a binary tree, determine whether it is a valid binary search tree: for every node, all values in its left subtree are smaller and all values in its right subtree are larger (strictly).

Function signature

def isValidBST(root: TreeNode) -> bool

EXAMPLES

Example 1	
Input	`[2, 1, 3]`
Output	`true`
Why	`2 with 1 on the left and 3 on the right.`

Example 2	
Input	`[5, 1, 4, null, null, 3, 6]`
Output	`false`
Why	`The 3 sits in the right subtree of 5 but is smaller than 5.`

Example 3	
Input	`[2, 2, 2]`
Output	`false`
Why	`A node equal to its parent breaks the strict rule.`

FULL CODE | main question first, then every follow-up, tests at the end

```
# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
class TreeNode:
    def __init__(self, val, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right


def value_fits(val, low, high, dup):
    # may `val` sit between the bounds from its ancestors (None = no bound)?
    # dup says where an EQUAL value is allowed: "none", "left" or "right"
    if low is not None:
        if val < low:
            return False
        if val == low and dup != "right":
            return False
    if high is not None:
        if val > high:
            return False
        if val == high and dup != "left":
```

```
            return False
    return True


# ---- main question: main function ----
# Idea: pass the allowed range (low, high) DOWN. The left child inherits
# (low, node.val) and the right child (node.val, high); the root starts
# with no bounds. Every node must lie strictly between its bounds.
# (dup is for follow-up 1; leave it as "none" for a normal BST.)
def isValidBST(root, dup="none"):
    stack = [(root, None, None)]
    while stack:
        node, low, high = stack.pop()
        if node is None:
            continue
```

```
        if not value_fits(node.val, low, high, dup):
            return False
        stack.append((node.left, low, node.val))
        stack.append((node.right, node.val, high))
    return True


# ---- main question: tests ----
def test_main_question():
    good = TreeNode(2, TreeNode(1), TreeNode(3))
    bad = TreeNode(5, TreeNode(1), TreeNode(4, TreeNode(3), TreeNode(6)))   # 6 is below 5, so it must be sma
ller than 5
    assert isValidBST(good)
    assert not isValidBST(bad)
    assert isValidBST(None)


# =====================================================
# FOLLOW-UP 1: Duplicates are allowed (all on the left, or all on the right)
# =====================================================
# Answer: make ONE of the two bounds inclusive. With duplicates on the
# left, a node may equal its upper bound; with duplicates on the right
# it may equal its lower bound (the dup parameter above).
# ---- follow-up 1: tests ----
def test_follow_up_1():
    left_dup = TreeNode(2, TreeNode(2), None)
    right_dup = TreeNode(2, None, TreeNode(2))
    assert not isValidBST(left_dup)
    assert isValidBST(left_dup, "left")
    assert not isValidBST(left_dup, "right")
    assert isValidBST(right_dup, "right")
    assert not isValidBST(right_dup, "left")


# =====================================================
# FOLLOW-UP 2: Do it with an in-order walk that keeps only the previous value
# =====================================================
# ---- follow-up 2: function ----
# Answer: an in-order walk must be strictly increasing, so keep only
# the previous value: O(height) space and an early exit.
def is_bst_inorder(root):
    stack = []
    node = root
    previous = None
    while stack or node:
        while node:
```

```
            stack.append(node)
            node = node.left
        node = stack.pop()
```

```
        if previous is not None and node.val <= previous:
            return False
        previous = node.val
        node = node.right
    return True


# ---- follow-up 2: tests ----
def test_follow_up_2():
    good = TreeNode(2, TreeNode(1), TreeNode(3))
    bad = TreeNode(5, TreeNode(1), TreeNode(4, TreeNode(3), TreeNode(6)))
    assert is_bst_inorder(good)
    assert not is_bst_inorder(bad)


# ======================================================
# FOLLOW-UP 3: Recover a BST where exactly two nodes were swapped (LeetCode 99)
# ======================================================
# ---- follow-up 3: function ----
# Answer: in an in-order walk the swap shows up as one or two places
# where a value is smaller than the one before it (a "descent"). The
# first node of the first descent and the second node of the last
# descent are the swapped pair. O(n) time, O(height) space.
def recover(root):
    stack = []
    node = root
    previous = None
    first = None
    second = None
    while stack or node:
        while node:
            stack.append(node)
            node = node.left
        node = stack.pop()
        if previous is not None and node.val < previous.val:
            second = node
            if first is None:
                first = previous
        previous = node
        node = node.right
    if first is not None:
        first.val, second.val = second.val, first.val
```

```
    return root


# ---- follow-up 3: tests ----
def test_follow_up_3():
    swapped = TreeNode(3, TreeNode(1), TreeNode(2))        # [2, 1, 3] with 3 and 2 swapped
    recover(swapped)
    assert swapped.val == 2 and swapped.left.val == 1 and swapped.right.val == 3
    assert isValidBST(swapped)


# ======================================================
# RUN EVERYTHING
# ======================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
```

```
run_all_tests()
```

252 Find mode in a trinary search tree

MEDIUM

Template fit: in-order walk (left, middle, right); equal values are neighbours, so one running count finds the mode(s)

QUESTION

You are given the root of a **trinary search tree** where each node has up to three children:

```
class Node { int val; Node left; // all values < val Node middle; // all values = val Node right; // all v
alues > val }
```

Assume the tree is intended to satisfy the ordering rules above.

Task

Return the **mode(s)** of the tree: the value(s) that occur most frequently among all nodes.

- If there is one mode, return a single-element list.
- If multiple values tie for highest frequency, return **all** such values (in any order).

Constraints (typical)

- Number of nodes **n** can be large (e.g., up to **`10^5`**).
- Values can be any 32-bit signed integers.

Follow-up

How would your approach change if the tree is **broken** (i.e., it may not respect the **<**, **=**, **>** constraints), but you still must return the correct mode(s)?

Part 1: Find Mode(s) in a Valid Trinary Search TreeYou are given a valid trinary search tree. Each node has an integer value and up to three children: left, middle, and right. For every node, all values in its left subtree are strictly less than the node's value, all values in its middle subtree are equal to the node's value, and all values in its right subtree are strictly greater than the node's value. The tree is encoded using parallel arrays instead of Node objects. Return the value or values that occur most frequently in the tree. To make the output deterministic, return the modes in increasing order. If the tree is empty, return an empty list.
Constraints
0 <= n <= 100000, where n == len(values)len(left) == len(middle) == len(right) == nEach child index is either -1 or an integer in [0, n - 1]root == -1 if n == 0; otherwise 0 <= root < nThe nodes reachable from root form a valid tree with no cyclesThe tree satisfies the trinary search ordering rules recursivelyEach value is a 32-bit signed integer

EXAMPLES

```
Input: ([], [], [], [], -1)
Expected Output: []
```

Explanation: The tree is empty, so there are no modes.

```
Input: ([42], [-1], [-1], [-1], 0)
Expected Output: [42]
```

Explanation: A single-node tree has that node's value as the only mode.

QUESTION

Hints

In a valid trinary search tree, an inorder traversal using left, node, middle, right visits values in nondecreasing order.Because equal values appear consecutively during that traversal, you can track only the current run length and the best run length so far.

Part 2: Find Mode(s) in a Broken Trinary TreeYou are given a trinary tree with integer values. Each node has up to three children: left, middle, and right. Unlike a valid trinary search tree, this tree may be broken: the left, middle, and right children do not necessarily follow any ordering rules. Return the value or values that occur most frequently among all reachable nodes. To make the output deterministic, return the modes in increasing order. If the tree is empty, return an empty list.

Constraints

0 <= n <= 100000, where n == len(values)len(left) == len(middle) == len(right) == nEach child index is either -1 or an integer in [0, n - 1]root == -1 if n == 0; otherwise 0 <= root < nThe nodes reachable from root form a tree with no cyclesNo ordering rule is guaranteed between a node and its childrenEach value is a 32-bit signed integer

Examples

Input: ([], [], [], [], -1)

Expected Output: []

Explanation: The tree is empty, so there are no modes.

Input: ([-2147483648], [-1], [-1], [-1], 0)

Expected Output: [-2147483648]

Explanation: A single-node tree has that node's value as the only mode.

Hints

Without the ordering property, equal values are not guaranteed to be adjacent in any traversal.Use a frequency table while traversing the entire tree.

FULL CODE | main question first, then every follow-up, tests at the end

```
# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ========================================================

# ========================================================
# MAIN QUESTION
# ========================================================
# ---- main question: helper functions ----
class TNode:
    # a trinary search tree node: everything in `left` is smaller than
    # val, `middle` holds a copy of val, everything in `right` is bigger
    def __init__(self, val, left=None, middle=None, right=None):
        self.val = val
        self.left = left
        self.middle = middle
        self.right = right


class ModeTracker:
    # values arrive in sorted order, so equal values come one after the
    # other and a running count is enough (no dictionary)
    def __init__(self):
        self.best = 0              # the longest run so far
        self.modes = []            # the values that have that run length
        self.current = None
        self.run = 0

    def add(self, value):
        if value == self.current:
```

```
            self.run += 1
        else:
            self.run = 1
        self.current = value
        if self.run > self.best:
            self.best = self.run
            self.modes = [value]
        elif self.run == self.best and value not in self.modes:
            self.modes.append(value)


# ---- main question: main function ----
# Idea: an IN-ORDER walk (left, middle chain, right) gives the values in
```

```
# sorted order. Use an explicit stack: O(height) extra space.
def find_modes(root):
    tracker = ModeTracker()
    stack = []
    node = root
    while stack or node:
        while node:                         # go as far left as possible
            stack.append(node)
            node = node.left
        node = stack.pop()
        tracker.add(node.val)
        middle = node.middle                # the middle chain repeats node.val once per node
        while middle:
            tracker.add(middle.val)
            middle = middle.middle
        node = node.right
    return sorted(tracker.modes)


# ---- main question: tests ----
#        5
#     /  |  \
#    3   5   8
#   /|       |
#  3 3       8
def example_tree():
    threes = TNode(3, None, TNode(3, None, TNode(3)))
    return TNode(5, threes, TNode(5), TNode(8, None, TNode(8)))


def test_main_question():
    assert find_modes(example_tree()) == [3]
    assert find_modes(None) == []
    tie = TNode(2, TNode(1), TNode(2), None)
    assert find_modes(tie) == [2]
    assert find_modes(TNode(2, TNode(1), None, TNode(3))) == [1, 2, 3]       # a three-way tie


# ========================================================
# FOLLOW-UP 1: The tree may be broken (it does not respect the ordering)
# ========================================================
# ---- follow-up 1: function ----
# Answer: an in-order walk no longer gives sorted values, so equal
# values need not be next to each other. Fall back to a frequency table
# over a full traversal: O(n) time, O(distinct values) space.
```

```
def find_modes_counter(root):
    counts = {}
    stack = []
    if root:
        stack.append(root)
    while stack:
```

```
        node = stack.pop()
        counts[node.val] = counts.get(node.val, 0) + 1
        for child in (node.left, node.middle, node.right):
            if child:
                stack.append(child)
    top = 0
    for count in counts.values():
        if count > top:
            top = count
    modes = []
    for value in counts:
        if counts[value] == top:
            modes.append(value)
    modes.sort()
    return modes


# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert find_modes_counter(example_tree()) == [3]
    assert find_modes_counter(TNode(2, TNode(1), TNode(2), None)) == [2]
    assert find_modes_counter(None) == []


# =============================================================
# FOLLOW-UP 2: Only O(1) extra space (no stack, no dictionary)
# =============================================================
# Answer: use Morris in-order traversal, which threads temporary links
# through the tree instead of using a stack (and restores them).


# =============================================================
# FOLLOW-UP 3: The tree is huge and values arrive in a stream
# =============================================================
# Answer: keep a ModeTracker and feed it the values as they arrive in
# sorted order; if they are unsorted use a counter and a "top" value.


# =============================================================
```

```
# RUN EVERYTHING
# =============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

253 Next Upcoming Birthday in a Family Tree — MEDIUM

Template fit: walk the ancestors with a stack and a seen set, keep the best birthday by the circular distance from today

QUESTION

You are given a family tree rooted at "me". Each Person node has a name , a birthday given as (month, day) , and pointers to parent1 and parent2 (either may be None). Following parent pointers reaches the whole ancestry; treat the reachable set of Person nodes as the tree to search (each distinct person counted once, even if reachable by two paths). Given today's date (today_month, today_day) , implement alarm_next_birthday(me, today_month, today_day) that scans every person and returns the (month, day) of the soonest upcoming birthday, where a birthday landing exactly on today counts as upcoming (distance 0) and the calendar wraps around the year. If the tree is empty (me is None), return the empty string "" . Use this ordering key to compare dates: key(m, d) = (m - 1) * 31 + d , and cyclic distance (key(person) - key(today)) mod 372 . Return the person with the smallest such distance; if several people share that soonest date, return that (month, day) .

Function signature

```
from typing import Optional, Union
class Person:
    def __init__(self, name: str, month: int, day: int,
                 parent1: "Optional[Person]" = None,
                 parent2: "Optional[Person]" = None) -> None:
        self.name = name
        self.month = month
        self.day = day
        self.parent1 = parent1
        self.parent2 = parent2
def alarm_next_birthday(me: Optional[Person],
                        today_month: int,
                        today_day: int) -> Union[tuple[int, int], str]:
    """Return (month, day) of the nearest upcoming birthday, or "" if there is
nobody."""
    ...
```

Follow-up (as reported): Support repeated alarms: after reporting one birthday, the next call returns the following-soonest, and once all have been reported, return "" .

EXAMPLES

Example 1	
Input	`me = Me(12,1) with parent1 = Alice(9,25), parent2 = Bob(1,10); today = (9, 12)`
Output	`(9, 25)`
Why	`keys — today=260, Alice=273 (dist 13), Me=342 (dist 82), Bob=10 (dist (10- 260) mod 372 = 122). Alice is soonest.`

Example 2	
Input	`me = None; today = (5, 5)`
Output	`""`
Why	`No people to check.`

Example 3	
Input	`me = Me(3,3) alone; today = (3, 3)`
Output	`(3, 3)`
Why	`The only birthday is today, distance 0.`

CONSTRAINTS

- 1 <= month <= 12 , 1 <= day <= 31 .

QUESTION

- Up to 10^5 people; parent pointers form a DAG (no cycles).

FULL CODE | main question first, then every follow-up, tests at the end

```
# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ========================================================

# ========================================================
# MAIN QUESTION
# ========================================================
# ---- main question: helper functions ----
class Person:
    def __init__(self, name, month, day, parent1=None, parent2=None):
        self.name = name
        self.month = month
        self.day = day
        self.parent1 = parent1
        self.parent2 = parent2


def day_number(month, day):
    # a number that grows through the year; 12 months of up to 31 days = 372
    return (month - 1) * 31 + day


def all_people(me):
    # `me` and every ancestor, each person once (a person can be reached
    # through both parents, so remember who was seen)
    people = []
    seen = set()
    stack = []
    if me:
        stack.append(me)
    while stack:
        person = stack.pop()
        if person in seen:
            continue
        seen.add(person)
        people.append(person)
        for parent in (person.parent1, person.parent2):
            if parent:
                stack.append(parent)
    return people
```

```
# ---- main question: main function ----
# Idea: look at every person once and keep the birthday that comes next:
# the number of days from today to the birthday, going round the year
# (the "% 372" wraps a birthday that already passed to next year).
def alarm_next_birthday(me, today_month, today_day):
    if me is None:
        return ""
    today = day_number(today_month, today_day)
    best_distance = None
    answer = ""
    for person in all_people(me):
        distance = (day_number(person.month, person.day) - today) % 372
        if best_distance is None or distance < best_distance:
            best_distance = distance
            answer = (person.month, person.day)
    return answer
```

```
# ---- main question: tests ----
def test_main_question():
    alice = Person("Alice", 9, 25)
    bob = Person("Bob", 1, 10)
    me = Person("Me", 12, 1, alice, bob)
    assert alarm_next_birthday(me, 9, 12) == (9, 25)
    assert alarm_next_birthday(None, 5, 5) == ""
    assert alarm_next_birthday(Person("Me", 3, 3), 3, 3) == (3, 3)      # today counts


# ========================================================
# FOLLOW-UP 1: Support repeated alarms: each call returns the next birthday after the previous one
# ========================================================
# ---- follow-up 1: function ----
# Answer: collect the distinct (days away, month, day) once, sort them,
# and keep an index: each call returns the next entry. If people can be
# added later use a heap keyed by the number of days instead.
class BirthdayAlarms:
    def __init__(self, me, today_month, today_day):
        today = day_number(today_month, today_day)
        dates = set()
        for person in all_people(me):
            distance = (day_number(person.month, person.day) - today) % 372
            dates.add((distance, person.month, person.day))
        self.order = sorted(dates)
        self.position = 0
```

```
    def next(self):
        if self.position >= len(self.order):
            return ""
        distance, month, day = self.order[self.position]
        self.position += 1
        return (month, day)


# ---- follow-up 1: tests ----
def test_follow_up_1():
    alice = Person("Alice", 9, 25)
    bob = Person("Bob", 1, 10)
    me = Person("Me", 12, 1, alice, bob)
    alarms = BirthdayAlarms(me, 9, 12)
    answers = []
    for i in range(4):
        answers.append(alarms.next())
    assert answers == [(9, 25), (12, 1), (1, 10), ""]


# ========================================================
# RUN EVERYTHING
# ========================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

GROUP 37 OF 43 | 5 QUESTIONS, ONE CODE TEMPLATE | 4 M 1 H

Random pick without repeats: swap the chosen item with the last one and pop

254 Random Generator Without Repeats

MEDIUM

Template fit: the template as written: values 0..n-1, every call returns a new one; raise when empty

QUESTION

Design a class RandomGenerator initialized with a positive integer n . Each call to generate() returns one integer chosen uniformly at random from the values in [0, n-1] that have not yet been returned. After n calls every value in [0, n-1] has been returned exactly once; a further call must raise IndexError (the pool is empty). Each generate() call must run in O(1) time. The intended approach maintains an array of the remaining values and, on each call, swaps a randomly chosen remaining value to the end of the active region and shrinks that region (partial Fisher-Yates).

Function signature

```
import random
class RandomGenerator:
    def __init__(self, n: int) -> None:
        """Prepare to yield a random permutation of [0, n-1], one value per
generate()."""
        ...
    def generate(self) -> int:
        """Return a uniformly random not-yet-returned value in O(1).
        Raise IndexError if all values have already been generated."""
        ...
```

Follow-up (as reported): Add a reset() that makes all n values available again in O(1) amortized.

EXAMPLES

Example 1

Input	`g = RandomGenerator(3); [g.generate() for _ in range(3)]`
Output	`e.g. [2, 0, 1]`
Why	`The three calls return a permutation of {0, 1, 2} with no repeats; any of the 6 permutations is a valid output. A 4th call would raise IndexError.`

Example 2

Input	`g = RandomGenerator(1); g.generate()`
Output	`0`
Why	`Only one value exists.`

CONSTRAINTS

- 1 <= n <= 10**6 .

FULL CODE | main question first, then every follow-up, tests at the end

```
# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

import random
```

```python
# ======================================================
# MAIN QUESTION
# ======================================================
# ---- main question: main function ----
# Idea (partial Fisher-Yates): keep the numbers 0..n-1 in an array and
# an "active region" at its start that holds the numbers not returned
# yet. To generate: pick a random position in the active region, swap
# that number to the END of the region, shrink the region by one and
# return the number that is now just outside it. O(1) per call.
class RandomGenerator:
    def __init__(self, n, seed=None):
        self.n = n
        self.values = list(range(n))
        self.size = n                                   # the active region is values[0:size]
        self.rng = random.Random(seed)

    def generate(self):
        if self.size == 0:
            raise IndexError("all values have been generated")
        i = self.rng.randrange(self.size)
        self.size -= 1
        self.values[i], self.values[self.size] = self.values[self.size], self.values[i]
        return self.values[self.size]


# ---- main question: tests ----
def test_main_question():
    for seed in range(20):
        gen = RandomGenerator(6, seed=seed)
        values = []
        for i in range(6):
            values.append(gen.generate())
        assert sorted(values) == [0, 1, 2, 3, 4, 5]
        try:
            gen.generate()
            raise AssertionError("an empty pool should raise")
        except IndexError:
            pass


# ======================================================
# FOLLOW-UP 1: Add reset() so all n values are available again in O(1)
# ======================================================
# ---- follow-up 1: function ----
# Answer: set `size` back to n. The array still holds every value 0..n-1
# (only their ORDER changed), and the next draws are uniform over the
# active region, so nothing has to be rebuilt.
class ResettableGenerator(RandomGenerator):
    def reset(self):
        self.size = self.n


# ---- follow-up 1: tests ----
def test_follow_up_1():
    gen = ResettableGenerator(4, seed=7)
    for i in range(4):
        gen.generate()
    gen.reset()
    values = []
    for i in range(4):
        values.append(gen.generate())
    assert sorted(values) == [0, 1, 2, 3]
```

```python
# ====================================================
# RUN EVERYTHING
# ====================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

255 Random Song Shuffler With a No-Repeat Window

MEDIUM

Template fit: the same pool, plus a deque of the last k songs; the played song leaves the pool and returns to it when it drops off the deque

QUESTION

Design a shuffler that plays songs at random from a fixed playlist, subject to one rule: the last k songs played may not be replayed. In other words, a song becomes eligible again only after k other (distinct) plays have occurred since it was last played. Implement the class so that each call to play() runs in O(1) time (no scanning of the playlist). The intended technique is a valid_songs list of currently eligible songs plus a song_to_index dictionary, using swap-and-pop to move a just-played song out of the eligible region and to re-admit the song that is leaving the window.

Function signature

```
import random
class SongShuffler:
    def __init__(self, songs: list[str], k: int) -> None:
        """Initialize with the playlist and the no-repeat window size k."""
        ...
    def play(self) -> str:
        """Return a uniformly random eligible song in O(1); it must not be
        one of the last k songs returned. Update state so the window slides."""
        ...
```

Follow-up (as reported): The interviewer wanted the naive deque solution replaced by O(1). Show why swap-and-pop with an index map achieves constant time per play() .

EXAMPLES

Example 1	
Input	`SongShuffler(["A", "B", "C"], k=1); then play(), play(), play(), play()`
Output	`e.g. "B", "A", "C", "B"`
Why	`k=1 forbids repeating the single most recent song. "B" then "A" is fine (A != B). Then "C" (C != A). Then "B" is allowed again because the only forbidden song is the immediately previous one ("C"). Any sequence with no two adjacent equal songs is valid.`

Example 2	
Input	`SongShuffler(["X", "Y"], k=0); play(), play()`
Output	`e.g. "Y", "Y"`
Why	`With k=0 there is no restriction, so repeats are allowed.`

CONSTRAINTS

- 1 <= len(songs) <= 10**5 , song titles are distinct strings.

QUESTION

- 0 <= k < len(songs) (so at least one song is always eligible). play() returns any eligible song chosen uniformly at random; every returned song must not be among the previous k returned songs.

FULL CODE | main question first, then every follow-up, tests at the end

```
# ======================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ======================================================

from collections import deque
import random


# ======================================================
# MAIN QUESTION
# ======================================================
# ---- main question: main function ----
# Idea: keep a list `valid` of the songs that may be played now, and a
# dictionary `index` with each song's place in that list. To play: pick a
# random index in `valid`. To REMOVE the played song in O(1), swap it
# with the LAST song of the list and pop (the swap-and-pop trick). The
# played song waits in a queue `recent` of the last k songs; when the
# queue is longer than k, the oldest song leaves it and is added back
# to `valid`. Every call is O(1).
class SongShuffler:
    def __init__(self, songs, k, seed=None):
        self.valid = list(songs)                    # songs that may be played now
        self.index = {}                             # song -> its place in valid
        for i in range(len(self.valid)):
            self.index[self.valid[i]] = i
        self.recent = deque()                       # the last k songs played, oldest first
        self.k = k
        self.rng = random.Random(seed)

    def play(self):
        i = self.rng.randrange(len(self.valid))
        song = self.valid[i]
        last = self.valid[-1]
        self.valid[i] = last                         # swap-and-pop: the last song takes the hole
        self.index[last] = i
        self.valid.pop()
        del self.index[song]
        self.recent.append(song)
        if len(self.recent) > self.k:
            back = self.recent.popleft()             # this song may be played again
            self.index[back] = len(self.valid)
            self.valid.append(back)
```

```
        return song


# ---- main question: tests ----
def test_main_question():
    tiny = SongShuffler(["A", "B", "C"], 1, seed=3)
    plays = []
    for i in range(40):
        plays.append(tiny.play())
    for i in range(1, len(plays)):
        assert plays[i] != plays[i - 1]                      # never the song just played
    shuffler = SongShuffler(["A", "B", "C", "D", "E"], 3, seed=4)
    plays = []
    for i in range(200):
```

```
        plays.append(shuffler.play())
    for i in range(len(plays)):
        for back in range(1, 4):
            if i - back >= 0:
                assert plays[i] != plays[i - back]            # not one of the last 3 songs
    assert set(plays) == {"A", "B", "C", "D", "E"}           # every song gets played
    assert len(SongShuffler(["X", "Y"], 0, seed=1).valid) == 2


# ========================================================
# FOLLOW-UP 1: Why is a plain deque or list not enough for O(1)?
# ========================================================
# Answer: picking a random ELIGIBLE song means choosing a random index of
# a list that holds only eligible songs. In a plain list, removing the
# played song from the middle shifts everything after it (O(n)); a deque
# is fast at its ends but has no O(1) random access in the middle.
# Swap-and-pop on a list plus an index dictionary removes a song in
# O(1), and the queue `recent` only touches its two ends.


# ========================================================
# RUN EVERYTHING
# ========================================================
def run_all_tests():
    test_main_question()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

256 Design set with O(1) random access

MEDIUM

Template fit: add appends and records the position; delete swaps with the last item and pops; get_random picks a random index

QUESTION

Design a data structure ("FancySet") that stores unique integers and supports the following operations, each in average O(1) time:

- **`add(x)`**: Insert value **`x`** if it is not already present. Return whether the insert happened.
- **`delete(x)`**: Remove value **`x`** if it exists. Return whether a deletion happened.
- **`getRandom()`**: Return a uniformly random element currently in the set. Requirements / follow-ups:
- All operations should be average O(1).
- Handle corner cases such as calling **`getRandom()`** on an empty set (define and justify behavior).
- Discuss how you would make this data structure thread-safe. Compare using a single lock for the whole set vs finer-grained locking (e.g., element-level locks), and explain trade-offs.

Design a data structure called FancySet that stores unique integers and supports these operations in average O(1) time: add(x), delete(x), and getRandom(). For this coding problem, implement a function **`solution(operations, values, seed)`** that simulates a newly created empty FancySet and returns the result of every operation. Use the standard dynamic-array + hash-map behavior:

- Store current elements in an array-like list.
- Store each element's current index in a hash map.

QUESTION

- When deleting an element that is not at the end of the list, move the last element into the removed slot, then pop the last slot. To make test cases deterministic, **`getRandom()`** does not use the language's built-in RNG. Instead, keep an internal 32-bit state initialized to **`seed`**. When **`getRandom()`** is called on a non-empty set, update the state with: **`state = (1664525 * state + 1013904223) mod 2^32`** and return the element at index **`state % current_size`** from the current internal list. If **`getRandom()`** is called on an empty set, return **`None`** and do not advance the state. In a real interview, picking a random index from the array gives uniform random access; the fixed generator above is only for reproducible grading. Follow-up discussion (not graded by the code runner): explain how you would make FancySet thread-safe, and compare using one lock for the whole structure versus finer-grained locking.

Constraints

1 <= len(operations) <= 2 * 10^5len(values) == len(operations)Each operation is one of 'add', 'delete', or 'getRandom'For 'add' and 'delete', values[i] contains exactly one integer; for 'getRandom', values[i] is []-10^9 <= x <= 10^9All operations should run in O(1) average time

EXAMPLES

```
Input: (['add','add','getRandom','delete','getRandom','add','getRandom'], [[1],[2],[],[1],[],[2],[]], 7)
Expected Output: [True, True, 1, True, 2, False, 2]
```

Explanation: After inserting 1 and 2, the first deterministic random choice picks index 0, so it returns 1. Deleting 1 leaves only 2. Adding 2 again fails because it is already present.

```
Input: (['getRandom','add','add','delete','delete','getRandom'], [[],[5],[5],[7],[5],[]], 1)
Expected Output: [None, True, False, False, True, None]
```

Explanation: Calling getRandom on an empty set returns None. Adding 5 twice inserts it only once. Deleting 7 fails because it is not present. After deleting 5, the set becomes empty again.

Hints

You need both fast membership/index lookup and fast access by numeric position. What two data structures combine to give that?Deleting from the middle of an array is expensive unless you replace the removed element with the last element and update that last element's stored index.

FULL CODE | main question first, then every follow-up, tests at the end

```python
# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ========================================================

import random
import threading


# ========================================================
# MAIN QUESTION
# ========================================================
# ---- main question: main function ----
# Idea: keep the values in a LIST (so a random one can be picked by a
# random index) and a dictionary from value to its index in the list (so
# add and delete can find a value at once).
#   add: append to the list and record the index.
#   delete: look up the index, move the LAST value into that hole (and
#   update its index), remove the last list slot and the dictionary entry.
# Every operation is O(1) on average.
class FancySet:
    def __init__(self, seed=None):
        self.items = []
        self.position = {}                          # value -> its index in items
```

```
        self.rng = random.Random(seed)

    def add(self, x):
        if x in self.position:
            return False
        self.position[x] = len(self.items)
        self.items.append(x)
        return True

    def delete(self, x):
        if x not in self.position:
            return False
        i = self.position[x]
        last = self.items[-1]
        self.items[i] = last                        # the last value fills the hole
        self.position[last] = i
        self.items.pop()
        del self.position[x]                        # (when x was the last one this removes it again)
        return True
```

```
    # follow-up 2: on an empty set there is nothing to return, so raise an
    # IndexError (like random.choice does) instead of returning a value
    # that could be mistaken for an element
    def getRandom(self):
        if len(self.items) == 0:
            raise IndexError("empty set")
        return self.items[self.rng.randrange(len(self.items))]


# ---- main question: tests ----
def test_main_question():
    s = FancySet(seed=1)
    assert s.add(1) and s.add(2) and not s.add(1)
    assert s.delete(1) and not s.delete(1)
    assert s.getRandom() == 2
    s.delete(2)
    try:
        s.getRandom()
        raise AssertionError("an empty set should raise")
    except IndexError:
        pass
    big = FancySet(seed=5)
    for v in range(4):
        big.add(v)
    picks = []
    for i in range(4000):
        picks.append(big.getRandom())
    for v in range(4):
        assert 850 < picks.count(v) < 1150           # every value is picked about equally often
    big.delete(0)
    big.delete(2)
    assert sorted(big.items) == [1, 3]
    for v in big.position:                           # deleting from the middle keeps the map consistent
        assert big.items[big.position[v]] == v


# =======================================================
# FOLLOW-UP 1: Make it thread-safe: one lock or finer-grained locking?
# =======================================================
# ---- follow-up 1: function ----
# Answer: one lock around every operation is the simplest correct choice:
# delete changes the list, the dictionary AND a second element's index
# together, so they must change as one unit. Locks per element would not
# protect the shared list and indexes (and risk deadlocks); a finer
```

```
# split only pays off if operations rarely touch the same data.
class SafeSet:
    def __init__(self, seed=None):
        self.inner = FancySet(seed)
        self.lock = threading.Lock()

    def add(self, x):
        with self.lock:
            return self.inner.add(x)

    def delete(self, x):
        with self.lock:
            return self.inner.delete(x)

    def getRandom(self):
        with self.lock:
            return self.inner.getRandom()


# ---- follow-up 1: tests ----
def test_follow_up_1():
    safe = SafeSet()
    assert safe.add(7) and safe.getRandom() == 7
    workers = []

    def add_many(start):
        for v in range(start, start + 500):
            safe.add(v)

    for start in (0, 250, 500):
        worker = threading.Thread(target=add_many, args=(start,))
        workers.append(worker)
        worker.start()
    for worker in workers:
        worker.join()
    assert len(safe.inner.items) == 1000 and len(safe.inner.position) == 1000


# ============================================================
# FOLLOW-UP 2: What should getRandom do on an empty set?
# ============================================================
# Answer: raise IndexError (see FancySet.getRandom). Returning None would
# be confused with a stored value, and an exception makes the caller
# decide. Whatever you choose, say it out loud and test it.
```

```
# ============================================================
# FOLLOW-UP 3: Duplicates allowed (a multiset, LeetCode 381)
# ============================================================
# ---- follow-up 3: function ----
# Answer: map each value to the SET of indexes where it sits in the list.
# Deleting one copy takes one index, moves the last element into that
# hole and fixes both index sets. getRandom stays a uniform pick over
# list positions, so a value is chosen in proportion to its count.
class RandomizedCollection:
    def __init__(self):
        self.items = []
        self.where = {}                              # value -> the set of indexes in items

    def insert(self, x):
        is_new = x not in self.where or len(self.where[x]) == 0
        if x not in self.where:
            self.where[x] = set()
        self.where[x].add(len(self.items))
        self.items.append(x)
        return is_new
```

```
    def remove(self, x):
        if x not in self.where or len(self.where[x]) == 0:
            return False
        i = self.where[x].pop()
        end = len(self.items) - 1
        if i != end:
            last = self.items[end]
            self.items[i] = last                    # the last item fills the hole
            self.where[last].discard(end)
            self.where[last].add(i)
        self.items.pop()
        return True

    def getRandom(self):
        return random.choice(self.items)


# ---- follow-up 3: tests ----
def test_follow_up_3():
    rc = RandomizedCollection()
    assert rc.insert(5)
    assert not rc.insert(5)
    assert rc.insert(6)
```

```
    assert rc.remove(5) and rc.remove(5)
    assert not rc.remove(5)
    assert rc.items == [6]
    assert rc.getRandom() == 6


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

257 Unique Random Number Generator — MEDIUM

Template fit: the lazy version for huge n: a dict 'moved' stores only the swapped slots, so memory stays O(calls)

QUESTION

Design a class UniqueRandom initialized with an integer n . Each call to generate() returns a number drawn uniformly at random from [0, n-1] that has not been returned before, so across successive calls no value ever repeats. After all n distinct values have been produced, further calls return None . Every unused value must remain equally likely on each draw (no bias), and generate() must run in O(1) time per call.

QUESTION

Function signature

import random
class UniqueRandom:
    def __init__(self, n: int, seed: int | None = None) -> None:
        ...
    def generate(self) -> int | None:
        """Return a not-yet-returned number in [0, n-1], or None once all n are
exhausted."""
        ...

Follow-up (as reported): Use a partial Fisher-Yates shuffle: keep a virtual array 0..n-1 , and on the i -th call swap a random unused slot into position i using a sparse dict of moved entries, giving O(1) time and O(number-of- draws) space.

EXAMPLES

Example 1	
Input	`g = UniqueRandom(3); [g.generate(), g.generate(), g.generate()]`
Output	`a permutation of [0, 1, 2] (e.g. [2, 0, 1])`
Why	`The three calls return all of 0,1,2 in some random order with no repeats.`

Example 2	
Input	`(continuing above) g.generate()`
Output	`None`
Why	`All 3 values already returned, so the pool is exhausted.`

Example 3	
Input	`g = UniqueRandom(1); [g.generate(), g.generate()]`
Output	`[0, None]`
Why	`The only value 0 is returned once, then the generator is exhausted.`

CONSTRAINTS

- 1 <= n <= 10^7 .
- Support an optional seed for reproducible tests.

FULL CODE | main question first, then every follow-up, tests at the end

```
# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

import random


# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: main function ----
# Idea (a lazy Fisher-Yates shuffle): imagine the numbers 0..n-1 in an
# array and shuffle it as you go. Draw number `drawn` by picking a random
# slot j from drawn..n-1; the value in slot j is the answer. Then the
# value that sat in slot `drawn` takes slot j's place. Only the slots
# that were MOVED are stored (in a dictionary); an untouched slot
# j simply holds the value j. O(1) per call, and O(draws) memory.
```

```
class UniqueRandom:
    def __init__(self, n, seed=None):
        self.n = n
        self.drawn = 0
        self.moved = {}                              # slot -> the value now sitting there
        self.rng = random.Random(seed)

    def generate(self):
        if self.drawn == self.n:
            return None                              # every number has been returned
        i = self.drawn
        j = self.rng.randrange(i, self.n)
        value = j
        if j in self.moved:
            value = self.moved[j]
        at_i = i
        if i in self.moved:
            at_i = self.moved[i]
        self.moved[j] = at_i                         # the value from slot i takes the place of the drawn one
        self.drawn += 1
        return value


# ---- main question: tests ----
def test_main_question():
```

```
    for seed in range(20):
        gen = UniqueRandom(6, seed=seed)
        values = []
        for i in range(6):
            values.append(gen.generate())
        assert sorted(values) == [0, 1, 2, 3, 4, 5]          # no repeats, nothing missing
        assert gen.generate() is None


# ========================================================
# FOLLOW-UP 1: Add a reset() so the same generator can run again
# ========================================================
# ---- follow-up 1: function ----
# Answer: forget the moved slots and set the draw counter back to zero.
# All n values are available again (the dictionary describes only the
# moves, so clearing it restores 0..n-1 in order).
class ResettableUniqueRandom(UniqueRandom):
    def reset(self):
        self.moved.clear()
        self.drawn = 0


# ---- follow-up 1: tests ----
def test_follow_up_1():
    gen = ResettableUniqueRandom(4, seed=2)
    first = []
    for i in range(4):
        first.append(gen.generate())
    gen.reset()
    second = []
    for i in range(4):
        second.append(gen.generate())
    assert sorted(first) == [0, 1, 2, 3] and sorted(second) == [0, 1, 2, 3]


# ========================================================
# RUN EVERYTHING
# ========================================================
def run_all_tests():
    test_main_question()
```

```
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
```

```
    run_all_tests()
```

258 Shuffle array using random integer API

HARD

Template fit: Fisher-Yates: for i from the end, j = rand_int(0, i), swap a[i] and a[j]

QUESTION

You have an array nums and no built-in shuffle, only rand_int(l, r), which returns a uniformly random integer in [l, r] (both ends included). Shuffle nums in place so that every one of the n! orders is equally likely.

Function signature

def shuffle(nums: list, rand_int) -> list

EXAMPLES

Example 1	

Input	`rand_int = lambda l, r: l # a fake 'random' source that always returns the lowest allowed index`
	`nums = [1, 2, 3, 4]`
	`shuffle(nums, rand_int)`

Output	`shuffle(nums, rand_int) -> [2, 3, 4, 1]`
Why	`Fisher-Yates walks i from the end and swaps nums[i] with nums[rand_int(0, i)]. With the fake source the swaps are (3,0), (2,0), (1,0), giving a predictable result; with a real source each of the 24 orders has probability 1/24.`

FULL CODE | main question first, then every follow-up, tests at the end

```
# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

import random


# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: main function ----
# Idea (Fisher-Yates): go from the LAST position down to position 1. For
# each position i pick j uniformly from 0..i (both included) and swap
# nums[i] with nums[j]. Every one of the n! orders is equally likely,
# because each step picks uniformly among i + 1 choices.
# rand_int(low, high) gives a random whole number from low to high.
def shuffle(nums, rand_int):
    for i in range(len(nums) - 1, 0, -1):
        j = rand_int(0, i)
        nums[i], nums[j] = nums[j], nums[i]
    return nums


# ---- main question: tests ----
def always_low(low, high):
```

```
    return low


def always_high(low, high):
    return high


def real_rand_int(low, high):
    return random.randint(low, high)


def test_main_question():
    assert shuffle([1, 2, 3, 4], always_low) == [2, 3, 4, 1]       # j is always 0
    assert shuffle([1, 2, 3, 4], always_high) == [1, 2, 3, 4]      # j is always i: nothing moves
    assert shuffle([], real_rand_int) == []
    random.seed(37)
    seen = set()
```

```
    for round_number in range(600):
        result = shuffle([1, 2, 3], real_rand_int)
        assert sorted(result) == [1, 2, 3]                          # always a rearrangement
        seen.add(tuple(result))
    assert len(seen) == 6                                           # all 3! orders occur


# =========================================================
# FOLLOW-UP 1: Return only k random elements
# =========================================================
# ---- follow-up 1: function ----
# Answer: run only k steps of Fisher-Yates from the FRONT: swap position i
# with a random position from i to the end, and return the first k.
# O(k) after copying, and still uniform.
def sample_k(nums, k, rand_int):
    items = list(nums)
    for i in range(min(k, len(items))):
        j = rand_int(i, len(items) - 1)
        items[i], items[j] = items[j], items[i]
    return items[:k]


# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert sample_k([1, 2, 3, 4], 2, always_low) == [1, 2]
    assert sample_k([1, 2, 3, 4], 2, always_high) == [4, 1]
    assert sample_k([1, 2, 3], 5, always_low) == [1, 2, 3]
    random.seed(38)
    for round_number in range(200):
        picked = sample_k([1, 2, 3, 4, 5], 3, real_rand_int)
        assert len(set(picked)) == 3                                # no element twice


# =========================================================
# FOLLOW-UP 2: Only a coin-flip API exists, no rand_int
# =========================================================
# ---- follow-up 2: function ----
# Answer: build a uniform number in [0, m) by REJECTION SAMPLING: draw
# enough random bits to cover m values, and try again if the number is
# m or more. Every value stays equally likely, unlike taking a
# remainder (which would favour small numbers).
def rand_below(m, flip):
    bits = 1
    while (1 << bits) < m:
        bits += 1                                   # the fewest bits that can count up to m
```

```
    while True:
        x = 0
```

```
        for i in range(bits):
            x = x * 2 + flip()
        if x < m:
            return x


# ---- follow-up 2: tests ----
def coin():
    return random.randint(0, 1)


def test_follow_up_2():
    random.seed(39)
    counts = [0] * 5
    for round_number in range(5000):
        counts[rand_below(5, coin)] += 1
    for count in counts:
        assert 800 < count < 1200                   # about 1000 each: no value is favoured
    assert rand_below(1, coin) == 0


# =============================================================
# FOLLOW-UP 3: Why is the biased version biased?
# =============================================================
# ---- follow-up 3: function ----
# Answer: the common WRONG version swaps every position with a random
# position from the WHOLE array. That makes n choices of n options,
# n^n equally likely paths, and n! does not divide n^n for n >= 3, so
# some orders must appear more often than others.
def biased_shuffle(nums, rand_int):
    for i in range(len(nums)):
        j = rand_int(0, len(nums) - 1)
        nums[i], nums[j] = nums[j], nums[i]
    return nums


# ---- follow-up 3: tests ----
def test_follow_up_3():
    counts = {}
    for a in range(3):                              # try all 27 possible sequences of random choices
        for b in range(3):
            for c in range(3):
                choices = [a, b, c]
                position = [0]

                def scripted(low, high):
                    value = choices[position[0]]
                    position[0] += 1
                    return value

                result = tuple(biased_shuffle([1, 2, 3], scripted))
                counts[result] = counts.get(result, 0) + 1
    assert sum(counts.values()) == 27
    assert len(set(counts.values())) > 1            # the 6 orders do NOT get equal shares of 27


# =============================================================
# RUN EVERYTHING
# =============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    test_follow_up_3()
    print("all tests passed")
```

```
if __name__ == "__main__":
    run_all_tests()
```

GROUP 38 OF 43 | 5 QUESTIONS, ONE CODE TEMPLATE | 3 M 2 H

Build and serialize: write a structure out, read it back with the same walk

259 Construct a binary tree given its preorder and inorder traversals. MEDIUM

Template fit: the first preorder value is the root; its position in a value-to-index map of the inorder list splits left from right

QUESTION

Given two integer arrays, preorder and inorder, which represent the pre-order and in-order traversal of a binary tree, construct and return the binary tree. Pre-order traversal visits nodes in this order: root -> left subtree -> right subtree In-order traversal visits nodes in this order: left subtree -> root -> right subtree You need to construct the binary tree and return its root node. Examples preorder = [3, 9, 20, 15, 7], inorder = [9, 3, 15, 20, 7] output: [3, 9, 20, null, null, 15, 7] Explanation: Given the two arrays, the binary tree constructed is: 3 / \ 9 20 / \ 15 7 preorder = [1, 2], inorder = [2, 1] output: [1, 2]
Explanation: Given the two arrays, the binary tree constructed is: 1 / 2 Constraints: The number of nodes in the tree is in the range [1, 3000]. preorder and inorder consist of unique integers. The elements of inorder will always be the same as the elements of preorder but in a different order.

EXAMPLES

Example 1

Input	`preorder = [3, 9, 20, 15, 7]` `inorder = [9, 3, 15, 20, 7]`
Output	`values(build_pre_in([3, 9, 20, 15, 7], [9, 3, 15, 20, 7])) -> [3, 9, 20, null, null, 15, 7]`
Why	`The first preorder value 3 is the root; in the inorder list 9 is left of it and 15, 20, 7 are right. The tree is shown level by level.`

FULL CODE | main question first, then every follow-up, tests at the end

```python
# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right


def preorder_of(root):
    # the values in pre-order (root, left, right), with a stack
    values = []
    stack = []
    if root:
        stack.append(root)
    while stack:
        node = stack.pop()
        values.append(node.val)
        if node.right:
            stack.append(node.right)
```

```
        if node.left:
            stack.append(node.left)
    return values


def inorder_of(root):
    # the values in in-order (left, root, right), with a stack
    values = []
    stack = []
    node = root
    while stack or node:
        while node:
            stack.append(node)
            node = node.left
        node = stack.pop()
        values.append(node.val)
```

```
        node = node.right
    return values


def positions(values):
    # value -> its index (the values are all different)
    where = {}
    for i in range(len(values)):
        where[values[i]] = i
    return where


# ---- main question: main function ----
# Idea: preorder is "root, left part, right part", so its next unused
# value is always the next ROOT. That root's position in the inorder
# list splits the inorder list into the left part (before it) and the
# right part (after it). Build the left subtree first, then the right.
def build_pre_in(preorder, inorder):
    where = positions(inorder)
    next_root = 0

    def build(low, high):
        # build the tree for inorder[low:high]
        nonlocal next_root
        if low >= high:
            return None
        root = TreeNode(preorder[next_root])
        next_root += 1
        middle = where[root.val]
        root.left = build(low, middle)
        root.right = build(middle + 1, high)
        return root

    return build(0, len(inorder))


# ---- main question: tests ----
PRE = [3, 9, 20, 15, 7]
INO = [9, 3, 15, 20, 7]
POST = [9, 15, 7, 20, 3]


def test_main_question():
    tree = build_pre_in(PRE, INO)
    assert preorder_of(tree) == PRE
```

```
    assert inorder_of(tree) == INO
    assert build_pre_in([], []) is None
```

```python
# ========================================================
# FOLLOW-UP 1: You are given postorder and inorder instead
# ========================================================
# ---- follow-up 1: function ----
# Answer: the root is the LAST postorder value. Postorder is "left, right,
# root", so reading it backwards gives root, right, left: build the
# RIGHT subtree before the left one.
def build_post_in(inorder, postorder):
    where = positions(inorder)
    next_root = len(postorder) - 1

    def build(low, high):
        nonlocal next_root
        if low >= high:
            return None
        root = TreeNode(postorder[next_root])
        next_root -= 1
        middle = where[root.val]
        root.right = build(middle + 1, high)
        root.left = build(low, middle)
        return root

    return build(0, len(inorder))


# ---- follow-up 1: tests ----
def test_follow_up_1():
    tree = build_post_in(INO, POST)
    assert preorder_of(tree) == PRE
    assert inorder_of(tree) == INO


# ========================================================
# FOLLOW-UP 2: Values can repeat
# ========================================================
# Answer: then the traversals alone do not determine the tree (the
# value-to-position table is ambiguous). Ask for extra information, such
# as the preorder together with the sizes of the left subtrees.


# ========================================================
```

```python
# FOLLOW-UP 3: The tree is a 100,000-node chain
# ========================================================
# ---- follow-up 3: function ----
# Answer: the recursion is as deep as the tree, so use your own stack.
# Go through the preorder values; a new node is the LEFT child of the top
# of the stack until the top's value equals the next inorder value (its
# left side is finished). Then pop back to the node this one is the right
# child of.
def build_pre_in_iterative(preorder, inorder):
    if not preorder:
        return None
    root = TreeNode(preorder[0])
    stack = [root]
    in_index = 0
    for i in range(1, len(preorder)):
        node = TreeNode(preorder[i])
        if stack[-1].val != inorder[in_index]:
            stack[-1].left = node                  # the left side of the top node is not finished
        else:
            parent = None
            while stack and stack[-1].val == inorder[in_index]:
                parent = stack.pop()
                in_index += 1
```

```
            parent.right = node
        stack.append(node)
    return root


# ---- follow-up 3: tests ----
def test_follow_up_3():
    tree = build_pre_in_iterative(PRE, INO)
    assert preorder_of(tree) == PRE
    assert inorder_of(tree) == INO
    assert build_pre_in_iterative([], []) is None
    chain = list(range(50000))                    # a very deep chain: no recursion
    reverse = list(range(49999, -1, -1))
    deep = build_pre_in_iterative(chain, reverse)
    assert preorder_of(deep) == chain


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
```

```
    test_follow_up_1()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

260 Write functions to serialize and deserialize a list of strings. MEDIUM

Template fit: write len + '#' + text for each string; read the length, then take exactly that many characters

QUESTION

Serialize and Deserialize a List of Strings. You are tasked with writing two functions: serialize and deserialize. The serialize function will take a list of strings as input and return a single string. The deserialize function will take this single string and reconstruct the original list of strings. Details The serialize function should concatenate all strings in the list into a single string, with each original string's length encoded as a single character prefix. The deserialize function should be able to take the single string and accurately split it back into the original list of strings, using the encoded length characters as guides. For example, consider a list ["hello", "world"]. The serialization of this list might be "\x05hello\x05world", where \x05 indicates that the following string has a length of 5 characters. Constraints The maximum length of any string is 65535 (0xFFFF). If a string's length exceeds the maximum, the serialize function should raise an exception. The deserialize function should accurately handle any edge cases, such as empty strings or single-character strings. Assume that all strings consist of ASCII characters only. Examples lst = ['hello', 'world'] serialized = serialize(lst)

serialized output

: "\x05hello\x05world" deserialized = deserialize(serialized)

deserialized output should be

: ['hello', 'world'] lst = [''] serialized = serialize(lst)

serialized output

: "\x00" deserialized = deserialize(serialized)

deserialized output should be

EXAMPLES

QUESTION

Example 1	
Input	items = ['hello', 'wor#ld', ''] serialize(items) deserialize(serialize(items)) == items
Output	serialize(items) -> '5#hello6#wor#ld0#' deserialize(serialize(items)) == items -> true
Why	Each string is written as its length, a #, and its text, so a # inside a string cannot confuse the reader.

FULL CODE | main question first, then every follow-up, tests at the end

```
# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

import json


# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: main functions ----
# Idea: write the LENGTH of every string before it: "<length>#<text>".
# The reader knows exactly how many characters to take, so the text may
# contain any character, even "#" and digits.
def serialize(items):
    text = ""
    for s in items:
        text += str(len(s)) + "#" + s
    return text


def deserialize(text):
    items = []
    i = 0
    while i < len(text):
        sep = text.index("#", i)                # the digits of the length end at the next "#"
        size = int(text[i:sep])
        start = sep + 1
        items.append(text[start:start + size])
        i = start + size
    return items


# ---- main question: tests ----
TRICKY = ["", "a", "12#x", "hello, world", "\\", ",,", "\u00e9\u4e16\u754c", "3#abc", "\n"]


def test_main_question():
    assert serialize(["ab", "c"]) == "2#ab1#c"
    for data in ([], [""], ["a"], TRICKY):
        assert deserialize(serialize(data)) == data
```

```
# ============================================================
# FOLLOW-UP 1: Use escaping instead of a length prefix
# ============================================================
# ---- follow-up 1: function ----
# Answer: write a backslash before every comma and every backslash
```

```
# inside a string, and end each item with a plain comma. The reader
# remembers whether the previous character was a backslash.
def serialize_escaped(items):
    text = ""
    for s in items:
        for ch in s:
            if ch == "\\" or ch == ",":
                text += "\\"
            text += ch
        text += ","
    return text


def deserialize_escaped(text):
    items = []
    current = ""
    escaped = False
    for ch in text:
        if escaped:
            current += ch                       # the character after a backslash is taken literally
            escaped = False
        elif ch == "\\":
            escaped = True
        elif ch == ",":
            items.append(current)
            current = ""
        else:
            current += ch
    if current != "" or escaped:
        raise ValueError("truncated input")
    return items


# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert serialize_escaped(["a,b"]) == "a\\,b,"
    assert serialize_escaped([""]) == ","
    assert serialize_escaped([]) == ""
    for data in ([], [""], ["a"], TRICKY):
```

```
        assert deserialize_escaped(serialize_escaped(data)) == data
    try:
        deserialize_escaped("abc")
        raise AssertionError("should fail")
    except ValueError:
        pass


# ============================================================
# FOLLOW-UP 2: The strings are long or binary
# ============================================================
# ---- follow-up 2: helper functions ----
def length_to_bytes(n):
    # a fixed 4-byte length, most significant byte first (big-endian)
    return bytes([n // 16777216 % 256, n // 65536 % 256, n // 256 % 256, n % 256])


def length_from_bytes(data, i):
    return data[i] * 16777216 + data[i + 1] * 65536 + data[i + 2] * 256 + data[i + 3]


# ---- follow-up 2: function ----
# Answer: work with bytes and a fixed-width length in front of every
# item. There are no digits to scan, and any byte value (even 0) is allowed.
def serialize_bytes(items):
    out = b""
```

```
    for s in items:
        raw = s
        if isinstance(s, str):
            raw = s.encode("utf-8")
        out += length_to_bytes(len(raw)) + raw
    return out


def deserialize_bytes(data):
    items = []
    i = 0
    while i < len(data):
        size = length_from_bytes(data, i)
        items.append(data[i + 4:i + 4 + size])
        i += 4 + size
    return items


# ---- follow-up 2: tests ----
```

```
def test_follow_up_2():
    assert serialize_bytes(["a"]) == b"\x00\x00\x00\x01a"
    for data in ([], [""], ["a"], TRICKY):
        raw = deserialize_bytes(serialize_bytes(data))
        decoded = []
        for item in raw:
            decoded.append(item.decode("utf-8"))
        assert decoded == data
    blob = [bytes([0, 255, 10])]                # binary data that is not text works too
    assert deserialize_bytes(serialize_bytes(blob)) == blob


# ============================================================
# FOLLOW-UP 3: Just use JSON?
# ============================================================
# ---- follow-up 3: function ----
# Answer: yes, if you are free to choose the format: json handles
# quoting and Unicode correctly. (Say what you gave up: you did not show
# that you can design a format yourself.)
def serialize_json(items):
    return json.dumps(items)


def deserialize_json(text):
    return json.loads(text)


# ---- follow-up 3: tests ----
def test_follow_up_3():
    for data in ([], [""], ["a"], TRICKY):
        assert deserialize_json(serialize_json(data)) == data


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
```

```
    run_all_tests()
```

261 Print a Directory Tree

MEDIUM

Template fit: build nested dicts from the path parts, then print them with a recursive walk that indents by depth

QUESTION

You are given a list of file/directory paths , each a '/' -separated string (for example "dir/subdir/file.txt"). Reproduce what a Unix directory listing tool prints: build the directory hierarchy implied by the paths and return it as a single newline-separated string. Rules: Each path component is a node. A component that appears under the same parent in several paths must be printed only once. Indent each node with 2 * depth spaces, where the top-level components have depth 0 . Children are printed in the order they are first encountered across the input list (preserve first-seen order; do not sort). The output must not have a trailing newline.

Function signature

def print_directory_tree(paths: list[str]) -> str:

    ...

Follow-up (as reported): Add box-drawing connectors (, ,) like the real tree command. Hint: track, per level, whether the current node is the last child of its parent.

EXAMPLES

Example 1	
Input	`paths = ["a/b/c", "a/b/d", "a/e"]`
Output	`"a\n b\n c\n d\n e"`
Why	`Rendered: a b c d e`

Example 2	
Input	`paths = ["src/main.py", "src/util.py", "README.md"]`
Output	`"src\n main.py\n util.py\nREADME.md"`
Why	`"src" and its two files come first (first-seen order), then the top-level "README.md".`

Example 3	
Input	`paths = []`
Output	`""`

CONSTRAINTS

- 0 <= len(paths) <= 10^4
- Each path is non-empty, has no leading/trailing '/' , and no empty components.
- If paths is empty, return the empty string "" .

FULL CODE | main question first, then every follow-up, tests at the end

```python
# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
def build_dirs(paths):
    # nested dictionaries: every folder or file name maps to its own contents
```

```
    root = {}
    for path in paths:
        node = root
        for part in path.split("/"):
            if part not in node:
                node[part] = {}
            node = node[part]
    return root


# ---- main question: main function ----
# Idea: put every path into a dictionary of dictionaries (a tree), then
# print it depth first with two spaces per level.
def print_directory_tree(paths):
    root = build_dirs(paths)
    lines = []

    def walk(node, depth):
        for name in node:
            lines.append("  " * depth + name)
            walk(node[name], depth + 1)

    walk(root, 0)
    return "\n".join(lines)


# ---- main question: tests ----
def test_main_question():
    assert print_directory_tree(["a/b/c", "a/b/d", "a/e"]) == "a\n  b\n    c\n    d\n  e"
    assert print_directory_tree(["src/main.py", "src/util.py", "README.md"]) == "src\n  main.py\n  util.py\nR
EADME.md"
    assert print_directory_tree([]) == ""
```

```
# =========================================================
# FOLLOW-UP 1: Print it with box-drawing connectors like the real tree command
# =========================================================
# ---- follow-up 1: function ----
# Answer: carry a prefix down the recursion. Every child gets a
# connector: the LAST child gets the corner and its contents continue
# with blank space; the others get a tee and their contents continue
# with a vertical bar.
def tree_with_connectors(paths):
    root = build_dirs(paths)
    lines = []
    tee = "\u251c\u2500\u2500 "
    corner = "\u2514\u2500\u2500 "
    bar = "\u2502   "

    def walk(node, prefix):
        names = list(node)
        for i in range(len(names)):
            is_last = i == len(names) - 1
            if is_last:
                lines.append(prefix + corner + names[i])
                walk(node[names[i]], prefix + "    ")
            else:
                lines.append(prefix + tee + names[i])
                walk(node[names[i]], prefix + bar)

    walk(root, "")
    return "\n".join(lines)


# ---- follow-up 1: tests ----
def test_follow_up_1():
```

```
    tee = "\u251c\u2500\u2500 "
    corner = "\u2514\u2500\u2500 "
    assert tree_with_connectors(["a/b", "a/c"]) == corner + "a\n    " + tee + "b\n    " + corner + "c"


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")
```

```
if __name__ == "__main__":
    run_all_tests()
```

262 Implement a decode and encode function of a general N-ary tree. HARD

Template fit: write value and child count for each node (preorder); read the count to know how many children to build

QUESTION

A general tree has a value in every node and any number of children. Write encode(root), which turns the tree into a string, and decode(text), which rebuilds the identical tree from that string.

Function signature

def encode(root: Node) -> str
def decode(text: str) -> Node

EXAMPLES

Example 1

Input	`root = Node(1, [Node(3, [Node(5), Node(6)]), Node(2), Node(4)])`
	`text = encode(root)`
	`text`
	`encode(decode(text)) == text`

Output	`text -> '[1, 3, 3, 2, 5, 0, 6, 0, 2, 0, 4, 0]'` `encode(decode(text)) == text -> true`
Why	`Each node is written as its value followed by its number of children, in pre-order: 1 has 3 children, 3 has 2, and so on. Decoding reads exactly that many children for each node, so encoding the decoded tree gives the same text.`

FULL CODE | main question first, then every follow-up, tests at the end

```
# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

from collections import deque
import json


# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
class Node:
```

```
    def __init__(self, val, children=None):
        self.val = val
        self.children = []
        if children:
            self.children = children


def preorder_pairs(root):
    # (value, number of children) for every node in pre-order. A stack
    # instead of recursion, so a very deep tree cannot overflow the call stack.
    pairs = []
    stack = []
    if root:
        stack.append(root)
    while stack:
        node = stack.pop()
        pairs.append((node.val, len(node.children)))
        for i in range(len(node.children) - 1, -1, -1):     # push the LAST child first so the first is poppe
d first
            stack.append(node.children[i])
    return pairs


def build_preorder(pairs):
    # the reverse: rebuild the tree from the same (value, child count) pairs
    if not pairs:
        return None
    root = Node(pairs[0][0])
    stack = [[root, pairs[0][1]]]               # [a node, how many children it still has to receive]
    position = 1
```

```
    while stack:
        top = stack[-1]
        if top[1] == 0:
            stack.pop()                         # this node is complete
            continue
        top[1] -= 1
        value, count = pairs[position]
        position += 1
        child = Node(value)
        top[0].children.append(child)
        stack.append([child, count])
    return root


def same(a, b):
    # do two trees have the same shape and values?
    return preorder_pairs(a) == preorder_pairs(b)


# ---- main question: main functions ----
# Idea: write every node as "value child_count" in PRE-ORDER (a node
# first, then its children in order). The counts say where each node's
# children end, so no brackets are needed. Values here are words without
# spaces (follow-up 1 handles any string).
def encode(root):
    parts = []
    for value, count in preorder_pairs(root):
        parts.append(str(value))
        parts.append(str(count))
    return " ".join(parts)


def decode(text):
    if text == "":
        return None
```

```
    tokens = text.split(" ")
    pairs = []
    for i in range(0, len(tokens), 2):
        pairs.append((tokens[i], int(tokens[i + 1])))
    return build_preorder(pairs)


# ---- main question: tests ----
def example_tree():
    return Node("a", [Node("b", [Node("c")]), Node("d"), Node("e", [Node("f"), Node("g")])])
```

```
def test_main_question():
    tree = example_tree()
    assert encode(tree) == "a 3 b 1 c 0 d 0 e 2 f 0 g 0"
    assert same(decode(encode(tree)), tree)
    assert encode(None) == "" and decode("") is None


# =============================================================
# FOLLOW-UP 1: The values are arbitrary strings that may contain spaces, commas or brackets
# =============================================================
# ---- follow-up 1: function ----
# Answer: do not invent delimiters a value could contain. Put the values
# in a real container: a JSON list of alternating value and child count
# (JSON does the quoting), or write the length before every value
# ("5#hello"). The counts, which carry the structure, stay the same.
def encode_any(root):
    flat = []
    for value, count in preorder_pairs(root):
        flat.append(value)
        flat.append(count)
    return json.dumps(flat)


def decode_any(text):
    flat = json.loads(text)
    pairs = []
    for i in range(0, len(flat), 2):
        pairs.append((flat[i], flat[i + 1]))
    return build_preorder(pairs)


# ---- follow-up 1: tests ----
def test_follow_up_1():
    tree = Node("a, b", [Node("[x]", [Node("")]), Node("\u00e9 \u4e16\u754c"), Node('q"uote', [Node("1"), Nod
e("2")])])
    assert same(decode_any(encode_any(tree)), tree)
    assert encode_any(None) == "[]" and decode_any("[]") is None
    chain = Node("0")                                       # a chain of 5000 nodes: nothing is recursive
    last = chain
    for i in range(1, 5000):
        last.children.append(Node(str(i)))
        last = last.children[0]
    assert same(decode_any(encode_any(chain)), chain)
```

```
# =============================================================
# FOLLOW-UP 2: The tree is huge and you want a compact binary encoding
# =============================================================
# ---- follow-up 2: helper functions ----
def put_varint(n, out):
    # a whole number in 7-bit pieces: the top bit of a byte says "another byte follows"
    while n >= 128:
        out.append(n % 128 + 128)
```

```
        n = n // 128
    out.append(n)


def get_varint(data, position):
    # read one varint starting at `position`; returns (number, position after it)
    n = 0
    shift = 0
    while data[position] >= 128:
        n += (data[position] - 128) << shift
        shift += 7
        position += 1
    n += data[position] << shift
    return n, position + 1


# ---- follow-up 2: function ----
# Answer: pack the lengths and counts as variable-length integers
# (small numbers take one byte). Keep the same pre-order "value, count"
# layout, so decoding is still one pass.
def encode_binary(root):
    out = bytearray([1])                          # a version byte first, so the format can change later
    for value, count in preorder_pairs(root):
        raw = value.encode("utf-8")
        put_varint(len(raw), out)
        out += raw
        put_varint(count, out)
    return bytes(out)


def decode_binary(data):
    if data[0] != 1:
        raise ValueError("unknown version")
    pairs = []
    position = 1
    while position < len(data):
```

```
        size, position = get_varint(data, position)
        value = data[position:position + size].decode("utf-8")
        count, position = get_varint(data, position + size)
        pairs.append((value, count))
    return build_preorder(pairs)


# ---- follow-up 2: tests ----
def test_follow_up_2():
    tree = example_tree()
    assert same(decode_binary(encode_binary(tree)), tree)
    assert decode_binary(encode_binary(None)) is None
    wide = Node("w", [Node(str(i)) for i in range(300)])       # 300 children need a two byte count
    assert same(decode_binary(encode_binary(wide)), wide)
    assert len(preorder_pairs(wide)) == 301


# ======================================================
# FOLLOW-UP 3: Encode in level order instead
# ======================================================
# ---- follow-up 3: function ----
# Answer: write each node's value and child count in level (BFS) order
# and decode with a queue: take a parent out, read its count, attach
# that many new nodes and put them in the queue. Handy when the reader
# wants to stream the tree level by level.
def level_pairs(root):
    pairs = []
    queue = deque()
    if root:
```

```
        queue.append(root)
    while queue:
        node = queue.popleft()
        pairs.append((node.val, len(node.children)))
        for child in node.children:
            queue.append(child)
    return pairs


def build_levels(pairs):
    if not pairs:
        return None
    root = Node(pairs[0][0])
    queue = deque([(root, pairs[0][1])])
    position = 1
    while queue:
```

```
        parent, count = queue.popleft()
        for i in range(count):
            value, kids = pairs[position]
            position += 1
            child = Node(value)
            parent.children.append(child)
            queue.append((child, kids))
    return root


# ---- follow-up 3: tests ----
def test_follow_up_3():
    tree = example_tree()
    assert level_pairs(tree)[:4] == [("a", 3), ("b", 1), ("d", 0), ("e", 2)]
    assert same(build_levels(level_pairs(tree)), tree)
    assert build_levels(level_pairs(None)) is None


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

263 Serialize and Deserialize a Graph — HARD

Template fit: BFS with a seen set; write one line per node: value, then the values of its neighbours; rebuild by creating all nodes first, then linking

QUESTION

Design a codec that can serialize a connected, undirected graph to a string and deserialize that string back into an identical graph. Each node has a unique integer val and a list of neighbors . The graph may contain cycles. After a round trip, the reconstructed graph must have the same node values and the same adjacency relationships as the original. You are given the reference node class below. Implement serialize (node -> string) and deserialize (string -> node). serialize(None) must round-trip to None via deserialize .

QUESTION

Function signature

```
class Node:
    def __init__(self, val: int = 0, neighbors: list["Node"] | None = None):
        self.val = val
        self.neighbors = neighbors if neighbors is not None else []
class Codec:
    def serialize(self, node: "Node | None") -> str:
        """Encode the whole graph reachable from node into a string."""
        ...
    def deserialize(self, data: str) -> "Node | None":
        """Rebuild the graph from the string produced by serialize."""
        ...
```

Follow-up (as reported): Support directed graphs (asymmetric adjacency). Hint: record each directed edge (u -> v) rather than assuming both directions during deserialization.

EXAMPLES

Example 1	
Input	`adjList = [[]] # a single node with no neighbors`
Output	`A single node with val 1 and an empty neighbor list.`

Example 2	
Input	`node = None # empty graph`
Output	`codec.deserialize(codec.serialize(None)) is None`

CONSTRAINTS

- 0 <= number of nodes <= 100
- Node values are unique.
- The graph is connected when non-empty; you receive any one node as the entry point.

FULL CODE | main question first, then every follow-up, tests at the end

```python
# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

from collections import deque
import random


# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
class Node:
    def __init__(self, val=0, neighbors=None):
        self.val = val
        self.neighbors = []
        if neighbors is not None:
            self.neighbors = neighbors


def build_graph(adj):
    # adj[i] lists the neighbours of node i + 1, e.g. [[2, 4], [1, 3], [2, 4], [1, 3]]
```

```
    nodes = []
    for i in range(len(adj)):
        nodes.append(Node(i + 1))
    for i in range(len(adj)):
        for j in adj[i]:
            nodes[i].neighbors.append(nodes[j - 1])
    if nodes:
        return nodes[0]
    return None


# ---- main question: main function ----
# Idea: one entry "value:neighbour,neighbour" per node, found with a
# BFS from the given node, separated by ";". To read it back, create
# every node FIRST and link the neighbours afterwards, so a neighbour
# always exists before it is linked (a graph can have cycles).
class Codec:
    def serialize(self, node):
        if node is None:
            return ""
```

```
        seen = {node.val}
        queue = deque([node])
        parts = []
        while queue:
            current = queue.popleft()
            names = []
            for neighbour in current.neighbors:
                names.append(str(neighbour.val))
                if neighbour.val not in seen:
                    seen.add(neighbour.val)
                    queue.append(neighbour)
            parts.append(str(current.val) + ":" + ",".join(names))
        return ";".join(parts)

    def deserialize(self, data):
        if data == "":
            return None
        rows = []
        for part in data.split(";"):
            rows.append(part.split(":"))
        nodes = {}
        for value, neighbour_text in rows:            # pass 1: create every node
            nodes[int(value)] = Node(int(value))
        for value, neighbour_text in rows:            # pass 2: link the neighbours
            if neighbour_text != "":
                for name in neighbour_text.split(","):
                    nodes[int(value)].neighbors.append(nodes[int(name)])
        return nodes[int(rows[0][0])]


# ---- main question: tests ----
def test_main_question():
    codec = Codec()
    assert codec.serialize(None) == "" and codec.deserialize("") is None
    square = build_graph([[2, 4], [1, 3], [2, 4], [1, 3]])       # a cycle of four nodes
    text = codec.serialize(square)
    assert text == "1:2,4;2:1,3;4:1,3;3:2,4"
    assert codec.serialize(codec.deserialize(text)) == text
    single = codec.deserialize(codec.serialize(build_graph([[]])))
    assert single.val == 1 and single.neighbors == []


# ========================================================
# FOLLOW-UP 1: Support directed graphs, where the adjacency is not symmetric
# ========================================================
```

```
# Answer: nothing to change: every node stores ITS OWN neighbour list, so
# a directed edge u -> v is saved as given and nothing assumes the
# reverse edge exists. (Reachability from the start node is what gets saved.)
# ---- follow-up 1: tests ----
def test_follow_up_1():
    codec = Codec()
    random.seed(25)
    for round_number in range(200):                # random graphs, edges in one direction only
        n = random.randint(1, 6)
        adj = []
        for i in range(n):
            adj.append(sorted(random.sample(range(1, n + 1), random.randint(0, min(3, n)))))
        start = build_graph(adj)
        text = codec.serialize(start)
        assert codec.serialize(codec.deserialize(text)) == text


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

GROUP 39 OF 43 | 5 QUESTIONS, ONE CODE TEMPLATE | 1 M 4 H

Games: win or lose with a cache, and guessing with feedback

264 Choose optimal guesses for green-only Wordle — MEDIUM

Template fit: bitmask per (position, letter) of the words that have it; the best guess splits the words into the most even groups

QUESTION

You are guessing a hidden word from a candidate list. After each guess you only learn which positions are green, that is, where your letter equals the answer's letter. best_guess(words) picks the candidate whose answer pattern splits the remaining words most evenly: the one with the smallest expected size of the group left (sum of squared group sizes divided by the number of words); ties go to the alphabetically smaller word.

Function signature

def green(guess: str, answer: str) -> tuple

def best_guess(words: list) -> str

EXAMPLES

Example 1	
Input	`'cat', 'car'`
Output	`(true, true, false)`
Why	`c and a match, the last letters differ.`

Example 2	
Input	`['cat', 'car', 'cap', 'dog']`
Output	`'cap'`
Why	`'cap', 'car' and 'cat' split the other words equally well and 'cap' wins the alphabetical tie-break; 'dog' shares no letter positions with the others, so it tells less.`

Example 3	
Input	`['tea']`
Output	`'tea'`
Why	`With a single candidate the answer is that candidate.`

FULL CODE | main question first, then every follow-up, tests at the end

```
# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

from fractions import Fraction


# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
def green(guess, answer):
    # green-only feedback: for every position, does the letter match exactly?
    feedback = []
    for i in range(len(guess)):
```

```
        feedback.append(guess[i] == answer[i])
    return tuple(feedback)


def partition(guess, words):
    # group the possible answers by the feedback they would give for this guess
    groups = {}
    for word in words:
        pattern = green(guess, word)
        if pattern not in groups:
            groups[pattern] = []
        groups[pattern].append(word)
    return list(groups.values())


def score(guess, words, weights=None):
    # expected number of answers left after this guess:
    # (sum of group size squared) / (number of answers). Lower is better.
    # weights (follow-up 2): word -> how likely it is the answer (default 1).
    def weight_of(word):
        if weights and word in weights:
            return weights[word]
        return 1

    total = 0
    for word in words:
        total += weight_of(word)
```

```
    spread = 0
    for group in partition(guess, words):
        group_weight = 0
        for word in group:
            group_weight += weight_of(word)
        spread += group_weight * group_weight
    return Fraction(spread, total)


# ---- main question: main function ----
# Idea: for every possible guess, split the remaining answers by the
# feedback that guess would give, and pick the guess that leaves the
# fewest answers on average. Ties go to the alphabetically first word,
# so the result is deterministic.
def best_guess(words, weights=None):
    best_word = None
    best_score = None
    for guess in words:
        s = score(guess, words, weights)
        if best_score is None or s < best_score or (s == best_score and guess < best_word):
            best_score = s
            best_word = guess
    return best_word


# a faster way: one bitmask per (position, letter); every position splits
# each group in two with two AND operations
def best_guess_bitsets(words):
    count = len(words)
    bits = []
    for pos in range(len(words[0])):
        bits.append({})
    for i in range(count):
        for pos in range(len(words[i])):
            letter = words[i][pos]
            bits[pos][letter] = bits[pos].get(letter, 0) | (1 << i)
    best_word = None
    best_score = None
```

```
    for guess in words:
        groups = [(1 << count) - 1]                # one group holding every word
        for pos in range(len(guess)):
            mask = bits[pos].get(guess[pos], 0)
            split = []
            for group in groups:
                matching = group & mask
```

```
                other = group & ~mask
                if matching:
                    split.append(matching)
                if other:
                    split.append(other)
            groups = split
        spread = 0
        for group in groups:
            size = bin(group).count("1")
            spread += size * size
        s = Fraction(spread, count)
        if best_score is None or s < best_score or (s == best_score and guess < best_word):
            best_score = s
            best_word = guess
    return best_word


# ---- main question: tests ----
WORDS = ["crane", "crate", "grape", "grade", "trade", "slate", "plate"]


def test_main_question():
    assert best_guess(WORDS) == best_guess_bitsets(WORDS)
    assert score("crane", WORDS) <= len(WORDS)               # a guess never leaves more than all the words
    assert partition("crane", ["crane"]) == [["crane"]]
    assert green("crane", "crate") == (True, True, True, False, True)


# ========================================================
# FOLLOW-UP 1: Play a whole game, using the feedback after each guess
# ========================================================
# ---- follow-up 1: function ----
# Answer: keep only the answers that would have produced exactly the
# feedback you saw, then pick the best guess among those. Each round
# removes at least the guess itself, so the game ends.
def filter_candidates(words, guess, pattern):
    kept = []
    for word in words:
        if green(guess, word) == pattern:
            kept.append(word)
    return kept


def play(words, secret):
    candidates = list(words)
```

```
    guesses = []
    while True:
        guess = best_guess(candidates)
        guesses.append(guess)
        if guess == secret:
            return guesses
        candidates = filter_candidates(candidates, guess, green(guess, secret))


# ---- follow-up 1: tests ----
def test_follow_up_1():
    for secret in WORDS:
```

```
        path = play(WORDS, secret)
        assert path[-1] == secret
        assert len(path) <= len(WORDS)


# ======================================================
# FOLLOW-UP 2: Answers are not equally likely (weights)
# ======================================================
# Answer: score() already takes a `weights` dictionary: a very likely
# answer counts for more in every group, so guesses that split it off
# become more attractive.
# ---- follow-up 2: tests ----
def test_follow_up_2():
    weights = {"slate": 100}
    assert score("slate", WORDS, weights) < score("crane", WORDS, weights)


# ======================================================
# FOLLOW-UP 3: Real Wordle also gives yellow feedback
# ======================================================
# Answer: the feedback becomes a pattern of green / yellow / grey per
# position (3^5 possible patterns). Only green() changes: count each
# letter's remaining copies so a repeated letter is yellow at most as
# many times as it really appears in the answer.


# ======================================================
# RUN EVERYTHING
# ======================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
```

```
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

265 Wordle-Style 5-Letter Minimum-Guess Strategy — HARD

Template fit: ask about the letter at a position that splits the remaining words most evenly (min of yes-count and no-count is largest)

QUESTION

A hidden word has 5 lowercase letters. The only thing you can ask is the oracle query(pos, letter), which says whether the letter is at that position (positions 0 to 4). Find the word with as few oracle calls as possible. by_frequency tries letters in order of English frequency; with_dictionary also knows a list of candidate words and always asks the question that splits the candidates most evenly. Both return (word, number of calls).

Function signature

def by_frequency(query) -> (str, int)
def with_dictionary(query, dictionary: list) -> (str, int)

EXAMPLES

QUESTION

Example 1

Input	`secret = 'crane'`
	`query = lambda pos, letter: secret[pos] == letter`
	`by_frequency(query)`
	`with_dictionary(query, ['crane', 'crate', 'slate', 'trace', 'grape'])`

Output	`by_frequency(query) -> ('crane', 31)` `with_dictionary(query, ['crane', 'crate', 'slate', 'trace', 'grape']) -> ('crane', 2)`
Why	Without a dictionary every position is searched letter by letter; with five candidate words a handful of well-chosen questions are enough.

FULL CODE | main question first, then every follow-up, tests at the end

```
# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ========================================================

# ========================================================
# MAIN QUESTION
# ========================================================
# ---- main question: helper functions ----
# letters from the most to the least common in English words
BY_FREQUENCY = "etaoinshrdlcumwfgypbvkjxqz"


def split_score(words, i, c):
    # how evenly does the question "is letter c at position i?" split the words?
    # The bigger the smaller side, the better: a perfect split is half and half.
    yes = 0
    for word in words:
        if word[i] == c:
            yes += 1
    no = len(words) - yes
    if yes < no:
        return yes
    return no


# ---- main question: main function ----
# query(i, c) is the black box: is letter c at position i of the secret?
# Idea (no dictionary): for each position ask the letters from the most
# to the least common and stop at the first "yes"; if 25 letters said
# "no" the 26th letter is implied and needs no call.
def by_frequency(query):
    calls = 0
    word = []
    for i in range(5):
        for k in range(26):
            c = BY_FREQUENCY[k]
            if k == 25:
                word.append(c)              # the last letter left: no call needed
                break
            calls += 1
            if query(i, c):
                word.append(c)
```

```
                break
    return "".join(word), calls
```

```
# Idea (a dictionary is known): keep the words that still fit every
# answer, and always ask the question that cuts the list closest to
# half, so each call removes about half of the words.
def with_dictionary(query, dictionary):
    words = list(dictionary)
    calls = 0
    while len(words) > 1:
        options = set()
        for word in words:
            for i in range(5):
                options.add((i, word[i]))
        best_option = None
        best_value = -1
        for option in sorted(options):
            value = split_score(words, option[0], option[1])
            if value > best_value:
                best_value = value
                best_option = option
        i, c = best_option
        calls += 1
        answer = query(i, c)
        remaining = []
        for word in words:
            if (word[i] == c) == answer:
                remaining.append(word)
        words = remaining
    return words[0], calls


# ---- main question: tests ----
def make_oracle(word):
    def query(i, c):
        return word[i] == c
    return query


def test_main_question():
    word, calls = by_frequency(make_oracle("grape"))
    assert word == "grape" and calls <= 125
    words = ["grape", "grade", "trade", "crane", "plane", "slate", "stare", "share"]
    fewest = 100
```

```
    for target in words:
        found, used = with_dictionary(make_oracle(target), words)
        assert found == target
        assert used <= 6                        # about log2(8) = 3 questions suffice; allow a few more
        if used < fewest:
            fewest = used
    assert fewest >= 3                          # one bit per call: at least ceil(log2(8)) = 3 calls


# ========================================================
# FOLLOW-UP 1: What is the lower bound and when is it reached?
# ========================================================
# Answer: every call returns at most one bit (yes or no), so at least
# ceil(log2(number of candidates)) calls are needed. The half-and-half
# question (split_score) reaches that bound only when a question can
# cut the candidates exactly in half every time; real dictionaries
# usually need a few more calls.


# ========================================================
# RUN EVERYTHING
# ========================================================
def run_all_tests():
    test_main_question()
```

```
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

266 Determine winner in stack-merging game

HARD

Template fit: state = the sorted tuple of stack sizes; a move merges two whose sum stays within the cap

QUESTION

A game starts with stacks of the given sizes. A move merges two stacks into one if their combined size is at most cap. The player who cannot move loses (in the misere variant, the player who cannot move wins). Return whether the first player wins with best play.

Function signature

def first_player_wins(stacks: list, cap: int, misere=False) -> bool

EXAMPLES

Example 1	
Input	`[1, 1], 2`
Output	`true`
Why	`Merge the two stacks of 1 into 2; the opponent has a single stack and cannot move.`

Example 2	
Input	`[1, 1, 1], 2`
Output	`true`
Why	`Any merge makes a stack of 2 next to a stack of 1, whose sum 3 exceeds the cap: the opponent is stuck.`

Example 3	
Input	`[3], 5`
Output	`false`
Why	`A single stack: the first player cannot move and loses.`

Example 4	
Input	`[3], 5, misere = true`
Output	`true`
Why	`The same single stack in the misere variant: being unable to move wins.`

FULL CODE | main question first, then every follow-up, tests at the end

```
# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
def moves(state, cap):
```

```
    # every position one move away: merge two stacks if the new stack fits under the cap.
    # `state` is a sorted tuple of stack sizes; equal results count as one move.
    results = []
    for i in range(len(state)):
        for j in range(i + 1, len(state)):
            if state[i] + state[j] > cap:
                continue
            rest = []
            for k in range(len(state)):
                if k != i and k != j:
                    rest.append(state[k])
            rest.append(state[i] + state[j])
            new_state = tuple(sorted(rest))
            if new_state not in results:
                results.append(new_state)
    return results


# ---- main question: main function ----
# Idea: merging does not depend on the order, so a sorted tuple
# identifies a position. Solve each position once and remember it
# (a dictionary), because many move orders reach the same position.
# A position is WON for the player to move when some move leaves the
# opponent in a LOST position. The player who cannot move loses
# (misere play, follow-up 2: the player who cannot move wins).
def first_player_wins(stacks, cap, misere=False):
    memo = {}

    def wins(state):
        if state in memo:
            return memo[state]
        options = moves(state, cap)
        if not options:
```

```
            answer = misere                 # no move: loses (or wins, in misere play)
        else:
            answer = False
            for next_state in options:
                if not wins(next_state):    # a winning move leaves the opponent lost
                    answer = True
                    break
        memo[state] = answer
        return answer

    return wins(tuple(sorted(stacks)))


# ---- main question: tests ----
def test_main_question():
    assert not first_player_wins([3], 5)            # one stack: no move, the first player loses
    assert first_player_wins([1, 1], 5)             # one merge ends the game, we win
    assert first_player_wins([1, 1, 1], 2) is True


# ======================================================
# FOLLOW-UP 1: The game is a sum of independent games (several separate boards or piles)
# ======================================================
# ---- follow-up 1: function ----
# Answer: compute each board's Grundy number (the smallest number that
# is not the Grundy number of any position one move away, the "mex")
# and XOR them. The first player wins exactly when the XOR is not zero.
def grundy(stacks, cap):
    memo = {}

    def number(state):
        if state in memo:
```

```
            return memo[state]
        reachable = set()
        for next_state in moves(state, cap):
            reachable.add(number(next_state))
        value = 0
        while value in reachable:
            value += 1
        memo[state] = value
        return value

    return number(tuple(sorted(stacks)))
```

```
def sum_of_games_wins(boards, cap):
    total = 0
    for board in boards:
        total ^= grundy(board, cap)
    return total != 0


# ---- follow-up 1: tests ----
def test_follow_up_1():
    for state in ([1, 2, 3], [2, 2, 2, 2], [1, 1, 4]):
        win = first_player_wins(state, 6)
        assert win == (grundy(state, 6) != 0)          # normal play: wins exactly when Grundy is not 0
    assert sum_of_games_wins([[1, 1], [1, 1]], 5) is False   # two equal boards cancel out


# ========================================================
# FOLLOW-UP 2: Misere play (the player who cannot move wins)
# ========================================================
# ---- follow-up 2: tests ----
# Answer: flip the base case of the search (no move means the player
# to move WINS): the `misere` parameter above. Grundy numbers no longer
# combine by XOR in general, so search the whole position.
def test_follow_up_2():
    assert first_player_wins([3], 5, misere=True)        # a single stack: no move, so the first player wins
    assert not first_player_wins([1, 1], 5, misere=True)


# ========================================================
# FOLLOW-UP 3: Count the number of winning first moves
# ========================================================
# ---- follow-up 3: function ----
# Answer: the same win / lose search; count the moves whose resulting
# position is lost for the opponent.
def winning_first_moves(stacks, cap):
    count = 0
    for next_state in moves(tuple(sorted(stacks)), cap):
        if not first_player_wins(list(next_state), cap):
            count += 1
    return count


# ---- follow-up 3: tests ----
def test_follow_up_3():
    assert winning_first_moves([1, 1, 1], 2) >= 1
    assert winning_first_moves([3], 5) == 0
```

```


# ========================================================
# RUN EVERYTHING
# ========================================================
def run_all_tests():
    test_main_question()
```

```
    test_follow_up_1()
    test_follow_up_2()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

267 Find secret word with match-count feedback HARD

Template fit: guess the word that minimises the size of the biggest feedback group; keep the words with the same match count

QUESTION

Problem

You are given a list of **unique** lowercase words, all of the same length (e.g., length = 6). One of these words is a **secret** word. You can interact with a black-box API:

- **`matchCount(guess) -> int`**: when you submit a word **`guess`** from the list, the API returns the number of positions **`i`** such that **`guess[i] == secret[i]`** (i.e., exact character match at the same index). You are allowed to call **`matchCount`**

at most 10 times.

Task

Design/implement a function (e.g., **`findSecretWord(words)`**) that uses the API to identify the secret word within the guess limit.

Notes / Constraints

- **`words`** contains between 1 and 100 words.
- All words have equal length **`L`** (commonly **`L = 6`**).
- Every guess must be a word from **`words`**.
- After each API response, you may adapt your next guess.

What to return

Depending on the interface, you may either: - return the identified secret word, or - stop once **`matchCount(guess) == L`** (found it). (Your interviewer may treat this as an interactive problem; focus on strategy + correctness under the 10-guess constraint.)

You are given a list of unique lowercase words, and every word has the same length **`L`**. Exactly one word in the list is the secret word. In the original interview version, you interact with a black-box API:

- **`matchCount(guess) -> int`**
- It returns how many positions **`i`** satisfy **`guess[i] == secret[i]`** You may call this API at most 10 times, and every guess must be a word from the given list. In this non-interactive version, implement **`solution(words, secret)`** and simulate the API inside your function. The parameter **`secret`** is provided only so the judge can verify your strategy. Your function should repeatedly make guesses, use the returned match counts to eliminate impossible candidates, and return the discovered secret word. If your strategy does not find the secret within 10 guesses, return an empty string **`""`**.

Constraints**`1 <= len(words) <= 1001 <= len(words[i]) <= 10`**All words are unique, lowercase, and have the same length **`Lsecret`** is guaranteed to be one of the words in **`words`**Your algorithm may use at most 10 simulated **`matchCount`** callsTest data is designed so that a correct elimination strategy can identify the secret within the limit

EXAMPLES

```
Input: (["acckzz", "ccbazz", "eiowzz", "abcczz"], "acckzz")
```

QUESTION

Expected Output: 'acckzz'Explanation: The secret is already one of the candidates. Using match-count feedback, the candidate set shrinks until **acckzz** is identified.

```
Input: (["secret"], "secret")
```

Expected Output: 'secret'Explanation: With only one word in the list, the first allowed guess must be the secret.

Hints

After a guess returns **k**, any valid remaining candidate must also have exactly **k** matching positions with that guess.A strong strategy is to choose the next guess that splits the remaining candidates as evenly as possible, minimizing the size of the largest feedback bucket.

FULL CODE | main question first, then every follow-up, tests at the end

```python
# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ========================================================

# ========================================================
# MAIN QUESTION
# ========================================================
# ---- main question: helper functions ----
def match(a, b):
    # the number of positions that hold the same letter
    count = 0
    for i in range(len(a)):
        if a[i] == b[i]:
            count += 1
    return count


def make_words(alphabet, length):
    # every word of the given length over the alphabet
    words = [""]
    for position in range(length):
        longer = []
        for word in words:
            for letter in alphabet:
                longer.append(word + letter)
        words = longer
    return words


# ---- main question: main function ----
# Idea: the secret agrees with every earlier guess in exactly as many
# positions as the API reported, so after each guess only the words
# with that same match count against the guess can still be the secret.
# Guess the word that makes the BIGGEST remaining group as small as
# possible (minimax), so the candidates shrink fast.
def best_guess(candidates):
    best_word = None
    best_worst = None
    for guess in candidates:
        group_sizes = {}
        for word in candidates:
            m = match(guess, word)
```

```python
            group_sizes[m] = group_sizes.get(m, 0) + 1
        worst = 0
        for size in group_sizes.values():
            if size > worst:
```

```
                worst = size
        if best_worst is None or worst < best_worst or (worst == best_worst and guess < best_word):
            best_worst = worst
            best_word = guess
    return best_word


def find_secret(words, match_count, limit=10):
    # match_count(guess) is the black-box API: how many positions match the secret
    candidates = list(words)
    guesses = 0
    while candidates and guesses < limit:
        guess = best_guess(candidates)
        guesses += 1
        score = match_count(guess)
        if score == len(guess):
            return guess, guesses
        remaining = []
        for word in candidates:
            if match(guess, word) == score:
                remaining.append(word)
        candidates = remaining
    return None, guesses


# ---- main question: tests ----
def test_main_question():
    words = make_words("abc", 4)
    worst_used = 0
    for secret in words:
        calls = []

        def api(guess, secret=secret):
            calls.append(guess)
            return match(guess, secret)

        found, used = find_secret(words, api)
        assert found == secret and used == len(calls)
        if used > worst_used:
            worst_used = used
    assert worst_used <= 10                      # all 81 words are found inside the limit of 10
```

```
    def always_right(guess):
        return 4

    assert find_secret(["only"], always_right) == ("only", 1)


# ======================================================
# FOLLOW-UP 1: Show how the candidate list shrinks
# ======================================================
# ---- follow-up 1: function ----
# Answer: after a guess whose answer is m, only words with match count
# m against that guess survive. Printing the sizes shows the list
# shrinking round by round.
def shrink_report(words, secret):
    sizes = [len(words)]
    candidates = list(words)
    while len(candidates) > 1:
        guess = best_guess(candidates)
        score = match(guess, secret)
        if score == len(secret):
            break
        remaining = []
        for word in candidates:
```

```
            if match(guess, word) == score:
                remaining.append(word)
        candidates = remaining
        sizes.append(len(candidates))
    return sizes


# ---- follow-up 1: tests ----
def test_follow_up_1():
    sizes = shrink_report(make_words("abc", 4), "abca")
    assert sizes[0] == 81
    for i in range(len(sizes) - 1):
        assert sizes[i] > sizes[i + 1]


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
```

```
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

268 Babylon Tile-Stack Merge Game

HARD

Template fit: state = the sorted tuple of stacks; a move merges two stacks that share a height or a colour; you win if some merge leaves the opponent without a winning reply

QUESTION

A two-player game on stacks of tiles. Each stack has a height and a colour (the colour of its top tile). A move places one stack on top of another when the two stacks have the same height or the same colour; the new stack has the combined height and the colour of the stack that was placed on top. The player who cannot move loses. Given the stacks as (height, colour) pairs, return whether the player to move wins with best play.

Function signature

def babylon_wins(stacks: list) -> bool

EXAMPLES

Example 1	
Input	`[(1, 'Y'), (1, 'W')]`
Output	`true`
Why	`Two stacks of height 1: the mover merges them, and the opponent has nothing left to move.`

Example 2	
Input	`[(1, 'Y')]`
Output	`false`
Why	`A single stack: the mover has no move and loses.`

QUESTION

Example 3	
Input	`[(1, 'Y'), (2, 'W')]`
Output	`false`
Why	`Heights differ and colours differ: no move is possible, so the mover loses.`

Example 4	
Input	`[(1, 'Y'), (1, 'Y'), (1, 'W'), (1, 'W')]`
Output	`true`
Why	`Four height-1 stacks of two colours: the mover has a winning plan.`

FULL CODE | main question first, then every follow-up, tests at the end

```
# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ========================================================

# ========================================================
# MAIN QUESTION
# ========================================================
# ---- main question: helper functions ----
class Babylon:
    # a stack is (height, colour of its top tile); players take turns putting
    # one stack on top of another, allowed when the two stacks have the same
    # height or the same colour on top. A player with no legal move loses.
    def __init__(self, stacks=None):
        if stacks is None:
            stacks = []
            for colour in "YWGB" * 3:               # the real start: 12 tiles, three of each colour
                stacks.append((1, colour))
        self.stacks = list(stacks)

    def legal_moves(self):
        # (i, j) means: put stack i on top of stack j
        moves = []
        for i in range(len(self.stacks)):
            for j in range(len(self.stacks)):
                if i == j:
                    continue
                height_i, colour_i = self.stacks[i]
                height_j, colour_j = self.stacks[j]
                if height_i == height_j or colour_i == colour_j:
                    moves.append((i, j))
        return moves

    def after(self, i, j):
        # the stacks after the move; the game itself is not changed
        height_i, colour_i = self.stacks[i]
        height_j, colour_j = self.stacks[j]
        rest = []
        for k in range(len(self.stacks)):
            if k != i and k != j:
                rest.append(self.stacks[k])
        rest.append((height_i + height_j, colour_i))      # heights add; the moved stack's colour is on top
        return rest
```

```
    def play(self, i, j):
```

```
        if (i, j) not in self.legal_moves():
            raise ValueError("illegal move")
        self.stacks = self.after(i, j)

    def is_over(self):
        return not self.legal_moves()


# ---- main question: main function ----
# Idea: a position is WON for the player to move when some legal move
# leaves the opponent in a LOST position (and a position with no move is
# lost). Remember every solved position; sort the stacks first so equal
# positions share one memory entry.
def babylon_wins(stacks):
    memo = {}

    def solve(state):
        if state in memo:
            return memo[state]
        game = Babylon(state)
        result = False
        for i, j in game.legal_moves():
            next_state = tuple(sorted(game.after(i, j)))
            if not solve(next_state):                    # a move to a position the opponent loses
                result = True
                break
        memo[state] = result
        return result

    return solve(tuple(sorted(stacks)))


def best_move(game):
    # a move that leaves the opponent lost, if there is one
    for i, j in game.legal_moves():
        if not babylon_wins(game.after(i, j)):
            return i, j
    moves = game.legal_moves()
    if moves:
        return moves[0]
    return None
```

```
# ---- main question: tests ----
def test_main_question():
    assert Babylon([(1, "Y")]).is_over()
    assert not babylon_wins([(1, "Y")])                  # one stack: no move, the player to move loses
    assert babylon_wins([(1, "Y"), (1, "W")])            # one move ends the game, we win
    game = Babylon([(1, "Y"), (1, "Y"), (1, "W")])
    assert (0, 1) in game.legal_moves()
    while not game.is_over():                             # play a whole game with the solver
        game.play(*best_move(game))
    assert len(game.stacks) >= 1
    try:
        Babylon([(1, "Y"), (2, "W")]).play(0, 1)
        raise AssertionError("move should be illegal")
    except ValueError:
        pass
    assert len(Babylon().legal_moves()) > 0


# ======================================================
# FOLLOW-UP 1: How big is the real state space?
# ======================================================
# Answer: the real start has 12 single tiles, 12 * 11 = 132 ordered moves,
# and a position is a multiset of (height, colour) stacks, so the number
```

```
# of different positions is far smaller than the number of move orders:
# the memory (sorted tuple as the key) is what makes the search finish.


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

GROUP 40 OF 43 | 5 QUESTIONS, ONE CODE TEMPLATE | 4 M 1 H

Sliding window: grow on the right, shrink on the left while it is too bad

269 Temperature Monitoring Data Structure — MEDIUM

Template fit: a deque of the last n readings with a running total; a second deque of decreasing values gives the maximum

QUESTION

Design a monitor over the last n temperature readings. insert(x) adds a reading (the oldest one drops out once more than n are stored), average() returns the mean of the stored readings, and maximum() returns the largest of them in O(1).

Function signature

class Monitor:
    def __init__(self, n: int)
    def insert(self, x: float) -> None
    def average(self) -> float
    def maximum(self) -> float

EXAMPLES

Example 1

Input	`m = Monitor(3)`
	`m.insert(20)`
	`m.insert(22)`
	`m.insert(19)`
	`m.average()`
	`m.insert(25)`
	`m.average()`
	`m.maximum()`
	`m.insert(10)`
	`m.maximum()`

Output	`m.average() -> 20.333333`
	`m.average() -> 22`
	`m.maximum() -> 25`
	`m.maximum() -> 25`

Why	`With n = 3 the window slides: after 25 arrives the window is 22, 19, 25 (average 22, maximum 25); after 10 it is 19, 25, 10, so the maximum is still 25.`

FULL CODE | main question first, then every follow-up, tests at the end

```
# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

from collections import deque


# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
class MonotoneQueue:
    # keeps the BEST value of the window at the front. keep_max=True keeps
    # the maximum, keep_max=False the minimum. A new value removes from the
    # back every value it beats: those can never be the best again, because
    # the new one stays in the window longer.
```

```
    def __init__(self, keep_max):
        self.items = deque()                        # (key, value) pairs, best first
        self.keep_max = keep_max

    def at_least_as_good(self, a, b):
        if self.keep_max:
            return a >= b
        return a <= b

    def push(self, key, value):
        while self.items and self.at_least_as_good(value, self.items[len(self.items) - 1][1]):
            self.items.pop()
        self.items.append((key, value))

    def expire(self, cutoff):
        # forget the entries whose key is at or below the cutoff (they left the window)
        while self.items and self.items[0][0] <= cutoff:
            self.items.popleft()

    def best(self):
        return self.items[0][1]


# The shared window: every reading with a key (its number or its day), a running
# total for the average, and two monotone queues for the maximum and the
# minimum (follow-up 2). Median needs more (two heaps with lazy deletion or a
```

```
# sorted container, O(log N) per update) because it is not decomposable like
# a sum or a maximum.
class SlidingStats:
    def __init__(self):
        self.items = deque()                        # (key, value), oldest first
        self.total = 0
        self.max_queue = MonotoneQueue(True)
        self.min_queue = MonotoneQueue(False)

    def add(self, key, value):
        self.items.append((key, value))
        self.total += value
        self.max_queue.push(key, value)
        self.min_queue.push(key, value)

    def expire(self, cutoff):
        # drop the readings with a key at or below the cutoff, everywhere
        while self.items and self.items[0][0] <= cutoff:
            self.total -= self.items.popleft()[1]
        self.max_queue.expire(cutoff)
        self.min_queue.expire(cutoff)

    def average(self):
        return self.total / len(self.items)

    def maximum(self):
        return self.max_queue.best()

    def minimum(self):
        return self.min_queue.best()


# ---- main question: main class ----
# insert(x) adds a reading; average() and maximum() describe the LAST n readings.
# Idea: keep the window in a deque with a running sum; on insert add the new
# value and drop the oldest when the window holds more than n. The maximum comes
# from a monotone queue, so every operation is O(1) amortised.
class Monitor(SlidingStats):
    def __init__(self, n):
```

```
        SlidingStats.__init__(self)
        self.n = n
        self.seen = 0

    def insert(self, x):
        self.add(self.seen, x)                    # the number of the reading is its key
        self.seen += 1
        self.expire(self.seen - self.n - 1)       # keep only the newest n readings


# ---- main question: tests ----
def test_main_question():
    m = Monitor(3)
    for x in [5, 1, 4, 2]:
        m.insert(x)
    assert m.maximum() == 4                       # the window is [1, 4, 2]
    assert m.average() == 7 / 3
    m.insert(0)
    assert m.maximum() == 4                       # the window is [4, 2, 0]


# ======================================================
# FOLLOW-UP 1: The window is the last D days (elapsed time), not the last N readings
# ======================================================
# ---- follow-up 1: function ----
# Answer: store (day, value) pairs and drop from the front while the day is
# at or before (today minus the span), in the window and in the monotone
# queues. Everything else is unchanged.
class TimedMonitor(SlidingStats):
    def __init__(self, span):
        SlidingStats.__init__(self)
        self.span = span

    def insert(self, day, x):
        self.add(day, x)
        self.expire(day - self.span)              # drop the readings at or before day - span


# ---- follow-up 1: tests ----
def test_follow_up_1():
    timed = TimedMonitor(2)
    for day, x in [(1, 9), (2, 3), (3, 5)]:
        timed.insert(day, x)
    assert timed.maximum() == 5 and timed.average() == 4      # day 1 is two days old: it left the window


# ======================================================
# FOLLOW-UP 2: Also support minimum, or the median
# ======================================================
# Answer: SlidingStats already has minimum() (a second monotone queue with
# the opposite comparison). The median needs two heaps with lazy deletion or a
# sorted container: O(log N) per update.
# ---- follow-up 2: tests ----
def test_follow_up_2():
    m = Monitor(3)
    for x in [5, 1, 4, 2]:
        m.insert(x)
    assert m.minimum() == 1
    m.insert(0)
    assert m.minimum() == 0


# ======================================================
# FOLLOW-UP 3: Floating-point drift in the running sum
```

```
# ============================================================
# Answer: a running sum that adds and subtracts floats slowly drifts. Add up
# the window again from scratch every so often, or keep the sum as whole
# numbers (e.g. tenths of a degree).


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

270 Shortest Subarray with K Distinct Integers — MEDIUM

Template fit: state = a count dict; while it holds at least k distinct values, record the window and shrink from the left

QUESTION

Given an array nums and an integer k, return the length of the shortest contiguous subarray that contains at least k different integers, or -1 if there is no such subarray.

Function signature

def shortest_k_distinct(nums: list, k: int) -> int

EXAMPLES

Example 1	
Input	[1, 2, 1, 3, 2], 3
Output	3
Why	[2, 1, 3] is the shortest window holding three different numbers.

Example 2	
Input	[1, 1, 1], 2
Output	-1
Why	Only one distinct number exists.

Example 3	
Input	[1, 2], 2
Output	2
Why	The whole array is needed.

CONSTRAINTS

- 1 <= nums.length <= 10^5
- 1 <= nums[i] <= 10^9
- 1 <= k <= nums.length

FULL CODE | main question first, then every follow-up, tests at the end

```
# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
```

```
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

import random


# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: main function ----
# The length of the shortest subarray with at least k distinct values (-1
# if none).
# Idea: a sliding window with a count of every value in it. Grow the right
# end; whenever the window holds at least k distinct values, record its length
# and shrink from the left as far as it stays valid (delete a value from the
# counts when its count reaches 0).
def shortest_k_distinct(nums, k):
    counts = {}
    left = 0
    best = None
    for right in range(len(nums)):
        counts[nums[right]] = counts.get(nums[right], 0) + 1
        while len(counts) >= k:
            size = right - left + 1
            if best is None or size < best:
                best = size
            counts[nums[left]] -= 1
            if counts[nums[left]] == 0:
                del counts[nums[left]]
            left += 1
    if best is None:
        return -1
    return best


# ---- main question: tests ----
def test_main_question():
    assert shortest_k_distinct([1, 2, 1, 3, 4], 3) == 3
    assert shortest_k_distinct([1, 1, 1], 2) == -1
    assert shortest_k_distinct([5], 1) == 1
    assert shortest_k_distinct([], 1) == -1
```

```
# ============================================================
# FOLLOW-UP 1: Count the subarrays with exactly k distinct values (LeetCode 992)
# ============================================================
# ---- follow-up 1: helper functions ----
def at_most(nums, k):
    # the number of subarrays with at most k distinct values: for every right
    # end, all windows that start between `left` and `right` qualify
    counts = {}
    left = 0
    total = 0
    for right in range(len(nums)):
        counts[nums[right]] = counts.get(nums[right], 0) + 1
        while len(counts) > k:
            counts[nums[left]] -= 1
            if counts[nums[left]] == 0:
                del counts[nums[left]]
            left += 1
        total += right - left + 1
    return total
```

```python
# ---- follow-up 1: function ----
# Answer: exactly(k) = atMost(k) - atMost(k - 1), where atMost is a sliding
# window that shrinks while it holds more than k distinct values. Both are O(n).
def exactly(nums, k):
    return at_most(nums, k) - at_most(nums, k - 1)


# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert exactly([1, 2, 1, 2, 3], 2) == 7
    assert exactly([1, 2, 1, 3, 4], 3) == 3
    random.seed(27)
    for round_number in range(300):                  # compare with counting every subarray
        nums = []
        for i in range(random.randint(0, 10)):
            nums.append(random.randint(1, 4))
        k = random.randint(1, 5)
        slow = 0
        for i in range(len(nums)):
            for j in range(i, len(nums)):
                if len(set(nums[i:j + 1])) == k:
                    slow += 1
        assert exactly(nums, k) == slow


# ========================================================
# FOLLOW-UP 2: Longest subarray with at most k distinct values
# ========================================================
# ---- follow-up 2: function ----
# Answer: the same window, but keep the LONGEST window whose number of
# distinct values never exceeds k (shrink while it does). O(n).
def longest_at_most(nums, k):
    counts = {}
    left = 0
    best = 0
    for right in range(len(nums)):
        counts[nums[right]] = counts.get(nums[right], 0) + 1
        while len(counts) > k:
            counts[nums[left]] -= 1
            if counts[nums[left]] == 0:
                del counts[nums[left]]
            left += 1
        if right - left + 1 > best:
            best = right - left + 1
    return best


# ---- follow-up 2: tests ----
def test_follow_up_2():
    assert longest_at_most([1, 2, 1, 2, 3], 2) == 4


# ========================================================
# FOLLOW-UP 3: The values are strings or large objects
# ========================================================
# ---- follow-up 3: function ----
# Answer: the counting dictionary works for any hashable value. For huge
# objects, map every distinct value to a small whole number once (O(n)) and
# run the same code on the numbers.
def to_ids(items):
    ids = {}
    numbers = []
    for item in items:
        if item not in ids:
            ids[item] = len(ids)
```

```
        numbers.append(ids[item])
    return numbers
```

```
# ---- follow-up 3: tests ----
def test_follow_up_3():
    assert shortest_k_distinct(to_ids(["a", "b", "a", "c"]), 3) == 3
    assert to_ids(["x", "y", "x"]) == [0, 1, 0]


# ======================================================
# RUN EVERYTHING
# ======================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

271 Find the minimum window substring. MEDIUM

Template fit: state = how many of each needed letter are still missing; while nothing is missing, record the window and shrink

QUESTION

Given strings s and t, return the shortest substring of s that contains every character of t (a character that appears twice in t must appear at least twice in the substring). Return the empty string if there is no such substring.

Function signature

def min_window(s: str, t: str) -> str

EXAMPLES

Example 1	
Input	`'ADOBECODEBANC', 'ABC'`
Output	`'BANC'`
Why	`BANC is the shortest window that holds an A, a B and a C.`

Example 2	
Input	`'a', 'a'`
Output	`'a'`
Why	`The whole string is the only window.`

Example 3	
Input	`'a', 'aa'`
Output	`''`
Why	`t needs two a's but s has only one.`

FULL CODE | main question first, then every follow-up, tests at the end

```
# ======================================================
# HOW THIS FILE IS ORGANISED
```

```
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
def valid_windows(s, t):
    # a sliding window: grow `right` until the window holds all of t, then
    # shrink `left` while it still does. Returns every (left, right) that is
    # valid at the moment it is seen. `need[c]` is how many more of the letter c
    # the window still requires; `missing` counts the letters of t still missing.
    need = {}
    for ch in t:
        need[ch] = need.get(ch, 0) + 1
    missing = len(t)
    left = 0
    windows = []
    for right in range(len(s)):
        ch = s[right]
        if need.get(ch, 0) > 0:
            missing -= 1
        need[ch] = need.get(ch, 0) - 1
        while missing == 0:
            windows.append((left, right))
            need[s[left]] += 1                      # drop the leftmost letter and see if it hurts
            if need[s[left]] > 0:
                missing += 1
            left += 1
    return windows


# ---- main question: main function ----
# The shortest substring of s that contains every letter of t (with its
# multiplicity); the first one if there is a tie; "" if there is none.
def min_window(s, t):
    best = None
    width = 0
    for left, right in valid_windows(s, t):
        if best is None or right - left < width:
            best = (left, right)
```

```
            width = right - left
    if best is None:
        return ""
    return s[best[0]:best[1] + 1]


# ---- main question: tests ----
def test_main_question():
    assert min_window("ADOBECODEBANC", "ABC") == "BANC"
    assert min_window("a", "a") == "a"
    assert min_window("a", "aa") == ""
    assert min_window("ab", "b") == "b"
    assert min_window("aab", "aab") == "aab"


# ============================================================
# FOLLOW-UP 1: The smallest window containing every distinct character of s itself
# ============================================================
# ---- follow-up 1: helper functions ----
def longest_k_distinct(s, k):
    # the related problem: the longest substring with at most k distinct
    # letters, a window that shrinks when it has too many
```

```
    counts = {}
    left = 0
    best = 0
    for right in range(len(s)):
        counts[s[right]] = counts.get(s[right], 0) + 1
        while len(counts) > k:
            counts[s[left]] -= 1
            if counts[s[left]] == 0:
                del counts[s[left]]
            left += 1
        if right - left + 1 > best:
            best = right - left + 1
    return best


# ---- follow-up 1: function ----
# Answer: use t = the distinct characters of s and run the same window.
def window_all_distinct(s):
    letters = ""
    for ch in set(s):
        letters += ch
    return min_window(s, letters)
```

```
# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert window_all_distinct("aabcbcdbca") == "dbca"
    assert longest_k_distinct("eceba", 2) == 3
    assert longest_k_distinct("aa", 1) == 2


# ========================================================
# FOLLOW-UP 2: Return all minimum windows, or the number of valid substrings
# ========================================================
# ---- follow-up 2: function ----
# Answer: for ALL minimum windows keep every window whose length equals the
# best length. For COUNTING: when the window [left, right] is valid, every
# longer end is valid too, so add len(s) - right for each left that is seen.
def all_min_windows(s, t):
    found = valid_windows(s, t)
    if not found:
        return []
    size = None
    for left, right in found:
        if size is None or right - left < size:
            size = right - left
    windows = []
    for left, right in found:
        if right - left == size:
            windows.append(s[left:right + 1])
    return windows


def count_valid_substrings(s, t):
    total = 0
    for left, right in valid_windows(s, t):
        total += len(s) - right
    return total


# ---- follow-up 2: tests ----
def test_follow_up_2():
    assert all_min_windows("abcabc", "abc") == ["abc", "bca", "cab", "abc"]
    assert all_min_windows("xyz", "q") == []
    assert count_valid_substrings("abcabc", "abc") == 10
```

```
# ============================================================
```

```
# FOLLOW-UP 3: Order matters: t must appear as a subsequence
# ============================================================
# ---- follow-up 3: function ----
# Answer: a different problem (LeetCode 727). From a start position match t
# in order going FORWARD to find the end of the window, then walk BACKWARDS from
# that end to find the tightest start; continue from just after that start.
def min_window_subsequence(s, t):
    best = None
    width = 0
    i = 0
    while i < len(s):
        j = 0
        k = i
        while k < len(s) and j < len(t):
            if s[k] == t[j]:
                j += 1
            k += 1
        if j < len(t):
            break                                   # the rest of s cannot contain t any more
        end = k - 1
        j = len(t) - 1
        k = end
        while j >= 0:
            if s[k] == t[j]:
                j -= 1
            k -= 1
        start = k + 1
        if best is None or end - start < width:
            best = (start, end)
            width = end - start
        i = start + 1
    if best is None:
        return ""
    return s[best[0]:best[1] + 1]


# ---- follow-up 3: tests ----
def test_follow_up_3():
    assert min_window_subsequence("abcdebdde", "bde") == "bcde"
    assert min_window_subsequence("abc", "ca") == ""


# ============================================================
# RUN EVERYTHING
# ============================================================
```

```
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

272 Footpath Beauty After Repainting

MEDIUM

Template fit: bad = a block that is not the wanted colour; allowed = the repaint budget m; the longest window is the answer

QUESTION

A footpath is built from n blocks laid in a line, painted with one of k colors. block_colors[i] is the color of block i (colors are integers in 1..k). The beauty value for a given favorite color c is the length of the longest run of consecutive blocks that are all color c . You are allowed to repaint at most m blocks (each to any color you choose). Return the maximum possible beauty value for favorite color c after repainting at most m blocks.

Function signature

def max_beauty_after_repaint(block_colors: list[int], c: int, m: int) -> int:

...

Follow-up (as reported): (Reported) Answer q queries, each with a different favorite color, and return the beauty value (with m fixed) for each. Hint: a sliding window per query, or precompute runs. (Reported) The base version with m = 0 was asked first; the repaint version is the twist.

EXAMPLES

Example 1	
Input	`block_colors = [1, 2, 1, 1, 2, 1], c = 1, m = 1`
Output	`4`
Why	`Repaint one of the 2's. The best window has at most one non-c block, e.g. indices 2..5 [1,1,2,1] -> repaint the 2 -> 4.`

Example 2	
Input	`block_colors = [3, 3, 3], c = 3, m = 0`
Output	`3`
Why	`All blocks are already color 3; no repaint needed.`

Example 3	
Input	`block_colors = [2, 2, 2], c = 1, m = 2`
Output	`2`
Why	`Only 2 repaints allowed, so at most 2 consecutive c blocks.`

CONSTRAINTS

- 1 <= n <= 10^5 , block_colors has length n
- 1 <= k , 1 <= c <= k , 1 <= block_colors[i] <= k
- 0 <= m <= n
- With m = 0 , the answer is simply the longest existing run of color c .

FULL CODE | main question first, then every follow-up, tests at the end

```
# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ========================================================

# ========================================================
# MAIN QUESTION
# ========================================================
# ---- main question: main function ----
# A footpath is a row of blocks with colours. You may repaint at most m
# blocks. The beauty is the length of the longest run of blocks of the
```

```
# favourite colour c. What is the best beauty?
# Idea: the answer is the longest window that holds at most m blocks of
# another colour (those are the ones to repaint). Grow the right end; while the
# window needs more than m repaints, shrink it from the left.
def max_beauty_after_repaint(block_colors, c, m):
    left = 0
    other = 0                                       # the blocks in the window that are not colour c
    best = 0
    for right in range(len(block_colors)):
        if block_colors[right] != c:
            other += 1
        while other > m:
            if block_colors[left] != c:
                other -= 1
            left += 1
        if right - left + 1 > best:
            best = right - left + 1
    return best


# ---- main question: tests ----
def test_main_question():
    assert max_beauty_after_repaint([1, 2, 1, 1, 2, 1], 1, 1) == 4
    assert max_beauty_after_repaint([3, 3, 3], 3, 0) == 3
    assert max_beauty_after_repaint([2, 2, 2], 1, 2) == 2


# ========================================================
# FOLLOW-UP 1: Answer many queries, each with a different favourite colour (m fixed)
# ========================================================
# ---- follow-up 1: function ----
# Answer: list the positions of every colour once. For one colour run a
```

```
# window over ITS positions: between the first and the last position of the
# window, the blocks that are not that colour must be repainted; keep that at
# most m. Then m more blocks (next to the window) can be repainted too, so
# the beauty is the number of positions in the window plus m, but never more
# than the whole path.
def beauty_queries(block_colors, queries, m):
    n = len(block_colors)
    where = {}
    for i in range(n):
        if block_colors[i] not in where:
            where[block_colors[i]] = []
        where[block_colors[i]].append(i)
    answers = []
    for c in queries:
        positions = where.get(c, [])
        left = 0
        best = 0
        for right in range(len(positions)):
            while positions[right] - positions[left] - (right - left) > m:
                left += 1                           # too many other blocks between the two ends
            size = right - left + 1 + m
            if size > n:
                size = n
            if size > best:
                best = size
        if not positions:
            best = min(n, m)
        answers.append(best)
    return answers


# ---- follow-up 1: tests ----
def test_follow_up_1():
```

```
    colors = [1, 2, 1, 1, 2, 1]
    assert beauty_queries(colors, [1, 2, 3], 1) == [4, 2, 1]
    for color in (1, 2, 3):
        assert beauty_queries(colors, [color], 1)[0] == max_beauty_after_repaint(colors, color, 1)


# ======================================================
# RUN EVERYTHING
# ======================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
```

```
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

273 Necklace Cut into Two Halves with Equal D/R Counts — HARD

Template fit: a fixed-size window of half the length slides around the doubled string, keeping a count of D

QUESTION

A necklace is a circular string of beads D and R. Cut it in two places into two arcs of equal length that contain the same number of D beads (and so the same number of R beads). Return the two cut positions and the two arcs, or None if no such cut exists.

Function signature

def necklace_two_cuts(s: str) -> tuple | None

(cut1, cut2, arc1, arc2)

EXAMPLES

Example 1	
Input	'DRDR'
Output	(0, 2, 'DR', 'DR')
Why	Cut at 0 and 2: both arcs are DR.

Example 2	
Input	'DDRR'
Output	(1, 3, 'DR', 'RD')
Why	Linear halves DD and RR are unbalanced, but rotating the cuts by one gives DR and RD.

Example 3	
Input	'DDR'
Output	null
Why	An odd number of beads cannot be split into equal arcs.

FULL CODE | main question first, then every follow-up, tests at the end

```
# ======================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
```

```
# ======================================================

# ======================================================
# MAIN QUESTION
# ======================================================
# ---- main question: helper functions ----
def count_letter(text, letter):
    count = 0
    for ch in text:
        if ch == letter:
            count += 1
    return count


# ---- main question: main function ----
# A circular necklace of D and R beads. Cut it in TWO places so that each of
# the two arcs holds half of the D's and half of the R's. Return
# (cut1, cut2, arc1, arc2) or None.
# Idea: if each arc has half of the D's and half of the R's, each arc has length
# n / 2 (and the total number of D's and of R's must both be even). So slide a
# window of length n / 2 once around the circle (work on s + s so the window may
# wrap) and keep its number of D's; when that equals half of all D's, the window
# and the rest of the circle are the two good arcs.
def necklace_two_cuts(s):
    n = len(s)
    d_total = count_letter(s, "D")
    r_total = count_letter(s, "R")
    if n % 2 == 1 or d_total % 2 == 1 or r_total % 2 == 1:
        return None                                      # each piece needs half of both letters
    half = n // 2
    target = d_total // 2
    doubled = s + s
    window = count_letter(doubled[:half], "D")           # the D's in the first window
    for i in range(n):
        if window == target:
            return i, (i + half) % n, doubled[i:i + half], doubled[i + half:i + n]
        if doubled[i + half] == "D":
            window += 1                                  # move the window one step: add the new bead ...
        if doubled[i] == "D":
            window -= 1                                  # ... and drop the old one
    return None


# the case of ONE cut (follow-up 1): the necklace is treated as a line
def single_cut(s):
    if len(s) % 2 == 1:
        return None
    half = len(s) // 2
    if count_letter(s[:half], "D") * 2 == count_letter(s, "D"):
        return half                                      # a prefix with half of everything
    return None


# ---- main question: tests ----
def test_main_question():
    cuts = necklace_two_cuts("DRDR")
    assert cuts is not None and count_letter(cuts[2], "D") == 1
    i, j, a, b = necklace_two_cuts("DDRR")               # DDRR needs two cuts on the circle: DR + DR
    assert sorted(a) == sorted(b) == ["D", "R"]
    assert necklace_two_cuts("DDR") is None              # odd totals have no answer
    assert necklace_two_cuts("DRDRDR") is None


# ======================================================
# FOLLOW-UP 1: When is a single cut enough?
```

```
# ============================================================
# Answer: on a LINE one cut must leave a prefix with half of the D's and
# half of the R's: single_cut checks that. DRDR works with one cut between the
# halves, DDRR does not (it needs two cuts on the circle).
# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert single_cut("DRDR") == 2
    assert single_cut("DDRR") is None


# ============================================================
# FOLLOW-UP 2: What if the two pieces must be equal strings, not just equal counts?
# ============================================================


# ============================================================
# FOLLOW-UP 3: The two pieces must be equal strings, not just equal counts
# ============================================================
```

```
# ============================================================
# FOLLOW-UP 4: The two pieces must be equal strings, not just have equal counts
# ============================================================
# ---- follow-up 2: function ----
# Answer: a different question: some ROTATION of the necklace must be of the
# form t + t. Try every rotation and compare its two halves (O(n^2)); or O(n)
# by searching s in (s + s)[1:-1] and checking the match at an offset of n / 2.
def equal_string_halves(s):
    n = len(s)
    if n % 2 == 1:
        return None
    half = n // 2
    doubled = s + s
    for i in range(n):
        if doubled[i:i + half] == doubled[i + half:i + n]:
            return i, (i + half) % n
    return None


# ---- follow-up 2: tests ----
def test_follow_up_2():
    assert equal_string_halves("DRDR") is not None            # DRDR is DR + DR ...
    assert equal_string_halves("DDRR") is None                # ... DDRR is not


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

GROUP 41 OF 43 | 5 QUESTIONS, ONE CODE TEMPLATE | 1 E 4 M

Parent pointers: store who is above each node and climb

274 Tree Node. EASY

Template fit: the SQL version of the same idea: p_id is the parent pointer, NULL means root, being someone's parent means inner, else leaf

QUESTION

A binary tree is stored in a table tree_node_table(id, p_id), where p_id is the id of the parent node (NULL for the root). Write a query that labels every node: 'Root' if it has no parent, 'Inner' if it has a parent and at least one child, and 'Leaf' if it has a parent and no children. The output has the columns id and node_types, ordered by id.

Function signature

-- columns: id, node_types

EXAMPLES

Example 1

Input	`CREATE TABLE tree_node_table (id INTEGER, p_id INTEGER);` `INSERT INTO tree_node_table VALUES (1, NULL), (2, 1), (3, 1), (4, 2), (5, 2);`

Output	`1, Root`
	`2, Inner`
	`3, Leaf`
	`4, Leaf`
	`5, Leaf`

Why	`Node 1 has no parent, node 2 has a parent and children 4 and 5, and nodes 3, 4 and 5 have no children.`

Example 2

Input	`CREATE TABLE tree_node_table (id INTEGER, p_id INTEGER);` `INSERT INTO tree_node_table VALUES (1, NULL);`
Output	`1, Root`
Why	`A tree with one node: it has no parent, so it is the Root (not a Leaf).`

FULL CODE | main question first, then every follow-up, tests at the end

```
# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ========================================================

import sqlite3


# ========================================================
# MAIN QUESTION
# ========================================================
# ---- main question: helper functions ----
# The table has one row per node; the root has p_id NULL.
SCHEMA = "CREATE TABLE tree_node_table (id INT, p_id INT)"
INSERT = "INSERT INTO tree_node_table VALUES (?, ?)"


def run(rows, sql):
    # run one query on a fresh in-memory database holding `rows`
    db = sqlite3.connect(":memory:")
```

```
    db.execute(SCHEMA)
    db.executemany(INSERT, rows)
    result = db.execute(sql).fetchall()
    db.close()
    return result


# ---- main question: main function ----
# Idea: one CASE expression, tested in order.
#   Root:  no parent.
#   Inner: has a parent AND somebody lists it as their parent.
#   Leaf:  has a parent and nobody lists it as their parent.
# The Root test comes first, so a one-node tree is a Root, not a Leaf.
TYPES_SQL = """
SELECT t.id,
  CASE
    WHEN t.p_id IS NULL THEN 'Root'
    WHEN EXISTS (SELECT 1 FROM tree_node_table c
                 WHERE c.p_id = t.id) THEN 'Inner'
    ELSE 'Leaf'
  END AS node_types
FROM tree_node_table t
ORDER BY t.id
```

```
"""


# ---- main question: tests ----
#       1
#      / \
#     2   3
#    / \
#   4   5
ROWS = [(1, None), (2, 1), (3, 1), (4, 2), (5, 2)]


def test_main_question():
    expected = [(1, "Root"), (2, "Inner"), (3, "Leaf"), (4, "Leaf"), (5, "Leaf")]
    assert run(ROWS, TYPES_SQL) == expected
    assert run([(7, None)], TYPES_SQL) == [(7, "Root")]      # a single node is the Root


# =====================================================
# FOLLOW-UP 1: Why not use NOT IN for the leaves?
# =====================================================
# ---- follow-up 1: function ----
# Answer: the list inside NOT IN contains the root's NULL p_id, and
# "id NOT IN (..., NULL)" is never true, so you get NO rows. Filter the
# NULL out of the list, or use NOT EXISTS (below).
NOT_IN_SQL = """
SELECT id FROM tree_node_table
WHERE id NOT IN (SELECT p_id FROM tree_node_table)
"""

LEAVES_SQL = """
SELECT t.id FROM tree_node_table t
WHERE NOT EXISTS (SELECT 1 FROM tree_node_table c
                  WHERE c.p_id = t.id)
ORDER BY t.id
"""


# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert run(ROWS, NOT_IN_SQL) == []                       # the NULL ruins NOT IN
    assert run(ROWS, LEAVES_SQL) == [(3,), (4,), (5,)]
```

```
# ============================================================
```

```
# FOLLOW-UP 2: The table has no row for the root's parent and p_id can point to missing ids
# ============================================================
# ---- follow-up 2: function ----
# Answer: look for orphans with a LEFT JOIN. A node whose p_id is not
# NULL but is missing from the table is a data error, not a root;
# decide whether to report it.
ORPHANS_SQL = """
SELECT t.id FROM tree_node_table t
LEFT JOIN tree_node_table p ON p.id = t.p_id
WHERE t.p_id IS NOT NULL AND p.id IS NULL
ORDER BY t.id
"""


# ---- follow-up 2: tests ----
def test_follow_up_2():
    assert run(ROWS + [(9, 42)], ORPHANS_SQL) == [(9,)]      # node 9 points at a parent that does not exist
    assert run(ROWS, ORPHANS_SQL) == []


# ============================================================
# FOLLOW-UP 3: Add the depth of each node
# ============================================================


# ============================================================
# FOLLOW-UP 4: Add the depth of each node
# ============================================================
# ---- follow-up 3: function ----
# Answer: a recursive query. Start from the roots (depth 0), then
# repeatedly join the table to the nodes found so far on p_id, adding 1.
# Every node appears once because it has one parent. One query, O(n)
# with an index on p_id.
DEPTH_SQL = """
WITH RECURSIVE walk(id, depth) AS (
  SELECT id, 0 FROM tree_node_table WHERE p_id IS NULL
  UNION ALL
  SELECT t.id, w.depth + 1
  FROM tree_node_table t JOIN walk w ON t.p_id = w.id
)
SELECT id, depth FROM walk ORDER BY id
"""


# ---- follow-up 3: tests ----
```

```
def test_follow_up_3():
    assert run(ROWS, DEPTH_SQL) == [(1, 0), (2, 1), (3, 1), (4, 2), (5, 2)]


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
```

```
    run_all_tests()
```

275 Employee Hierarchy Add / Delete — MEDIUM

Template fit: manager[emp] and a set of reports per manager; delete re-attaches the reports to the deleted person's manager

QUESTION

Maintain a company hierarchy. Org(ceo) starts with the CEO. add(emp, mgr) adds a new employee under an existing manager (False if emp already exists or mgr does not). delete(emp) removes an employee and their direct reports move up to the deleted person's manager. The CEO can only be deleted when exactly one person reports to them, who then becomes the CEO.

Function signature

class Org:
  def __init__(self, ceo)
  def add(self, emp, mgr) -> bool
  def delete(self, emp) -> bool

EXAMPLES

Example 1

Input	`o = Org('ceo')`
	`o.add('vp', 'ceo')`
	`o.add('dev', 'vp')`
	`o.add('qa', 'vp')`
	`o.add('dev', 'ceo')`
	`o.delete('vp')`
	`o.manager['dev']`
	`o.delete('ceo')`

Output	`o.add('vp', 'ceo') -> true`
	`o.add('dev', 'vp') -> true`
	`o.add('qa', 'vp') -> true`
	`o.add('dev', 'ceo') -> false`
	`o.delete('vp') -> true`
	`o.manager['dev'] -> 'ceo'`
	`o.delete('ceo') -> false`

Why	`'dev' already exists so the second add fails. Deleting 'vp' moves dev and qa up to the ceo. The ceo now has two reports, so the ceo cannot be deleted.`

FULL CODE | main question first, then every follow-up, tests at the end

```python
# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: main class ----
# Idea: keep two dictionaries: manager[emp] (who is above) and reports[emp]
# (the set of people directly below). add() fills both; delete() hands
# each report of the leaving person to that person's manager and removes
# the person from the manager's reports.
class Org:
    def __init__(self, ceo):
        self.root = ceo
```

```
        self.manager = {ceo: None}
        self.reports = {ceo: set()}
        self.hired = {ceo: 0}          # hire order: a smaller number is more senior (follow-ups 2 and 3)
        self.clock = 1

    def add(self, emp, mgr):
        if emp in self.manager or mgr not in self.manager:
            return False               # already here, or the manager does not exist
        self.manager[emp] = mgr
        self.reports[emp] = set()
        self.reports[mgr].add(emp)
        self.hired[emp] = self.clock
        self.clock += 1
        return True

    def reassign(self, emp, new_mgr):
        # move emp under new_mgr (take it out of the old manager's reports first)
        old = self.manager[emp]
        if old is not None:
            self.reports[old].discard(emp)
        self.manager[emp] = new_mgr
        self.reports[new_mgr].add(emp)

    def delete(self, emp):
        # the CEO cannot be deleted here (follow-up 3 handles it)
        if emp not in self.manager or self.manager[emp] is None:
            return False
```

```
        boss = self.manager[emp]
        for report in list(self.reports[emp]):
            self.reassign(report, boss)        # the reports go to the grandparent
        self.reports[boss].discard(emp)
        del self.manager[emp]
        del self.reports[emp]
        return True


# ---- main question: tests ----
def test_main_question():
    o = Org("ceo")
    for emp, mgr in [("a", "ceo"), ("b", "a"), ("c", "a")]:
        assert o.add(emp, mgr)
    assert not o.add("a", "ceo")               # already there
    assert not o.add("x", "nobody")            # unknown manager
    assert o.delete("a")                       # deleting a manager: reports go to the grandparent
    assert o.manager["b"] == "ceo" and o.manager["c"] == "ceo"
    assert not o.delete("nobody")
    assert not o.delete("ceo")                 # the CEO needs the rules of follow-up 3


# ========================================================
# FOLLOW-UP 1: Support moving an employee under another manager and reject cycles
# ========================================================
# ---- follow-up 1: function ----
# Answer: walk UP from the proposed new manager through the manager
# links. If you meet the employee being moved, the move would put the
# employee under their own descendant (a cycle): reject it. O(depth).
def move(org, emp, new_mgr):
    node = new_mgr
    while node is not None:
        if node == emp:
            return False
        node = org.manager[node]
    org.reassign(emp, new_mgr)
    return True
```

```
# ---- follow-up 1: tests ----
def test_follow_up_1():
    o = Org("ceo")
    o.add("a", "ceo")
    o.add("b", "a")
    o.add("d", "b")
```

```
    assert not move(o, "b", "d")              # d is below b: a cycle
    assert move(o, "d", "a")
    assert o.manager["d"] == "a"


# ========================================================
# FOLLOW-UP 2: When a manager leaves, promote the most senior report instead of the grandparent
# ========================================================
# ---- follow-up 2: helper functions ----
def most_senior(org, people):
    # the person hired earliest
    best = None
    for person in people:
        if best is None or org.hired[person] < org.hired[best]:
            best = person
    return best


# ---- follow-up 2: function ----
# Answer: the most senior report takes the leaver's place: the other
# reports now report to them, and they report to the leaver's manager.
def delete_promote_senior(org, emp):
    if emp not in org.manager or org.manager[emp] is None:
        return False
    boss = org.manager[emp]
    reports = list(org.reports[emp])
    if reports:
        heir = most_senior(org, reports)
        for report in reports:
            if report != heir:
                org.reassign(report, heir)
        org.reassign(heir, boss)
    org.reports[boss].discard(emp)
    del org.manager[emp]
    del org.reports[emp]
    return True


# ---- follow-up 2: tests ----
def test_follow_up_2():
    o = Org("ceo")
    for emp, mgr in [("m", "ceo"), ("x", "m"), ("y", "m")]:
        o.add(emp, mgr)
    assert delete_promote_senior(o, "m")
    assert o.manager["x"] == "ceo"             # x was hired before y, so x is promoted
```

```
    assert o.manager["y"] == "x"


# ========================================================
# FOLLOW-UP 3: The CEO leaves
# ========================================================
# ---- follow-up 3: function ----
# Answer: the most senior report becomes the new CEO and the other
# reports go under them. A CEO with nobody below cannot be replaced.
def delete_ceo(org):
    reports = list(org.reports[org.root])
    if not reports:
```

```
        return False
    heir = most_senior(org, reports)
    for report in reports:
        if report != heir:
            org.reassign(report, heir)
    del org.manager[org.root]
    del org.reports[org.root]
    org.root = heir
    org.manager[heir] = None
    return True


# ---- follow-up 3: tests ----
def test_follow_up_3():
    o = Org("ceo")
    o.add("a", "ceo")
    o.add("b", "ceo")
    assert delete_ceo(o)
    assert o.root == "a" and o.manager["b"] == "a"
    assert not delete_ceo(Org("solo"))


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    test_follow_up_3()
    print("all tests passed")
```

```
if __name__ == "__main__":
    run_all_tests()
```

276 Validate parent array forms a tree — MEDIUM

Template fit: climb from every node with three states (unseen, on this walk, verified); meeting your own walk is a cycle; there must be exactly one root

QUESTION

Problem

You are given an integer array **`parent`** of length **`n`** describing a directed parent pointer for each node **`i`** (nodes are labeled **`0..n-1`**).

- **`parent[i]`** is the parent of node **`i`**.
- Exactly one node should be the root, indicated by **`parent[root] = -1`**. Determine whether these **`n`** nodes form a **valid rooted tree**. A valid rooted tree must satisfy all of the following: 1.

Exactly one root (exactly one index **`i`** with **`parent[i] = -1`**). 2.
No cycles (following parent pointers from any node must eventually reach the root). 3.
Connectivity (every node is reachable from the root; equivalently, every node has exactly one simple path to the root).

Input

- **`parent`**: integer array of length **`n`** where each value is either **`-1`** or in **`[0, n-1]`**.

Output

- Return **`true`** if **`parent`** represents a valid rooted tree; otherwise return **`false`**.

Notes

/ Edge Cases - **`n`** can be 1 (then the only valid tree is **`parent[0] = -1`**).

QUESTION

- Self-parenting like **`parent[i] = i`** is invalid.
- Multiple **`-1`** entries (multiple roots) is invalid.
- A cycle among non-root nodes is invalid.
- A disconnected component (some nodes not reachable from the root) is invalid.

You are given an integer array **`parent`** of length **`n`** describing a parent pointer for each node **`i`** where nodes are labeled **`0`** to **`n-1`**.

- **`parent[i]`** is the parent of node **`i`**.
- Exactly one node should be the root, indicated by **`parent[root] = -1`**. Determine whether these nodes form a valid rooted tree. A valid rooted tree must satisfy all of the following:

1. Exactly one root.
2. No cycles.
3. Every node belongs to the same connected structure and has a path to the root. Return **`True`** if **`parent`** represents a valid rooted tree; otherwise return **`False`**.

Constraints

1 <= n <= 2 * 10^5Each **`parent[i]`** is either **`-1`** or an integer in **`[0, n-1]parent[i] = i`** is invalid

EXAMPLES

```
Input: [-1, 0, 0, 1, 1]
Expected Output: True
```

Explanation: Node 0 is the only root. All other nodes are connected under it, and there are no cycles.

```
Input: [2, -1, 1, 2]
Expected Output: True
Explanation: Node 1 is the root. The paths are 0->2->1, 2->1, and 3->2->1, so every node reaches the
    same root.
```

Hints

First count how many roots there are. A valid tree must have exactly one **`-1`** entry.Build children lists from the parent array, then traverse from the root. If you do not visit every node, the structure is disconnected or contains a cycle away from the root.

FULL CODE | main question first, then every follow-up, tests at the end

```
# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ========================================================

# ========================================================
# MAIN QUESTION
# ========================================================
# ---- main question: helper functions ----
def reason(parent):
    # None if parent[] describes a tree, else the first rule that fails.
    # parent[i] is the parent of node i; the root has parent -1.
    n = len(parent)
    roots = 0
    for p in parent:
        if p == -1:
            roots += 1
    if roots != 1:
        return str(roots) + " roots (need exactly 1)"
    for i in range(n):
        p = parent[i]
```

```
        if p != -1 and not (0 <= p < n):
            return "node " + str(i) + ": out-of-range parent " + str(p)
        if p == i:
            return "node " + str(i) + " is its own parent"
    # walk up from every node; 0 = unseen, 1 = on the current walk, 2 = already checked
    state = [0] * n
    for start in range(n):
        path = []
        x = start
        while x != -1 and state[x] == 0:
            state[x] = 1
            path.append(x)
            x = parent[x]
        if x != -1 and state[x] == 1:
            return "cycle through node " + str(x)        # met a node twice on one walk
        for v in path:
            state[v] = 2
    return None


# ---- main question: main function ----
# Idea: exactly one root (-1), every other parent is a real node, and no
```

```
# walk upwards ever meets a node twice (that would be a cycle). Each
# node is coloured once, so the whole check is O(n).
def validate_parent_array(parent):
    return reason(parent) is None


# ---- main question: tests ----
def test_main_question():
    assert validate_parent_array([-1, 0, 0, 1])
    assert validate_parent_array([-1])
    assert not validate_parent_array([-1, -1])             # two roots
    assert not validate_parent_array([1, 0])               # no root, a cycle
    assert not validate_parent_array([-1, 5])              # parent out of range


# ========================================================
# FOLLOW-UP 1: The graph is given as undirected edges (LeetCode 261)
# ========================================================
# ---- follow-up 1: function ----
# Answer: a tree on n nodes has exactly n - 1 edges and no cycle. Check
# the edge count, then union the ends of each edge (union-find) and fail
# on an edge whose ends are already in the same group.
def valid_tree_edges(n, edges):
    if len(edges) != n - 1:
        return False
    leader = list(range(n))

    def find(x):
        while leader[x] != x:
            leader[x] = leader[leader[x]]
            x = leader[x]
        return x

    for u, v in edges:
        a = find(u)
        b = find(v)
        if a == b:
            return False                                   # this edge closes a cycle
        leader[a] = b
    return True


# ---- follow-up 1: tests ----
```

```
def test_follow_up_1():
    assert valid_tree_edges(5, [(0, 1), (0, 2), (0, 3), (1, 4)])
    assert not valid_tree_edges(4, [(0, 1), (1, 2), (2, 0)])


# ============================================================
# FOLLOW-UP 2: Report the reason it is invalid
# ============================================================
# Answer: reason() above already returns which rule failed: the number of
# roots, an out-of-range parent, a node that is its own parent, or the
# node where a cycle was found.
# ---- follow-up 2: tests ----
def test_follow_up_2():
    assert "cycle" in reason([-1, 2, 1])
    assert "out-of-range" in reason([-1, 5])
    assert "own parent" in reason([-1, 1])
    assert "2 roots" in reason([-1, -1])
    assert reason([-1, 0]) is None


# ============================================================
# FOLLOW-UP 3: A binary tree given as child arrays (LeetCode 1361)
# ============================================================
# ---- follow-up 3: function ----
# Answer: exactly one node has nobody pointing to it (the root), every
# other node is pointed to exactly once, and walking from the root
# reaches all n nodes. -1 means "no child".
def valid_binary_tree(n, left, right):
    pointed_to = [0] * n
    for children in (left, right):
        for c in children:
            if c != -1:
                pointed_to[c] += 1
    roots = []
    for i in range(n):
        if pointed_to[i] == 0:
            roots.append(i)
        if pointed_to[i] > 1:
            return False                                    # two parents
    if len(roots) != 1:
        return False
    seen = {roots[0]}
    stack = [roots[0]]
    while stack:
        u = stack.pop()
        for c in (left[u], right[u]):
            if c != -1:
                if c in seen:
                    return False
                seen.add(c)
                stack.append(c)
    return len(seen) == n                                   # every node must be reachable from the root


# ---- follow-up 3: tests ----
def test_follow_up_3():
    assert valid_binary_tree(4, [1, -1, 3, -1], [2, -1, -1, -1])
    assert not valid_binary_tree(2, [1, 0], [-1, -1])


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
```

```
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

277 Lowest Common Ancestor of a Binary Tree III — MEDIUM

Template fit: two pointers climb; when one falls off the top it restarts at the other node, so both meet at the common ancestor

QUESTION

Given two nodes p and q of a binary tree, return their lowest common ancestor: the deepest node that has both of them as descendants (a node counts as a descendant of itself). Every node has a parent pointer, but you are not given the root.

Function signature

def lca_swap(p: PNode, q: PNode) -> PNode

EXAMPLES

Example 1

Input	`class PNode:`
	`def __init__(self, val):`
	`self.val, self.parent = val, None`
	`root, a, b, c = PNode(3), PNode(5), PNode(1), PNode(6)`
	`a.parent = b.parent = root`
	`c.parent = a`
	`lca_swap(c, b).val`
	`lca_swap(c, a).val`
	`lca_swap(a, b).val`

Output	`lca_swap(c, b).val -> 3`
	`lca_swap(c, a).val -> 5`
	`lca_swap(a, b).val -> 3`

Why	`Node 6 hangs under 5, which hangs under the root 3 next to node 1. The ancestor of 6 and 1 is the root 3; the ancestor of 6 and its own parent 5 is 5 itself.`

CONSTRAINTS

- Number of nodes: [2, 10^5]
- -10^9 <= Node.val <= 10^9
- All node values are unique
- p != q
- p and q exist in the tree

FULL CODE | main question first, then every follow-up, tests at the end

```
# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

import random


# ============================================================
```

```
# MAIN QUESTION
# ======================================================
# ---- main question: helper functions ----
class Node:
    # every node knows its parent (the root's parent is None)
    def __init__(self, val, parent=None):
        self.val = val
        self.parent = parent
        self.left = None
        self.right = None


def attach(parent, val, side):
    # create a child on the "left" or "right" of parent and link it both ways
    child = Node(val, parent)
    setattr(parent, side, child)
    return child


def lca_set(p, q):
    # the first idea: write down every ancestor of p (including p), then
    # climb from q until we meet one of them. O(h) time and space.
    seen = set()
    while p:
        seen.add(p)
        p = p.parent
    while q:
        if q in seen:
            return q
        q = q.parent
    return None


# ---- main question: main function ----
```

```
# Idea: two pointers climb towards the root, and when one falls off the
# top it restarts at the OTHER node. Both then walk the same total
# distance (the two path lengths added), so they meet exactly at the
# lowest common ancestor. O(h) time, O(1) space. If the nodes are in
# different trees both end up as None together.
def lca_swap(p, q):
    a = p
    b = q
    while a is not b:
        if a:
            a = a.parent
        else:
            a = q
        if b:
            b = b.parent
        else:
            b = p
    return a


# ---- main question: tests ----
def example_tree():
    root = Node(3)
    five = attach(root, 5, "left")
    one = attach(root, 1, "right")
    six = attach(five, 6, "left")
    two = attach(five, 2, "right")
    four = attach(two, 4, "left")
    return root, five, one, six, two, four
```

```
def test_main_question():
    root, five, one, six, two, four = example_tree()
    assert lca_swap(six, four) is five
    assert lca_swap(five, four) is five
    assert lca_swap(six, one) is root
    assert lca_swap(root, root) is root
    assert lca_swap(six, Node(99)) is None                # a node of another tree
    assert lca_set(six, four) is five


# ============================================================
# FOLLOW-UP 1: Do it without the swap trick (it feels like magic)
# ============================================================
# ---- follow-up 1: helper functions ----
```

```
def depth(node):
    # how many steps up to the root
    steps = 0
    while node.parent:
        node = node.parent
        steps += 1
    return steps


# ---- follow-up 1: function ----
# Answer: measure each node's depth, lift the deeper one up by the
# difference, then climb both together until they are the same node.
# Same O(h) time and O(1) space, and easier to justify.
def lca_by_depth(p, q):
    dp = depth(p)
    dq = depth(q)
    while dp > dq:
        p = p.parent
        dp -= 1
    while dq > dp:
        q = q.parent
        dq -= 1
    while p is not q:
        p = p.parent
        q = q.parent
        if p is None or q is None:
            return None                                   # they never met: different trees
    return p


# ---- follow-up 1: tests ----
def test_follow_up_1():
    root, five, one, six, two, four = example_tree()
    assert lca_by_depth(six, Node(99)) is None
    assert lca_by_depth(four, one) is root
    assert lca_by_depth(six, four) is five


# ============================================================
# FOLLOW-UP 2: Many LCA queries on the same tree
# ============================================================
# ---- follow-up 2: function ----
# Answer: preprocess with BINARY LIFTING: up[k][v] is the 2^k-th
# ancestor of node v. Building costs O(n log n); each query is O(log n):
# first lift the deeper node to the other's depth, then lift both as
```

```
# high as possible while their ancestors still differ. (Or an Euler tour
# with a range-minimum table for O(1) queries.)
class LiftingLCA:
    def __init__(self, root):
        self.index = {root: 0}              # node -> its number
```

```
        self.nodes = [root]                 # number -> node (breadth first order)
        parent = [0]
        self.depth = [0]
        i = 0
        while i < len(self.nodes):
            node = self.nodes[i]
            for child in (node.left, node.right):
                if child:
                    self.index[child] = len(self.nodes)
                    self.nodes.append(child)
                    parent.append(i)
                    self.depth.append(self.depth[i] + 1)
            i += 1
        self.up = [parent]
        count = len(self.nodes)
        while (1 << len(self.up)) < count:
            previous = self.up[-1]
            row = []
            for v in range(count):
                row.append(previous[previous[v]])   # two jumps of 2^k make one jump of 2^(k+1)
            self.up.append(row)

    def query(self, p, q):
        a = self.index[p]
        b = self.index[q]
        if self.depth[a] < self.depth[b]:
            a, b = b, a
        gap = self.depth[a] - self.depth[b]
        k = 0
        while gap > 0:                          # lift a by `gap`, one binary digit at a time
            if gap % 2 == 1:
                a = self.up[k][a]
            gap = gap // 2
            k += 1
        if a == b:
            return self.nodes[a]
        for k in range(len(self.up) - 1, -1, -1):
            if self.up[k][a] != self.up[k][b]:
                a = self.up[k][a]
                b = self.up[k][b]
```

```
        return self.nodes[self.up[0][a]]


# ---- follow-up 2: tests ----
def random_tree(size):
    root = Node(0)
    nodes = [root]
    while len(nodes) < size:
        parent = random.choice(nodes)
        side = random.choice(["left", "right"])
        if getattr(parent, side) is None:
            nodes.append(attach(parent, len(nodes), side))
    return nodes


def test_follow_up_2():
    root, five, one, six, two, four = example_tree()
    index = LiftingLCA(root)
    assert index.query(six, four) is five
    assert index.query(root, four) is root
    random.seed(5)
    for round_number in range(50):                          # random trees: all versions must agree
        nodes = random_tree(random.randint(1, 12))
        index = LiftingLCA(nodes[0])
        p = random.choice(nodes)
```

```
        q = random.choice(nodes)
        assert index.query(p, q) is lca_swap(p, q) is lca_by_depth(p, q)


# ======================================================
# FOLLOW-UP 3: The nodes might not be in the same tree
# ======================================================
# Answer: lca_swap returns None when the nodes are in different trees
# (both pointers reach None together), and lca_by_depth stops with None
# when a pointer runs off the top first.


# ======================================================
# RUN EVERYTHING
# ======================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    print("all tests passed")
```

```
if __name__ == "__main__":
    run_all_tests()
```

278 Root a Tree So It Becomes a Binary Tree — MEDIUM

Template fit: the root has no parent so it needs degree <= 2; everyone else spends one edge on its parent; answer = the first node of degree <= 2

QUESTION

You are given a connected, undirected, acyclic graph (i.e. a tree) with n nodes labeled 0 .. n-1 , described by an edge list. It is guaranteed that every node has at most 3 neighbors. Choose a node to be the root. When rooted at node r , one neighbor of each non-root node becomes its parent and the rest become its children; the root has no parent, so all of its neighbors become children. The rooted tree is a binary tree if every node has at most 2 children. Return the smallest label of a node that can serve as such a root. If no node works, return -1 . Key observation (the trick): a non-root node with degree d has d - 1 children, so it needs d <= 3 ; the root has degree children, so the root needs degree <= 2 . Since all nodes already have degree <= 3 , any node with degree <= 2 is a valid root — and leaves (degree 1) always exist in a nonempty tree, so a valid root always exists for n >= 1 .

Function signature

```
from typing import List, Tuple
def find_binary_tree_root(n: int, edges: List[Tuple[int, int]]) -> int:
    """Return the smallest node label that can root the tree as a binary tree
    (every node has at most 2 children), or -1 if none exists."""
    ...
```

Follow-up (as reported): Now nodes must be 2-colored (black/white) so that every parent differs in color from its children and all nodes at the same depth share a color. Determine whether a valid root plus coloring exists and return one. Hint: coloring each node by depth % 2 from any chosen root always satisfies both rules in a tree, so a valid coloring exists whenever a valid root does.

EXAMPLES

QUESTION

Example 1	
Input	`n = 4, edges = [(0, 1), (0, 2), (0, 3)]`
Output	`1`
Why	`Node 0 has degree 3, so as root it would have 3 children (invalid). Nodes 1, 2, 3 have degree 1; the smallest valid root is 1.`

Example 2	
Input	`n = 3, edges = [(0, 1), (1, 2)]`
Output	`0`
Why	`A path; every node has degree <= 2, so the smallest label 0 works.`

Example 3	
Input	`n = 1, edges = []`
Output	`0`
Why	`A single node is trivially a binary tree.`

CONSTRAINTS

- 1 <= n <= 10^5 ; the input is a valid tree with n - 1 edges (for n == 1 , no edges).
- Every node has at most 3 neighbors.
- Return the smallest valid root label.

FULL CODE | main question first, then every follow-up, tests at the end

```
# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ========================================================

from collections import deque
import random


# ========================================================
# MAIN QUESTION
# ========================================================
# ---- main question: main function ----
# Idea: in a tree where every node may have at most 2 children, only the
# root has as many children as its degree; every other node has one
# FEWER (one edge goes to its parent). So any node of degree 2 or less
# can be the root, and the first one is a valid choice. -1 if there is none.
def find_binary_tree_root(n, edges):
    degree = [0] * n
    for a, b in edges:
        degree[a] += 1
        degree[b] += 1
    for node in range(n):
        if degree[node] <= 2:
            return node
    return -1


# ---- main question: tests ----
def test_main_question():
    assert find_binary_tree_root(4, [(0, 1), (0, 2), (0, 3)]) == 1     # node 0 has three neighbours
    assert find_binary_tree_root(3, [(0, 1), (1, 2)]) == 0
```

```
    assert find_binary_tree_root(1, []) == 0


# ======================================================
# FOLLOW-UP 1: Also 2-colour the nodes so a parent and its children differ and equal depths share a colour
# ======================================================
# ---- follow-up 1: function ----
# Answer: colour each node by its depth from the chosen root, modulo 2
# (0, 1, 0, 1 ...). In a tree that gives a parent and its children
# different colours and every depth one colour, so a valid colouring
# exists whenever a valid root does.
```

```
def root_and_coloring(n, edges):
    root = find_binary_tree_root(n, edges)
    if root == -1:
        return None
    adj = []
    for i in range(n):
        adj.append([])
    for a, b in edges:
        adj[a].append(b)
        adj[b].append(a)
    color = {root: 0}
    queue = deque([root])
    while queue:
        u = queue.popleft()
        for v in adj[u]:
            if v not in color:
                color[v] = 1 - color[u]
                queue.append(v)
    colors = []
    for i in range(n):
        colors.append(color[i])
    return root, colors


# ---- follow-up 1: tests ----
def test_follow_up_1():
    random.seed(81)
    for round_number in range(300):                          # random trees where no node has more than 3 neig
hbours
        n = random.randint(1, 10)
        degree = [0] * n
        edges = []
        for v in range(1, n):
            options = []
            for u in range(v):
                if degree[u] < 3:
                    options.append(u)
            u = random.choice(options)
            if random.random() < 0.5:
                edges.append((u, v))
            else:
                edges.append((v, u))
            degree[u] += 1
            degree[v] += 1
        root, color = root_and_coloring(n, edges)
        assert color[root] == 0
```

```
        for a, b in edges:
            assert color[a] != color[b]


# ======================================================
# RUN EVERYTHING
# ======================================================
```

```
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

Build or transform a grid with index arithmetic

279 Filled ASCII Diamond

EASY

*Template fit: row i has pad = |i - R| underscores on each side and 2 * (R - pad) + 1 x's in the middle*

QUESTION

Given a non-negative integer radius R , render a filled diamond on a square grid of side 2*R + 1 . The grid is indexed by (i, j) with 0 <= i, j <= 2*R , and its center is (R, R) . A cell belongs to the diamond (draw 'x') if its Manhattan distance from the center is at most R , i.e. abs(i - R) + abs(j - R) <= R . Every other cell is background (draw '_'). Return the picture as a list of 2*R + 1 strings, one per row (top to bottom), each of length 2*R + 1 .

Function signature

def render_diamond(R: int) -> list[str]:

    ...

Follow-up (as reported): The naive fill is O(R^2). Can you produce each row directly from its width instead of testing every cell? (Row i is 'x' repeated 2*(R - abs(i-R)) + 1 times, padded with '_' .)

EXAMPLES

Example 1	
Input	`R = 1`
Output	`["_x_", "xxx", "_x_"]`
Why	`Center (1,1). Row 0 has distance 1 from center vertically, so only column 1 (distance 0) is inside.`

Example 2	
Input	`R = 2`
Output	`["__x__", "_xxx_", "xxxxx", "_xxx_", "__x__"]`
Why	`Each row i allows columns within (R - abs(i-R)) of the center.`

Example 3	
Input	`R = 0`
Output	`["x"]`
Why	`A 1x1 grid whose only cell is the center.`

CONSTRAINTS

- 0 <= R <= 500 .
- For R = 0 the result is the single cell ["x"] .

FULL CODE | main question first, then every follow-up, tests at the end

```
# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: main function ----
# Idea: the diamond has 2 * R + 1 rows. Row i is `pad` = |i - R| steps away
```

```
# from the middle row, so it has `pad` underscores on each side and
# 2 * (R - pad) + 1 letters "x" in the middle.
def render_diamond(R):
    rows = []
    for i in range(2 * R + 1):
        pad = i - R
        if pad < 0:
            pad = -pad                                  # the distance from the middle row
        rows.append("_" * pad + "x" * (2 * (R - pad) + 1) + "_" * pad)
    return rows


# ---- main question: tests ----
def test_main_question():
    assert render_diamond(1) == ["_x_", "xxx", "_x_"]
    assert render_diamond(2) == ["__x__", "_xxx_", "xxxxx", "_xxx_", "__x__"]
    assert render_diamond(0) == ["x"]


# ========================================================
# FOLLOW-UP 1: Draw only the outline of the diamond
# ========================================================
# ---- follow-up 1: function ----
# Answer: keep the same loop but put "x" only at the two ends of the
# letter stretch of each row (positions pad and 2R - pad); everything
# inside becomes background. At the tips both ends are the same cell.
def render_hollow_diamond(R):
    rows = []
    for i in range(2 * R + 1):
        pad = i - R
        if pad < 0:
            pad = -pad
        row = ["_"] * (2 * R + 1)
        row[pad] = "x"
        row[2 * R - pad] = "x"
        rows.append("".join(row))
    return rows


# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert render_hollow_diamond(2) == ["__x__", "_x_x_", "x___x", "_x_x_", "__x__"]
    assert render_hollow_diamond(0) == ["x"]
    assert render_hollow_diamond(1) == ["_x_", "x_x", "_x_"]


# ========================================================
# RUN EVERYTHING
# ========================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

280 Connected Crop Arrangement on a Grid

MEDIUM

Template fit: fill the cells in snake order (left to right, then right to left) so equal crops stay in one connected strip

QUESTION

A garden is an n x m grid. There are k crop types numbered 1..k, and crop i must fill exactly counts[i - 1] cells; the counts add up to n * m so every cell is used. Fill the grid so that the cells of each crop form one connected region (cells joined up, down, left or right). Return any valid grid.

Function signature

def arrange_crops(n: int, m: int, counts: list) -> list

EXAMPLES

Example 1	
Input	`2, 3, [2, 3, 1]`
Output	`[[1, 1, 2], [3, 2, 2]]`
Why	`One valid answer: crop 3 is the single cell at the bottom left; crops 1 and 2 are each connected.`

Example 2	
Input	`1, 4, [1, 3]`
Output	`[[1, 2, 2, 2]]`
Why	`A single row: the crops simply sit side by side.`

FULL CODE | main question first, then every follow-up, tests at the end

```
# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
def snake_cells(n, m):
    # the cells of an n x m grid in "snake" order: row 0 left to right,
    # row 1 right to left, row 2 left to right, ... Consecutive cells in
    # this order always share an edge.
    path = []
    for i in range(n):
        for j in range(m):
            if i % 2 == 0:
                path.append((i, j))
            else:
                path.append((i, m - 1 - j))
    return path


def fill_along(grid, path, counts):
    # hand out the path: the first counts[0] cells to crop 1, the next
    # counts[1] cells to crop 2, and so on
    k = 0
    for crop in range(1, len(counts) + 1):
        for i in range(counts[crop - 1]):
            r, c = path[k]
            grid[r][c] = crop
            k += 1
    return grid
```

```
# ---- main question: main function ----
# Idea: every crop must be one connected piece. Take the snake order of
# the cells and give each crop a consecutive stretch of it. Neighbouring
# cells of the snake share an edge, so each stretch (one crop) is connected.
# counts[i] is how many cells crop i + 1 needs; together they fill the grid.
def arrange_crops(n, m, counts):
    total = 0
    for count in counts:
```

```
        total += count
    if total != n * m:
        raise ValueError("the counts must add up to n * m")
    grid = []
    for r in range(n):
        grid.append([0] * m)
    return fill_along(grid, snake_cells(n, m), counts)


# ---- main question: tests ----
def crops_valid(grid, counts):
    # does every crop have the right number of cells, all in one connected piece?
    cells = {}
    for i in range(len(grid)):
        for j in range(len(grid[0])):
            crop = grid[i][j]
            if crop != 0:
                if crop not in cells:
                    cells[crop] = set()
                cells[crop].add((i, j))
    for crop in range(1, len(counts) + 1):
        group = cells.get(crop, set())
        if len(group) != counts[crop - 1]:
            return False
        start = None
        for cell in group:
            start = cell
        seen = set()
        stack = [start]
        while stack:                                    # a flood fill from one cell must reach them all
            r, c = stack.pop()
            if (r, c) in seen:
                continue
            seen.add((r, c))
            for neighbour in ((r + 1, c), (r - 1, c), (r, c + 1), (r, c - 1)):
                if neighbour in group:
                    stack.append(neighbour)
        if seen != group:
            return False
    return True


def test_main_question():
    assert crops_valid(arrange_crops(2, 3, [2, 4]), [2, 4])
    ones = [1] * 9
```

```
    assert crops_valid(arrange_crops(3, 3, ones), ones)            # many tiny crops still connect
    assert crops_valid(arrange_crops(4, 5, [7, 1, 12]), [7, 1, 12])
    try:
        arrange_crops(2, 2, [1, 1])
        raise AssertionError("the counts do not fill the grid")
    except ValueError:
        pass
```

```
# ============================================================
# FOLLOW-UP 1: Two adjacent square gardens (N x N next to M x M)
# ============================================================
# ---- follow-up 1: helper functions ----
def two_gardens_path(n, m):
    # garden A is n x n on the left, garden B is m x m on its right (both start at row 0)
    path = []
    for c in range(n):                                  # snake down and up the columns of garden A
        going_down = (c % 2 == 0) == (n % 2 == 0)       # chosen so that the LAST column ends in row 0
        if going_down:
            rows = list(range(n))
        else:
            rows = list(range(n - 1, -1, -1))
        for r in rows:
            path.append((r, c))
    for r in range(m):                                  # step into garden B at row 0 and snake through it
        if r % 2 == 0:
            cols = list(range(n, n + m))
        else:
            cols = list(range(n + m - 1, n - 1, -1))
        for c in cols:
            path.append((r, c))
    return path


# ---- follow-up 1: function ----
# Answer: use a different path through the combined area: snake down and
# up the columns of the first garden so that its last column ends in the
# top row, step into the second garden there and snake through it. Every
# step of this path moves to a neighbouring cell, so each crop is
# connected, and the crops are handed out along the path as before.
def arrange_two_gardens(n, m, counts):
    total = 0
    for count in counts:
        total += count
    if total != n * n + m * m:
        raise ValueError("the counts must add up to n * n + m * m")
    height = n
    if m > n:
        height = m
    grid = []
    for r in range(height):
        grid.append([0] * (n + m))
    return fill_along(grid, two_gardens_path(n, m), counts)


# ---- follow-up 1: tests ----
def test_follow_up_1():
    counts = [5, 4, 4]                                  # 2 x 2 next to 3 x 3: 13 cells
    assert crops_valid(arrange_two_gardens(2, 3, counts), counts)
    path = two_gardens_path(2, 3)
    assert len(set(path)) == 13                         # every cell exactly once
    for i in range(len(path) - 1):
        a = path[i]
        b = path[i + 1]
        assert abs(a[0] - b[0]) + abs(a[1] - b[1]) == 1     # each step goes to a neighbouring cell
    path = two_gardens_path(3, 2)
    for i in range(len(path) - 1):
        a = path[i]
        b = path[i + 1]
        assert abs(a[0] - b[0]) + abs(a[1] - b[1]) == 1


# ============================================================
# RUN EVERYTHING
```

```python
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

281 Rotate Any m×n 2D Matrix

MEDIUM

*Template fit: clockwise: new[r][c] = old[rows - 1 - c][r] on a cols-by-rows result (or zip(*matrix[::-1]))*

QUESTION

Rotate an m x n matrix 90 degrees clockwise and return the resulting n x m matrix.

Function signature

def rotate_cw(matrix: list) -> list

EXAMPLES

Example 1	
Input	`[[1, 2, 3], [4, 5, 6]]`
Output	`[[4, 1], [5, 2], [6, 3]]`
Why	`The last row becomes the first column: [4, 1] is the old first column read from the bottom.`

Example 2	
Input	`[[1, 2], [3, 4]]`
Output	`[[3, 1], [4, 2]]`
Why	`A square matrix.`

Example 3	
Input	`[[1, 2, 3]]`
Output	`[[1], [2], [3]]`
Why	`A single row becomes a single column.`

FULL CODE | main question first, then every follow-up, tests at the end

```python
# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: main function ----
# Idea: turning a matrix 90 degrees CLOCKWISE moves the item at row i,
# column j of the old m x n matrix to row j, column m - 1 - i of the
# new n x m matrix (the last row becomes the first column). Build the new
# matrix once with that rule. A non-square matrix changes its shape, which
# is why it cannot be rotated in place the way the square LeetCode 48 can.
def rotate_cw(matrix):
    m = len(matrix)
```

```
    if m == 0:
        return []
    n = len(matrix[0])
    rotated = []
    for j in range(n):
        rotated.append([None] * m)
    for i in range(m):
        for j in range(n):
            rotated[j][m - 1 - i] = matrix[i][j]
    return rotated


# the other direction: item (i, j) goes to row n - 1 - j, column i
def rotate_ccw(matrix):
    m = len(matrix)
    if m == 0:
        return []
    n = len(matrix[0])
    rotated = []
    for j in range(n):
        rotated.append([None] * m)
    for i in range(m):
        for j in range(n):
            rotated[n - 1 - j][i] = matrix[i][j]
    return rotated
```

```
# ---- main question: tests ----
A = [[1, 2, 3], [4, 5, 6]]


def test_main_question():
    assert rotate_cw(A) == [[4, 1], [5, 2], [6, 3]]          # the 2 x 3 example becomes 3 x 2
    assert rotate_ccw(A) == [[3, 6], [2, 5], [1, 4]]
    assert rotate_ccw(rotate_cw(A)) == A                      # a clockwise and an anticlockwise turn cancel
out
    assert rotate_cw([[7]]) == [[7]]
    assert rotate_cw([]) == []


# ============================================================
# FOLLOW-UP 1: Can a non-square matrix be rotated in place?
# ============================================================
# Answer: no, not in the same array: an m x n matrix becomes n x m, so the
# shape changes. You can only do it in place for a square matrix
# (transpose, then reverse every row), or by storing the result in the same
# flat list with new indexes.


# ============================================================
# FOLLOW-UP 2: How would you rotate by k * 90 degrees?
# ============================================================


# ============================================================
# FOLLOW-UP 3: Rotate by k quarter turns
# ============================================================
# ---- follow-up 2: function ----
# Answer: four quarter turns are the identity, so reduce k modulo 4 first.
# Then apply the single-turn function that many times (at most three),
# or write the three index formulas directly so the answer is built once.
def rotate_k(matrix, k):
    for turn in range(k % 4):
        matrix = rotate_cw(matrix)
    return matrix
```

```python
# ---- follow-up 2: tests ----
def test_follow_up_2():
    assert rotate_k(A, 4) == A
    assert rotate_k(A, 2) == [[6, 5, 4], [3, 2, 1]]
    assert rotate_k(A, 5) == rotate_cw(A)
    assert rotate_k(A, 3) == rotate_ccw(A)
```

```python
# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_2()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

282 Solve matrix groups and recipe inventory

MEDIUM

Template fit: block (r // k, c // k) holds the minimum of every cell that maps to it; the recipe part is a topological sort

QUESTION

Two tasks. (1) block_mins(matrix, k): cut the matrix into k x k blocks (blocks at the right and bottom edge may be smaller) and return the minimum of every block as a smaller matrix. (2) possible_recipes(recipes, supplies): recipes maps a recipe name to the ingredients it needs, and an ingredient may itself be another recipe; return the recipes that can be made from the supplies.

Function signature

def block_mins(matrix: list, k: int) -> list

def possible_recipes(recipes: dict, supplies: list) -> list

EXAMPLES

Example 1	
Input	`[[3, 1, 4, 1], [5, 9, 2, 6], [5, 3, 5, 8]], 2`
Output	`[[1, 1], [3, 5]]`
Why	`Top-left block (3, 1, 5, 9) has minimum 1, top-right (4, 1, 2, 6) has 1; the bottom row forms smaller blocks with minimum 3 and 5.`

Example 2	
Input	`{'bread': ['flour', 'yeast'], 'sandwich': ['bread', 'ham'], 'cake': ['flour', 'egg', 'sugar']}, ['flour', 'yeast', 'ham', 'egg']`
Output	`['bread', 'sandwich']`
Why	`bread needs flour and yeast (available); sandwich needs bread and ham; cake needs sugar, which is missing.`

FULL CODE | main question first, then every follow-up, tests at the end

```python
# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
```

```python
# ============================================================

from collections import deque


# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: main functions ----
# PART 1: the minimum of every k x k block of a matrix.
# Idea: cell (r, c) belongs to block (r // k, c // k). Go over the matrix
# once and keep the smallest value seen for each block. The blocks at
# the right and bottom edges may be smaller when k does not divide the
# size; r // k and c // k handle that by themselves.
def block_mins(matrix, k):
    rows = len(matrix)
    cols = len(matrix[0])
    block_rows = (rows + k - 1) // k
    block_cols = (cols + k - 1) // k
    mins = []
    for i in range(block_rows):
        mins.append([float("inf")] * block_cols)
    for r in range(rows):
        for c in range(cols):
            if matrix[r][c] < mins[r // k][c // k]:
                mins[r // k][c // k] = matrix[r][c]
    return mins


# PART 2: which recipes can be made? recipes maps a name to its list of
# ingredients, which may be supplies or other recipes.
# Idea (Kahn's algorithm): count for each recipe how many ingredients are
# still missing. A recipe with none missing is made; making it supplies
# an ingredient to every recipe that uses it, lowering their counts.
def possible_recipes(recipes, supplies):
    have = set(supplies)
    missing = {}                                        # recipe -> how many different ingredients it still lack
s
    users = {}                                          # ingredient -> the recipes that need it
    for name in recipes:
        missing[name] = 0
```

```python
        for item in set(recipes[name]):
            if item not in have:
                missing[name] += 1
                if item not in users:
                    users[item] = []
                users[item].append(name)
    queue = deque()
    for name in missing:
        if missing[name] == 0:
            queue.append(name)
    made = []
    while queue:
        name = queue.popleft()
        made.append(name)
        for other in users.get(name, []):               # a made recipe is a new ingredient for its users
            missing[other] -= 1
            if missing[other] == 0:
                queue.append(other)
    return made


# ---- main question: tests ----
def test_main_question():
    matrix = [[5, 3, 8, 1], [2, 9, 4, 7], [6, 0, 3, 2]]
```

```
    assert block_mins(matrix, 2) == [[2, 1], [0, 2]]          # the last row of blocks is only 1 row tall
    assert block_mins([[4]], 3) == [[4]]
    recipes = {"bread": ["flour", "yeast"],
               "sandwich": ["bread", "meat"],
               "burger": ["bread", "meat", "bun"]}
    assert possible_recipes(recipes, ["flour", "yeast", "meat"]) == ["bread", "sandwich"]
    assert possible_recipes({"a": ["b"], "b": ["a"], "c": ["x"]}, []) == []     # a cycle, and a missing item


# ========================================================
# FOLLOW-UP 1: Ingredients have quantities and a shared ingredient can only be used once
# ========================================================
# ---- follow-up 1: function ----
# Answer: now "is it available" is not enough: scarce items are USED UP, so
# the order matters. Here recipes are tried in the given priority order
# and each is made at most once while the stock allows (a made recipe
# becomes new stock). This is a heuristic: the exact best choice is a flow
# problem. needs[i] is {ingredient: amount} for recipes[i].
def recipes_with_quantities(recipes, needs, stock):
    stock = dict(stock)
    made = []
```

```
    progress = True
    while progress:
        progress = False
        for i in range(len(recipes)):
            name = recipes[i]
            if name in made:
                continue
            enough = True
            for item in needs[i]:
                if stock.get(item, 0) < needs[i][item]:
                    enough = False
            if enough:
                for item in needs[i]:
                    stock[item] -= needs[i][item]
                stock[name] = stock.get(name, 0) + 1
                made.append(name)
                progress = True
    return made


# ---- follow-up 1: tests ----
def test_follow_up_1():
    names = ["bread", "cake"]
    needs = [{"flour": 2}, {"flour": 2}]
    assert recipes_with_quantities(names, needs, {"flour": 2}) == ["bread"]      # the only 2 flour go to the
 bread
    assert recipes_with_quantities(names, needs, {"flour": 4}) == ["bread", "cake"]


# ========================================================
# FOLLOW-UP 2: Blocks that do not tile the matrix exactly
# ========================================================
# Answer: already handled: the edge blocks are simply smaller, because a
# cell is assigned to block (r // k, c // k) and the number of blocks is
# the size divided by k ROUNDED UP.


# ========================================================
# FOLLOW-UP 3: Detect which recipes are impossible because of a cycle
# ========================================================
# ---- follow-up 3: function ----
# Answer: after Kahn's algorithm, the recipes that were never made are
# either missing a raw ingredient or sit on a cycle. Tell them apart: if
# an ingredient is neither a supply nor a recipe it is missing; otherwise
```

```
# the recipe waits for something that waits for it (a cycle).
def why_impossible(recipes, supplies):
```

```
    made = set(possible_recipes(recipes, supplies))
    known = set(supplies) | set(recipes)
    reasons = {}
    for name in recipes:
        if name in made:
            continue
        reasons[name] = "cycle"
        for item in recipes[name]:
            if item not in known:
                reasons[name] = "missing " + item
                break
    return reasons


# ---- follow-up 3: tests ----
def test_follow_up_3():
    cyc = {"a": ["b"], "b": ["a"], "c": ["x"]}
    assert why_impossible(cyc, []) == {"a": "cycle", "b": "cycle", "c": "missing x"}


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

283 Range Sum Query 2D (Immutable Matrix) MEDIUM

Template fit: prefix table p[r + 1][c + 1] = cell + p[r][c + 1] + p[r + 1][c] - p[r][c]; a query is four lookups

QUESTION

Given a 2D integer matrix, handle multiple queries of the following type: return the sum of the elements inside the rectangle defined by its upper-left corner (row1, col1) and lower-right corner (row2, col2) , inclusive. Implement a class NumMatrix that is initialized once with the matrix and can then answer many sum_region queries efficiently (each query in O(1) after preprocessing).

Function signature

```
class NumMatrix:
    def __init__(self, matrix: list[list[int]]) -> None:
        """Preprocess the matrix (e.g., build a 2D prefix-sum table)."""
        ...
    def sum_region(self, row1: int, col1: int, row2: int, col2: int) -> int:
        """Return the sum of the submatrix with corners (row1, col1) and (row2, col2),
inclusive."""
        ...
```

Follow-up (as reported): Follow-up constraint change from the report: what if the matrix is mutable (values can be updated between queries)? Hint: switch to a 2D Binary Indexed Tree (Fenwick) for O(log rows * log cols) updates and queries.

EXAMPLES

QUESTION

Example 1	
Input	`matrix = [[3, 0, 1, 4, 2], [5, 6, 3, 2, 1], [1, 2, 0, 1, 5], [4, 1, 0, 1, 7], [1, 0, 3, 0, 5]] sum_region(2, 1, 4, 3)`
Output	`8`
Why	`The 3x3 block rows 2..4, cols 1..3 is [[2,0,1],[1,0,1],[0,3,0]] which sums to 8.`

Example 2	
Input	`same matrix; sum_region(1, 1, 2, 2)`
Output	`11`
Why	`Block [[6,3],[2,0]] sums to 11.`

Example 3	
Input	`same matrix; sum_region(1, 2, 2, 4)`
Output	`12`
Why	`Block [[3,2,1],[0,1,5]] sums to 12.`

CONSTRAINTS

- 1 <= rows, cols <= 200
- -10^5 <= matrix[i][j] <= 10^5
- 0 <= row1 <= row2 < rows , 0 <= col1 <= col2 < cols
- There may be up to 10^4 calls to sum_region .

FULL CODE | main question first, then every follow-up, tests at the end

```python
# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ========================================================

import random


# ========================================================
# MAIN QUESTION
# ========================================================
# ---- main question: main function ----
# Idea (2D prefix sums): p[r][c] is the sum of the rectangle from the
# top-left corner (0, 0) up to (but not including) row r and column c.
# Building p: a cell's value plus the block above plus the block to the
# left, minus the corner block that was counted twice. A query then
# combines FOUR entries of p (inclusion-exclusion): big rectangle, minus
# the strip above, minus the strip on the left, plus the corner that was
# subtracted twice. Each query is O(1).
class NumMatrix:
    def __init__(self, matrix):
        rows = len(matrix)
        cols = 0
        if rows > 0:
            cols = len(matrix[0])
        self.p = []
        for r in range(rows + 1):
            self.p.append([0] * (cols + 1))
        for r in range(rows):
            for c in range(cols):
                self.p[r + 1][c + 1] = matrix[r][c] + self.p[r][c + 1] + self.p[r + 1][c] - self.p[r][c]
```

```
    def sum_region(self, r1, c1, r2, c2):
        p = self.p
        return p[r2 + 1][c2 + 1] - p[r1][c2 + 1] - p[r2 + 1][c1] + p[r1][c1]


# ---- main question: tests ----
GRID = [[3, 0, 1, 4, 2],
        [5, 6, 3, 2, 1],
        [1, 2, 0, 1, 5],
        [4, 1, 0, 1, 7],
        [1, 0, 3, 0, 5]]
```

```
def slow_sum(matrix, r1, c1, r2, c2):
    total = 0
    for r in range(r1, r2 + 1):
        for c in range(c1, c2 + 1):
            total += matrix[r][c]
    return total


def test_main_question():
    nm = NumMatrix(GRID)
    assert nm.sum_region(2, 1, 4, 3) == 8
    assert nm.sum_region(1, 1, 2, 2) == 11
    assert nm.sum_region(1, 2, 2, 4) == 12
    random.seed(2)
    for round_number in range(300):                 # compare with adding up the rectangle cell by cell
        r1 = random.randint(0, 4)
        r2 = random.randint(r1, 4)
        c1 = random.randint(0, 4)
        c2 = random.randint(c1, 4)
        assert nm.sum_region(r1, c1, r2, c2) == slow_sum(GRID, r1, c1, r2, c2)


# =========================================================
# FOLLOW-UP 1: The matrix can change between queries (update a cell). What do you use?
# =========================================================
# ---- follow-up 1: function ----
# Answer: a 2D Binary Indexed Tree (Fenwick tree). update(r, c, value)
# walks both indexes UP by their lowest set bit (i += i & -i), and a prefix
# query walks them DOWN (i -= i & -i). Each operation costs
# O(log R * log C), and a rectangle is four prefix queries.
class MutableMatrix:
    def __init__(self, matrix):
        self.rows = len(matrix)
        self.cols = len(matrix[0])
        self.values = []                             # the current value of every cell
        self.tree = []
        for r in range(self.rows):
            self.values.append([0] * self.cols)
        for r in range(self.rows + 1):
            self.tree.append([0] * (self.cols + 1))
        for r in range(self.rows):
            for c in range(self.cols):
                self.update(r, c, matrix[r][c])
```

```
    def update(self, r, c, value):
        delta = value - self.values[r][c]            # the tree stores sums, so add the CHANGE
        self.values[r][c] = value
        i = r + 1
        while i <= self.rows:
            j = c + 1
```

```
            while j <= self.cols:
                self.tree[i][j] += delta
                j += j & -j
            i += i & -i

    def prefix(self, r, c):
        # the sum of rows 0 .. r - 1 and columns 0 .. c - 1
        total = 0
        i = r
        while i > 0:
            j = c
            while j > 0:
                total += self.tree[i][j]
                j -= j & -j
            i -= i & -i
        return total

    def sum_region(self, r1, c1, r2, c2):
        return self.prefix(r2 + 1, c2 + 1) - self.prefix(r1, c2 + 1) - self.prefix(r2 + 1, c1) + self.prefix(
r1, c1)


# ---- follow-up 1: tests ----
def test_follow_up_1():
    mm = MutableMatrix(GRID)
    assert mm.sum_region(2, 1, 4, 3) == 8
    mm.update(3, 2, 2)                                  # the 0 at row 3, column 2 becomes 2
    assert mm.sum_region(2, 1, 4, 3) == 10
    current = [row[:] for row in GRID]
    current[3][2] = 2
    random.seed(3)
    for round_number in range(200):
        r = random.randint(0, 4)
        c = random.randint(0, 4)
        value = random.randint(0, 9)
        mm.update(r, c, value)
        current[r][c] = value
        r1 = random.randint(0, 4)
        r2 = random.randint(r1, 4)
```

```
        c1 = random.randint(0, 4)
        c2 = random.randint(c1, 4)
        assert mm.sum_region(r1, c1, r2, c2) == slow_sum(current, r1, c1, r2, c2)


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

GROUP 43 OF 43 | 5 QUESTIONS, ONE CODE TEMPLATE | 1 E 2 M 2 H

Split into pieces, fix each piece, join (and pack words into lines)

284 Implement string formatting and request tracking tasks

EASY

Template fit: split, fix each piece, join; the request-tracking part is a small dict

QUESTION

Four small tasks. (A) snake_to_camel(s): convert snake_case to camelCase, ignoring leading, trailing and repeated underscores. (B) format_thousands(n): write an integer with a comma between every group of three digits (keep the minus sign). (C) ocr_valid(t, rot): an OCR scanner may read a label upside down; given the rotation table rot (character -> character seen after turning 180 degrees), return true if t, or t turned 180 degrees (reversed and mapped through rot), is an integer from 1 to 200 without leading zeros. (D) Server: begin(rid, ts) and end(rid, ts) arrive in any order for each request id; drain(watermark) returns the completed requests as 'id begin=b end=e' lines in order of end time, only those with end at most the watermark (all of them when no watermark is given).

Function signature

```
def snake_to_camel(s: str) -> str
def format_thousands(n: int) -> str
def ocr_valid(t: str, rot: dict) -> bool
class Server:
    def begin(self, rid, ts) -> None
    def end(self, rid, ts) -> None
    def drain(self, watermark=None) -> list
```

EXAMPLES

Example 1	
Input	`'make__it_camel_'`
Output	`'makeItCamel'`
Why	`Repeated and trailing underscores are ignored.`

Example 2	
Input	`1234567`
Output	`'1,234,567'`
Why	`A comma before every third digit from the right.`

Example 3	
Input	`-1000`
Output	`'-1,000'`
Why	`The sign stays at the front.`

QUESTION

Example 4

Input	rot = {'0': '0', '1': '1', '2': '2', '5': '5', '6': '9', '8': '8', '9': '6'} ocr_valid('186', rot) ocr_valid('981', rot) ocr_valid('500', rot)
Output	ocr_valid('186', rot) -> true ocr_valid('981', rot) -> true ocr_valid('500', rot) -> false
Why	981 is above 200, but turned upside down it reads 186, which is valid. 500 is too large and its rotation 005 has leading zeros.

Example 5

Input	sv = Server() sv.begin('r2', 5) sv.end('r1', 4) sv.begin('r1', 1) sv.end('r2', 9) sv.drain(8) sv.drain()
Output	sv.drain(8) -> ['r1 begin=1 end=4'] sv.drain() -> ['r2 begin=5 end=9']
Why	r1 finished at 4 and r2 at 9. Draining with watermark 8 prints only r1; draining everything then prints r2.

FULL CODE | main question first, then every follow-up, tests at the end

```python
# ========================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ========================================================

import heapq


# ========================================================
# MAIN QUESTION
# ========================================================
# ---- main question: main functions ----
# TASK A: snake_case to camelCase, ignoring leading, trailing and
# repeated underscores.
# Idea: split on "_" and skip the empty parts (that handles every odd
# underscore in one move). The first word stays as it is; every later
# word gets a capital first letter.
def snake_to_camel(s):
    words = []
    for part in s.split("_"):
        if part != "":
            words.append(part)
    if len(words) == 0:
        return ""
    result = words[0]
    for word in words[1:]:
        result += word.capitalize()
    return result


# TASK B: write an integer with a comma between every group of three
```

```
# digits (keep the minus sign).
# Idea: peel three digits at a time from the RIGHT end of the digit text.
def format_thousands(n):
    sign = ""
    if n < 0:
        sign = "-"
    digits = str(abs(n))
    groups = []
    while len(digits) > 3:
        groups.append(digits[-3:])
        digits = digits[:-3]
    groups.append(digits)
```

```
    groups.reverse()
    return sign + ",".join(groups)


# TASK C: an OCR scanner may read a label upside down. ocr_valid is True
# if the text, or the text turned 180 degrees (reversed, then every
# character mapped through the table `rot`), is an integer from 1 to
# 200 without leading zeros.
def is_valid_number(text):
    # only digits, no leading zero, value 1 to 200
    if text == "" or text[0] == "0":
        return False
    for ch in text:
        if ch < "0" or ch > "9":
            return False
    return 1 <= int(text) <= 200


def turn_around(text, rot):
    # the text as seen after turning it 180 degrees, or None if a character cannot be turned
    result = ""
    for i in range(len(text) - 1, -1, -1):
        if text[i] not in rot:
            return None
        result += rot[text[i]]
    return result


def ocr_valid(t, rot):
    if is_valid_number(t):
        return True
    turned = turn_around(t, rot)
    return turned is not None and is_valid_number(turned)


# TASK D: requests arrive as begin(rid, ts) and end(rid, ts), in any
# order for each id. drain(watermark) returns the COMPLETED requests (both
# events seen) as "id begin=b end=e" lines, in order of end time, only
# those with end <= watermark (all of them when no watermark is given).
# Idea: remember the half that arrived first; when the other half arrives
# the request is complete and goes into a min-heap ordered by end time;
# drain() pops the heap while the end time is within the watermark.
class Server:
    def __init__(self):
        self.begins = {}                  # id -> begin time, waiting for its end
```

```
        self.ends = {}                    # id -> end time, waiting for its begin
        self.done = []                    # heap of (end time, id, begin time)

    def begin(self, rid, ts):
        if rid in self.begins:
            raise ValueError("duplicate begin")        # follow-up 2
        self.begins[rid] = ts
```

```
        self.check_complete(rid)

    def end(self, rid, ts):
        if rid in self.ends:
            raise ValueError("duplicate end")
        self.ends[rid] = ts
        self.check_complete(rid)

    def check_complete(self, rid):
        if rid in self.begins and rid in self.ends:
            begin_time = self.begins.pop(rid)
            end_time = self.ends.pop(rid)
            if end_time < begin_time:
                raise ValueError("ends before it begins")      # follow-up 2
            heapq.heappush(self.done, (end_time, rid, begin_time))

    def drain(self, watermark=None):
        lines = []
        while self.done and (watermark is None or self.done[0][0] <= watermark):
            end_time, rid, begin_time = heapq.heappop(self.done)
            lines.append(str(rid) + " begin=" + str(begin_time) + " end=" + str(end_time))
        return lines


# ---- main question: tests ----
ROT = {"0": "0", "1": "1", "6": "9", "8": "8", "9": "6"}


def test_main_question():
    assert snake_to_camel("make__it_camel_") == "makeItCamel"
    assert snake_to_camel("__a__b_") == "aB"
    assert snake_to_camel("") == ""
    assert format_thousands(1234567) == "1,234,567"
    assert format_thousands(-1000) == "-1,000"
    assert format_thousands(999) == "999"
    assert format_thousands(0) == "0"
    assert ocr_valid("196", ROT)                # as written 196 is fine
    assert ocr_valid("961", ROT)                # turned around it reads 196
```

```
    assert ocr_valid("09", ROT)                 # a leading zero, but turned around it reads 60
    assert not ocr_valid("201", ROT)            # too big, and the 2 cannot be turned
    assert not ocr_valid("0", ROT)
    assert not ocr_valid("", ROT)
    s = Server()
    s.begin("a", 1)
    s.begin("b", 2)
    s.end("b", 3)
    s.end("a", 5)
    assert s.drain(3) == ["b begin=2 end=3"]    # only what ended by the watermark
    assert s.drain() == ["a begin=1 end=5"]     # the rest
    assert s.drain() == []
    s.end("c", 9)                               # the end can arrive before the begin
    s.begin("c", 4)
    assert s.drain() == ["c begin=4 end=9"]


# ========================================================
# FOLLOW-UP 1: Why not print as soon as a request completes in the follow-up?
# ========================================================
# Answer: requests complete in any order. A request that ends at time 9
# might complete (be reported) before one that ended at time 5, so printing
# at once would break the order of end times. It is only safe to print
# everything up to a watermark: a time before which no more completions can
# arrive. That is why drain() takes a watermark.
```

```
# ============================================================
# FOLLOW-UP 2: How do you handle bad input (duplicate begin, end before begin, never-completed requests)?
# ============================================================
# ---- follow-up 2: tests ----
# Answer: a duplicate begin or end, or an end time before the begin time,
# raises ValueError (see Server). Requests that never complete stay in
# `begins` or `ends`; report them separately or drop them after a timeout.
def test_follow_up_2():
    s = Server()
    s.begin("a", 1)
    for call in ("duplicate", "backwards"):
        try:
            if call == "duplicate":
                s.begin("a", 2)
            else:
                s.end("a", 0)
            raise AssertionError("should fail")
        except ValueError:
```

```
            pass


# ============================================================
# FOLLOW-UP 3: Rotation map provided at runtime with extra pairs
# ============================================================
# Answer: ocr_valid already takes the table as an argument, so any
# extra pair (for example "2" and "5" turning into each other) just goes
# into the dictionary.
# ---- follow-up 3: tests ----
def test_follow_up_3():
    extended = dict(ROT)
    extended["2"] = "5"
    extended["5"] = "2"
    assert ocr_valid("102", extended)            # as written 102 is fine
    assert ocr_valid("501", extended) is True    # turned: reversed "105" -> "102"... valid


# ============================================================
# FOLLOW-UP 4: The tracker must bound its memory
# ============================================================
# Answer: half-finished requests (only a begin or only an end) are the
# memory risk: drop the ones older than a timeout, and keep the heap small
# by draining often. Duplicate checks need the ids, so keep them only
# for the pending requests.


# ============================================================
# FOLLOW-UP 5: Another locale: a different separator or grouping (Indian style 12,34,567)
# ============================================================
# ---- follow-up 5: function ----
# Answer: make the separator and the group sizes parameters. The lowest
# group may differ from the others (3 digits, then groups of 2 in the
# Indian system). The loop peels one group from the right each time.
def format_grouped(n, sep=",", first=3, rest=3):
    sign = ""
    if n < 0:
        sign = "-"
    digits = str(abs(n))
    groups = []
    size = first
    while len(digits) > size:
        groups.append(digits[len(digits) - size:])
        digits = digits[:len(digits) - size]
        size = rest
```

```
    groups.append(digits)
    groups.reverse()
    return sign + sep.join(groups)


# ---- follow-up 5: tests ----
def test_follow_up_5():
    assert format_grouped(1234567) == "1,234,567"
    assert format_grouped(1234567, first=3, rest=2) == "12,34,567"
    assert format_grouped(1234, sep=".") == "1.234"
    assert format_grouped(-1234567, first=3, rest=2) == "-12,34,567"


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_2()
    test_follow_up_3()
    test_follow_up_5()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

285 Snake Case to Camel Case with Repository-Wide Refactor

MEDIUM

Template fit: the same function applied to every identifier found in the files

QUESTION

Convert a valid snake_case identifier to camelCase, then design a safe repository-wide migration for variables and function names without changing third-party symbols, public interfaces, or the tests' ability to detect regressions.

Implement a conversion from a valid snake_case identifier to camelCase.
The base input is guaranteed valid: no leading underscore, trailing underscore, or doubled separator.
Example: foo_bar -> fooBar.
Extend the discussion to a repository too large for manual review: replace snake-case variables and functions across the codebase.
Explain how to verify that the repository still works after the automated rewrite, including when test-package identifiers are rewritten too.
Preserve third-party functions and public interfaces; explain how the migration distinguishes symbols that may change from contracts that must remain stable.

EXAMPLES

```
foo_bar -> fooBar
```

FULL CODE | main question first, then every follow-up, tests at the end

```
# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

import ast
import io
import tokenize
```

```
# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
def snake_to_camel(name):
    # foo_bar -> fooBar (the input is a valid snake_case name)
    parts = name.split("_")
    result = parts[0]
    for part in parts[1:]:
        result += part[:1].upper() + part[1:]
    return result


def owned_names(source, keep=()):
    # the names this code defines itself (function names, arguments,
    # variables that are assigned): only these are safe to rename. The syntax
    # tree (ast) tells us what is defined, so imported or library names
    # stay untouched. Names in `keep` are never renamed.
    names = set()
    for node in ast.walk(ast.parse(source)):
        if isinstance(node, ast.FunctionDef):
            names.add(node.name)
            for arg in node.args.args + node.args.kwonlyargs:
                names.add(arg.arg)
        elif isinstance(node, ast.Name) and isinstance(node.ctx, ast.Store):
            names.add(node.id)
    result = set()
    for name in names:
        is_snake = "_" in name.strip("_")             # a real snake_case name
        if is_snake and not name.startswith("_") and name not in keep:
            result.add(name)
    return result
```

```
# ---- main question: main function ----
# Idea: snake_to_camel capitalises each part after the first. To rename a
# whole file, find the names the code OWNS (owned_names) and rewrite only
# those, working on TOKENS so that strings and comments are never touched.
# A name right after a dot (an attribute) may belong to a library, so it
# stays as written.
def rename_identifiers(source, keep=()):
    owned = owned_names(source, keep)
    tokens = []
    previous = None
    for token in tokenize.generate_tokens(io.StringIO(source).readline):
        after_dot = previous is not None and previous.string == "."
        if token.type == tokenize.NAME and token.string in owned and not after_dot:
            token = token._replace(string=snake_to_camel(token.string))
        tokens.append(token)
        previous = token
    return tokenize.untokenize(tokens)


# ---- main question: tests ----
SOURCE = (
    "import os\n"
    "def load_data(file_path):\n"
    "    full_path = os.path.join(file_path, 'x_y')\n"
    "    return full_path  # load_data is fine\n"
    "result_value = load_data('a')\n")


def test_main_question():
    assert snake_to_camel("foo_bar") == "fooBar"
    assert snake_to_camel("a_b_c") == "aBC"
    assert snake_to_camel("name") == "name"
```

```
    new = rename_identifiers(SOURCE, keep={"file_path"})
    assert "def loadData(file_path)" in new
    assert "fullPath = os.path.join" in new
    assert "'x_y'" in new and "# load_data is fine" in new        # strings and comments are intact
    assert "resultValue = loadData('a')" in new


# ============================================================
# FOLLOW-UP 1: How do you prove the migration is safe?
# ============================================================
# ---- follow-up 1: tests ----
# Answer: run the full test suite before and after (green both times),
```

```
# compare the public API surface (the exported names) before and after,
# and migrate in small reviewable commits with a codemod that only touches
# owned symbols. A cheap check: the migrated code must still compile and
# give the same results.
def test_follow_up_1():
    new = rename_identifiers(SOURCE, keep={"file_path"})
    scope = {}
    exec(compile(new, "x", "exec"), scope)
    assert scope["resultValue"] == "a/x_y"


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
    test_follow_up_1()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

286 Google Doc Line Wrap (Word Wrap with Whitespace and Newlines) MEDIUM

Template fit: count lines: go paragraph by paragraph, add a word if it fits (current + 1 + length <= width), else start a new line

QUESTION

EXAMPLES

```
text  = "Hello world\nFoo bar baz qux"
width = 10
Line 1: "Hello"            (5 chars; "world" + space would be 11 -> break)
Line 2: "world"
Line 3: "Foo bar"           (7 chars; "baz" -> "Foo bar baz" = 11 -> break)
Line 4: "baz qux"
Output: 4
```

FULL CODE | main question first, then every follow-up, tests at the end

```
# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
```

```python
def split_long(word, width):
    # a word longer than the line is cut into pieces of `width` letters
    pieces = []
    for start in range(0, len(word), width):
        pieces.append(word[start:start + width])
    return pieces


def wrap_lines(text, width, break_long=False):
    # the wrapped lines themselves (follow-ups 1 and 2). A newline always
    # ends a line. Each line is built from a list of words and joined once.
    lines = []
    for paragraph in text.split("\n"):
        words = []
        for word in paragraph.split():
            if break_long:
                words += split_long(word, width)
            else:
                words.append(word)
        current = ""
        for word in words:
            if current == "":
                current = word
            elif len(current) + 1 + len(word) <= width:
                current += " " + word               # the word and one space still fit
            else:
                lines.append(current)
                current = word
        lines.append(current)                       # the last (or only, or empty) line of the paragraph
    return lines


# ---- main question: main function ----
# Idea: greedy. Split on newlines first (a newline always ends a line, and
```

```python
# an empty paragraph is an empty line). Then for each word compare
# current length + 1 + length of the word with the width. Only the LENGTH
# of the current line is tracked, never the joined text.
def wrap_count(text, width):
    lines = 0
    for paragraph in text.split("\n"):
        lines += 1                                  # every paragraph has at least one line
        current_length = 0                          # 0 means: nothing on the line yet
        for word in paragraph.split():
            if current_length == 0:
                current_length = len(word)
            elif current_length + 1 + len(word) <= width:
                current_length += 1 + len(word)
            else:
                lines += 1
                current_length = len(word)
    return lines


# ---- main question: tests ----
def test_main_question():
    text = "Hello world\nFoo bar baz qux"
    assert wrap_count(text, 10) == 4
    assert wrap_count("a\n\nb", 5) == 3             # blank lines count as lines
    assert wrap_count("", 5) == 1
    assert wrap_count("one two three four", 9) == 3


# ======================================================
# FOLLOW-UP 1: Return the lines, not only the count
# ======================================================
```

```
# ============================================================
# FOLLOW-UP 2: A word longer than the width must be broken
# ============================================================
# ---- follow-up 1: tests ----
# Answer: the same loop, but append each finished line to a list
# (wrap_lines above). Build every line from a list of words and join once,
# so each word is copied once. For a word longer than the width, flush the
# current line and cut the word into width-sized pieces (split_long) before
# placing them like normal words.
def test_follow_up_1():
    text = "Hello world\nFoo bar baz qux"
    assert wrap_lines(text, 10) == ["Hello", "world", "Foo bar", "baz qux"]
```

```
    assert wrap_lines("abcdefgh", 3, True) == ["abc", "def", "gh"]
    assert wrap_lines("abcdefgh", 3) == ["abcdefgh"]
    for sample in ("a b c d e f", "xx yy zz", "a\n\nb c", ""):
        assert wrap_count(sample, 4) == len(wrap_lines(sample, 4))


# ============================================================
# FOLLOW-UP 3: Minimise raggedness instead of lines
# ============================================================
# ---- follow-up 3: function ----
# Answer: Knuth's word-wrap DP. cost[i] is the best total cost of laying
# out words i, i + 1, ...; a line costs (spare space)^2 except the last
# one, which is free. O(n * words per line). Greedy is NOT optimal for
# this goal.
def min_raggedness(words, width):
    n = len(words)
    cost = [float("inf")] * (n + 1)
    next_start = [0] * (n + 1)
    cost[n] = 0
    for i in range(n - 1, -1, -1):
        length = -1
        for j in range(i, n):                          # a line holds words i .. j
            length += len(words[j]) + 1
            if length > width:
                break
            extra = 0
            if j < n - 1:
                extra = (width - length) ** 2
            if extra + cost[j + 1] < cost[i]:
                cost[i] = extra + cost[j + 1]
                next_start[i] = j + 1
    lines = []
    i = 0
    while i < n:
        lines.append(" ".join(words[i:next_start[i]]))
        i = next_start[i]
    return cost[0], lines


# ---- follow-up 3: tests ----
def test_follow_up_3():
    cost, lines = min_raggedness("aaa bb cc ddddd".split(), 6)      # beats the greedy layout
    assert lines == ["aaa", "bb cc", "ddddd"]
    assert cost == 9 + 1
```

```
# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
    test_main_question()
```

```
    test_follow_up_1()
    test_follow_up_3()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

287 Implement a function to justify a list of words within a given width. HARD

Template fit: pack_lines for the words, then spread the extra spaces over the gaps (the left gaps get the remainder); the last line is left-aligned

QUESTION

Given a list of words and a line width, pack the words into lines of exactly that width (full justification): put as many words on a line as fit (with at least one space between them), then spread the extra spaces between the words as evenly as possible, giving the leftmost gaps the extra ones. The last line, and any line with a single word, is left-justified and padded with spaces on the right. Return the list of lines.

Function signature

def fullJustify(words: list, max_width: int) -> list

EXAMPLES

Example 1	
Input	`['This', 'is', 'an', 'example', 'of', 'text', 'justification.'], 16`
Output	`['This is an', 'example of text', 'justification. ']`
Why	Three lines of width 16: the first two are spread out, the last is left-aligned and padded.

Example 2	
Input	`['a', 'b'], 5`
Output	`['a b ']`
Why	Both words fit on one line, which is the last line: left-aligned with single spaces and padded on the right.

FULL CODE | main question first, then every follow-up, tests at the end

```
# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

import random


# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: helper functions ----
# `measure(text)` is how wide a piece of text is. It is len() for plain
# characters; follow-ups 2 and 4 pass another function (wide characters,
# pixel widths) and nothing else has to change.
def pack_greedy(words, width, measure=len):
    # put as many words as fit on every line (one space between words)
    lines = []
    line = []
    used = 0                                    # width of the line so far, spaces included
```

```
    for word in words:
        size = measure(word)
        if line and used + 1 + size > width:
            lines.append(line)                # the word does not fit: close the line
            line = []
            used = 0
        if line:
            used += 1
        used += size
        line.append(word)
    if line:
        lines.append(line)
    return lines


def spread(line, width, measure=len):
    # fully justified: the spare space is shared over the gaps, and the EXTRA
    # spaces (when it does not divide evenly) go to the LEFT gaps first.
    # A line with one word is left-aligned.
    if len(line) == 1:
        return line[0] + " " * (width - measure(line[0]))
    letters = 0
    for word in line:
```

```
        letters += measure(word)
    spaces = width - letters
    gaps = len(line) - 1
    each = spaces // gaps
    extra = spaces % gaps
    text = ""
    for i in range(gaps):
        text += line[i] + " " * each
        if i < extra:
            text += " "
    return text + line[-1]


def left_align(line, width, measure=len):
    text = " ".join(line)
    return text + " " * (width - measure(text))


def center(line, width, measure=len):
    # the spare spaces are split on both sides, the odd one goes right
    text = " ".join(line)
    spare = width - measure(text)
    left = spare // 2
    return " " * left + text + " " * (spare - left)


# ---- main question: main function ----
# Idea: greedy line packing. Put as many words as fit on a line (counting
# one space between words). Spread the spare spaces over the gaps, giving
# the extra ones to the leftmost gaps. The LAST line is left-aligned.
def fullJustify(words, max_width):
    lines = pack_greedy(words, max_width)
    result = []
    for i in range(len(lines)):
        if i < len(lines) - 1:
            result.append(spread(lines[i], max_width))
        else:
            result.append(left_align(lines[i], max_width))
    return result


# ---- main question: tests ----
```

```
def test_main_question():
    text = "This is an example of text justification.".split()
    assert fullJustify(text, 16) == ["This    is    an", "example  of text", "justification.  "]
```

```
    words = "What must be acknowledgment shall be".split()
    got = fullJustify(words, 16)
    assert got == ["What   must   be", "acknowledgment  ", "shall be        "]
    for line in got:
        assert len(line) == 16
    assert fullJustify(["a"], 3) == ["a  "]
    assert fullJustify([], 5) == []


# =====================================================
# FOLLOW-UP 1: How would you change it to fully justify the last line too, or to center the text?
# =====================================================


# =====================================================
# FOLLOW-UP 3: Fully justify the last line too
# =====================================================
# ---- follow-up 1: function ----
# Answer: only the branch for the last line changes. To justify it too, drop
# the special case (a line with several words is spread like the others;
# a one-word line stays left-aligned). To center it, use center().
def justify_options(words, width, last="left", measure=len):
    lines = pack_greedy(words, width, measure)
    result = []
    for i in range(len(lines)):
        if i < len(lines) - 1 or last == "full":
            result.append(spread(lines[i], width, measure))
        elif last == "center":
            result.append(center(lines[i], width, measure))
        else:
            result.append(left_align(lines[i], width, measure))
    return result


# ---- follow-up 1: tests ----
def test_follow_up_1():
    words = "What must be acknowledgment shall be".split()
    assert justify_options(words, 16, last="full")[2] == "shall         be"
    assert justify_options(words, 16, last="center")[2] == "    shall be    "
    assert justify_options(words, 16) == fullJustify(words, 16)


# =====================================================
# FOLLOW-UP 2: What if the words contain wide characters or different pixel widths?
# =====================================================
```

```


# =====================================================
# FOLLOW-UP 4: Words have pixel widths (a proportional font)
# =====================================================
# ---- follow-up 2: tests ----
# Answer: replace len(word) by a measure(word) function (font metrics, or
# East Asian character width) and compare against the pixel width. Extra
# space is still handed out in whole space units, so the remainder rule
# stays the same. The helpers above already take a `measure` argument.
def double_width(text):
    return 2 * len(text)                    # pretend every character is 2 units wide


def test_follow_up_2():
    lines = justify_options(["ab", "cd", "ef"], 12, measure=double_width)
```

```
    assert lines == ["ab    cd", "ef" + " " * 8]


# ======================================================
# FOLLOW-UP 5: Minimise raggedness instead of packing greedily
# ======================================================
# ---- follow-up 5: function ----
# Answer: this is the Knuth-Plass idea, a DP: the cost of a line is
# (spare space)^2 (the last line is free), and best[i] is the cheapest
# way to set the first i words. Try every possible start j of the last
# line. O(n * words per line). Greedy packing is not optimal for this goal.
def pack_optimal(words, width):
    n = len(words)
    best = [0] + [float("inf")] * n
    previous = [0] * (n + 1)
    for i in range(1, n + 1):
        size = -1
        for j in range(i - 1, -1, -1):            # a line holds words j .. i - 1; try the longest first
            size += len(words[j]) + 1
            if size > width:
                break
            cost = 0
            if i < n:
                cost = (width - size) ** 2
            if best[j] + cost < best[i]:
                best[i] = best[j] + cost
                previous[i] = j
    lines = []
    i = n
```

```
    while i > 0:
        lines.append(words[previous[i]:i])
        i = previous[i]
    lines.reverse()
    return lines


# ---- follow-up 5: tests ----
def raggedness(lines, width):
    # the sum of the squares of the unused space (the last line is free)
    cost = 0
    for line in lines[:-1]:
        cost += (width - len(" ".join(line))) ** 2
    return cost


def test_follow_up_5():
    random.seed(34)
    for round_number in range(200):
        words = []
        for i in range(random.randint(1, 9)):
            words.append("x" * random.randint(1, 6))
        optimal = pack_optimal(words, 8)
        flat = []
        for line in optimal:
            flat += line
        assert flat == words
        for line in optimal:
            assert len(" ".join(line)) <= 8
        assert raggedness(optimal, 8) <= raggedness(pack_greedy(words, 8), 8)


# ======================================================
# RUN EVERYTHING
# ======================================================
def run_all_tests():
```

```
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    test_follow_up_5()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```

288 Convert Valid Snake Case to Lower Camel Case

HARD

Template fit: reformat(): split on '_', keep the first piece, capitalise the others

QUESTION

Given a valid snake_case identifier (one or more lowercase words separated by single underscores, with no leading, trailing or doubled underscores), convert it to lowerCamelCase: keep the first word as it is, remove the underscores and capitalise the first letter of every later word.

Function signature

def snake_to_camel(s: str) -> str

EXAMPLES

Example 1	
Input	`'get_http_response_code'`
Output	`'getHttpResponseCode'`
Why	Words are joined and every word after the first is capitalised.

Example 2	
Input	`'user_id'`
Output	`'userId'`
Why	Two words.

Example 3	
Input	`'name'`
Output	`'name'`
Why	A single word has nothing to change.

FULL CODE | main question first, then every follow-up, tests at the end

```
# ============================================================
# HOW THIS FILE IS ORGANISED
#   MAIN QUESTION: its helpers, its main function, its tests.
#   FOLLOW-UP 1, 2, 3 ...: for each, its helpers, its function(s), its tests.
#   At the bottom, run_all_tests() calls every test function in this order.
# ============================================================

# ============================================================
# MAIN QUESTION
# ============================================================
# ---- main question: main function ----
# Idea: the input is valid, so there are no empty parts. Split on "_",
# keep the first word as it is, and give every other word a capital first
# letter.
def snake_to_camel(s):
    parts = s.split("_")
```

```
    result = parts[0]
    for part in parts[1:]:
        result += part[:1].upper() + part[1:]
    return result


# ---- main question: tests ----
def test_main_question():
    assert snake_to_camel("foo_bar") == "fooBar"
    assert snake_to_camel("a_b_c") == "aBC"
    assert snake_to_camel("name") == "name"
    assert snake_to_camel("get_http_response") == "getHttpResponse"


# ======================================================
# FOLLOW-UP 1: The input may have leading, trailing or repeated underscores, or mixed case
# ======================================================
# ---- follow-up 1: function ----
# Answer: drop the empty parts after the split, lowercase the first word
# and capitalize the rest. Say what you do with digits and other
# characters, and keep the rule in one place.
def snake_to_camel_tolerant(s):
    words = []
    for part in s.split("_"):
        if part != "":
            words.append(part)
    if len(words) == 0:
        return ""
    result = words[0].lower()
```

```
    for word in words[1:]:
        result += word.capitalize()
    return result


# ---- follow-up 1: tests ----
def test_follow_up_1():
    assert snake_to_camel_tolerant("_Foo__BAR_") == "fooBar"
    assert snake_to_camel_tolerant("___") == ""


# ======================================================
# FOLLOW-UP 2: Convert back from camel case to snake case
# ======================================================
# ---- follow-up 2: function ----
# Answer: put an underscore before an uppercase letter and lowercase it.
# Acronyms such as HTTPServer need a smarter rule: an underscore goes
# before an uppercase letter when the previous letter is lowercase or a
# digit, or when it is uppercase but the NEXT letter is lowercase (the
# start of a new word after an acronym).
def camel_to_snake(s):
    result = ""
    for i in range(len(s)):
        ch = s[i]
        if ch.isupper() and i > 0:
            previous = s[i - 1]
            next_is_lower = i + 1 < len(s) and s[i + 1].islower()
            if previous.islower() or previous.isdigit() or (previous.isupper() and next_is_lower):
                result += "_"
        result += ch.lower()
    return result


# ---- follow-up 2: tests ----
def test_follow_up_2():
    assert camel_to_snake("fooBarBaz") == "foo_bar_baz"
```

```
    assert camel_to_snake("HTTPServer") == "http_server"
    assert camel_to_snake("x") == "x"
    assert camel_to_snake(snake_to_camel("my_var_2")) == "my_var2"       # and back again


# ============================================================
# RUN EVERYTHING
# ============================================================
def run_all_tests():
```

```
    test_main_question()
    test_follow_up_1()
    test_follow_up_2()
    print("all tests passed")


if __name__ == "__main__":
    run_all_tests()
```